

ALGORITHMS, DATA STRUCTURES, AND APPLICATIONS

Computational Geometry

Chaman Singh Verma

July 2026

Preface

Computational geometry is the study of algorithms for solving geometric problems. It sits at the intersection of mathematics, computer science, and engineering, providing the theoretical foundations and practical tools for manipulating geometric objects in two and three dimensions.

This book presents a comprehensive collection of algorithms drawn from the **CompGeom** library. Each chapter provides a self-contained treatment of a family of related algorithms, including formal definitions, theoretical analysis, pseudocode implementations, complexity bounds, practical applications, and edge-case considerations.

The material is organized into twelve parts, progressing from foundations and mathematical preliminaries through fundamental planar algorithms, spatial data structures, proximity and tessellations, arrangements and duality, geometric optimization, curves and surfaces, motion and robotics, shape analysis, advanced algorithmic techniques, application domains, and finally modern research-level topics.

How to Use This Book

Each algorithm entry follows a consistent structure:

1. **Overview** — a high-level description of the problem and its significance.
2. **Definitions** — precise mathematical terminology.
3. **Theory** — the mathematical foundations and proofs.
4. **Pseudocode** — a readable implementation sketch.
5. **Parameter Selection** — practical tuning advice.
6. **Complexity** — time and space bounds.
7. **Applications** — real-world use cases.
8. **Edge Cases** — testing strategies and degenerate inputs.
9. **References** — original papers and textbooks.

Readers may approach the book topically or use it as a reference for individual algorithms. Cross-references between chapters are provided where algorithms build upon one another.

Introduction

Euclidean Geometry

Euclidean geometry studies points, lines, angles, distances, and shapes in a flat space governed by Euclid’s parallel postulate. In $\mathbb{R}^2$ and $\mathbb{R}^3$, the distance between points is measured by the Euclidean norm,

$$\|x - y\|_2 = \sqrt{\sum_i (x_i - y_i)^2},$$

and straight lines are the shortest paths between their points. Most classical computational-geometry algorithms in this book are Euclidean: orientation tests, convex hulls, Delaunay triangulations, Voronoi diagrams, line and segment intersections, polygon operations, and mesh algorithms all use this flat-space model either directly or as their local approximation.

Non-Euclidean Geometries

Non-Euclidean geometry changes one or more assumptions of Euclidean geometry, especially the behavior of parallel lines and the definition of distance. The two principal examples are:

- **Spherical or elliptic geometry:** points lie on a curved surface such as a sphere. “Straight” paths are great-circle arcs, and initially parallel geodesics can meet.
- **Hyperbolic geometry:** the space has negative curvature. A point and a line can have multiple distinct parallels through that point, and triangle angles sum to less than π .

More generally, differential geometry describes a curved space using a manifold, a metric tensor, and geodesics. This viewpoint is important for surface parameterization, geodesic distance, curvature, discrete exterior calculus, and spectral geometry. Algorithms designed for Euclidean input cannot automatically be transferred to curved spaces: Euclidean dot products, cross products, circumcenters, and straight-line interpolation may need to be replaced by metric-aware operations, local tangent-plane constructions, or geodesic computations. Several later chapters introduce these extensions.

Prerequisites

A basic understanding of data structures, algorithms, and linear algebra is assumed. Familiarity with discrete mathematics and introductory geometry will be helpful but is not strictly required, as key concepts are introduced inline.

Notation

Symbol	Meaning
$\mathbb{R}^d$	Euclidean space of dimension d
$\mathbb{R}^2, \mathbb{R}^3$	The 2D and 3D Euclidean planes
n	Number of input points or elements
k	Number of output features (intersections, layers, etc.)
$P = \{p_1, \dots, p_n\}$	A set of points
$CH(P)$	Convex hull of P
$DT(P)$	Delaunay triangulation of P
$VD(P)$	Voronoi diagram of P
$\mathcal{O}(f(n))$	Big-O complexity notation
$\Omega(f(n))$	Big-Omega complexity notation
$\Theta(f(n))$	Big-Theta complexity notation
$\alpha(n)$	Inverse Ackermann function
Δ	Laplace–Beltrami operator (or cotangent Laplacian)
d, δ	Exterior derivative and codifferential
$\star$	Hodge star operator
$\oplus, \ominus$	Minkowski sum and difference
$\langle \cdot, \cdot \rangle$	Inner product
$ \cdot $	Euclidean norm (or cardinality, context-dependent)
∂K	Boundary of a simplicial complex K
$\chi(K)$	Euler characteristic of K
β_p	p -th Betti number

Table 1: Principal notation used throughout the book.

Contents

Preface	iii
Introduction	v
Notation	vii
I Foundations	1
1 Introduction to Computational Geometry	3
1.1 What Is Computational Geometry?	4
1.2 Geometry as an Algorithmic Problem	4
1.3 Historical Development	4
1.4 Geometric Objects and Representations	4
1.5 Points, Lines, Segments, Rays, and Polygons	4
1.6 Combinatorial versus Numerical Geometry	4
1.7 The Real-RAM Model	4
1.8 Computational Models and Complexity	4
1.9 Degeneracies and General Position	4
1.10 Robustness and Numerical Precision	4
1.11 Symbolic Perturbation	4
1.12 Exact Geometric Computation	4
1.13 Major Families of Geometric Algorithms	4
1.14 Applications of Computational Geometry	4
1.15 Software Libraries and Experimental Geometry	4
1.16 Chapter Problems	4
1.17 Computational Projects	4
2 Mathematical Foundations	5
2.1 Vectors and Affine Spaces	5
2.2 Inner Products and Euclidean Distance	5
2.3 Lines and Hyperplanes	5
2.4 Affine Combinations	5
2.5 Convex Combinations	5
2.6 Orientation and Determinants	5
2.7 Signed Area and Signed Volume	5
2.8 Barycentric Coordinates	5
2.9 Half-Spaces	5
2.10 Hyperplanes and Separating Surfaces	5
2.11 Norms and Metrics	5
2.12 Transformations	5

2.13	Homogeneous Coordinates	5
2.14	Duality	5
2.15	Topological Preliminaries	5
2.16	Exercises	5
3	Geometric Predicates and Robust Computation	7
3.1	Orientation Predicate	7
3.2	Three-dimensional Orientation	7
3.3	In-circle Predicate	8
3.4	Robust Evaluation	8
3.5	Floating-Point Filters	8
3.6	Exact Arithmetic	9
3.7	Rational Arithmetic in Computational Geometry	9
3.8	Simulation of Simplicity	10
3.9	Robustness Failure Examples	10
II	Fundamental Planar Algorithms	11
4	Convex Hulls	13
4.1	Graham Scan	13
4.2	Monotone Chain	16
4.3	Chan's Algorithm	18
4.4	QuickHull	19
4.5	Convex Hull Peeling	21
5	Line-Segment Intersection	25
5.1	The Intersection Problem	25
5.2	Pairwise Intersection Testing	25
5.3	Sweep-Line Paradigm	25
5.4	Events and Event Queues	25
5.5	Sweep-Line Status Structures	25
5.6	Bentley–Ottmann Algorithm	25
5.7	Correctness	25
5.8	Complexity Analysis	25
5.9	Degenerate Intersections	25
5.10	Reporting versus Detecting Intersections	25
5.11	Red–Blue Segment Intersection	25
5.12	Contour of the Union of Rectangles	25
5.13	Applications	26
5.14	Exercises	26
5.15	Project: Interactive Sweep-Line Visualizer	26
6	Polygon Geometry	27
6.1	Polygon Boolean Operations	27
6.2	Line Arrangements	30
6.3	Minkowski Sums	33
6.4	Industrial Applications of Minkowski Sums	35
6.5	Polygon Symmetry	37
6.6	Polygon Matching	39
6.7	Line and Polygon Clipping	42
6.8	Polygon Partitioning	44

6.9	Convex Decomposition	44
6.10	Convex Partitioning	45
7	Polygon Triangulation	49
7.1	Ear Clipping	49
7.2	Monotone Decomposition	52
7.3	Incremental Delaunay Triangulation	54
7.4	Dynamic Delaunay Triangulation	57
7.5	Delaunay Flips	59
7.6	Constrained Delaunay Triangulation	62
7.7	Non-Obtuse Triangulation	63
III	Geometric Searching and Spatial Data Structures	65
8	Point Location	67
8.1	The Planar Point-Location Problem	67
8.2	Subdivision Representations	67
8.3	Slab Decomposition	67
8.4	Trapezoidal Maps	67
8.5	Randomized Incremental Construction	67
8.6	Search DAGs	67
8.7	Kirkpatrick's Hierarchy	67
8.8	Point Location in Triangulations	67
8.9	Dynamic Point Location	67
8.10	Practical Implementations	67
8.11	Exercises	67
9	Orthogonal Range Searching	69
9.1	Range Queries	69
9.2	One-Dimensional Range Searching	69
9.3	k-d Trees	69
9.4	Range Trees	69
9.5	Multilevel Data Structures	69
9.6	Fractional Cascading	69
9.7	Priority Search Trees	69
9.8	Reporting versus Counting	69
9.9	Higher-Dimensional Range Searching	69
9.10	Dynamic Range Searching	69
9.11	Curse of Dimensionality	69
9.12	Simplex Range Searching	69
9.13	Exercises	71
9.14	Project: Spatial Query Engine	71
10	Spatial Partitioning	73
10.1	K-Dimensional Trees	73
10.2	Quadtrees	77
10.3	R-Trees	81
10.4	Interval Trees	84
10.5	Segment Trees	86
10.6	Uniform Grids	89
10.7	Fixed-Radius Neighbor Search	91

10.8 Binary Space Partitioning Trees	93
10.9 Industrial Applications of BSP Trees	95
10.10 Space-Filling Curves	96
10.11 Hilbert Curve	96
10.12 Morton Curve	99
10.13 Peano Curve	102
10.14 Zigzag Curve	104
10.15 Spiral Walk	107
 IV Proximity and Tessellations	 111
 11 Voronoi Diagrams	 113
11.1 Voronoi Diagram	113
11.2 Voronoi Diagrams of Line Segments	114
11.3 Farthest-Point Voronoi Diagrams	116
11.4 Kinetic Voronoi Diagrams	117
11.5 Industrial Applications of Voronoi Diagrams	118
 12 Delaunay Triangulations	 119
12.1 Voronoi–Delaunay Duality	119
12.2 Empty-Circumcircle Property	119
12.3 Local Delaunay Criterion	119
12.4 Edge Flipping	119
12.5 Incremental Construction	119
12.6 Randomized Incremental Algorithm	119
12.7 Divide-and-Conquer Construction	119
12.8 Lifting to a Paraboloid	119
12.9 Relationship to Convex Hulls	119
12.10 Constrained Delaunay Triangulation	119
12.11 Quality Properties	119
12.12 Delaunay Refinement	119
12.13 Higher-Dimensional Delaunay Complexes	119
12.14 Intrinsic Delaunay Triangulation	119
12.15 Applications to Mesh Generation	122
12.16 Exercises	123
12.17 Project: Delaunay Triangulation from Scratch	123
 13 Proximity Problems	 125
13.1 Closest Pair of Points	125
13.2 Proximity Queries	128
13.3 Industrial Applications of Nearest Neighbor Search	130
13.4 Range Search	131
13.5 Point in Polygon (Winding Number)	134
13.6 Largest Empty Circle	136
13.7 Minimum Bounding Boxes	138
13.8 Maximum Overlap	140
13.9 Segment Intersection (Bentley-Ottmann)	143
13.10 Point Location via Trapezoidal Maps	146
13.11 Point Location in Triangulations (Walking)	151

V	Arrangements and Duality	155
14	Arrangements of Lines and Curves	157
14.1	Davenport–Schinzel Sequences	157
14.2	Lower Envelopes	160
14.3	Levels in Arrangements and Half-Plane Discrepancy	163
14.4	Union Volume Estimation	165
15	Geometric Duality	169
15.1	Point-Line Duality	169
15.2	Incidence Preservation	169
15.3	Order Reversal	169
15.4	Dual Arrangements	169
15.5	Convex Hulls through Duality	169
15.6	Half-Plane Intersection	169
15.7	Linear Programming through Duality	169
15.8	Applications to Optimization	169
15.9	Exercises	169
VI	Convexity and Geometric Optimization	171
16	Convex Polytopes	173
16.1	Convex Polytopes	173
16.2	V-Representations and H-Representations	173
16.3	Faces and Face Lattices	173
16.4	Supporting Hyperplanes	173
16.5	Separating Hyperplane Theorems	173
16.6	Carathéodory’s Theorem	173
16.7	Helly’s Theorem	173
16.8	Radon’s Theorem	173
16.9	Euler–Poincaré Relations	173
16.10	Polarity	173
16.11	Computing Higher-Dimensional Hulls	173
16.12	Exercises	173
17	Linear Programming in Geometry	175
17.1	Geometric Linear Programs	175
17.2	Half-Plane Intersection	175
17.3	Low-Dimensional Linear Programming	175
17.4	Duality and Applications	176
18	Intersection and Distance of Convex Objects	177
18.1	Convex Polygon Intersection	177
18.2	Separating Axis Theorem	177
18.3	Distance between Convex Sets	177
18.4	Support Functions	177
18.5	Minkowski Difference	177
18.6	Gilbert–Johnson–Keerthi Algorithm	177
18.7	Penetration Depth	177
18.8	Collision Detection	177
18.9	Continuous Collision Detection	177

18.10	Applications in Robotics and Simulation	177
18.11	Exercises	177
18.12	Enclosure and Convex Distance	177
18.13	Welzl's Algorithm	177
18.14	Convex Polygon Distance	180
VII	Curves, Surfaces, and Three-Dimensional Geometry	185
19	Computational Geometry in Three Dimensions	187
19.1	Polyhedral Representation	187
19.2	Three-Dimensional Predicates	187
19.3	Incremental Three-Dimensional Hulls	187
19.4	Degeneracies and Validation	188
19.5	Applications	188
20	Curves and Surfaces	189
20.1	Surface Modeling and Mesh Generation	189
20.2	Bézier Surfaces	189
20.3	NURBS Surfaces	192
20.4	Mesh Voxelization	194
20.5	Winding-Filtered Tetrahedral Meshing	196
20.6	Mesh Refinement	197
20.7	Approximate Convex Decomposition	201
20.8	Triangle to Quad Conversion	203
20.9	Voxel Grid Point Simplification	205
20.10	Surface Reconstruction	207
20.11	α -Shapes	212
20.12	Surface Parameterization and Geometry	216
20.13	BFF Parameterization	216
20.14	Harmonic Mapping	217
20.15	Functional Maps	220
20.16	Diffusion Distance	222
20.17	Mean Curvature Flow	223
20.18	Angle-Bounded Remeshing	226
20.19	Affine Heat Flow	227
20.20	Geodesic Distance	229
20.21	Medial Axis Transform	231
20.22	Straight Skeleton	234
20.23	Mesh Cut Loops	236
20.24	Mesh Tunnel Loops	238
20.25	Iterative Closest Point (ICP) Registration	240
21	Half-Edge Data Structure	247
21.1	Half-Edge Representation	247
21.2	Vertices, Half-Edges, and Faces	247
21.3	Twin, Next, and Prev Operators	247
21.4	Boundary and Manifold Handling	247
21.5	Adjacency and Neighborhood Queries	247
21.6	Traversal Operations	247
21.7	Comparison with Other Representations	247
21.8	Incidence and Storage Complexity	247

21.9 Construction from Polygon Soup	247
21.10 Applications in Mesh Processing	247
21.11 Exercises	247
22 Mesh Generation	249
22.1 Computational Meshes	249
22.2 Simplicial Complexes	249
22.3 Triangular and Tetrahedral Meshes	249
22.4 Mesh Quality Measures	249
22.5 Delaunay Mesh Generation	249
22.6 Constrained Delaunay Meshing	249
22.7 Delaunay Refinement	249
22.8 Ruppert's Algorithm	249
22.9 Chew's Algorithm	249
22.10 Advancing-Front Methods	249
22.11 Adaptive Meshing	249
22.12 Mesh Optimization	249
22.13 Finite-Element Applications	249
22.14 Exercises	249
22.15 Project: Adaptive Mesh Generator	249
23 Voxel Geometry and OpenVDB	251
23.1 Voxel Grids	251
23.2 Signed Distance Fields	253
23.3 Sparse Voxel Representations	254
23.4 OpenVDB: Hierarchical Sparse Volumes	255
23.5 Voxel-to-Mesh Conversion	257
23.6 Applications	258
VIII Motion, Visibility, and Robotics	259
24 Visibility	261
24.1 Visibility Problems	261
24.2 Visibility inside Polygons	261
24.3 Visibility Polygons	261
24.4 Computing Visibility from a Point	261
24.5 Guarding Polygons	261
24.6 Visibility Graphs	261
24.7 Shortest Paths among Obstacles	261
24.8 Geodesic Paths	261
24.9 Applications	261
24.10 Exercises	261
24.11 Distance Transforms and Shortest Paths	261
24.12 Fast Marching Method	261
24.13 Distance Map	265
24.14 Dijkstra's Algorithm	268
24.15 K-Geodesics	271
24.16 Fréchet Distance	273

25 Art Gallery Problem	277
25.1 Art Gallery Guard Placement	277
25.2 Polygon Kernel and Star-Shaped Polygons	280
26 Motion Planning	283
26.1 Configuration Space	283
26.2 Configuration-Space Obstacles	283
26.3 Exact Planning Methods	284
26.4 Roadmaps and Sampling-Based Planning	284
26.5 Planning Constraints and Validation	284
26.6 Applications	285
 IX Shape Analysis and Geometric Data	 287
27 Sampling	289
27.1 Poisson Disk Sampling	289
27.2 Halton Points	292
27.3 Uniform Random Point Generation	295
27.4 Random Line Segment Generation	298
27.5 Random Polygon Generation	300
27.6 Random Walk	302
27.7 Self-Avoiding Walk	305
28 Shape Representation and Reconstruction	309
28.1 Shape Representation	309
28.2 Implicit and Parametric Representations	309
28.3 Reconstruction from Point Clouds	309
28.4 Surface Reconstruction	309
28.5 Signed Distance Functions	309
28.6 Level Sets and Front Propagation	309
28.7 Poisson Surface Reconstruction	309
28.8 Applications	309
28.9 Exercises	309
29 Geometric Matching	311
29.1 Comparing Geometric Objects	311
29.2 Hausdorff Distance	311
29.3 Fréchet Distance	311
29.4 Discrete Fréchet Distance	311
29.5 Shape Matching	311
29.6 Point-Set Registration	311
29.7 Rigid Transformations	311
29.8 Iterative Closest Point	311
29.9 Geometric Hashing	311
29.10 Applications in Computer Vision	311
29.11 Exercises	311
30 Computational Topology	313
30.1 Mesh Analysis and Topology	313
30.2 Mesh Connected Components	313
30.3 Mesh Boundary Detection	316

30.4	Mesh Orientability	318
30.5	Manifold Verification	321
30.6	Euler Characteristic	324
30.7	Node Degree and Valence	327
30.8	Mesh Coloring	329
30.9	Mesh Reordering	332
30.10	Mesh Rigidity	334
30.11	Mesh Decimation via Quadric Error Metrics	337
30.12	Spectral Geometry and Shape Analysis	342
30.13	Shape Spectra	342
30.14	Shape Space Spectra	345
30.15	Odeco Frame Fields	346
30.16	Discrete Exterior Calculus	348
30.17	Hodge Star Computation	348
30.18	Exterior Derivative	351
30.19	Hodge Decomposition	354
30.20	Discrete Morse Theory	357
X	Advanced Algorithmic Techniques	361
31	Randomized Geometric Algorithms	363
31.1	Why Randomization Helps	363
31.2	Randomized Incremental Construction	363
31.3	Backward Analysis	363
31.4	Conflict Graphs	363
31.5	Clarkson–Shor Technique	363
31.6	Random Sampling	363
31.7	ϵ -Nets	363
31.8	Applications to Convex Hulls	363
31.9	Applications to Delaunay Triangulations	363
31.10	Applications to Linear Programming	363
31.11	Exercises	363
32	Approximation Algorithms in Geometry	365
32.1	Why Approximation?	365
32.2	Geometric Approximation	365
32.3	ϵ -Approximations	365
32.4	Coresets	365
32.5	Approximate Nearest Neighbors	365
32.6	Locality-Sensitive Hashing	365
32.7	Well-Separated Pair Decomposition	365
32.8	Approximate Distance Queries	365
32.9	Geometric Clustering	365
32.10	k -Center and k -Median Problems	365
32.11	Minimum Enclosing Shapes	365
32.12	High-Dimensional Problems	365
32.13	Exercises	365
32.14	Project: Approximate Nearest-Neighbor Search	365

33 Packing Algorithms	367
33.1 Circle Packing	367
33.2 Rectangle Packing	370
33.3 Hopper Optimization	373
34 Dynamic and Kinetic Geometry	377
34.1 Dynamic Geometric Problems	377
34.2 Dynamic Point Sets	377
34.3 Dynamic Convex Hulls	377
34.4 Dynamic Proximity Structures	377
34.5 Moving Geometric Objects	377
34.6 Kinetic Data Structures	377
34.7 Certificates and Events	377
34.8 Kinetic Sorting	377
34.9 Kinetic Convex Hulls	377
34.10 Kinetic Closest Pairs	377
34.11 Applications to Tracking and Simulation	377
34.12 Exercises	377
 XI Applications	 379
35 Computational Geometry for Computer Graphics	381
35.1 Geometric Modeling	381
35.2 Spatial Acceleration Structures	381
35.3 Bounding Volume Hierarchies	381
35.4 Ray–Object Intersection	381
35.5 Hidden-Surface Problems	381
35.6 Clipping	381
35.7 Mesh Processing	381
35.8 Collision Detection	381
35.9 Real-Time Geometry	381
35.10 GPU-Based Geometric Computation	381
35.11 Case Study: Geometry Pipeline	381
35.12 Project	381
 36 Computational Geometry for GIS	 383
36.1 Geographic Data	383
36.2 Spatial Indexing	383
36.3 Map Overlay	383
36.4 Point-in-Region Queries	383
36.5 Proximity Analysis	383
36.6 Voronoi-Based Service Regions	383
36.7 Terrain Representation	383
36.8 Triangulated Irregular Networks	383
36.9 Digital Elevation Models	383
36.10 Map Generalization	383
36.11 Large-Scale Spatial Data	383
36.12 Case Study: Geographic Facility Location	383
36.13 Project	383

37 Computational Geometry for Robotics	385
37.1 Robot Geometry	385
37.2 Collision Detection	385
37.3 Configuration Spaces	385
37.4 Path Planning	385
37.5 Sensor Geometry	385
37.6 Localization	385
37.7 Mapping	385
37.8 SLAM and Geometric Structures	385
37.9 Manipulation Planning	385
37.10 Autonomous Navigation	385
37.11 Case Study: Mobile-Robot Planning	385
37.12 Project	385
38 Computational Geometry for Scientific Computing	387
38.1 Geometry in Numerical Simulation	387
38.2 Mesh Generation	387
38.3 Finite-Element Geometry	387
38.4 Domain Discretization	387
38.5 Adaptive Refinement	387
38.6 Particle Methods	387
38.7 Nearest-Neighbor Computations	387
38.8 Spatial Decomposition	387
38.9 Parallel Geometric Algorithms	387
38.10 Large-Scale Scientific Geometry	387
38.11 Case Study: PDE Mesh Construction	387
38.12 Project	387
38.13 Stochastic and Meshless PDE Methods	387
38.14 Stochastic PDE Solvers	387
38.15 Wave Kernel Signature	389
XII Modern and Research-Level Topics	393
39 High-Dimensional Computational Geometry	395
39.1 Geometry beyond Three Dimensions	395
39.2 High-Dimensional Convex Hulls	395
39.3 Voronoi and Delaunay Structures	395
39.4 Nearest-Neighbor Search	395
39.5 Dimensionality Reduction	395
39.6 Johnson–Lindenstrauss Lemma	395
39.7 Random Projections	395
39.8 Concentration of Measure	395
39.9 Curse of Dimensionality	395
39.10 Applications to Machine Learning	395
39.11 Exercises	395
40 Geometry of Data and Machine Learning	397
40.1 Geometric View of Data	397
40.2 Metric Spaces	397
40.3 Nearest-Neighbor Classification	397
40.4 Clustering Geometry	397

40.5	Convex Geometry in Machine Learning	397
40.6	Kernel Geometry	397
40.7	Manifold Learning	397
40.8	Geometric Embeddings	397
40.9	Graph-Based Geometric Learning	397
40.10	Optimal Transport: A Geometric Introduction	397
40.11	Computational Geometry for AI	397
40.12	Research Directions	397
40.13	Project	397
41	Research Frontiers	399
41.1	Robust and Certified Geometry	400
41.2	Massive Geometric Data Sets	400
41.3	Streaming Geometry	400
41.4	External-Memory Geometry	400
41.5	Geometric Algorithms for Autonomous Systems	400
41.6	Geometry in Machine Learning	400
41.7	Topological Data Analysis	400
41.8	Shape and Surface Reconstruction	400
41.9	Geometric Deep Learning	400
41.10	Differentiable Geometry	400
41.11	Geometry Processing	400
41.12	Open Problems in Computational Geometry	400
41.13	How to Read Computational Geometry Research	400
41.14	Designing Computational Experiments	400
41.15	From Graduate Study to Research	400
41.16	Computational Algebra and Similarity	400
41.17	Buchberger’s Algorithm	400
41.18	Resultant-Based Elimination	404
A	Mathematical Reference	407
A.1	Linear Algebra	407
A.2	Vector Calculus	407
A.3	Determinants	407
A.4	Affine Geometry	407
A.5	Convexity	407
A.6	Probability	407
A.7	Graph Theory	407
A.8	Asymptotic Analysis	407
B	Algorithms and Data Structures	409
B.1	Sorting	409
B.2	Binary Search	409
B.3	Balanced Search Trees	409
B.4	Priority Queues	409
B.5	Hashing	409
B.6	Graph Algorithms	409
B.7	Union–Find	409
B.8	Randomized Algorithms	409

C	Geometry Formula Reference	411
C.1	Vector Operations	411
C.2	Orientation Tests	411
C.3	Distance Formulas	411
C.4	Intersection Formulas	411
C.5	Circle Predicates	411
C.6	Areas and Volumes	411
C.7	Coordinate Transformations	411
C.8	Barycentric Coordinates	411
D	Implementation Guide	413
D.1	Designing a Geometry Kernel	413
D.2	Choosing Numeric Types	413
D.3	Floating-Point Error	413
D.4	Exact Predicates	413
D.5	Testing Geometric Software	413
D.6	Generating Adversarial Test Cases	413
D.7	Benchmarking	413
D.8	Visualization and Debugging	413
E	Computational Geometry Software	415
E.1	CGAL	415
E.2	GEOSE	415
E.3	Boost.Geometry	415
E.4	Qhull	415
E.5	Triangle	415
E.6	TetGen	415
E.7	SciPy Spatial Algorithms	415
E.8	Python Geometry Ecosystem	415
E.9	Choosing a Library	415
F	Graduate Projects	417
F.1	Convex Hull Laboratory	417
F.2	Sweep-Line Intersection Engine	417
F.3	Voronoi/Delaunay Package	417
F.4	Spatial Database	417
F.5	Mesh Generator	417
F.6	Collision-Detection System	417
F.7	Robot Path Planner	417
F.8	Point-Cloud Reconstruction	417
F.9	GIS Spatial Analysis System	417
F.10	Persistent-Homology Explorer	417
F.11	GPU Geometry Engine	417
F.12	Research Replication Project	417
G	Selected Solutions and Hints	419

List of Algorithms

1	Two-dimensional orientation test	7
2	Graham Scan for Convex Hull	15
3	Monotone Chain (Andrew's Algorithm)	17
4	Chan's Convex Hull Algorithm	19
5	QuickHull Algorithm	20
6	Convex Hull Peeling (Onion Peeling)	22
7	Convex Decomposition via Triangle Merging	45
8	Minimum Convex Partition via Dynamic Programming	46
9	Ear Clipping Triangulation	51
10	Monotone Polygon Decomposition	53
11	Incremental Delaunay Triangulation	55
12	Dynamic Point Insertion	58
13	Dynamic Point Deletion	58
14	Delaunay Edge Flip Algorithm	61
15	Constrained Delaunay Triangulation	63
16	Non-Obtuse Triangulation	64
17	Simplex Range Counting with a Partition Tree	70
18	Fixed-Radius Neighbor Search	92
19	Hilbert Curve: Coordinates to Index	98
20	Morton Code: Coordinates to Index	100
21	Peano Curve: Coordinates to Index	103
22	Snake Scan Traversal	105
23	Diagonal Zigzag Scan (JPEG)	106
24	Square Spiral Walk	108
25	Voronoi Diagram via Delaunay Dual	114
26	Segment Voronoi Diagram by Plane Sweep	115
27	Smallest Enclosing Circle via Farthest-Point Voronoi	117
28	Kinetic Voronoi Maintenance	117
29	Intrinsic Delaunay Flip Algorithm	121
30	Divide-and-Conquer Closest Pair	127
31	BVH Distance Query	129
32	Orthogonal Range Search (Tree-based)	132
33	Point-in-Polygon via Winding Number	135
34	Largest Empty Circle	137
35	Minimum Bounding Box via Rotating Calipers	139
36	Maximum Overlap (1D Sweep)	142
37	Bentley-Ottmann Segment Intersection	145
38	Trapezoidal Map Query	149
39	Trapezoidal Map Construction (Incremental)	149
40	Locate in a Triangulation by Visibility Walk	152
41	Compute the k-Level of a Line Arrangement	164

42	Half-Plane Intersection	176
43	Incremental Three-Dimensional Convex Hull	188
44	Evaluate Bézier Surface	191
45	Evaluate NURBS Surface	193
46	Voxelize Mesh	195
47	Winding-Filtered Tet Meshing	197
48	Refine Mesh	199
49	CoACD	202
50	Greedy Triangle to Quad	204
51	Voxel Grid Simplify	206
52	Ball Pivoting Algorithm Reconstruction	210
53	α -Shape Computation (2D)	214
54	BFF Parameterization	217
55	Transfer Topology	219
56	Compute Functional Map	221
57	Compute Diffusion Distance	223
58	Mean Curvature Flow	225
59	Angle-Bounded Remesh	226
60	Affine Polygon Smoothing	228
61	Compute Geodesic via Heat Method	230
62	Medial Axis	232
63	Straight Skeleton	235
64	Find Cut Graph	237
65	Find Tunnel Loops	239
66	Iterative Closest Point (ICP) Registration	243
67	Fast Marching Method	263
68	Meijster’s Distance Transform	266
69	Dijkstra’s Shortest Path Algorithm	269
70	K-Geodesics Clustering	272
71	Discrete Fréchet Distance	274
72	Probabilistic Roadmap Planner	284
73	Poisson Disk Sampling	291
74	Halton Sequence Generator	294
75	Uniform Random Points in Basic Shapes	297
76	Random Line Segment Generation	299
77	Random Polygon Generation	301
78	2D Random Walk	304
79	Random Walk on a Mesh	304
80	Generate Self-Avoiding Walk (Recursive)	307
81	Find Mesh Components	315
82	Detect Mesh Boundaries	317
83	Orient Mesh	320
84	Is Manifold	323
85	Calculate Euler Characteristic	326
86	Calculate Vertex Degrees	328
87	Get Vertex Degree	328
88	Color Mesh	331
89	RCM Reorder	333
90	Analyze Mesh Rigidity	336
91	Mesh Decimation via QEM	340
92	Shape-DNA Computation	344

93	Shape Space SpectralNet Usage	346
94	Odeco Frame Field Design	348
95	Compute Hodge Star Operators	350
96	Compute Exterior Derivatives	353
97	Hodge Decomposition of a 1-Form	356
98	Greedy Construction of Discrete Gradient Field	359
99	Stochastic PDE Solver Usage	389
100	Wave Kernel Signature	391

List of Figures

4.1	Graham Scan: the anchor point p_1 (lowest y) sorts all other points by polar angle. The hull vertices (red) are traced in counter-clockwise order.	14
4.2	Convex hull peeling (onion peeling): successive convex hulls are removed, producing nested layers.	22
6.1	Polygon boolean operations: Union $A \cup B$ (center) and Intersection $A \cap B$ (right) of two overlapping rectangles.	28
6.2	Minkowski sum of a triangle A and a unit square B . The result is a pentagon with at most $n + m$ vertices.	34
7.1	Ear clipping: vertex v_3 is an “ear” since triangle (v_2, v_3, v_4) is entirely inside the polygon and contains no other vertices.	50
7.2	Delaunay property: (left) edge AC is illegal because D lies inside the circumcircle of ABC . (right) Flipping to edge BD satisfies the empty circumcircle property.	60
10.1	KD-tree: (left) recursive axis-aligned partitioning of 2D space; (right) the corresponding binary tree structure.	76
10.2	Quadtree: recursive 2D space partitioning. Quadrants with multiple points are subdivided; leaf quadrants contain at most one point.	78
10.3	Hilbert curve construction: each order recursively places four rotated copies in a U-shaped pattern.	97
10.4	Morton curve: the Z-shaped traversal is obtained by interleaving the binary representations of x and y	100
11.1	Voronoi diagram (colored cells) and its Delaunay triangulation dual (dashed gray edges). Each Voronoi vertex is the circumcenter of a Delaunay triangle.	114
13.1	Closest pair: divide-and-conquer splits points along a vertical line. The closest pair may cross the dividing strip.	126
13.2	Bentley–Ottmann sweep line: the vertical line moves left-to-right. The <i>status</i> maintains the vertical order of intersecting segments.	144
14.1	Lower envelope of three functions: the thick black curve is the pointwise minimum $\min\{f_1, f_2, f_3\}$	161
18.1	Welzl’s algorithm: the smallest enclosing circle is defined by at most 3 support points (red).	178
20.1	Bézier surface: a tensor-product surface defined by a control net of points.	190
20.2	1-to-4 triangle refinement: each edge is split at its midpoint, producing four congruent sub-triangles.	199

20.3 Geodesic vs. Euclidean distance on a curved surface. The geodesic (blue) follows the terrain.	230
24.1 Fast Marching Method: the wavefront propagates from the source. Points are categorized as Accepted, Trial, or Far.	263
25.1 Art gallery problem: a comb polygon with $n = 12$ vertices requires $\lfloor 12/3 \rfloor = 4$ guards in the worst case.	278
27.1 Poisson disk sampling: points are placed with minimum distance r between any pair.	290
27.2 Random walk on a grid: at each step, move to a uniformly random neighboring vertex.	303
30.1 A triangle mesh with vertices v_i , edges, and faces f_j . The Euler characteristic $\chi = V - E + F$ is a topological invariant.	314
30.2 (Left) Manifold mesh: every edge is shared by exactly two faces. (Right) Non-manifold: an edge shared by three faces violates the manifold property.	322
30.3 First three eigenfunctions of the Laplacian on a circle. The eigenvalues λ_k form the shape spectrum.	344
30.4 Discrete exterior derivative: d^0 maps vertex values to edge differences; d^1 maps edge values to face circulations.	352
30.5 Hodge decomposition: any 1-form ω splits uniquely into exact, co-exact, and harmonic components.	355
33.1 Circle packing: placing non-overlapping disks of varying radii inside a bounding region.	368
38.1 Walk on Spheres: from each interior point, jump to a random point on the largest inscribed sphere.	388
41.1 Buchberger's algorithm: S-polynomials are formed from pairs of generators and reduced.	402

Part I

Foundations

Chapter 1

Introduction to Computational Geometry

- 1.1 What Is Computational Geometry?
- 1.2 Geometry as an Algorithmic Problem
- 1.3 Historical Development
- 1.4 Geometric Objects and Representations
- 1.5 Points, Lines, Segments, Rays, and Polygons
- 1.6 Combinatorial versus Numerical Geometry
- 1.7 The Real-RAM Model
- 1.8 Computational Models and Complexity
- 1.9 Degeneracies and General Position
- 1.10 Robustness and Numerical Precision
- 1.11 Symbolic Perturbation
- 1.12 Exact Geometric Computation
- 1.13 Major Families of Geometric Algorithms
- 1.14 Applications of Computational Geometry
- 1.15 Software Libraries and Experimental Geometry
- 1.16 Chapter Problems
- 1.17 Computational Projects

Chapter 2

Mathematical Foundations

- 2.1 Vectors and Affine Spaces
- 2.2 Inner Products and Euclidean Distance
- 2.3 Lines and Hyperplanes
- 2.4 Affine Combinations
- 2.5 Convex Combinations
- 2.6 Orientation and Determinants
- 2.7 Signed Area and Signed Volume
- 2.8 Barycentric Coordinates
- 2.9 Half-Spaces
- 2.10 Hyperplanes and Separating Surfaces
- 2.11 Norms and Metrics
- 2.12 Transformations
- 2.13 Homogeneous Coordinates
- 2.14 Duality
- 2.15 Topological Preliminaries
- 2.16 Exercises

Chapter 3

Geometric Predicates and Robust Computation

Geometric predicates are exact-sign tests that determine the combinatorial relationship between geometric objects. They are the foundation of the point, line, triangle, quadrilateral, tetrahedral, and hexahedral algorithms that follow. A predicate should return a discrete result such as positive, zero, or negative, rather than a constructed geometric object.

3.1 Orientation Predicate

For points $A, B, C \in \mathbb{R}^2$, define

$$\text{orient2d}(A, B, C) = (B_x - A_x)(C_y - A_y) - (B_y - A_y)(C_x - A_x).$$

The sign is positive when C lies to the left of the directed line $A \rightarrow B$, negative when it lies to the right, and zero when the three points are collinear. This test is used by convex hull, intersection, point-location, and triangulation algorithms.

Algorithm 1 Two-dimensional orientation test

```
1: function ORIENT2D( $A, B, C$ )  
2:    $u \leftarrow B_x - A_x$   
3:    $v \leftarrow B_y - A_y$   
4:    $w \leftarrow C_x - A_x$   
5:    $z \leftarrow C_y - A_y$   
6:   return  $uz - vw$   
7: end function
```

3.2 Three-dimensional Orientation

For points $A, B, C, D \in \mathbb{R}^3$, the signed volume predicate is

$$\text{orient3d}(A, B, C, D) = \det \begin{pmatrix} B - A \\ C - A \\ D - A \end{pmatrix}.$$

Its sign identifies which side of the oriented plane ABC contains D . Its zero case detects coplanarity. Tetrahedralization, volume-mesh validation, and tetrahedron–hexahedron conversion use this predicate.

3.3 In-circle Predicate

Given a counter-clockwise triangle ABC and a point D , the in-circle predicate determines whether D lies inside, on, or outside the circumcircle of ABC . It is evaluated from the sign of

$$\text{incircle}(A, B, C, D) = \det \begin{pmatrix} A_x & A_y & A_x^2 + A_y^2 & 1 \\ B_x & B_y & B_x^2 + B_y^2 & 1 \\ C_x & C_y & C_x^2 + C_y^2 & 1 \\ D_x & D_y & D_x^2 + D_y^2 & 1 \end{pmatrix}.$$

The test is central to Delaunay triangulation and edge-flip algorithms.

3.4 Robust Evaluation

Floating-point roundoff can change the sign of a nearly zero determinant and therefore change an algorithm's topology. Implementations should evaluate predicates with adaptive precision or exact arithmetic when the rounded result is uncertain. Epsilon thresholds may be useful for application-level tolerances, but they do not by themselves provide a consistent combinatorial result. Symbolic perturbation is another way to define deterministic answers for coincident, collinear, coplanar, or co-circular inputs.

The predicates in this chapter have constant arithmetic cost for fixed dimension. Their practical cost is dominated by the precision required to certify the sign.

3.5 Floating-Point Filters

3.5.1 Overview

A **floating-point filter** evaluates a predicate in floating point first and certifies the sign using an *a priori* error bound; only when the rounded result is too close to zero does it fall back to a slower, exact evaluation. Because ill-conditioned inputs are rare in practice, filters give the robustness of exact arithmetic at nearly floating-point speed [She97].

3.5.2 Static and Dynamic Filters

A **static filter** derives a fixed error bound from the magnitudes of the input coordinates before the computation begins; it is cheap but pessimistic. A **dynamic filter** computes the bound during the evaluation, tightening it as intermediate results are refined, so the exact path is entered only when it is genuinely needed [BBP01].

3.5.3 Adaptive Predicates

Shewchuk's adaptive predicates evaluate the orientation and in-circle determinants as a sequence of increasingly accurate approximations built from **expansion arithmetic**, stopping as soon as the sign is certified [She97]. The arithmetic rests on error-free transformations such as the *two-sum* and *two-product* primitives of Knuth and Priest: for IEEE 754 inputs these recover the roundoff error exactly, so the exact sign is obtained with no explicit multiple-precision representation [Pri91, Gol91]. In the common nondegenerate case the predicates run at roughly the speed of plain floating point.

3.5.4 Complexity

Each error-free transformation costs a small constant number of floating operations. The adaptive stage recomputes the determinant with increasing precision only along the path where roundoff

threatens the sign; the exact stage has cost proportional to the number of bits that must be carried.

3.6 Exact Arithmetic

3.6.1 Overview

The **exact geometric computation** (EGC) paradigm treats robustness as a non-issue by computing predicates exactly, and only when a new geometric object is constructed does it degrade to a certified numerical value [Yap97]. Exactness is achieved with arbitrary-precision integers, rationals, or floating-point expansions.

3.6.2 Integer and Rational Kernels

When input coordinates are given as integers (or rationals), every predicate sign is an exact integer computation. Fortune and Van Wyk generate specialized, unrolled evaluation code from an expression compiler so that the cost tracks the actual bit-length of the operands rather than paying a general-purpose library’s overhead [FVW93, FVW96]. The bit length grows only by a constant factor for each predicate: a 2D orientation of τ -bit integers needs $O(\tau)$ bits, and an in-circle test needs $O(\tau)$ as well.

3.6.3 Expansions

An **expansion** represents an exact value as the exact sum of non-overlapping floating-point components. The two-sum and two-product identities split a single floating-point addition or product into the rounded result and its exact error, and these identities compose into exact sums and products of arbitrary length [Pri91, She97]. This is the representation used by robust implementations of the orientation and in-circle predicates.

3.6.4 Exact Geometric Computation

Yap’s EGC framework adds *precision-driven* evaluation: expressions are evaluated to successively higher precision until a requested bound is met. It underwrites libraries such as CORE that combine rational and algebraic number types, and it extends exact computation beyond predicates to constructed objects like intersection points [Yap97, HMY04].

3.7 Rational Arithmetic in Computational Geometry

3.7.1 Overview

Rational arithmetic represents every number as a ratio of arbitrary-precision integers, so all predicates and all constructed points remain exact. It is the natural arithmetic for algorithms whose input is integer or rational, and it avoids the roundoff of floating point at the price of normalizing fractions and managing large numerators.

3.7.2 Application to Delaunay Triangulation

Karasick, Lieber, and Nackman showed that a Delaunay triangulation driven by rational arithmetic eliminates the failures caused by floating-point in-circle tests, at a cost that is acceptable for engineering input [KLN91]. The in-circle predicate for rational inputs reduces to a fixed combination of integer operations whose size is bounded by the input bit-length.

3.7.3 Trade-offs

Rational arithmetic is exact and uniform, but numerators can grow large when intersection points are repeatedly constructed. Hybrid designs combine a floating-point filter with a rational or integer fallback so that typical queries never touch the expensive exact kernel [FVW93, KLN91].

3.8 Simulation of Simplicity

3.8.1 Overview

Simulation of Simplicity (SoS) is a general technique for coping with degenerate inputs: it simulates an infinitesimal perturbation of the data that eliminates all coincident, collinear, coplanar, and co-circular configurations, yet the perturbation is never actually computed [EM90]. The programmer writes the algorithm for general position only; the tie-breaking is hidden inside the predicate implementations.

3.8.2 Construction

Each coordinate x_i is conceptually replaced by $x_i(\epsilon)$, a polynomial in an indeterminate ϵ , so that the perturbed predicate $P(x_0(\epsilon), \dots, x_{n-1}(\epsilon))$ is a nonzero polynomial whose sign at $\epsilon \rightarrow 0^+$ is the lexicographically first nonzero term. Evaluating only that leading term yields a deterministic, consistent answer for every degenerate input. Yap proved consistency for the corresponding symbolic perturbation scheme and extended the technique to arbitrary predicates [Yap88, Yap90].

3.8.3 Cost and Limitations

For determinant predicates the perturbed test costs only a small constant factor over the unperturbed one. SoS cannot make an inconsistent *algorithm* robust, and it ties degeneracies to a fixed symbolic order rather than to any geometric meaning; exact arithmetic with explicit degeneracy handling remains the alternative when the tie-break matters [EM90, HMY04].

3.9 Robustness Failure Examples

Floating-point predicates do not merely produce slightly wrong answers; they can make an algorithm crash, loop forever, or output an inconsistent structure. Kettner, Mehlhorn, Pion, Schirra, and Yap collected a suite of small, reproducible examples — such as a convex-hull routine whose topology changes under rotation, and a segment intersection where roundoff contradicts the planar subdivision — that expose these failures and are now used as regression tests for libraries like CGAL [KMP⁺08]. These examples motivate the filter and exact-arithmetic machinery above: certifying the sign of a predicate is the only way to guarantee a consistent combinatorial result.

Part II

Fundamental Planar Algorithms

Chapter 4

Convex Hulls

4.1 Graham Scan

The Graham scan is an efficient method for computing the convex hull of a finite set of points in the Euclidean plane. It was published by Ronald Graham in 1972 [Gra72] and is a fundamental algorithm in computational geometry. The algorithm finds all vertices of the convex hull ordered along its boundary.

4.1.1 Definitions

Definition 4.1 (Convex Hull). The *convex hull* of a set $S \subseteq \mathbb{R}^2$ is the smallest convex set containing all the points in S . Equivalently, it is the intersection of all convex sets containing S , or the set of all convex combinations of points in S :

$$\text{conv}(S) = \left\{ \sum_{i=1}^k \lambda_i p_i \mid k \in \mathbb{N}, p_i \in S, \lambda_i \geq 0, \sum_{i=1}^k \lambda_i = 1 \right\}.$$

Definition 4.2 (Cross Product in $\mathbb{R}^2$). For three points $A, B, C \in \mathbb{R}^2$, the *cross product* is defined as

$$\text{cross}(A, B, C) = (B.x - A.x)(C.y - A.y) - (B.y - A.y)(C.x - A.x).$$

It determines the orientation of the ordered triple (A, B, C) :

- Positive: Left turn (counter-clockwise).
- Negative: Right turn (clockwise).
- Zero: Collinear.

Geometrically, $|\text{cross}(A, B, C)|$ equals twice the signed area of triangle $\triangle ABC$.

Definition 4.3 (Anchor Point). The *anchor point* of a point set is the point with the lowest y -coordinate (and lowest x -coordinate in case of ties). It is guaranteed to lie on the convex hull.

4.1.2 Theory

The algorithm operates by first identifying an extreme point (the anchor) and sorting all other points based on the polar angle they make with this anchor. It then performs a linear scan through the sorted points, maintaining a stack of points that potentially form the hull. For each new point, the algorithm “backtracks” by popping points from the stack that would create a non-convex (right) turn. This ensures that the final stack contains only the vertices of the convex hull in counter-clockwise order.

Theorem 4.4 (Correctness of the Graham Scan). *Let P be a set of $n \geq 3$ points in general position in $\mathbb{R}^2$, and let p_0 be the anchor point. After sorting the remaining points by polar angle around p_0 , the Graham scan produces the vertices of $\text{conv}(P)$ in counter-clockwise order.*

Proof. The key invariant is that the stack always stores a sequence of points forming a left-turn chain. When a new point p_i is examined, any point(s) on the stack that would create a right turn (i.e., $\text{cross}(s_{k-1}, s_k, p_i) \leq 0$) are removed. Since every vertex of the convex hull must appear as a left turn when traversed counter-clockwise, no hull vertex is ever popped. Conversely, every non-hull point is eventually popped because it must lie inside the convex polygon formed by the hull vertices, and hence some subsequent hull vertex will create a right turn with it. The remaining stack therefore contains exactly the hull vertices in order. $\square$

Theorem 4.5 (Complexity of the Graham Scan). *The Graham scan computes the convex hull of n points in $O(n \log n)$ time and $O(n)$ space.*

Proof. The sorting step dominates with cost $O(n \log n)$. In the scan phase, each point is pushed onto the stack exactly once. Each point is popped at most once (since once popped, it is never re-examined). Thus the total number of push and pop operations is $O(n)$, making the scan phase $O(n)$ amortized. Space is $O(n)$ for the sorted list and stack. $\square$

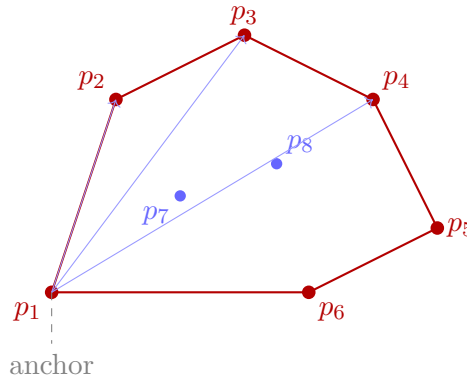


Figure 4.1: Graham Scan: the anchor point p_1 (lowest y) sorts all other points by polar angle. The hull vertices (red) are traced in counter-clockwise order.

4.1.3 Pseudocode

4.1.4 Parameters

- **Input:** A list of `Point2D` objects.
- **Sorting Criterion:** Polar angle relative to the anchor. In the implementation, `math.atan2` is often used, but cross-products can provide a more robust (and sometimes faster) comparison.

4.1.5 Complexity

- **Time Complexity:** $O(n \log n)$, where n is the number of points. This is dominated by the sorting step. The subsequent scan takes $O(n)$ time because each point is pushed and popped at most once.
- **Space Complexity:** $O(n)$ to store the sorted points and the stack.

Example 4.6 (Graham Scan Trace). Consider the point set $P = \{(0, 0), (1, 1), (2, 0), (1, -1), (0.5, 0.5)\}$.

Algorithm 2 Graham Scan for Convex Hull

```

1: function GRAHAMSCAN(points)
2:   if len(points) ≤ 2 then
3:     return points
4:   end if
5:   anchor ← min(points, key = λp : (p.y, p.x))
6:   sorted_points ← sorted(points, key = polar_angle_with_anchor)
7:   hull ← []
8:   for p in sorted_points do
9:     while len(hull) ≥ 2 and cross_product(hull[-2], hull[-1], p) ≤ 0 do
10:      hull.pop()
11:    end while
12:    hull.append(p)
13:  end for
14:  return hull
15: end function

```

1. The anchor point is (0,0) (lowest y , then lowest x).
2. Sorting by polar angle around (0,0) yields (approximately): (1, -1), (2, 0), (1, 1), (0.5, 0.5).
3. The scan proceeds: (0,0) and (1, -1) are placed on the stack. When (2,0) arrives, $\text{cross}((0,0), (1, -1), (2,0)) = 1 \cdot 0 - (-1) \cdot 2 = 2 > 0$, so we keep the chain. When (1,1) arrives, we check $\text{cross}((1, -1), (2,0), (1,1)) = 1 \cdot 1 - (-1) \cdot (-1) = 2 > 0$: no pop. When (0.5, 0.5) arrives, $\text{cross}((2,0), (1,1), (0.5,0.5)) = (-1)(0.5) - (1)(-1.5) = -0.5 + 1.5 = 1 > 0$: kept, but since (0.5, 0.5) lies in the interior of the hull formed by the other points, it will be interior to a final triangle and ultimately will not appear on the convex hull. (In practice, (0.5, 0.5) would be popped by a subsequent hull vertex check.)

The resulting hull is $\{(0,0), (1, -1), (2,0), (1,1)\}$.

4.1.6 Usages

- Collision detection in physics engines.
- Shape analysis and pattern recognition.
- Geographic Information Systems (GIS) for defining boundary regions.
- Minimum bounding box calculations.

4.1.7 Testing and Edge Cases

- **Few Points:** 0, 1, or 2 points should return the input points.
- **Collinear Points:** Points lying on the same line. The algorithm should handle them by either including them or only keeping the endpoints (depending on the `cross_product <= 0` vs `< 0` logic).
- **Duplicate Points:** Multiple points at the same coordinates.
- **Degenerate Cases:** All points at the same location.
- **Vertical/Horizontal alignment:** Points forming a perfect grid.

4.1.8 References

- Graham, R. L. (1972). “An Efficient Algorithm for Determining the Convex Hull of a Finite Planar Set”. Information Processing Letters.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). “Introduction to Algorithms”. MIT Press.
- Wikipedia: Graham scan.

4.2 Monotone Chain

The Monotone Chain algorithm (also known as Andrew’s algorithm) is a method for computing the convex hull of a finite set of points in the plane. It was first described by A. M. Andrew in 1979 [And79]. It is generally faster than the Graham scan because it avoids the calculation of polar angles and trigonometric functions, relying solely on sorting and cross-product tests.

4.2.1 Definitions

Definition 4.7 (Upper and Lower Hull). Let P be a set of points sorted lexicographically. The *lower hull* is the sequence of vertices of $\text{conv}(P)$ from the leftmost to the rightmost point, traversed along the bottom boundary. The *upper hull* is the analogous sequence traversed along the top boundary. Together they partition the hull boundary into two monotone chains.

Definition 4.8 (Orientation Test). The *orientation test* for three points $A, B, C \in \mathbb{R}^2$ computes the sign of $\text{cross}(A, B, C)$. It determines whether the path $A \rightarrow B \rightarrow C$ makes a left turn, right turn, or is collinear.

4.2.2 Theory

The algorithm takes advantage of the fact that the convex hull can be split into upper and lower chains. By sorting the points lexicographically (first by x -coordinate, then by y -coordinate), we can build the lower hull by scanning the sorted points from left to right. A similar process (scanning either the reversed points or right-to-left) yields the upper hull. For each chain, the algorithm maintains a stack and removes the last point if it doesn’t form a “valid” turn relative to the previous points.

Theorem 4.9 (Correctness of Monotone Chain). *Let P be a set of $n \geq 2$ points in $\mathbb{R}^2$. After sorting P lexicographically, the Monotone Chain algorithm correctly computes $\text{conv}(P)$.*

Proof. The sorted order ensures the lower hull is built left-to-right with monotonically increasing x -coordinates. At each step, the invariant is that the stack maintains a chain with all cross products positive (left turns). If adding a point p would create a non-positive cross product with the last two stack points, the middle point cannot be a vertex of the convex hull (it is “inside” the angle formed by the other two). Removing it preserves the invariant. The upper hull is built symmetrically. The concatenation of lower and upper hulls (with endpoints removed to avoid duplication) gives the full convex hull boundary. $\square$

4.2.3 Pseudocode

4.2.4 Parameters

- **Input:** A list of `Point2D` objects.
- **Sorting Key:** Points are sorted primarily by their x -coordinate and secondarily by their y -coordinate.

Algorithm 3 Monotone Chain (Andrew's Algorithm)

```

1: function MONOTONECHAIN(points)
2:   if len(points) ≤ 2 then
3:     return points
4:   end if
5:   points.sort()
6:   lower ← []
7:   for p in points do
8:     while len(lower) ≥ 2 and cross_product(lower[-2], lower[-1], p) ≤ 0 do
9:       lower.pop()
10:    end while
11:    lower.append(p)
12:  end for
13:  upper ← []
14:  for p in reversed(points) do
15:    while len(upper) ≥ 2 and cross_product(upper[-2], upper[-1], p) ≤ 0 do
16:      upper.pop()
17:    end while
18:    upper.append(p)
19:  end for
20:  return lower[: -1] + upper[: -1]
21: end function

```

4.2.5 Complexity

- **Time Complexity:** $O(n \log n)$ due to sorting. The construction phase is strictly $O(n)$, as each point is added and removed at most once for each hull.
- **Space Complexity:** $O(n)$ to hold the sorted points and the resulting hull.

4.2.6 Usages

- Standard 2D convex hull calculation in many libraries (e.g., SciPy as a fallback).
- Calculating the area of a set of points.
- Simplifying complex polygons.
- GIS and map boundary creation.

4.2.7 Testing and Edge Cases

- **Vertical Lines:** Points with the same x -coordinate but different y -coordinates.
- **Collinear Points:** Handled by the ≥ 2 and $\text{cross_product} \leq 0$ conditions.
- **Identical Points:** Lexicographical sort handles duplicates naturally.
- **Small Sets:** Sets of 0, 1, or 2 points return early.
- **Closed Hulls:** Ensure the first point and last point of each chain match up correctly at the extremities.

4.2.8 References

- Andrew, A. M. (1979). “Another Efficient Algorithm for Convex Hulls in Two Dimensions”. Information Processing Letters.
- Preparata, F. P., & Shamos, M. I. (1985). “Computational Geometry: An Introduction”. Springer-Verlag.
- Wikipedia: Monotone chain.

4.3 Chan’s Algorithm

Chan’s algorithm is an **output-sensitive** algorithm for computing the convex hull of a set of n points in 2D or 3D. It combines the strengths of Graham scan (or Algorithm 3) and Jarvis march to achieve an optimal $O(n \log h)$ running time, where h is the number of vertices on the convex hull. It was introduced by Chan in 1996 [Cha96].

4.3.1 Definitions

Definition 4.10 (Output-Sensitive Algorithm). An algorithm is *output-sensitive* if its running time depends not only on the input size n but also on the output size (in this case, h , the number of hull vertices).

4.3.2 Theory

Chan’s algorithm partitions the point set into n/m groups of size m . It computes the convex hull of each group in $O(m \log m)$ time. Then, it uses a modified Jarvis march to find the global hull by performing binary searches (tangent queries) on the mini-hulls. By iteratively guessing $m = 2^{2^t}$, it reaches the optimal complexity.

Theorem 4.11 (Complexity of Chan’s Algorithm). *Chan’s algorithm computes the convex hull of n points in $O(n \log h)$ time and $O(n)$ space, where h is the number of vertices on the convex hull.*

Proof. The algorithm tries group sizes $m = 2, 4, 16, 256, \dots, 2^{2^t}$. For the correct guess where $m \geq h$, each group hull is computed in $O(m \log m)$ time, and the Jarvis march finds the hull in $O(m \log m)$ tangent queries (each taking $O(\log m)$ time on a group hull of size $\leq m$). The total work is $O((n/m) \cdot m \log m) = O(n \log m) = O(n \log h)$. All previous incorrect guesses $m' < h$ contribute at most $\sum_t O(n \log 2^{2^t}) = O(n)$ total work by a geometric series argument. $\square$

4.3.3 Pseudocode

4.3.4 Parameters

- **Group Size (m):** Controlled by the iteration parameter t .

4.3.5 Complexity

- **Time:** $O(n \log h)$, which is optimal.
- **Space:** $O(n)$.

4.3.6 Usages

Implemented in `compgeom.polygon.convex_hull.Chan`. Used for efficient hull calculation when the number of hull vertices is small.

Algorithm 4 Chan’s Convex Hull Algorithm

```

1: function CHANSCONVEXHULL( $P$ )
2:   for  $t = 1, 2, \dots$  do
3:      $m \leftarrow \min(2^{2^t}, n)$ 
4:     Partition  $P$  into groups of size  $m$ 
5:     Compute Monotone Chain hull for each group
6:      $\triangleright$  Try to find global hull using Jarvis march in  $m$  steps
7:     From current point, find next hull point by tangent query on each mini-hull
8:     if hull is closed then
9:       return hull
10:    end if
11:  end for
12: end function

```

4.3.7 Testing and Edge Cases

- **Testing:** Compare output with Graham Scan or SciPy’s Qhull. Verify all original points are inside the hull.
- **Edge Cases:** Collinear points, coincident points, small point sets ($n < 3$).

4.3.8 References

- Chan, T. M. (1996). “Optimal output-sensitive convex hull algorithms in two and three dimensions”. Discrete & Computational Geometry.
- Wikipedia: Chan’s algorithm.

4.4 QuickHull

QuickHull is a divide-and-conquer algorithm for computing the convex hull of a finite set of points. It is similar in design to the Quicksort sorting algorithm. It was independently published by several researchers, including Barber et al. (1996) [BDH96] and Eddy (1977) [Edd77], and is the basis of the widely used Qhull library.

4.4.1 Definitions

Definition 4.12 (Distance from a Line). The *perpendicular distance* from a point P to a line through points A and B is

$$d(P, \overline{AB}) = \frac{|\text{cross}(A, B, P)|}{\|B - A\|}.$$

Points farthest from a line segment are guaranteed to be vertices of the convex hull.

4.4.2 Theory

The algorithm begins by finding the leftmost and rightmost points, which are guaranteed to be on the hull. These points divide the point set into two subsets by a line. For each subset, the point farthest from the line is identified. This point, along with the two original points, forms a triangle. Points inside this triangle cannot be on the convex hull and are discarded. The process is then recursively applied to the two edges of the triangle that are not part of the original line.

Theorem 4.13 (QuickHull Average-Case Complexity). *The average-case time complexity of QuickHull is $O(n \log n)$ when points are distributed according to a smooth distribution (e.g., uniformly in a convex region).*

Proof. The argument mirrors that of Quicksort. On average, the farthest point partitions the remaining points into two roughly balanced subsets. If each recursive step discards a constant fraction of the remaining points, the depth of recursion is $O(\log n)$ and the total work is $O(n \log n)$. In the worst case (e.g., points on a circle), the partition is maximally unbalanced and the complexity degrades to $O(n^2)$. $\square$

4.4.3 Pseudocode

Algorithm 5 QuickHull Algorithm

```

1: function QUICKHULL(points)
2:   leftmost  $\leftarrow$  min(points)
3:   rightmost  $\leftarrow$  max(points)
4:   upper  $\leftarrow$  points above line (leftmost, rightmost)
5:   lower  $\leftarrow$  points below line (leftmost, rightmost)
6:   return [leftmost] + FindHull(leftmost, rightmost, upper) + [rightmost] +
      FindHull(rightmost, leftmost, lower)
7: end function
8: function FINDHULL(A, B, candidates)
9:   if candidates is empty then
10:    return []
11:  end if
12:  P  $\leftarrow$  arg maxp ∈ candidates dist_from_line(A, B, p)
13:  left_of_AP  $\leftarrow$  [p | p ∈ candidates ∧ is_left_of(A, P, p)]
14:  left_of_PB  $\leftarrow$  [p | p ∈ candidates ∧ is_left_of(P, B, p)]
15:  return FindHull(A, P, left_of_AP) + [P] + FindHull(P, B, left_of_PB)
16: end function

```

4.4.4 Parameters

- **Input:** A set of points (list of `Point2D`).
- **Recursive termination:** When the subset of candidate points for a segment is empty.

4.4.5 Complexity

- **Time Complexity:** Average case $O(n \log n)$. Worst case $O(n^2)$ when points are distributed such that the partition is highly unbalanced (e.g., points on a circle).
- **Space Complexity:** $O(n)$ to store the points and recursion stack.

4.4.6 Usages

- The primary algorithm in the `Qhull` library, used by SciPy, MATLAB, and R.
- General-purpose convex hull calculation in 2D and 3D.
- Collision detection and mesh simplification.

4.4.7 Testing and Edge Cases

- **Highly Symmetric Points:** Points on a circle can lead to the worst-case $O(n^2)$ performance.
- **Collinear Points:** Farthest point might not be unique; handle tie-breaking consistently.
- **Points inside the hull:** Ensure that the pruning (ignoring points inside triangles) is correct.
- **Numerical Precision:** Use an epsilon for distance and cross-product comparisons to avoid issues with floating-point arithmetic.

4.4.8 References

- Barber, C. B., Dobkin, D. P., & Huhdanpaa, H. (1996). “The Quickhull Algorithm for Convex Hulls”. ACM Transactions on Mathematical Software (TOMS).
- Eddy, W. F. (1977). “A New Convex Hull Algorithm for Planar Sets”. ACM Transactions on Mathematical Software (TOMS).
- Bykat, A. (1978). “Convex Hull of a Finite Set of Points in Two Dimensions”. Information Processing Letters.
- Wikipedia: Quickhull.

4.5 Convex Hull Peeling

Convex hull peeling, also known as “onion peeling”, is a recursive process of identifying and removing the convex hull of a point set. After the first hull is removed, the process is repeated on the remaining points until no points are left. This decomposition organizes a set of points into nested “layers,” similar to the layers of an onion. It was studied by Chazelle [Cha85], Overmars and van Leeuwen [OvL81], and is closely related to the concept of Tukey depth introduced by Tukey [Tuk75]. It is a fundamental technique for robust statistics, data depth analysis, and identifying outliers.

4.5.1 Definitions

Definition 4.14 (Convex Hull Layer). A *convex hull layer* is the set of points forming the boundary of the convex hull at a specific iteration of the peeling process.

Definition 4.15 (Convex Hull Depth). The *convex hull depth* (or layer number) of a point is the iteration number at which it is removed. Points on the outermost hull have depth 1. The innermost layer defines the **Tukey depth** of the deepest points.

4.5.2 Theory

The onion peeling of a point set P provides a natural way to measure the “centrality” of points.

1. Compute the convex hull $CH(P)$.
2. The points on the boundary of $CH(P)$ form the first layer L_1 .
3. Let $P' = P \setminus L_1$.
4. Repeat the process on P' to find $L_2, L_3, \dots, L_k$.

Theorem 4.16 (Layer Count). *For n points distributed uniformly at random in a unit square, the expected number of layers is $\Theta(n^{2/3})$.*

Proof. Each convex hull of a random point set in the unit square has $O(n^{1/3})$ vertices on average. Removing these vertices reduces the remaining set by this amount. The number of layers before the set is exhausted is thus $O(n/n^{1/3}) = O(n^{2/3})$. A matching lower bound follows from concentration arguments on the extreme point count. $\square$

In 2D, the total number of layers k is roughly $O(n^{2/3})$ for points distributed uniformly in a square. The structure of the layers can be used to define the Tukey Depth of points in a multivariate dataset.

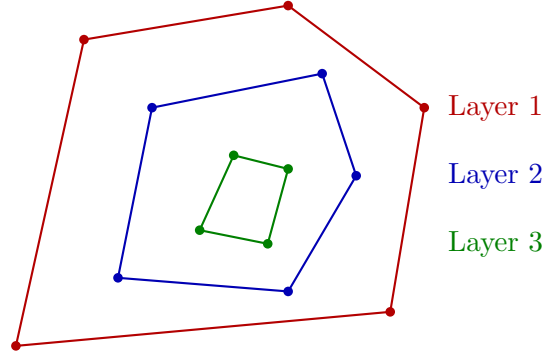


Figure 4.2: Convex hull peeling (onion peeling): successive convex hulls are removed, producing nested layers.

4.5.3 Pseudocode

Algorithm 6 Convex Hull Peeling (Onion Peeling)

```

1: function CONVEXHULLPEELING(points)
2:   layers  $\leftarrow []$ 
3:   current_points  $\leftarrow \text{set}(\text{points})$ 
4:   while current_points is not empty do
5:     if len(current_points) < 3 then
6:       layers.append(list(current_points))
7:       break
8:     end if
9:     hull_indices  $\leftarrow \text{ComputeConvexHull}(\text{list}(\text{current\_points}))$ 
10:    hull_points  $\leftarrow [\text{list}(\text{current\_points})[i] \mid i \in \text{hull\_indices}]$ 
11:    layers.append(hull_points)
12:    for p in hull_points do
13:      current_points.remove(p)
14:    end for
15:  end while
16:  return layers
17: end function

```

4.5.4 Parameters

- **Hull Algorithm:** Monotone Chain or Graham Scan are ideal for 2D. For 3D, QuickHull is standard.

- **Handling Collinearity:** Points that lie exactly on the edges of the convex hull can either be included in the current layer or treated as interior points for the next layer. Including them is more common in statistics.

4.5.5 Complexity

- **Naive Complexity:** $O(n^2)$ if a new $O(n \log n)$ hull is calculated for every point removed.
- **Efficient 2D Complexity:** $O(n \log n)$ using Chazelle’s algorithm [Cha85], which uses a more complex data structure to update the hull as points are removed.
- **Space Complexity:** $O(n)$ to store the point set and the layer indices.

4.5.6 Usages

- **Robust Statistics:** Identifying the “geometric median” of a point set by finding the deepest onion layer.
- **Outlier Detection:** Points in the first few layers are candidates for being outliers or “extreme” values.
- **Data Visualization:** Creating “bagplots” or “boxplots” for 2D data.
- **Pattern Recognition:** Describing the density and distribution of a point cloud.
- **Computational Geometry:** A preprocessing step for certain range-searching and visibility problems.

Example 4.17 (Onion Peeling by Hand). Consider a 3×3 grid of points $P = \{(i, j) : i, j \in \{0, 1, 2\}\}$ with 9 total points.

1. Layer 1 (outermost hull): The 8 boundary points $\{(0, 0), (1, 0), (2, 0), (2, 1), (2, 2), (1, 2), (0, 2), (0, 1)\}$.
2. After peeling, only $\{(1, 1)\}$ remains.
3. Layer 2: The single point $\{(1, 1)\}$.

The center point has hull depth 2, while all other points have depth 1.

4.5.7 Testing and Edge Cases

- **Perfectly Collinear Points:** All points should be removed in the first layer.
- **Points on a Circle:** All points should be removed in the first layer.
- **Grid Arrangement:** Verify that the peeling correctly identifies the nested squares or rectangles.
- **Small Sets:** Test with $n = 1, 2, 3$.
- **Symmetry:** For a symmetric distribution, the layers should maintain that symmetry.

4.5.8 References

- Chazelle, B. (1985). “On the convex layers of a planar set”. *IEEE Transactions on Information Theory*.
- Overmars, M. H., & Leeuwen, J. van. (1981). “Maintenance of configurations in the plane”. *Journal of Computer and System Sciences*.
- Tukey, J. W. (1975). “Mathematics and the picturing of data”. *Proceedings of the International Congress of Mathematicians*.
- Wikipedia: Convex layers.

Chapter 5

Line-Segment Intersection

- 5.1 The Intersection Problem
- 5.2 Pairwise Intersection Testing
- 5.3 Sweep-Line Paradigm
- 5.4 Events and Event Queues
- 5.5 Sweep-Line Status Structures
- 5.6 Bentley–Ottmann Algorithm
- 5.7 Correctness
- 5.8 Complexity Analysis
- 5.9 Degenerate Intersections
- 5.10 Reporting versus Detecting Intersections
- 5.11 Red–Blue Segment Intersection
- 5.12 Contour of the Union of Rectangles

5.12.1 Overview

The contour of the union of rectangles problem asks for the boundary of the region covered by a set of axis-parallel rectangles. It is a classic application of the sweep-line paradigm: a vertical line sweeps the plane while a status structure maintains the active set of rectangles, and the contour is assembled from the events at which the union boundary changes. Applications include VLSI mask checking, building footprints in GIS, and windowing in computer graphics [BCKO08].

5.12.2 Definitions

Definition 5.1 (Union Contour). The *contour* of a set of rectangles is the boundary of their union: the set of points on the union’s frontier that are not interior to any rectangle.

Definition 5.2 (Contour Break Point). A *break point* is a point on the contour where the boundary changes direction, occurring at rectangle corners or at intersections between rectangle edges.

5.12.3 Complexity

With n axis-parallel rectangles, the sweep processes $2n$ vertical edge events, each involving an $O(\log n)$ update of the interval status structure. The running time is $O(n \log n)$; the output has $O(k)$ segments, where k is the number of contour break points.

5.13 Applications

5.14 Exercises

5.15 Project: Interactive Sweep-Line Visualizer

Chapter 6

Polygon Geometry

6.1 Polygon Boolean Operations

6.1.1 1. Overview

Polygon Boolean operations are a set of geometric algorithms used to compute the result of applying set operations (union, intersection, difference, and symmetric difference) to two or more polygons [Vat92]. These operations are essential in computer graphics, CAD/CAM, and GIS for manipulating complex shapes and defining regions of interest.

6.1.2 2. Definitions

Definition 6.1 (Polygon Union). The *union* $A \cup B$ of two polygons A and B is the region covered by at least one of the input polygons.

Definition 6.2 (Polygon Intersection). The *intersection* $A \cap B$ of two polygons A and B is the region shared by both input polygons.

Definition 6.3 (Polygon Difference). The *difference* $A - B$ is the region inside polygon A but outside polygon B .

Definition 6.4 (Symmetric Difference). The *symmetric difference* $A \Delta B$ is the region inside either A or B , but not both. Formally, $A \Delta B = (A - B) \cup (B - A)$.

Definition 6.5 (Winding Rule). A *winding rule* (e.g., Even-Odd or Non-Zero winding number) is a rule used to determine the “inside” of a self-intersecting or hole-containing polygon. The Even-Odd rule considers a point inside if a ray from it crosses an odd number of edges; the Non-Zero rule considers the winding number (total signed crossings).

6.1.3 3. Theory

The most common approach to polygon Boolean operations is based on **edge clipping and subdivision** [GH98]. The algorithm typically involves:

1. **Intersection Detection:** Finding all points where edges of polygon A intersect edges of polygon B .
2. **Edge Subdivision:** Splitting edges at intersection points into smaller segments.
3. **Edge Classification:** For each segment, determining if it is “Inside,” “Outside,” or “Shared” relative to the other polygon.

4. **Loop Assembly:** Collecting the appropriate segments based on the desired operation and assembling them into new closed loops.

Theorem 6.6 (Polygon Boolean Complexity). *Let A and B be two simple polygons with n and m vertices respectively, producing k intersection points. The Boolean operations $A \cup B$, $A \cap B$, $A - B$, and $A \Delta B$ can all be computed in $O((n + m + k) \log(n + m))$ time using a sweep-line algorithm [Vat92].*

Proof. The algorithm proceeds in three phases. In phase 1, we find all k edge-edge intersections using a Bentley–Ottmann sweep line in $O((n + m + k) \log(n + m))$ time. In phase 2, we subdivide and classify each edge segment in $O(1)$ per segment (total $O(n + m + k)$). In phase 3, we assemble result polygons by traversing the doubly-linked edge structures in $O(n + m + k)$ time. The overall bound is dominated by the sweep. $\square$

Example 6.7 (Polygon Intersection Trace). Let A be the square $[0, 3] \times [0, 3]$ and B the square $[1, 4] \times [1, 4]$.

1. **Intersection detection:** The edges intersect at $(1, 3)$, $(3, 1)$, $(1, 1)$, and $(3, 3)$. Wait: A 's right edge $x = 3$ intersects B 's bottom edge $y = 1$ at $(3, 1)$; A 's top edge $y = 3$ intersects B 's left edge $x = 1$ at $(1, 3)$. There are $k = 2$ intersection points: $(3, 1)$ and $(1, 3)$.
2. **Edge subdivision:** A 's right edge is split into $[(3, 0), (3, 1)]$ and $[(3, 1), (3, 3)]$; A 's top edge is split into $[(0, 3), (1, 3)]$ and $[(1, 3), (3, 3)]$. Similarly for B 's edges.
3. **Classification:** For $A \cap B$, we keep only segments that lie inside both polygons: $[(3, 1), (3, 3)]$ (right edge of A , inside B), $[(1, 3), (3, 3)]$ (top edge of A , inside B), plus segments from B inside A .
4. **Assembly:** The resulting intersection is the square $[1, 3] \times [1, 3]$ with vertices $(1, 1)$, $(3, 1)$, $(3, 3)$, $(1, 3)$.

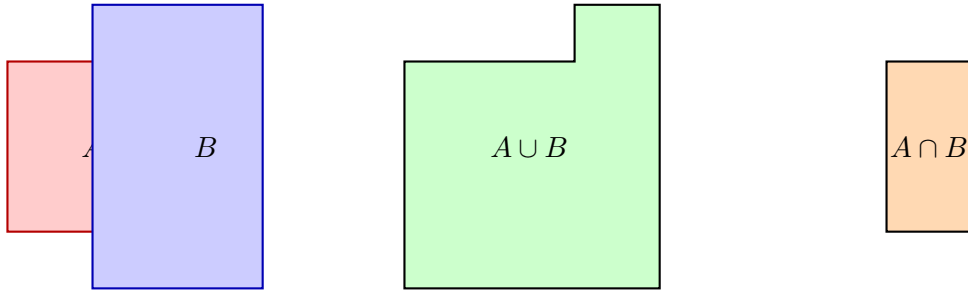


Figure 6.1: Polygon boolean operations: Union $A \cup B$ (center) and Intersection $A \cap B$ (right) of two overlapping rectangles.

6.1.4 4. Pseudo code

6.1.5 5. Parameters Selections

Definition 6.8 (Precision (Epsilon)). The *precision* ϵ is a floating-point tolerance used to determine if a vertex lies on an edge or if two edges are collinear. It is crucial for numerical robustness.

Definition 6.9 (Consistent Winding). *Consistent winding* requires that both input polygons use the same orientation convention (e.g., both counter-clockwise). Most algorithms assume counter-clockwise orientation for all inputs.

```

1: function POLYGONBOOLEAN(A, B, operation)      ▷ 1. Split edges at intersection points
2:   subdivided_A ← SplitEdges(A, B)
3:   subdivided_B ← SplitEdges(B, A)
                                                    ▷ 2. Classify segments
4:   classified_A ← Classify(subdivided_A, B)
5:   classified_B ← Classify(subdivided_B, A)
                                                    ▷ 3. Filter segments based on operation
6:   result_segments ← []
7:   if operation == "UNION" then
8:     result_segments += [s for s in classified_A if s.is_outside]
9:     result_segments += [s for s in classified_B if s.is_outside]
10:  else if operation == "INTERSECTION" then
11:    result_segments += [s for s in classified_A if s.is_inside]
12:    result_segments += [s for s in classified_B if s.is_inside]
13:  end if
                                                    ▷ 4. Assemble into polygons
14:  return AssemblePolygons(result_segments)
15: end function

```

6.1.6 6. Complexity

Theorem 6.10 (Polygon Boolean Complexity Summary). *For two simple polygons A (n vertices) and B (m vertices) with k intersection points:*

1. **Time:** $O((n + m + k) \log(n + m))$ using sweep-line intersection detection [Vat92].
2. **Space:** $O(n + m + k)$ for the subdivided edges and output polygon.

6.1.7 7. Usages

- **CSG (Constructive Solid Geometry):** Building complex 3D objects from simple primitives.
- **GIS Clipping:** Extracting spatial data within a specific boundary (e.g., finding all roads in a county).
- **PCB Design:** Calculating overlapping areas of copper layers or solder masks.
- **UI Layout:** Determining visible regions of windows and buttons.

6.1.8 8. Testing methods and Edge cases

- **Collinear Edges:** Edges that overlap exactly or partially.
- **Shared Vertices:** Polygons touching at a single point or along a common edge.
- **Holes:** Polygons containing internal voids or “islands.”
- **Self-Intersecting Inputs:** May require preprocessing into a set of simple polygons.
- **Degeneracies:** Zero-area polygons resulting from subtraction.

6.1.9 9. References

- Vatti, B. R. (1992). “A generic solution to polygon clipping”. Communications of the ACM.
- Greiner, G., & Hormann, K. (1998). “Efficient clipping of arbitrary polygons”. ACM TOG.
- Weiler, K., & Atherton, P. (1977). “Hidden surface removal using polygon area subdivision”. SIGGRAPH.
- [Wikipedia: Boolean operations on polygons](#)

6.2 Line Arrangements

6.2.1 1. Overview

A **line arrangement** is the subdivision of the 2D plane induced by a set of n lines. It is a fundamental structure in computational geometry that encodes all intersection points and the topological relationships between the lines (vertices, edges, and faces) [EOS86]. Arrangements provide a geometric framework for solving problems involving duality, range searching, and visibility.

6.2.2 2. Definitions

Definition 6.11 (Arrangement). Given a set L of n lines, the *arrangement* $\mathcal{A}(L)$ is the subdivision of the plane into vertices (intersection points), edges (maximal connected portions of lines between vertices), and faces (maximal connected regions of the plane not containing any line).

Definition 6.12 (Zone). The *zone* of a line ℓ in an arrangement $\mathcal{A}(L)$ is the set of faces in $\mathcal{A}(L)$ that are intersected by ℓ .

Definition 6.13 (DCEL). The *Doubly-Connected Edge List* (DCEL) is the standard data structure for representing arrangements. It stores for each edge: an origin vertex, a destination vertex, a pointer to the next edge around the face, and a pointer to the twin (opposite) edge.

6.2.3 3. Theory

For n lines in **general position** (no two lines parallel, no three lines concurrent):

- Number of vertices: $\binom{n}{2} = \frac{n(n-1)}{2}$.
- Number of edges: n^2 .
- Number of faces: $\frac{n^2+n+2}{2}$.

The standard way to build an arrangement is through **incremental construction**:

1. Start with a simple arrangement of two lines.
2. Add lines one by one. For each new line ℓ_i :
 - Find the face containing one end of ℓ_i .
 - Trace the path of ℓ_i through the arrangement using the **Zone Theorem**.
 - Split edges and faces as ℓ_i passes through them.

Theorem 6.14 (Zone Theorem). *In an arrangement of n lines, the zone of a single line ℓ has $O(n)$ complexity, i.e., it intersects $O(n)$ edges and faces [EOS86].*

Proof. Let ℓ be the new line. As ℓ traverses the arrangement from left to right, each time it crosses an edge of the arrangement, it enters a new face. Each existing line l_j intersects ℓ in at most one point, contributing at most 2 edges to the zone boundary (one above ℓ , one below). Since there are $n - 1$ other lines, the total zone complexity is at most $2(n - 1) + 1 = O(n)$. $\square$

Theorem 6.15 (Arrangement Construction Complexity). *An arrangement of n lines can be constructed in $O(n^2)$ time and stored in $O(n^2)$ space using the incremental algorithm based on the Zone Theorem [EOS86].*

Proof. We add n lines one at a time. For the i -th line, the Zone Theorem guarantees that its zone in the arrangement of the first $i - 1$ lines has $O(i)$ complexity. The cost of inserting line i is therefore $O(i)$. Summing over all n lines: $\sum_{i=1}^n O(i) = O(n^2)$. $\square$

Example 6.16 (Arrangement of 3 Lines). Consider three lines in general position: $\ell_1 : y = 0$, $\ell_2 : y = x$, $\ell_3 : y = -x + 2$.

- $\binom{3}{2} = 3$ vertices: $(0, 0)$, $(1, 0)$, $(1, 1)$.
- $3^2 = 9$ edges.
- $\frac{9+3+2}{2} = 7$ faces: 1 bounded triangle and 6 unbounded regions.

The bounded face is the triangle with vertices $(0, 0)$, $(1, 0)$, and $(1, 1)$.

6.2.4 4. Pseudo code

<pre> 1: function BUILDARRANGEMENT(lines) 2: arrangement $\leftarrow$ InitializeDCEL() 3: for line in lines do 4: face $\leftarrow$ LocateFace(arrangement, line.start_point) 5: current_p $\leftarrow$ line.start_point 6: while not IsFinished(current_p) do 7: exit_edge $\leftarrow$ FindExitEdge(face, line) 8: arrangement.SplitFace(face, line, exit_edge) 9: face $\leftarrow$ arrangement.GetAdjacentFace(exit_edge) 10: current_p $\leftarrow$ arrangement.GetIntersection(line, exit_edge) 11: end while 12: end for 13: return arrangement 14: end function </pre>	<pre> $\triangleright$ 1. Initialize with a large bounding box $\triangleright$ 2. Add lines incrementally $\triangleright$ 3. Locate start face $\triangleright$ 4. Trace the line through the arrangement (Zone Walk) $\triangleright$ Find which edge of the current face the line exits through $\triangleright$ Split the face and update the DCEL $\triangleright$ Move to the adjacent face </pre>
---	--

6.2.5 5. Parameters Selections

Definition 6.17 (DCEL Representation). The *Doubly-Connected Edge List (DCEL)* is the standard choice for representing arrangements as it allows for efficient traversal of vertices, edges, and faces in $O(1)$ per step.

Definition 6.18 (General Position Handling). If lines can be parallel or concurrent (degenerate), the construction algorithm must use robust geometric predicates and degeneracy handling. In practice, a symbolic perturbation or an epsilon-based approach is used.

6.2.6 6. Complexity

Theorem 6.19 (Arrangement Complexity Summary). *For n lines in general position:*

1. **Time:** $O(n^2)$ for construction using the incremental algorithm with the Zone Theorem.
2. **Space:** $O(n^2)$ to store all vertices, edges, and faces.
3. **Face Count:** $\frac{n^2+n+2}{2} = O(n^2)$.

6.2.7 7. Usages

- **Duality Problems:** Many problems involving points can be solved by mapping them to lines in a dual plane and analyzing their arrangement (e.g., finding the maximum number of collinear points) [CGL85].
- **Visibility:** Computing the visibility graph of a set of obstacles.
- **Motion Planning:** Analyzing the complexity of the configuration space for a robot.
- **Range Searching:** Identifying all points within a specific region.
- **Statistical Analysis:** Finding the Theil-Sen estimator or other robust regression lines.

6.2.8 8. Testing methods and Edge cases

- **Parallel Lines:** Verify that the algorithm correctly creates regions between lines that never intersect.
- **Concurrent Lines:** Ensure that three or more lines meeting at a single point are handled without creating zero-length edges.
- **Horizontal/Vertical Lines:** Test with lines aligned to the axes.
- **Small n :** Verify the formula for $n = 1, 2, 3$.
- **Bounding Box:** Ensure the arrangement handles lines that extend to infinity correctly by using a sufficiently large bounding volume or symbolic coordinates.

6.2.9 9. References

- Edelsbrunner, H., O'Rourke, J., & Seidel, R. (1986). "Constructing arrangements of lines and hyperplanes with applications". SIAM Journal on Computing.
- Chazelle, B., Guibas, L. J., & Lee, D. T. (1985). "The power of geometric duality". BIT Numerical Mathematics.
- Berg, M. de, Cheong, O., Kreveld, M. van, & Overmars, M. (2008). "Computational Geometry: Algorithms and Applications". Springer-Verlag.
- Wikipedia: [Arrangement of lines](#)

6.3 Minkowski Sums

6.3.1 1. Overview

The **Minkowski sum** of two sets of points A and B is the set of all possible sums of a point from A and a point from B [LP83]. In computational geometry, this operation is primarily used with polygons (2D) or polyhedra (3D). It is a fundamental tool for motion planning, collision detection, and calculating the offset of shapes.

6.3.2 2. Definitions

Definition 6.20 (Minkowski Sum). The *Minkowski sum* of two point sets $A, B \subseteq \mathbb{R}^2$ is defined as $A \oplus B = \{a + b \mid a \in A, b \in B\}$.

Definition 6.21 (Minkowski Difference). The *Minkowski difference* is $A \ominus B = \{a - b \mid a \in A, b \in B\} = A \oplus (-B)$. Two shapes A and B intersect if and only if $\mathbf{0} \in A \ominus B$.

Definition 6.22 (Convex Polygon Minkowski Sum). For two *convex polygons* P and Q with n and m vertices respectively, their Minkowski sum $P \oplus Q$ is also a convex polygon with at most $n + m$ vertices.

6.3.3 3. Theory

For two convex polygons P and Q with n and m vertices, the Minkowski sum is computed by:

1. Collect all the edges of P and Q .
2. Orient all edges counter-clockwise.
3. Sort all edges by their polar angle.
4. Chain the edges together starting from the sum of the bottom-left vertices of P and Q .

Theorem 6.23 (Convex Minkowski Sum Complexity). *The Minkowski sum of two convex polygons P (n vertices) and Q (m vertices) can be computed in $O(n + m)$ time and space [GS87].*

Proof. The edges of both polygons, when sorted by polar angle, form two sorted sequences (since both are convex). Merging two sorted sequences of total length $n + m$ takes $O(n + m)$ time. The resulting edge chain is traversed in $O(n + m)$ to form the sum polygon. $\square$

For **non-convex polygons**, the Minkowski sum is much more complex. The standard approach is to:

1. Decompose both non-convex polygons into convex pieces (e.g., using convex decomposition).
2. Compute the Minkowski sum for every pair of convex pieces ($P_i \oplus Q_j$).
3. Compute the union of all the resulting convex Minkowski sums.

Theorem 6.24 (Non-Convex Minkowski Sum Complexity). *For two simple (possibly non-convex) polygons P (n vertices) and Q (m vertices), the Minkowski sum can be computed in $O(n^2 m^2)$ worst-case time.*

Proof. A convex decomposition of P yields at most $O(n)$ convex pieces, and similarly $O(m)$ for Q . Computing each pairwise sum $P_i \oplus Q_j$ takes $O(|P_i| + |Q_j|)$ time by Theorem 6.23. The total work for $O(nm)$ pairwise sums is $O(n^2 m^2)$ in the worst case (accounting for the union step). $\square$

Example 6.25 (Minkowski Sum of Two Squares). Let $P = \{(0, 0), (1, 0), (1, 1), (0, 1)\}$ (unit square) and $Q = \{(0, 0), (2, 0), (2, 2), (0, 2)\}$ (2×2 square). Both are CCW.

1. Edges of P : $(1, 0), (0, 1), (-1, 0), (0, -1)$.
2. Edges of Q : $(2, 0), (0, 2), (-2, 0), (0, -2)$.
3. Sorted by angle: $(1, 0), (2, 0), (0, 1), (0, 2), (-1, 0), (-2, 0), (0, -1), (0, -2)$.
4. Starting from $(0, 0) + (0, 0) = (0, 0)$, chain the edges: $(0, 0) \rightarrow (1, 0) \rightarrow (3, 0) \rightarrow (3, 1) \rightarrow (3, 3) \rightarrow (2, 3) \rightarrow (0, 3) \rightarrow (0, 2) \rightarrow (0, 0)$.
5. Result: A 3×3 square (after merging collinear edges), with 4 vertices.

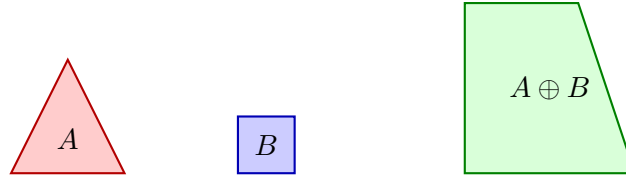


Figure 6.2: Minkowski sum of a triangle A and a unit square B . The result is a pentagon with at most $n + m$ vertices.

6.3.4 4. Pseudo code

```

1: function MINKOWSKI SUM CONVEX(P, Q)           ▷ P and Q are CCW convex polygons
2:   edges_P ← GetOrientedEdges(P)
3:   edges_Q ← GetOrientedEdges(Q)
4:   all_edges ← sorted(edges_P + edges_Q, key=lambda e: atan2(e.dy, e.dx))
                                                    ▷ Sort all edges by polar angle
                                                    ▷ Start at the sum of bottom-left vertices
5:   start_P ← min(P, key=lambda p: (p.y, p.x))
6:   start_Q ← min(Q, key=lambda p: (p.y, p.x))
7:   current ← start_P + start_Q
8:   result ← [current]
9:   for e in all_edges do
10:     current ← current + e.vector
11:     result.append(current)
12:   end for
13:   return result[:-1]                             ▷ Remove duplicate start/end
14: end function

```

6.3.5 5. Parameters Selections

Definition 6.26 (Input Orientation Requirement). Polygons MUST be oriented consistently (e.g., both counter-clockwise) before computing the Minkowski sum. Inconsistent orientation produces incorrect edge orderings.

Definition 6.27 (Convex Decomposition Method). The choice of convex decomposition (exact vs. approximate) for non-convex inputs significantly affects the performance and quality of the non-convex Minkowski sum.

6.3.6 6. Complexity

Theorem 6.28 (Minkowski Sum Complexity Summary). 1. **Convex Case:** $O(n+m)$ time and space for convex polygons P (n vertices) and Q (m vertices) [GS87].

2. **Non-Convex Case:** $O(n^2m^2)$ worst-case time for simple polygons.

6.3.7 7. Usages

- **Motion Planning (Configuration Space):** Expanding obstacles by the shape of the robot. If the robot's center is at x , it collides with obstacle O if $x \in O \oplus (-\text{Robot})$ [LP83].
- **Collision Detection:** The GJK (Gilbert-Johnson-Keerthi) algorithm uses Minkowski differences to check for intersections efficiently.
- **CAD Offsetting:** Creating a “buffer” or “padding” around a shape (mitered offsets).
- **Morphological Operations:** Dilation of 2D images or 3D volumes.

6.3.8 8. Testing methods and Edge cases

- **Points and Lines:** Minkowski sum of a polygon and a single point is just a translation. Sum of a polygon and a line segment is a “swept” version of the polygon.
- **Collinear Edges:** Edges from P and Q with the same angle should be merged into a single longer edge in the result.
- **Degenerate Shapes:** Polygons with zero area or zero vertices.
- **Non-Convex Union:** For non-convex sums, verify that internal “voids” are correctly handled or filled if necessary.
- **Origin Symmetry:** Minkowski sum of P and its reflection $-P$ is always centrally symmetric.

6.3.9 9. References

- Guibas, L. J., & Seidel, R. (1987). “Computing convolutions by reciprocal search”. Discrete & Computational Geometry.
- Lozano-Pérez, T. (1983). “Spatial Planning: A Configuration Space Approach”. IEEE Transactions on Computers.
- Fogel, E., & Halperin, D. (2006). “Exact Minkowski Sums of Convex Polygons”. CGAL Documentation.
- [Wikipedia: Minkowski addition](#)

6.4 Industrial Applications of Minkowski Sums

The Minkowski sum $A \oplus B = \{a + b \mid a \in A, b \in B\}$ (Section 6.3) is the algebraic engine behind an unusually broad set of industrial problems, because any operation that “grows” one shape by another, or that checks whether two shapes can touch, is a Minkowski computation. The following are the principal industrial use cases.

- **Robot Motion Planning (Configuration Space):** Growing every obstacle by the reflected robot shape $O \oplus (-\text{Robot})$ reduces the moving-robot collision problem to a point-robot problem: the robot at translation x collides iff $x \in O \oplus (-\text{Robot})$. This is the classical **C-space** construction of Lozano-Pérez, and it underlies the collision checking of commercial offline-programming and simulation suites for industrial manipulators (for example, ABB RobotStudio and FANUC Roboguide) (Section 26.2) [LP83].
- **Collision Detection and GJK:** The Gilbert–Johnson–Keerthi distance algorithm computes the distance between convex shapes from a simplex inside the Minkowski difference $P \ominus Q$; the origin lies in $P \ominus Q$ exactly when the shapes intersect. GJK powers the collision layer of commercial physics middleware (PhysX, Havok, Bullet) used in games, film, and robotics simulation (Section 18.6).
- **Assembly and Nesting (No-Fit Polygon):** The no-fit polygon (NFP) used in 2D irregular cutting and packing is the Minkowski sum of one piece with the reflection of the other (Section 33.3 and the definition 33.15); the NFP encodes all placements where the pieces touch, driving the nesting engines of commercial sheet-metal, textile, and packaging software (SigmaNEST, Lantek, Optimation).
- **Machine Toolpath Generation:** Offsetting a polygonal pocket by its tool radius via the Minkowski sum of the pocket with a disk yields the set of admissible tool-center positions, from which parallel or contour-parallel toolpaths are derived in commercial CAM systems (Mastercam, Siemens NX CAM, Autodesk Fusion 360) for CNC machining and laser/waterjet cutting.
- **Vehicle Swept-Path Analysis:** Computing the swept region of a vehicle turning along a path is a Minkowski sum of the chassis footprint with the path curve. It is the core of AutoTURN (Transoft Solutions), the industry-standard vehicle swept-path package used by over 90% of U.S. state departments of transportation for truck-turning clearance, bus terminals, and loading-dock design, and of the turning templates in the AASHTO Green Book.
- **Autonomous Driving (Occupancy Grids):** Inflating detected obstacles by the vehicle’s safety margin (a Minkowski sum with a disk or box) builds the expanded cost map over which planners find collision-free paths; the same growth bounds the car’s shape against lane geometry in commercial automated-driving stacks.
- **Image Morphology (Dilation) in Industrial Vision:** Grayscale and binary dilation is the Minkowski sum of the image set with a structuring element, and erosion is the Minkowski difference. These are the workhorses of defect detection, OCR, and inspection in commercial machine-vision systems (Cognex, Keyence).
- **Geometric Tolerancing (GD&T):** The maximum material condition and envelope tolerances in mechanical tolerancing are computed with Minkowski sums of the nominal profile with tolerance zones, bounding the worst-case assembled geometry in commercial tolerance-analysis packages.

6.4.1 Industrial-Scale Software

Minkowski-sum-based operations are implemented in CGAL (Minkowski_sum_2, NFP and offsetting), clipper and Clipper2 (polygon offsetting; Clipper is the slicing kernel of Ultimaker Cura), Boost.Polygon (polygon Boolean operations and offsetting), OMPL (C-space construction, the planning library of the robotics middleware MoveIt), and the bullet, PhysX, and Havok physics engines (GJK for collision). See also the motion-planning treatment in Section 26.2 [LP83, GS87].

6.5 Polygon Symmetry

6.5.1 1. Overview

Polygon symmetry detection is the problem of identifying whether a given polygon possesses symmetries such as rotational symmetry or reflectional (axial) symmetry. A polygon is symmetric if it looks identical after a specific geometric transformation. Detecting these symmetries is a classic problem in computational geometry with applications in computer vision, robotics, and geometric design [Ata85].

6.5.2 2. Definitions

Definition 6.29 (Reflectional Symmetry). A polygon has *reflectional (axial) symmetry* if there exists a line ℓ (the axis of symmetry) such that reflecting the polygon across ℓ maps it onto itself.

Definition 6.30 (Rotational Symmetry). A polygon has *rotational symmetry* of order $k > 1$ if rotating it by $360^\circ/k$ around its centroid leaves it unchanged.

Definition 6.31 (Symmetry Centroid). The *centroid* of a polygon is the average position of all the points (or vertices, for a regular polygon). For a polygon with rotational symmetry, the centroid must be the fixed point of all rotations.

6.5.3 3. Theory

The most efficient algorithm for detecting polygon symmetry is based on **string matching** [Ata85], similar to the polygon congruence algorithm.

1. Represent the polygon as a sequence of (angle, edge-length) pairs: $S = (a_1, l_1, a_2, l_2, \dots, a_n, l_n)$.
2. **Rotational Symmetry**: Find all cyclic shifts of S that are identical to S itself. This can be done using the **Knuth-Morris-Pratt (KMP)** algorithm by searching for S in $S + S$ (excluding the trivial match at position 0).
3. **Reflectional Symmetry**: Check if the sequence S is a cyclic shift of its own reverse sequence S^R . If it is, the axes of symmetry must pass through either a vertex or the midpoint of an edge.

Theorem 6.32 (Polygon Symmetry Detection Complexity). *All rotational and reflectional symmetries of a polygon with n vertices can be detected in $O(n)$ time using the string-matching approach [Hig86].*

Proof. The signature generation takes $O(n)$ time. KMP preprocessing of the pattern S takes $O(n)$, and searching S in $S + S$ takes $O(n)$, giving $O(n)$ for rotational detection. Similarly, KMP on the reversed signature takes $O(n)$ for reflectional detection. The total is $O(n)$. $\square$

Example 6.33 (Symmetry of a Regular Hexagon). A regular hexagon has:

- Rotational symmetry of order 6 ($k = 6$): rotations by $0^\circ, 60^\circ, 120^\circ, 180^\circ, 240^\circ, 300^\circ$.
- 6 axes of reflectional symmetry: 3 passing through opposite vertices, 3 passing through midpoints of opposite edges.

The signature S of a regular hexagon is $(120^\circ, s, 120^\circ, s, \dots)$ for edge length s . All 6 cyclic shifts of S match S itself, confirming order 6. The reversed signature S^R matches S (cyclically), confirming 6 reflection axes.

```

1: function DETECTSYMMETRIES(polygon)
2:   sig ← GenerateSignature(polygon)
3:   n ← len(polygon)
4:    $\triangleright$  1. Detect Rotational Symmetry  $\triangleright$  Find all occurrences of sig in sig + sig
5:   rotations ← KMP_AllMatches(sig + sig[:-1], sig)
6:   order_k ← len(rotations)
7:    $\triangleright$  2. Detect Reflectional Symmetry
8:   sig_rev ← ReverseSignature(sig)
9:   reflection_matches ← KMP_AllMatches(sig + sig, sig_rev)
10:  axes ← []
11:  for match_pos in reflection_matches do  $\triangleright$  Calculate the axis line based on match position
12:    axis ← CalculateAxis(match_pos, polygon)
13:    axes.append(axis)
14:  end for
15:  return order_k, axes
16: end function
17: function GENERATESIGNATURE(polygon)  $\triangleright$  Returns (alpha_1, d_1, alpha_2, l_2, ...)  $\triangleright$ 
    where alpha is interior angle and d is edge length
18:   pass
19: end function

```

6.5.4 4. Pseudo code

6.5.5 5. Parameters Selections

Definition 6.34 (Symmetry Precision). Symmetry detection is highly sensitive to noise. An *epsilon tolerance* should be used when comparing angles and lengths to account for floating-point imprecision.

Definition 6.35 (Signature Choice). Using interior angles and edge lengths makes the detection invariant to translation, rotation, and scaling. The signature uniquely determines a polygon's shape up to congruence (or similarity if lengths are normalized).

6.5.6 6. Complexity

Theorem 6.36 (Polygon Symmetry Complexity Summary). *For a polygon with n vertices:*

1. *Signature Generation: $O(n)$.*
2. *Rotational Check: $O(n)$ using KMP.*
3. *Reflectional Check: $O(n)$ using KMP on the reversed signature.*
4. *Total Time: $O(n)$ [Ata85].*
5. *Space: $O(n)$ for the signature.*

6.5.7 7. Usages

- **Computer Vision:** Recognizing symmetric objects (like tools, symbols, or parts) in images regardless of their orientation.
- **Robotics:** Planning grasps for symmetric objects (where multiple orientations are equivalent).

- **Geometric Design (CAD):** Enforcing symmetry constraints during modeling.
- **Chemistry:** Identifying the point group symmetry of molecular structures represented as polygons or polyhedra.
- **Art and Architecture:** Analyzing and generating symmetric patterns and tilings.

6.5.8 8. Testing methods and Edge cases

- **Regular Polygons:** A regular n -gon should have rotational symmetry of order n and n axes of reflectional symmetry.
- **Isosceles Triangle:** Should have one axis of symmetry and no rotational symmetry ($k = 1$).
- **Rectangle:** Should have rotational symmetry of order 2 and two axes of symmetry.
- **Parallelogram:** Should have rotational symmetry of order 2 but no axes of symmetry (in general).
- **No Symmetry:** Test on random, scalene polygons to ensure they return order 1 and zero axes.
- **Precision Issues:** Test with nearly symmetric polygons to verify the epsilon handling.

6.5.9 9. References

- Atallah, M. J. (1985). “On symmetry detection”. IEEE Transactions on Computers.
- Highnam, P. T. (1986). “Optimal algorithms for finding the symmetries of a planar point set”. Information Processing Letters.
- Wolter, J. D., Woo, T. C., & Volz, R. A. (1985). “Optimal algorithms for detecting relevant symmetries in polygons”. IEEE Transactions on Robotics and Automation.
- Wikipedia: [Symmetry in geometry](#)

6.6 Polygon Matching

6.6.1 1. Overview

Polygon matching is the problem of determining whether two given polygons P and Q are identical under a set of transformations (congruence) or whether one is a scaled version of the other (similarity) [ACH⁺91]. This is a specialized form of shape matching that focuses on the exact geometric properties of the boundary. It is essential for pattern recognition, computer vision, and verifying geometric designs.

6.6.2 2. Definitions

Definition 6.37 (Polygon Congruence). Two polygons are *congruent* if they have the same shape and size. They can be mapped to each other via translation, rotation, and (optionally) reflection.

Definition 6.38 (Polygon Similarity). Two polygons are *similar* if they have the same shape but possibly different sizes. They can be mapped via congruence plus uniform scaling.

Definition 6.39 (Geometric Signature). A *geometric signature* is a sequence of values (like edge lengths and angles) that uniquely describes a polygon’s shape and is invariant under certain transformations.

6.6.3 3. Theory

The most robust way to check for polygon matching is to construct a **string representation** of the polygon and use string-matching algorithms [ACH⁺91].

1. For each vertex, calculate its **internal angle** and the **length of the edge** following it.
2. Form a sequence: $S = (a_1, l_1, a_2, l_2, \dots, a_n, l_n)$.
3. To handle rotation, treat the sequence as a circular string.
4. Two polygons are congruent if their sequences are cyclic shifts of each other (or one is a cyclic shift of the reversed other, for reflection).
5. Two polygons are similar if their angle sequences are identical and their edge length sequences are proportional.

The **Knuth-Morris-Pratt (KMP)** algorithm can be used to find cyclic shifts in linear time by searching for sequence S in $S + S$.

Theorem 6.40 (Polygon Congruence via String Matching). *Two simple polygons P and Q with n and m vertices respectively can be tested for congruence in $O(n + m)$ time, or $O(n)$ if $n = m$ [ACH⁺91].*

Proof. Generate the geometric signature S_P for P in $O(n)$ and S_Q for Q in $O(m)$. If $n \neq m$, reject. Otherwise, use KMP to search for S_Q in $S_P + S_P$ (for rotation) and in $S_P + S_P$ for S_Q^R (for reflection). Each KMP search takes $O(n)$ time. $\square$

Example 6.41 (Congruence Check). Consider two triangles:

- $P = \{(0, 0), (3, 0), (0, 4)\}$: Edges $(3, 0), (-3, 4), (0, -4)$. Angles: 90° at $(0, 0)$, $\arctan(4/3) \approx 53.1^\circ$ at $(3, 0)$, $\arctan(3/4) \approx 36.9^\circ$ at $(0, 4)$.
- $Q = \{(1, 1), (4, 1), (1, 5)\}$: Same edge lengths $(3, 5, 4)$ and same angles. Signature of Q is a cyclic shift of signature of P .

KMP finds the match, confirming congruence.

6.6.4 4. Pseudo code

6.6.5 5. Parameters Selections

Definition 6.42 (Matching Precision). Matching should use a small epsilon for floating-point comparisons of angles and lengths. The choice of epsilon affects both false positives and false negatives.

Definition 6.43 (Allowed Transformations). Decide if reflection (mirror symmetry) is allowed. If not, only check for cyclic shifts (rotation and translation). For similarity matching, divide all edge lengths by the total perimeter or the longest edge.

6.6.6 6. Complexity

Theorem 6.44 (Polygon Matching Complexity Summary). *For two polygons with n vertices each:*

1. **Signature Generation:** $O(n)$ each.
2. **Matching:** $O(n)$ using KMP for cyclic shifts [ACH⁺91].
3. **Total Time:** $O(n)$ for congruence; $O(n)$ for similarity (after normalization).
4. **Space:** $O(n)$ to store the signatures.

```

1: function ISCONGRUENT(P, Q)
2:   if len(P) != len(Q) then
3:     return False
4:   end if

5:   sigP ← GenerateSignature(P)
6:   sigQ ← GenerateSignature(Q)

7:   if IsCyclicShift(sigP, sigQ) then
8:     return True
9:   end if
10:  if IsCyclicShift(sigP, Reverse(sigQ)) then
11:    return True
12:  end if
13:  return False
14: end function

15: function GENERATESIGNATURE(polygon)
16:  sig ← []
17:  n ← len(polygon)
18:  for i in range(n) do
19:    p1 ← polygon[i-1]
20:    p2 ← polygon[i]
21:    p3 ← polygon[(i+1)%n]
22:    angle ← CalculateAngle(p1, p2, p3)
23:    length ← Distance(p2, p3)
24:    sig.append((angle, length))
25:  end for
26:  return sig
27: end function

28: function ISCYCLICSHIFT(S1, S2)
29:   return KMP_Search(S1 + S1, S2)
30: end function

```

▷ 1. Generate signatures

▷ 2. Handle reflection (optional)

▷ KMP search for S2 in S1 + S1

6.6.7 7. Usages

- **Computer Vision:** Identifying an object in an image by matching its contour to a template.
- **CAD/CAM:** Finding duplicate parts in a complex mechanical assembly.
- **Architecture:** Identifying repeating patterns or tiles in a floor plan.
- **Geographic Information Systems (GIS):** Matching building footprints across different map revisions.
- **Character Recognition:** Matching the strokes of a drawn letter to a font database.

6.6.8 8. Testing methods and Edge cases

- **Rotated Polygons:** Verify that a polygon matches its rotated version.
- **Mirrored Polygons:** Test if the algorithm correctly identifies reflections.
- **Regular Polygons:** A regular n -gon should have n different valid cyclic shifts.
- **Degenerate Polygons:** Test on triangles or polygons with collinear vertices.
- **Self-Intersecting Polygons:** Ensure the signature is robust for non-simple polygons.
- **Numerical Sensitivity:** Test with polygons having very small edges or angles.

6.6.9 9. References

- Arkin, E. M., Chew, L. P., Huttenlocher, D. P., Kedem, K., & Mitchell, J. S. (1991). “An efficiently computable metric for comparing polygonal shapes”. IEEE Transactions on Pattern Analysis and Machine Intelligence.
- Manber, U. (1989). “Introduction to Algorithms: A Creative Approach”. Addison-Wesley.
- Wikipedia: [Congruence \(geometry\)](#)

6.7 Line and Polygon Clipping

6.7.1 1. Overview

Clipping removes the portions of geometric objects lying outside a given window. Line clipping restricts a line segment to a convex window, and polygon clipping restricts a polygon to a convex window, yielding a new polygon. These operations are central to graphics viewport clipping and to Boolean operations on polygons [SH74, CB78].

6.7.2 2. Definitions

Definition 6.45 (Clip Window). A *clip window* is a (usually convex) region against which an object is clipped; the result is the intersection of the object with the window.

Definition 6.46 (Cyrus–Beck Clipping). The *Cyrus–Beck* algorithm clips a line segment to a convex polygon by computing, for each boundary edge, the interval of parameters $t \in [0, 1]$ for which the segment lies on the inner side, and intersecting these intervals [CB78].

Definition 6.47 (Sutherland–Hodgman Clipping). The *Sutherland–Hodgman* algorithm clips a polygon to a convex window by successively clipping the polygon against each boundary edge of the window, producing an output polygon at every stage [SH74].

6.7.3 3. Theory

Cyrus–Beck expresses the segment as $p(t) = p_0 + t(p_1 - p_0)$ and, for each window edge with inward normal n_i through point q_i , keeps parameters satisfying $n_i \cdot (p(t) - q_i) \geq 0$. Intersecting all such t -intervals yields the clipped segment. Sutherland–Hodgman clips the vertex list once per window edge, retaining vertices on the inside and inserting intersection points where an edge crosses the clipping line.

6.7.4 4. Pseudo code

```

function CLIPPOLYGON( $P$ , window  $W$ )
   $output \leftarrow P$ 
  for each boundary edge  $e$  of  $W$  do
     $input \leftarrow output$ ,  $output \leftarrow \emptyset$ 
    for each edge  $(s, p)$  of  $input$  do
      if  $p$  inside  $e$  then
        if  $s$  not inside  $e$  then
           $output \leftarrow$  intersection of  $(s, p)$  with  $e$ 
        end if
         $output \leftarrow p$ 
      else if  $s$  inside  $e$  then
         $output \leftarrow$  intersection of  $(s, p)$  with  $e$ 
      end if
    end for
  end for
  return  $output$ 
end function

```

6.7.5 5. Parameters Selections

- **Window convexity:** Sutherland–Hodgman assumes a convex window; for non-convex windows the polygon is first decomposed or the operation handled by general Boolean intersection.
- **Tolerance:** use a small epsilon when classifying vertices as inside or on the boundary.

6.7.6 6. Complexity

Cyrus–Beck clips a segment against an m -edge convex window in $O(m)$ time. Sutherland–Hodgman clips an n -vertex polygon against an m -edge window in $O(nm)$ time.

6.7.7 7. Usages

- **Graphics:** viewport and scissor clipping in the rendering pipeline.
- **CAD and GIS:** windowing queries and map clipping to regions of interest.
- **Boolean operations:** clipping is the primitive behind polygon intersection [Vat92].

6.7.8 8. Testing methods and Edge cases

- **Window fully inside:** the polygon is returned unchanged.
- **Polygon fully outside:** an empty polygon is returned.

- **Vertex on boundary:** verify the on-edge case is not counted twice.
- **Degenerate output:** clipping to a triangle or segment must not produce repeated vertices.

6.7.9 9. References

- Sutherland, I. E., & Hodgman, G. W. (1974). “Reentrant polygon clipping”. Communications of the ACM.
- Cyrus, M., & Beck, J. (1978). “Generalized two- and three-dimensional clipping”. Computers & Graphics.
- Wikipedia: [Sutherland–Hodgman algorithm](#)

6.8 Polygon Partitioning

6.9 Convex Decomposition

Convex decomposition is the process of partitioning a non-convex (concave) polygon into a set of disjoint convex polygons. This is a fundamental operation in computational geometry, as many algorithms work more efficiently on convex shapes.

6.9.1 Definitions

Definition 6.48 (Convex Polygon). A polygon P is *convex* if for every pair of points $a, b \in P$, the line segment $\overline{ab}$ lies entirely within P .

Definition 6.49 (Diagonal). A *diagonal* of a polygon is a line segment connecting two non-adjacent vertices that lies entirely within the interior of the polygon.

6.9.2 Theory

The implementation uses a **triangle merging** approach.

1. Triangulate the polygon (e.g., using ear clipping [Mei75]).
2. For each shared diagonal between two adjacent triangles/polygons, check if removing the diagonal results in a convex polygon.
3. If it does, merge the two parts.

Theorem 6.50 (Hertel–Mehlhorn Bound). *Any convex decomposition produced by the Hertel–Mehlhorn triangle-merging heuristic has at most $4k^*$ parts, where k^* is the optimal (minimum) number of convex parts.*

6.9.3 Pseudocode

6.9.4 Parameters

- **Triangulation Algorithm:** Ear clipping is used for simplicity.

6.9.5 Complexity

- **Time:** $O(n^2)$ for simple merging heuristics.
- **Space:** $O(n)$.

Algorithm 7 Convex Decomposition via Triangle Merging

```

1: function CONVEXDECOMPOSITION( $P$ )
2:    $T \leftarrow \text{Triangulate}(P)$ 
3:    $D \leftarrow$  list of all interior diagonals of  $T$ 
4:   for each  $d$  in  $D$  do
5:     Let  $P_1, P_2$  be the parts sharing diagonal  $d$ 
6:     if Merged( $P_1, P_2$ ) is convex at endpoints of  $d$  then
7:       Merge  $P_1$  and  $P_2$  into a single part
8:     end if
9:   end for
10:  return final parts
11: end function

```

6.9.6 Usages

Implemented in `compgeom.polygon.polygon.decomposer.convex_decompose_polygon`. Used in physics engines, motion planning, and collision detection.

6.9.7 Testing and Edge Cases

- **Testing:** Verify all resulting parts are convex. Check that the union of parts matches the original area.
- **Edge Cases:** Polygons with holes, self-intersecting polygons (unsupported), highly narrow “snaky” polygons.

6.9.8 References

- Hertel, S., & Mehlhorn, K. (1983). “Fast triangulation of simple polygons”. FCT.
- Wikipedia: Polygon decomposition.

6.10 Convex Partitioning

Convex partitioning is the problem of dividing a non-convex (concave) polygon into the **minimum possible number** of convex sub-polygons. While a simple decomposition (like triangulation) results in $n - 2$ convex parts, an optimal partition typically results in significantly fewer. This problem is solvable in polynomial time for simple polygons, making it a powerful tool for geometric simplification.

6.10.1 Definitions

Definition 6.51 (Simple Polygon). A polygon is *simple* if it has no self-intersections and no holes.

Definition 6.52 (Reflex Vertex). A vertex v of a polygon is *reflex* if the interior angle at v exceeds π (i.e., 180°). Reflex vertices are the source of non-convexity.

Definition 6.53 (Optimal Convex Partition). An *optimal convex partition* of a polygon P is a decomposition into the minimum number k^* of convex sub-polygons whose union equals P and whose interiors are pairwise disjoint.

6.10.2 Theory

The minimum number of convex parts is determined by how reflex vertices are “resolved” by diagonals. Each reflex vertex must be incident to at least one diagonal that splits its reflex angle.

The **Keil–Snoeyink algorithm** uses **dynamic programming** to find the optimal partition of a simple polygon in $O(n^3)$ time (or $O(r^2n)$ where r is the number of reflex vertices).

1. Define $DP[i][j]$ as the minimum number of convex parts in the sub-polygon formed by vertices $V_i, \dots, V_j$ and the diagonal V_iV_j .
2. The algorithm systematically builds up solutions for larger sub-polygons by combining solutions for smaller ones and checking if the resulting combination is convex.
3. The optimal solution handles cases where multiple reflex vertices are resolved by a single diagonal.

Theorem 6.54 (Lower Bound on Convex Parts). *Any convex partition of a simple polygon with r reflex vertices requires at least $\lceil r/2 \rceil + 1$ convex parts.*

Proof. Each reflex vertex must be “cut” by at least one diagonal to reduce its interior angle below π . A single diagonal can resolve at most two reflex vertices (one at each endpoint). Thus at least $\lceil r/2 \rceil$ diagonals are needed, producing at least $\lceil r/2 \rceil + 1$ parts. $\square$

6.10.3 Pseudocode

Algorithm 8 Minimum Convex Partition via Dynamic Programming

```

1: function MINIMUMCONVEXPARTITION(polygon)
2:    $n \leftarrow \text{len}(\text{polygon.vertices})$ 
3:    $\text{reflex} \leftarrow [v \mid v \in \text{polygon} \wedge \text{IsReflex}(v)]$ 
4:   if  $\text{reflex}$  is empty then
5:     return  $[\text{polygon}]$ 
6:   end if
7:    $dp \leftarrow \text{array}[n][n]$ 
8:   for  $\text{length} = 2$  to  $n - 1$  do
9:     for  $i = 0$  to  $n - \text{length} - 1$  do
10:       $j \leftarrow i + \text{length}$ 
11:      if  $\neg \text{IsDiagonal}(i, j, \text{polygon})$  then
12:         $dp[i][j] \leftarrow \infty$ 
13:        continue
14:      end if
15:       $dp[i][j] \leftarrow \min_{k \in (i+1, \dots, j)} (dp[i][k] + dp[k][j])$ 
16:      if  $\text{ResolvesReflex}(i, j)$  then
17:         $dp[i][j] \leftarrow dp[i][j] - 1$ 
18:      end if
19:    end for
20:  end for
21:  return  $\text{ExtractPartition}(dp, \text{polygon})$ 
22: end function

```

6.10.4 Parameters

- **Polygon Type:** This optimal algorithm is for **simple polygons**. For polygons with holes, the problem is NP-hard.

- **Goal:** This specifically minimizes the number of **parts**. Other algorithms might minimize the **total length** of diagonals (Minimum Weight Convex Partition).

6.10.5 Complexity

- **Time Complexity:** $O(n^3)$ for the general dynamic programming approach. More optimized versions can reach $O(r^2n)$.
- **Space Complexity:** $O(n^2)$ to store the DP table.

6.10.6 Usages

- **Path Planning:** Breaking a complex floor plan into the fewest convex “rooms” to simplify robot navigation.
- **Computer Vision:** Representing a complex silhouette as a small number of convex components for shape matching.
- **Physics Engines:** Reducing the number of primitives needed for collision detection while maintaining accuracy.
- **Pattern Recognition:** Identifying structural features in complex polygonal datasets.

Example 6.55 (Optimal Partition of an L-Shape). Consider an L-shaped polygon with vertices (in order):

$$(0, 0), (3, 0), (3, 1), (1, 1), (1, 3), (0, 3).$$

This polygon has $r = 1$ reflex vertex at $(1, 1)$. By the lower bound (Theorem 6.54), at least $\lceil 1/2 \rceil + 1 = 2$ convex parts are needed. A single diagonal from $(1, 1)$ to $(0, 0)$ or from $(1, 1)$ to $(3, 3)$ would resolve the reflex vertex, yielding exactly 2 convex parts. This is optimal.

6.10.7 Testing and Edge Cases

- **Convex Polygon:** Should return the original polygon (1 part).
- **Star Polygon:** An optimal partition might be smaller than the number of reflex vertices.
- **“C” Shape:** Should be split into 3 convex parts optimally.
- **“L” Shape:** Should be split into 2 parts.
- **Numerical Precision:** Correctness of `IsDiagonal` and `IsReflex` is critical.
- **Holes:** Verify the algorithm correctly identifies that holes make the optimal solution much harder.

6.10.8 References

- Keil, J. M., & Snoeyink, J. S. (2002). “On the time complexity of optimal convex polygon partitioning”. Computational Geometry.
- Hertel, S., & Mehlhorn, K. (1983). “Fast triangulation of simple polygons”. FCT.
- Chazelle, B., & Dobkin, D. P. (1985). “Optimal convex decompositions”. Computational Geometry.
- Wikipedia: Convex polygon.

Chapter 7

Polygon Triangulation

7.1 Ear Clipping

Ear clipping is a simple and intuitive algorithm for triangulating a simple polygon. It is based on the Two-Ears Theorem, which states that every simple polygon with at least four vertices has at least two “ears” that can be removed to form a smaller simple polygon. By repeatedly clipping these ears, the entire polygon is eventually decomposed into triangles. The result was proved by Meisters [Mei75].

7.1.1 Definitions

Definition 7.1 (Simple Polygon). A polygon is *simple* if it does not self-intersect and has no holes.

Definition 7.2 (Convex Vertex). A vertex V_i of a polygon is *convex* if the interior angle at V_i is less than π (180°).

Definition 7.3 (Ear). A vertex V_i of a simple polygon is an *ear* if the triangle formed by V_{i-1}, V_i, V_{i+1} lies entirely within the polygon and contains no other vertices of the polygon in its interior.

Definition 7.4 (Barycentric Coordinates). *Barycentric coordinates* provide a coordinate system relative to a triangle. For a point P and triangle $\triangle ABC$, the barycentric coordinates $(\lambda_1, \lambda_2, \lambda_3)$ satisfy $P = \lambda_1 A + \lambda_2 B + \lambda_3 C$ with $\lambda_1 + \lambda_2 + \lambda_3 = 1$ and $\lambda_i \geq 0$ if and only if P lies inside $\triangle ABC$.

7.1.2 Theory

The algorithm proceeds by identifying a vertex that satisfies two conditions:

1. The vertex is **convex** (the turn from V_{i-1} to V_i to V_{i+1} is counter-clockwise for CCW polygons).
2. The triangle formed by V_{i-1}, V_i, V_{i+1} contains no other vertices of the polygon in its interior.

Once such an ear is found, the triangle (V_{i-1}, V_i, V_{i+1}) is recorded, and the vertex V_i is removed from the polygon’s vertex list. This process is repeated until only three vertices remain, which form the final triangle.

Theorem 7.5 (Two-Ears Theorem). *Every simple polygon with $n \geq 4$ vertices has at least two non-overlapping ears.*

Theorem 7.6 (Ear Clipping Correctness). *The ear clipping algorithm produces a triangulation of any simple polygon with n vertices, consisting of exactly $n - 2$ triangles.*

Proof. We proceed by induction on n . For $n = 3$, the polygon is already a triangle. For $n \geq 4$, by the Two-Ears Theorem [Mei75], the polygon has at least one ear. Removing this ear produces a simple polygon with $n - 1$ vertices. By the inductive hypothesis, the smaller polygon can be triangulated into $(n - 1) - 2 = n - 3$ triangles. Adding the ear triangle yields $n - 2$ triangles in total. $\square$

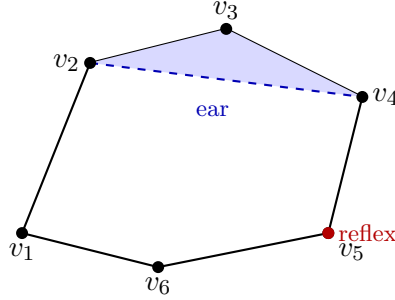


Figure 7.1: Ear clipping: vertex v_3 is an “ear” since triangle (v_2, v_3, v_4) is entirely inside the polygon and contains no other vertices.

7.1.3 Pseudocode

7.1.4 Parameters

- **Input:** A list of `Point2D` representing a simple polygon in counter-clockwise order.
- **Precision:** An epsilon should be used in cross-product and point-in-triangle tests to handle floating-point inaccuracies.

7.1.5 Complexity

- **Time Complexity:** $O(n^2)$ for a polygon with n vertices. In each step, we check $O(n)$ vertices for “ear-ness,” and each check takes $O(n)$ time to verify that no other points are inside.
- **Space Complexity:** $O(n)$ to store the triangles and the remaining vertex indices.

Example 7.7 (Ear Clipping Trace). Consider a quadrilateral with vertices (CCW): $V_0 = (0, 0)$, $V_1 = (2, 0)$, $V_2 = (2, 1)$, $V_3 = (0, 1)$. This is a convex rectangle, so every vertex is an ear.

1. Check V_0 : $\text{cross}(V_3, V_0, V_1) > 0$ and no other vertex is inside $\triangle V_3 V_0 V_1$. It is an ear.
2. Record triangle (V_3, V_0, V_1) and remove V_0 .
3. Remaining vertices: V_1, V_2, V_3 form a triangle.

Result: Two triangles (V_3, V_0, V_1) and (V_1, V_2, V_3) , which is the expected $n - 2 = 2$ triangles for a quadrilateral.

7.1.6 Usages

- Converting arbitrary polygons into triangles for rendering via OpenGL or DirectX.
- Preprocessing geometry for physics simulation and collision detection.
- Calculating the area or centroid of complex shapes.

Algorithm 9 Ear Clipping Triangulation

```
1: function TRIANGULATE(polygon)
2:   indices  $\leftarrow [0, 1, \dots, n - 1]$ 
3:   triangles  $\leftarrow []$ 
4:   while len(indices) > 3 do
5:     for i = 0 to len(indices) - 1 do
6:       u  $\leftarrow$  indices[i - 1]
7:       v  $\leftarrow$  indices[i]
8:       w  $\leftarrow$  indices[(i + 1) mod len(indices)]
9:       if IsEar(u, v, w, indices, polygon) then
10:        triangles.append((u, v, w))
11:        indices.pop(i)
12:        break
13:      end if
14:    end for
15:  end while
16:  triangles.append(tuple(indices))
17:  return triangles
18: end function
19: function ISEAR(u, v, w, current_indices, polygon)
20:  if cross_product(polygon[u], polygon[v], polygon[w])  $\leq 0$  then
21:    return False
22:  end if
23:  for index in current_indices do
24:    if index  $\in \{u, v, w\}$  then
25:      continue
26:    end if
27:    if PointInTriangle(polygon[index], polygon[u], polygon[v], polygon[w]) then
28:      return False
29:    end if
30:  end for
31:  return True
32: end function
```

7.1.7 Testing and Edge Cases

- **Concave Polygons:** Ensure the algorithm correctly skips reflex (non-convex) vertices.
- **Collinear Vertices:** Handled by strict inequality in the convexity check.
- **Self-intersecting Polygons:** Basic ear clipping may fail or produce incorrect results; the input must be simple.
- **Sliver Triangles:** Very thin polygons can lead to numerical stability issues.
- **Degenerate Polygons:** Polygons with 0, 1, or 2 vertices.

7.1.8 References

- Meisters, G. H. (1975). “Polygons have ears”. The American Mathematical Monthly.
- Eberly, D. (2002). “Triangulation by Ear Clipping”. Geometric Tools.
- Wikipedia: Polygon triangulation.

7.2 Monotone Decomposition

Monotone decomposition is the process of partitioning a simple polygon into a set of monotone polygons. A polygon is monotone with respect to a line L if every line orthogonal to L intersects the polygon in at most one connected component (a single line segment). This is typically a preprocessing step for more efficient polygon triangulation, as monotone polygons can be triangulated in linear time [PS85].

7.2.1 Definitions

Definition 7.8 (Monotone Polygon). A polygon is *monotone* with respect to a direction d if every line orthogonal to d intersects the polygon in at most one connected component (at most one line segment).

Definition 7.9 (Y-Monotone Polygon). A polygon is *y-monotone* if every horizontal line intersects the interior in at most one connected component. Equivalently, the polygon has no “cusp” vertices where both neighbors have y -coordinates on the same side.

Definition 7.10 (Cusp Vertex). A *cusp vertex* is a vertex where both adjacent vertices have y -coordinates either all above or all below it. Cusps violate y -monotonicity and are classified as:

- **Start/Split vertex:** interior is below, neighbors have higher y .
- **End/Merge vertex:** interior is above, neighbors have lower y .

7.2.2 Theory

The algorithm uses a **plane sweep** approach, moving a horizontal line from top to bottom. The key to making a polygon y -monotone is to eliminate all split and merge vertices by adding diagonals. A split vertex is connected to a vertex above it (the “helper” of the edge to its left), and a merge vertex is connected to a vertex below it when the sweep line reaches it. By the time the sweep is finished, all cusps that violated monotonicity are resolved.

Theorem 7.11 (Monotone Decomposition Correctness). *The sweep-line algorithm adds at most r diagonals to produce a decomposition of a simple polygon with r reflex vertices into at most $r + 1$ y -monotone sub-polygons.*

Proof. Each split or merge vertex requires exactly one diagonal to resolve the cusp. There are at most r such vertices (each reflex vertex is either a split, merge, or regular vertex; only split and merge vertices require diagonals). Adding one diagonal per such vertex resolves all monotonicity violations, yielding a partition into monotone pieces. $\square$

7.2.3 Pseudocode

Algorithm 10 Monotone Polygon Decomposition

```

1: function MONOTONEDECOMPOSITION(polygon)
2:   events  $\leftarrow$  sorted(polygon.vertices, key =  $\lambda v : (-v.y, v.x)$ )
3:   status  $\leftarrow$  BalancedBST()
4:   diagonals  $\leftarrow []$ 
5:   for v in events do
6:     if IsStartVertex(v) then
7:       HandleStartVertex(v, status)
8:     else if IsEndVertex(v) then
9:       HandleEndVertex(v, status, diagonals)
10:    else if IsSplitVertex(v) then
11:      HandleSplitVertex(v, status, diagonals)
12:    else if IsMergeVertex(v) then
13:      HandleMergeVertex(v, status, diagonals)
14:    else
15:      HandleRegularVertex(v, status, diagonals)
16:    end if
17:  end for
18:  return SplitPolygon(polygon, diagonals)
19: end function

```

7.2.4 Parameters

- **Sweep Direction:** Usually vertical (y -axis), but can be any arbitrary direction if the polygon is rotated.
- **Data Structures:** A balanced binary search tree (BST) for the status and a priority queue for events are standard.

7.2.5 Complexity

- **Time Complexity:** $O(n \log n)$ due to the sorting of vertices and the $O(\log n)$ operations on the balanced BST for each vertex.
- **Space Complexity:** $O(n)$ to store the polygon, the status tree, and the resulting diagonals.

7.2.6 Usages

- **Triangulation:** The first stage of $O(n \log n)$ polygon triangulation algorithms.
- **Trapezoidal Decomposition:** Often used in conjunction with monotone partitioning.
- **Path Planning:** Simplifying complex obstacles into monotone regions for easier navigation.

7.2.7 Testing and Edge Cases

- **Horizontal Edges:** Vertices with identical y -coordinates require consistent tie-breaking in the sort.
- **Complex Cusps:** Multiple split/merge vertices at the same height.
- **Strict Monotonicity:** Ensuring that horizontal lines intersecting a vertex or edge are handled correctly.
- **Non-Simple Polygons:** Monotone decomposition is generally defined for simple polygons; holes require special handling (treating them as merge/split events).

7.2.8 References

- Berg, M. de, Cheong, O., Kreveld, M. van, & Overmars, M. (2008). “Computational Geometry: Algorithms and Applications”. Springer-Verlag.
- Lee, D. T., & Preparata, F. P. (1977). “Location of a point in a planar subdivision and its applications”. SIAM Journal on Computing.
- Wikipedia: Monotone polygon.

7.3 Incremental Delaunay Triangulation

Incremental Delaunay triangulation is a fundamental algorithm for constructing a Delaunay triangulation of a set of points in the plane. The algorithm builds the triangulation by adding points one at a time and locally updating the mesh to maintain the Delaunay property. It was independently proposed by Bowyer [Bow81] and Watson [Wat81]. It is valued for its relative simplicity and efficiency, especially when combined with spatial sorting techniques.

7.3.1 Definitions

Definition 7.12 (Delaunay Property). A triangulation of a point set P is *Delaunay* if the circumcircle of every triangle contains no other point of P in its interior.

Definition 7.13 (In-Circle Test). The *in-circle test* determines whether a point D lies inside, on, or outside the circumcircle of triangle ABC . Given the determinant:

$$\text{InCircle}(A, B, C, D) = \begin{vmatrix} a_x - d_x & a_y - d_y & (a_x - d_x)^2 + (a_y - d_y)^2 \\ b_x - d_x & b_y - d_y & (b_x - d_x)^2 + (b_y - d_y)^2 \\ c_x - d_x & c_y - d_y & (c_x - d_x)^2 + (c_y - d_y)^2 \end{vmatrix}$$

the sign indicates: positive (inside), zero (on), negative (outside), assuming counter-clockwise orientation of ABC .

Definition 7.14 (Edge Flip). An *edge flip* replaces the common edge of two adjacent triangles in a triangulation with the other diagonal of their union quadrilateral.

7.3.2 Theory

The algorithm relies on the fact that any triangulation can be converted into a Delaunay triangulation through a series of edge flips. When a new point P is inserted into a triangle T , it is connected to the three vertices of T , creating three new triangles. The edges of these new triangles (which were the edges of T) may now be “illegal.” The algorithm recursively checks and flips these edges until the Delaunay property is restored globally.

Theorem 7.15 (Delaunay Triangulation Existence and Uniqueness). *For a set of n points in $\mathbb{R}^2$ in general position (no four points co-circular), the Delaunay triangulation exists and is unique.*

Theorem 7.16 (Incremental Delaunay Correctness). *The incremental insertion algorithm with edge legalization produces a valid Delaunay triangulation.*

Proof. The proof proceeds by induction on the number of inserted points. After inserting the first three points, the single triangle trivially satisfies the Delaunay property. When point p is inserted into triangle T , the three new triangles may violate the Delaunay property only along the edges shared with neighboring triangles. The LEGALIZEEDGE procedure checks the in-circle condition for each such edge. If violated, flipping the edge locally restores the Delaunay property for that edge. Since each flip can only create at most two new potentially illegal edges (among the new neighbors), the process terminates. The total number of flips is $O(n)$ in the expected case. $\square$

7.3.3 Pseudocode

Algorithm 11 Incremental Delaunay Triangulation

```

1: function INCREMENTALDELAUNAY(points)
2:    $ST \leftarrow \text{CreateSuperTriangle}(\text{points})$ 
3:    $\text{Triangulation} \leftarrow \{ST\}$ 
4:   for  $p$  in  $\text{points}$  do
5:      $T \leftarrow \text{FindTriangleContaining}(p, \text{Triangulation})$ 
6:      $\text{SplitTriangle}(T, p, \text{Triangulation})$ 
7:   end for
8:    $\text{RemoveSuperTriangleVertices}(\text{Triangulation})$ 
9:   return  $\text{Triangulation}$ 
10: end function
11: function SPLITTRIANGLE( $T, p, \text{Triangulation}$ )
12:    $(A, B, C) \leftarrow T.\text{vertices}$ 
13:    $\text{NewTris} \leftarrow [(p, A, B), (p, B, C), (p, C, A)]$ 
14:   Replace  $T$  with  $\text{NewTris}$  in  $\text{Triangulation}$ 
15:   for each  $\text{edge}$  in  $\{AB, BC, CA\}$  do
16:      $\text{LegalizeEdge}(p, \text{edge}, \text{Triangulation})$ 
17:   end for
18: end function
19: function LEGALIZEEDGE( $p, \text{edge}(U, V), \text{Triangulation}$ )
20:    $\text{Neighbor} \leftarrow \text{FindNeighbor}(\text{edge}, \text{Triangulation})$ 
21:   if  $\text{Neighbor}$  is None then
22:     return
23:   end if
24:    $W \leftarrow \text{Neighbor.vertex\_opposite\_to}(\text{edge})$ 
25:   if  $\text{InCircle}(p, U, V, W)$  then
26:      $\text{FlipEdge}(\text{edge}(U, V))$  to  $\text{edge}(p, W)$ 
27:      $\text{LegalizeEdge}(p, \text{edge}(U, W), \text{Triangulation})$ 
28:      $\text{LegalizeEdge}(p, \text{edge}(W, V), \text{Triangulation})$ 
29:   end if
30: end function

```

7.3.4 Parameters

- **Point Location:** Using a **Visibility Walk** or a **Point Grid** (spatial index) drastically speeds up finding the triangle containing the new point.
- **Insertion Order:** Sorting points along a **Hilbert Curve** or **Morton Code** ensures that subsequent points are geographically close, keeping the walk short.

7.3.5 Complexity

- **Time Complexity:** Average case $O(n \log n)$ with spatial sorting and efficient point location. Worst case $O(n^2)$ for highly structured point sets without randomization or sorting.
- **Space Complexity:** $O(n)$ to store the triangles and point coordinates.

7.3.6 Usages

- Generating high-quality meshes for Finite Element Method (FEM) analysis.
- Interpolation of scattered data (e.g., creating contour maps from elevation points).
- Pathfinding in robotics and video games.
- Dual representation of Voronoi diagrams.

7.3.7 Testing and Edge Cases

- **Co-circular Points:** Four or more points on the same circle. The algorithm will choose one valid triangulation, but the result may not be unique.
- **Collinear Points:** Points lying on a single line. Requires robust predicates or a small perturbation (epsilon).
- **Points on Edges:** If a point falls exactly on an existing edge, the edge should be split into four triangles instead of three.
- **Super-Triangle Selection:** Must be large enough to contain all points, but not so large that it causes numerical overflow.

7.3.8 References

- Bowyer, A. (1981). “Computing Dirichlet tessellations”. The Computer Journal.
- Watson, D. F. (1981). “Computing the n-dimensional Delaunay tessellation with application to Voronoi polytopes”. The Computer Journal.
- Guibas, L., & Stolfi, J. (1985). “Primitives for the manipulation of general subdivisions and the computation of Voronoi diagrams”. ACM Transactions on Graphics.
- Wikipedia: Delaunay triangulation.

7.4 Dynamic Delaunay Triangulation

Dynamic Delaunay triangulation is the process of maintaining a Delaunay triangulation as points are inserted into or deleted from the set. Unlike static construction, where the entire point set is known beforehand, dynamic algorithms must update the mesh locally to restore the Delaunay property while maintaining efficiency. This is critical for applications involving moving objects, real-time data streams, and interactive geometry editing.

7.4.1 Definitions

Definition 7.17 (Dynamic Triangulation). A *dynamic triangulation* maintains a valid triangulation under a sequence of point insertions and deletions, guaranteeing the triangulation invariant (here, the Delaunay property) after each update.

7.4.2 Theory

Insertion

Insertion typically uses the **Incremental Algorithm** (Algorithm 11):

1. **Locate**: Find triangle T containing new point P .
2. **Split**: Split T into three new triangles by connecting P to its vertices.
3. **Legalize**: Recursively check the edges of the new triangles and perform **Delaunay Flips** until all edges are legal (satisfy the empty circumcircle property).

Deletion

Deletion is more complex:

1. **Remove**: Remove the vertex V and all incident edges, leaving a star-shaped polygon (the “hole”).
2. **Re-triangulate**: Triangulate the hole. A simple way is to use a recursive “ear clipping” approach that specifically chooses diagonals satisfying the Delaunay property.
3. **Legalize**: Alternatively, perform any triangulation of the hole and then flip edges until it is Delaunay.

Theorem 7.18 (Expected Flip Complexity). *For n points inserted in random order, the total number of edge flips during incremental Delaunay triangulation is $O(n)$ in expectation.*

Proof. Guibas, Knuth, and Sharir [GS85] showed that the randomized analysis of edge flips leads to an amortized $O(1)$ flips per insertion. The key observation is that each flip can be “charged” to a pair of points whose circumcircle relation changes, and the total number of such chargeable events is bounded by $O(n)$ by a planar graph argument. $\square$

7.4.3 Pseudocode

Dynamic Insertion

Dynamic Deletion

7.4.4 Parameters

- **Point Location Strategy**: For high performance, use a **Stochastic Walk** or a **Directed Acyclic Graph (DAG)** (e.g., the Delaunay Tree) to achieve $O(\log n)$ location time.

Algorithm 12 Dynamic Point Insertion

```
1: function INSERTPOINT(triangulation,  $p$ )
2:    $T \leftarrow \text{FindTriangleContaining}(\text{triangulation}, p)$ 
3:    $\text{new\_triangles} \leftarrow \text{SplitTriangle}(T, p)$ 
4:    $\text{edges\_to\_check} \leftarrow \text{GetOuterEdges}(\text{new\_triangles})$ 
5:   while  $\text{edges\_to\_check}$  is not empty do
6:      $e \leftarrow \text{edges\_to\_check.pop}()$ 
7:     if  $\text{IsIllegal}(e, \text{triangulation})$  then
8:        $\text{new\_e} \leftarrow \text{Flip}(e, \text{triangulation})$ 
9:        $\text{edges\_to\_check.extend}(\text{GetSurroundingEdges}(\text{new\_e}))$ 
10:    end if
11:  end while
12: end function
```

Algorithm 13 Dynamic Point Deletion

```
1: function DELETEPOINT(triangulation,  $v$ )
2:    $\text{neighbors} \leftarrow \text{GetAdjacentVertices}(v, \text{triangulation})$ 
3:    $\text{RemoveVertex}(v, \text{triangulation})$ 
4:    $\text{FillHoleDelaunay}(\text{neighbors}, \text{triangulation})$ 
5: end function
```

- **Robustness:** Use robust predicates to prevent infinite loops during flipping caused by precision errors.

7.4.5 Complexity

- **Insertion:** $O(\log n)$ average for location, $O(1)$ expected for flipping (though $O(n)$ worst-case).
- **Deletion:** $O(\log n)$ average to find the vertex, $O(k)$ to re-triangulate where k is the degree of the vertex.
- **Space:** $O(n)$ to store the triangles and point location structures.

7.4.6 Usages

- **Terrain Modeling:** Adding new survey points to a digital elevation model.
- **Robotics:** Updating a map as a robot moves and discovers new obstacles.
- **Fluid Simulation:** Re-meshing around a moving boundary (e.g., a boat moving through water).
- **Mesh Refinement:** Splitting triangles to improve resolution in high-gradient areas.
- **Data Visualization:** Real-time plotting of scattered data points.

7.4.7 Testing and Edge Cases

- **Co-circular Points:** Ensure both insertion and deletion handle cases where four or more points lie on a circle (Delaunay triangulation is not unique).
- **Points on Boundary:** Handle points added to or removed from the convex hull correctly.

- **Degenerate Deletion:** Deleting a vertex that reduces the convex hull to a line or a single point.
- **Multiple Insertions at same position:** Handle duplicate points by ignoring them or updating metadata.
- **High-Valence Vertices:** Test deletion of vertices shared by many (e.g., 50+) triangles.

7.4.8 References

- Bowyer, A. (1981). “Computing Dirichlet tessellations”. The Computer Journal.
- Watson, D. F. (1981). “Computing the n-dimensional Delaunay tessellation with application to Voronoi polytopes”. The Computer Journal.
- Guibas, L., Knuth, D. E., & Sharir, M. (1992). “Randomized incremental construction of Delaunay and Voronoi diagrams”. Algorithmica.
- Devillers, O. (1999). “On handling transitions between different structures in geometric modeling”. Proceedings of the 11th Canadian Conference on Computational Geometry.
- Wikipedia: Delaunay triangulation.

7.5 Delaunay Flips

The edge flip algorithm is a local transformation technique used to convert any valid triangulation of a set of points into a **Delaunay triangulation**. It is based on the Lawson Flip Theorem [Law77], which states that any two triangulations of the same set of points are connected by a series of edge flips. This process iteratively improves the “quality” of the triangles by ensuring they satisfy the Delaunay (empty circumcircle) property.

7.5.1 Definitions

Definition 7.19 (Illegal Edge). An interior edge e shared by triangles $T_1 = (A, B, C)$ and $T_2 = (B, C, D)$ is *illegal* if D lies inside the circumcircle of $\triangle ABC$ (equivalently, if $\text{InCircle}(A, B, C, D) > 0$).

Definition 7.20 (Angle-Vector). The *angle-vector* of a triangulation is the sorted (non-decreasing) list of all interior angles. Delaunay flips lexicographically maximize this vector, yielding the triangulation that maximizes the minimum angle.

7.5.2 Theory

For any interior edge e shared by triangles $T_1 = (A, B, C)$ and $T_2 = (B, C, D)$, we check the **In-Circle** condition for vertex D relative to triangle ABC . If D is inside the circumcircle of ABC , then the edge BC is illegal.

Flipping an illegal edge BC produces a new edge AD and two new triangles ABD and ACD . It is a proven property that:

1. Flipping an illegal edge increases the minimum angle of the triangulation.
2. An illegal edge can only exist if the four vertices form a convex quadrilateral.
3. By repeatedly flipping illegal edges, the algorithm will eventually converge to the unique Delaunay triangulation (assuming no four points are co-circular).

Theorem 7.21 (Lawson’s Flip Theorem). *Any triangulation of a point set in general position can be transformed into the Delaunay triangulation by a finite sequence of edge flips. Furthermore, the number of flips is at most $O(n^2)$.*

Proof. Each flip strictly increases the angle-vector in lexicographic order. Since there are finitely many triangulations of n points (at most $O(27^n)$ by a result of Sharir and others), the process must terminate. The bound of $O(n^2)$ flips follows from an amortized analysis of the number of illegal edges created per flip. $\square$

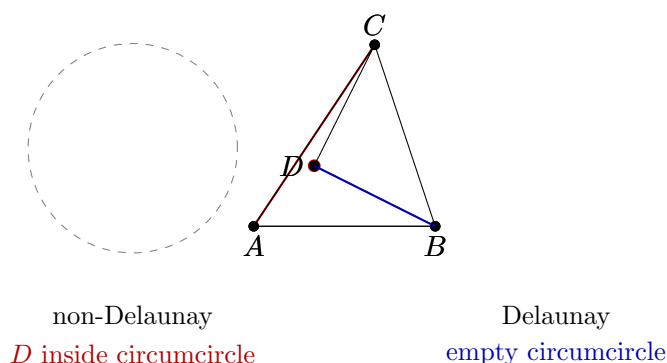


Figure 7.2: Delaunay property: (left) edge AC is illegal because D lies inside the circumcircle of ABC . (right) Flipping to edge BD satisfies the empty circumcircle property.

7.5.3 Pseudocode

7.5.4 Parameters

- **Robust Predicates:** The `InCircle` test must be implemented using robust geometric predicates (e.g., using `orient2d` and `incircle` from Shewchuk [She97]) to handle floating-point precision issues.
- **Data Structure:** A **Half-Edge** or **Quad-Edge** data structure is essential for efficiently finding neighboring faces and edges.

7.5.5 Complexity

- **Time Complexity:** In the worst case, $O(n^2)$ flips may be required for n points. However, starting from a “good” initial triangulation (like one from a sweep-line), it is often much faster.
- **Space Complexity:** $O(n)$ to store the triangulation and the queue of edges.

7.5.6 Usages

- **Mesh Refinement:** Improving the quality of an existing mesh for Finite Element analysis.
- **Incremental Construction:** Maintaining a Delaunay triangulation as points are added or moved.
- **Terrain Modeling:** Converting a set of elevation samples into a high-quality TIN (Triangulated Irregular Network).
- **Fluid Simulation:** Re-meshing to avoid thin, sliver triangles that cause numerical instability.

Algorithm 14 Delaunay Edge Flip Algorithm

```
1: function DELAUNAYFLIP(triangulation)
2:   illegal_edges  $\leftarrow$  Queue()
3:   for edge in triangulation.interior_edges do
4:     if IsIllegal(edge, triangulation) then
5:       illegal_edges.push(edge)
6:     end if
7:   end for
8:   while illegal_edges is not empty do
9:     edge  $\leftarrow$  illegal_edges.pop()
10:    if IsIllegal(edge, triangulation) then
11:      new_edge  $\leftarrow$  Flip(edge, triangulation)
12:      for neighbor in GetNeighboringEdges(new_edge) do
13:        if neighbor.is_interior then
14:          illegal_edges.push(neighbor)
15:        end if
16:      end for
17:    end if
18:  end while
19:  return triangulation
20: end function
21: function ISILLEGAL(edge(B, C), triangulation)
22:   T1  $\leftarrow$  triangulation.GetFace(edge)
23:   T2  $\leftarrow$  triangulation.GetOppositeFace(edge)
24:   A  $\leftarrow$  T1.vertex_opposite(edge)
25:   D  $\leftarrow$  T2.vertex_opposite(edge)
26:   return InCircle(A, B, C, D)
27: end function
```

7.5.7 Testing and Edge Cases

- **Co-circular Points:** Four points on a circle mean both diagonals are “legal.” The algorithm should terminate consistently.
- **Non-Convex Quadrilateral:** Ensure the flip logic only applies to convex quadrilaterals (though an illegal edge is guaranteed to be a diagonal of a convex quad).
- **Boundary Edges:** Edges on the convex hull cannot be flipped.
- **Sliver Triangles:** Verify that flips successfully remove extremely thin triangles.
- **Convergence:** Test on highly structured point sets (like a grid) to ensure the algorithm doesn’t loop infinitely due to precision errors.

7.5.8 References

- Lawson, C. L. (1977). “Software for C^1 surface interpolation”. Mathematical Software.
- Guibas, L., & Stolfi, J. (1985). “Primitives for the manipulation of general subdivisions and the computation of Voronoi diagrams”. ACM Transactions on Graphics.
- Shewchuk, J. R. (1997). “Adaptive Precision Floating-Point Arithmetic and Fast Robust Geometric Predicates”. Discrete & Computational Geometry.
- Wikipedia: Delaunay triangulation.

7.6 Constrained Delaunay Triangulation

Constrained Delaunay Triangulation (CDT) is a version of Delaunay triangulation that forces certain edges (constraints) into the triangulation. It is essential for representing domains with specific boundaries, holes, or internal features that must be preserved.

7.6.1 Overview

While a standard Delaunay triangulation maximizes the minimum angle and satisfies the “empty circumcircle” property for all points, it may not respect the boundaries of a non-convex polygon. CDT modifies the Delaunay criteria to ensure that all constraint edges are present in the final mesh, while maintaining the Delaunay property for all other edges as much as possible.

7.6.2 Implementation Details

The CDT is implemented using an iterative edge-flip approach:

1. **Initial Triangulation:** The domain (including holes) is initially triangulated using a standard polygon triangulation algorithm.
2. **Constraint Enforcement:** Bridge edges are added to connect holes to the outer boundary, creating a single degenerate polygon that is then triangulated.
3. **Edge Flipping:** All internal edges that are not part of the constraints are placed in a queue. Edges are flipped if they violate the Delaunay condition, provided the flip doesn’t cross a constraint and maintains the domain’s validity.

Algorithm 15 Constrained Delaunay Triangulation

```

1: function CONSTRAINEDDELAUNAYTRIANGULATION(outer, holes)
2:   triangulation  $\leftarrow$  TriangulateDomain(outer, holes)
3:   edge_queue  $\leftarrow$  GetInteriorNonConstraintEdges(triangulation)
4:   while edge_queue is not empty do
5:     e  $\leftarrow$  edge_queue.pop()
6:     if IsIllegal(e) and  $\neg$  CrossesConstraint(e) then
7:       Flip(e, triangulation)
8:       edge_queue.extend(GetAffectedEdges(e))
9:     end if
10:  end while
11:  return triangulation
12: end function

```

7.6.3 Pseudocode**7.6.4 Usage Example**

```

from compgeom.mesh import constrained_delaunay_triangulation
from compgeom import Point2D

outer = [Point2D(0,0), Point2D(10,0), Point2D(10,10), Point2D(0,10)]
holes = [[Point2D(4,4), Point2D(6,4), Point2D(6,6), Point2D(4,6)]]

triangles, constrained_edges = constrained_delaunay_triangulation(outer, holes)

```

7.6.5 References

- Chew, L. Paul. “Constrained Delaunay Triangulations.” *Algorithmica*, 1989.
- Sloan, S. W. “A Fast Algorithm for Generating Constrained Delaunay Triangulations.” *Computers & Structures*, 1993.

7.7 Non-Obtuse Triangulation

Non-Obtuse Triangulation is the process of subdividing a 2D domain into triangles such that no internal angle exceeds 90° .

7.7.1 Overview

Most standard triangulation algorithms (including Delaunay) may produce obtuse triangles. For certain numerical simulations (like finite element analysis or fluid dynamics), obtuse triangles can introduce errors or numerical instability. A non-obtuse triangulation ensures that all angles are acute or right angles.

7.7.2 Implementation Details

The `NonObtuseTriangulator` uses an iterative Steiner point insertion strategy:

1. **Initial Mesh:** Start with a Delaunay triangulation of the input point set.
2. **Detection:** Identify triangles with an angle $> 90^\circ$.

3. **Refinement:** For each obtuse triangle, insert a new vertex (Steiner point) at the midpoint of its longest edge.
4. **Re-triangulation:** Update the Delaunay triangulation with the new point.
5. **Iteration:** Repeat until no obtuse triangles remain or a maximum number of Steiner points is reached.

7.7.3 Pseudocode

Algorithm 16 Non-Obtuse Triangulation

```
1: function NONOBTUSETRIANGULATE(points, max_steiner)
2:   mesh  $\leftarrow$  DelaunayTriangulation(points)
3:   steiner_count  $\leftarrow$  0
4:   while hasObtuse(mesh) and steiner_count < max_steiner do
5:     T  $\leftarrow$  findObtuseTriangle(mesh)
6:     edge  $\leftarrow$  longestEdge(T)
7:     midpoint  $\leftarrow$  midpoint(edge)
8:     InsertPoint(mesh, midpoint)
9:     steiner_count  $\leftarrow$  steiner_count + 1
10:  end while
11:  return mesh
12: end function
```

7.7.4 Usage Example

```
from compgeom.mesh.surface.trimesh.non_obtuse_triangulation import NonObtuseTriangulator
from compgeom import Point2D
```

```
pts = [Point2D(0, 0), Point2D(10, 0), Point2D(5, 0.1)]
triangulator = NonObtuseTriangulator(pts)
mesh = triangulator.triangulate(max_steiner=100)
```

7.7.5 References

- Bern, M., et al. “Provably Good Mesh Generation.” Journal of Computer and System Sciences, 1994.
- CG:SHOP 2025 Challenge: “Minimum Non-Obtuse Triangulation.”

Part III

Geometric Searching and Spatial Data Structures

Chapter 8

Point Location

8.1 The Planar Point-Location Problem

8.2 Subdivision Representations

8.3 Slab Decomposition

8.4 Trapezoidal Maps

8.5 Randomized Incremental Construction

8.6 Search DAGs

8.7 Kirkpatrick's Hierarchy

8.8 Point Location in Triangulations

Walking methods locate a query point by crossing mesh edges (2D) or faces (3D) from a starting simplex toward the query point. The visibility walk and jump-and-walk are described in Section 13.11, which covers both triangles and tetrahedra and the expected $O(\log n)$ walk length with a spatial-index start.

8.9 Dynamic Point Location

8.10 Practical Implementations

8.11 Exercises

Chapter 9

Orthogonal Range Searching

9.1 Range Queries

9.2 One-Dimensional Range Searching

9.3 k-d Trees

K-d trees support orthogonal range queries by recursively partitioning the point set with axis-aligned splitting hyperplanes. A brief outline appears here; the full data structure – construction, balancing, and query pruning – is developed in Section 10.1 (Chapter 10).

9.4 Range Trees

9.5 Multilevel Data Structures

9.6 Fractional Cascading

9.7 Priority Search Trees

9.8 Reporting versus Counting

9.9 Higher-Dimensional Range Searching

9.10 Dynamic Range Searching

9.11 Curse of Dimensionality

9.12 Simplex Range Searching

9.12.1 Overview

Range searching asks for the set of points of a fixed point set P that lie inside a query region Q . When Q is a simplex (a half-plane, triangle, or half-space), the problem is called **simplex range searching**. It is one of the most fundamental query problems in computational geometry, with applications in databases, GIS, and computer graphics. The standard solutions use **partition trees**, **cuttings**, and ε -**nets** to answer counting queries in roughly $O(n^{1-1/d})$ time with linear storage, which is optimal in the worst case for near-linear space.

9.12.2 Definitions

Definition 9.1 (Simplex Range Counting). Given a set P of n points in $\mathbb{R}^d$ and a query simplex Δ , the *simplex range counting* problem asks for $|P \cap \Delta|$. The *reporting* variant asks for all points in $P \cap \Delta$.

Definition 9.2 (ε -Net). Let $(X, \mathcal{R})$ be a range space of points and subsets (ranges) with VC dimension d . A subset $N \subseteq X$ is an ε -net if every range that contains at least $\varepsilon|X|$ points also contains a point of N .

9.12.3 Theory

A random sample of size $O(\frac{d}{\varepsilon} \log \frac{1}{\varepsilon})$ is an ε -net with high probability [HW87]. Cuttings generalize this idea: a $(1/r)$ -cutting of an arrangement of n lines subdivides the plane into $O(r^2)$ cells, each crossed by at most n/r lines. Cuttings can be built deterministically in $O(nr)$ time [CF90], and hierarchical cuttings yield partition trees.

Theorem 9.3 (Simplex Range Searching Bounds). *For a set of n points in $\mathbb{R}^d$, a partition tree of linear size can answer a simplex counting query in $O(n^{1-1/d} (\log n)^{O(1)})$ time and a reporting query in $O(n^{1-1/d} (\log n)^{O(1)} + k)$ time, where k is the number of reported points [Mat92, CF90]. This is optimal for near-linear space: any data structure using $O(n \text{ polylog } n)$ space requires $\Omega(n^{1-1/d})$ query time in the worst case.*

9.12.4 Pseudocode

Algorithm 17 Simplex Range Counting with a Partition Tree

```

1: function COUNTSIMPLEX(node,  $\Delta$ )
2:                                      $\triangleright$  node is a cell with an associated canonical subset
3:   if node.cell  $\cap \Delta = \emptyset$  then
4:     return 0
5:   end if
6:   if node.cell  $\subseteq \Delta$  then
7:     return node.count
8:   end if
9:   return  $\sum_{child} \text{COUNTSIMPLEX}(child, \Delta)$ 
10: end function

```

9.12.5 Complexity

- **Storage:** $O(n)$ for a partition tree; cutting-based structures use $O(n \text{ polylog } n)$.
- **Counting Query:** $O(n^{1-1/d} (\log n)^{O(1)})$ time.
- **Reporting Query:** add $O(k)$ output time.
- **Construction:** $O(n \log n)$ with randomized incremental methods, or $O(nr)$ for a single cutting.

9.12.6 Usages

- **Database and GIS** polygon windowing queries.
- **Geometric statistics:** counting points in a query half-space for classification and density estimation.

- **Levels and duality:** by duality, half-plane range counting is equivalent to counting vertices of an arrangement below the query line, connecting to levels (Section 14.3).

9.12.7 Testing Methods and Edge Cases

- **Degenerate queries:** simplex boundaries through points, empty and full queries.
- **Consistency:** compare counting against brute force for small n ; compare reporting to counting by checking the output size.

9.12.8 References

- Haussler, D., & Welzl, E. (1987). “ ϵ -Nets and Simplex Range Queries”. *Discrete & Computational Geometry*.
- Matoušek, J. (1992). “Reporting Points in Halfspaces” *Computational Geometry: Theory and Applications*.
- Chazelle, B., & Friedman, J. (1990). “A Deterministic View of Random Sampling and Its Use in Geometry”. *Combinatorica*.
- Matoušek, J. (1999). “Geometric Discrepancy: An Illustrated Guide”. Springer.

9.13 Exercises

9.14 Project: Spatial Query Engine

Chapter 10

Spatial Partitioning

Geometric algorithms depend on data structures that organize points, lines, regions, and higher-dimensional objects for efficient construction and query. This chapter presents hierarchical and ordered structures for spatial searching: KD-trees, quadtrees, R-trees, interval trees, and segment trees. Together they support nearest-neighbor queries, range searching, collision detection, intersection reporting, and other geometric operations.

10.1 K-Dimensional Trees

10.1.1 1. Overview

A **KD-tree** is a space-partitioning data structure for organizing points in a k -dimensional space [Ben75]. It is a binary tree where every node is a k -dimensional point and every non-leaf node represents a splitting hyperplane. This structure is extremely efficient for range searches and nearest neighbor queries in low-to-medium dimensional spaces. A first treatment of k -d trees in the context of orthogonal range queries appears in Section 9.3; this section gives the full data structure, including construction, balancing, and query pruning.

10.1.2 2. Definitions

Definition 10.1 (Splitting Hyperplane). A *splitting hyperplane* is a $(k - 1)$ -dimensional surface that divides the point set into two subsets. It is always perpendicular to one of the coordinate axes.

Definition 10.2 (Depth). The *depth* d of a node is its level in the tree. The splitting axis is chosen as $\text{axis} = d \bmod k$.

Definition 10.3 (KD-tree Median). The *median* is the point chosen to split the set at each node, ensuring the tree is balanced. Choosing the exact median guarantees that both subtrees contain at most $\lfloor n/2 \rfloor$ points.

Definition 10.4 (Bounding Box). The *bounding box* of a node is the region of space represented by that node and all its descendants. At depth d , the bounding box is an axis-aligned rectangle restricted along axis $d \bmod k$ to at most one side of the splitting hyperplane.

10.1.3 3. Theory

A KD-tree is built by recursively partitioning the set of points. At each level, the algorithm chooses one of the k dimensions and splits the points into two groups: those whose coordinate in that dimension is less than the median and those whose coordinate is greater.

Theorem 10.5 (KD-tree Construction Complexity). *Building a KD-tree on n points in $\mathbb{R}^k$ takes $O(n \log n)$ time when using a linear-time median selection algorithm at each level, and $O(nk \log n)$ time when sorting at each level.*

Proof. At each level of recursion we partition at most n points. If we use an $O(n)$ median-finding algorithm (e.g., quickselect), the total work at level i is $O(n)$, and the tree has $O(\log n)$ levels (assuming balanced splits), giving $O(n \log n)$ total. With sorting, each level costs $O(n \log n)$ for a total of $O(n \log^2 n)$, but a pre-sorted approach or median-of-medians reduces this to $O(n \log n)$. $\square$

For **Nearest Neighbor Search**, the tree is traversed from the root down to the leaf containing the query point. The distance from the query to this leaf is our initial “best” distance. Then, the algorithm backtracks up the tree, checking if any other branches could contain a closer point. This is done by checking if the query point’s distance to a node’s splitting hyperplane is smaller than the current best distance.

Theorem 10.6 (KD-tree Nearest Neighbor Search). *For n points in $\mathbb{R}^k$ with fixed k , the expected nearest neighbor search time in a randomly built KD-tree is $O(\log n)$.*

Proof. The expected number of nodes visited during a nearest neighbor query is $O(2^k \log n)$ for uniformly distributed points [FBF77]. For fixed k , this simplifies to $O(\log n)$. The key insight is that the pruning criterion—skipping a subtree when the splitting hyperplane is farther than the current best—eliminates most branches. In the worst case (e.g., all points equidistant from the query), no pruning occurs and $O(n)$ nodes are visited. $\square$

Example 10.7 (KD-tree Construction in $\mathbb{R}^2$). Consider the point set $S = \{(2, 3), (5, 4), (9, 6), (4, 7), (8, 1), (7, 2)\}$.

1. **Root (depth 0, axis x):** Median x -coordinate is 5 (from sorted x : 2, 4, 5, 7, 8, 9). Node: (5, 4). Left subtree: $\{(2, 3), (4, 7)\}$; right subtree: $\{(9, 6), (8, 1), (7, 2)\}$.
2. **Left child (depth 1, axis y):** Median y -coordinate of $\{(2, 3), (4, 7)\}$ is 3 or 7. Node: (2, 3). Right child: (4, 7).
3. **Right child (depth 1, axis y):** Median y -coordinate of $\{(9, 6), (8, 1), (7, 2)\}$ is 2. Node: (7, 2). Left child: (8, 1); right child: (9, 6).

The resulting tree partitions $\mathbb{R}^2$ into rectangular regions, each containing exactly one point.

10.1.4 4. Pseudo code

10.1.5 5. Parameters Selections

Definition 10.8 (Dimension Cycling). *Dimension cycling* is the strategy for choosing the splitting axis at each level of the KD-tree. The standard choice is $\mathbf{d} \% \mathbf{k}$, but choosing the dimension with the largest variance (spread) can lead to more efficient searches.

Definition 10.9 (Leaf Size). The *leaf size* is the maximum number of points stored in a single leaf node. Storing a few points (e.g., 5–10) in each leaf instead of one reduces recursion depth and can improve cache performance.

10.1.6 6. Complexity

Theorem 10.10 (KD-tree Complexity Summary). *For n points in $\mathbb{R}^k$:*

1. **Construction:** $O(n \log n)$ time and $O(n)$ space.

```

1: function BUILDKDTree(points, depth=0)
2:   if not points then
3:     return None
4:   end if
5:    $k \leftarrow \text{len}(\text{points}[0])$ 
6:    $\text{axis} \leftarrow \text{depth} \bmod k$ 
7:   Sort points by axis
8:    $\text{mid} \leftarrow \text{len}(\text{points})/2$ 
9:    $\text{node} \leftarrow \text{Node}(\text{points}[\text{mid}])$ 
10:   $\text{node.left} \leftarrow \text{BuildKDTree}(\text{points}[:\text{mid}], \text{depth} + 1)$ 
11:   $\text{node.right} \leftarrow \text{BuildKDTree}(\text{points}[\text{mid}+1:], \text{depth} + 1)$ 
12:  return node
13: end function
14: function NEARESTNEIGHBOR(root, query, depth=0, best=None)
15:  if root is None then
16:    return best
17:  end if
18:   $k \leftarrow \text{len}(\text{query})$ 
19:   $\text{axis} \leftarrow \text{depth} \bmod k$ 
20:   $\text{dist} \leftarrow \text{EuclideanDistance}(\text{root.point}, \text{query})$ 
21:  if best is None or  $\text{dist} < \text{EuclideanDistance}(\text{best.point}, \text{query})$  then
22:     $\text{best} \leftarrow \text{root}$ 
23:  end if
24:   $\text{next\_branch} \leftarrow \begin{cases} \text{root.left} & \text{if } \text{query}[\text{axis}] < \text{root.point}[\text{axis}] \\ \text{root.right} & \text{otherwise} \end{cases}$ 
25:   $\text{other\_branch} \leftarrow \begin{cases} \text{root.right} & \text{if } \text{query}[\text{axis}] < \text{root.point}[\text{axis}] \\ \text{root.left} & \text{otherwise} \end{cases}$ 
26:   $\text{best} \leftarrow \text{NearestNeighbor}(\text{next\_branch}, \text{query}, \text{depth} + 1, \text{best})$ 
27:  if  $|\text{query}[\text{axis}] - \text{root.point}[\text{axis}]| < \text{EuclideanDistance}(\text{best.point}, \text{query})$  then
28:     $\text{best} \leftarrow \text{NearestNeighbor}(\text{other\_branch}, \text{query}, \text{depth} + 1, \text{best})$ 
29:  end if
30:  return best
31: end function

```

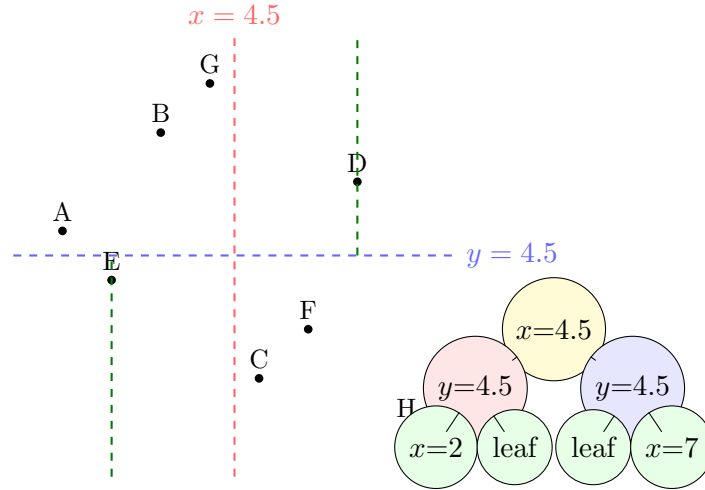


Figure 10.1: KD-tree: (left) recursive axis-aligned partitioning of 2D space; (right) the corresponding binary tree structure.

2. **Nearest Neighbor Search:** $O(\log n)$ expected time for fixed k ; $O(n)$ worst case.

3. **Range Search:** $O(n^{1-1/k} + t)$ time for reporting t points in a rectangular range, for $k \geq 2$ [FBF77].

Proof. Construction follows from the median-based partitioning argument (Theorem 10.5). The range search bound $O(n^{1-1/k} + t)$ is obtained by observing that at each level i , at most $O(2^{ki/k})$ nodes can straddle the query boundary, and summing over $O(\log n)$ levels yields the stated bound. The space is $O(n)$ since each point is stored in exactly one leaf. $\square$

Remark 10.11. The “curse of dimensionality” causes the range search bound to degrade to $O(n)$ when k is large (e.g., $k > 20$). In such regimes, alternative structures like locality-sensitive hashing or random-projection trees are preferred.

10.1.7 7. Usages

- **Computational Geometry:** Fast point location and range searching.
- **Machine Learning:** K-Nearest Neighbors (KNN) classification and clustering.
- **Computer Graphics:** Ray tracing (finding the first object a ray intersects) and photon mapping.
- **Data Compression:** Efficient representation of multidimensional datasets.
- **GIS:** Finding the closest point of interest (e.g., “find the nearest gas station”).

10.1.8 8. Testing methods and Edge cases

- **Duplicate Points:** Ensure the tree correctly handles multiple points at the same coordinates.
- **Points on Boundaries:** Correctly assign points that lie exactly on a splitting hyperplane.
- **Empty Tree:** Handle $n = 0$ cases gracefully.
- **Very High Dimensions:** The “Curse of Dimensionality” affects KD-trees ($k > 20$); performance should be monitored.

- **Skewed Data:** Test on points that are nearly collinear or form very narrow clusters.

10.1.9 9. References

- Bentley, J. L. (1975). “Multidimensional binary search trees used for associative searching”. Communications of the ACM.
- Friedman, J. H., Bentley, J. L., & Finkel, R. A. (1977). “An Algorithm for Finding Best Matches in Logarithmic Expected Time”. ACM Transactions on Mathematical Software.
- Samet, H. (2006). “Foundations of Multidimensional and Metric Data Structures”. Morgan Kaufmann.
- Wikipedia: [k-d tree](#)

10.2 Quadtrees

10.2.1 1. Overview

A **quadtree** is a tree data structure in which each internal node has exactly four children. It is most commonly used to partition a two-dimensional space by recursively subdividing it into four quadrants (North-West, North-East, South-West, South-East) [FB74]. Quadtrees are the 2D equivalent of Octrees (used in 3D) and are essential for efficient spatial querying, image compression, and collision detection.

10.2.2 2. Definitions

Definition 10.12 (Quadtree Root). The *root* of a quadtree is the top-level node representing the entire bounding area.

Definition 10.13 (Quadtree Leaf). A *leaf* is a node that contains actual data points or a uniform value and has no children.

Definition 10.14 (Quadrant). A *quadrant* is one of the four rectangular regions created by splitting a parent node’s region in half along both the x and y axes. The four quadrants are denoted NW, NE, SW, and SE.

Definition 10.15 (Quadtree Capacity). The *capacity* of a quadtree node is the maximum number of points a single node can hold before it must be split into four children.

10.2.3 3. Theory

A quadtree is built by starting with a bounding box and inserting points one by one. If a point is inserted into a leaf node that is already at its capacity, the node “subdivides” into four children, and all points in the node are redistributed to the correct child.

Theorem 10.16 (Quadtree Subdivision Property). *When a quadtree node with capacity c is subdivided, at most 4 nodes of depth $d + 1$ are created, each covering exactly one quarter of the parent’s area. After subdivision, each child contains at most c of the original points.*

Proof. By construction, the four quadrants partition the parent’s bounding box into four disjoint rectangular regions whose union equals the parent region. Each of the original c (or fewer) points in the parent falls into exactly one quadrant based on its coordinates, so no point is lost or duplicated. $\square$

For **Range Querying** (e.g., finding all points within a circle or rectangle), the quadtree allows us to quickly discard entire branches of the tree if their bounding box does not intersect the query area. This pruning results in $O(\log n)$ performance for localized searches.

Example 10.17 (Quadtree Insertion Trace). Consider a quadtree with capacity $c = 1$ and bounding box $[0, 8] \times [0, 8]$.

1. Insert $(2, 3)$: Root is empty, store directly.
2. Insert $(6, 7)$: Root is at capacity, subdivide into four quadrants centered at $(4, 4)$. $(2, 3)$ falls in SW, $(6, 7)$ falls in NE.
3. Insert $(5, 1)$: Root subdivided, $(5, 1)$ falls in SE quadrant (empty), stored there.
4. Insert $(1, 6)$: Falls in NW quadrant (empty), stored there.

All four children now each hold one point, and no further subdivision is needed until a fifth point lands in an occupied quadrant.

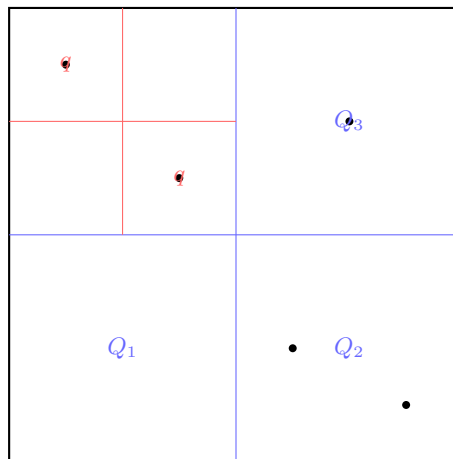


Figure 10.2: Quadtree: recursive 2D space partitioning. Quadrants with multiple points are subdivided; leaf quadrants contain at most one point.

10.2.4 4. Pseudo code

10.2.5 5. Parameters Selections

Definition 10.18 (Quadtree Node Capacity). The *node capacity* is typically set to 4–16 points. Higher capacity reduces the number of internal nodes but increases the linear search time within each node.

Definition 10.19 (Maximum Depth). The *maximum depth* is a limit on subdivision to prevent infinite recursion in the case of overlapping or very dense points.

10.2.6 6. Complexity

Theorem 10.20 (Quadtree Complexity). For n uniformly distributed points in a quadtree with capacity c :

1. **Construction:** $O(n \log n)$ on average; $O(n \cdot \text{depth})$ worst case for clustered data.
2. **Range Query:** $O(\log n + t)$ for localized queries reporting t points [Sam84].

```
1: function INSERT(node, point)
2:   if not node.boundary.contains(point) then
3:     return False
4:   end if
5:   if len(node.points) ≥ node.capacity and not node.is_divided then
6:     node.points.append(point)
7:     return True
8:   end if
9:   if not node.is_divided then
10:    node.subdivide()
11:  end if
12:  if node.nw.insert(point) then
13:    return True
14:  end if
15:  if node.ne.insert(point) then
16:    return True
17:  end if
18:  if node.sw.insert(point) then
19:    return True
20:  end if
21:  if node.se.insert(point) then
22:    return True
23:  end if
24: end function
25: function QUERY(node, range, result)
26:   if not node.boundary.intersects(range) then
27:     return
28:   end if
29:   for p in node.points do
30:     if range.contains(p) then
31:       result.append(p)
32:     end if
33:   end for
34:   if node.is_divided then
35:     Query(node.nw, range, result)
36:     Query(node.ne, range, result)
37:     Query(node.sw, range, result)
38:     Query(node.se, range, result)
39:   end if
40: end function
```

▷ Try inserting into children

▷ Check points in current node

▷ Check children

3. **Space:** $O(n)$ on average, though internal node overhead can grow with depth.

Remark 10.21. For highly non-uniform point distributions, quadtree depth can reach $O(n)$ in the worst case. Using a **point quadtree** variant (instead of region quadtree) or a loose quadtree can mitigate this.

10.2.7 7. Usages

- **Collision Detection:** Quickly finding pairs of objects that are close enough to collide.
- **Image Compression:** Representing regions of uniform color with a single large node (e.g., in the TGA or PNG-like formats).
- **Video Games:** View frustum culling (rendering only objects that are visible within the camera’s FOV).
- **GIS:** Storing and querying map features like road networks or city locations.
- **Particle Systems:** Efficiently calculating local forces (e.g., gravity or repulsion) in many-body simulations.

10.2.8 8. Testing methods and Edge cases

- **Uniform Point Distribution:** Verify that the tree divides evenly into a balanced structure.
- **Points on Boundaries:** Ensure points exactly on the dividing lines are assigned to exactly one child.
- **High-Density Clusters:** Ensure the “capacity” and “max depth” parameters prevent excessive subdivision.
- **Empty Areas:** Verify that large regions with no points are represented by a single empty leaf node.
- **Resizing:** Test if the quadtree can handle a dynamic bounding box or if it requires a fixed initial area.

10.2.9 9. References

- Finkel, R. A., & Bentley, J. L. (1974). “Quad trees: A data structure for retrieval on composite keys”. *Acta Informatica*.
- Samet, H. (1984). “The Quadtree and Related Hierarchical Data Structures”. *ACM Computing Surveys*.
- Samet, H. (2006). “Foundations of Multidimensional and Metric Data Structures”. Morgan Kaufmann.
- Wikipedia: [Quadtree](#)

10.3 R-Trees

10.3.1 1. Overview

An **R-tree** is a tree data structure used for efficient spatial indexing of multidimensional information, particularly objects represented as Minimum Bounding Rectangles (MBRs) [Gut84]. Unlike KD-trees or quadtrees, which partition the space itself, R-trees group the objects into hierarchical, potentially overlapping regions. This makes R-trees exceptionally well-suited for disk-based databases and large-scale spatial datasets.

10.3.2 2. Definitions

Definition 10.22 (Minimum Bounding Rectangle). The *Minimum Bounding Rectangle (MBR)* of a node is the smallest rectangle (axis-aligned) that contains all the elements in that node's subtree.

Definition 10.23 (R-tree Leaf Node). A *leaf node* stores pointers to the actual geometric objects and their MBRs.

Definition 10.24 (R-tree Internal Node). An *internal node* stores a pointer to a child node and the MBR of that child.

Definition 10.25 (R-tree Balance Property). An R-tree is always *height-balanced*: all leaf nodes appear at the same depth. This is analogous to the B-tree balance property.

10.3.3 3. Theory

R-trees are built by grouping nearby objects and enclosing them in their MBR.

1. **Insertion:** To insert an object, we start at the root and recursively choose a child node whose MBR requires the least expansion to include the new object.
2. **Node Splitting:** When a node exceeds its capacity, it must be split into two. Since the choice of split affects search performance, heuristics are used to minimize the area and overlap of the two new MBRs (e.g., Quadratic Split or Linear Split).
3. **Search:** To find objects in a query region, we traverse all branches whose MBRs intersect the query. Because MBRs at the same level can overlap, multiple branches may need to be explored.

Theorem 10.26 (R-tree Search Complexity). *For n objects stored in an R-tree with node capacity M and height $h = O(\log_M n)$, a window query reports t objects in expected time $O(M^{h-1} + t)$ under uniform object distribution. For the R*-tree [BKSS90], the average number of disk accesses per query is significantly reduced due to minimized MBR overlap.*

Remark 10.27. The worst-case search time for an R-tree is $O(n)$ when all MBRs overlap heavily, effectively degrading to a linear scan. The R*-tree [BKSS90] mitigates this by using a more sophisticated split heuristic and forced reinsertion.

10.3.4 4. Pseudo code

Example 10.28 (R-tree Insertion). Consider inserting objects $A = [0, 1] \times [0, 1]$, $B = [3, 4] \times [0, 1]$, $C = [0, 1] \times [3, 4]$, $D = [3, 4] \times [3, 4]$, and $E = [1, 2] \times [1, 2]$ into an R-tree with capacity $M = 2$.

1. Insert A : root is a leaf with $[A]$.
2. Insert B : root becomes $[A, B]$. MBR is $[0, 4] \times [0, 1]$.

```
1: function SEARCH(node, query_rect)
2:   results  $\leftarrow$  []
3:   if node.is_leaf then
4:     for obj in node.objects do
5:       if Intersects(obj.mbr, query_rect) then
6:         results.append(obj)
7:       end if
8:     end for
9:   else
10:    for child in node.children do
11:      if Intersects(child.mbr, query_rect) then
12:        results.extend(Search(child, query_rect))
13:      end if
14:    end for
15:  end if
16:  return results
17: end function
18: function INSERT(node, obj)
19:   if node.is_leaf then
20:     node.add(obj)
21:     if node.count  $\geq$  MAX_CAPACITY then
22:       return Split(node)  $\triangleright$  Returns two nodes
23:     end if
24:   else  $\triangleright$  1. Choose subtree that requires least area expansion
25:     child  $\leftarrow$  ChooseBestSubtree(node, obj)
26:     split_result  $\leftarrow$  Insert(child, obj)
27:      $\triangleright$  2. Update node's MBR
28:     node.mbr  $\leftarrow$  Union(node.mbr, obj.mbr)
29:      $\triangleright$  3. Handle split if it occurred in child
30:     if split_result is not None then
31:       node.add_child(split_result)
32:       if node.child_count  $\geq$  MAX_CAPACITY then
33:         return Split(node)
34:       end if
35:     end if
36:   end if
37:   return None
38: end function
```

3. Insert C : root overflows. Split into two nodes: $\{A, C\}$ (MBR $[0, 1] \times [0, 4]$) and $\{B\}$ (MBR $[3, 4] \times [0, 1]$). A new root is created.
4. Insert D : descends to the B -node, making it $\{B, D\}$.
5. Insert E : the MBR of $\{A, C\}$ is $[0, 1] \times [0, 4]$ and the MBR of $\{B, D\}$ is $[3, 4] \times [0, 4]$. Object $E = [1, 2] \times [1, 2]$ overlaps neither MBR exactly, but whichever node requires least expansion is chosen.

10.3.5 5. Parameters Selections

Definition 10.29 (R-tree Split Algorithm). **Guttman’s Quadratic Split** [Gut84] is a common splitting heuristic. The **R*-tree** variant [BKSS90] uses a more complex split that also considers overlap and perimeter, leading to significantly better search performance.

Definition 10.30 (R-tree Min/Max Capacity). Typically, nodes are required to be at least 40% full to maintain balance and storage efficiency. The maximum capacity M depends on the disk page size.

10.3.6 6. Complexity

Theorem 10.31 (R-tree Complexity Summary). *For n objects stored in an R-tree with height h :*

1. **Insertion:** $O(\log_M n)$ expected. Worst case $O(n)$ if many splits propagate.
2. **Search:** $O(M^{h-1} + t) = O(n^{1-1/\alpha} + t)$ for localized queries [Gut84].
3. **Space:** $O(n)$ for storage, plus $O(n/M)$ internal nodes.

10.3.7 7. Usages

- **Spatial Databases:** The core indexing mechanism for PostGIS, Oracle Spatial, and MySQL Spatial.
- **GIS:** Indexing road networks, parcels, and satellite imagery.
- **Network Analysis:** Routing and nearest-neighbor searches in complex maps.
- **Computer Graphics:** Managing large scenes with many individual models.
- **Internet of Things (IoT):** Real-time tracking and querying of moving sensors or vehicles.

10.3.8 8. Testing methods and Edge cases

- **Highly Overlapping Objects:** Ensure the tree doesn’t degrade into a linked list when many objects occupy the same space.
- **Large Objects:** A single large rectangle can intersect many MBRs, causing search to slow down.
- **Empty Search:** Verify the algorithm correctly identifies when no objects are in the query area.
- **Bulk Loading:** For static datasets, use a “Sort-Tile-Recursive” (STR) bulk loader to build a nearly optimal tree.
- **Deletion:** R-trees handle deletions by merging nodes if they fall below minimum capacity.

10.3.9 9. References

- Guttman, A. (1984). “R-trees: A dynamic index structure for spatial searching”. SIGMOD.
- Beckmann, N., Kriegel, H. P., Schneider, R., & Seeger, B. (1990). “The R*-tree: An efficient and robust access method for points and rectangles”. SIGMOD.
- Samet, H. (2006). “Foundations of Multidimensional and Metric Data Structures”. Morgan Kaufmann.
- Wikipedia: [R-tree](#)

10.4 Interval Trees

10.4.1 1. Overview

An **interval tree** is a balanced binary search tree used to store segments (intervals) and allow for efficient range queries [Ede80]. Specifically, it is designed to answer the question: “Which of the stored intervals overlap with a given query interval $[x, y]$?” Interval trees are an essential tool in computational geometry for solving 1D segment intersection problems and are a key component of more complex spatial structures.

10.4.2 2. Definitions

Definition 10.32 (Interval). An *interval* is a pair of numbers $[start, end]$ with $start \leq end$.

Definition 10.33 (Interval Overlap). Two intervals $[a, b]$ and $[c, d]$ *overlap* if and only if $a \leq d$ and $c \leq b$.

Definition 10.34 (Interval Tree Node). Each node in an interval tree stores a single interval and the maximum value *max* found in its subtree. For any node N , $N.max = \max(N.interval.end, N.left.max, N.right.max)$.

10.4.3 3. Theory

The interval tree is built using the **start point** of each interval as the key in a standard binary search tree. Each node additionally maintains the maximum endpoint of all intervals in its subtree. This “max” property is key for efficient pruning during a query.

To search for an interval overlapping with $[q_{start}, q_{end}]$:

1. Check if the current node’s interval overlaps.
2. If the left child exists and its *max* is $\geq q_{start}$, we must search the left branch (there could be an overlap there).
3. Otherwise, we only search the right branch.

This pruning logic ensures that we don’t explore subtrees that cannot possibly contain an overlapping interval.

Theorem 10.35 (Interval Tree Query Complexity). *Given n intervals stored in an interval tree, all intervals overlapping a query interval $[q_{start}, q_{end}]$ can be reported in $O(k + \log n)$ time, where k is the number of reported intervals [Ede80].*

Proof. The search descends from the root to a leaf along at most $O(\log n)$ nodes. At each node, checking for overlap costs $O(1)$. The left subtree of a node is explored only if its *max* value exceeds q_{start} . By the max property, any interval in the left subtree with $end \geq q_{start}$ must have been accounted for, so we visit at most $O(k)$ additional nodes. The total cost is $O(\log n + k)$. $\square$

Example 10.36 (Interval Tree Overlap Query). Build an interval tree on $\{[15, 20], [10, 30], [17, 19], [5, 20], [12, 15]\}$ ordered by start point.

- Root: $[15, 20]$, $max = 40$.
- Query $[18, 22]$: Check root $[15, 20]$: overlaps ($15 \leq 22$ and $18 \leq 20$). Report it. Left child exists and $max = 30 \geq 18$, so recurse left. Right child: $[30, 40]$, $max = 40 \geq 18$, so recurse right. Continue pruning based on max values.

Reported intervals: $[15, 20]$, $[10, 30]$, $[17, 19]$, $[5, 20]$.

10.4.4 4. Pseudo code

```

1: function INSERT(node, interval)
2:   if node is None then
3:     return Node(interval, interval.end)
4:   end if
5:   if interval.start < node.interval.start then
6:     node.left  $\leftarrow$  Insert(node.left, interval)
7:   else
8:     node.right  $\leftarrow$  Insert(node.right, interval)
9:   end if
                                      $\triangleright$  Update the max endpoint in this subtree
10:  node.max  $\leftarrow$  max(node.max, interval.end)
11:  return node
12: end function
13: function FINDOVERLAPS(node, query, results)
14:   if node is None then
15:     return
16:   end if
                                      $\triangleright$  Check current node
17:   if Overlaps(node.interval, query) then
18:     results.append(node.interval)
19:   end if
                                      $\triangleright$  Choose branch
20:   if node.left is not None and node.left.max  $\geq$  query.start then
21:     FindOverlaps(node.left, query, results)
22:   end if
                                      $\triangleright$  Always check right if there's potentially an overlap
                                      $\triangleright$  We can skip if
                                     node.interval.start < query.end
23:   if node.right is not None and node.right.max  $\geq$  query.start then
24:     FindOverlaps(node.right, query, results)
25:   end if
26: end function

```

10.4.5 5. Parameters Selections

Definition 10.37 (Interval Tree Balancing). To maintain $O(\log n)$ performance, the underlying BST should be a self-balancing tree such as an **AVL Tree** or a **Red-Black Tree**.

Definition 10.38 (Interval Type). The interval tree can handle open, closed, or half-open intervals with consistent comparison logic by adjusting the overlap predicate.

10.4.6 6. Complexity

Theorem 10.39 (Interval Tree Complexity Summary). *For n intervals:*

1. **Construction:** $O(n \log n)$ to insert n intervals into a balanced tree.
2. **Query:** $O(k + \log n)$ where k is the number of overlapping intervals found [[Ede80](#)].
3. **Space:** $O(n)$ to store each interval exactly once.

10.4.7 7. Usages

- **Resource Allocation:** Finding all booked time slots that conflict with a new appointment.
- **VLSI Design:** Design rule checking (checking for overlapping wires on a chip).
- **Database Indexing:** Efficiently searching through time-series data or ranges.
- **Ray Tracing:** 1D interval trees are used to find all objects a ray passes through.
- **Video Games:** Managing “life spans” or “event timelines.”

10.4.8 8. Testing methods and Edge cases

- **Contained Intervals:** An interval $[5, 10]$ fully containing another $[6, 7]$.
- **Sharing Boundaries:** Intervals $[5, 10]$ and $[10, 15]$ overlapping only at 10.
- **Large and Small Max values:** Ensure the `node.max` correctly propagates from the leaves to the root.
- **Disconnected Intervals:** Searching for overlaps in a region with no intervals.
- **Identical Intervals:** Multiple nodes with the exact same $[start, end]$.

10.4.9 9. References

- Edelsbrunner, H. (1980). “Dynamic data structures for orthogonal intersection queries”. Technical Report.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). “Introduction to Algorithms”. MIT Press.
- Wikipedia: [Interval tree](#)

10.5 Segment Trees

10.5.1 1. Overview

A **segment tree** is a versatile tree data structure used for storing information about intervals or segments [[Ben77](#)]. It is primarily used for range queries where we need to find the sum, minimum, or maximum of a specified range of values in an array that can be updated. In computational geometry, segment trees are used to solve problems involving orthogonal segments and rectangles.

10.5.2 2. Definitions

Definition 10.40 (Segment Tree). A *segment tree* is a binary tree where each node represents an interval of an array. The root represents $[0, n - 1]$, and each internal node representing $[L, R]$ has children representing $[L, \lfloor (L + R)/2 \rfloor]$ and $[\lfloor (L + R)/2 \rfloor + 1, R]$.

Definition 10.41 (Segment Tree Leaf Node). A *leaf node* represents a single element of the array.

Definition 10.42 (Lazy Propagation). *Lazy propagation* is a technique to optimize range updates by storing pending updates at internal nodes and pushing them down only when those children are accessed, maintaining $O(\log n)$ performance per operation.

10.5.3 3. Theory

A segment tree for an array of n elements is built by recursively splitting the array in half. The root node represents the interval $[0, n - 1]$. Each internal node representing $[L, R]$ has children representing $[L, mid]$ and $[mid + 1, R]$.

To solve a **Range Sum Query** for $[qL, qR]$:

1. If the current node's interval is fully within $[qL, qR]$, return the node's sum.
2. If the current node's interval is completely outside $[qL, qR]$, return 0.
3. Otherwise, recurse to both children and return the sum of their results.

Theorem 10.43 (Segment Tree Query Complexity). *Any range query (sum, min, max, etc.) on a segment tree of size n can be answered in $O(\log n)$ time.*

Proof. Any query range $[qL, qR]$ can be decomposed into at most $2 \log_2 n$ disjoint node intervals. This follows from the observation that at each level of the tree, at most two nodes are “partially overlapping” (one on each side of the decomposition), while all others are either fully inside or fully outside. Thus we visit $O(\log n)$ nodes at each of the $O(\log n)$ levels. $\square$

For **Range Updates**, a naive approach would update all relevant leaves, taking $O(n)$ time. **Lazy Propagation** stores the update at the highest relevant internal node and pushes it down only when those children are accessed, maintaining $O(\log n)$ performance.

Theorem 10.44 (Lazy Propagation Complexity). *With lazy propagation, both point updates and range updates on a segment tree of size n take $O(\log n)$ time.*

Proof. A range update modifies at most $O(\log n)$ nodes (the same decomposition argument as in Theorem 10.43). Each node's lazy tag is pushed down at most once before its children are accessed, contributing $O(1)$ amortized work per level. Thus the total work per update is $O(\log n)$. $\square$

Example 10.45 (Segment Tree Range Query). Build a segment tree on $A = [2, 1, 5, 3, 4, 2]$.

- Root: interval $[0, 5]$, sum = 17.
- Left child $[0, 2]$: sum = 8; right child $[3, 5]$: sum = 9.
- Query $[1, 4]$: The range $[1, 4]$ partially overlaps $[0, 2]$ and $[3, 5]$. Recurse into $[1, 2]$ (sum = $1 + 5 = 6$) and $[3, 4]$ (sum = $3 + 4 = 7$). Result: $6 + 7 = 13$.

```
1: function BUILD(arr, node, start, end)
2:   if start == end then
3:     tree[node] ← arr[start]
4:     return
5:   end if
6:   mid ← (start + end) / 2
7:   Build(arr, 2*node, start, mid)
8:   Build(arr, 2*node+1, mid+1, end)
9:   tree[node] ← tree[2*node] + tree[2*node+1]
10: end function
11: function RANGEQUERY(node, start, end, qL, qR)
12:   if qR < start or end < qL then
13:     return 0
14:   end if
15:   if qL ≤ start and end ≤ qR then
16:     return tree[node]
17:   end if
18:   mid ← (start + end) / 2
19:   p1 ← RangeQuery(2*node, start, mid, qL, qR)
20:   p2 ← RangeQuery(2*node+1, mid+1, end, qL, qR)
21:   return p1 + p2
22: end function
```

▷ Completely outside

▷ Completely inside

10.5.4 4. Pseudo code

10.5.5 5. Parameters Selections

Definition 10.46 (Segment Tree Operator). The segment tree can store *any associative operator*: Sum, Min, Max, GCD, matrix product, etc. The choice of operator determines the type of query the tree supports.

Definition 10.47 (Segment Tree Size). For an array of n elements, the segment tree requires $4n$ space in an array-based implementation (since a complete binary tree on n leaves has fewer than $4n$ nodes).

10.5.6 6. Complexity

Theorem 10.48 (Segment Tree Complexity Summary). *For an array of n elements stored in a segment tree:*

1. **Build**: $O(n)$ time and $O(n)$ space.
2. **Point Update**: $O(\log n)$ time.
3. **Range Query**: $O(\log n)$ time.
4. **Range Update (lazy)**: $O(\log n)$ time.

10.5.7 7. Usages

- **Range Minimum Query (RMQ)**: Finding the smallest value in a subarray.
- **Computational Geometry (Bentley-Ottmann)**: Used as the status structure for identifying intersections between vertical segments during a horizontal sweep.

- **Rectangle Union:** Calculating the area of the union of several rectangles (Klee’s measure problem).
- **Dynamic Connectivity:** Keeping track of connectivity in a graph that is undergoing edge insertions and deletions.
- **Statistics:** Calculating running averages or variances in real-time data streams.

10.5.8 8. Testing methods and Edge cases

- **Single Element Ranges:** Querying and updating a range where $qL == qR$.
- **Full Array Query:** Querying the entire range $[0, n - 1]$ should return the root node’s value.
- **Overlapping Updates:** Multiple range updates on overlapping segments should be correctly accumulated by lazy propagation.
- **Power of Two vs. Arbitrary n :** Ensure the tree correctly handles array sizes that are not powers of two.
- **Negative Values:** If using Min/Max, ensure negative numbers are handled correctly (e.g., initial values should be $-\infty$ or $+\infty$).

10.5.9 9. References

- Bentley, J. L. (1977). “Algorithms for Klee’s measure problem”. Technical Report.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). “Introduction to Algorithms”. MIT Press.
- Wikipedia: [Segment tree](#)

10.6 Uniform Grids

10.6.1 1. Overview

A **uniform grid** partitions the plane into congruent axis-aligned cells of fixed side length δ , storing each point in the cell that contains it. Queries inspect only the cells that can possibly contain the answer, giving a simple and very fast spatial index for uniformly distributed data [BCKO08]. Grids are also the workhorse of spatial sorting, spatial hashing, and image-based algorithms.

10.6.2 2. Definitions

Definition 10.49 (Uniform Grid). A *uniform grid* of side length δ maps each point p to the cell

$$\text{cell}(p) = \left(\left\lfloor \frac{p_x}{\delta} \right\rfloor, \left\lfloor \frac{p_y}{\delta} \right\rfloor \right),$$

and a cell index stores all points that map to it, typically in a hash table keyed by the cell coordinates.

Definition 10.50 (Query Neighborhood). A query point q examines the cells within distance r of q : for a nearest-neighbor query with search radius r , only the $(2\lceil r/\delta \rceil + 1)^2$ cells in the Chebyshev neighborhood of $\text{cell}(q)$ need inspection.

10.6.3 3. Theory

The choice of δ trades cell density against the number of cells examined per query. For n points distributed uniformly over an area A , taking $\delta \approx \sqrt{A/n}$ keeps a constant expected number of points per cell, so a range query of radius r inspects $O((r/\delta)^2)$ cells. The grid degenerates when the data is highly non-uniform; hierarchical grids that adapt cell size locally are one remedy [BCKO08].

10.6.4 4. Pseudo code

```
function BUILDGRID( $P, \delta$ )  
   $T \leftarrow$  empty hash table  
  for each  $p \in P$  do  
     $T[\text{cell}(p)] \leftarrow T[\text{cell}(p)] \cup \{p\}$   
  end for  
  return  $T$   
end function  
function NEARESTCANDIDATECELLS( $q, r, \delta$ )  
  for each cell  $c$  with  $\|c - \text{cell}(q)\|_\infty \leq \lceil r/\delta \rceil$  do  
    emit  $T[c]$   
  end for  
end function
```

10.6.5 5. Parameters Selections

- **Cell side length δ :** choose $\delta \approx \sqrt{A/n}$ for uniform data; for queries, a common heuristic is δ equal to the average query radius.
- **Hash function:** use a multiplicative hash of the cell coordinates (e.g., “hashing the cell pair”) to spread adjacent cells across buckets.
- **Boundary points:** points exactly on a cell boundary must be assigned to exactly one cell, consistently.

10.6.6 6. Complexity

Build time and space are $O(n)$. A range query of radius r inspects $O((r/\delta)^2)$ cells, each holding an expected constant number of points for uniform data; worst-case behavior is poor for clustered or pathological inputs [BCKO08].

10.6.7 7. Usages

- **Collision detection:** pairs of objects that cannot collide are those in distant cells.
- **Nearest neighbor queries:** spatial sorting in Delaunay construction, and “point grid” acceleration in triangulation [BCKO08].
- **Image processing and games:** pixel neighborhoods and broad-phase culling in a game engine.

10.6.8 8. Testing methods and Edge cases

- **Empty cells:** queries near the boundary must tolerate cells with no stored points.

- **Non-uniform data:** verify query cost on strongly clustered inputs; consider an adaptive or hierarchical variant.
- **Large coordinates:** guard against overflow when computing $\lfloor p_x/\delta \rfloor$.

10.6.9 9. References

- de Berg, M., Cheong, O., van Kreveld, M., & Overmars, M. (2008). “Computational Geometry: Algorithms and Applications”. Springer.
- Wikipedia: [Spatial hashing](#)

10.7 Fixed-Radius Neighbor Search

10.7.1 1. Overview

Fixed-radius neighbor search (also called **ball query** or **range search with radius r**) reports, for a query point q , all stored points within a prescribed distance r . It is the natural tool for billion-point 2D/3D point clouds—sensor networks, particle simulations, and point-cloud processing—where the query distance is known in advance. The problem is a special case of range search (Section 13.4) with a ball as the range, but the fixed radius permits data structures, most notably the uniform grid, that answer queries in time proportional to the output size plus the boundary of the query ball [BCKO08].

10.7.2 2. Definitions

Definition 10.51 (Fixed-Radius Query). Given a set P of n points in $\mathbb{R}^d$ and a radius r , a *fixed-radius query* at q reports

$$N_r(q) = \{p \in P : \|p - q\|_2 \leq r\}.$$

The answer count $|N_r(q)|$ is the *ball count*.

Definition 10.52 (Grid Search Region). In a uniform grid of cell side δ , the cells that may contain points within distance r of q are those whose Chebyshev distance from $\text{cell}(q)$ is at most $\lceil r/\delta \rceil$; every point of $N_r(q)$ lies in this region because a point farther than r in one coordinate is automatically farther than r in Euclidean distance.

10.7.3 3. Theory

A uniform grid with cell side $\delta \geq r$ guarantees that only the $(2\lceil r/\delta \rceil + 1)^d$ cells of the Chebyshev neighborhood of $\text{cell}(q)$ need to be scanned. Setting $\delta = r$ minimizes the number of cells examined to 3^d while keeping every answer within the candidate region; each candidate point is then accepted or rejected by the exact Euclidean test $\|p - q\|_2 \leq r$.

Theorem 10.53 (Grid Query Cost). *For n points distributed uniformly over a region of volume V and a query radius r with cell side $\delta = r$, a fixed-radius query examines $O(1)$ cells in the worst case over the choice of query point, and each cell holds an expected $O(nr^d/V)$ points. Hence the expected query time is $O(nr^d/V + |N_r(q)|)$, which for dense clouds (large nr^d/V) reduces to output-size dominated work [BCKO08].*

For sparse clouds or non-uniform data, tree structures give better worst-case bounds: a k -d tree (Section 10.1) or octree (Section 10.2) reports $N_r(q)$ in $O(n^{1-1/d} + k)$ time with $O(n)$ space, where $k = |N_r(q)|$.

Algorithm 18 Fixed-Radius Neighbor Search

```
function BUILDFIXEDRADIUSINDEX(points, r)
     $\delta \leftarrow r$ 
     $T \leftarrow$  empty hash table keyed by cell coordinates
    for each  $p \in P$  do
         $T[\text{cell}(p)] \leftarrow T[\text{cell}(p)] \cup \{p\}$ 
    end for
    return  $T$ 
end function
function FIXEDRADIUSQUERY( $q, r, T$ )
     $c \leftarrow \text{cell}(q)$ 
     $out \leftarrow \emptyset$ 
    for each cell  $c'$  with  $\|c' - c\|_\infty \leq 1$  do
        for each  $p \in T[c']$  do
            if  $\|p - q\|_2 \leq r$  then  $\triangleright$  exact Euclidean test
                 $out \leftarrow out \cup \{p\}$ 
            end if
        end for
    end for
    return  $out$ 
end function
```

10.7.4 4. Pseudo code**10.7.5 5. Parameter Selection**

- **Cell side δ :** setting $\delta = r$ keeps the query neighborhood to 3^d cells. Smaller cells reduce per-cell work but increase the number of cells scanned; larger cells invert the trade-off.
- **Hash function:** hash the cell coordinates to a dense bucket array so that adjacent cells spread evenly (as in Section 10.6).

10.7.6 6. Complexity

- **Space:** $O(n)$ cells plus $O(n)$ stored points.
- **Build time:** $O(n)$.
- **Query time:** expected $O(nr^d/V + |N_r(q)|)$ on a grid; $O(n^{1-1/d} + |N_r(q)|)$ worst case on a k-d tree or octree.

10.7.7 7. Usages

- **Sensor networks:** finding all sensors within communication range of a query point.
- **Particle simulations:** building interaction lists within the cutoff radius.
- **Point-cloud processing:** pairing points within a tolerance for surface reconstruction and registration.
- **Collision detection:** broad-phase culling of objects within a blast or detection radius.

10.7.8 8. Testing methods and Edge cases

- **Boundary cells:** points exactly at distance r may lie on a shared cell boundary; assign each point to exactly one cell and rely on the exact test $\|p - q\|_2 \leq r$ for inclusion.
- **Ball count:** compare against a brute-force scan for small n ; verify the reported set has exactly the brute-force size.
- **Clustered data:** test strongly non-uniform inputs, where the grid degenerates and the tree variant is preferable.
- **Outside domain:** queries whose ball extends past the domain bounds must skip empty hash buckets.
- **Large coordinates:** guard against overflow when computing $\lfloor p_x/r \rfloor$.

10.7.9 9. References

- de Berg, M., Cheong, O., van Kreveld, M., & Overmars, M. (2008). “Computational Geometry: Algorithms and Applications”. Springer.
- Wikipedia: [Nearest neighbor search](#)

10.8 Binary Space Partitioning Trees

10.8.1 1. Overview

A **binary space partitioning (BSP) tree** recursively partitions space by a hyperplane into two half-spaces, building a binary tree whose leaves correspond to convex regions of the partition [FKN80]. BSP trees were introduced for hidden-surface removal and are used for painter’s-algorithm sorting, collision detection, and image compression; the k-d tree of Section 10.1 is the special case where every splitting plane is axis-aligned.

10.8.2 2. Definitions

Definition 10.54 (BSP Tree). A *BSP tree* on a set of objects S is a binary tree where each internal node stores a hyperplane h ; the left subtree contains the objects of S entirely in the back half-space of h , the right subtree those in the front half-space, and objects crossing h are split and stored in both subtrees. Leaves are homogeneous convex regions.

Definition 10.55 (Autopartition). An *autopartition* uses, at every node, the supporting line of one of the input segments as the splitting line. Restricting splitting lines to input edges bounds both the number of pieces and the size of the tree.

10.8.3 3. Theory

For a set of n disjoint segments in the plane, a BSP tree is built by recursive subdivision: choose a splitting line h , split every segment that crosses h into two fragments, and recurse on each half-plane. The number of fragments can grow, but for disjoint line segments there is an autopartitioning scheme producing a tree of size $O(n^2)$ in the worst case and $O(n \log n)$ expected under random choices [BCKO08]. In $\mathbb{R}^3$, BSP trees of linear size exist for suitably restricted inputs, and the tree supports painter’s-algorithm depth sorting and point-location queries in $O(n)$ time.

10.8.4 4. Pseudo code

```
function BUILD BSP( $S$ )  
  if  $|S| \leq 1$  then  
    return leaf with  $S$   
  end if  
  choose a splitting line  $h$  from  $S$   
   $S^+ \leftarrow \emptyset$ ,  $S^- \leftarrow \emptyset$   
  for each segment  $s \in S$  do  
    if  $s$  lies in front of  $h$  then  
       $S^+ \leftarrow S^+ \cup \{s\}$   
    else if  $s$  lies in back of  $h$  then  
       $S^- \leftarrow S^- \cup \{s\}$   
    else  
      split  $s$  into  $s_1, s_2$  at  $h$ ; add to  $S^+$  and  $S^-$   
    end if  
  end for  
  return node( $h$ , BSP( $S^-$ ), BSP( $S^+$ ))  
end function
```

10.8.5 5. Parameters Selections

- **Splitting strategy:** random autopartitioning gives $O(n \log n)$ expected tree size and is easy to implement; greedy strategies that minimize splits improve performance on real data [NAT90].
- **Axis-aligned splits:** restricting to vertical or horizontal lines yields a k-d tree, trading generality for a simpler construction.
- **Robustness:** fragmenting segments requires exact predicates to classify points against the splitting line.

10.8.6 6. Complexity

For n disjoint segments in the plane, an autopartition can be built in $O(n \log n)$ expected time and yields $O(n \log n)$ expected fragments; worst-case size is $O(n^2)$. A point-location or visibility query descends one root-to-leaf path in $O(n)$ time [BCKO08, FKN80].

10.8.7 7. Usages

- **Hidden-surface removal:** the painter's algorithm draws the back-to-front order given by the tree [FKN80].
- **Collision detection:** convex leaf regions allow fast rejection tests in robotics and games.
- **Discrete BSP / image compression:** the Doom engine's rendering and classic BSP-based image coding [NAT90].

10.8.8 8. Testing methods and Edge cases

- **Coincident segments:** overlapping or identical input segments must be assigned consistently to one side.
- **Segments on the splitting line:** define a deterministic tie-break for segments lying exactly on h .

- **Non-uniform inputs:** verify tree size does not blow up to $O(n^2)$ on adversarial configurations.

10.8.9 9. References

- Fuchs, H., Kedem, Z. M., & Naylor, B. F. (1980). “On visible surface generation by a priori tree structures”. *Computer Graphics (SIGGRAPH)*.
- Naylor, B., Amanatides, J., & Thibault, W. (1990). “Merging BSP trees yields polyhedral set operations”. *Computer Graphics (SIGGRAPH)*.
- de Berg, M., Cheong, O., van Kreveld, M., & Overmars, M. (2008). “Computational Geometry: Algorithms and Applications”. Springer.
- Wikipedia: [Binary space partitioning](#)

10.9 Industrial Applications of BSP Trees

BSP trees were invented for the painter’s algorithm in graphics, but they became the structural foundation of the first-generation real-time 3D game engines, and the axis-aligned special case (the kd-tree) remains the ray-traversal kernel of commercial production renderers [FKN80, NAT90]. The following are the principal industrial use cases.

- **First-Person Shooter Engines (Licensed Middleware):** The Doom engine stored each level as a BSP tree of subsectors, giving back-to-front visibility without per-polygon sorting. The lineage continued in the Quake engines (id Tech 2 and id Tech 3), which were licensed to outside studios—most notably a heavily modified id Tech 3, which shipped as *Call of Duty* (2003), the first release of a franchise that has generated billions in revenue. The BSP leaves in these engines also serve the collision and line-of-sight queries of the game physics [NAT90].
- **Brush Worlds and CSG in Commercial Level Design:** Level geometry in the Source engine is assembled from convex BSP brushes; the editor performs BSP-based CSG (union, subtraction) to carve rooms and archways, and the resulting tree provides visibility culling, portal connectivity, and collision in a single structure. Source powered commercial titles such as *Counter-Strike: Source*, *Team Fortress 2*, and *Half-Life 2*.
- **Shadow Volumes in the Doom 3 Engine:** The BSP tree’s leaf depth ordering lets a stencil shadow volume terminate where it enters an occluded cell. Doom 3 (id Tech 4) popularized this combination, and the technique entered the real-time shadow pipelines of subsequent commercial engines.
- **Production Renderers (kd-Tree Ray Traversal):** The axis-aligned BSP, or kd-tree (Section 10.1), accelerates ray-vs-scene traversal by pruning half of space at each node, and has long served as the acceleration structure of commercial offline renderers such as V-Ray (Chaos), which is licensed by architecture, product-design, and visual-effects firms for photorealistic lighting, shadows, and global illumination.
- **Robotics and Automation Collision Checking:** Each BSP node’s half-space test rejects whole branches of the tree in $O(1)$ time, giving broad-phase collision rejection for robotic manipulators and automated guided vehicles operating against static polygonal cell and warehouse models.

10.9.1 Industrial-Scale Software

BSP-based structures are implemented in the **Doom** and **Quake**/ id Tech engines (level geometry, PVS, and collision), the **Source** engine (brush worlds), and the kd-tree ray-traversal cores of commercial renderers such as V-Ray [FKN80, NAT90].

10.10 Space-Filling Curves

10.11 Hilbert Curve

10.11.1 Overview

The Hilbert curve is a continuous fractal space-filling curve first described by David Hilbert in 1891 [Hil91]. It is a mapping between 1D and 2D (or higher-dimensional) space that preserves locality: points that are close together on the 1D curve are also close together in the 2D plane. This property makes the Hilbert curve an invaluable tool for multidimensional indexing, data compression, and spatial analysis [Sag94].

10.11.2 Definitions

Definition 10.56 (Space-Filling Curve). A continuous function $f: [0, 1] \rightarrow [0, 1]^2$ whose range contains the entire 2-dimensional unit square. Formally, f is *surjective*: for every point $(x, y) \in [0, 1]^2$, there exists $t \in [0, 1]$ such that $f(t) = (x, y)$ [Sag94].

Definition 10.57 (Order (n)). The number of recursive subdivisions. An order- n Hilbert curve covers a grid of $2^n \times 2^n$ cells and visits all 4^n cells exactly once. The total curve length at order n is $2^n - 1$ cell-to-cell transitions.

Definition 10.58 (Locality Preservation). The property where distance on the curve (Hilbert index) correlates strongly with Euclidean distance in space. Specifically, for points $p, q \in [0, 1]^2$ with Hilbert indices $h(p), h(q)$, the Hilbert distance $|h(p) - h(q)|$ is bounded by a polynomial function of the Euclidean distance $\|p - q\|$ [MJFS01].

Definition 10.59 (U-shape). The basic motif of the Hilbert curve, which is rotated and reflected to form higher orders. The four orientations of the U-shape determine how the sub-curves connect at each level of recursion.

10.11.3 Theory

The Hilbert curve is constructed recursively. For order $n = 1$, it is a simple U-shape connecting four cells. To create order $n + 1$:

1. The space is divided into four quadrants.
2. Four order- n curves are placed in the quadrants.
3. The bottom-left curve is rotated 90° clockwise.
4. The bottom-right curve is rotated 90° counter-clockwise and reflected.
5. The four curves are connected by three segments.

The transformation from 2D coordinates (x, y) to a 1D index d (and vice versa) involves bitwise operations and coordinate rotations. As the order $n \rightarrow \infty$, the curve fills the entire square.

Theorem 10.60 (Locality Bound of the Hilbert Curve). *For any two points $p, q \in [0, 1]^2$ with Hilbert indices $h(p)$ and $h(q)$, there exists a constant $c > 0$ such that:*

$$|h(p) - h(q)| \leq c \cdot \|p - q\|_1 \cdot \max\left(\frac{1}{\|p - q\|_1}, 1\right)^{\log_2 d},$$

where d is the dimension. In 2D, this simplifies to the bound that points within a Hilbert-index window of width w are contained in a square of side length $O(\sqrt{w})$ [MJFS01].

Proof. The proof follows from the self-similar structure of the Hilbert curve. At each level k of the recursion, the space is partitioned into 4^k cells, each of side length 2^{-k} . Two points in the same level- k cell have Hilbert indices differing by at most 4^k . Conversely, if their Hilbert indices differ by Δ , they must share a common ancestor cell at level $k = \lceil \log_4 \Delta \rceil$, which has side length $2^{-k} \leq 2\sqrt{\Delta}$. This gives the $O(\sqrt{w})$ spatial bound for a window of width w [MJFS01]. $\square$

Example 10.61 (Hilbert Curve of Order 2). An order-2 Hilbert curve visits all $4^2 = 16$ cells in a 4×4 grid. Starting from the bottom-left corner $(0, 0)$:

$$\begin{aligned} (0, 0) &\rightarrow (1, 0) \rightarrow (1, 1) \rightarrow (0, 1) \rightarrow (0, 2) \rightarrow (0, 3) \rightarrow (1, 3) \rightarrow (1, 2) \rightarrow \\ &(2, 2) \rightarrow (2, 3) \rightarrow (3, 3) \rightarrow (3, 2) \rightarrow (3, 1) \rightarrow (2, 1) \rightarrow (2, 0) \rightarrow (3, 0) \end{aligned}$$

Points that are consecutive on the curve are always Manhattan neighbors (differing in exactly one coordinate by 1). The four quadrants each contain an order-1 U-shape, rotated and reflected as prescribed by the recursive construction [Sag94].

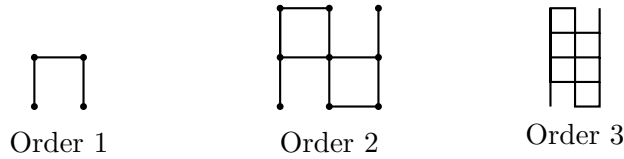


Figure 10.3: Hilbert curve construction: each order recursively places four rotated copies in a U-shaped pattern.

10.11.4 Pseudocode

Algorithm 19 converts 2D coordinates to a 1D Hilbert index using $O(n)$ bitwise operations. The inverse mapping `HilbertToXY` follows the same logic in reverse [Hil91, Sag94].

10.11.5 Parameter Selection

- **Order** (n): Typically chosen so that 2^n matches the resolution of the spatial grid.
- **Dimension**: While most common in 2D, the Hilbert curve can be extended to any number of dimensions k .

10.11.6 Complexity

- **Transformation**: $O(n)$ bitwise operations for an order- n curve. For a fixed grid size, this is essentially $O(1)$.
- **Space**: $O(1)$ auxiliary space to perform the coordinate mapping.

Algorithm 19 Hilbert Curve: Coordinates to Index

```
function XYTOHILBERT( $n, x, y$ )
   $d \leftarrow 0$ 
   $s \leftarrow 2^{n-1}$ 
  while  $s > 0$  do
     $rx \leftarrow (x \ \& \ s) > 0$ 
     $ry \leftarrow (y \ \& \ s) > 0$ 
     $d \leftarrow d + s \cdot s \cdot ((3 \cdot rx) \oplus ry)$   $\triangleright$  Rotate and flip
     $x, y \leftarrow \text{RotateHilbert}(s, x, y, rx, ry)$ 
     $s \leftarrow \lfloor s/2 \rfloor$ 
  end while return  $d$ 
end function

function ROTATEHILBERT( $n, x, y, rx, ry$ )
  if  $ry = 0$  then
    if  $rx = 1$  then
       $x \leftarrow n - 1 - x$ 
       $y \leftarrow n - 1 - y$ 
    end if return  $y, x$ 
  end if return  $x, y$ 
end function
```

10.11.7 Usages

- **Spatial Indexing:** Organizing data in databases (like MongoDB or GeoJSON) for faster range queries.
- **Image Dithering:** Ordering pixels for error diffusion to minimize visual artifacts.
- **Parallel Computing:** Mapping multidimensional tasks to linear processing units while maintaining local data relationships (e.g., in domain decomposition).
- **Data Compression:** Improving the performance of compression algorithms by clustering similar nearby values.
- **Cache Optimization:** Storing 2D arrays in Hilbert order to improve the L1/L2 cache hit rate for local access patterns.

10.11.8 Testing Methods and Edge Cases

- **Invertibility:** Verify that $\text{HilbertToXY}(\text{XYtoHilbert}(x, y)) == (x, y)$ for all points in a grid.
- **Adjacency:** Verify that points with sequential indices $d, d + 1$ are always Manhattan neighbors in the 2D grid.
- **Large Order:** Test the transformation with $n = 31$ to ensure bitwise operations don't overflow 64-bit integers.
- **All Quadrants:** Test points in each of the four main quadrants to ensure rotation and reflection logic is correct.

10.11.9 References

- Hilbert, D. (1891). “Über die stetige Abbildung einer Linie auf ein Flächenstück”. *Mathematische Annalen*.
- Sagan, H. (1994). “Space-Filling Curves”. Universitext.
- Moon, B., Jagadish, H. V., Faloutsos, C., & Saltz, J. H. (2001). “Analysis of the Clustering Properties of the Hilbert Space-Filling Curve”. *IEEE Transactions on Knowledge and Data Engineering*.
- Wikipedia: Hilbert curve

10.12 Morton Curve

10.12.1 Overview

The Morton curve (also known as Z-order) is a space-filling curve that maps multidimensional data to one dimension while maintaining some level of spatial locality. It is much simpler to implement than the Hilbert curve, as it is based on bit-interleaving of coordinates [Mor66]. The resulting path forms a “Z” shape at each scale, which is why it is commonly called Z-order. It is a fundamental technique for spatial indexing in computer graphics and databases [Sam84].

10.12.2 Definitions

Definition 10.62 (Bit-Interleaving). Creating a single number by alternating bits from two or more source numbers. For 2D coordinates, the i -th bit of the Morton code is formed by placing the i -th bit of y at position $2i + 1$ and the i -th bit of x at position $2i$.

Definition 10.63 (Z-Order Index). The 1D index (Morton code) resulting from bit-interleaving. For (x, y) with binary representations $x = (x_n \dots x_0)_2$ and $y = (y_n \dots y_0)_2$:

$$Z(x, y) = (y_n x_n y_{n-1} x_{n-1} \dots y_0 x_0)_2.$$

Definition 10.64 (Locality of Z-Order). The property where points close in space are also close on the 1D curve. Z-order preserves locality well, but not as consistently as the Hilbert curve—there are “jumps” at certain power-of-two boundaries where spatially distant cells receive consecutive indices [Sam84].

10.12.3 Theory

For 2D coordinates (x, y) , the Morton code is computed by taking the binary representation of x and y and interleaving them:

- $x = (x_n x_{n-1} \dots x_1 x_0)_2$
- $y = (y_n y_{n-1} \dots y_1 y_0)_2$
- $Z = (y_n x_n y_{n-1} x_{n-1} \dots y_0 x_0)_2$

This process is extremely fast because it can be implemented with simple bitwise shifts and masks. Geometrically, this recursively partitions the space into four quadrants, ordering them in a Z-pattern: $(0, 0) \rightarrow (1, 0) \rightarrow (0, 1) \rightarrow (1, 1)$.

Theorem 10.65 (Morton Code Uniqueness). *The bit-interleaving map $Z: \mathbb{Z}_{\geq 0}^2 \rightarrow \mathbb{Z}_{\geq 0}$ is a bijection. Every pair of non-negative integers (x, y) maps to a unique Morton code $Z(x, y)$, and the inverse map (de-interleaving) uniquely recovers (x, y) from Z .*

Proof. The map separates the even and odd bit positions: the even-position bits of Z are exactly the bits of x , and the odd-position bits are exactly the bits of y . Since the bit extraction operations (shifting and masking) are deterministic and invertible, the map is a bijection. Concretely, $x = Z \& 0x55555555$ and $y = (Z \gg 1) \& 0x55555555$ recover the original coordinates. $\square$

Example 10.66 (Morton Code for (3, 5)). Convert $(x, y) = (3, 5)$ to a Morton code. In binary: $x = (011)_2$ and $y = (101)_2$. Interleaving:

$$Z = (y_2 x_2 y_1 x_1 y_0 x_0)_2 = (101111)_2 = 47_{10}.$$

Verifying the inverse: extract even bits of $47 = (101111)_2$: positions 0, 2, 4 give $110_2 = 3 = x$. Extract odd bits (positions 1, 3, 5): $101_2 = 5 = y$. The mapping is invertible as guaranteed by Theorem 10.65 [Mor66].

5	6	9	10
4	7	8	11
3	2	13	12
0	1	14	15

Morton (Z-order) curve: bit-interleaving

Figure 10.4: Morton curve: the Z-shaped traversal is obtained by interleaving the binary representations of x and y .

10.12.4 Pseudocode

Algorithm 20 Morton Code: Coordinates to Index

```

function XYTOMORTON( $x, y$ )
     $morton\_code \leftarrow 0$  ▷ Process bits one by one
    for  $i$  in range( $MAX\_BITS$ ) do ▷ Extract  $i$ -th bit from  $x$  and  $y$ 
         $bit\_x \leftarrow (x \gg i) \& 1$ 
         $bit\_y \leftarrow (y \gg i) \& 1$ 
        ▷ Interleave bits:  $y_i$  at position  $2i + 1$ ,  $x_i$  at position  $2i$ 
         $morton\_code \leftarrow morton\_code | (bit\_x \ll (2 \cdot i))$ 
         $morton\_code \leftarrow morton\_code | (bit\_y \ll (2 \cdot i + 1))$ 
    end for
    return  $morton\_code$ 
end function

▷ Faster version using bit manipulation magic (for 16-bit  $x, y$ )
function INTERLEAVEBITS( $x$ )
     $x \leftarrow (x | (x \ll 8)) \& 0x00FF00FF$ 
     $x \leftarrow (x | (x \ll 4)) \& 0x0F0F0F0F$ 
     $x \leftarrow (x | (x \ll 2)) \& 0x33333333$ 
     $x \leftarrow (x | (x \ll 1)) \& 0x55555555$  return  $x$ 
end function

```

Algorithm 20 shows both the straightforward bit-by-bit interleaving and the optimized “bit manipulation magic” version that performs the interleave in $O(1)$ constant time using only shifts and masks [Mor66].

10.12.5 Parameter Selection

- **Bit Depth:** The number of bits used for each coordinate (e.g., 16 bits for a $2^{16} \times 2^{16}$ grid).
- **Interleaving Order:** Conventionally y bits are placed in higher positions, but x -first is also used.

10.12.6 Complexity

- **Transformation:** $O(1)$ for fixed-size integers using bit manipulation (or $O(n)$ where n is bits).
- **Space:** $O(1)$ auxiliary space.

10.12.7 Usages

- **Quadtrees/Octrees:** Morton codes can be used to uniquely represent nodes in a linear quadtree/octree, making them extremely fast to navigate [Sam84].
- **Texture Mapping:** Storing textures in Morton order (often called “swizzling”) significantly improves GPU texture cache hit rates by ensuring local pixels are stored close together in memory.
- **Spatial Databases:** Efficient range queries and nearest neighbor searches.
- **Video Compression:** Partitioning macroblocks in a locality-preserving order.
- **GPGPU Optimization:** Ordering parallel threads to match memory access patterns.

10.12.8 Testing Methods and Edge Cases

- **Invertibility:** Ensure bit-interleaving can be reversed (de-interleaving) to recover the original (x, y) (Theorem 10.65).
- **Boundary Points:** Test the mapping at $(0, 0)$ and the maximum possible coordinate (e.g., $(2^{16} - 1, 2^{16} - 1)$).
- **Adjacency:** Note that Z-order has “jumps” where sequential points on the curve are geographically distant. Verify that these occur only at specific power-of-two boundaries.
- **High Dimensions:** Test that the code works correctly for 3D (x, y, z) coordinates.

10.12.9 References

- Morton, G. M. (1966). “A computer-oriented geodetic data base and a new technique in file sequencing”. IBM Technical Report.
- Samet, H. (1984). “The Quadtree and Related Hierarchical Data Structures”. ACM Computing Surveys.
- Wikipedia: Z-order curve

10.13 Peano Curve

10.13.1 Overview

The Peano curve is the first known example of a space-filling curve, discovered by Giuseppe Peano in 1890 [Pea90]. It is a continuous mapping from a 1D unit interval to a 2D unit square. Unlike the Hilbert curve, which is based on a 2×2 subdivision, the Peano curve is based on a 3×3 subdivision. It is an important foundational structure in mathematical analysis and fractal geometry [Sag94].

10.13.2 Definitions

Definition 10.67 (Space-Filling Curve (Peano)). A continuous function $f: [0, 1] \rightarrow [0, 1]^2$ whose image is all of $[0, 1]^2$. The Peano curve was the first such function to be constructed, answering a question posed by Cantor about whether a continuous bijection between $[0, 1]$ and $[0, 1]^2$ exists [Pea90].

Definition 10.68 (Order (n) of the Peano Curve). The number of recursive subdivisions. An order- n Peano curve covers a grid of $3^n \times 3^n$ cells, visiting all 9^n cells exactly once.

Definition 10.69 (Locality of the Peano Curve). The property of mapping nearby 1D points to nearby 2D points. The Peano curve preserves locality, but with a different structure than the Hilbert curve due to its ternary subdivision [Sag94].

10.13.3 Theory

The Peano curve is constructed recursively using a 3×3 grid [Sag94].

1. For order $n = 1$, the space is divided into 9 squares.
2. The curve visits these squares in an S-like pattern, ensuring that it enters and exits each square in a way that allows for continuous connection between neighbors.
3. Each of the 9 squares is then recursively replaced by a smaller 3×3 Peano curve. Some of these sub-curves must be reflected or rotated to maintain the overall continuity.

Mathematically, the Peano curve is defined by a base-3 representation of coordinates. Interleaving these base-3 digits (trit) provides the 1D index, though the digits must be flipped for certain sub-quadrants to preserve the curve's continuity.

Theorem 10.70 (Peano Curve is Continuous and Surjective). *The Peano curve $f: [0, 1] \rightarrow [0, 1]^2$ is a continuous surjection: for every $(x, y) \in [0, 1]^2$, there exists $t \in [0, 1]$ such that $f(t) = (x, y)$. However, f is not injective: there exist points in $[0, 1]^2$ with multiple pre-images.*

Proof. Continuity follows from the uniform convergence of the sequence of piecewise-linear approximations f_n . At order n , the curve visits all 9^n cells in the $3^n \times 3^n$ grid, and consecutive cells share an edge. The maximum distance between $f_n(t)$ and $f_{n+1}(t)$ is $O(3^{-n})$, so $\{f_n\}$ converges uniformly. The uniform limit of continuous functions is continuous. Surjectivity follows because every cell in the grid is visited at every order, so every point in $[0, 1]^2$ is a limit of visited cells. Non-injectivity arises because the curve must “double back” at certain junction points to maintain continuity [Pea90, Sag94]. $\square$

Example 10.71 (Peano Curve of Order 1). An order-1 Peano curve visits all 9 cells in a 3×3 grid. A standard traversal in S-pattern order (with appropriate reflections) visits:

$$(0, 0) \rightarrow (1, 0) \rightarrow (2, 0) \rightarrow (2, 1) \rightarrow (1, 1) \rightarrow (0, 1) \rightarrow$$

$$(0, 2) \rightarrow (1, 2) \rightarrow (2, 2)$$

This forms a backward-S shape. For order 2, each of these 9 cells is replaced by a 3×3 sub-grid with appropriate reflections to maintain continuity at the junctions.

10.13.4 Pseudocode

Algorithm 21 Peano Curve: Coordinates to Index

```

function PEANOINDEX( $n, x, y$ )  ▷ Mapping coordinates  $(x, y)$  to a 1D index  ▷ Based on
base-3 representation
   $idx \leftarrow 0$ 
  for  $i$  in range( $n - 1, -1, -1$ ) do
     $trit\_x \leftarrow \lfloor x/3^i \rfloor \bmod 3$ 
     $trit\_y \leftarrow \lfloor y/3^i \rfloor \bmod 3$ 
    ▷ Handle reflections for even-numbered segments
    if  $x\_is\_reversed$  then
       $trit\_x \leftarrow 2 - trit\_x$ 
    end if
    if  $y\_is\_reversed$  then
       $trit\_y \leftarrow 2 - trit\_y$ 
    end if
    ▷ Segment index (0–8)
     $segment \leftarrow trit\_y \cdot 3 + trit\_x$ 
     $idx \leftarrow idx \cdot 9 + segment$ 
    ▷ Update reflection state for next level
    UpdateReflectionState( $trit\_x, trit\_y$ )
  end for
  return  $idx$ 
end function

```

10.13.5 Parameter Selection

- **Order (n):** Determines the resolution of the grid ($3^n \times 3^n$).
- **Grid Size:** Must be a power of 3.

10.13.6 Complexity

- **Transformation:** $O(n)$ base-3 digit operations.
- **Space:** $O(1)$ auxiliary space.

10.13.7 Usages

- **Theoretical Analysis:** A fundamental counter-example in topology and real analysis (showing a continuous mapping from 1D to 2D) [Pea90].
- **Image Processing:** Scanning images in a way that respects local structures, similar to Hilbert scans.
- **Data Organization:** Multidimensional indexing (though base-2 curves like Hilbert and Morton are much more common in digital hardware).
- **Fractal Design:** Used in generative art and architectural patterns.

10.13.8 Testing Methods and Edge Cases

- **Continuity:** Verify that for any index i , the Euclidean distance between `Peano(i)` and `Peano(i+1)` is exactly 1 unit in the grid.
- **Full Coverage:** Verify that every (x, y) coordinate in a $3^n \times 3^n$ grid is mapped to a unique index $0 \dots 9^n - 1$.
- **Boundary Cases:** Test coordinates $(0, 0)$ and the maximum coordinate $(3^n - 1, 3^n - 1)$.
- **Reflections:** Carefully test the “flipping” logic in the S-pattern to ensure the curve doesn’t have breaks.

10.13.9 References

- Peano, G. (1890). “Sur une courbe, qui remplit toute une aire plane”. *Mathematische Annalen*.
- Sagan, H. (1994). “Space-Filling Curves”. Universitext.
- Wikipedia: Peano curve

10.14 Zigzag Curve

10.14.1 Overview

The zigzag curve (or snake scan) is a simple space-filling curve that visits every cell in a 2D grid in a back-and-forth pattern. Unlike Hilbert or Morton curves, it does not have fractal properties and is not designed for recursive locality preservation. Instead, it is highly efficient for sequential data access in rectangular arrays and is a common technique in image and video compression [PM92].

10.14.2 Definitions

Definition 10.72 (Zigzag Scan). A path that traverses a grid row-by-row, alternating the direction of traversal for each row. Even-indexed rows are traversed left-to-right, and odd-indexed rows are traversed right-to-left.

Definition 10.73 (Locality of Zigzag Scan). Sequential points on a zigzag curve are geographically close within a row, but there are large jumps when moving between rows. The horizontal locality is perfect (adjacent indices map to adjacent cells), but vertical locality is poor compared to fractal curves.

Definition 10.74 (Raster Scan). A similar scan that always goes in the same direction (e.g., left-to-right). Zigzag is preferred in some cases because it maintains some horizontal locality at the row transitions, avoiding the large horizontal jumps that a raster scan has at each row boundary.

10.14.3 Theory

The zigzag scan is constructed by iterating through the grid row by row.

- For **even-indexed rows** $(0, 2, \dots)$, the curve moves from left to right.
- For **odd-indexed rows** $(1, 3, \dots)$, the curve moves from right to left.

In image compression (like JPEG), a more specific **diagonal zigzag scan** is used. This scan traverses an 8×8 block of frequency coefficients in order of increasing frequency. This ensures that the low-frequency coefficients (which carry most of the image information) are grouped together, making them easier to compress [PM92].

Theorem 10.75 (Zigzag Scan Covers All Cells). *For a $W \times H$ grid, the snake scan produces a path of length $W \cdot H$ that visits every cell (x, y) with $0 \leq x < W$ and $0 \leq y < H$ exactly once. Consecutive cells in the path are always Manhattan neighbors.*

Proof. The scan processes H rows, each containing W cells. For each row y , the cells are enumerated in order: if y is even, the cells $(0, y), (1, y), \dots, (W-1, y)$ are visited; if y is odd, the cells $(W-1, y), (W-2, y), \dots, (0, y)$ are visited. Within a row, consecutive cells differ by exactly 1 in the x -coordinate. Between rows, the last cell of row y and the first cell of row $y+1$ share the same x -coordinate (since row $y+1$ starts from the opposite side), so they are Manhattan neighbors. The total number of cells visited is $H \cdot W$, and since each (x, y) with $0 \leq x < W$, $0 \leq y < H$ appears in exactly one row, the coverage is complete and without repetition. $\square$

Example 10.76 (Snake Scan on a 4×3 Grid). A 4×3 grid traversed by snake scan:

$(0, 0) \rightarrow (1, 0) \rightarrow (2, 0) \rightarrow (3, 0) \rightarrow (3, 1) \rightarrow (2, 1) \rightarrow (1, 1) \rightarrow (0, 1) \rightarrow (0, 2) \rightarrow (1, 2) \rightarrow (2, 2) \rightarrow (3, 2)$

Row 0 (even): left to right. Row 1 (odd): right to left. Row 2 (even): left to right. All 12 cells are visited exactly once, and consecutive cells are Manhattan neighbors as guaranteed by Theorem 10.75.

10.14.4 Pseudocode

Standard Snake Scan

Algorithm 22 Snake Scan Traversal

```

function SNAKESCAN(width, height)
     $path \leftarrow []$ 
    for  $y$  in range(height) do
        if  $y \bmod 2 = 0$  then ▷ Even row: left to right
            for  $x$  in range(width) do
                 $path.append((x, y))$ 
            end for
        else ▷ Odd row: right to left
            for  $x$  in range(width - 1, -1, -1) do
                 $path.append((x, y))$ 
            end for
        end if
    end for return  $path$ 
end function

```

Diagonal Zigzag (JPEG style)

The diagonal zigzag (Algorithm 23) is particularly important in JPEG compression [PM92], where it orders DCT coefficients so that low-frequency entries come first, enabling efficient run-length encoding.

Algorithm 23 Diagonal Zigzag Scan (JPEG)

```
function DIAGONALZIGZAG(size)                                ▷ Scan a square block diagonally
    path ← []
    for s in range(2 · size − 1) do
        if s mod 2 = 0 then                                    ▷ Up-right diagonal
            for y in range(size) do
                x ← s − y
                if 0 ≤ x < size then
                    path.append((x, y))
                end if
            end for
        else                                                    ▷ Down-left diagonal
            for x in range(size) do
                y ← s − x
                if 0 ≤ y < size then
                    path.append((x, y))
                end if
            end for
        end if
    end for return path
end function
```

10.14.5 Parameter Selection

- **Direction:** Can be row-major (standard snake) or diagonal (compression).
- **Block Size:** For JPEG, the block size is fixed at 8×8 .

10.14.6 Complexity

- **Transformation:** $O(1)$ to compute a coordinate from an index (and vice versa) using modular arithmetic.
- **Space:** $O(1)$ auxiliary space.

10.14.7 Usages

- **Image/Video Compression (JPEG, MPEG):** Ordering DCT coefficients to group low-frequency data, which improves the efficiency of run-length encoding (RLE) [PM92].
- **Display Technology:** Scanning pixels in CRT or LCD screens.
- **Robotics/3D Printing:** Planning the path for a nozzle or sensor to cover a flat area (rasterizing a plane).
- **Sensor Networks:** Ordering data collection from a grid of sensors.

10.14.8 Testing Methods and Edge Cases

- **Full Coverage:** Verify that every (x, y) in the grid appears exactly once in the scan.
- **Continuity:** Verify that consecutive points in the path are Manhattan neighbors.
- **Non-Square Grids:** Ensure the algorithm handles $W \neq H$ correctly.
- **Zero/One dimensions:** Test on $1 \times N$ or $N \times 1$ grids.

10.14.9 References

- Pennebaker, W. B., & Mitchell, J. L. (1992). “JPEG: Still Image Data Compression Standard”. Van Nostrand Reinhold.
- Wikipedia: Zigzag

10.15 Spiral Walk

10.15.1 Overview

A spiral walk is a space-filling traversal that starts at a center point and visits cells in a 2D grid in an expanding spiral pattern. Unlike Hilbert or Morton curves, which are recursive and locality-preserving, the spiral walk is intuitive and ensures that the distance from the starting point increases monotonically (on average). It is used for localized searching, image scanning, and data visualization.

10.15.2 Definitions

Definition 10.77 (Archimedean Spiral). A continuous curve where the distance from the origin is proportional to the angle: $r = a + b\theta$. The discrete square spiral is an approximation to the Archimedean spiral on an integer grid.

Definition 10.78 (Square Spiral). A discrete version of the spiral restricted to a square grid. It visits cells in layers, where layer k consists of the $8k$ cells forming the perimeter of a $(2k + 1) \times (2k + 1)$ square centered at the origin.

Definition 10.79 (Radius (r) of a Spiral Walk). The maximum Manhattan or Chebyshev distance from the starting point during the walk. After visiting all cells within radius r , the total number of cells visited is $(2r + 1)^2$.

10.15.3 Theory

The square spiral walk is typically performed by moving in layers [SUW64].

1. **Layer 0:** The starting cell $(0, 0)$.
2. **Layer 1:** The 8 neighbors surrounding the center.
3. **Layer k :** The $8k$ cells that form the perimeter of a $(2k + 1) \times (2k + 1)$ square.

The walk follows a repeating pattern of directions (e.g., Right, Up, Left, Down) with increasing segment lengths. For example:

- Right 1, Up 1
- Left 2, Down 2
- Right 3, Up 3
- Left 4, Down 4
- ...

Theorem 10.80 (Spiral Walk Coverage). After completing layer k (i.e., after $N = (2k + 1)^2$ steps), the spiral walk has visited every cell (x, y) with $|x| \leq k$ and $|y| \leq k$. The total number of steps to cover a square of radius r is $(2r + 1)^2$.

Proof. Layer k adds the cells on the perimeter of the $(2k+1) \times (2k+1)$ square that are not already covered by layers $0, \dots, k-1$. The number of cells on this perimeter is $(2k+1)^2 - (2(k-1)+1)^2 = (2k+1)^2 - (2k-1)^2 = 8k$. By induction, after completing layer k , the walk has visited all cells within the $(2k+1) \times (2k+1)$ square, totaling $\sum_{j=0}^k 8j + 1 = (2k+1)^2$ cells [SUW64]. $\square$

Example 10.81 (First 25 Steps of a Square Spiral). The first 25 steps of the square spiral (covering a 5×5 square):

$(0, 0) \rightarrow (1, 0) \rightarrow (1, 1) \rightarrow (0, 1) \rightarrow (-1, 1) \rightarrow (-1, 0) \rightarrow (-1, -1) \rightarrow (0, -1) \rightarrow (1, -1) \rightarrow$
 $(2, -1) \rightarrow (2, 0) \rightarrow (2, 1) \rightarrow (2, 2) \rightarrow (1, 2) \rightarrow (0, 2) \rightarrow (-1, 2) \rightarrow (-2, 2) \rightarrow (-2, 1) \rightarrow$
 $(-2, 0) \rightarrow (-2, -1) \rightarrow (-2, -2) \rightarrow (-1, -2) \rightarrow (0, -2) \rightarrow (1, -2) \rightarrow (2, -2)$

After $25 = (2 \cdot 2 + 1)^2$ steps, all cells in the 5×5 square are covered, as guaranteed by Theorem 10.80.

10.15.4 Pseudocode

Algorithm 24 Square Spiral Walk

```

function SPIRALWALK(num_steps)
     $x, y \leftarrow 0, 0$ 
     $path \leftarrow [(x, y)]$ 
    ▷ Directions: Right, Up, Left, Down

     $dx \leftarrow [1, 0, -1, 0]$ 
     $dy \leftarrow [0, 1, 0, -1]$ 
     $step\_limit \leftarrow 1$ 
     $direction \leftarrow 0$ 
    while  $\text{len}(path) < num\_steps$  do ▷ Move in current direction for 'step_limit' steps
        for  $\_$  in  $\text{range}(2)$  do ▷ Two directions share the same limit
            for  $\_$  in  $\text{range}(step\_limit)$  do
                if  $\text{len}(path) \geq num\_steps$  then
                    break
                end if
                 $x \leftarrow x + dx[direction]$ 
                 $y \leftarrow y + dy[direction]$ 
                 $path.append((x, y))$ 
            end for
            ▷ Change direction
             $direction \leftarrow (direction + 1) \bmod 4$ 
        end for
        ▷ Increase segment length after every two turns
         $step\_limit \leftarrow step\_limit + 1$ 
    end while
    return  $path$ 
end function

```

10.15.5 Parameter Selection

- **Starting Direction:** Can be any of the four cardinal directions.
- **Orientation:** Clockwise vs. Counter-clockwise.
- **Step Size:** Standard is 1 unit, but can be larger for sparse sampling.

10.15.6 Complexity

- **Time Complexity:** $O(n)$ where n is the number of steps.
- **Space Complexity:** $O(n)$ to store the path coordinates.

10.15.7 Usages

- **Search Algorithms:** Finding the nearest point of interest starting from a query location (e.g., “nearest gas station” in a local grid).
- **Computer Vision:** Scanning a local neighborhood around a keypoint or feature.
- **Procedural Generation:** Growing a city or a structure from a central “seed” location.
- **Data Visualization:** Arranging objects around a center based on their priority or timestamp (e.g., a “spiral timeline”).
- **Cryptography:** Permuting bits or pixels in a non-linear but deterministic way.

10.15.8 Testing Methods and Edge Cases

- **Full Coverage:** Verify that every (x, y) coordinate within a k -layer square is visited exactly once.
- **Monotonicity:** Ensure that for any two sequential points P_i, P_{i+1} , their coordinates change by exactly 1 unit.
- **Large Step Count:** Verify the coordinates don’t drift or skip cells for $n > 10^6$.
- **Start at Edge:** Handle cases where the spiral is constrained within a bounding box and must stop at the edges.

10.15.9 References

- Stein, M. L., Ulam, S. M., & Wells, M. B. (1964). “A Visual Display of Some Properties of the Distribution of Primes”. The American Mathematical Monthly. (The Ulam Spiral).
- Gardner, M. (1971). “The Extraordinary Properties of the Mathematical Constant Pi”. Scientific American.
- Wikipedia: Spiral

Part IV

Proximity and Tessellations

Chapter 11

Voronoi Diagrams

11.1 Voronoi Diagram

A Voronoi diagram partitions the plane into regions based on distance to a set of sites. The cell of a site contains the points closer to it than to any other site. The Voronoi diagram is the geometric dual of the Delaunay triangulation [Vor08, Aur91].

11.1.1 Definitions

Definition 11.1 (Voronoi Cell). For sites $S = \{s_1, \dots, s_n\} \subset \mathbb{R}^2$, the Voronoi cell of s_i is

$$V(s_i) = \{p \in \mathbb{R}^2 \mid \|p - s_i\| \leq \|p - s_j\| \text{ for all } j \neq i\}.$$

Definition 11.2 (Voronoi Vertex). A Voronoi vertex is a point where three or more cells meet. It is equidistant from its closest sites and corresponds to the circumcenter of a Delaunay triangle.

11.1.2 Delaunay Duality

The duality is given by the following correspondences:

1. A Delaunay vertex corresponds to a Voronoi cell.
2. A Delaunay triangle corresponds to a Voronoi vertex.
3. A Delaunay edge corresponds to a Voronoi edge on a perpendicular bisector.

Theorem 11.3 (Duality of Delaunay and Voronoi Diagrams). *For sites in general position, the Delaunay triangulation and Voronoi diagram are planar duals and both have linear complexity.*

11.1.3 Pseudocode

11.1.4 Complexity and Applications

Construction through a Delaunay triangulation takes $O(n \log n)$ time and $O(n)$ space. A bounding box is required to cap cells belonging to sites on the convex hull. Applications include facility location, robot path planning, territory modeling, and nearest-neighbor search.

11.1.5 Testing and Edge Cases

Sites on the convex hull produce unbounded cells, co-circular sites produce non-unique vertices, collinear sites create degeneracies, and duplicate sites must be merged or rejected.

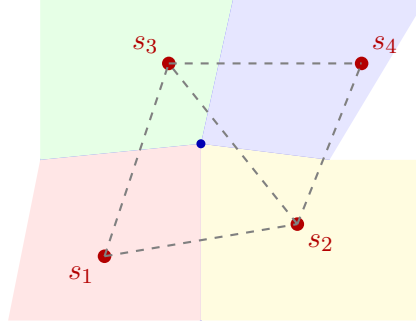


Figure 11.1: Voronoi diagram (colored cells) and its Delaunay triangulation dual (dashed gray edges). Each Voronoi vertex is the circumcenter of a Delaunay triangle.

Algorithm 25 Voronoi Diagram via Delaunay Dual

```

1: function COMPUTEVORONOI(sites, bounding_box)
2:    $DT \leftarrow \text{DelaunayTriangulation}(\text{sites})$ 
3:    $C \leftarrow \{\text{Circumcenter}(T) : T \in DT.\text{triangles}\}$ 
4:   for  $site$  in  $sites$  do
5:      $star \leftarrow \text{FindTrianglesSharingVertex}(site, DT)$ 
6:      $boundary \leftarrow \text{SortAround}(site, C[star])$ 
7:      $cell \leftarrow \text{ClipPolygon}(boundary, bounding\_box)$ 
8:     Store  $cell$ 
9:   end for
10:  return all stored cells
11: end function

```

11.2 Voronoi Diagrams of Line Segments

The generalized Voronoi diagram of a set of disjoint line segments $S = \{s_1, \dots, s_n\}$ partitions the plane according to the Euclidean distance to the closest point of each segment. Segment sites arise whenever features are modeled as one-dimensional objects: road networks, rivers, and walls in GIS, or the edges of polygonal obstacles in robot motion planning [LD81, OBSC00]. The medial axis of a polygonal domain is a subset of this diagram.

11.2.1 Definitions

Definition 11.4 (Distance to a Segment). The distance from a point p to a closed segment $s = \overline{ab}$ is

$$\text{dist}(p, s) = \min_{q \in s} \|p - q\|_2,$$

attained at the orthogonal projection of p onto the supporting line of s clamped to $[a, b]$; if the projection falls outside the segment, the minimum is attained at the nearer endpoint.

Definition 11.5 (Voronoi Cell of a Segment). The Voronoi cell of segment s_i is

$$V(s_i) = \{p \in \mathbb{R}^2 \mid \text{dist}(p, s_i) \leq \text{dist}(p, s_j) \text{ for all } j \neq i\}.$$

11.2.2 Theory

The bisector of two sites is the locus of points equidistant from them. For segment sites the bisector is composed of

- a **line** between two points (or between two parallel segment interiors),

- a **parabola** whose focus is a segment endpoint and whose directrix is the supporting line of the other segment, or
- an **angle bisector** between two non-parallel supporting lines.

Consequently the edges of a segment-site Voronoi diagram are straight segments or parabolic arcs, and its cells are star-shaped but not necessarily convex.

Theorem 11.6 (Complexity of the Segment Voronoi Diagram). *The Voronoi diagram of n disjoint line segments has $O(n)$ vertices, edges, and cells, and can be computed in $O(n \log n)$ time.*

Proof sketch. Each segment contributes a bounded number of features, and the planar structure of the diagram bounds the total complexity linearly. The $O(n \log n)$ construction time is achieved by Fortune’s sweep-line algorithm generalized to segment sites—the beach line then alternates between parabolic arcs (foci at segment endpoints) and line segments—and by divide-and-conquer constructions [For87, LD81]. $\square$

11.2.3 Pseudocode

Algorithm 26 Segment Voronoi Diagram by Plane Sweep

```

1: function SEGMENTVORONOI(segments)
2:    $Q \leftarrow$  site events (segment endpoints) and circle events
3:    $T \leftarrow$  balanced tree holding the beach line of lines and parabolas
4:   initialize output structure for each segment
5:   while  $Q$  is not empty do
6:      $e \leftarrow Q.\text{pop}()$ 
7:     if  $e$  is a site event then
8:       insert the new beach-line arc into  $T$ 
9:       detect and schedule new circle events
10:    else ▷ circle event: an arc disappears
11:      create a Voronoi vertex and its incident edges
12:      remove the arc and update circle events
13:    end if
14:  end while
15:  return edges and vertices, clipped to the bounding box
16: end function

```

11.2.4 Complexity and Applications

The diagram needs $O(n \log n)$ time and $O(n)$ space. Applications include

- the **medial axis** of a polygonal domain (a subset of the diagram),
- **nearest-obstacle** and clearance queries for a robot among polygonal obstacles, and
- **1-skeleton** neighborhood queries in road and river networks.

11.2.5 Testing and Edge Cases

Collinear overlapping segments, parallel segments (whose bisectors are parallel lines), shared endpoints, and segments touching the bounding box must be handled; predicates for parabolic bisectors need care near focus degeneracies.

11.3 Farthest-Point Voronoi Diagrams

The farthest-point Voronoi diagram assigns to each site the region of points for which that site is the *farthest* among all sites. It is the exact analogue of the ordinary Voronoi diagram with min replaced by max, and it is the key structure for computing the diameter and the smallest enclosing circle of a point set [SH75, OBSC00].

11.3.1 Definitions

Definition 11.7 (Farthest-Point Voronoi Cell). For sites $S = \{s_1, \dots, s_n\} \subset \mathbb{R}^2$, the farthest-point Voronoi cell of s_i is

$$V^*(s_i) = \{p \in \mathbb{R}^2 \mid \|p - s_i\| \geq \|p - s_j\| \text{ for all } j \neq i\}.$$

11.3.2 Theory

Theorem 11.8 (Structure of the Farthest-Point Voronoi Diagram). *A site has a non-empty farthest-point Voronoi cell if and only if it is a vertex of the convex hull of S . If the hull $\mathcal{H}(S)$ has m vertices, the farthest-point Voronoi diagram has m cells and $m - 2$ vertices, is a tree, and its edges lie on perpendicular bisectors of pairs of hull vertices.*

The diagram is the dual of the Delaunay triangulation of the hull vertices restricted to the upper side of the lifted paraboloid: a farthest-point Voronoi vertex is the center of a circle through three hull vertices that contains all of S . Both the nearest- and farthest-point diagrams can be obtained from the same sweep-line construction by duality [For87].

11.3.3 Applications: Diameter and Smallest Enclosing Circle

The principal use of the farthest-point Voronoi diagram is optimization over the point set:

- **Diameter:** the two points realizing the diameter of S are the endpoints of an edge of the farthest-point Delaunay diagram of the hull; the diameter is found in $O(n \log n)$ time by rotating calipers on the hull.
- **Smallest enclosing circle:** the center of the minimum-radius circle enclosing S is either a farthest-point Voronoi vertex (the center of a circle through three hull vertices that contains all points) or the midpoint of a diameter pair. Welzl's randomized algorithm [Wel91] solves the same problem directly in expected linear time.

11.3.4 Pseudocode

11.3.5 Complexity and Applications

The hull takes $O(n \log n)$ time; the farthest-point Voronoi diagram of the m hull vertices is a tree with m cells and takes $O(m)$ time and space once the hull is known. Scanning its vertices costs $O(m)$, for an overall $O(n \log n)$.

11.3.6 Testing and Edge Cases

Collinear point sets (the diagram degenerates to parallel slabs), co-circular hull vertices, and hulls with very few vertices (with two points every cell is a half-plane) are the main edge cases.

Algorithm 27 Smallest Enclosing Circle via Farthest-Point Voronoi

```

1: function SMALLESTENCLOSINGCIRCLE(points)
2:    $H \leftarrow \text{ConvexHull}(\text{points})$ 
3:    $FPV \leftarrow \text{FarthestPointVoronoi}(H)$ 
4:    $best \leftarrow$  circle through the two points of the diameter of  $H$ 
5:   for vertex  $v$  of  $FPV$  do
6:      $C \leftarrow$  circle centered at  $v$  through its three hull vertices
7:     if  $\text{radius}(C) < \text{radius}(best)$  and  $C$  encloses all points then
8:        $best \leftarrow C$ 
9:     end if
10:  end for
11:  return  $best$ 
12: end function

```

11.3.7 References

- Shamos, M. I. (1978). “Computational Geometry”. PhD thesis, Yale University.
- Fortune, S. (1987). “A sweepline algorithm for Voronoi diagrams.”
- Okabe, A., Boots, B., Sugihara, K., & Chiu, S. N. (2000). “Spatial Tessellations: Concepts and Applications of Voronoi Diagrams”. Wiley.

11.4 Kinetic Voronoi Diagrams

A kinetic Voronoi diagram maintains the diagram of moving sites. Its dual is maintained through a kinetic Delaunay triangulation. Certificates based on the in-circle predicate predict when an edge becomes invalid; the next event is then processed by an edge flip.

Algorithm 28 Kinetic Voronoi Maintenance

```

1: function MAINTAINKINETICVORONOI(points, trajectories)
2:    $DT \leftarrow \text{DelaunayTriangulation}(\text{points})$ 
3:    $Q \leftarrow \text{PriorityQueueOfCertificateFailures}(DT, \text{trajectories})$ 
4:   while  $Q$  is not empty do
5:      $event \leftarrow Q.\text{pop}()$ 
6:     Flip the invalid edge in  $DT$ 
7:     Update certificates for affected edges
8:   end while
9: end function

```

The event-driven structure processes $O(\log n)$ priority-queue work per event. Degenerate cases include zero-velocity sites, collisions, simultaneous co-circular events, and sites leaving the region of interest.

- Voronoi, G. (1908). “Nouvelles applications des paramètres continus.”
- Fortune, S. (1987). “A sweepline algorithm for Voronoi diagrams.”
- Aurenhammer, F. (1991). “Voronoi Diagrams: A Survey.”

11.5 Industrial Applications of Voronoi Diagrams

Voronoi diagrams appear throughout industry because the dual Delaunay triangulation (Section 12.1) and the nearest-site assignment they encode are the exact solution to many “assign each point to the closest resource” problems. The following are the principal industrial use cases.

- **Retail and Service-Area Analysis (GIS):** Retail chains, banks, and emergency services use the Voronoi diagram (“Thiessen polygons”) of existing sites to compute service territories and to find the location of a new facility that maximizes the underserved area—the site whose insertion grows the smallest uncovered Voronoi cell the most. Thiessen-polygon generation is a standard tool in commercial GIS—Esri ArcGIS (3D Analyst) and MapInfo—sold to thousands of planning departments [OBSC00].
- **Wireless and Cellular Network Planning:** The Voronoi cells of base stations or small cells are the coverage regions under nearest-site association; load balancing and cell planning are performed directly on these cells, and the Delaunay graph gives the interference and handover topology. Commercial RF-planning suites (Atoll, Planet) sold to carriers and network operators build on this decomposition [OBSC00].
- **Logistics and Field-Service Territory Design:** Voronoi territories of distribution centers or service technicians partition demand points so that every customer is served by the closest resource; commercial territory- and route-optimization products (for example, ORTEC and Descartes) ship Voronoi-based territory design, recomputed as fleet and demand change.
- **Reservoir Simulation Grids (PEBI/Voronoi):** The perpendicular-bisector (PEBI) simulation grids of the oil and gas industry are Voronoi diagrams whose cells honor wells, faults, and fracture networks. Commercial reservoir-modeling software such as Roxar RMS (Emerson) sells this gridding as part of its seismic-to-simulation workflow.
- **Exact Nearest-Site Queries in Geodata Stacks:** Databases answer “nearest X to each location” exactly by materializing Voronoi cells: PostGIS (ST_VoronoiPolygons, via GEOS) exposes this in the SQL stacks of commercial geodata platforms and location services.

11.5.1 Industrial-Scale Software

Production implementations of the Voronoi diagram and its Delaunay dual are available in CGAL (2D and 3D Voronoi), `scipy.spatial` (Voronoi, Delaunay), `Boost.Polygon`, `voro++` (3D parallel Voronoi for molecular and physics simulations), and PostGIS (ST_VoronoiPolygons for spatial databases) [Aur91, OBSC00].

Chapter 12

Delaunay Triangulations

- 12.1 Voronoi–Delaunay Duality
- 12.2 Empty-Circumcircle Property
- 12.3 Local Delaunay Criterion
- 12.4 Edge Flipping
- 12.5 Incremental Construction
- 12.6 Randomized Incremental Algorithm
- 12.7 Divide-and-Conquer Construction
- 12.8 Lifting to a Paraboloid
- 12.9 Relationship to Convex Hulls
- 12.10 Constrained Delaunay Triangulation
- 12.11 Quality Properties
- 12.12 Delaunay Refinement
- 12.13 Higher-Dimensional Delaunay Complexes
- 12.14 Intrinsic Delaunay Triangulation

The Euclidean Delaunay triangulation is defined through empty circumcircles, a notion that depends on the ambient metric of $\mathbb{R}^2$. On a polyhedral surface $\mathcal{M}$ triangulated by triangles that do not lie in a common plane, this extrinsic test is meaningless: the two triangles sharing an edge lie in different tangent planes, so their circumcircles (as circles in $\mathbb{R}^3$) cannot be compared. An *intrinsic Delaunay triangulation* instead uses only the intrinsic metric of the surface, i.e., the edge lengths of the mesh, to decide legality. This section follows the flip-based construction of Bobenko and Springborn [BS07] and its practical implementation by Fisher et al. [FSBS07].

Definition 12.1 (Intrinsic Delaunay Edge). Let $e = (i, j)$ be an interior edge of a triangulated polyhedral surface, shared by triangles T_1 and T_2 , and let α_e and β_e be the angles opposite e in T_1 and T_2 , respectively. The edge e is *intrinsically Delaunay* if

$$\alpha_e + \beta_e \leq \pi.$$

A triangulation is *intrinsic Delaunay* if every interior edge is intrinsically Delaunay.

Because each triangle of the surface is intrinsically a Euclidean triangle, the angles α_e and β_e are computed from the three edge lengths of the respective triangles using the law of cosines. The criterion therefore depends only on the intrinsic metric, never on how the triangles are embedded in $\mathbb{R}^3$.

Definition 12.2 (Cotangent Weight). For an interior edge e , the *cotangent weight* is $w_e = \cot \alpha_e + \cot \beta_e$, where α_e, β_e are the angles opposite e . The identity

$$\cot \alpha_e + \cot \beta_e = \frac{\sin(\alpha_e + \beta_e)}{\sin \alpha_e \sin \beta_e}$$

shows that $w_e \geq 0$ if and only if $\alpha_e + \beta_e \leq \pi$. Thus a triangulation is intrinsic Delaunay if and only if all cotangent weights are non-negative.

The connection to the Laplace–Beltrami operator motivates the definition. The cotangent Laplacian of the mesh has off-diagonal entries $-\frac{1}{2}w_e$ and diagonal entries summing to zero; it is positive semi-definite exactly when the triangulation is intrinsic Delaunay. Negative weights, which can appear in an ordinary triangulation containing obtuse triangles, are responsible for spurious oscillations in discrete harmonic functions and for unstable heat propagation. Intrinsic Delaunay re-triangulation removes this defect at no cost in resolution.

12.14.1 Algorithm

The construction proceeds by flipping illegal edges exactly as in the planar case (Section 7.5), but with two differences:

1. The legality test uses the intrinsic angle sum $\alpha_e + \beta_e$ rather than the planar in-circle predicate.
2. After a flip, the new diagonal e' of the quadrilateral must be given an intrinsic length. The two triangles sharing e are, intrinsically, two Euclidean triangles glued along e ; unfolding them into a common plane yields a flat (possibly non-convex) quadrilateral, and the new edge length is the length of the other diagonal of this quadrilateral, computed from the four known edge lengths.

Theorem 12.3 (Existence of Intrinsic Delaunay Triangulation). *Every triangulation of a polyhedral surface admits an intrinsic Delaunay re-triangulation of the same vertices, obtainable by a finite sequence of edge flips [BS07, FSBS07].*

Proof sketch. Each flip replaces an edge whose opposite angles sum to more than π by the other diagonal. An angle-sum decrease argument shows that every flip strictly decreases a global potential over the set of triangulations of the intrinsic metric; since the set of triangulations of a fixed vertex set is finite, the process terminates. When the intrinsic Delaunay condition determines every diagonal uniquely, the resulting triangulation is unique; otherwise ties are broken consistently, for example lexicographically over edges. $\square$

Remark 12.4. The intrinsic Delaunay triangulation of a mesh is the dual of the *geodesic Voronoi diagram* of its vertices on the surface, just as the planar Delaunay triangulation is the dual of the planar Voronoi diagram. This explains why the intrinsic construction is the “correct” generalization for surface processing.

Algorithm 29 Intrinsic Delaunay Flip Algorithm

```

1: function INTRINSICDELAUNAY(triangulation)
2:   queue  $\leftarrow$  all interior edges of triangulation
3:   while queue is not empty do
4:      $e \leftarrow \text{queue.pop}()$ 
5:      $(\alpha_e, \beta_e) \leftarrow$  angles opposite  $e$  in its two incident triangles
6:     if  $\alpha_e + \beta_e > \pi$  then
7:        $e' \leftarrow$  other diagonal of the quadrilateral formed by the two triangles
8:        $\ell(e') \leftarrow$  intrinsic length of  $e'$  from the unfolded quadrilateral
9:       flip  $e$  to  $e'$ 
10:      for  $f$  in the two newly created edges do
11:        push(queue,  $f$ )
12:      end for
13:    end if
14:  end while
15:  return triangulation
16: end function

```

12.14.2 Properties

- **Metric invariance:** The result depends only on edge lengths, so it is invariant under isometric deformations of the surface (bending without stretching).
- **Non-negative weights:** All cotangent weights are non-negative, making the cotangent Laplacian positive semi-definite and well-behaved for PDEs, harmonic maps, and the heat method.
- **Same vertex set:** The algorithm changes only connectivity, never vertex positions; it is a pure re-tiling of the intrinsic metric.
- **Compatibility:** The intrinsic flip criterion coincides with the planar one on flat regions of the mesh, so the algorithm reproduces the usual Delaunay triangulation on planar surfaces.

12.14.3 Complexity

- **Time:** Each flip is $O(1)$; the total number of flips is $O(n^2)$ in the worst case and $O(n)$ in typical cases for meshes with reasonable initial quality [FSBS07].
- **Space:** $O(n)$ for the mesh and the queue of candidate edges.

12.14.4 Applications

- **Discrete Laplace–Beltrami:** Guaranteeing positive semi-definiteness of the cotangent operator for spectral analysis and shape signature computation.
- **Heat methods:** Stabilizing the heat kernel and geodesic-distance approximation on meshes with obtuse triangles.
- **Harmonic and biharmonic fields:** Removing spurious extrema caused by negative cotangent weights in, e.g., parameterization and deformation [BS07].
- **Discrete exterior calculus:** Restoring the non-negativity of the Hodge-star edge weights discussed in the discrete exterior calculus chapter (Section 30.17).

12.15 Applications to Mesh Generation

Delaunay meshes are the workhorse of industrial mesh generation because the empty-circumcircle (empty-circumsphere in 3D) property and the associated refinement algorithms of Section 12.12 produce element-quality guarantees that are provably unobtainable with other triangulation schemes. The following are the principal industrial use cases.

- **Finite Element Analysis (FEM):** Delaunay meshes with the refinement guarantees of Section 12.12 bound the minimum angle of every triangle away from zero, which is necessary for the conditioning of the stiffness matrix and the convergence of FEM solvers in structural and thermal analysis. The commercial FEA market—**ANSYS**, **Abaqus** (Dassault Systèmes), **COMSOL**, and **Altair**—sells solvers whose mesh generators are exactly this refinement scheme [She98].
- **Computational Fluid Dynamics (CFD):** Commercial CFD meshers (**ANSYS** Fluent meshing, **SIMULIA**, **STAR-CCM+**, and **Pointwise**) generate quality tetrahedral meshes from CAD geometry; the empty-sphere property keeps high-aspect-ratio elements near the anisotropic inflation layers while preserving the angle bounds that stabilize pressure–velocity coupling on aircraft, turbine, and vehicle models.
- **Reservoir Simulation (Voronoi/PEBI Simulation Grids):** The perpendicular-bisector (PEBI) grids used for reservoir flow simulation are Voronoi diagrams, and their construction is driven by the dual Delaunay triangulation. Commercial reservoir-modeling suites such as **Roxar RMS** (Emerson) build unstructured simulation grids whose cells honor wells, faults, and fracture networks, on which multiphase flow and transport solvers run.
- **Medical Imaging and Surgical Planning:** Delaunay tetrahedralization of segmented CT and MRI volumes produces the volumetric meshes used in finite-element modeling of bone, soft tissue, and blood flow. Commercial planning platforms such as **Materialise Mimics** sell patient-specific modeling for surgical guides, implants, and biomechanical simulation.
- **Reverse Engineering and Metrology:** Surface reconstruction from laser-scan point clouds builds a Delaunay (or alpha-shape, Section 12.13) complex of the scanned part. Commercial scanning pipelines such as **PolyWorks** (InnovMetric) and **Geomagic** (3D Systems) ship this reconstruction and the resulting mesh to downstream CAD and inspection.
- **Geographic Information Systems (GIS):** Conforming Delaunay triangulations of terrain point clouds produce the triangulated irregular networks (TINs) that back digital elevation models, visibility analysis, and contouring in **Esri ArcGIS 3D Analyst** and comparable commercial GIS; the conforming refinement also generates the meshes for watershed and flood modeling.
- **Shape and Topology Optimization:** The conforming mesh produced by Delaunay refinement with sizing fields adapts element density to the stress field, accelerating the density-based optimization loop in commercial tools such as **Altair OptiStruct**, **SIMULIA Tosca**, and **nTopology** for automotive and aerospace lightweighting.

12.15.1 Industrial-Scale Software

The algorithms of this chapter are implemented at production scale in the open-source meshers **TetGen** (tetrahedralization and constrained Delaunay), **CGAL** (2D and 3D Delaunay and regular triangulations), **Triangle** (2D CDT and refinement, embedded under license in commercial GIS and numerical products), **Gmsh** (unstructured mesh generation), and **MMG** (remeshing and

metric adaptation), as well as in commercial tools such as **ANSYS**, **COMSOL**, and **Pointwise** for CFD. Their shared core is the incremental insertion and flip algorithm of Sections [12.5](#) and [12.4](#), coupled with the quality refinement guarantees of Section [12.12](#).

12.16 Exercises

12.17 Project: Delaunay Triangulation from Scratch

Chapter 13

Proximity Problems

13.1 Closest Pair of Points

The closest pair of points problem is a fundamental challenge in computational geometry that involves finding the two points in a set of n points that have the minimum Euclidean distance between them. While a naive brute-force approach takes $O(n^2)$ time, more efficient algorithms exist, including a deterministic $O(n \log n)$ divide-and-conquer approach by Shamos and Hoey [SH75] and Bentley and Shamos [BS76], and a randomized $O(n)$ grid-based approach.

13.1.1 Definitions

Definition 13.1 (Closest Pair). Given a set P of n points in $\mathbb{R}^2$, the *closest pair* is the pair (p_i, p_j) with $i \neq j$ that minimizes the Euclidean distance $\|p_i - p_j\|$.

Definition 13.2 (Euclidean Distance). For two points $P_1(x_1, y_1)$ and $P_2(x_2, y_2)$, the *Euclidean distance* is $d(P_1, P_2) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$.

Definition 13.3 (Divide and Conquer Strategy). A *divide-and-conquer* strategy recursively partitions the input into two halves, solves each half independently, and merges the results. For the closest pair problem, the merge step requires examining only a narrow strip of width $2d$ centered on the dividing line, where $d = \min(d_L, d_R)$.

13.1.2 Theory

Divide and Conquer Approach

The set of points is first sorted by x -coordinate and then split into two halves by a vertical line. The minimum distance is recursively found in the left half (d_L) and the right half (d_R). Let $d = \min(d_L, d_R)$. The global minimum could still be a pair of points where one is in the left half and one is in the right half. To check this, we only need to consider points within a vertical “strip” of width $2d$ centered on the dividing line.

Within this strip, for any point P , we only need to check points Q that are within distance d of P . If the strip points are sorted by y -coordinate, it can be proven that each point only needs to be compared with a constant number (at most 7) of succeeding points in the sorted list to find a pair closer than d .

Theorem 13.4 (Strip Lemma). *Let S be a set of points in a $2d \times d$ rectangle, sorted by y -coordinate. For each point $p \in S$, at most 7 other points in S can lie within distance d of p .*

Proof. Divide the $2d \times d$ rectangle into 8 squares of side $d/2$. By the pigeonhole principle, if two points lie in the same square, their distance is at most $\frac{d}{2}\sqrt{2} < d$. However, each square can

contain at most one point from a pair with mutual distance $\geq d$, giving at most 8 points total (including p). Thus p needs to compare with at most 7 others. $\square$

Theorem 13.5 (Closest Pair Correctness). *The divide-and-conquer closest pair algorithm correctly finds the closest pair of n points in $\mathbb{R}^2$.*

Proof. By induction on n . The base cases ($n \leq 3$) are solved by brute force. For the recursive case, the minimum distance is either entirely within the left half, entirely within the right half, or crosses the dividing line. The recursive calls find d_L and d_R , and $d = \min(d_L, d_R)$. By the Strip Lemma (Theorem 13.4), only points within the strip need to be checked for a closer cross-pair, and the constant-bounded comparisons ensure no cross-pair closer than d is missed. $\square$

Randomized Grid Approach

This approach uses a grid with cell size $d \times d$. Each cell in the grid contains at most one point (if d is the current minimum distance). When a new point is added, we check its own cell and the 8 neighboring cells. If a closer point is found, the minimum distance d is updated, and the grid is rebuilt with the new cell size. In the randomized version, the expected number of rebuilds is $O(\log n)$, leading to an overall expected time of $O(n)$.

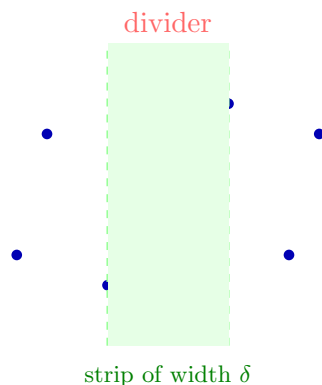


Figure 13.1: Closest pair: divide-and-conquer splits points along a vertical line. The closest pair may cross the dividing strip.

13.1.3 Pseudocode

13.1.4 Parameters

- **Algorithm Choice:** Use D&C for general-purpose deterministic results. Use Grid-based for massive streaming datasets where $O(n \log n)$ is too slow.
- **Base Case:** Brute force for $n < 4$ to avoid excessive recursion overhead.

13.1.5 Complexity

- **Divide and Conquer:** $O(n \log n)$ time, $O(n)$ space.
- **Grid-based:** $O(n)$ expected time, $O(n)$ space.

Example 13.6 (Closest Pair by Hand). Consider $P = \{(0, 0), (1, 3), (2, 2), (4, 1), (5, 4)\}$, sorted by x .

1. Split at $x = 2$: left $L = \{(0, 0), (1, 3), (2, 2)\}$, right $R = \{(4, 1), (5, 4)\}$.

Algorithm 30 Divide-and-Conquer Closest Pair

```

1: function CLOSESTPAIR(points_sorted_x, points_sorted_y)
2:   if len(points_sorted_x) ≤ 3 then
3:     return BruteForce(points_sorted_x)
4:   end if
5:   mid ← len(points_sorted_x) // 2
6:   left_x ← points_sorted_x[: mid]
7:   right_x ← points_sorted_x[mid :]
8:   (d_L, p1_L, p2_L) ← ClosestPair(left_x, points_sorted_y_left)
9:   (d_R, p1_R, p2_R) ← ClosestPair(right_x, points_sorted_y_right)
10:  d ← min(d_L, d_R)
11:  best_pair ← (p1_L, p2_L) if d_L < d_R else (p1_R, p2_R)
12:  strip ← [p | p ∈ points_sorted_y ∧ |p.x − points_sorted_x[mid].x| < d]
13:  for i = 0 to len(strip) − 1 do
14:    for j = i + 1 to min(i + 8, len(strip)) − 1 do
15:      if dist(strip[i], strip[j]) < d then
16:        d ← dist(strip[i], strip[j])
17:        best_pair ← (strip[i], strip[j])
18:      end if
19:    end for
20:  end for
21:  return (d, best_pair)
22: end function

```

2. Recurse on *L*: brute force gives $d_L = \|(1, 3) - (2, 2)\| = \sqrt{2}$.
3. Recurse on *R*: brute force gives $d_R = \|(4, 1) - (5, 4)\| = \sqrt{10}$.
4. $d = \sqrt{2}$. Strip: points with $|x - 2| < \sqrt{2}$, i.e., $x \in (0.586, 3.414)$: $\{(1, 3), (2, 2)\}$.
5. Check strip pairs: only $(1, 3)$ and $(2, 2)$ at distance $\sqrt{2}$: no improvement.
6. Result: closest pair is $(1, 3)$ and $(2, 2)$ with distance $\sqrt{2} \approx 1.414$.

13.1.6 Usages

- **Collision Detection:** Finding the two closest objects in a physics simulation.
- **Cluster Analysis:** Identifying tight groupings of data points.
- **Air Traffic Control:** Detecting potential collisions between aircraft.
- **Bioinformatics:** Comparing molecular structures or sequences.

13.1.7 Testing and Edge Cases

- **Identical Points:** Points with the exact same coordinates (distance 0).
- **Vertical/Horizontal alignment:** Points forming a perfect grid or line.
- **Large Coordinates:** Precision issues with squared distances $(x^2 + y^2)$ for very large x, y .
- **Sparse vs. Dense:** Performance on points scattered far apart vs. points clustered tightly together.
- **Verification:** Always compare with an $O(n^2)$ brute-force implementation for small inputs during testing.

13.1.8 References

- Shamos, M. I., & Hoey, D. (1975). “Closest-point problems”. IEEE FOCS.
- Khuller, S., & Matias, Y. (1995). “A simple randomized sieve algorithm for the closest-pair problem”. Information and Computation.
- Bentley, J. L., & Shamos, M. I. (1976). “Divide-and-conquer in multidimensional space”. ACM STOC.
- Wikipedia: Closest pair of points problem.

13.2 Proximity Queries

Proximity queries are a class of geometric problems focused on finding the spatial relationships between objects, specifically their distances and relative positions. These queries answer questions like “Which object is closest to this point?”, “Are these two objects intersecting?”, or “How far apart are these two shapes?” Proximity queries are the foundation of collision detection, spatial data mining, and robotics.

13.2.1 Definitions

Definition 13.7 (Nearest Neighbor). Given a set $P \subseteq \mathbb{R}^d$ and a query point q , the *nearest neighbor* of q in P is the point $p^* \in P$ minimizing $\|q - p^*\|$.

Definition 13.8 (K-Nearest Neighbors). The *k-nearest neighbors* of a query point q in a set P are the k points of P closest to q , ordered by distance.

13.2.2 Theory

Proximity queries are categorized by the type of geometric objects involved:

1. **Point-Point**: Handled using **KD-Trees** [Ben75] or **Quadtrees** for efficient $O(\log n)$ retrieval.
2. **Point-Mesh**: Finding the closest point on a surface. This involves searching a spatial index (like an **AABB Tree** or **BVH**) and calculating the distance to each triangle in the candidate leaves.
3. **Mesh-Mesh**: Calculating the distance between two 3D models. The **GJK (Gilbert-Johnson-Keerthi)** algorithm is the standard for convex shapes, while BVH hierarchies are used for general non-convex meshes.
4. **All-Pairs**: Finding all pairs of objects within a distance r . This is solved using **Spatial Hashing** or sweep-line algorithms.

The efficiency of these queries relies heavily on **Spatial Partitioning** (dividing the space) or **Bounding Volume Hierarchies** (grouping the objects).

Theorem 13.9 (KD-Tree Nearest Neighbor). A KD-tree built on n points in $\mathbb{R}^d$ supports nearest neighbor queries in $O(n^{1-1/d})$ worst-case time.

Algorithm 31 BVH Distance Query

```

1: function DISTANCE(nodeA, nodeB, current_min)
2:   if BoxDistance(nodeA.bbox, nodeB.bbox)  $\geq$  current_min then
3:     return current_min
4:   end if
5:   if nodeA.is_leaf and nodeB.is_leaf then
6:     for primA in nodeA.primitives do
7:       for primB in nodeB.primitives do
8:          $d \leftarrow$  PrimitiveDistance(primA, primB)
9:          $current\_min \leftarrow \min(current\_min, d)$ 
10:      end for
11:    end for
12:    return current_min
13:  end if
14:  if nodeA.volume  $>$  nodeB.volume then
15:    for childA in nodeA.children do
16:       $current\_min \leftarrow$  Distance(childA, nodeB, current_min)
17:    end for
18:  else
19:    for childB in nodeB.children do
20:       $current\_min \leftarrow$  Distance(nodeA, childB, current_min)
21:    end for
22:  end if
23:  return current_min
24: end function

```

13.2.3 Pseudocode**13.2.4 Parameters**

- **Bounding Volume Type:** **AABB** (fast intersection), **OBB** (tighter fit), or **Bounding Spheres** (rotation invariant).
- **Spatial Index:** Use **KD-Trees** for static points and **R-Trees** or **BVHs** for moving objects.

13.2.5 Complexity

- **Point NN:** $O(\log n)$ average using KD-Trees. $O(n^{1-1/d})$ worst case in d dimensions [Ben75].
- **Closest Pair:** $O(n \log n)$ deterministic or $O(n)$ randomized.
- **Collision Detection:** $O(n \log n)$ for broad-phase, $O(1)$ for narrow-phase (convex primitives).

13.2.6 Usages

- **Physics Engines:** Real-time collision detection in video games (e.g., PhysX, Bullet).
- **Machine Learning:** Clustering (K-Means) and classification (KNN).
- **Robotics:** Obstacle avoidance and sensor fusion.
- **Molecular Modeling:** Detecting van der Waals overlaps between atoms.
- **GIS:** Finding the nearest points of interest (e.g., hospitals or fire stations).

13.2.7 Testing and Edge Cases

- **Identical Objects:** Distance should be zero.
- **Touching Objects:** Distance should be zero or very small.
- **Nested Objects:** Ensure the algorithm detects if one object is entirely inside another.
- **Large Aspect Ratios:** Test with very long and thin objects which can “leak” through poorly fitted bounding volumes.
- **Precision:** Verify distances for objects separated by many orders of magnitude.

13.2.8 References

- Gilbert, E. G., Johnson, D. W., & Keerthi, S. S. (1988). “A fast procedure for computing the distance between complex objects in three-dimensional space”. *IEEE Journal on Robotics and Automation*.
- Ericson, C. (2004). “Real-Time Collision Detection”. Morgan Kaufmann.
- Samet, H. (2006). “Foundations of Multidimensional and Metric Data Structures”. Elsevier.
- Wikipedia: Nearest neighbor search.

13.3 Industrial Applications of Nearest Neighbor Search

Nearest neighbor search (NNS)—finding the stored point closest to a query point, or its k -nearest generalization of Definition 13.8—is the most frequently used geometric primitive in industry because a surprising number of problems reduce to “find the most similar known item.” The data structures of Sections 10.1 and 10.7 (and, for high dimensions, locality-sensitive hashing) make it practical at billions of points [Ben75, Sam06]. The following are the principal industrial use cases.

- **Recommendation Systems:** Annoy was built at Spotify specifically for music recommendations—millions of tracks and users in a low-dimensional embedding space, queried for similar items—and the same nearest-neighbor formulation powers the recommendation engines of the largest web platforms.
- **Semantic Search and Vector Databases:** Embedding-based semantic search is served by approximate nearest-neighbor (ANNS) indexes (HNSW, FAISS, DiskANN) at the core of commercial vector databases (Pinecone, Weaviate, Qdrant, Milvus, Elasticsearch kNN), a market sold to enterprise search, retrieval-augmented generation, and chatbot deployments [MY20].
- **Reverse Image and Visual Search:** Content-based retrieval encodes images (or frames) into embedding vectors and returns the k most visually similar items by nearest-neighbor search in the embedding space, powering reverse image search, product visual search, and duplicate detection in retail and content platforms.
- **Geospatial Services:** Mapping, ride-hailing, and navigation apps answer “closest restaurant” or “nearest pickup point” through spatial nearest-neighbor queries over point-of-interest databases (Section 10.1), a query type materialized at billion-query scale by location platforms.

- **Autonomous Driving and Robotics:** Nearest-neighbor search over a precomputed point cloud aligns sensor scans (point-cloud registration, ICP), matches features for SLAM loop closure, and finds the nearest obstacle for collision checking and path planning in commercial autonomous-driving and robotics stacks.
- **Drug Discovery and Molecular Similarity:** Molecular similarity by fingerprints or descriptors is a nearest-neighbor query over compound databases, used to find the closest known active molecule to a candidate drug; structure- and ligand-based virtual screening is sold commercially by computational-chemistry vendors.
- **Fraud Detection and Anomaly Detection:** Transactions embedded into a feature space are flagged when their nearest neighbors are predominantly fraudulent or when the distance to all known-good neighbors exceeds a learned threshold—an anomaly-detection formulation of NNS shipped in commercial fraud and security analytics platforms.
- **Bioinformatics and Genomics:** DNA and protein sequences compared by edit distance use nearest-neighbor search over a string-space index to find the closest gene, detect homology, and cluster metagenomic samples, powering commercial sequencing-analysis pipelines.
- **Pattern Recognition and Classification:** The k -nearest neighbors classifier (k -NN) labels a query point by the majority class of its k nearest training points; it remains a production baseline in biometric identification, OCR, and spam filtering.

13.3.1 Industrial-Scale Software

Exact and approximate NNS is implemented in `scipy.spatial.cKDTree` (KD-tree), `FLANN` and `FAISS` (approximate, billion-scale), `HNSWlib` (graph-based ANNS), `Annoy` (random-projection forests), and `PostGIS` (spatial nearest-neighbor SQL), in addition to the KD-tree and fixed-radius structures of Sections 10.1 and 10.7 [MY20].

13.4 Range Search

Range searching is a fundamental task in computational geometry where the goal is to efficiently retrieve all geometric objects (typically points) that lie within a specified query region (the “range”). Query regions are usually simple shapes like axis-aligned rectangles (orthogonal range search), circles, or half-planes. Efficient range searching is the backbone of spatial databases, geographic information systems (GIS), and computer graphics. Foundational work on range search data structures was done by Bentley [Ben75] and Lueker [Lue78].

13.4.1 Definitions

Definition 13.10 (Range Search). Given a set P of n points in $\mathbb{R}^d$ and a query region R , the *range search* problem asks to either report all points $p \in P \cap R$ (range reporting) or count $|P \cap R|$ (range counting).

Definition 13.11 (Orthogonal Range Query). An *orthogonal range query* is a range search where the query region is an axis-aligned hyper-rectangle $R = [x_1, x_2] \times [y_1, y_2] \times \dots$.

13.4.2 Theory

The efficiency of range search depends on the dimensionality and the type of range.

1. **1D Range Search:** Solved using a balanced binary search tree in $O(\log n + k)$ time.

2. Orthogonal Range Search (d -dimensions):

- **KD-Trees:** Good for medium dimensions; $O(\sqrt{n} + k)$ query time in 2D.
 - **Range Trees:** Faster query time $O(\log^d n + k)$ but higher space $O(n \log^{d-1} n)$.
 - **Quadrees:** Effective for non-uniform data distributions.
3. **Circular/Spherical Range Search:** Often handled using KD-trees with a distance-based pruning condition or specialized structures like **Ball Trees**.
4. **Non-Orthogonal Range Search:** General shapes can be approximated by rectangles or handled using more complex structures like **Simplex Range Reporting**.

The fundamental tradeoff in range search is between **Query Time**, **Preprocessing Time**, and **Space Complexity**.

Theorem 13.12 (Range Tree Query Time). *A range tree on n points in $\mathbb{R}^d$ uses $O(n \log^{d-1} n)$ space and supports orthogonal range reporting queries in $O(\log^d n + k)$ time, where k is the number of reported points.*

13.4.3 Pseudocode

Algorithm 32 Orthogonal Range Search (Tree-based)

```

1: function RANGESEARCH(node, query_range, results)
2:   if node is None then
3:     return
4:   end if
5:   if  $\neg$ Intersect(node.bbox, query_range) then
6:     return
7:   end if
8:   if FullyContains(query_range, node.bbox) then
9:     results.extend(node.all_points)
10:    return
11:  end if
12:  if node.is_leaf then
13:    for  $p$  in node.points do
14:      if query_range.contains( $p$ ) then
15:        results.append( $p$ )
16:      end if
17:    end for
18:  else
19:    for child in node.children do
20:      RangeSearch(child, query_range, results)
21:    end for
22:  end if
23: end function

```

13.4.4 Parameters

- **Dimensionality:** For $d > 20$, most spatial data structures degrade to linear search (the “Curse of Dimensionality”).

- **Data Distribution:** Use **Quadtrees** or **R-Trees** for highly clustered data; use **KD-Trees** or **Range Trees** for more uniform data.
- **Memory:** If memory is tight, KD-trees are preferred ($O(n)$ space).

13.4.5 Complexity Summary

Structure	Space	Query (2D Reporting)
KD-Tree	$O(n)$	$O(\sqrt{n} + k)$
Range Tree	$O(n \log n)$	$O(\log^2 n + k)$
Quadtree	$O(\text{depth} \cdot n)$	$O(\text{depth} + k)$
R-Tree	$O(n)$	$O(\log n + k)$ (avg)

13.4.6 Usages

- **Spatial Databases:** Finding all customers within a specific zip code or radius.
- **Computer Graphics:** Frustum culling and identifying all objects hit by a blast radius.
- **GIS:** Overlaying different map layers (e.g., finding all houses in a flood zone).
- **Machine Learning:** Preprocessing for K-Nearest Neighbors (KNN).
- **Collision Detection:** Finding all pairs of objects that might be colliding.

Example 13.13 (1D Range Search). Given sorted points $P = \{1, 3, 5, 7, 9, 11, 13\}$ and query range $R = [4, 10]$, a binary search locates the first point ≥ 4 (which is 5) and the last point ≤ 10 (which is 9). All points in this contiguous subsequence $\{5, 7, 9\}$ are returned in $O(\log n + k)$ time, where $k = 3$.

13.4.7 Testing and Edge Cases

- **Empty Query:** Ensure the algorithm returns an empty list for a range containing no points.
- **Full Range Query:** Verify that a range covering the entire domain returns all n points.
- **Points on Boundaries:** Consistently handle points exactly on the edge of the query rectangle.
- **Overlapping Points:** Ensure duplicate points are correctly counted/reported.
- **Very Large Coordinates:** Precision check for queries far from the origin.

13.4.8 References

- Bentley, J. L. (1975). “Multidimensional binary search trees used for associative searching”. Communications of the ACM.
- Lueker, G. S. (1978). “A data structure for orthogonal range queries”. IEEE FOCS.
- Berg, M. de, Cheong, O., Kreveld, M. van, & Overmars, M. (2008). “Computational Geometry: Algorithms and Applications”. Springer-Verlag.
- Wikipedia: Range searching.

13.5 Point in Polygon (Winding Number)

The Winding Number algorithm is a robust method for determining whether a point lies inside a given polygon. It calculates the total number of times the polygon's boundary wraps around the test point. A winding number of zero indicates the point is outside the polygon, while a non-zero value indicates it is inside. This method is particularly superior to the Ray Casting algorithm when dealing with self-intersecting polygons. Analysis of the winding number method for arbitrary polygons is given by Sunday [Sun01] and Hörmann and Agathos [HA01].

13.5.1 Definitions

Definition 13.14 (Winding Number). The *winding number* $wn(P, C)$ of a point P with respect to a closed curve C is the total number of times C winds counter-clockwise around P . Formally, it equals $\frac{1}{2\pi} \oint_C d\theta$ where θ is the angle from P to a point on C .

Definition 13.15 (Is-Left Test). The *Is-Left test* determines whether a point P is to the left of the directed line from V_1 to V_2 :

$$\text{IsLeft}(V_1, V_2, P) = (V_2.x - V_1.x)(P.y - V_1.y) - (P.x - V_1.x)(V_2.y - V_1.y).$$

This is equivalent to the signed area of $\triangle V_1 V_2 P$ (up to a factor of 2).

13.5.2 Theory

The algorithm counts the net number of full revolutions the polygon makes around the point P . As we traverse the polygon's edges:

- Every time an edge crosses the positive x -ray of P going **upward**, we increment the winding number.
- Every time an edge crosses the positive x -ray of P going **downward**, we decrement the winding number.

Crucially, an edge is only counted if the point P is actually to the left of the upward edge (or to the right of the downward edge), ensuring that the crossing actually “wraps” around the point.

Theorem 13.16 (Winding Number Correctness). *A point P lies inside a simple polygon C if and only if $wn(P, C) \neq 0$. More precisely, the winding number counts the number of times C encircles P .*

Proof. The winding number is a topological invariant related to the degree of the map $\theta : C \rightarrow S^1$ defined by $\theta(q) = \frac{q-P}{\|q-P\|}$. By the Jordan Curve Theorem, for a simple closed curve, $wn(P, C) = \pm 1$ for interior points and 0 for exterior points. The sign depends on the orientation of C . The incremental counting algorithm correctly tracks the accumulated angle by detecting upward and downward crossings of the horizontal ray from P . $\square$

13.5.3 Pseudocode

13.5.4 Parameters

- **Input:** A test `Point2D` and a list of `Point2D` vertices representing the polygon.
- **Winding Rule:** Most applications use the **Non-Zero Winding Rule** (inside if $wn \neq 0$), though some use the **Odd-Even Rule** (wn is odd).

Algorithm 33 Point-in-Polygon via Winding Number

```

1: function POINTWINDINGNUMBER( $P$ , polygon)
2:    $wn \leftarrow 0$ 
3:   for each edge  $(V1, V2)$  in polygon do
4:     if  $V1.y \leq P.y$  then
5:       if  $V2.y > P.y$  then
6:         if  $\text{IsLeft}(V1, V2, P) > 0$  then
7:            $wn \leftarrow wn + 1$ 
8:         end if
9:       end if
10:    else
11:      if  $V2.y \leq P.y$  then
12:        if  $\text{IsLeft}(V1, V2, P) < 0$  then
13:           $wn \leftarrow wn - 1$ 
14:        end if
15:      end if
16:    end if
17:  end for
18:  return  $wn$ 
19: end function
20: function ISLEFT( $V1, V2, P$ )
21:  return  $(V2.x - V1.x) * (P.y - V1.y) - (P.x - V1.x) * (V2.y - V1.y)$ 
22: end function

```

13.5.5 Complexity

- **Time Complexity:** $O(n)$ where n is the number of vertices. Each edge is visited exactly once.
- **Space Complexity:** $O(1)$ auxiliary space, assuming the polygon vertices are already stored.

Example 13.17 (Winding Number for a Square). Consider the unit square with vertices $(0,0), (1,0), (1,1), (0,1)$ traversed counter-clockwise. Testing point $P = (0.5, 0.5)$:

1. Edge $(0,0) \rightarrow (1,0)$: $V1.y = 0 \leq 0.5$, $V2.y = 0 \not> 0.5$: no crossing.
2. Edge $(1,0) \rightarrow (1,1)$: $V1.y = 0 \leq 0.5$, $V2.y = 1 > 0.5$: upward crossing. $\text{IsLeft} = (1-1)(0.5-0) - (0.5-1)(1-0) = 0 + 0.5 = 0.5 > 0$: $wn \leftarrow 1$.
3. Edge $(1,1) \rightarrow (0,1)$: $V1.y = 1 \not\leq 0.5$: downward branch. $V2.y = 1 \not\leq 0.5$: no crossing.
4. Edge $(0,1) \rightarrow (0,0)$: $V1.y = 1 \not\leq 0.5$: downward branch. $V2.y = 0 \leq 0.5$: downward crossing. $\text{IsLeft} = (0-0)(0.5-1) - (0.5-0)(0-1) = 0 + 0.5 = 0.5 > 0$, but we need < 0 : not counted (this is correct since the edge goes down and P is to its right).

Final $wn = 1 \neq 0$: point is inside. For $P = (2, 0.5)$ (outside), no crossings contribute: $wn = 0$.

13.5.6 Usages

- **SVG/PostScript Rendering:** Determining which areas of a complex path should be filled.
- **GIS Analysis:** Checking if a GPS coordinate falls within a specific territorial boundary.
- **Video Games:** Hitbox detection for complex, non-convex characters or objects.

13.5.7 Testing and Edge Cases

- **Point on Edge/Vertex:** The algorithm's behavior on the boundary depends on the strictness of the inequalities ($<$ vs $\leq$).
- **Horizontal Edges:** Handled correctly because they fail both the $V1.y \leq P.y < V2.y$ and $V2.y \leq P.y < V1.y$ conditions.
- **Self-intersecting Polygons:** A “figure-eight” polygon will have regions with winding numbers of 1, -1 , and 0.
- **Degenerate Polygons:** Polygons with fewer than 3 vertices or zero area.

13.5.8 References

- Sunday, D. (2001). “Inclusion of a Point in a Polygon”. Journal of Graphics Tools.
- Hörmann, K., & Agathos, A. (2001). “The point in polygon problem for arbitrary polygons”. Computational Geometry.
- Wikipedia: Point in polygon.

13.6 Largest Empty Circle

The largest empty circle problem asks for the circle of maximum radius whose center is within the convex hull of a given set of points P , and whose interior contains no points from P . This problem is a fundamental challenge in computational geometry with direct applications in facility location and urban planning.

13.6.1 Definitions

Definition 13.18 (Largest Empty Circle). Given a set $P \subseteq \mathbb{R}^2$, the *largest empty circle* is the circle C^* with maximum radius r^* such that C^* contains no point of P in its interior and its center lies within $\text{conv}(P)$.

13.6.2 Theory

The largest empty circle must have its center at one of two types of locations:

1. **Voronoi Vertices:** A Voronoi vertex is the circumcenter of three points from P . If this vertex is inside the convex hull, it defines an empty circle passing through those three points.
2. **Intersection of Voronoi Edges and Convex Hull:** The center could be at the intersection of a Voronoi edge and an edge of the convex hull.

The algorithm for finding the largest empty circle is:

1. Compute the **Voronoi Diagram** (Algorithm 25) of the points P .
2. Iterate through all Voronoi vertices. For each vertex inside the convex hull, calculate the distance to its closest point (its radius).
3. Iterate through all intersections of Voronoi edges with the convex hull boundary. For each intersection, calculate the radius of the empty circle centered there.
4. The largest of all these radii defines the largest empty circle.

Theorem 13.19 (Optimality of Voronoi Vertices). *The center of the largest empty circle of a point set P in general position is either a Voronoi vertex of P or lies on the intersection of a Voronoi edge with the boundary of $\text{conv}(P)$.*

13.6.3 Pseudocode

Algorithm 34 Largest Empty Circle

```

1: function LARGESTEMPTYCIRCLE(points)
2:    $CH \leftarrow \text{ConvexHull}(\text{points})$ 
3:    $VD \leftarrow \text{Voronoi}(\text{points})$ 
4:    $\text{max\_radius} \leftarrow 0$ 
5:    $\text{best\_center} \leftarrow \text{None}$ 
6:   for  $v$  in  $VD.\text{vertices}$  do
7:     if  $\text{PointInPolygon}(v, CH)$  then
8:        $r \leftarrow \text{Distance}(v, VD.\text{GetClosestSite}(v))$ 
9:       if  $r > \text{max\_radius}$  then
10:         $\text{max\_radius} \leftarrow r$ 
11:         $\text{best\_center} \leftarrow v$ 
12:       end if
13:     end if
14:   end for
15:   for  $\text{edge}$  in  $VD.\text{edges}$  do
16:     for  $\text{ch\_edge}$  in  $CH.\text{edges}$  do
17:        $p \leftarrow \text{Intersect}(\text{edge}, \text{ch\_edge})$ 
18:       if  $p$  is not  $\text{None}$  then
19:          $r \leftarrow \text{Distance}(p, VD.\text{GetClosestSite}(p))$ 
20:         if  $r > \text{max\_radius}$  then
21:            $\text{max\_radius} \leftarrow r$ 
22:            $\text{best\_center} \leftarrow p$ 
23:         end if
24:       end if
25:     end for
26:   end for
27:   return  $\text{best\_center}, \text{max\_radius}$ 
28: end function

```

13.6.4 Parameters

- **Precision:** Finding the intersection of Voronoi edges and the convex hull requires careful handling of floating-point inaccuracies.
- **Region Constraint:** The problem can be generalized to find the largest empty circle within any bounding polygon (not just the convex hull).

13.6.5 Complexity

- **Time Complexity:** $O(n \log n)$ to build the Voronoi diagram and convex hull. Finding intersections takes $O(n \log n)$ or $O(n)$ depending on the implementation.
- **Space Complexity:** $O(n)$ to store the Voronoi and convex hull structures.

13.6.6 Usages

- **Facility Location:** Finding the location for a new facility (e.g., a toxic waste dump or a new shopping center) that is as far as possible from all existing points of interest.
- **Urban Planning:** Determining where to place a new park to maximize the distance from the nearest noise sources.
- **Robotics:** Path planning to stay as far from obstacles as possible (the “safest” path follows Voronoi edges).
- **Signal Coverage:** Identifying the largest “blind spot” in a cellular or Wi-Fi network.

13.6.7 Testing and Edge Cases

- **Square/Rectangle Alignment:** For four points in a square, the center should be at the centroid.
- **Points on a Circle:** All Voronoi vertices should coincide at the center of the circle.
- **Collinear Points:** The convex hull is a line segment, so the circle will have zero radius.
- **Sparse vs. Dense:** Ensure the algorithm handles large empty spaces correctly.
- **Large Coordinates:** Precision check for very large x, y .

13.6.8 References

- Shamos, M. I., & Hoey, D. (1975). “Closest-point problems”. IEEE FOCS.
- Preparata, F. P., & Shamos, M. I. (1985). “Computational Geometry: An Introduction”. Springer-Verlag.
- Wikipedia: Largest empty sphere.

13.7 Minimum Bounding Boxes

A minimum bounding box (MBB) is the smallest rectangle (in terms of area or perimeter) that contains all the points in a given set P . Unlike the axis-aligned bounding box (AABB), which is restricted to the coordinate axes, the minimum bounding box can be rotated to find the optimal fit. This problem is fundamental in object recognition, shape analysis, and spatial data compression.

13.7.1 Definitions

Definition 13.20 (Minimum Bounding Box). The *minimum bounding box* of a point set P is the rectangle of minimum area (or perimeter) containing all points of P , with no restriction on orientation.

Definition 13.21 (Rotating Calipers). The *rotating calipers* technique is an algorithmic paradigm that uses a set of lines (“calipers”) touching a convex polygon at extreme points and rotates them simultaneously to compute geometric properties in linear time [Tou83].

13.7.2 Theory

The minimum bounding box of a point set P is identical to the MBB of its convex hull $\text{conv}(P)$. Freeman and Shapira (1975) proved that one edge of the minimum-area bounding box **MUST** be collinear with an edge of the convex hull.

The **Rotating Calipers** algorithm uses this property:

1. Compute the convex hull $\text{conv}(P)$.
2. Place four “calipers” (lines) touching the convex hull at its extreme points $(x_{\min}, x_{\max}, y_{\min}, y_{\max})$.
3. Rotate the four lines simultaneously around the hull.
4. Each time a line becomes collinear with an edge of the hull, calculate the area of the rectangle formed by the four lines.
5. After a 90° rotation, the minimum area found is the MBB.

Theorem 13.22 (Rotating Calipers Correctness). *At least one edge of the minimum-area bounding rectangle is collinear with an edge of the convex hull.*

Proof. Let R^* be the minimum-area bounding rectangle, and let e be an edge of R^* . If no edge of $\text{conv}(P)$ is parallel to e , then we can rotate R^* slightly to decrease its area while still containing all points, contradicting optimality. Hence at least one edge of R^* must be parallel to an edge of $\text{conv}(P)$. $\square$

13.7.3 Pseudocode

Algorithm 35 Minimum Bounding Box via Rotating Calipers

```

1: function MINIMUMBOUNDINGBOX(points)
2:    $CH \leftarrow \text{ConvexHull}(\text{points})$ 
3:    $\text{min\_area} \leftarrow \infty$ 
4:    $\text{best\_rectangle} \leftarrow \text{None}$ 
5:   for  $i = 0$  to  $\text{len}(CH) - 1$  do
6:      $\text{edge} \leftarrow CH[i + 1] - CH[i]$ 
7:      $\text{angle} \leftarrow \text{atan2}(\text{edge.y}, \text{edge.x})$ 
8:      $\text{rotated\_hull} \leftarrow \text{Rotate}(CH, -\text{angle})$ 
9:      $\text{min\_x}, \text{max\_x}, \text{min\_y}, \text{max\_y} \leftarrow \text{GetBounds}(\text{rotated\_hull})$ 
10:     $\text{area} \leftarrow (\text{max\_x} - \text{min\_x}) * (\text{max\_y} - \text{min\_y})$ 
11:    if  $\text{area} < \text{min\_area}$  then
12:       $\text{min\_area} \leftarrow \text{area}$ 
13:       $\text{best\_rectangle} \leftarrow \text{RotateBack}(\text{Rectangle}(\text{min\_x}, \text{max\_x}, \text{min\_y}, \text{max\_y}), \text{angle})$ 
14:    end if
15:  end for
16:  return  $\text{best\_rectangle}$ 
17: end function

```

13.7.4 Parameters

- **Objective Function:** Usually **Area** is minimized, but **Perimeter** can also be an objective.
- **Convex Hull Algorithm:** Graham Scan (Algorithm 2) or Monotone Chain (Algorithm 3) are suitable.

13.7.5 Complexity

- **Time Complexity:** $O(n \log n)$ to build the convex hull, and $O(n)$ for the rotating calipers scan.
- **Space Complexity:** $O(n)$ to store the convex hull.

13.7.6 Usages

- **Object Recognition:** Normalizing the orientation of an object before comparing it to a database.
- **Packaging and Bin Packing:** Finding the most space-efficient way to fit an object into a shipping container.
- **Spatial Indexing:** Using tighter-fitting MBBs instead of AABBs in R-trees to reduce overlap and improve search performance.
- **Computer Vision:** Finding the orientation of a rectangular object (e.g., a credit card or a license plate).
- **Geographic Information Systems (GIS):** Describing the extent of spatial features.

13.7.7 Testing and Edge Cases

- **Points forming a Circle:** The MBB should be a square.
- **Already Axis-Aligned Rectangle:** The MBB should match the AABB.
- **Points on a Line:** The MBB will have zero area.
- **Rotated Rectangle:** Ensure the algorithm finds the correct orientation and area.
- **Precision:** Use robust arithmetic for cross-products and distance calculations.

13.7.8 References

- Freeman, H., & Shapira, R. (1975). “Determining the minimum-area encasing rectangle for an arbitrary closed curve”. *Communications of the ACM*.
- Toussaint, G. T. (1983). “Solving geometric problems with the rotating calipers”. *Proceedings of MELECON '83*.
- Wikipedia: Minimum bounding box.

13.8 Maximum Overlap

The maximum overlap problem asks for a point (or region) in space that is covered by the maximum number of geometric objects from a given set. For example, given a set of 1D intervals, finding the time when the most tasks are active simultaneously. In 2D, this often involves finding the location covered by the most rectangles or disks. It is a fundamental problem in scheduling, resource allocation, and signal processing.

13.8.1 Definitions

Definition 13.23 (Depth of a Point). The *depth* of a point p with respect to a set of objects $\mathcal{F}$ is the number of objects in $\mathcal{F}$ that contain p .

Definition 13.24 (Maximum Depth). The *maximum depth* is $\max_{p \in \mathbb{R}^d} \text{depth}(p, \mathcal{F})$, i.e., the maximum number of overlapping objects at any point.

13.8.2 Theory

1D Case (Intervals)

For n intervals $[s_i, e_i]$, the maximum overlap can be found using a **sweep-line** approach:

1. Treat all start and end points as “events.”
2. Sort events by coordinate.
3. Traverse events: increment a counter for every start point and decrement for every end point.
4. The maximum value reached by the counter is the maximum overlap.

2D Case (Rectangles)

For axis-aligned rectangles, the problem is solved by sweeping a vertical line across the plane:

1. As the sweep line hits the left edge of a rectangle, the corresponding vertical interval is added to a **Segment Tree**.
2. As it hits the right edge, the interval is removed.
3. The Segment Tree maintains the maximum “coverage” count across its entire range efficiently.

Theorem 13.25 (1D Maximum Overlap Sweep Correctness). *The sweep-line algorithm correctly computes the maximum overlap of n intervals in $O(n \log n)$ time.*

13.8.3 Pseudocode

13.8.4 Parameters

- **Boundary Inclusion:** Decide if intervals are open (a, b) or closed $[a, b]$. This affects how events at the same coordinate are handled during the sort.
- **Data Structure:** For 2D, a Segment Tree with lazy propagation is required to achieve $O(n \log n)$ time.

13.8.5 Complexity

- **1D Complexity:** $O(n \log n)$ due to sorting the $2n$ endpoints.
- **2D Complexity:** $O(n \log n)$ using a sweep-line and a Segment Tree.
- **Space Complexity:** $O(n)$ to store the events and the tree structure.

Example 13.26 (1D Sweep Trace). Consider intervals $[1, 4]$, $[2, 5]$, $[3, 6]$. Events sorted: $(1, +1)$, $(2, +1)$, $(3, +1)$, $(4, -1)$, $(5, -1)$, $(6, -1)$. The sweep yields counts: 1, 2, 3, 2, 1, 0. Maximum overlap is 3, achieved at any point in $[3, 4]$.

Algorithm 36 Maximum Overlap (1D Sweep)

```
1: function MAXIMUMOVERLAP1D(intervals)
2:   events  $\leftarrow []$ 
3:   for s, e in intervals do
4:     events.append((s, +1))
5:     events.append((e, -1))
6:   end for
7:   events.sort(key =  $\lambda x : (x[0], -x[1])$ )
8:   max_count  $\leftarrow 0$ 
9:   current_count  $\leftarrow 0$ 
10:  best_pos  $\leftarrow \text{None}$ 
11:  for pos, type in events do
12:    current_count  $\leftarrow \text{current\_count} + \text{type}$ 
13:    if current_count > max_count then
14:      max_count  $\leftarrow \text{current\_count}$ 
15:      best_pos  $\leftarrow \text{pos}$ 
16:    end if
17:  end for
18:  return max_count, best_pos
19: end function
```

13.8.6 Usages

- **Scheduling:** Finding the peak number of concurrent users or tasks.
- **Genomics:** Identifying “hotspots” where many DNA reads map to the same genomic location.
- **VLSI Design:** Finding regions with the highest density of circuit components to prevent overheating.
- **Finance:** Detecting when the most stock market orders are active in a specific price range.
- **Sensor Networks:** Identifying regions covered by the most sensors (redundancy analysis).

13.8.7 Testing and Edge Cases

- **Disjoint Intervals:** Maximum overlap should be 1.
- **Identical Intervals:** Maximum overlap should be n .
- **Nested Intervals:** Ensure the algorithm handles intervals fully contained within each other.
- **Zero-Length Intervals:** Points treated as intervals.
- **Large Coordinates:** Precision check for very distant intervals.
- **Touching Boundaries:** Verify behavior when intervals touch at a single point (overlap of 1 or 2 depending on inclusion rule).

13.8.8 References

- Bentley, J. L., & Wood, D. (1980). “Algorithms for spectral and overlapping rectangles”. SIAM Journal on Computing.

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). “Introduction to Algorithms”. MIT Press.
- Wikipedia: Interval tree.

13.9 Segment Intersection (Bentley-Ottmann)

The segment intersection problem asks for all intersection points among a set of n line segments in the 2D plane. While a brute-force approach checks every pair of segments in $O(n^2)$ time, the Bentley-Ottmann algorithm [BO79] uses a **sweep-line paradigm** to achieve a complexity that is sensitive to the number of intersections k . It is a cornerstone algorithm in computational geometry for applications in CAD, GIS, and computer graphics.

13.9.1 Definitions

Definition 13.27 (Sweep Line). A *sweep line* is an imaginary vertical line that moves from left to right across the plane, processing events in order of x -coordinate.

Definition 13.28 (Sweep-Line Status). The *sweep-line status* is a data structure (typically a balanced BST) that maintains the vertical order of segments currently intersecting the sweep line, ordered by their y -coordinates at the sweep line’s current position.

Definition 13.29 (Output-Sensitive Algorithm). An algorithm is *output-sensitive* if its running time depends on both the input size and the output size. For Bentley-Ottmann, the time is $O((n + k) \log n)$ where k is the number of intersection points reported.

13.9.2 Theory

The Bentley-Ottmann algorithm is based on the observation that two segments can only intersect if they are **adjacent** in the vertical order at some position along the sweep line.

1. Initialize the event queue with all segment endpoints.
2. Pop the next event point P .
3. **If P is a left endpoint:** Insert the segment into the status structure. Check for intersections with its immediate neighbors (above and below).
4. **If P is a right endpoint:** Check if its neighbors (above and below) now intersect each other. Remove the segment from the status structure.
5. **If P is an intersection point:** Report the intersection. Swap the order of the two intersecting segments in the status structure. Check for new intersections with their new neighbors.

This process ensures that all k intersections are found by only checking pairs of segments that are “geographically” close.

Theorem 13.30 (Bentley-Ottmann Correctness). *The Bentley-Ottmann algorithm correctly reports all intersection points among n line segments. Each intersection is reported exactly once.*

Theorem 13.31 (Bentley-Ottmann Complexity). *The Bentley-Ottmann algorithm runs in $O((n + k) \log n)$ time and $O(n + k)$ space, where n is the number of segments and k is the number of intersections.*

Proof. There are $2n$ endpoint events and k intersection events. Each event requires $O(\log n)$ time for BST operations (insert, delete, swap) and $O(1)$ neighbor checks. Each intersection event is generated at most once (when two segments become adjacent), and each event is processed once. The total number of events is $O(n + k)$, each costing $O(\log n)$, giving $O((n + k) \log n)$ total time. Space is $O(n + k)$ for the event queue (which stores at most $O(n + k)$ events at any time) and $O(n)$ for the status structure. $\square$

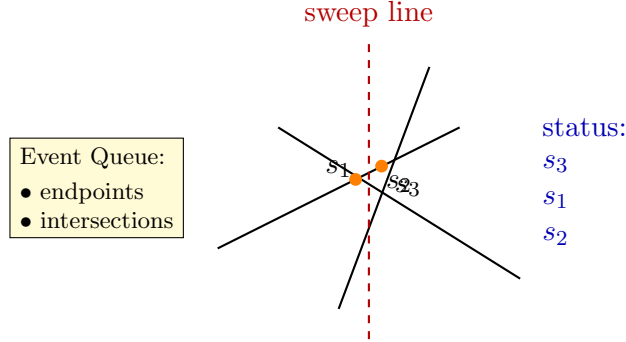


Figure 13.2: Bentley–Ottmann sweep line: the vertical line moves left-to-right. The *status* maintains the vertical order of intersecting segments.

13.9.3 Pseudocode

13.9.4 Parameters

- **Precision:** Highly sensitive to floating-point errors. Use robust geometric predicates for intersection calculation and segment ordering.
- **Degeneracies:** Multiple segments intersecting at the same point or vertical segments require careful handling.

13.9.5 Complexity

- **Time Complexity:** $O((n + k) \log n)$, where n is the number of segments and k is the number of intersections.
- **Space Complexity:** $O(n + k)$ to store the event queue and status structure.

Example 13.32 (Bentley-Ottmann Trace). Consider three segments: $s_1 = \{(0, 0), (3, 3)\}$, $s_2 = \{(0, 3), (3, 0)\}$, $s_3 = \{(1, 0), (1, 3)\}$.

1. Events sorted by x : $(0, \text{left}, s_1)$, $(0, \text{left}, s_2)$, $(1, \text{left}, s_3)$, $(3, \text{right}, s_1)$, $(3, \text{right}, s_2)$.
2. After processing $x = 0$: Status order (at $x = 0^+$) is s_2 (above), s_1 (below). Check intersection: they meet at $(1.5, 1.5)$: event enqueued.
3. At $x = 1$: s_3 inserted. Status: s_2, s_3, s_1 . Check s_3 with s_2 : they meet at $(1, 2)$: event enqueued. Check s_3 with s_1 : meet at $(1, 1)$: event enqueued.
4. At $x = 1$ (intersection $(1, 1)$): swap s_1, s_3 . New neighbors checked.
5. At $x = 1$ (intersection $(1, 2)$): swap s_2, s_3 .
6. At $x = 1.5$ (intersection $(1.5, 1.5)$): swap s_1, s_2 .
7. After $x = 3$: all segments removed. Three intersection points reported.

Algorithm 37 Bentley-Ottmann Segment Intersection

```

1: function BENTLEYOTTMANN(segments)
2:   events  $\leftarrow$  PriorityQueue()
3:   for s in segments do
4:     events.push(Event(s.left, LEFT_ENDPOINT, s))
5:     events.push(Event(s.right, RIGHT_ENDPOINT, s))
6:   end for
7:   status  $\leftarrow$  BalancedBST()
8:   intersections  $\leftarrow$  []
9:   while events is not empty do
10:    event  $\leftarrow$  events.pop()
11:    p  $\leftarrow$  event.point
12:    if event.type == LEFT_ENDPOINT then
13:      s  $\leftarrow$  event.segment
14:      status.insert(s, p.x)
15:      above, below  $\leftarrow$  status.neighbors(s)
16:      CheckIntersection(s, above, events)
17:      CheckIntersection(s, below, events)
18:    else if event.type == RIGHT_ENDPOINT then
19:      s  $\leftarrow$  event.segment
20:      above, below  $\leftarrow$  status.neighbors(s)
21:      status.remove(s)
22:      CheckIntersection(above, below, events)
23:    else if event.type == INTERSECTION then
24:      s1, s2  $\leftarrow$  event.segments
25:      intersections.append(p)
26:      status.swap(s1, s2)
27:      new_above  $\leftarrow$  status.above(s2)
28:      new_below  $\leftarrow$  status.below(s1)
29:      CheckIntersection(s2, new_above, events)
30:      CheckIntersection(s1, new_below, events)
31:    end if
32:  end while
33:  return intersections
34: end function

```

13.9.6 Usages

- **Map Overlay:** Combining two maps (e.g., roads and counties) to find where they cross.
- **Boolean Operations:** Preprocessing for polygon union/intersection.
- **CAD/CAM:** Detecting errors in mechanical drawings (e.g., overlapping paths).
- **VLSI Design:** Checking for short circuits in circuit board traces.
- **Path Planning:** Determining if a path intersects any obstacles.

13.9.7 Testing and Edge Cases

- **No Intersections:** The algorithm should only process $2n$ events and return empty.
- **All segments intersect at one point:** Verify that the event queue handles multiple events at the same location.
- **Vertical Segments:** Ensure the ordering logic correctly handles segments with the same x -coordinate.
- **Collinear Segments:** Overlapping segments on the same line.
- **Duplicate Intersections:** Ensure each intersection point is reported only once if multiple segments meet.

13.9.8 References

- Bentley, J. L., & Ottmann, T. A. (1979). “Algorithms for reporting and counting geometric intersections”. IEEE Transactions on Computers.
- Berg, M. de, Cheong, O., Kreveld, M. van, & Overmars, M. (2008). “Computational Geometry: Algorithms and Applications”. Springer-Verlag.
- Wikipedia: Bentley–Ottmann algorithm.

13.10 Point Location via Trapezoidal Maps

The point location problem asks, given a planar subdivision and a query point, which face of the subdivision contains that point. It is one of the most fundamental problems in computational geometry, appearing as a subroutine in polygon operations, mesh processing, GIS overlays, and visibility computations. A classical solution is the *trapezoidal map* combined with a directed acyclic graph (DAG) search structure, which supports $O(\log n)$ expected query time after $O(n \log n)$ expected preprocessing [Mul90, Sei91]. The randomized incremental construction avoids the degeneracy analysis required by deterministic approaches.

13.10.1 Definitions

Definition 13.33 (Trapezoid). A *trapezoid* T in a trapezoidal map is a region bounded by two vertical walls (left and right sides), a *top edge* e_{top} , and a *bottom edge* e_{bot} . The left and right boundaries are vertical segments from the bottom edge to the top edge. Each trapezoid is uniquely identified by its left x -coordinate, right x -coordinate, and the two edges that bound it from above and below.

Definition 13.34 (Trapezoidal Map). Given a set S of n non-intersecting line segments in $\mathbb{R}^2$, the *trapezoidal map* $\mathcal{T}(S)$ is the subdivision of the plane induced by extending vertical rays from every endpoint of every segment in S upward and downward until they hit a segment or a bounding box. The result is a partition of the plane into $O(n)$ trapezoids.

Definition 13.35 (DAG Search Structure). The *DAG search structure* $D(\mathcal{T})$ is a directed acyclic graph with leaf nodes representing the trapezoids of $\mathcal{T}$ and internal nodes representing *branching queries*. There are two types of internal nodes: *x-nodes* (containing a point p , branching left if the query has $x < p.x$) and *y-nodes* (containing a segment $e = (p_1, p_2)$, branching left if the query is above e and right otherwise). A root-to-leaf path encodes a sequence of decisions that identifies exactly one trapezoid.

Definition 13.36 (*x-Node*). An *x-node* stores a point $p = (p.x, p.y)$. For a query point q , the search proceeds to the left child if $q.x < p.x$ and to the right child otherwise. An *x-node* corresponds to an endpoint of a segment.

Definition 13.37 (*y-Node*). A *y-node* stores a segment $e = (p_1, p_2)$ with $p_1.x < p_2.x$. For a query point q , the search proceeds to the left child if q lies above e (i.e., the orientation of (p_1, p_2, q) is counter-clockwise) and to the right child otherwise. A *y-node* corresponds to a segment in S .

13.10.2 Theory

Construction

The trapezoidal map is built via a *randomized incremental algorithm*. Starting with a single bounding trapezoid that encompasses the entire input, the segments $s_1, s_2, \dots, s_n$ are inserted in a uniformly random order. When a new segment s_k is inserted, the existing trapezoids intersected by s_k are located by walking along the trapezoid adjacency graph. Each intersected trapezoid is split into at most four new trapezoids, and the DAG is updated accordingly. Because the insertion order is random, the expected depth of the DAG is $O(\log n)$.

The DAG is a *dag* (directed acyclic graph), not a tree, because multiple internal nodes can share the same child. This sharing is critical: when a segment is inserted and splits an existing trapezoid, the new *x-node* and *y-node* are wired into the DAG at the position of the old leaf, so other paths to that leaf are implicitly updated without duplicating the subtree.

Query Algorithm

To locate a query point q , we descend from the root of the DAG:

1. At an *x-node* with point p : go left if $q.x < p.x$, right otherwise.
2. At a *y-node* with segment $e = (p_1, p_2)$: go left if q is above e (counter-clockwise orientation), right otherwise.
3. At a leaf: return the trapezoid stored at that leaf.

The above-below test is performed using the orientation predicate (cross product):

$$\text{IsAbove}(q, (p_1, p_2)) \iff (p_2.x - p_1.x)(q.y - p_1.y) - (p_2.y - p_1.y)(q.x - p_1.x) > 0.$$

Theorem 13.38 (Trapezoidal Map Complexity). *Let S be a set of n non-intersecting line segments in $\mathbb{R}^2$. The randomized incremental trapezoidal map construction with a DAG search structure has expected preprocessing time $O(n \log n)$ and expected space $O(n)$. The expected query time to locate a point in the trapezoidal map is $O(\log n)$, where the expectation is over the random insertion order.*

Proof. We analyze the expected size of the search structure using backwards analysis. Consider the DAG after k segments have been inserted. For any trapezoid T that exists after all n segments are inserted, T was created at some step $k < n$ when the last segment that affects T was inserted. The probability that a particular segment among $\{s_1, \dots, s_k\}$ is the last to affect T is at most $2/k$ (since T is bounded by at most two segments, and any one of them could be the last inserted among the k). Therefore, the expected number of trapezoids created at step k is at most 2. Summing over $k = 1, \dots, n$, the total expected number of trapezoids (and hence DAG nodes) is $O(n)$.

The expected depth of a leaf in the DAG is bounded by the expected number of segments whose x -nodes or y -nodes lie on the root-to-leaf path. By the same backwards analysis, the expected depth of the deepest leaf is $O(\log n)$. Since each level of the DAG is traversed in $O(1)$ time, the expected query time is $O(\log n)$.

The expected total work across all n insertions is $O(n \log n)$ by an amortized argument: at step k , the new segment intersects $O(1)$ expected trapezoids, each of which is split in $O(1)$ time, and the DAG update is $O(1)$ per affected node. $\square$

Example 13.39 (Trapezoidal Map Construction). Consider two non-intersecting segments $s_1 = \{(1, 1), (3, 3)\}$ and $s_2 = \{(2, 0), (4, 2)\}$ inside a bounding box $[0, 5] \times [0, 5]$.

1. **Initialization:** The bounding trapezoid T_0 spans the entire box. The DAG root is a leaf containing T_0 .
2. **Insert s_1 :** The segment s_1 splits T_0 into trapezoids above and below it. A y -node for s_1 is created at the root. Its left child is a leaf for the trapezoid above s_1 ; its right child is a leaf for the trapezoid below s_1 .
3. **Insert s_2 :** The segment s_2 crosses through the lower trapezoid (the right child of the s_1 y -node). That trapezoid is split: new x -nodes for the endpoints $(2, 0)$ and $(4, 2)$ are inserted, and additional y -nodes for s_2 are created. The upper trapezoid (above s_1) is unaffected.
4. **Query for $q = (3, 2.5)$:** Starting at the root (y -node for s_1): Is $(3, 2.5)$ above s_1 ? Cross product: $(3-1)(2.5-1) - (3-1)(3-1) = 2 \cdot 1.5 - 2 \cdot 2 = -1 < 0$, so q is below s_1 : go right. Descend through the DAG nodes for the lower region until a leaf is reached, identifying the trapezoid containing q .

13.10.3 Pseudocode

The following pseudocode is adapted from the `TrapezoidalMap` class in the source implementation.

13.10.4 Parameters

- **Bounding Box:** A sufficiently large bounding box must be chosen to contain all segments and query points. The bounding box defines the initial trapezoid and the vertical extent of the map.
- **Precision:** The orientation predicate `IsAbove()` uses a cross-product computation. For inputs with nearly collinear segments, a robust predicate with an epsilon tolerance is recommended to avoid numerical instability.
- **Randomization:** The insertion order of segments should be uniformly random to guarantee the expected $O(\log n)$ query depth. A poor (adversarial) ordering can lead to $O(n)$ worst-case depth.

Algorithm 38 Trapezoidal Map Query

```

1: function LOCATE( $root, q$ )
2:    $curr \leftarrow root$ 
3:   while  $curr$  is not a leaf do
4:     if  $curr$  is an  $x$ -node then
5:        $p \leftarrow curr.value$   $\triangleright$  a point
6:       if  $q.x < p.x$  then
7:          $curr \leftarrow curr.left\_child$ 
8:       else
9:          $curr \leftarrow curr.right\_child$ 
10:      end if
11:    else  $\triangleright y$ -node
12:       $e \leftarrow curr.value$   $\triangleright$  a segment  $(p_1, p_2)$ 
13:      if IsAbove( $q, e$ ) then
14:         $curr \leftarrow curr.left\_child$ 
15:      else
16:         $curr \leftarrow curr.right\_child$ 
17:      end if
18:    end if
19:  end while
20:  return  $curr.value$   $\triangleright$  the containing trapezoid
21: end function
22: function ISABOVE( $q, e = (p_1, p_2)$ )
23:  return  $(p_2.x - p_1.x) \cdot (q.y - p_1.y) - (p_2.y - p_1.y) \cdot (q.x - p_1.x) > 0$ 
24: end function

```

Algorithm 39 Trapezoidal Map Construction (Incremental)

```

1: function TRAPEZOIDALMAP( $bbox$ )
2:    $p_1, p_2 \leftarrow bbox$   $\triangleright$  bounding box corners
3:    $T_0 \leftarrow$  new Trapezoid( $p_1, p_2$ , top-edge, bottom-edge)
4:    $root \leftarrow$  new node containing  $T_0$ 
5:    $T_0.node \leftarrow root$ 
6:   return  $root$ 
7: end function
8: function INSERTSEGMENT( $root, s_k$ )
9:    $T_{start} \leftarrow$  Locate( $root, s_k$ )
10:  Walk from  $T_{start}$  to the right along adjacent trapezoids intersected by  $s_k$ 
11:  for each intersected trapezoid  $T$  do
12:    Split  $T$  into at most four new trapezoids using  $s_k$ 
13:    Replace the leaf for  $T$  in the DAG with a new subtree of  $x$ -nodes and  $y$ -nodes
14:  end for
15: end function

```

- **Segment Validity:** The input segments must be non-intersecting (except possibly at shared endpoints). Intersecting segments must be split at their intersection points before building the trapezoidal map.

13.10.5 Complexity

- **Preprocessing Time:** $O(n \log n)$ expected, where n is the number of segments. Each insertion processes $O(1)$ expected trapezoids, and the walk to find intersected trapezoids is amortized $O(1)$ per segment.
- **Space:** $O(n)$ expected. The DAG stores $O(n)$ nodes (both internal and leaf), and each trapezoid stores $O(1)$ data plus adjacency pointers.
- **Query Time:** $O(\log n)$ expected. The DAG depth is $O(\log n)$ by backwards analysis, and each level is processed in $O(1)$ time.
- **Worst Case:** $O(n)$ query time and $O(n^2)$ space in the worst case (adversarial input ordering), though this is extremely unlikely with randomized construction.

13.10.6 Usages

- **Planar Subdivision Queries:** Determining which region of a Voronoi diagram, triangulation, or map overlay contains a given point.
- **Polygon Operations:** Point location is a subroutine in polygon clipping, union, and intersection algorithms (e.g., Weiler–Atherton).
- **Mesh Processing:** Finding which triangle or tetrahedron contains a query point in finite element analysis and computer graphics.
- **GIS and Cartography:** Identifying which administrative region, soil type, or land-use zone a GPS coordinate falls within.
- **Visibility and Ray Tracing:** Determining the first surface hit by a ray in a 2D scene represented as a planar subdivision.
- **Robotics:** Localization within a known map by matching sensor observations to map regions.

13.10.7 Testing and Edge Cases

- **Point on Segment Boundary:** The orientation predicate may return zero for points exactly on a segment. The implementation should define a consistent convention (e.g., treat collinear as “not above”).
- **Point on Vertical Wall:** Queries on the vertical boundary between two trapezoids should consistently return one of the two trapezoids.
- **Empty Map:** A map with no segments should have a single bounding trapezoid; all queries should return that trapezoid.
- **Single Segment:** The map should contain exactly two trapezoids (above and below the segment) inside the bounding box.
- **Degenerate Segments:** Horizontal or vertical segments should be handled without creating zero-width trapezoids.

- **Collinear Endpoints:** Multiple segments sharing an endpoint should produce consistent x -node ordering in the DAG.
- **Large Input:** Verify that the expected $O(\log n)$ query depth holds empirically for $n > 10,000$ segments by measuring average path length.

13.10.8 References

- Mulmuley, K. (1990). “A fast planar partition algorithm, I”. *Journal of Symbolic Computation*.
- Seidel, R. (1991). “A simple and fast incremental randomized algorithm for computing trapezoidal decompositions and for point location”. *Computational Geometry: Theory and Applications*.
- Berg, M. de, Cheong, O., Kreveld, M. van, & Overmars, M. (2008). “Computational Geometry: Algorithms and Applications”. Springer-Verlag.
- Guibas, L. J., & Sedgwick, R. (1983). “A dichromatic framework for balanced trees”. *IEEE FOCS*.
- Wikipedia: Point location.

13.11 Point Location in Triangulations (Walking)

13.11.1 Overview

Given a triangulation or tetrahedral mesh with billions of elements, the **walking** method locates the simplex containing a query point by moving across edges (2D) or faces (3D) from a starting simplex toward the query point. Unlike the trapezoidal map of Section 13.10 (see also the dedicated point location chapter, Section 8.8), which is restricted to planar subdivisions, walking works directly on a mesh in any dimension and needs no auxiliary search structure beyond the mesh adjacency itself. It is the method of choice inside incremental Delaunay construction (Section 12.5 and 12.4) and for FEM and particle codes that repeatedly query a fixed mesh.

13.11.2 Definitions

Definition 13.40 (Visibility Walk). A *visibility walk* from a starting simplex σ_0 toward query point q repeatedly crosses the facet of the current simplex that is *visible* from q : among the facets whose separating hyperplane leaves q in the opposite half-space, move across the first such facet found. The walk terminates at a simplex containing q .

Definition 13.41 (Jump-and-Walk). A *jump-and-walk* first selects a starting simplex σ_0 using a spatial index (a grid, k -d tree, or Delaunay tree) that returns a simplex close to q , then walks from σ_0 . The jump makes the walk length shorter in expectation than a walk from a fixed or random start.

Definition 13.42 (Straight Walk). A *straight walk* from σ_0 moves, at every step, across the facet intersected by the line segment from the current position to q . It tends to take fewer steps than a visibility walk but can fail on non-convex or non-Delaunay meshes; the visibility walk is robust on any triangulation.

13.11.3 Theory

Theorem 13.43 (Visibility Walk Terminates). *A visibility walk terminates at a simplex containing q on any triangulation of a convex domain. On a Delaunay triangulation it terminates on any simple domain [Dev02].*

Proof sketch. Each crossing strictly decreases the distance from the current simplex to q in the sense of the supporting lines of the crossed facets: the walk cannot cross the same facet twice, because the crossed hyperplanes separate q from the already visited simplices. Finiteness follows from the finite number of facets. The Delaunay case follows because every vertex sees at most one successive facet, which guarantees progress on non-convex domains as well [Dev02]. $\square$

Theorem 13.44 (Expected Walk Length). *On a Delaunay triangulation of n points, the expected number of steps of a walk starting at a random simplex is $O(\sqrt{n})$; with a jump-and-walk whose starting simplex is chosen near q by a grid or k -d tree, the expected number of steps is $O(\log n)$ for uniformly distributed points [Dev02].*

Proof sketch. The expected $O(\sqrt{n})$ bound follows from the fact that the walk crosses $O(\sqrt{n})$ cells when n points are distributed over a bounded region: the number of simplices crossed is bounded by the number of simplices intersected by a segment, which is $O(\sqrt{n})$ in expectation for the Delaunay triangulation of a uniform set. A spatial-index jump reduces the distance from the start to the query point, which reduces the expected crossing count to $O(\log n)$ [Dev02]. $\square$

13.11.4 Pseudocode

Algorithm 40 Locate in a Triangulation by Visibility Walk

```

1: function LOCATEWALK(mesh, q, start)
2:    $\sigma \leftarrow \text{start}$ 
3:   while true do
4:      $\text{facet} \leftarrow$  first facet of  $\sigma$  such that  $q$  and  $\sigma \setminus \text{facet}$  lie on opposite sides of  $\text{aff}(\text{facet})$ 
5:     if no such facet exists then  $\triangleright q$  is inside  $\sigma$ 
6:       return  $\sigma$ 
7:     end if
8:      $\sigma \leftarrow$  neighbor of  $\sigma$  across  $\text{facet}$   $\triangleright$  cross the visible facet
9:   end while
10: end function

11: function JUMPANDWALK(mesh, q, grid)
12:    $\text{cells} \leftarrow \text{NearestCandidateCells}(q, r, \delta)$   $\triangleright$  grid jump, Section 10.7
13:    $\sigma_0 \leftarrow$  a simplex intersecting a cell in  $\text{cells}$ , closest to  $q$ 
14:   return LOCATEWALK(mesh, q,  $\sigma_0$ )
15: end function

```

The 3D case is identical with *facets* replaced by the four triangular faces of a tetrahedron: cross the face whose plane separates q from the tetrahedron's opposite vertex. In 2D the facet is the single edge separating q from the opposite vertex.

13.11.5 Parameter Selection

- **Start simplex:** for Delaunay construction with spatially sorted (Hilbert or Morton) insertion, reuse the simplex found for the previous point; for queries, use a grid or k -d tree jump.

- **Robust predicates:** use adaptive orientation tests (`orient2d`, `orient3d`) to avoid looping near degeneracies [She97].
- **Tie-breaking:** when q lies exactly on a facet, pick one adjacent simplex consistently to avoid infinite loops.

13.11.6 Complexity

- **Time:** expected $O(\sqrt{n})$ steps from a random start; $O(\log n)$ with a grid or k-d tree jump on uniform data.
- **Space:** $O(1)$ beyond the mesh itself; the grid jump adds $O(n)$.
- **No preprocessing:** unlike the trapezoidal map, walking needs no auxiliary DAG; it uses only the mesh adjacency.

13.11.7 Usages

- **Incremental Delaunay construction:** locating the triangle containing each inserted point [Bow81, Wat81].
- **FEM post-processing:** reporting which element contains a query point or interpolation target.
- **Ray tracing and probing** of volumetric (tetrahedral) meshes.

13.11.8 Testing and Edge Cases

- **Termination:** compare against the trapezoidal map location (Section 13.10) for 2D on the same mesh; results must agree.
- **Boundary points:** queries exactly on an edge or vertex must return a consistent simplex.
- **Non-convex meshes:** use the visibility walk (not the straight walk) and verify termination on meshes with concavities.
- **Large input:** measure average walk length over n queries and confirm it matches the theoretical $O(\log n)$ or $O(\sqrt{n})$ growth.
- **Counterexample checks:** the straight walk can fail on non-Delaunay meshes; assert the visibility walk is always used there.

13.11.9 References

- Devillers, O. (2002). “The Delaunay hierarchy”. International Journal of Foundations of Computer Science.
- Bowyer, A. (1981). “Computing Dirichlet tessellations”. The Computer Journal.
- Watson, D. F. (1981). “Computing the n -dimensional Delaunay tessellation”. The Computer Journal.
- Shewchuk, J. R. (1997). “Adaptive precision floating-point arithmetic and fast robust geometric predicates”. Discrete & Computational Geometry.

Part V

Arrangements and Duality

Chapter 14

Arrangements of Lines and Curves

14.1 Davenport–Schinzel Sequences

14.1.1 Overview

A Davenport–Schinzel sequence is a sequence of symbols that satisfies specific constraints on the alternating sub-sequences it can contain. These sequences provide a powerful combinatorial framework for analyzing the complexity of the **lower envelope** of a set of functions. In computational geometry, they are used to bound the complexity of arrangements of curves, motion planning visibility, and dynamic geometric structures. The theory originated with Davenport and Schinzel [DS65] and was developed into a comprehensive geometric theory by Sharir and Agarwal [SA95].

14.1.2 Definitions

Definition 14.1 (Davenport–Schinzel Sequence). A sequence $U = (u_1, u_2, \dots, u_m)$ over an alphabet A of n symbols is a **Davenport–Schinzel sequence of order s** (denoted $DS(n, s)$) if:

1. No two adjacent symbols are the same: $u_i \neq u_{i+1}$ for all $1 \leq i < m$.
2. It does not contain any alternating sub-sequence of length $s + 2$ of the form $a, b, a, b, \dots$ (alternating $s + 1$ times between two distinct symbols $a, b \in A$).

Definition 14.2 ($\lambda_s(n)$). $\lambda_s(n)$ denotes the maximum possible length of a Davenport–Schinzel sequence of order s on n distinct symbols. These functions grow near-linearly for any fixed s :

$$\lambda_0(n) = 1, \quad \lambda_1(n) = n, \quad \lambda_2(n) = 2n - 1, \quad \lambda_3(n) = \Theta(n \alpha(n)),$$

where $\alpha(n)$ is the inverse Ackermann function.

Definition 14.3 (Inverse Ackermann Function). The **inverse Ackermann function** $\alpha(n)$ is the function such that $\alpha(n) \leq 4$ for all practical values of n (up to $n = 10^{80}$, the number of atoms in the observable universe). Formally, $\alpha(n)$ is the inverse of the Ackermann function $A(k)$, which grows faster than any primitive recursive function.

Definition 14.4 (Lower Envelope). The **lower envelope** of a collection of n functions $\{f_1, \dots, f_n\}$ defined on a common domain D is the function $L(x) = \min_{1 \leq i \leq n} f_i(x)$. The *complexity* of the lower envelope is the number of maximal continuous pieces that compose it.

14.1.3 Theory

The importance of DS sequences stems from their relationship to function intersections. If we have n continuous functions such that any two functions intersect at most s times, then the sequence of function indices that form the **lower envelope** is a Davenport–Schinzel sequence of order s .

For example:

- **Order** $s = 1$: Functions intersect at most once (e.g., lines). $\lambda_1(n) = n$.
- **Order** $s = 2$: Functions intersect at most twice (e.g., parabolas). $\lambda_2(n) = 2n - 1$.
- **Order** $s = 3$: Functions intersect at most three times. $\lambda_3(n) = \Theta(n\alpha(n))$, where $\alpha(n)$ is the extremely slow-growing inverse Ackermann function.

This theory allows us to prove that the complexity of the lower envelope of n line segments is $O(n\alpha(n))$, which is much smaller than the total number of intersections $O(n^2)$.

Theorem 14.5 (DS Sequence Length Bounds). *For $s \geq 1$ and $n \geq 1$:*

1. $\lambda_1(n) = n$.
2. $\lambda_2(n) = 2n - 1$.
3. For $s \geq 3$, $\lambda_s(n) = n \cdot \alpha(n)^{O(1)}$, where $\alpha(n)$ is the inverse Ackermann function. More precisely, $\lambda_3(n) = 2n\alpha(n) + O(n)$.

These bounds are tight.

Proof. The cases $s = 1$ and $s = 2$ are elementary. For $s \geq 3$, the upper bound was established by Sharir [SA95] using a charging argument on the “blocks” of the sequence and the properties of the Ackermann hierarchy. The matching lower bound is obtained by an explicit construction based on the composition sequences of the Ackermann function. See [SA95, Chapters 2–4] for the complete proof. $\square$

Observation 14.6. The growth rate of $\lambda_s(n)$ for increasing s reflects the increasing complexity of the lower envelope as the number of allowed intersections between function pairs grows. For $s = 1$ (lines), the envelope has $O(n)$ complexity. For $s = 3$ (line segments), the complexity jumps to $O(n\alpha(n))$ —a subtlety that arises from the non-monotonic behavior of segments.

14.1.4 Pseudocode

Computing the Lower Envelope (Recursive)

14.1.5 Parameter Selection

- **Order** (s): Depends on the type of geometric primitives (e.g., $s = 1$ for lines, $s = 2$ for circles, $s = 3$ for line segments).
- **Alphabet Size** (n): The number of geometric objects.

14.1.6 Complexity

- **Maximum Length:** $\lambda_s(n)$ is near-linear for any fixed s . For example, $\lambda_3(n) = O(n\alpha(n))$.
- **Construction Time:** The lower envelope can be computed in $O(\lambda_s(n) \log n)$ time using a divide-and-conquer approach.

```

1: function LOWERENVELOPE(functions, start, end)
2:   if |functions| = 1 then
3:     return functions[0]
4:   end if
5:   mid  $\leftarrow$  |functions|/2
6:   left_env  $\leftarrow$  LowerEnvelope(functions[: mid], start, end)
7:   right_env  $\leftarrow$  LowerEnvelope(functions[mid :], start, end)
8:   merged_env  $\leftarrow$  MergeEnvelopes(left_env, right_env)
9:   return merged_env
10: end function
11: function MERGEENVELOPES( $E1$ ,  $E2$ )
12:   intersections  $\leftarrow$  FindIntersections( $E1$ ,  $E2$ )
13:   return ResultingSequence( $E1$ ,  $E2$ , intersections)
14: end function

```

14.1.7 Usages

- **Arrangements:** Bounding the number of edges on a single cell or the lower envelope of an arrangement of curves.
- **Motion Planning:** Analyzing the complexity of the “free space” for a robot moving among obstacles.
- **Visibility:** Calculating the complexity of the region visible from a point in a dynamic environment.
- **Kinetic Data Structures:** Tracking the minimum value among a set of moving points.
- **Shortest Paths:** Computing the visibility graph of a set of segments.

14.1.8 Testing Methods and Edge Cases

- **Non-Intersecting Functions:** The envelope should simply follow the lowest function for the entire range.
- **Multiple Intersections:** Ensure the algorithm correctly captures all points where the lower envelope switches from one function to another.
- **Touching vs. Crossing:** Handle cases where functions are tangent to each other without crossing.
- **Vertical Segments:** Ensure the interval logic handles vertical discontinuities.
- **Complexity Check:** Verify that the number of segments in the resulting envelope is within the DS bound $\lambda_s(n)$.

14.1.9 References

- Davenport, H., & Schinzel, A. (1965). “A combinatorial problem connected with differential equations”. *American Journal of Mathematics*.
- Sharir, M., & Agarwal, P. K. (1995). “Davenport–Schinzel Sequences and Their Geometric Applications”. Cambridge University Press.
- Atallah, M. J. (1985). “Some dynamic computational geometry problems”. *Computers & Mathematics with Applications*.

- Wikipedia: [Davenport–Schinzel sequence](#)

14.2 Lower Envelopes

14.2.1 Overview

The lower envelope of a set of functions $f_1, f_2, \dots, f_n$ is the pointwise minimum of those functions: $L(x) = \min_i f_i(x)$. In computational geometry, the lower envelope is a fundamental structure used to solve problems related to visibility, optimization, and arrangements. For example, the lower envelope of a set of lines forms the boundary of their intersection (a convex polygon in the dual plane). The computational aspects of envelopes were studied extensively by Edelsbrunner [Ede87] and Hershberger [Her89].

14.2.2 Definitions

Definition 14.7 (Lower Envelope). The **lower envelope** of a family of functions $\{f_1, \dots, f_n\}$ on a domain $D \subseteq \mathbb{R}$ is the function $L : D \rightarrow \mathbb{R}$ defined by $L(x) = \min_{1 \leq i \leq n} f_i(x)$. The envelope is piecewise composed of segments of the individual functions.

Definition 14.8 (Upper Envelope). The **upper envelope** of a family of functions $\{f_1, \dots, f_n\}$ is $U(x) = \max_{1 \leq i \leq n} f_i(x)$. The upper envelope of a set of functions is the lower envelope of their negations.

Definition 14.9 (Minimization Diagram). A **minimization diagram** is the partition of the domain D into maximal regions where a specific function f_i achieves the minimum. Each region is called a *cell* of the diagram, and the boundary between two cells is formed by the intersection points of the corresponding functions.

Definition 14.10 (Complexity of an Envelope). The **complexity** of a lower envelope is the number of maximal continuous pieces (arcs) that compose it. Each arc is a maximal connected subset of the domain on which a single function f_i achieves the pointwise minimum.

14.2.3 Theory

The complexity of the lower envelope depends on how many times any two functions can intersect.

1. **Lines:** Any two lines intersect at most once. The lower envelope of n lines is a convex chain with at most n segments.
2. **Line Segments:** Two segments can intersect at most once, but the envelope can have up to $O(n\alpha(n))$ segments due to the “visibility” of segments behind each other.
3. **Parabolas/Circles:** Two functions intersect at most twice. The envelope complexity is $O(n)$.
4. **General Curves:** If any two curves intersect at most s times, the complexity is bounded by the Davenport–Schinzel sequence $\lambda_s(n)$.

The lower envelope can be computed using a **Divide and Conquer** algorithm:

1. Divide the set of functions into two halves.
2. Recursively compute the lower envelope of each half.
3. Merge the two envelopes by finding their intersection points and keeping the lower segments.

Theorem 14.11 (Lower Envelope Complexity for Lines). *The lower envelope of n lines in $\mathbb{R}^2$ consists of at most n segments and can be computed in $O(n \log n)$ time.*

Proof. Any two distinct lines intersect in at most one point. By the DS sequence framework with $s = 1$, the envelope is a $DS(n, 1)$ sequence, which has length at most n . The divide-and-conquer algorithm solves the recurrence $T(n) = 2T(n/2) + O(n)$, giving $T(n) = O(n \log n)$. $\square$

Theorem 14.12 (Lower Envelope Complexity for Segments). *The lower envelope of n line segments in $\mathbb{R}^2$ has $O(n \alpha(n))$ complexity, where $\alpha(n)$ is the inverse Ackermann function, and can be computed in $O(n \alpha(n) \log n)$ time.*

Proof. Any two line segments intersect in at most one point (since they are linear), so the envelope is a DS sequence of order $s = 3$ (segments can cross, touch, or miss). The DS bound $\lambda_3(n) = O(n \alpha(n))$ gives the complexity upper bound. The divide-and-conquer algorithm merges two sub-envelopes in time proportional to their combined complexity times $\log n$, yielding $O(n \alpha(n) \log n)$ total time. See Edelsbrunner [Eds87, Chapter 4]. $\square$

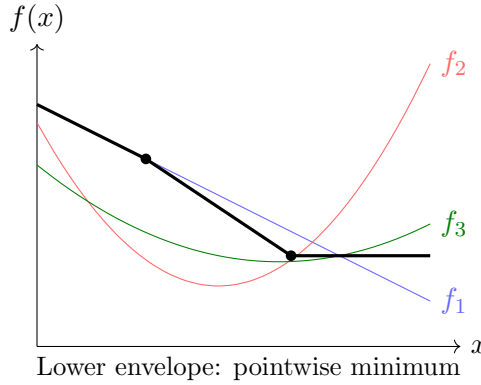


Figure 14.1: Lower envelope of three functions: the thick black curve is the pointwise minimum $\min\{f_1, f_2, f_3\}$.

Example 14.13 (Lower Envelope of Lines). Consider four lines: $f_1(x) = 2 - x$, $f_2(x) = -1 + 2x$, $f_3(x) = 3 - 0.5x$, $f_4(x) = x - 2$. The lower envelope is:

$$L(x) = \min(2 - x, -1 + 2x, 3 - 0.5x, x - 2)$$

On $(-\infty, 0)$: $f_4(x) = x - 2$ is the minimum. On $[0, 1]$: $f_1(x) = 2 - x$ is minimum. On $[1, 2]$: $f_2(x) = -1 + 2x$ is minimum. On $(2, \infty)$: $f_4(x) = x - 2$ is minimum again. The envelope has 4 pieces, which does not exceed $n = 4$.

14.2.4 Pseudocode

14.2.5 Parameter Selection

- **Function Type:** Linear, polynomial, or arbitrary continuous functions.
- **Intersection Solver:** A robust numerical or symbolic solver is needed to find where $f_i(x) = f_j(x)$.

14.2.6 Complexity

- **Complexity of Envelope:** $O(\lambda_s(n))$ where s is the number of intersections.
- **Construction Time:** $O(\lambda_s(n) \log n)$ using divide and conquer.
- **Space Complexity:** $O(\lambda_s(n))$ to store the resulting segments.

```
1: function COMPUTELOWERENVELOPE(functions,  $x_{\min}$ ,  $x_{\max}$ )
2:   if |functions| = 1 then
3:     return [(functions[0],  $x_{\min}$ ,  $x_{\max}$ )]
4:   end if
5:   mid  $\leftarrow$  |functions|/2
6:   left_env  $\leftarrow$  ComputeLowerEnvelope(functions[: mid],  $x_{\min}$ ,  $x_{\max}$ )
7:   right_env  $\leftarrow$  ComputeLowerEnvelope(functions[mid :],  $x_{\min}$ ,  $x_{\max}$ )
8:   merged  $\leftarrow$  []
9:   for segment_L in left_env do
10:    for segment_R in right_env do
11:      common_start  $\leftarrow$  max(segment_L.start, segment_R.start)
12:      common_end  $\leftarrow$  min(segment_L.end, segment_R.end)
13:      if common_start < common_end then
14:        intersections  $\leftarrow$  FindIntersections(segment_L.f, segment_R.f, common_start, common_end)
15:        merged.extend(PickLower(segment_L.f, segment_R.f, common_start, common_end, intersections))
16:      end if
17:    end for
18:  end for
19:  return Simplify(merged)
20: end function
```

14.2.7 Usages

- **Visibility:** Calculating the “horizon” of a set of mountains or the region visible from a point.
- **Motion Planning:** Finding the safest path by maximizing the distance to the nearest obstacle (lower envelope of distance functions).
- **Duality:** Solving problems like the “Width of a Point Set” or “Diameter” in the dual plane.
- **Dynamic Programming:** Optimizing decisions over time where costs are modeled as functions.
- **Statistics:** Calculating the “support” of a probability distribution.

14.2.8 Testing Methods and Edge Cases

- **Non-Intersecting Functions:** The envelope should simply be the single function that is globally minimum.
- **Vertical Functions:** Ensure the algorithm handles discontinuities or vertical lines.
- **Touching Functions:** Verify that tangencies (where functions touch but don’t cross) don’t create unnecessary segments.
- **Overlap:** Handle cases where two functions are identical over an interval.
- **Large n :** Verify the near-linear growth of the result size.

14.2.9 References

- Sharir, M., & Agarwal, P. K. (1995). “Davenport–Schinzel Sequences and Their Geometric Applications”. Cambridge University Press.

- Hershberger, J. (1989). “Finding the upper envelope of n line segments in $O(n \log n)$ time”. Information Processing Letters.
- Edelsbrunner, H. (1987). “Algorithms in Combinatorial Geometry”. Springer-Verlag.
- Wikipedia: [Lower envelope](#)

14.3 Levels in Arrangements and Half-Plane Discrepancy

14.3.1 Overview

In an arrangement of lines, every point not on a line has a well-defined *level*: the number of lines strictly below it. The set of points of a fixed level is a polygonal chain called the **k-level**, which generalizes the lower envelope (level 0) and the upper envelope (level $n - 1$). Levels are the bridge between arrangements and point sets: by duality, the vertices of the k -level of an arrangement of lines correspond exactly to the **k-sets** of the dual point set—subsets of size k separable from the rest by a line. Levels also control the **half-plane discrepancy** of a point set, a measure of how well a set of n points approximates the uniform distribution that arises, for example, in the supersampling analysis of ray tracing [SA95].

14.3.2 Definitions

Definition 14.14 (Level of a Point). In an arrangement $\mathcal{A}(\mathcal{L})$ of n lines, the *level* of a point p that lies on no line is the number of lines of $\mathcal{L}$ strictly below p . The k -level is the set of points whose level is exactly k ; it is an x -monotone polygonal chain. The *complexity* of the k -level is its number of vertices.

Definition 14.15 (k-Set). A k -set of a point set P is a subset $K \subseteq P$ with $|K| = k$ that can be separated from $P \setminus K$ by a line. Under point–line duality, the k -sets of P correspond one-to-one with the vertices of the k -th level of the arrangement of lines dual to P .

Definition 14.16 (Half-Plane Discrepancy). Let P be a set of n points inside a region B of area 1. The *half-plane discrepancy* of P is

$$D(P) = \max_{h \text{ half-plane}} \left| |P \cap h| - n \cdot \text{area}(h \cap B) \right|.$$

The discrepancy $D(n)$ is the minimum of $D(P)$ over all n -point sets P .

14.3.3 Theory

The lower envelope studied in Section 14.2 is precisely the 0-level, and the upper envelope is the $(n - 1)$ -level. The complexity of intermediate levels is the central open problem of the area; the best known results are as follows.

Theorem 14.17 (Complexity of the k-Level). *The complexity of the k -th level in an arrangement of n lines is $O(nk^{1/3})$ for every k [Dey98]; for the middle level ($k \approx n/2$) this is $O(n^{4/3})$. The best known lower bound is $n 2^{\Omega(\sqrt{\log k})}$.*

Proof sketch. The older bound $O(n\sqrt{k})$ of Lovász follows by a potential-function argument: sweep the arrangement and charge the $\Omega(\sqrt{k})$ left-turning vertices of the level. Dey’s improvement applies the crossing lemma to the union of k x -monotone concave chains that cover the region below the level, yielding the $O(nk^{1/3})$ bound [Dey98]. $\square$

The connection between levels and discrepancy is that a half-plane whose incidence count deviates strongly from the area expectation must contain many vertices of a level in the dual arrangement; hence a low-discrepancy point set forces every level to be simple.

Theorem 14.18 (Half-Plane Discrepancy of n Points). *The half-plane discrepancy of every set of n points is at least $cn^{1/4}$, and there exist n -point sets (such as suitable lattices) whose half-plane discrepancy is $O(n^{1/4})$. In particular $D(n) = \Theta(n^{1/4})$.*

Proof sketch. The lower bound is due to Beck and Alexander using Fourier-analytic (integral-geometric) techniques [Mat99]. The matching upper bound is due to Matoušek, who constructed point sets—related to scaled integer lattices—for which every half-plane counts the expected number of points up to an error of $O(n^{1/4})$ [Mat95, Mat99]. $\square$

14.3.4 Pseudocode

A simple incremental construction of the k -level of an arrangement of n lines uses the duality between levels and k -sets.

Algorithm 41 Compute the k -Level of a Line Arrangement

```

1: function COMPUTEKLEVEL( $lines, k$ )
2:    $\mathcal{A} \leftarrow \text{LineArrangement}(lines)$ 
3:    $level \leftarrow 0$ 
4:    $p \leftarrow$  point at  $x = -\infty$  on the lowest line of  $\mathcal{A}$ 
5:   while  $x \neq +\infty$  do
6:     if  $level = k$  then
7:       append the current edge to the output chain
8:     end if
9:     advance to the next vertex of  $\mathcal{A}$ 
10:     $level \leftarrow level \pm 1$   $\triangleright$  crossing a line
11:  end while
12:  return the output chain
13: end function

```

14.3.5 Parameter Selection

- **Level k :** choose $k = 0$ for the lower envelope and $k = n - 1$ for the upper envelope; intermediate k give the k -sets.
- **Duality:** for discrepancy problems, dualize the point set to lines and operate on levels of the arrangement.

14.3.6 Complexity

- **Level Complexity:** $O(nk^{1/3})$ vertices (Dey); the middle level has $O(n^{4/3})$ complexity.
- **Discrepancy:** $D(n) = \Theta(n^{1/4})$ for half-planes.
- **Construction Time:** $O(n^2)$ for a full arrangement; a single level can be traced in time proportional to its complexity plus the sorting of the lines.

14.3.7 Usages

- **Ray tracing:** bounding the discrepancy of sampling patterns for anti-aliased rendering.
- **k -sets:** counting the number of subsets of size k separable by a line (used in statistical classification and pattern recognition).

- **Geometric data analysis:** measuring how evenly a point set approximates a uniform distribution over a region.
- **Range counting:** levels dualize to half-plane range counting, connecting arrangements to the simplex range searching chapter.

14.3.8 Testing Methods and Edge Cases

- **Parallel lines:** the k -level may be empty or unbounded.
- **Multiple lines through one point:** perturb or handle coincident vertices consistently.
- **Grid construction:** verify that a $m \times m$ grid point set indeed achieves discrepancy $O(m^{1/2}) = O(n^{1/4})$.
- **Small n :** for $n = 4$, any set has a half-plane containing 3 of the points when all four are in convex position, matching the $n^{1/4}$ order.

14.3.9 References

- Dey, T. K. (1998). “Improved bounds on planar k -sets and related problems”. Discrete & Computational Geometry.
- Matoušek, J. (1995). “Tight upper bounds for the discrepancy of half-spaces”. Discrete & Computational Geometry.
- Matoušek, J. (1999). “Geometric Discrepancy: An Illustrated Guide”. Springer.
- Sharir, M., & Agarwal, P. K. (1995). “Davenport–Schinzel Sequences and Their Geometric Applications”. Cambridge University Press.

14.4 Union Volume Estimation

14.4.1 Overview

The union volume estimation problem, also known as **Klee’s measure problem**, involves calculating the total volume (or area in 2D) occupied by the union of n geometric objects (typically axis-aligned boxes). This is a fundamental challenge because the union can have a very complex topology even if the individual objects are simple. The problem was posed by Klee [Kle77], and significant algorithmic advances were made by Overmars and Yap [OY91] and Bringmann [Bri12]. While exact algorithms exist for boxes, estimation methods are often preferred for more complex shapes like spheres or non-convex meshes.

14.4.2 Definitions

Definition 14.19 (Union Measure). For a family of n measurable sets $\{O_1, \dots, O_n\}$ in $\mathbb{R}^d$, the **union measure** is

$$\mu\left(\bigcup_{i=1}^n O_i\right),$$

where μ denotes Lebesgue measure (length in 1D, area in 2D, volume in 3D).

Definition 14.20 (Monte Carlo Estimation). A **Monte Carlo estimator** for the volume of a region $\Omega \subset B$ (where B is a bounding box of known volume V_B) is

$$\hat{V} = V_B \cdot \frac{k}{N},$$

where N points are sampled uniformly at random from B and k of them fall inside Ω . The estimator is unbiased, with $\mathbb{E}[\hat{V}] = V(\Omega)$ and standard deviation $\sigma = V_B \sqrt{\frac{V(\Omega)(V_B - V(\Omega))}{N \cdot V_B^2}}$.

Definition 14.21 (Sweep-Line Algorithm). A **sweep-line algorithm** for union area computation processes events at all unique x -coordinates of vertical edges. Between consecutive events, the union area reduces to computing the union of 1D intervals along the y -axis.

14.4.3 Theory

Exact 2D Calculation (Sweep-Line)

For n axis-aligned rectangles, the exact area can be calculated in $O(n \log n)$ time:

1. Identify all unique x -coordinates of the vertical edges. These form a set of “strips.”
2. For each strip, find the total length of the union of 1D intervals along the y -axis (using a Segment Tree).
3. The area is the sum of (strip width $\times$ union length).

Monte Carlo Estimation (N -Dimensions)

For more complex or higher-dimensional shapes:

1. Enclose all objects in a large bounding box B with known volume V_B .
2. Sample N random points $P_1, \dots, P_N$ inside B .
3. For each point P_i , check if it lies inside **any** object O_j (using a spatial index like an R-tree for speed).
4. If k points are inside the union, the estimated volume is $\hat{V} = V_B \cdot \frac{k}{N}$.

Theorem 14.22 (Sweep-Line Union Complexity). *The area of the union of n axis-aligned rectangles in $\mathbb{R}^2$ can be computed in $O(n \log n)$ time and $O(n)$ space.*

Proof. The algorithm sweeps a vertical line from left to right across all $2n$ unique x -coordinates of vertical edges. At each event, it adds or removes a horizontal interval from a 1D interval set. The union length of the active intervals is maintained using a segment tree, supporting $O(\log n)$ insertions, deletions, and queries per event. With $O(n)$ events, the total time is $O(n \log n)$. Space is $O(n)$ for the interval set and event queue. See Klee [Kle77] for the original formulation. $\square$

Theorem 14.23 (Klee’s Measure Problem Bounds). *In d dimensions with $d \geq 3$, the exact measure of the union of n axis-aligned boxes can be computed in $O(n^{d/2})$ time for even d and $O(n^{(d+1)/2})$ time for odd d , improving upon the naive $O(n^d)$ approach.*

Proof. The result follows from the observation that in d dimensions, the complexity of the union of n boxes is $O(n^{\lfloor d/2 \rfloor})$ by the Davenport–Schinzel framework applied level by level. Overmars and Yap [OY91] achieved $O(n^{d/2})$ for even d , and Bringmann [Bri12] improved bounds for small fixed d . The algorithm uses a multi-level data structure that maintains partial unions across dimensions. $\square$

Example 14.24 (Monte Carlo Volume Estimation). Estimate the area of the union of two axis-aligned squares: $S_1 = [0, 2] \times [0, 2]$ (area 4) and $S_2 = [1, 3] \times [1, 3]$ (area 4), overlapping on $[1, 2] \times [1, 2]$ (area 1). The true union area is $4 + 4 - 1 = 7$.

Bounding box: $B = [0, 3] \times [0, 3]$, $V_B = 9$. Sample $N = 1000$ random points from B . Suppose $k = 778$ fall inside $S_1 \cup S_2$. Then $\hat{V} = 9 \cdot \frac{778}{1000} = 7.002$, which is close to the true value of 7.

14.4.4 Pseudocode

Monte Carlo Estimation

```

1: function ESTIMATEUNIONVOLUME(objects, num_samples)
2:   bbox  $\leftarrow$  Union([obj.bbox | obj  $\in$  objects])
3:   total_volume  $\leftarrow$  bbox.volume()
4:   index  $\leftarrow$  BuildRTree(objects)
5:   inside_count  $\leftarrow$  0
6:   for _ in range(num_samples) do
7:     p  $\leftarrow$  bbox.RandomPoint()
8:     if index.IsPointInsideAny(p) then
9:       inside_count  $\leftarrow$  inside_count + 1
10:    end if
11:  end for
12:  return total_volume  $\cdot$  (inside_count/num_samples)
13: end function

```

14.4.5 Parameter Selection

- **Sample Count (N):** Higher N leads to lower variance but increased computation. Accuracy improves as $1/\sqrt{N}$.
- **Bounding Box:** A “tight” bounding box reduces the number of samples that fall outside the union, improving efficiency.

14.4.6 Complexity

- **Exact 2D:** $O(n \log n)$ using sweep-line.
- **Exact 3D:** $O(n^{1.5})$ or $O(n \log n)$ with complex data structures.
- **Monte Carlo:** $O(N \cdot \log n)$ where N is samples and n is objects.

14.4.7 Usages

- **CAD/CAM:** Calculating the material required for a complex part built from boolean primitives.
- **Computational Biology:** Estimating the volume of a protein or a cellular structure modeled as a union of spheres (Van der Waals volume).
- **Robotics:** Calculating the total “reachable volume” of a robot arm.
- **GIS:** Determining the total area covered by a set of circular sensor ranges.
- **Computer Graphics:** Ambient occlusion and visibility calculations.

14.4.8 Testing Methods and Edge Cases

- **Disjoint Objects:** The union volume should be the sum of individual volumes.
- **Identical Objects:** The union volume should be equal to a single object’s volume.
- **Nested Objects:** Ensure that a small object inside a large one doesn’t add to the volume.

- **Analytical Shapes:** Test with spheres or boxes where the exact union volume can be calculated manually.
- **High Overlap:** Verify that the algorithm doesn't double-count overlapping regions.

14.4.9 References

- Klee, V. (1977). "Can the measure of $\cup[a_i, b_i]$ be computed in less than $O(n \log n)$ steps?". American Mathematical Monthly.
- Overmars, M. H., & Yap, C. K. (1991). "New upper bounds in Klee's measure problem". SIAM Journal on Computing.
- Bringmann, K. (2012). "New upper bounds for Klee's measure problem in small dimensions". Computational Geometry.
- Wikipedia: [Klee's measure problem](#)

Chapter 15

Geometric Duality

- 15.1 Point-Line Duality
- 15.2 Incidence Preservation
- 15.3 Order Reversal
- 15.4 Dual Arrangements
- 15.5 Convex Hulls through Duality
- 15.6 Half-Plane Intersection
- 15.7 Linear Programming through Duality
- 15.8 Applications to Optimization
- 15.9 Exercises

Part VI

Convexity and Geometric
Optimization

Chapter 16

Convex Polytopes

- 16.1 Convex Polytopes
- 16.2 V-Representations and H-Representations
- 16.3 Faces and Face Lattices
- 16.4 Supporting Hyperplanes
- 16.5 Separating Hyperplane Theorems
- 16.6 Carathéodory's Theorem
- 16.7 Helly's Theorem
- 16.8 Radon's Theorem
- 16.9 Euler–Poincaré Relations
- 16.10 Polarity
- 16.11 Computing Higher-Dimensional Hulls
- 16.12 Exercises

Chapter 17

Linear Programming in Geometry

Linear programming provides a common language for geometric optimization. Many geometric problems ask for a point satisfying linear inequalities while maximizing or minimizing a linear objective. In two dimensions, the feasible region is an intersection of half-planes and therefore a convex polygon.

17.1 Geometric Linear Programs

A linear program has the form

$$\text{maximize } c^T x \quad \text{subject to} \quad Ax \leq b.$$

Each row of $Ax \leq b$ defines a half-plane. The feasible set is convex, and when it is non-empty and bounded, an optimum occurs at a vertex of the feasible polytope. In two dimensions this reduces optimization to a convex polygon; in three dimensions it reduces to a convex polyhedron.

Applications include smallest enclosing regions, collision separation, supporting-line queries, facility placement, and motion-planning constraints.

17.2 Half-Plane Intersection

For an oriented line through points p and q , the left half-plane is

$$H(p, q) = \{x \in \mathbb{R}^2 : \text{orient2d}(p, q, x) \geq 0\}.$$

The intersection of a finite collection of half-planes is a convex polygon, possibly empty or unbounded. A bounding box can be added when a finite output polygon is required.

One robust approach sorts the boundary lines by direction and maintains a deque of candidate lines. Whenever the intersection of the last two lines is outside the next half-plane, the last line is removed. A symmetric check is performed at the front of the deque.

After sorting, the deque algorithm takes $O(n)$ time and $O(n)$ space. Parallel or coincident boundary lines require a consistent rule: retain the more restrictive half-plane when their feasible sides agree, and report an empty intersection when they conflict.

17.3 Low-Dimensional Linear Programming

In fixed dimension, randomized incremental algorithms solve linear programs efficiently by processing constraints in random order. When a new constraint violates the current optimum, the optimum must lie on that constraint's boundary, reducing the problem dimension by one.

Algorithm 42 Half-Plane Intersection

```
1: function HALFPLANEINTERSECTION( $H$ )
2:   Sort half-planes by boundary direction
3:    $D \leftarrow$  empty deque
4:   for  $h$  in  $H$  do
5:     while  $|D| \geq 2$  and intersection of the last two lines is outside  $h$  do
6:       Remove the last half-plane from  $D$ 
7:     end while
8:     while  $|D| \geq 2$  and intersection of the first two lines is outside  $h$  do
9:       Remove the first half-plane from  $D$ 
10:    end while
11:    Append  $h$  to  $D$ 
12:  end for
13:  while  $|D| \geq 3$  and the last intersection is outside the first half-plane do
14:    Remove the last half-plane from  $D$ 
15:  end while
16:  while  $|D| \geq 3$  and the first intersection is outside the last half-plane do
17:    Remove the first half-plane from  $D$ 
18:  end while
19:  return polygon formed by consecutive boundary intersections
20: end function
```

This gives expected linear time for two-dimensional linear programming and expected linear time for fixed-dimensional LP under the usual boundedness and constant-dimension assumptions.

The same boundary-reduction idea appears in smallest-enclosing-circle algorithms, support-point queries, and randomized convex-hull construction. It is therefore a useful bridge between predicates, convexity, optimization, and geometric data structures.

17.4 Duality and Applications

Point-line duality maps a point (a, b) to a line such as $y = ax - b$. Under a consistent duality transform, incidence and above/below relationships are preserved or reversed in a controlled way. This converts convex-hull and half-plane questions into upper- or lower-envelope questions.

Linear constraints also describe collision-free configuration-space regions, visibility restrictions, separating planes, and feasible placement regions. Numerical implementations should use the robust orientation predicates from Chapter 3 when testing boundary intersections.

Chapter 18

Intersection and Distance of Convex Objects

18.1 Convex Polygon Intersection

18.2 Separating Axis Theorem

18.3 Distance between Convex Sets

18.4 Support Functions

18.5 Minkowski Difference

18.6 Gilbert–Johnson–Keerthi Algorithm

18.7 Penetration Depth

18.8 Collision Detection

18.9 Continuous Collision Detection

18.10 Applications in Robotics and Simulation

18.11 Exercises

18.12 Enclosure and Convex Distance

18.13 Welzl’s Algorithm

18.13.1 Overview

Welzl’s algorithm is a randomized algorithm for finding the smallest enclosing circle (the minimum-area circle that contains all points in a set). Originally described by Emo Welzl in 1991 [Wel91], it is a brilliant application of the linear programming paradigm to a geometric problem. The algorithm builds on earlier linear-time results for fixed-dimension linear programming by Megiddo [Meg83]. It achieves an expected $O(n)$ time complexity, making it highly efficient for point sets in 2D.

18.13.2 Definitions

Definition 18.1 (Smallest Enclosing Circle). The **smallest enclosing circle (SEC)** of a point set $P \subset \mathbb{R}^2$ is the unique circle of minimum radius that contains every point of P . The SEC is also called the *minimum covering circle* or *minidisk*.

Definition 18.2 (Support Points). The **support points** of the SEC are the points of P that lie on the boundary of the SEC. The SEC is uniquely determined by at most three support points: a single point (the center), two points (the circle having them as diameter), or three non-collinear points (the circumcircle).

18.13.3 Theory

The algorithm is based on a recursive reduction of the point set. The key insight is that if a new point P_i is already inside the SEC of the previous $i - 1$ points, then the SEC remains the same. If P_i is outside, it **MUST** lie on the boundary of the new SEC. This allows us to recursively solve the problem with P_i fixed as a “boundary point.”

The recursion continues until we have a set of 0, 1, 2, or 3 points that must lie on the boundary. These “fixed” points uniquely define the circle (0: none, 1: point as center, 2: segment as diameter, 3: circumcircle).

Theorem 18.3 (Expected Linear Time). *Welzl’s algorithm computes the smallest enclosing circle of n points in $\mathbb{R}^2$ in expected $O(n)$ time.*

Proof. The proof uses a backwards analysis argument. Fix a random permutation π of the n points. Consider the moment when the algorithm first inserts a point $p_{\pi(i)}$ onto the boundary (i.e., the recursion rejects the current disk and adds $p_{\pi(i)}$ to R). By backwards analysis, conditioning on this event, $p_{\pi(i)}$ is equally likely to be any of the i points seen so far. The expected number of boundary insertions across all n points is $\sum_{i=1}^n \frac{3}{i} = O(\log n)$ (since at most 3 points are ever on the boundary and each insertion reduces the remaining budget by 1). The total work is therefore $O(n + \log n) = O(n)$. See [Wel91] and Megiddo [Meg83] for the full analysis. $\square$

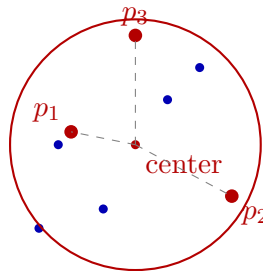


Figure 18.1: Welzl’s algorithm: the smallest enclosing circle is defined by at most 3 support points (red).

Example 18.4 (Welzl Recursion Trace). Consider points $P = \{(0, 0), (4, 0), (2, 3), (2, 1)\}$ processed in order:

1. Recurse with $R = \{\}$: pop $(2, 1)$, compute SEC of remaining 3 points. The SEC of $\{(0, 0), (4, 0), (2, 3)\}$ is the circumcircle: center $(2, 0.833)$, radius ≈ 2.404 .
2. Check $(2, 1)$: distance from center is $\approx 0.167 < 2.404$ —inside! Return this SEC.

The algorithm finishes in one pass because $(2, 1)$ happens to be interior to the circumcircle of the other three.

18.13.4 Pseudocode

```

1: function WELZL( $P$ ,  $R = []$ )
2:   if  $|P| = 0$  or  $|R| = 3$  then
3:     return CircleFromBoundary( $R$ )
4:   end if
5:    $p \leftarrow P.\text{pop}()$ 
6:    $D \leftarrow \text{Welzl}(P.\text{copy}(), R)$ 
7:   if  $p$  is in  $D$  then
8:     return  $D$ 
9:   end if
10:  return Welzl( $P.\text{copy}(), R + [p]$ )
11: end function

```

▷ To start:

```

12: random.shuffle(points)
13: SEC  $\leftarrow$  Welzl(points)

```

18.13.5 Parameter Selection

- **Randomization:** Shuffling the input points at the beginning is crucial. Without randomization, structured input (like points on a line) can cause $O(n^2)$ worst-case performance.
- **Numerical Precision:** Circumcircle calculations for 3 points can be sensitive to collinearity. An epsilon should be used to check if 3 points are nearly on a line.

18.13.6 Complexity

- **Time Complexity:** $O(n)$ expected. Although the recursion depth is n , the probability that a point falls outside the current circle decreases rapidly, leading to the $O(n)$ bound.
- **Space Complexity:** $O(n)$ for the recursion stack and the copies of the point set.

18.13.7 Usages

- **Collision Detection:** Generating a bounding sphere for an object as a fast first-pass check [Eri04].
- **Facility Location:** Finding the center for a new resource (e.g., a cell tower) to cover all clients with minimum range.
- **Cluster Analysis:** Determining the spatial extent of a group of points.
- **Camera Control:** Finding the minimum zoom/pan to keep all characters in a scene visible.

18.13.8 Testing Methods and Edge Cases

- **Collinear Points:** The SEC should have its diameter equal to the distance between the two furthest points.
- **Duplicate Points:** Handled naturally by the inclusion check.
- **Small Sets:** Verify with 0, 1, and 2 points.

- **Symmetry:** Regular polygons should result in a circle centered at the polygon’s centroid.
- **Redundant Points:** Verify that a million points inside a circle don’t change the SEC defined by 3 points on the boundary.

18.13.9 References

- Welzl, E. (1991). “Smallest enclosing disks (balls and ellipsoids)”. Lecture Notes in Computer Science.
- Megiddo, N. (1983). “Linear-time algorithms for linear programming in $\mathbb{R}^3$ and related problems”. SIAM Journal on Computing.
- Wikipedia: [Smallest-circle problem](#)

18.14 Convex Polygon Distance

18.14.1 Overview

The convex polygon distance problem involves finding the minimum Euclidean distance between two non-intersecting convex polygons P and Q . This is a fundamental operation in collision detection and motion planning. While a naive $O(n \cdot m)$ approach checks all pairs of edges, more efficient algorithms leverage the convexity of the shapes to achieve $O(n + m)$ or even $O(\log n + \log m)$ performance. The rotating calipers approach was popularized by Toussaint [Tou83], and the GJK algorithm was introduced by Gilbert, Johnson, and Keerthi [GJK88]. Edelsbrunner [Eds85] provides further complexity bounds.

18.14.2 Definitions

Definition 18.5 (Euclidean Distance between Polygons). The **Euclidean distance** between two polygons P and Q is

$$\text{dist}(P, Q) = \min_{p \in P, q \in Q} \|p - q\|_2.$$

For non-intersecting convex polygons, this minimum is achieved between two vertices or a vertex and an edge.

Definition 18.6 (Separating Axis). A **separating axis** is a line ℓ such that the orthogonal projections of two convex shapes P and Q onto ℓ do not overlap. By the separating axis theorem, two convex shapes do not intersect if and only if a separating axis exists.

Definition 18.7 (Minkowski Difference). The **Minkowski difference** of two sets $P, Q \subset \mathbb{R}^2$ is

$$P \ominus Q = \{p - q \mid p \in P, q \in Q\}.$$

For convex polygons with n and m vertices, $P \ominus Q$ is a convex polygon with at most $n + m$ vertices. The distance $\text{dist}(P, Q)$ equals the distance from the origin to $P \ominus Q$.

Definition 18.8 (GJK Algorithm). The **Gilbert–Johnson–Keerthi (GJK) algorithm** computes the distance between two convex shapes by iteratively constructing a simplex inside the Minkowski difference $P \ominus Q$ that converges toward the origin. If the origin lies inside $P \ominus Q$, the shapes intersect.

18.14.3 Theory

The most common approach for linear-time distance calculation is based on the **Rotating Calipers** method.

1. Find the pair of points (one on each polygon) that minimize the distance. For non-intersecting convex polygons, this minimum distance occurs between two vertices, or a vertex and an edge.
2. Use the rotating calipers technique to “crawl” around both polygons simultaneously. Because the polygons are convex, the distance function is unimodal along the boundary, allowing us to find the global minimum in a single pass.

Another powerful approach is the **GJK Algorithm**:

1. Compute the **Minkowski Difference** $D = P \ominus Q$.
2. The distance between P and Q is the distance from the origin $(0,0)$ to the set D .
3. GJK iteratively builds a simplex inside D that gets closer and closer to the origin.

Theorem 18.9 (Convex Distance Lower Bound). *For two convex polygons P with n vertices and Q with m vertices (both in convex position), the minimum Euclidean distance can be computed in $O(n + m)$ time using rotating calipers, and in $O(\log n + \log m)$ time using binary search if the polygons have been preprocessed.*

Proof. The $O(n + m)$ bound follows from the rotating calipers observation: at each step, at least one caliper advances to the next vertex, and the total number of advances is at most $n + m$. The distance between the two caliper positions is evaluated at each step, and the minimum is returned. The $O(\log n + \log m)$ bound follows from the fact that for each edge of one polygon, the distance to the other polygon is a unimodal function along its boundary, which can be minimized by binary search. $\square$

18.14.4 Pseudocode

Rotating Calipers for Distance

```

1: function CONVEXDISTANCE( $P, Q$ )
2:    $i \leftarrow \text{index\_of\_min\_y}(P)$ 
3:    $j \leftarrow \text{index\_of\_max\_y}(Q)$ 
4:    $\text{min\_dist} \leftarrow \infty$ 
5:   for  $\_$  in  $\text{range}(|P| + |Q|)$  do
6:      $d \leftarrow \text{Distance}(P[i], Q[j])$ 
7:      $\text{min\_dist} \leftarrow \min(\text{min\_dist}, d)$ 
8:      $\text{angle\_P} \leftarrow \text{AngleOfEdge}(P, i)$ 
9:      $\text{angle\_Q} \leftarrow \text{AngleOfEdge}(Q, j)$ 
10:    if  $\text{angle\_P} < \text{angle\_Q}$  then
11:       $i \leftarrow (i + 1) \bmod |P|$ 
12:    else
13:       $j \leftarrow (j + 1) \bmod |Q|$ 
14:    end if
15:  end for
16:  return  $\text{min\_dist}$ 
17: end function

```

Example 18.10 (Rotating Calipers Walkthrough). Consider $P = \{(0, 0), (3, 0), (3, 2), (0, 2)\}$ (a 3×2 rectangle) and $Q = \{(5, 1), (7, 1), (7, 3), (5, 3)\}$ (a 2×2 rectangle). Both are CCW.

1. Start: $i = \text{index of min } y \text{ in } P = 0$ (point $(0, 0)$), $j = \text{index of max } y \text{ in } Q = 2$ (point $(7, 3)$).
2. Distance from $(0, 0)$ to $(7, 3)$: $\sqrt{49 + 9} = \sqrt{58} \approx 7.62$. Advance i (edge $P[0] \rightarrow P[1]$ has angle 0° , smaller than Q 's edge).
3. Distance from $(3, 0)$ to $(7, 3)$: $\sqrt{16 + 9} = 5.0$. Advance j .
4. Continue until both calipers complete a full cycle. The minimum distance is 2.0 (from edge $x = 3$ of P to edge $x = 5$ of Q).

18.14.5 Parameter Selection

- **Polygon Orientation:** Polygons must be consistently oriented (e.g., both CCW).
- **Initial Points:** Starting at extreme points ensures the calipers are correctly synchronized.

18.14.6 Complexity

- **Time Complexity:** $O(n + m)$ using rotating calipers or GJK. Binary search methods can achieve $O(\log n + \log m)$ for preprocessed polygons.
- **Space Complexity:** $O(1)$ auxiliary space if the polygons are already stored.

18.14.7 Usages

- **Collision Avoidance:** Maintaining a “safety buffer” between a robot and obstacles.
- **Physics Simulation:** Calculating the time of impact or the depth of penetration.
- **Tolerance Analysis:** Determining if two mechanical parts will fit together with a required clearance.
- **UI Layout:** Calculating the spacing between complex geometric buttons or containers.

18.14.8 Testing Methods and Edge Cases

- **Intersecting Polygons:** The distance should be 0. (GJK handles this by detecting that the origin is inside the Minkowski difference).
- **Parallel Edges:** The minimum distance might be achieved along an entire segment.
- **Points and Lines:** Test with “polygons” that are actually single points or line segments.
- **Very Thin Polygons:** Ensure numerical stability for nearly collinear vertices.
- **Vast Scales:** Precision check for polygons that are very far apart.

18.14.9 References

- Toussaint, G. T. (1983). “Solving geometric problems with the rotating calipers”. Proceedings of MELECON '83.
- Gilbert, E. G., Johnson, D. W., & Keerthi, S. S. (1988). “A fast procedure for computing the distance between complex objects in three-dimensional space”. IEEE Journal on Robotics and Automation.
- Edelsbrunner, H. (1985). “Computing the extreme distances between two convex polygons”. Journal of Algorithms.
- Wikipedia: [Gilbert–Johnson–Keerthi distance algorithm](#)

Part VII

Curves, Surfaces, and Three-Dimensional Geometry

Chapter 19

Computational Geometry in Three Dimensions

The convex hull of a finite set of points in $\mathbb{R}^3$ is the smallest convex polyhedron containing them. It is the three-dimensional analogue of the planar convex hull and is a foundation for collision detection, visibility, solid modeling, meshing, and geometric optimization.

19.1 Polyhedral Representation

A polyhedron can be represented by its vertices, edges, and polygonal faces, or by half-spaces of the form $a_i^\top x \leq b_i$. A valid boundary representation records face cycles and the adjacency between faces and edges. For a closed connected convex polyhedron, Euler's formula is

$$V - E + F = 2.$$

The boundary orientation must be consistent so that all outward normals point in the same direction.

19.2 Three-Dimensional Predicates

The signed volume test

$$\text{orient3d}(A, B, C, D) = \det \begin{pmatrix} (B - A)^\top \\ (C - A)^\top \\ (D - A)^\top \end{pmatrix}$$

determines which side of the oriented plane ABC contains D . A positive value indicates one side, a negative value the other, and zero coplanarity. Robust evaluation is essential: an incorrect sign can create a non-manifold polyhedron or reverse a face during hull construction.

19.3 Incremental Three-Dimensional Hulls

An incremental hull algorithm starts with a non-degenerate tetrahedron and inserts points one at a time. For each point, it finds the faces visible from that point, removes those faces, extracts the horizon cycle between visible and invisible faces, and connects the new point to the horizon edges.

The visible-face search can be accelerated with a conflict graph storing the points visible from each face. With suitable randomized ordering, expected running time is $O(n \log n + k)$ for n input points and k output features; the worst case is output-sensitive and can be quadratic for poorly ordered inputs.

Algorithm 43 Incremental Three-Dimensional Convex Hull

```
1: function CONVEXHULL3D( $P$ )
2:   Select four non-coplanar points and initialize tetrahedron  $H$ 
3:   for each remaining point  $p$  do
4:      $V \leftarrow$  faces of  $H$  visible from  $p$ 
5:     if  $V$  is non-empty then
6:        $R \leftarrow$  boundary edges between visible and invisible faces
7:       Remove  $V$  from  $H$ 
8:       for each edge  $(u, v)$  in  $R$  do
9:         Add face  $(u, v, p)$  with consistent outward orientation
10:      end for
11:    end if
12:  end for
13:  return  $H$ 
14: end function
```

19.4 Degeneracies and Validation

Implementations must define behavior for coplanar points, duplicate points, collinear subsets, cospherical configurations, and points lying on existing faces. Symbolic perturbation or exact predicates can make the combinatorial result deterministic. A resulting hull should be checked for positive face orientation, edge incidence, connectedness, Euler's formula, and containment of every input point.

19.5 Applications

Three-dimensional convex hulls provide collision proxies, bounding volumes, support mappings for GJK, visibility boundaries, polyhedral approximations, and input domains for tetrahedral and hexahedral mesh generation.

Chapter 20

Curves and Surfaces

20.1 Surface Modeling and Mesh Generation

20.2 Bézier Surfaces

20.2.1 Overview

A Bézier surface is a smooth surface used in computer graphics and computer-aided design (CAD) that is controlled by a rectangular grid of points, called control points [B68, Far02]. It is a 2D extension of the Bézier curve. The surface is defined by a pair of parameters (u, v) , both in the range $[0, 1]$. Evaluating a Bézier surface means calculating the 3D coordinates (x, y, z) corresponding to a given (u, v) coordinate.

20.2.2 Definitions

Definition 20.1 (Control Points). A grid of $m + 1$ by $n + 1$ points P_{ij} in 3D space that define the shape of the Bézier surface.

Definition 20.2 (Bernstein Polynomials). The basis functions $B_{i,n}(t) = \binom{n}{i} t^i (1 - t)^{n-i}$ used to weight the control points in Bézier surface evaluation.

Definition 20.3 (Tensor Product). The method of forming a surface by multiplying two orthogonal Bézier curves, i.e., $Q(u, v) = \sum_{i,j} B_{i,m}(u) B_{j,n}(v) P_{ij}$.

Definition 20.4 (de Casteljau's Algorithm). A recursive algorithm for evaluating Bézier curves and surfaces that is numerically stable and geometrically intuitive.

20.2.3 Theory

A Bézier surface point $Q(u, v)$ is calculated using the formula:

$$Q(u, v) = \sum_{i=0}^m \sum_{j=0}^n B_{i,m}(u) B_{j,n}(v) P_{i,j}$$

The most efficient way to evaluate this is by using **de Casteljau's algorithm** [Far02] in two steps:

1. Evaluate $m + 1$ separate Bézier curves in the v -direction using parameter v . This produces a temporary set of control points $P'_0, \dots, P'_m$ that depend only on v .
2. Evaluate the single Bézier curve defined by P'_i using parameter u .

This “tensor product” approach effectively reduces the surface evaluation to several simpler curve evaluations.

Example 20.5 (Bicubic Bézier Surface Evaluation). Consider a 4×4 bicubic Bézier surface (degree $m = n = 3$) with 16 control points forming a 4×4 grid. To evaluate at $(u, v) = (0.25, 0.75)$:

1. For each of the 4 rows, apply de Casteljau’s algorithm in the v -direction with $v = 0.75$, producing 4 intermediate points P'_0, P'_1, P'_2, P'_3 .
2. Apply de Casteljau’s algorithm to $P'_0, \dots, P'_3$ in the u -direction with $u = 0.25$, yielding the final 3D point $Q(0.25, 0.75)$.

Each de Casteljau step requires $O(m)$ linear interpolations, so the total cost is $O(m \cdot n)$ per evaluation.

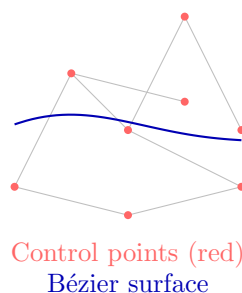


Figure 20.1: Bézier surface: a tensor-product surface defined by a control net of points.

20.2.4 Pseudocode

20.2.5 Parameter Selection

- **Parameter Range:** Parameters u, v must be strictly within $[0, 1]$.
- **Order:** The number of control points is $(m + 1) \times (n + 1)$. Typically $m = 3, n = 3$ for bicubic surfaces.

20.2.6 Complexity

- **Time Complexity:** $O(m \cdot n^2 + m^2)$ using the two-step de Casteljau approach. For bicubic surfaces ($m = 3, n = 3$), this is a small constant.
- **Space Complexity:** $O(m \cdot n)$ to store the control point grid.

20.2.7 Usages

- **CAD (Computer Aided Design):** Modeling car bodies, aircraft wings, and consumer products.
- **Animation:** Smooth character skinning and facial deformations.
- **TrueType/PostScript Fonts:** While fonts are 2D curves, they can be interpreted as 1D surfaces.
- **Geometric Design:** Creating smooth transitions (blends) between different parts of a model.
- **Fluid Simulation:** Representing the surface of a liquid with a set of smooth patches.

Algorithm 44 Evaluate Bézier Surface

```

1: function EVALUATEBEZIERSURFACE(control_points, u, v)
2:    $m \leftarrow \text{num\_rows} - 1$ 
3:    $n \leftarrow \text{num\_cols} - 1$  ▷ Reduce in the  $v$  direction
4:   temp_points  $\leftarrow \emptyset$ 
5:   for  $i \leftarrow 0$  to  $m$  do
6:     row  $\leftarrow \text{control\_points}[i]$ 
7:     EvaluateBezierCurve(row, v)
8:     temp_points.append( $p'$ )
9:   end for ▷ Reduce in the  $u$  direction
10:  return EvaluateBezierCurve(temp_points, u)
11: end function
12: function EVALUATEBEZIERCURVE(points, t)
13:  current  $\leftarrow \text{points}[:, ]$ 
14:  while  $|\text{current}| > 1$  do
15:    next_level  $\leftarrow \emptyset$ 
16:    for  $i \leftarrow 0$  to  $|\text{current}| - 2$  do
17:       $p \leftarrow (1 - t) \cdot \text{current}[i] + t \cdot \text{current}[i + 1]$ 
18:      next_level.append( $p$ )
19:    end for
20:    current  $\leftarrow \text{next\_level}$ 
21:  end while
22:  return current[0]
23: end function

```

20.2.8 Testing Methods and Edge Cases

- **Corners:** Verify that $Q(0, 0), Q(0, 1), Q(1, 0), Q(1, 1)$ exactly match the four corner control points.
- **Flat Surface:** If all control points lie on a plane, the entire evaluated surface must lie on that plane.
- **Degree Elevation:** Increasing the number of control points without changing the surface shape.
- **Precision:** Ensure the surface remains smooth and doesn't "break" at $u, v \approx 0.5$.
- **Continuity:** Checking that adjacent Bézier patches meet smoothly (G1 or C1 continuity).

20.2.9 References

- Bézier, P. [B68] "How Renault Uses Numerical Control for Car Body Design and Tooling". SAE paper.
- Farin, G. [Far02] "Curves and Surfaces for CAGD: A Practical Guide". Morgan Kaufmann.
- Piegl, L., & Tiller, W. [PT97] "The NURBS Book". Springer.
- Wikipedia: [Bézier surface](#)

20.3 NURBS Surfaces

20.3.1 Overview

NURBS (Non-Uniform Rational B-Splines) are a mathematical model commonly used in computer graphics and computer-aided design (CAD) for generating and representing curves and surfaces [PT97]. NURBS surfaces provide a highly flexible and unified way to represent both analytic shapes (like spheres, cones, and cylinders) and complex, free-form organic shapes. Evaluating a NURBS surface involves calculating the 3D coordinates (x, y, z) at a given parameter pair (u, v) within the surface's domain.

20.3.2 Definitions

Definition 20.6 (Knot Vector). A non-decreasing sequence of real numbers $t_0 \leq t_1 \leq \dots \leq t_{n+p+1}$ that determines how the control points affect the NURBS surface.

Definition 20.7 (NURBS Weights). A scalar $w_{ij} > 0$ associated with each control point P_{ij} . A higher weight “pulls” the surface closer to that control point.

Definition 20.8 (B-Spline Basis Functions). Piecewise polynomial functions $N_{i,p}$ of degree p defined over a knot vector via the Cox-de Boor recursion formula.

Definition 20.9 (Rational Surface). The term “rational” refers to the use of weights and a division step, allowing for the exact representation of conic sections (circles, ellipses).

20.3.3 Theory

A point on a NURBS surface $S(u, v)$ is defined as:

$$S(u, v) = \frac{\sum_{i=0}^n \sum_{j=0}^m N_{i,p}(u) N_{j,q}(v) w_{i,j} P_{i,j}}{\sum_{i=0}^n \sum_{j=0}^m N_{i,p}(u) N_{j,q}(v) w_{i,j}}$$

where $P_{i,j}$ are the control points, $w_{i,j}$ are weights, and $N_{i,p}, N_{j,q}$ are the B-spline basis functions of degrees p and q .

The calculation typically follows these steps:

1. **Locate Knots:** Find the indices of the knots u and v in their respective knot vectors.
2. **Evaluate Basis Functions:** Compute the values of all non-zero basis functions at u and v using the Cox-de Boor recursion formula.
3. **Summation:** Compute the weighted sum of the control points and the sum of the weights.
4. **Division:** Divide the weighted point sum by the total weight to find the final 3D point.

Example 20.10 (NURBS Circle Representation). A circle of radius r can be exactly represented by a single quadratic NURBS curve ($p = 2$) with 9 control points and a specific knot vector. The key insight is that the weights w_i for the control points at the cardinal directions are $\cos(\pi/4) = \sqrt{2}/2$, while the weights for the diagonal control points are 1. This rational weighting is what allows NURBS to represent conic sections exactly, which polynomial Bézier curves cannot do.

20.3.4 Pseudocode

20.3.5 Parameter Selection

- **Degree (p, q) :** Usually $p = 3, q = 3$ (bicubic) for smooth modeling.
- **Knot Vector Type: Clamped** knot vectors (where the first and last $p + 1$ knots are identical) ensure the surface passes exactly through its corner control points.

Algorithm 45 Evaluate NURBS Surface

```

1: function EVALUATENURBSSURFACE(control_points, weights, knots_u, knots_v, p, q, u, v)
2:   FindKnotSpan(n,p,u,knots_u)
3:   FindKnotSpan(m,q,v,knots_v)
4:   BasisFunctions(span_u,u,p,knots_u)
5:   BasisFunctions(span_v,v,q,knots_v)
6:   sum_wp ← 0
7:   sum_w ← 0
8:   for l ← 0 to q do
9:     temp ← 0
10:    temp_w ← 0
11:    for k ← 0 to p do
12:      idx_u ← span_u - p + k
13:      idx_v ← span_v - q + l
14:      w ← weights[idx_u][idx_v]
15:      p_val ← control_points[idx_u][idx_v]
16:      sum_wp ← sum_wp +  $N_u[k] \cdot N_v[l] \cdot w \cdot p_{\text{val}}$ 
17:      sum_w ← sum_w +  $N_u[k] \cdot N_v[l] \cdot w$ 
18:    end for
19:  end for
20:  return sum_wp / sum_w
21: end function

```

20.3.6 Complexity

- **Time Complexity:** $O(p^2 + q^2)$ to evaluate the basis functions, plus $O(p \cdot q)$ for the final summation. This is extremely efficient for low degrees.
- **Space Complexity:** $O(n \cdot m)$ to store the control point and weight grids.

20.3.7 Usages

- **Industrial CAD:** Standard data format (IGES, STEP) for exchanging 3D models between different engineering software (SolidWorks, Rhino, CATIA).
- **Reverse Engineering:** Fitting smooth surfaces to scanned point cloud data.
- **Scientific Visualization:** Smoothly interpolating scattered spatial data (e.g., bathymetry or atmospheric pressure).
- **High-End Animation:** Rendering complex character models (Pixar's RenderMan uses NURBS extensively).
- **Naval Architecture:** Designing boat hulls for optimal fluid flow.

20.3.8 Testing Methods and Edge Cases

- **Analytic Shapes:** Verify that a NURBS surface with appropriate weights can exactly represent a sphere or a cylinder.
- **Weights of 1:** If all weights are 1.0, the surface must behave identically to a non-rational B-spline surface.
- **Boundary Conditions:** Verify that the surface interpolates its corner control points (for clamped knots).

- **Numerical Stability:** Ensure the denominator (total weight) does not approach zero.
- **Derivative Calculation:** Verify that the surface normals (calculated from u and v derivatives) are smooth and consistent.

20.3.9 References

- Piegl, L., & Tiller, W. [PT97] “The NURBS Book”. Springer.
- Rogers, D. F. (2001). “An Introduction to NURBS: With Historical Perspective”. Morgan Kaufmann.
- Farin, G. [Far02] “Curves and Surfaces for CAGD: A Practical Guide”. Morgan Kaufmann.
- Wikipedia: [Non-uniform rational B-spline](#)

20.4 Mesh Voxelization

20.4.1 Overview

Mesh voxelization is the process of converting a 3D surface mesh (typically composed of triangles) into a volumetric representation called a **voxel grid** [AM01,NT03]. A voxel (volume element) is the 3D equivalent of a pixel. Voxelization is used to simplify complex geometry, perform volumetric analysis, and prepare models for physics simulations or 3D printing. The process can result in a “binary” voxel grid (occupied vs. empty) or a “solid” voxelization (distinguishing interior from exterior).

20.4.2 Definitions

Definition 20.11 (Voxel). A unit of volume on a regular 3D grid.

Definition 20.12 (Conservative Voxelization). A method that ensures a voxel is marked as occupied if any part of the triangle passes through it.

Definition 20.13 (Parity Rule). A technique for solid voxelization where a voxel’s “insideness” is determined by counting intersections of a ray with the mesh boundary using the even-odd rule.

20.4.3 Theory

The most common approach to surface voxelization is based on **triangle-box intersection** tests [AM01].

1. Define a 3D grid that encloses the mesh’s bounding box.
2. For each triangle in the mesh:
 - Determine the range of voxels it might intersect (its AABB in grid coordinates).
 - For every voxel in that range, perform a rigorous triangle-box intersection test (e.g., using the Akenine-Möller algorithm based on the Separating Axis Theorem).
3. To perform **solid voxelization** (filling the interior):
 - After surface voxelization, use a flood-fill algorithm starting from the grid boundaries to identify “outside” voxels [NT03].
 - Everything not reachable from the outside is “inside.”

- Alternatively, use ray-casting along each grid line and apply the even-odd rule.

Example 20.14 (Voxelizing a Cube). Consider a unit cube (8 vertices, 12 triangular faces) voxelized at resolution $8 \times 8 \times 8$. The bounding box is $[0, 1]^3$ with voxel size $1/8$. Each of the 12 triangles is tested against its AABB in grid coordinates. Surface voxels on each face are marked. Then flood-fill from a corner voxel identifies all voxels on the exterior. The remaining interior voxels form a $6 \times 6 \times 6$ solid block of 216 inside voxels (the boundary voxels separate inside from outside).

20.4.4 Pseudocode

Algorithm 46 Voxelize Mesh

```

1: function VoxelizeMesh(mesh, resolution)
2:   bbox  $\leftarrow$  mesh.GetBoundingBox()
3:   voxel_size  $\leftarrow$  bbox.size.max() / resolution
4:   grid  $\leftarrow$  array[resolution]3 initialized to EMPTY ▷ Surface Voxelization
5:   for each triangle in mesh.faces do
6:     WorldToGrid(triangle.min, bbox, voxel_size)
7:     WorldToGrid(triangle.max, bbox, voxel_size)
8:     for  $x \leftarrow \text{min}_v.x$  to  $\text{max}_v.x$  do
9:       for  $y \leftarrow \text{min}_v.y$  to  $\text{max}_v.y$  do
10:        for  $z \leftarrow \text{min}_v.z$  to  $\text{max}_v.z$  do
11:          GetVoxelBox( $x, y, z$ , bbox, voxel_size)
12:          if TriangleBoxIntersect(triangle, voxel_box) then
13:            grid[ $x$ ][ $y$ ][ $z$ ]  $\leftarrow$  SURFACE
14:          end if
15:        end for
16:      end for
17:    end for
18:  ▷ Solid Voxelization (Optional)
19:  FloodFill(grid, start_pos=(0,0,0), target = EMPTY, replacement = OUTSIDE)
20:  for ( $x, y, z$ ) in grid do
21:    if grid[ $x$ ][ $y$ ][ $z$ ] = EMPTY then
22:      grid[ $x$ ][ $y$ ][ $z$ ]  $\leftarrow$  INSIDE
23:    end if
24:  end for
25:  return grid
26: end function

```

20.4.5 Parameter Selection

- **Resolution:** Higher resolution captures more detail but increases memory consumption cubically ($O(N^3)$).
- **Intersection Strategy:** Conservative voxelization is preferred for tasks like collision detection to avoid “gaps” in the surface.

20.4.6 Complexity

- **Time Complexity:** $O(F \cdot K^3)$ where F is the face count and K is the local grid size per triangle. For solid filling, $O(N^3)$ where N is the resolution.

- **Space Complexity:** $O(N^3)$ to store the 3D grid. Sparse representations like **OpenVDB** or **Octrees** can reduce this significantly.

20.4.7 Usages

- **3D Printing:** Slicing software voxelizes models to generate toolpaths and infill patterns.
- **Physics Simulation:** Converting complex meshes into voxels for fluid dynamics (Lattice Boltzmann Method) or smoke simulation.
- **Collision Detection:** Using a voxel grid as a spatial index for fast intersection queries.
- **Medical Imaging:** Modeling organs or bones from surface scans for surgical planning.
- **Game Development:** Destructible environments (e.g., Minecraft or Teardown) and GPU-based global illumination.

20.4.8 Testing Methods and Edge Cases

- **Watertightness:** Verify that solid voxelization produces no “leaks” for a closed mesh.
- **Thin Features:** Ensure triangles thinner than a voxel are still captured (conservative test).
- **Non-Manifold Mesh:** Test how the solid filler handles meshes with holes (usually results in the entire model being marked as outside).
- **Numerical Precision:** Triangle-box tests are sensitive to floating-point errors; use robust epsilon values.
- **Memory Limit:** Check performance and stability at high resolutions (e.g., 1024^3).

20.4.9 References

- Akenine-Möller, T. [AM01] “Fast 3D Triangle-Box Overlap Testing”. Journal of Graphics Tools.
- Nooruddin, F. S., & Turk, G. [NT03] “Simplification and Repair of Polygonal Models Using Volumetric Techniques”. IEEE TVCG.
- Schwarz, M., & Seidel, H. P. (2010). “Fast Parallel Surface and Solid Voxelization on GPUs”. ACM TOG.
- Wikipedia: [Voxel](#)

20.5 Winding-Filtered Tetrahedral Meshing

20.5.1 Overview

Winding-Filtered Tetrahedral Meshing is a robust technique for generating a volumetric tetrahedral mesh from a surface mesh, even if the surface mesh is “dirty” (i.e., contains self-intersections or holes) [H⁺18].

Traditional tetrahedralization algorithms (like constrained Delaunay) often require a clean, manifold surface. In contrast, winding-filtered meshing uses the Generalized Winding Number (GWN) to classify tetrahedra as interior or exterior, which is robust even for non-manifold inputs.

20.5.2 Implementation Details

In `CompGeom`, the `WindingFilteredTetMesher` uses the following process (inspired by `fTetWild` [H⁺18]):

1. **Bounding Box Expansion:** Create a large super-tetrahedron containing the entire surface mesh.
2. **Point Insertion:** Add surface vertices and a dense grid of internal Steiner points to the tetrahedralization.
3. **Delaunay Tetrahedralization:** Construct a global Delaunay tetrahedralization of the point set.
4. **Winding-Based Filtering:** For each tetrahedron, compute the Generalized Winding Number of its centroid with respect to the original surface mesh.
5. **Output:** Retain only tetrahedra where the $\text{GWN} > 0.5$ (indicating the interior of the domain).

Example 20.15 (Winding Number Classification). Consider a closed tetrahedron surface mesh. For a point inside the tetrahedron, the generalized winding number is 1.0 (the surface wraps around it exactly once). For a point outside, the winding number is 0.0. The threshold of 0.5 cleanly separates interior from exterior. Even if the surface has a small gap (non-watertight), the winding number degrades gracefully, correctly classifying most interior tetrahedra.

20.5.3 Usage

Algorithm 47 Winding-Filtered Tet Meshing

```

1: from compgeom.mesh.volume.tetmesh.winding_filtered_mesher import
   WindingFilteredTetMesher
2: import numpy as np
3:                                     ▷ surface vertices and faces as numpy arrays
4: vertices ← np.array([[0, 0, 0], [1, 0, 0], [0, 1, 0], [0, 0, 1]])
5: faces ← np.array([[0, 1, 2], [0, 2, 3], [0, 3, 1], [1, 2, 3]])
6:
7: WindingFilteredTetMesher(vertices, faces)
8: tet_mesh ← mesher.mesh(refinement_factor=1.0)
```

20.5.4 References

- Hu, Y., et al. “fTetWild: Robust Tetrahedral Meshing in the Wild.” *ACM Transactions on Graphics (SIGGRAPH)*, 2018.
- Jacobson, A., et al. “Robust Inside-Outside Segmentation using Generalized Winding Numbers.” *ACM Transactions on Graphics (SIGGRAPH)*, 2013.

20.6 Mesh Refinement

20.6.1 Overview

Mesh refinement is the process of increasing the resolution of a mesh by subdividing its elements (faces and edges) into smaller ones. The goal is to improve the mesh’s quality, accuracy for

numerical simulations, or visual smoothness [Loo87, CC78]. Refinement can be **global** (applied to the entire mesh) or **local** (applied only to specific regions of interest). Common algorithms include Loop subdivision (for triangles) and Catmull-Clark subdivision (for quads).

20.6.2 Definitions

Definition 20.16 (Subdivision). Splitting an existing face into multiple smaller faces.

Definition 20.17 (Edge Split). Inserting a new vertex at the midpoint of an edge and splitting the edge into two.

Definition 20.18 (Face Split). Connecting a new vertex at the centroid of a face to its original vertices.

Definition 20.19 (Adaptive Refinement). Refinement that occurs only where a specific error metric exceeds a threshold.

Definition 20.20 (Stencil). A weighted average formula used to determine the position of new and original vertices after subdivision.

20.6.3 Theory

Most refinement schemes follow a two-step process:

1. **Topological Split:** The mesh connectivity is modified to create more faces (e.g., 1-to-4 triangle split).
2. **Smoothing (Geometric Update):** The positions of the new and existing vertices are adjusted based on their neighbors to achieve a specific level of smoothness (e.g., C^1 or C^2 continuity).

For example, in **Loop Subdivision** [Loo87]:

- Each triangle is split into four smaller triangles.
- New “edge vertices” are positioned using a weighted average of the endpoints and the opposite vertices of the two incident faces.
- Original “even vertices” are repositioned based on their neighbors to maintain surface smoothness.

For **Catmull-Clark Subdivision** [CC78]:

- Each quad is split into four smaller quads by inserting vertices at face centers and edge midpoints.
- The limiting surface is C^2 everywhere except at extraordinary vertices (where C^1 continuity holds).

Example 20.21 (1-to-4 Triangle Refinement). Start with a single equilateral triangle with vertices A , B , C . After one level of 1-to-4 refinement:

- Three new edge midpoints are created: M_{AB} , M_{BC} , M_{CA} .
- Four new triangles are formed: (A, M_{AB}, M_{CA}) , (B, M_{BC}, M_{AB}) , (C, M_{CA}, M_{BC}) , (M_{AB}, M_{BC}, M_{CA}) .
- The number of faces goes from 1 to 4, edges from 3 to 9, and vertices from 3 to 6.
- The Euler characteristic is preserved: $\chi = 6 - 9 + 4 = 1$ (disk), same as the original $\chi = 3 - 3 + 1 = 1$.

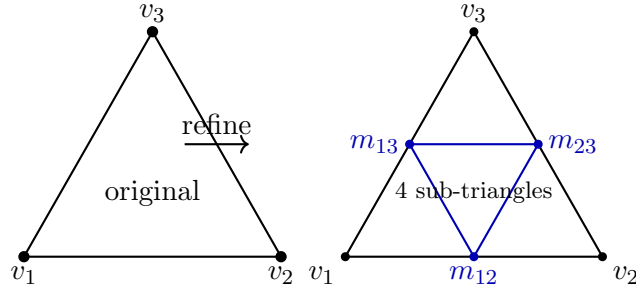


Figure 20.2: 1-to-4 triangle refinement: each edge is split at its midpoint, producing four congruent sub-triangles.

Algorithm 48 Refine Mesh

```

1: function REFINEMESH(mesh)
2:   new_vertices  $\leftarrow$  mesh.vertices[:]
3:   new_faces  $\leftarrow \emptyset$ 
4:   edge_midpoints  $\leftarrow \{\}$  ▷ Create new vertices at every edge midpoint
5:   for each edge in mesh.edges do
6:      $(v_1, v_2) \leftarrow$  edge
7:     mid_p  $\leftarrow$  (mesh.vertices[v1] + mesh.vertices[v2]) / 2.0
8:     edge_midpoints[sorted(edge)]  $\leftarrow$  [new_vertices]
9:     new_vertices.append(mid_p)
10:  end for ▷ Create four new triangles for every original face
11:  for each face in mesh.faces do
12:     $(v_1, v_2, v_3) \leftarrow$  face
13:    m1  $\leftarrow$  edge_midpoints[sorted(v1, v2)]
14:    m2  $\leftarrow$  edge_midpoints[sorted(v2, v3)]
15:    m3  $\leftarrow$  edge_midpoints[sorted(v3, v1)]
16:    new_faces.append((v1, m1, m3))
17:    new_faces.append((v2, m2, m1))
18:    new_faces.append((v3, m3, m2))
19:    new_faces.append((m1, m2, m3))
20:  end for
21:  return Mesh(new_vertices, new_faces)
22: end function

```

20.6.4 Pseudocode: Simple 1-to-4 Triangle Refinement

20.6.5 Parameter Selection

- **Refinement Level:** The number of iterations. Each iteration of 1-to-4 split quadruples the number of faces.
- **Metric:** For adaptive refinement, common metrics include curvature, gradient of a simulated field (e.g., stress or temperature), or edge length.

20.6.6 Complexity

- **Time Complexity:** $O(F \cdot 4^k)$ where F is the original face count and k is the refinement level.
- **Space Complexity:** $O(F \cdot 4^k)$ to store the refined mesh.

20.6.7 Usages

- **Computer Graphics:** Generating high-fidelity models from low-poly base meshes (e.g., characters in movies or games).
- **Finite Element Method (FEM):** Improving solution accuracy in regions with high stress or fluid turbulence.
- **3D Printing:** Increasing the resolution of a mesh to ensure smooth curved surfaces after slicing.
- **Terrain Modeling:** Adding detail to a coarse elevation grid where the landscape is complex.
- **Surface Reconstruction:** Improving the density of a mesh fitted to a sparse point cloud.

20.6.8 Testing Methods and Edge Cases

- **Consistency:** Verify that the refined mesh occupies the exact same volume as the original (for linear refinement).
- **Manifold Property:** Ensure refinement doesn't introduce non-manifold edges or vertices.
- **Boundaries:** Verify that boundary edges are correctly split and handled (often boundary rules are different from interior rules).
- **Watertightness:** Check that no gaps are introduced between adjacent sub-triangles.
- **High Valence:** Test on meshes with “poles” (vertices with many incident edges).

20.6.9 References

- Loop, C. [Loo87] “Smooth subdivision surfaces based on triangle meshes”. Master’s thesis, University of Utah.
- Catmull, E., & Clark, J. [CC78] “Recursively generated B-spline surfaces on arbitrary topological meshes”. Computer-Aided Design.
- Botsch, M., Kobbelt, L., Pauly, M., Alliez, P., & Lévy, B. [BKP⁺10] “Polygon Mesh Processing”. CRC Press.
- Wikipedia: [Subdivision surface](#)

20.7 Approximate Convex Decomposition

20.7.1 Overview

Approximate Convex Decomposition (ACD) is the process of partitioning a 3D surface mesh into a set of nearly convex parts [WLW22]. Unlike exact convex decomposition, which can produce an enormous number of tiny, thin pieces for complex organic shapes, ACD allows for a controllable level of “concavity tolerance.” This results in a small number of visually meaningful parts that are easy to handle in physics engines and motion planning. CoACD (Collision-oriented ACD) is a modern, state-of-the-art implementation that prioritizes accuracy and performance for collision detection.

20.7.2 Definitions

Definition 20.22 (Concavity). A measure of how much a shape deviates from being convex. It is often measured as the maximum distance from the surface to its convex hull.

Definition 20.23 (Convex Decomposition). A collection of parts whose union matches the original shape and whose interiors are disjoint.

Definition 20.24 (Cutting Plane). A plane used to split a non-convex part into two smaller, more convex parts.

20.7.3 Theory

CoACD operates by recursively splitting the mesh using a **best-fit cutting plane** [WLW22].

1. **Voxelization:** The input mesh is first converted into a high-resolution voxel grid to simplify the geometry and handle potential topological defects (like holes or self-intersections).
2. **Concavity Estimation:** For each part, the maximum distance from the surface to its convex hull is calculated. If this distance is below the **threshold**, the part is accepted as “approximately convex.”
3. **Plane Selection:** If a part is too concave, the algorithm searches for a cutting plane that minimizes the sum of concavities of the two resulting parts. This is often done by sampling candidate planes from the mesh’s dual graph or using a manifold-aware search.
4. **Merging:** After decomposition, adjacent parts that are nearly convex together can be merged to further reduce the total part count.

Example 20.25 (CoACD of an L-Shape). Consider an L-shaped polyhedron. Its convex hull is much larger than the original shape, so the concavity is large. CoACD with threshold 0.05 might split it along the inner corner, producing two rectangular prisms (each with small concavity relative to its own convex hull). With a larger threshold (e.g., 0.5), the algorithm might accept the L-shape as a single part, since the concavity is below the threshold.

20.7.4 Pseudocode

20.7.5 Parameter Selection

- **Concavity Threshold:** The most important parameter. A smaller threshold (e.g., 0.01) produces many accurate parts; a larger threshold (e.g., 0.1) produces a few coarse parts.
- **Resolution:** The voxel grid resolution (e.g., 50–100 voxels per side). Higher resolution is slower but more accurate.
- **Max Parts:** A hard limit on the total number of pieces to prevent excessive fragmentation.

Algorithm 49 CoACD

```
1: function CoACD(mesh, threshold, max_parts)
2:   Voxelize(mesh)
3:   parts  $\leftarrow$  [v_mesh]
4:   result  $\leftarrow$   $\emptyset$ 
5:   while parts  $\neq$   $\emptyset$  do
6:     current  $\leftarrow$  parts.pop()
7:     CalculateConcavity(current)
8:     if concavity  $\leq$  threshold or |result|  $\geq$  max_parts then
9:       result.append(current)
10:    else
11:      FindBestCuttingPlane(current)
12:      Split(current, best_plane)
13:      parts.extend([ $P_1$ ,  $P_2$ ])
14:    end if
15:  end while
16:  MergeSmallParts(result, threshold)
17:  return MapToOriginalSurface(final_parts, mesh)
18: end function
```

20.7.6 Complexity

- **Time Complexity:** $O(K \cdot N \log N)$ where N is the number of voxels/vertices and K is the number of resulting parts. The plane search is the most expensive step.
- **Space Complexity:** $O(N)$ to store the voxel grid and the sub-meshes.

20.7.7 Usages

- **Physics Simulation (Collision Shapes):** Modern engines like PhysX, Bullet, and Havok work much faster with a set of convex hulls than with a single complex triangle mesh.
- **Robotics:** Path planning for grippers or mobile robots, where obstacles are simplified into convex hulls for fast distance queries.
- **Computer Vision:** Segmenting a complex scanned object into its functional components (e.g., a chair into legs, seat, and back).
- **Animation:** Generating ragdoll physics rigs for characters.

20.7.8 Testing Methods and Edge Cases

- **Perfectly Convex Mesh:** The algorithm should return the original mesh as a single part.
- **Non-Manifold Geometry:** Verify that the voxelization step correctly “heals” the mesh.
- **Sharp Spikes:** These create high local concavity; ensure the threshold correctly identifies them.
- **Thin Structures:** Very thin regions (like a piece of paper) can be tricky for voxelization; check for “leaking” or vanishing parts.
- **Holes:** Ensure the decomposition doesn’t try to “fill” holes unless intended.

20.7.9 References

- Wei, X., Liu, J., & Wang, J. [WLW22] “CoACD: Collision-oriented Approximate Convex Decomposition”. SIGGRAPH.
- Mamou, K., & Ghorbel, F. (2009). “A Simple and Efficient Approach for 3D Mesh Approximate Convex Decomposition”. SEGEF.
- Lien, J. M., & Amato, N. M. (2007). “Approximate Convex Decomposition of Polyhedra”. ACM TOG.
- [GitHub: CoACD repository](#)

20.8 Triangle to Quad Conversion

20.8.1 Overview

Triangle to quad conversion is the process of transforming a triangle-only mesh into a quad-dominant or pure quadrilateral mesh [R⁺12, BLP⁺13]. Quadrilateral meshes are often preferred in computer graphics and numerical simulations (like Finite Element Analysis) because they follow principal curvature directions better and provide more natural coordinate systems for texture mapping and subdivision.

20.8.2 Definitions

Definition 20.26 (Dual Graph (Triangle Mesh)). A graph where each triangle is a node and edges connect nodes representing triangles that share an edge.

Definition 20.27 (Perfect Matching). A set of edges in the dual graph such that every node is incident to exactly one edge. In our context, this corresponds to a pairing of every triangle into a quad.

Definition 20.28 (Quad-Dominant Mesh). A mesh containing mostly quads but some triangles.

Definition 20.29 (Pure Quad Mesh). A mesh where every face has exactly four vertices.

20.8.3 Theory

The most common approach to triangle-to-quad conversion is based on **pairing adjacent triangles**.

1. Represent the mesh as a dual graph.
2. Assign a weight to each edge in the dual graph based on the “quality” of the potential quad (e.g., how close it is to a square or a rectangle, flatness, etc.).
3. Find a maximum weight matching in this graph.
4. Each matched pair of triangles is merged into a quadrilateral by removing their shared edge.
5. Unmatched triangles remain as triangles in the final mesh.

To achieve a **pure quad mesh**, more aggressive techniques are needed, such as:

- **Catmull-Clark Subdivision**: Every triangle is split into three quads by adding a vertex at the centroid and midpoints of edges. This always results in a pure quad mesh but significantly increases the face count.
- **Q-Morph**: A front-advancing algorithm that builds quads from the boundary inward.

20.8.4 Pseudocode: Greedy Triangle Merging

Algorithm 50 Greedy Triangle to Quad

```
1: function GREEDYTRIANGLETOQUAD(mesh)
2:   BuildDualGraph(mesh)
3:   CalculateQuadQualityWeights(dual_graph, mesh)
4:   sorted_pairs  $\leftarrow$  sort(weights, key = weights, reverse = True)
5:   matched_triangles  $\leftarrow \emptyset$ 
6:   quads  $\leftarrow \emptyset$ 
7:   for ( $T_1, T_2$ ) in sorted_pairs do
8:     if  $T_1 \notin$  matched_triangles and  $T_2 \notin$  matched_triangles then
9:       MergeTriangles( $T_1, T_2$ , mesh)
10:      quads.append(new_quad)
11:      matched_triangles.add( $T_1$ )
12:      matched_triangles.add( $T_2$ )
13:    end if
14:  end for
15:  remaining_tris  $\leftarrow [T \in \text{mesh.faces} \mid T \notin \text{matched\_triangles}]$ 
16:  return Mesh(mesh.vertices, quads+remaining_tris)
17: end function
```

20.8.5 Parameter Selection

- **Quality Metric:** A common metric is the ratio of the quad's diagonals or the deviation of its internal angles from 90° .
- **Matching Algorithm:** Greedy matching is fast; Blossom's algorithm finds the global maximum weight matching but is much slower.

20.8.6 Complexity

- **Graph Construction:** $O(F)$ where F is the number of faces.
- **Greedy Matching:** $O(F \log F)$ for sorting the dual edges.
- **Blossom Algorithm:** $O(E \cdot V^2)$, which is $O(F^3)$ for meshes.

20.8.7 Usages

- **Finite Element Method (FEM):** Quads are often more accurate for structural and thermal analysis.
- **Animation:** Quad meshes deform more predictably during character rigging and skinning.
- **Subdivision Modeling:** Catmull-Clark subdivision is the industry standard for creating smooth surfaces from quad cages.
- **Texture Mapping:** Providing a more natural UV grid for 2D textures.

20.8.8 Testing Methods and Edge Cases

- **Planar Mesh:** Merging two triangles into a quad should preserve the flatness.

- **Non-Convex Pairs:** Merging two triangles into a concave quad should be avoided or flagged (using the quality metric).
- **Singularities:** Identify “extraordinary vertices” (those not shared by 4 quads).
- **Watertightness:** Ensure no holes are created at the boundaries of merged quads.
- **Manifoldness:** The merging process must preserve the manifold property of the mesh.

20.8.9 References

- Remacle, J. F., et al. [R⁺12] “A Frontal Delaunay Quad Mesh Generator”. International Journal for Numerical Methods in Engineering.
- Owen, S. J. (1998). “A Survey of Unstructured Mesh Generation Technology”. IMR.
- Bommers, D., Lévy, B., Pietroni, N., Puppo, E., Silva, C., Tarini, M., & Zayer, R. [BLP⁺13] “State of the Art in Quad-Meshing”. Eurographics STAR.
- Wikipedia: [Types of mesh](#)

20.9 Voxel Grid Point Simplification

20.9.1 Overview

Voxel Grid Decimation is a point cloud simplification algorithm that reduces the number of points in a dataset while maintaining its spatial distribution [BKP⁺10]. It works by partitioning the space into a regular grid of “voxels” (3D pixels) and replacing all points within each voxel with a single representative point. This is an extremely efficient way to downsample large point clouds, such as those generated by LiDAR or photogrammetry.

20.9.2 Definitions

- **Voxel:** A volume element representing a value on a regular grid in three-dimensional space.
- **Decimation:** The process of reducing the sampling rate of a signal or data set.
- **Grid Hashing:** Mapping a 2D or 3D coordinate (x, y, z) to an integer triple (i, j, k) based on a fixed cell size, used as a key for fast lookups.
- **Representative Point:** The point chosen to represent a voxel; typically the first point found, the centroid of all points in the voxel, or the point closest to the voxel’s center.

20.9.3 Theory

The algorithm discretizes the continuous space inhabited by the point cloud. By defining a voxel size L , any point (x, y, z) can be assigned to a voxel with indices $(\lfloor x/L \rfloor, \lfloor y/L \rfloor, \lfloor z/L \rfloor)$.

In the simplest form, the first point that falls into a previously empty voxel is kept, and all subsequent points falling into the same voxel are discarded. More sophisticated versions calculate the average position (centroid) of all points within the voxel to better preserve the underlying surface geometry. This process results in a point cloud where no two points are closer than the voxel size (approximately), leading to a uniform density.

Algorithm 51 Voxel Grid Simplify

```
1: function VOXELGRIDSIMPLIFY(points, voxel_size)
2:   grid  $\leftarrow$  hash_map()
3:   simplified_points  $\leftarrow \emptyset$ 
4:   for each  $p$  in points do
5:     gx  $\leftarrow \lfloor p.x / \text{voxel\_size} \rfloor$ 
6:     gy  $\leftarrow \lfloor p.y / \text{voxel\_size} \rfloor$ 
7:     gz  $\leftarrow \lfloor p.z / \text{voxel\_size} \rfloor$ 
8:     key  $\leftarrow$  (gx, gy, gz)
9:     if key  $\notin$  grid then
10:      if NoPointTooCloseInNeighbors( $p$ , key, grid, voxel_size) then
11:        grid[key]  $\leftarrow p$ 
12:        simplified_points.append( $p$ )
13:      end if
14:    end if
15:  end for
16:  return simplified_points
17: end function
```

20.9.4 Pseudocode**20.9.5 Parameter Selection**

- **Voxel Size (L):** Often determined by a **ratio** of the bounding box diagonal (e.g., $L = 0.01 \times \text{diagonal}$). A larger L results in more aggressive simplification.
- **Protected Points:** Specific points (like corners or edges) can be flagged by ID to be excluded from decimation, ensuring critical features are preserved.

20.9.6 Complexity

- **Time Complexity:** $O(n)$, where n is the number of points. Hash map insertions and lookups are $O(1)$ on average.
- **Space Complexity:** $O(n)$ in the worst case (if every point ends up in a different voxel), but typically much less as points are decimated.

20.9.7 Usages

- **LiDAR Processing:** Reducing millions of points to a manageable number for terrain modeling.
- **Real-time Rendering:** Generating Levels of Detail (LOD) for 3D environments.
- **Collision Detection:** Simplifying complex meshes into point-representative volumes for faster intersection tests.
- **Robotics:** Downsampling sensor data for simultaneous localization and mapping (SLAM).

20.9.8 Testing Methods and Edge Cases

- **Uniform Points:** A perfectly regular grid of points should be decimated predictably.
- **High-Density Clusters:** Ensure that thousands of points in a single voxel are correctly reduced to one.

- **Voxel Size > Bounding Box:** Should result in a single point (the first one processed).
- **Voxel Size ≈ 0 :** Should return the original point cloud.
- **Precision:** Handle floating-point errors at voxel boundaries.

20.9.9 References

- PCL (Point Cloud Library) Documentation: `pcl::VoxelGrid`.
- Botsch, M., Kobbelt, L., Pauly, M., Alliez, P., & Lévy, B. [BKP⁺10] “Polygon Mesh Processing”. CRC Press.
- Wikipedia: [Point cloud](#)

20.10 Surface Reconstruction

20.10.1 Overview

Surface reconstruction is the process of recovering a continuous surface representation from a discrete set of sampled points in three-dimensional space. This problem arises in numerous applications: 3D scanning devices such as LiDAR and structured-light scanners produce dense point clouds that must be converted into watertight meshes for downstream processing. Photogrammetry pipelines generate point clouds from photographs. Medical imaging yields volumetric point data that benefits from surface extraction. The two principal families of reconstruction algorithms are *local* methods, which grow a mesh directly from the point set by pivoting a probing ball across the surface, and *global* methods, which recover an implicit function whose iso-surface is then extracted.

20.10.2 Definitions

Definition 20.30 (Surface Reconstruction). Given a set $P = \{p_1, \dots, p_n\} \subset \mathbb{R}^3$ sampled from an unknown surface S , the surface reconstruction problem is to compute a mesh M that approximates S in a topologically and geometrically faithful manner.

Definition 20.31 (Ball Pivoting Algorithm). A local mesh-growing algorithm introduced by Bernardini et al. [BMR⁺99] that reconstructs a triangle mesh from a point cloud by rolling a ball of radius ρ over the point set. The ball pivots around the boundary edges of the current mesh, seeking new triangles.

Definition 20.32 (Empty Ball Property). A ball of radius ρ centered at c satisfies the *empty ball property* with respect to a triangle $\triangle(p_i, p_j, p_k)$ if no point of $P \setminus \{p_i, p_j, p_k\}$ lies strictly inside the ball, and the ball passes through p_i , p_j , and p_k .

Definition 20.33 (Seed Triangle). A triangle $\triangle(p_i, p_j, p_k)$ formed by three points of P such that a ball of radius ρ rests on the three vertices and contains no other point of P in its interior. The seed triangle initiates the mesh-growing process.

Definition 20.34 (Boundary Edge Front). The set of boundary edges $\mathcal{F}$ maintained during ball pivoting. A boundary edge is an edge of the current mesh shared by exactly one triangle. The algorithm iteratively pivots around edges in $\mathcal{F}$ to discover new triangles.

Definition 20.35 (Poisson Equation). Given a vector field $\mathbf{V}$ defined over a domain Ω , the Poisson equation seeks a scalar function ϕ satisfying $\Delta\phi = \nabla \cdot \mathbf{V}$, where Δ is the Laplacian operator.

Definition 20.36 (Indicator Function). A binary function $\chi_S : \mathbb{R}^3 \rightarrow \{0, 1\}$ that equals 1 inside the solid whose boundary is the surface S and 0 outside. Recovering χ_S from oriented point samples is the goal of Poisson reconstruction.

Definition 20.37 (Marching Cubes). An algorithm for extracting an iso-surface from a scalar field sampled on a regular grid. Each grid cell is classified into one of 256 topological cases, and triangulated accordingly [LC87].

20.10.3 Theory

Ball Pivoting Algorithm

The BPA proceeds in three phases: seed selection, pivoting, and front propagation.

Seed Triangle Finding. The algorithm searches for three points $p_i, p_j, p_k \in P$ such that a ball of radius ρ can be placed on the three vertices without enclosing any other point. In the implementation (`BallPivotingReconstructor`), a triple is accepted when (i) the circumradius r of $\triangle(p_i, p_j, p_k)$ satisfies $r \leq \rho$, and (ii) the ball centered at the computed center c contains no point of P other than the three vertices.

Ball Center Computation. Given three non-collinear points p_1, p_2, p_3 with circumradius $r < \rho$, the ball center lies on the line perpendicular to the triangle plane through its circumcenter m . Setting $h = \sqrt{\rho^2 - r^2}$, the center is $c = m + h \hat{n}$, where $\hat{n}$ is the unit normal to the triangle plane oriented consistently with the surface orientation.

Pivoting Around a Boundary Edge. Given a boundary edge (v_1, v_2) with an existing triangle $\triangle(v_1, v_2, v_{\text{opp}})$, the ball is rotated around the edge until it touches a new point p_k . The pivot point is selected as the candidate minimizing the rotation angle such that the new ball on (v_1, v_2, p_k) satisfies the empty ball property. This guarantees a Delaunay-like local optimality.

Front Propagation. Each new triangle $\triangle(v_1, p_k, v_2)$ adds three edges to the mesh. Edges that are shared by exactly one triangle become boundary edges and are appended to the front $\mathcal{F}$. The algorithm terminates when the front is empty.

Poisson Surface Reconstruction

Vector Field Construction. Given P with oriented normals $\{n_i\}$, the algorithm rasterizes the normals into a regular grid of resolution res^3 , computing an averaged normal vector $\mathbf{V}(g)$ at each grid cell g from the points that fall within it.

Divergence Computation. The divergence of $\mathbf{V}$ is approximated on the grid using finite differences:

$$\nabla \cdot \mathbf{V} \approx \frac{V_x(i+1, j, k) - V_x(i, j, k)}{\Delta x} + \frac{V_y(i, j+1, k) - V_y(i, j, k)}{\Delta y} + \frac{V_z(i, j, k+1) - V_z(i, j, k)}{\Delta z}.$$

Solving the Poisson Equation. The discrete Laplacian $\Delta\phi = \nabla \cdot \mathbf{V}$ is assembled as a sparse linear system $L\phi = \text{div}(\mathbf{V})$ and solved via a sparse direct solver. Boundary conditions are set to Dirichlet ($\phi = 0$) on the grid boundary.

Iso-Surface Extraction. The isovalue τ is chosen as the median of ϕ evaluated at the input point positions. Marching cubes [LC87] then extracts the triangulated iso-surface $\{x : \phi(x) = \tau\}$.

Theorem 20.38 (BPA Topology Recovery). *Let S be a compact surface without boundary, and let P be an ε -sample of S , meaning every point $q \in S$ has some $p_i \in P$ with $\|q - p_i\| \leq \varepsilon \cdot \text{med}(S)$, where $\text{med}(S)$ is the medial axis distance. If $\rho \geq \varepsilon \cdot \text{med}(S)$ and the sampling is sufficiently dense, then the Ball Pivoting Algorithm with ball radius ρ recovers a mesh homeomorphic to S .*

Proof Sketch. By the ε -sampling condition, every point on S is within distance $\varepsilon \cdot \text{med}(S)$ of a sample. The medial axis guarantees that the local feature size controls the sampling density needed for topological correctness. The ball of radius ρ is large enough to bridge adjacent samples while small enough to avoid puncturing through thin features of S . The empty ball property ensures that each pivoting step selects a point lying on the tangent plane of the local surface patch, preserving the local Delaunay property. The front propagation covers S without introducing topological artifacts because the ε -sample condition prevents the ball from “jumping” across surface features. A complete proof requires showing that the front never creates handles or tunnels, which follows from the feature-size argument of Amenta and Bern [ABE98]. $\square$

Example 20.39 (Ball Pivoting on Four Points). Consider four points forming a tetrahedron: $p_0 = (0, 0, 0)$, $p_1 = (1, 0, 0)$, $p_2 = (0, 1, 0)$, $p_3 = (0, 0, 1)$, with ball radius $\rho = 1.2$.

1. **Seed triangle:** The circumradius of $\triangle(p_0, p_1, p_2)$ is $\frac{\sqrt{2}}{2} \approx 0.707 < \rho$. The ball center is at $c = (\frac{1}{2}, \frac{1}{2}, h)$ with $h = \sqrt{\rho^2 - 0.5} \approx 0.872$. The empty ball check confirms that p_3 lies outside (distance to $c \approx 0.866 > \rho$). The seed triangle $\triangle(p_0, p_1, p_2)$ is accepted.
2. **Pivoting edge (p_0, p_1) :** The ball pivots around (p_0, p_1) starting from p_2 . The candidate p_3 yields a new ball center with circumradius of $\triangle(p_0, p_1, p_3) = \frac{\sqrt{2}}{2}$, same empty ball check. The triangle $\triangle(p_0, p_1, p_3)$ is added.
3. The process continues until all boundary edges are covered. The result is four triangles forming the tetrahedron surface.

20.10.4 Pseudocode

20.10.5 Parameter Selection

- **Ball Radius (ρ):** The most critical parameter for BPA. Typically set to $\rho \approx 2\text{--}5 \times$ the average nearest-neighbor distance. Too small a radius results in holes in the reconstruction; too large a radius causes the ball to bridge across concavities and produce an over-smoothed surface.
- **Grid Resolution (Poisson):** Controls the spatial fidelity of the implicit function. Higher resolution yields finer detail but increases memory and solve time as $O(\text{res}^3)$. A typical starting value is $\text{res} = 32$, increased to 64–128 for fine detail.
- **Isovalue (Poisson):** Set as the median of the implicit function ϕ evaluated at the input points. This places the extracted surface “on” the samples.

20.10.6 Complexity

- **BPA Time Complexity:** In the worst case $O(n^2)$ per seed search and $O(n)$ per pivot step. With spatial hashing for the empty ball test, the average-case pivoting step is $O(k \log n)$ where k is the number of candidate points, yielding an overall $O(n \cdot k \log n)$ reconstruction.
- **BPA Space Complexity:** $O(n + t)$ where n is the number of points and t is the number of output triangles.

Algorithm 52 Ball Pivoting Algorithm Reconstruction

```

1: function RECONSTRUCT(points,  $\rho$ )
2:   triangles  $\leftarrow \emptyset$ 
3:   front  $\leftarrow \emptyset$  ▷ Boundary edge front
4:   used  $\leftarrow \emptyset$ 
5:   edge_count  $\leftarrow$  empty map ▷ Edge  $\rightarrow$  triangle count
6:   seed  $\leftarrow$  FindSeedTriangle(points,  $\rho$ )
7:   if seed = null then
8:     return triangles
9:   end if
10:  AddTriangle(triangles, front, edge_count, seed)
11:  while front  $\neq \emptyset$  do
12:     $(v_1, v_2, v_{\text{opp}}) \leftarrow$  front.pop(0)
13:     $e \leftarrow$  sort( $v_1, v_2$ )
14:    if edge_count[e]  $\geq 2$  then
15:      continue ▷ Interior edge
16:    end if
17:     $p_k \leftarrow$  PivotAroundEdge( $v_1, v_2, v_{\text{opp}}$ , points,  $\rho$ )
18:    if  $p_k \neq$  null then
19:      AddTriangle(triangles, front, edge_count,  $(v_1, p_k, v_2)$ )
20:    end if
21:  end while
22:  return triangles
23: end function
24: function FINDSEEDTRIANGLE(points,  $\rho$ )
25:   for  $i \leftarrow 0$  to  $n - 1$  do
26:     for  $j \leftarrow i + 1$  to  $n - 1$  do
27:       if  $\|p_i - p_j\|^2 > 4\rho^2$  then continue
28:       end if
29:       for  $k \leftarrow j + 1$  to  $n - 1$  do
30:          $c \leftarrow$  BallCenter( $p_i, p_j, p_k, \rho$ )
31:         if  $c \neq$  null and IsEmptyBall( $c, \{i, j, k\}$ , points,  $\rho$ ) then
32:           return  $(i, j, k)$ 
33:         end if
34:       end for
35:     end for
36:   end for
37:   return null
38: end function
39: function PIVOTAROUNDEDGE( $v_1, v_2, v_{\text{opp}}$ , points,  $\rho$ )
40:   best  $\leftarrow$  null, min_angle  $\leftarrow \infty$ 
41:    $c_{\text{old}} \leftarrow$  BallCenter( $p_{v_1}, p_{v_2}, p_{v_{\text{opp}}}, \rho$ )
42:   mid  $\leftarrow \frac{1}{2}(p_{v_1} + p_{v_2})$ 
43:   for  $k \leftarrow 0$  to  $n - 1$  do
44:     if  $k = v_1$  or  $k = v_2$  or  $k = v_{\text{opp}}$  then continue
45:     end if
46:     if  $\|p_k - \text{mid}\|^2 > 4\rho^2$  then continue
47:     end if
48:      $c_{\text{new}} \leftarrow$  BallCenter( $p_{v_1}, p_{v_2}, p_k, \rho$ )
49:     if  $c_{\text{new}} =$  null then continue
50:     end if
51:     if not IsEmptyBall( $c_{\text{new}}, \{v_1, v_2, k\}$ , points,  $\rho$ ) then continue
52:     end if
53:      $\alpha \leftarrow$  PivotAngle(mid,  $p_{v_2} - p_{v_1}, c_{\text{old}}, c_{\text{new}})$ 
54:     if  $\alpha < \text{min\_angle}$  then
55:       min_angle  $\leftarrow \alpha$ , best  $\leftarrow k$ 
56:     end if

```

- **Poisson Time Complexity:** $O(N^3 \log N)$ where $N = \text{res}$, dominated by the sparse solve of the Laplacian system on the N^3 grid.
- **Poisson Space Complexity:** $O(N^3)$ for the grid and sparse matrix.

20.10.7 Usages

- **3D Scanning:** Reconstructing object surfaces from structured-light or time-of-flight scanner data.
- **LiDAR Processing:** Generating terrain and building meshes from aerial or terrestrial LiDAR point clouds.
- **Photogrammetry:** Converting multi-view stereo point clouds into watertight 3D models.
- **Cultural Heritage:** Digital preservation of sculptures, buildings, and archaeological sites.
- **Medical Imaging:** Extracting organ surfaces from volumetric scans (CT, MRI) for surgical planning.

20.10.8 Testing Methods and Edge Cases

- **Sphere Sampling:** Reconstructing a sphere from uniformly sampled points should yield a mesh homeomorphic to S^2 with correct genus.
- **Sparse Points:** Verify graceful degradation when the point cloud is too sparse for the chosen ball radius.
- **Collinear Points:** The ball center computation must return `null` for degenerate collinear triples.
- **Duplicate Points:** Duplicates should not cause infinite loops or degenerate triangles.
- **Isolated Components:** Point clouds with disconnected parts should each be reconstructed independently.
- **Poisson with Noisy Normals:** Incorrect normals lead to inverted or distorted iso-surfaces; test with synthetic noise.

20.10.9 References

- Bernardini, F., Mittleman, J., Rushmeier, H., Silva, C., & Taubin, G. [BMR⁺99] “The Ball-Pivoting Algorithm for Surface Reconstruction”. IEEE TVCG.
- Kazhdan, M., Bolitho, M., & Hoppe, H. [KBH06] “Poisson Surface Reconstruction”. Eurographics Symposium on Geometry Processing.
- Amenta, N., & Bern, M. [ABE98] “Surface Reconstruction by Voronoi Filtering”. Discrete & Computational Geometry.
- Lorensen, W. E., & Cline, H. E. [LC87] “Marching Cubes: A High Resolution 3D Surface Construction Algorithm”. ACM SIGGRAPH.

20.11 α -Shapes

20.11.1 Overview

An α -shape is a family of geometric structures that generalizes the convex hull of a point set, providing a notion of “concave hull” or “crust” that captures the underlying shape at multiple scales. Introduced by Edelsbrunner, Kirkpatrick, and Seidel [ES83], α -shapes are obtained by filtering the simplices of a Delaunay triangulation according to a radius parameter α . As α varies from small to large values, the α -shape transitions from a collection of disconnected components to the full convex hull. This multi-scale property makes α -shapes valuable for extracting meaningful boundary geometry from point clouds in both 2D and 3D.

20.11.2 Definitions

Definition 20.40 (α -Shape). Given a finite point set $P \subset \mathbb{R}^d$ and a real parameter $\alpha > 0$, the α -shape of P is the polyhedral complex obtained by retaining every Delaunay simplex (edge in 2D, triangle in 2D, tetrahedron in 3D) whose *circumradius* is strictly less than α .

Definition 20.41 (Concave Hull). A non-convex boundary that tightly encloses a point set, capturing concavities that the convex hull omits. The α -shape boundary is a canonical example of a concave hull.

Definition 20.42 (Rolling Ball). A conceptual construction in which a ball of radius α is “rolled” along the exterior of the point set. Portions of the boundary that the ball cannot reach (because it is too large to fit into concavities) are excluded from the α -shape.

Definition 20.43 (Circumradius). The circumradius R of a simplex is the radius of its circumscribed sphere. For a triangle with side lengths a, b, c and area A , the circumradius is $R = \frac{abc}{4A}$. For a tetrahedron, the circumradius is the radius of the unique sphere passing through all four vertices.

Definition 20.44 (α -Complex). The α -complex is the abstract simplicial complex consisting of all Delaunay simplices with circumradius less than α . The α -shape is the geometric realization of this complex.

20.11.3 Theory

2D α -Shapes

Delaunay Triangulation. The algorithm begins by computing the Delaunay triangulation of the input point set P . In 2D, this yields a triangulation of the convex hull in which every triangle has the empty circumcircle property.

Circumradius Filtering. For each Delaunay triangle T_i with vertices (p_a, p_b, p_c) , the circumradius R_i is computed as $R_i = \frac{abc}{4A}$, where a, b, c are the side lengths and A is the area of T_i . If $R_i < \alpha$, the triangle is *accepted*; otherwise, it is *rejected*.

Boundary Edge Extraction. The α -shape boundary consists of all edges that belong to exactly one accepted triangle. Symmetric difference logic achieves this: each edge is added to a set when encountered in an accepted triangle; if the edge is encountered a second time, it is removed (it is an interior edge).

3D α -Shapes

In 3D, the Delaunay triangulation yields tetrahedra. Each tetrahedron is accepted or rejected based on its circumradius. The boundary of the α -shape consists of the triangular faces shared by exactly one accepted tetrahedron, again computed via symmetric difference.

Relationship to the Convex Hull. As $\alpha \rightarrow \infty$, every Delaunay simplex has circumradius less than α , so the α -shape equals the convex hull. As $\alpha \rightarrow 0$, only simplices with vanishing circumradius survive, eventually yielding only the original point set.

Theorem 20.45 (α -Shape Topology Recovery). *Let $S \subset \mathbb{R}^d$ be a compact set with a well-defined boundary, and let P be an ε -sample of S with ε sufficiently small relative to the local feature size of S . Then there exists $\alpha > 0$ such that the boundary of the α -shape of P is homeomorphic to ∂S .*

Proof Sketch. The proof relies on the stability of Delaunay triangulations under perturbation (the *smoothed analysis* framework of Amenta, Bern, and Eppstein [ABE98]). For sufficiently dense sampling (ε small enough), the Delaunay triangulation of P contains simplices whose circumradii approximate the local feature size of S . Choosing α between the maximum circumradius of simplices that cross the medial axis and the minimum circumradius of simplices that lie on the boundary surface ensures that exactly the boundary-crossing simplices are retained. The resulting α -complex is then a topological ball (or the appropriate surface type), establishing the homeomorphism. The key geometric argument is that the ε -sample condition prevents the Delaunay triangulation from introducing spurious features at scales smaller than ε . $\square$

Example 20.46 (α -Shape of Five Points in 2D). Consider five points in $\mathbb{R}^2$: $p_0 = (0, 0)$, $p_1 = (2, 0)$, $p_2 = (2, 2)$, $p_3 = (0, 2)$, $p_4 = (1, 1)$. The Delaunay triangulation produces four triangles: $T_1 = \triangle(p_0, p_1, p_4)$, $T_2 = \triangle(p_1, p_2, p_4)$, $T_3 = \triangle(p_2, p_3, p_4)$, $T_4 = \triangle(p_3, p_0, p_4)$.

1. For $\alpha = 1.5$: The circumradius of each T_i is $R = \frac{\sqrt{2}}{2} \approx 0.707 < 1.5$, so all four triangles are accepted. The boundary is the outer square (p_0, p_1, p_2, p_3) , which equals the convex hull.
2. For $\alpha = 0.5$: The circumradius $R \approx 0.707 > 0.5$, so all triangles are rejected. The α -shape degenerates to the point set alone.
3. For $\alpha = 1.0$: All four triangles are accepted (since $0.707 < 1.0$). The α -shape boundary is the square, same as the convex hull in this case.

Now consider a sixth point $p_5 = (3, 1)$ outside the square. With $\alpha = 1.0$, the Delaunay triangle $\triangle(p_1, p_2, p_5)$ has circumradius $R = 1.0$. Since $R = \alpha$ and the filter uses strict inequality, this triangle is rejected, creating a concavity in the α -shape boundary. This illustrates how α -shapes capture concavities that the convex hull does not.

20.11.4 Pseudocode

20.11.5 Parameter Selection

- **Alpha (α):** The radius of the rolling ball. For uniform point distributions, a good starting value is $\alpha \approx 2\text{--}3 \times$ the average nearest-neighbor distance. Smaller α values reveal finer concavities; larger values smooth them out. Setting $\alpha = \infty$ recovers the convex hull.
- **Adaptive Selection:** In practice, α can be chosen adaptively based on local sampling density, using the local feature size estimated from the ε -sampling condition.

Algorithm 53 α -Shape Computation (2D)

```
1: function COMPUTE2D(points,  $\alpha$ )
2:   mesh  $\leftarrow$  DelaunayTriangulation(points)
3:   boundary  $\leftarrow$  empty set  $\triangleright$  Symmetric difference of edges
4:   for each triangle  $T$  in mesh do
5:      $(v_0, v_1, v_2) \leftarrow T$ .vertices
6:      $a \leftarrow \|p_{v_1} - p_{v_0}\|$ ,  $b \leftarrow \|p_{v_2} - p_{v_1}\|$ ,  $c \leftarrow \|p_{v_0} - p_{v_2}\|$ 
7:      $s \leftarrow \frac{a+b+c}{2}$ 
8:      $A \leftarrow \sqrt{\max(0, s(s-a)(s-b)(s-c))}$ 
9:     if  $A < 10^{-12}$  then
10:       continue
11:     end if  $\triangleright$  Degenerate triangle
12:      $R \leftarrow \frac{abc}{4A}$ 
13:     if  $R < \alpha$  then
14:       for  $i \leftarrow 0$  to 2 do
15:          $e \leftarrow \text{sort}(v_i, v_{(i+1) \bmod 3})$ 
16:         if  $e \in \text{boundary}$  then
17:           boundary.remove( $e$ )
18:         else
19:           boundary.insert( $e$ )
20:         end if
21:       end for
22:     end if
23:   end for
24:   return boundary
25: end function
```

20.11.6 Complexity

- **Time Complexity:** $O(n \log n)$, dominated by the Delaunay triangulation in 2D (or $O(n^2)$ in the worst case for 3D Delaunay). The circumradius filtering and boundary extraction are $O(n)$.
- **Space Complexity:** $O(n)$ to store the Delaunay triangulation and the boundary edge set.

20.11.7 Usages

- **Molecular Biology:** Computing the solvent-accessible surface and packing defects of protein structures.
- **Geographic Information Systems:** Extracting coastline or terrain boundaries from scattered elevation points.
- **Material Science:** Analyzing the grain structure and porosity of polycrystalline materials.
- **Point Cloud Processing:** Providing a concave hull alternative to convex hull for bounding irregular point distributions.
- **Robotics:** Generating obstacle boundaries from sensor point clouds with configurable tightness.

20.11.8 Testing Methods and Edge Cases

- **Convex Hull Limit:** For $\alpha \rightarrow \infty$, the α -shape must equal the convex hull of the point set.
- **Degenerate Triangles:** Collinear triples produce zero-area triangles; the circumradius computation must handle this gracefully.
- **Duplicate Points:** Identical points create degenerate simplices that must be filtered or handled.
- **Uniform Grid:** A regular grid of points should produce an α -shape whose boundary closely approximates the bounding rectangle.
- **Ring Topology:** Points on a circle should yield an α -shape that is a polygon approximating the circle for appropriate α .
- **3D Boundary Consistency:** In 3D, the boundary of the α -shape must form a closed manifold (no dangling faces).

20.11.9 References

- Edelsbrunner, H., Kirkpatrick, D., & Seidel, R. [ES83] “On the Shape of a Set of Points in the Plane”. IEEE Transactions on Information Theory.
- Amenta, N., Bern, M., & Eppstein, D. [ABE98] “The Crust and the β -Skeleton: Combinatorial Curve Reconstruction”. Graphical Models and Image Processing.
- Edelsbrunner, H. & Mücke, E. P. [EM94a] “Three-Dimensional Alpha Shapes”. ACM Transactions on Graphics.
- Botsch, M., Kobbelt, L., Pauly, M., Alliez, P., & Lévy, B. [BKP⁺10] “Polygon Mesh Processing”. CRC Press.

20.12 Surface Parameterization and Geometry

20.13 BFF Parameterization

20.13.1 1. Overview

Boundary First Flattening (BFF) is a state-of-the-art method for **conformal parameterization** of surface meshes, introduced by Keenan Crane et al. in 2017 [CWW17]. Unlike traditional methods like Harmonic Mapping or LSCM, BFF allows the user to prescribe the target boundary curvature, providing unmatched control over the flattening process (e.g., flattening to a disk, a rectangle, or a specific shape).

20.13.2 2. Definitions

Definition 20.47 (Conformal Mapping). A mapping that locally preserves angles between curves on a surface.

Definition 20.48 (Laplacian Matrix). A discrete representation of the Laplace-Beltrami operator, typically constructed using cotangent weights.

Definition 20.49 (Scale Factor). A scalar field u on the mesh representing the local scaling needed to achieve the target metric under a conformal transformation $g' = e^{2u}g$.

Definition 20.50 (Geodesic Curvature). The curvature of a curve on a surface, relative to the surface itself.

20.13.3 3. Theory

BFF is rooted in the **Yamabe equation**, which describes how the metric changes under a conformal transformation $g' = e^{2u}g$ [CWW17]. The target geodesic curvature κ' on the boundary relates to the original curvature κ and the normal derivative of the scale factor u by:

$$\kappa' = e^{-u} \left(\kappa + \frac{\partial u}{\partial n} \right)$$

BFF solves this by:

1. Determining boundary scale factors u that satisfy the target curvature κ' .
2. Extending u to the interior via a Dirichlet problem.
3. Integrating the flattened boundary and extending the UV coordinates harmonically to the interior.

Example 20.51 (BFF Disk Flattening). Consider a hemisphere mesh with a circular boundary. To flatten it to a disk, BFF sets the target geodesic curvature to $\kappa' = 2\pi/n$ at each of the n boundary vertices (uniform curvature). The boundary system $L_{bb} \cdot u_b = \kappa' - \kappa$ is solved for the boundary scale factors, which encode how much each boundary edge must shrink or stretch. The interior scale factors are then determined by the Dirichlet problem, and the final UV coordinates are obtained by integrating the scaled edge lengths along the boundary and extending harmonically to the interior.

Algorithm 54 BFF Parameterization

```

1: function BFFPARAMETERIZATION( $M$ )
2:   Build the cotangent Laplacian matrix  $L$  for mesh  $M$ 
3:   Identify the boundary loop of  $M$ 
4:   Compute the current geodesic curvature  $\kappa$  at each boundary vertex
5:   Define the target curvature  $\kappa_{\text{target}}$  (e.g.,  $2\pi/n$  for disk)
6:   Solve the boundary system for scale factors  $u_b$ :  $L_{bb} \cdot u_b = \kappa_{\text{target}} - \kappa$ 
7:   Solve the Dirichlet problem for interior scale factors  $u_i$ :  $L_{ii} \cdot u_i = -L_{ib} \cdot u_b$ 
8:   Integrate boundary coordinates  $uv_b$  using  $l_{\text{flat}} = l_{\text{orig}} \cdot \exp\left(\frac{u_i + u_j}{2}\right)$ 
9:   Extend  $uv_b$  to the interior via harmonic mapping:  $L_{ii} \cdot uv_i = -L_{ib} \cdot uv_b$ 
10:  return  $UV$  coordinates for each vertex
11: end function

```

20.13.4 4. Pseudocode**20.13.5** 5. Parameter Selection

- **Target Curvature:** For a disk, set to $2\pi/n$. For a rectangle, set to $\pi/2$ at four vertices and 0 elsewhere.
- **Solver:** Sparse direct solvers (like SuperLU) are recommended for solving the Laplacian systems.

20.13.6 6. Complexity

- **Construction:** $O(N)$ for building the Laplacian.
- **Solving:** $O(N^{1.5})$ to $O(N^2)$ depending on mesh connectivity and sparse solver performance.

20.13.7 7. Usages

Implemented in `compgeom.mesh.surface.parameterization_bff.BFFParameterizer`. Used for UV unwrapping, texture mapping, and surface remeshing.

20.13.8 8. Testing Methods and Edge Cases

- **Testing:** Verify local angle preservation (conformal property). Check boundary curvature error.
- **Edge Cases:** Non-manifold meshes, meshes with no boundary (unsupported), and self-intersecting UV outputs.

20.13.9 9. References

- Crane, K., Weischedel, C., & Wardetzky, M. [CWW17] “Boundary first flattening.” ACM TOG.
- Wikipedia: Conformal mapping.

20.14 Harmonic Mapping**20.14.1** 1. Overview

Mesh Topology Transfer is the process of adapting a source mesh’s connectivity (its faces and edges) to a new geometric boundary. This is achieved using Harmonic Mapping, a technique

that positions interior vertices to minimize a “stretching” energy (Dirichlet energy). The result is a new mesh that shares the same topology as the source but fits the geometry of the target domain with minimal distortion.

20.14.2 2. Definitions

Definition 20.52 (Topology). The connectivity of the mesh (which vertices form which faces), independent of their spatial coordinates.

Definition 20.53 (Harmonic Map). A mapping between geometric domains that satisfies the Laplace equation $\Delta f = 0$.

Definition 20.54 (Barycentric Mapping). A discrete version of harmonic mapping where every interior vertex is the weighted average of its neighbors.

Definition 20.55 (Dirichlet Energy). A measure of how much a mapping “stretches” the domain; harmonic maps are the minimizers of this energy functional.

20.14.3 3. Theory

Based on Tutte’s Embedding Theorem [Tut63], a valid planar embedding of a graph can be found by fixing its boundary to a convex polygon and solving for the interior positions such that each vertex is at the centroid of its neighbors.

Theorem 20.56 (Tutte’s Embedding Theorem). *Let G be a planar graph with a distinguished boundary cycle mapped to a convex polygon. If every interior vertex is placed at the convex combination of its neighbors with strictly positive weights, the resulting embedding is a planar embedding with no edge crossings.*

Proof. Tutte’s proof [Tut63] uses a variational argument. The embedding minimizes the Dirichlet energy $E = \sum_{(i,j) \in E} w_{ij} \|p_i - p_j\|^2$ subject to the boundary constraints. Since the domain is convex and the weights are positive, the energy functional is strictly convex on the interior variables, guaranteeing a unique minimum. At the minimum, each interior vertex satisfies $\nabla_{p_i} E = 0$, which gives $p_i = \frac{1}{\sum_j w_{ij}} \sum_j w_{ij} p_j$ —exactly the weighted average. The convexity of the target polygon, combined with the maximum principle for harmonic functions, ensures that no edge crossings occur. $\square$

Mathematically, for each interior vertex i , we solve:

$$V_i = \frac{1}{\sum_{j \in N(i)} w_{ij}} \sum_{j \in N(i)} w_{ij} V_j$$

Where w_{ij} are weights (often $w_{ij} = 1$ for simple barycentric mapping or cotangent weights for more accurate harmonic maps). This forms a system of linear equations $LV = B$, where L is the Laplacian matrix and B contains boundary constraints.

Example 20.57 (Harmonic Mapping of a Square). Consider a 4×4 grid of vertices (a planar quad mesh) with the boundary vertices fixed to a unit square. Using uniform weights ($w_{ij} = 1$), each interior vertex is positioned at the arithmetic mean of its 4 neighbors. After convergence (typically 50–100 Gauss-Seidel iterations), the interior vertices distribute uniformly, producing a smooth parameterization of the square domain. This is the discrete harmonic map—the unique minimizer of the Dirichlet energy.

Algorithm 55 Transfer Topology

```

1: function TRANSFERTOPOLOGY(source_mesh, target_boundary)
2:   source_boundary  $\leftarrow$  GetBoundaryCycle(source_mesh)
3:   MapBoundary(source_boundary, target_boundary) ▷ Using arc-length
4:   target_centroid  $\leftarrow$  Centroid(target_boundary)
5:   for  $v \in \text{InteriorVertices}(\textit{source\_mesh})$  do
6:      $v.\textit{position} \leftarrow \textit{target\_centroid}$ 
7:   end for
8:   max_delta  $\leftarrow \infty$ 
9:   while max_delta >  $\epsilon$  do
10:    max_delta  $\leftarrow 0$ 
11:    for  $v \in \text{InteriorVertices}(\textit{source\_mesh})$  do
12:      old_pos  $\leftarrow v.\textit{position}$ 
13:      neighbors  $\leftarrow \text{GetNeighbors}(v, \textit{source\_mesh})$ 
14:       $v.\textit{position} \leftarrow \frac{\sum n.\textit{position}}{|\textit{neighbors}|}$ 
15:      max_delta  $\leftarrow \max(\textit{max\_delta}, \text{dist}(\textit{old\_pos}, v.\textit{position}))$ 
16:    end for
17:  end while
18:  return CreateMesh(new_positions, source_mesh.faces)
19: end function

```

20.14.4 4. Pseudocode**20.14.5 5. Parameter Selection**

- **Iteration Count:** Typically 100–1000 iterations for the Gauss-Seidel solver, or use a direct sparse solver for large meshes.
- **Boundary Mapping:** Usually based on normalized arc-length to prevent the boundary vertices from “bunching up” in corners.

20.14.6 6. Complexity

- **Time Complexity:** $O(n \cdot i)$ for the iterative solver (where n is vertices, i is iterations). $O(n \log n)$ or $O(n^{1.5})$ for direct sparse solvers.
- **Space Complexity:** $O(n + e)$ to store the mesh connectivity and vertex positions.

20.14.7 7. Usages

- **Shape Morphing:** Smoothly transitioning one 3D model into another with the same topology.
- **Texture Mapping:** Unwrapping a 3D surface onto a 2D plane (UV mapping).
- **Remeshing:** Transferring a high-quality triangulation onto a newly defined boundary.
- **Surface Reconstruction:** Mapping a template mesh onto scanned point cloud data.

20.14.8 8. Testing Methods and Edge Cases

- **Non-Convex Boundaries:** Tutte’s theorem only guarantees no edge crossings for convex boundaries. For non-convex boundaries, self-intersections (flipped triangles) may occur.

- **Disconnected Components:** The algorithm must be applied to each connected component of the mesh separately.
- **High Valence Vertices:** Vertices with many neighbors may converge more slowly.
- **Mesheres with Holes:** Requires mapping multiple boundary cycles (e.g., one to the outer boundary, others to internal “islands”).

20.14.9 9. References

- Tutte, W. T. [Tut63] “How to draw a graph.” Proceedings of the London Mathematical Society.
- Floater, M. S. (1997). “Parametrization and smooth approximation of surface triangulations.” Computer Aided Geometric Design.
- Pinkall, U., & Polthier, K. (1993). “Computing discrete minimal surfaces and their conjugates.” Computing.
- Wikipedia: Tutte embedding.

20.15 Functional Maps

20.15.1 1. Overview

Functional maps are a powerful framework for finding correspondences between 3D shapes [OBGS⁺12]. Instead of mapping points directly to points, which is a combinatorial problem, functional maps represent the correspondence as a linear operator between **functions** defined on the surfaces. This transformation from a point-based to a functional representation turns the non-convex matching problem into a much simpler linear algebra problem in a reduced basis.

20.15.2 2. Definitions

Definition 20.58 (Basis Functions). A set of smooth functions $\{\phi_i\}$ defined on a surface, typically the eigenfunctions of the Laplace-Beltrami operator.

Definition 20.59 (Functional Map). A matrix C that maps coefficients of a function in shape A ’s basis to coefficients in shape B ’s basis.

Definition 20.60 (Correspondence). A mapping Π from vertices of shape A to vertices of shape B .

Definition 20.61 (Commutativity). The property that a functional map should commute with other geometric operators (like the Laplacian), i.e., $C\Delta_A \approx \Delta_B C$.

20.15.3 3. Theory

Given two shapes M and N , we compute the first k eigenfunctions of their respective Laplacians: $\Phi = [\phi_1, \dots, \phi_k]$ and $\Psi = [\psi_1, \dots, \psi_k]$. Any function f on surface M can be approximated as $f \approx \Phi \mathbf{a}$.

A correspondence between M and N can be represented by a matrix C such that if f is a function on M and g is its corresponding function on N , then $\mathbf{b} = C\mathbf{a}$, where $\mathbf{b}$ are the coefficients of g in basis Ψ .

The matrix C is found by solving an optimization problem:

$$\min_C \|CA - B\|^2 + \alpha \|C\Delta_M - \Delta_N C\|^2$$

where A and B are coefficients of “descriptor functions” (like curvature or HKS [CL06]) that should be preserved under the mapping.

Theorem 20.62 (Functional Map from Pointwise Map). *Given a pointwise correspondence $\Pi : M \rightarrow N$ between two shapes, the induced functional map is $C = \Psi^\dagger \Pi^\top \Phi$, where $\dagger$ denotes the pseudoinverse.*

Proof. For a function f on M with coefficient vector $\mathbf{a} = \Phi^\dagger f$, the pullback $\Pi_* f = f \circ \Pi^{-1}$ has coefficients $\mathbf{b} = \Psi^\dagger(f \circ \Pi^{-1})$. Substituting $f = \Phi \mathbf{a}$, we get $\mathbf{b} = \Psi^\dagger \Pi^\top \Phi \mathbf{a}$, so $C = \Psi^\dagger \Pi^\top \Phi$. $\square$

Example 20.63 (Functional Map for Shape Matching). Consider two discretizations of a sphere (shapes A and B) with a 90° rotation between them. The first k eigenfunctions of the Laplacian on both spheres are the spherical harmonics. The functional map C corresponding to the rotation is a $k \times k$ block-diagonal matrix where each block is a rotation matrix acting on the harmonic subspace of a given frequency. With $k = 50$ basis functions, C captures the rotation precisely, and the pointwise correspondence can be recovered by comparing the columns of ΨC with the columns of Φ .

20.15.4 4. Pseudocode

Algorithm 56 Compute Functional Map

```

1: function COMPUTEFUNCTIONALMAP( $mesh\_A, mesh\_B, num\_basis = 50$ )
2:    $L_A, M_A \leftarrow \text{ComputeLaplacianAndMass}(mesh\_A)$ 
3:    $L_B, M_B \leftarrow \text{ComputeLaplacianAndMass}(mesh\_B)$ 
4:    $evals\_A, \Phi \leftarrow \text{eigsh}(L_A, M_A, k = num\_basis)$ 
5:    $evals\_B, \Psi \leftarrow \text{eigsh}(L_B, M_B, k = num\_basis)$ 
6:    $desc\_A \leftarrow \text{ComputeHKS}(mesh\_A, \Phi, evals\_A)$ 
7:    $desc\_B \leftarrow \text{ComputeHKS}(mesh\_B, \Psi, evals\_B)$ 
8:    $A\_coeffs \leftarrow \Phi^\top \cdot M_A \cdot desc\_A$ 
9:    $B\_coeffs \leftarrow \Psi^\top \cdot M_B \cdot desc\_B$ 
10:   $C \leftarrow \text{SolveLeastSquares}(A\_coeffs^\top, B\_coeffs^\top)^\top$ 
11:   $C \leftarrow \text{RefineMap}(C, \Phi, \Psi)$  ▷ Optional ICP refinement
12:  return  $C$ 
13: end function

```

20.15.5 5. Parameter Selection

- **Number of Basis Functions (k):** Typically 30–100. More basis functions capture more detail but increase computation and noise sensitivity.
- **Descriptors:** High-quality descriptors (like HKS, WKS, or SHOT) are critical for a good initial map.
- **Regularization:** Weighting the Laplacian commutativity term helps ensure the map is “smooth” and “geometric.”

20.15.6 6. Complexity

- **Eigen-decomposition:** $O(N^{1.5})$ for sparse Laplacian matrices.
- **Map Calculation:** $O(k^3 + k^2 \cdot D)$ where D is the number of descriptors.
- **Point Recovery:** $O(N \cdot k)$ using nearest neighbor search in the spectral embedding.

20.15.7 7. Usages

- **Shape Matching:** Finding how one 3D character corresponds to another for animation transfer.
- **Statistical Shape Analysis:** Building a “mean shape” from a collection of scanned objects.
- **Texture Transfer:** Mapping a texture from one model to another with a different topology.
- **Symmetry Detection:** Finding self-correspondences within a single shape.
- **Shape Interpolation:** Generating intermediate shapes between two models.

20.15.8 8. Testing Methods and Edge Cases

- **Identical Shapes:** The functional map C should be an identity matrix.
- **Isometries:** For shapes that are bent but not stretched (like a human in different poses), C should be nearly orthogonal ($C^\top C \approx I$).
- **Partial Matching:** Test how the algorithm handles shapes where parts are missing (requires more advanced “Partial Functional Maps” logic).
- **Topology Changes:** Verify that functional maps can still find a reasonable matching even if one shape has a different genus.
- **Low Resolution:** Ensure the basis is stable even on coarse meshes.

20.15.9 9. References

- Ovsjanikov, M., Ben-Chen, M., Solomon, J., Butscher, A., & Guibas, L. [OBCS⁺12] “Functional maps: a flexible representation of maps between shapes.” ACM TOG.
- Solomon, J., Nguyen, A., Butscher, A., Guibas, L., & Ovsjanikov, M. (2016). “Soft maps between surfaces.” Computer Graphics Forum.
- Ovsjanikov, M., et al. (2017). “Computing and Processing Correspondences with Functional Maps.” SIGGRAPH Course.
- Wikipedia: Spectral shape analysis.

20.16 Diffusion Distance

Diffusion distance is an intrinsic metric on a Riemannian manifold (or its discrete mesh representation) that measures the connectivity of points through a diffusion process [CL06].

20.16.1 Overview

Unlike Euclidean distance, which ignores the underlying shape, or Geodesic distance, which is sensitive to small topological changes (like “short circuits”), diffusion distance is robust to noise and captures the global structure of the shape. It is based on the Heat Kernel $K_t(x, y)$, which represents the probability of a random walker moving from x to y in time t .

The diffusion distance $d_t(x, y)$ is defined as:

$$d_t^2(x, y) = \int_M |K_t(x, u) - K_t(y, u)|^2 du$$

Definition 20.64 (Diffusion Distance). Given a manifold M and a time parameter $t > 0$, the diffusion distance between points $x, y \in M$ is $d_t(x, y) = \left(\int_M |K_t(x, u) - K_t(y, u)|^2 du \right)^{1/2}$, where K_t is the heat kernel at time t .

20.16.2 Spectral Representation

In the discrete setting, diffusion distance can be computed efficiently using the eigenvalues λ_i and eigenvectors ϕ_i of the Laplace-Beltrami operator:

$$d_t^2(x, y) = \sum_{i=1}^k e^{-2\lambda_i t} (\phi_i(x) - \phi_i(y))^2$$

This allows for a **diffusion embedding** where each vertex x is mapped to a point in $\mathbb{R}^k$:

$$\Psi_t(x) = \left(e^{-\lambda_1 t} \phi_1(x), e^{-\lambda_2 t} \phi_2(x), \dots, e^{-\lambda_k t} \phi_k(x) \right)$$

The Euclidean distance in this embedding space is exactly the diffusion distance.

20.16.3 Usage

Algorithm 57 Compute Diffusion Distance

```

1: function COMPUTEDIFFUSIONDISTANCE(mesh, i, j, t)
2:   Compute eigenvalues  $\lambda$  and eigenvectors  $\phi$  of the Laplace-Beltrami operator
3:    $d^2 \leftarrow \sum_k e^{-2\lambda_k t} (\phi_k(i) - \phi_k(j))^2$ 
4:   return  $\sqrt{d^2}$ 
5: end function
6:
7: function COMPUTEDIFFUSIONEMBEDDING(mesh, t, k)
8:   Compute eigenvalues  $\lambda$  and eigenvectors  $\phi$  of the Laplace-Beltrami operator
9:   for each vertex  $x$  do
10:     $\Psi_t(x) \leftarrow (e^{-\lambda_1 t} \phi_1(x), \dots, e^{-\lambda_k t} \phi_k(x))$ 
11:   end for
12:   return  $\Psi_t$ 
13: end function

```

20.16.4 References

- Coifman, R. R., & Lafon, S. [CL06] “Diffusion maps.” Applied and Computational Harmonic Analysis.
- Bronstein, A. M., et al. “Numerical Geometry of Non-Rigid Shapes.” Springer, 2008.

20.17 Mean Curvature Flow

20.17.1 1. Overview

Polygonal Mean Curvature Flow is a geometric process used to smooth the boundaries of a polygon [GH86, Gra87]. It is a discrete version of the curve-shortening flow, where each point on a curve moves in the direction of its curvature vector. Over time, this process eliminates high-frequency noise and sharp corners, causing the polygon to become smoother and more circular.

20.17.2 2. Definitions

Definition 20.65 (Mean Curvature (Curves)). In the context of curves, this is the rate at which the tangent direction changes along the curve, i.e., $\kappa = \frac{d\theta}{ds}$ where θ is the tangent angle and s is arc length.

Definition 20.66 (Discrete Laplacian). An operator that approximates the second derivative of the vertex positions. For a vertex V_i in a polygon, $L_i = V_{i-1} - 2V_i + V_{i+1}$.

Definition 20.67 (Time Step). A parameter Δt controlling the magnitude of displacement in each iteration. Must satisfy $\Delta t < 0.5$ for numerical stability.

20.17.3 3. Theory

The flow is based on the diffusion equation (heat equation) applied to geometry: $\frac{\partial P}{\partial t} = \Delta P$. In a discrete polygon, moving a vertex V_i by its Laplacian is equivalent to moving it toward the midpoint of its two neighbors (V_{i-1} and V_{i+1}). This local averaging effect reduces the “sharpness” of vertices.

Theorem 20.68 (Gage–Hamilton–Grayson Theorem). *Any simple closed plane curve evolving under curve-shortening flow (mean curvature flow) becomes convex in finite time and subsequently shrinks to a round point.*

Proof. The proof proceeds in two stages. Gage and Hamilton (1986) [GH86] showed that any simple closed curve with non-negative curvature evolves to become convex. Grayson (1987) [Gra87] completed the argument by showing that if a curve is not initially convex, the flow eventually makes it convex, after which the Gage–Hamilton result applies. The key ingredient is the monotonicity of the width of the curve under the flow and the maximum principle applied to the curvature. $\square$

A known property of this flow is that any simple closed curve will eventually become convex and then shrink to a circular point. To use this for smoothing without losing the shape entirely, the process is usually limited to a few iterations or includes a scaling step to preserve the original perimeter or area.

Example 20.69 (Mean Curvature Flow on a Star Shape). Consider a 5-pointed star polygon with 10 vertices (5 tips, 5 valleys). At each iteration with $\Delta t = 0.1$:

- **Step 0:** The tips have large negative curvature (sharp outward corners) and the valleys have large positive curvature.
- **Step 1:** Tips move inward (toward midpoints of their neighbors), valleys move outward. The star becomes less pointy.
- **Step 5:** The polygon is noticeably smoother. The concavities are reduced.
- **Step 20:** The polygon is nearly convex and approximately elliptical.

After enough iterations (without size preservation), it would shrink to a point. With perimeter preservation, it converges to a smooth, near-circular shape.

20.17.4 4. Pseudocode**20.17.5 5. Parameter Selection**

- **Time Step (Δt):** Typically set between 0.01 and 0.2. Values above 0.5 can lead to numerical instability where the polygon “explodes.”

Algorithm 58 Mean Curvature Flow

```

1: function MEANCURVATUREFLOW(polygon, iterations, dt, preserve_size)
2:    $n \leftarrow \text{len}(\text{polygon})$ 
3:    $\text{current} \leftarrow \text{polygon}$ 
4:    $\text{original\_perimeter} \leftarrow \text{CalculatePerimeter}(\text{polygon})$ 
5:   for  $k = 1$  to  $\text{iterations}$  do
6:      $\text{next\_gen} \leftarrow []$ 
7:     for  $i = 0$  to  $n - 1$  do
8:        $L_x \leftarrow \text{current}[i-1].x - 2 \cdot \text{current}[i].x + \text{current}[i+1].x$ 
9:        $L_y \leftarrow \text{current}[i-1].y - 2 \cdot \text{current}[i].y + \text{current}[i+1].y$ 
10:       $\text{new\_x} \leftarrow \text{current}[i].x + dt \cdot L_x$ 
11:       $\text{new\_y} \leftarrow \text{current}[i].y + dt \cdot L_y$ 
12:       $\text{append}(\text{next\_gen}, \text{Point}(\text{new\_x}, \text{new\_y}))$ 
13:    end for
14:    if  $\text{preserve\_size}$  then
15:       $\text{current\_perimeter} \leftarrow \text{CalculatePerimeter}(\text{next\_gen})$ 
16:       $\text{scale} \leftarrow \text{original\_perimeter} / \text{current\_perimeter}$ 
17:       $\text{next\_gen} \leftarrow \text{Scale}(\text{next\_gen}, \text{scale})$ 
18:    end if
19:     $\text{current} \leftarrow \text{next\_gen}$ 
20:  end for
21:  return  $\text{current}$ 
22: end function

```

- **Iterations:** Depends on the level of noise. Usually 10–100 steps are sufficient for smoothing.
- **Preservation:** `keep_perimeter` or `keep_area` prevents the polygon from shrinking to a point.

20.17.6 6. Complexity

- **Time Complexity:** $O(n \cdot i)$, where n is the number of vertices and i is the number of iterations.
- **Space Complexity:** $O(n)$ to store the vertex positions.

20.17.7 7. Usages

- **Image Processing:** Smoothing the results of contour extraction.
- **GIS:** Generalizing map boundaries for different zoom levels.
- **Computer Graphics:** Creating organic-looking shapes from blocky initial geometry.
- **CAD:** Removing artifacts from digitized or scanned blueprints.

20.17.8 8. Testing Methods and Edge Cases

- **Jagged Input:** Verify that “zig-zag” patterns are flattened.
- **Stability:** Test with large Δt to identify the breaking point.
- **Self-intersection:** Check if the flow resolves or creates new self-intersections.
- **Collinear Vertices:** Verify that straight edges remain straight.

- **Drift:** Ensure the centroid remains relatively stationary if re-centering is applied.

20.17.9 9. References

- Gage, M., & Hamilton, R. S. [GH86] “The heat equation shrinking convex plane curves.” *Journal of Differential Geometry*.
- Grayson, M. A. [Gra87] “The heat equation shrinks embedded plane curves to round points.” *Journal of Differential Geometry*.
- Desbrun, M., Meyer, M., Schröder, P., & Barr, A. H. (1999). “Implicit fairing of irregular meshes using diffusion and curvature flow.” *SIGGRAPH*.
- Wikipedia: Curve-shortening flow.

20.18 Angle-Bounded Remeshing

Angle-bounded remeshing is a surface mesh refinement process that improves the quality of a triangular mesh by enforcing specific bounds on the internal angles of all triangles.

20.18.1 Overview

High-quality triangular meshes are essential for accurate physical simulations [BKP⁺10]. Small or large angles can lead to numerical instability. This remeshing technique (inspired by TriWild) iteratively modifies the mesh topology and geometry to improve the overall quality, targeting a minimum angle of 30° and a maximum angle of 120°.

20.18.2 Implementation Details

The `AngleBoundedRemesher` in `CompGeom` uses a sequence of local mesh operations:

1. **Edge Splits:** Long edges are split to reduce triangle size.
2. **Edge Collapses:** Short edges are collapsed to remove small or sliver triangles.
3. **Edge Flips:** Edges are flipped to maximize the minimum angle in the local quadrilateral.
4. **Vertex Smoothing:** Vertices are moved to their local centroids (Laplacian smoothing) to create more uniform distributions.

20.18.3 Usage

Algorithm 59 Angle-Bounded Remesh

```
1: function ANGLEBOUNDEDREMESH(mesh, min_angle, max_angle, iterations)
2:   for k = 1 to iterations do
3:     SplitLongEdges(mesh, max_angle)
4:     CollapseShortEdges(mesh, min_angle)
5:     FlipEdges(mesh, min_angle, max_angle)
6:     SmoothVertices(mesh)
7:   end for
8:   return mesh
9: end function
```

20.18.4 References

- Hu, Y., et al. “TriWild: Robust Remeshing with Angle Bounds.” ACM Transactions on Graphics (SIGGRAPH), 2021.
- Botsch, M., et al. “Polygon Mesh Processing.” CRC Press, 2010.

20.19 Affine Heat Flow

20.19.1 1. Overview

Affine heat flow is a geometric smoothing process for polygons and curves that is invariant under affine transformations (rotation, scaling, translation, and shearing). While standard heat flow (curve shortening flow) tends to evolve shapes toward circles, affine heat flow evolves them toward ellipses. It is used in image processing and computer vision for shape simplification and denoising where the affine structure of the object must be preserved.

20.19.2 2. Definitions

Definition 20.70 (Affine Invariance). A property where the result of the flow on a transformed shape is the same as the transformation applied to the flow result.

Definition 20.71 (Euclidean Curvature). The standard rate of change of the tangent direction: $\kappa = d\theta/ds$.

Definition 20.72 (Affine Curvature). A measure of curvature that remains unchanged under affine maps, defined as $\kappa_a = \kappa^{1/3}$ for planar curves.

20.19.3 3. Theory

Standard curve shortening flow is defined by $\frac{\partial P}{\partial t} = \kappa \mathbf{N}$.

Affine heat flow is defined by $\frac{\partial P}{\partial t} = \kappa^{1/3} \mathbf{N}$.

This specific power of 1/3 is what provides the affine invariance. In the discrete case (for polygons), the flow is often implemented by iteratively updating vertex positions using a weighted average of their neighbors, but with weights that account for the local area or affine metric.

A common discrete approximation for a vertex V_i with neighbors V_{i-1} and V_{i+1} is:

$$V_i^{\text{new}} = V_i + \Delta t \cdot \text{Area}(V_{i-1}, V_i, V_{i+1})^{1/3} \cdot \mathbf{N}_i$$

where $\mathbf{N}_i$ is the discrete normal vector.

Example 20.73 (Affine Heat Flow on a Sheared Square). Consider a square with vertices $(0,0), (1,0), (1,1), (0,1)$ that is sheared by the transformation $(x,y) \mapsto (x + 0.5y, y)$, giving vertices $(0,0), (1,0), (1.5,1), (0.5,1)$. Under standard curve-shortening flow, this shape would evolve toward a circle. Under affine heat flow, it evolves toward an ellipse whose eccentricity matches the affine structure of the original sheared square. If we then “un-shear” the result, we recover the same shape as applying affine heat flow directly to the original square.

20.19.4 4. Pseudocode

20.19.5 5. Parameter Selection

- **Time Step (dt):** Must be small enough to ensure stability (CFL condition).
- **Iterations:** More iterations result in smoother, more simplified shapes.
- **Area Threshold:** Small areas (near-collinear vertices) can lead to numerical instability; an epsilon threshold is often used.

Algorithm 60 Affine Polygon Smoothing

```
1: function AFFINEPOLYGONSMOOTHING(polygon, dt, iterations)
2:   current_vertices  $\leftarrow$  polygon.vertices
3:   for  $\_ = 1$  to iterations do
4:     new_vertices  $\leftarrow$  []
5:     n  $\leftarrow$  len(current_vertices)
6:     for i = 0 to n - 1 do
7:       vprev  $\leftarrow$  current_vertices[(i - 1) mod n]
8:       vcurr  $\leftarrow$  current_vertices[i]
9:       vnext  $\leftarrow$  current_vertices[(i + 1) mod n]
10:      edge1  $\leftarrow$  vcurr - vprev
11:      edge2  $\leftarrow$  vnext - vcurr
12:      area  $\leftarrow$  0.5 · |cross_product(edge1, edge2)|
13:      normal  $\leftarrow$  Normalize (Perpendicular(edge2) + Perpendicular(edge1))
14:      displacement  $\leftarrow$  area1/3 · normal · dt
15:      append(new_vertices, vcurr + displacement)
16:    end for
17:    current_vertices  $\leftarrow$  new_vertices
18:  end for
19:  return Polygon(current_vertices)
20: end function
```

20.19.6 6. Complexity

- **Time Complexity:** $O(I \cdot n)$ where I is iterations and n is vertex count.
- **Space Complexity:** $O(n)$ to store the vertex positions.

20.19.7 7. Usages

- **Shape Recognition:** Normalizing shapes to a “canonical” form before matching in a database.
- **Computer Vision:** Preprocessing object silhouettes to remove pixelation or noise while preserving geometric structure.
- **Cartography:** Simplifying coastlines or boundaries for small-scale maps.
- **Generative Design:** Creating organic, smooth shapes from rough polyline sketches.
- **Image Analysis:** Analyzing the “affine scale space” of an image.

20.19.8 8. Testing Methods and Edge Cases

- **Ellipses:** An ellipse should remain an ellipse under affine heat flow (only its size should change).
- **Squares:** A square should gradually round its corners and evolve toward an ellipse.
- **Affine Symmetry:** Verify that shearing a polygon, smoothing it, and then “un-shearing” it gives the same result as smoothing the original polygon.
- **Self-Intersections:** Ensure the flow doesn’t cause the polygon to cross itself (though this is theoretically possible for non-convex shapes).
- **Zero Area:** Handle perfectly collinear triplets of vertices.

20.19.9 9. References

- Sapiro, G., & Tannenbaum, A. (1993). “Affine invariant scale-space.” *International Journal of Computer Vision*.
- Olver, P. J., Sapiro, G., & Tannenbaum, A. (1994). “Affine invariant geometric evolutions of planar curves.” *SIAM Journal on Applied Mathematics*.
- Angenent, S., Sapiro, G., & Tannenbaum, A. (1998). “On the affine heat equation for non-convex curves.” *Journal of the American Mathematical Society*.
- Wikipedia: Affine shape adaptation.

20.20 Geodesic Distance

20.20.1 1. Overview

Finding the shortest path between two points on a 3D surface mesh (the geodesic distance) is a fundamental problem in geometric modeling [MMP87, CWW13b, Set96]. Unlike Euclidean distance, which ignores the surface, the geodesic distance must follow the “terrain” of the mesh. While the Fast Marching Method (FMM) is the most famous numerical approach, other algorithms like the MMP algorithm (exact) and the Heat Method (highly efficient) provide different trade-offs between speed and accuracy.

20.20.2 2. Definitions

Definition 20.74 (Geodesic). The shortest path between two points on a surface, restricted to stay on that surface.

Definition 20.75 (MMP Algorithm). An exact algorithm (Mitchell, Mount, Papadimitriou [MMP87]) that generalizes Dijkstra’s algorithm to continuous surfaces by propagating “windows” along edges.

Definition 20.76 (Heat Method). A modern method (Crane et al. [CWW13b]) that approximates geodesics by observing the flow of heat from a source point.

Definition 20.77 (Edge-Walking). A simple approximation that treats the mesh as a graph and uses Dijkstra’s algorithm on edges, which usually provides poor results as it cannot “cut” across faces.

20.20.3 3. Theory

MMP Algorithm (Exact)

The MMP algorithm (Mitchell, Mount, and Papadimitriou [MMP87]) works by propagating “intervals” of potential paths across the surface. When a wavefront reaches an edge, it creates a new set of intervals for the next triangle. This results in an exact polyhedral geodesic but is mathematically complex to implement.

The Heat Method (Fast Approximation)

The Heat Method [CWW13b], introduced by Keenan Crane et al. (2013), uses a three-step process based on the heat equation:

1. **Heat Flow:** Solve the heat equation $\dot{u} = \Delta u$ for a short time t , with a source at the starting vertex.

2. **Gradient Normalization:** Calculate the gradient of the heat field ∇u and normalize it to have unit length ($X = -\nabla u / |\nabla u|$).
3. **Poisson Solve:** Solve the Poisson equation $\Delta\phi = \nabla \cdot X$. The resulting scalar field ϕ is the geodesic distance to the source.

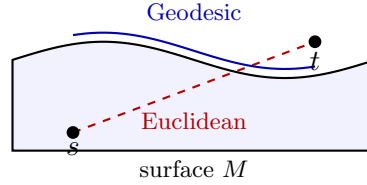


Figure 20.3: Geodesic vs. Euclidean distance on a curved surface. The geodesic (blue) follows the terrain.

Example 20.78 (Geodesic on a Cylinder). Consider a cylinder of radius r and height h . The geodesic between two points on opposite sides of the cylinder does not travel “through” the cylinder but wraps around it. If the two points are at the same height but separated by angle π (opposite sides), the geodesic distance is πr (half the circumference), not $2r$ (the Euclidean distance through the interior). The Heat Method computes this by solving the heat equation with a point source, normalizing the gradient, and solving a Poisson equation.

20.20.4 4. Pseudocode

Heat Method

Algorithm 61 Compute Geodesic via Heat Method

```

1: function COMPUTEGEODESICHEAT(mesh, source_vertex)
2:    $L \leftarrow \text{ComputeCotangentLaplacian}(\textit{mesh})$ 
3:    $A \leftarrow \text{ComputeVertexAreas}(\textit{mesh})$ 
4:    $t \leftarrow \text{mean\_edge\_length}(\textit{mesh})^2$ 
5:    $u \leftarrow \text{SolveLinearSystem}(A - t \cdot L, \textit{source\_indicator})$ 
6:    $X \leftarrow []$ 
7:   for  $\textit{face} \in \textit{mesh.faces}$  do
8:      $\textit{grad\_u} \leftarrow \text{ComputeFaceGradient}(\textit{face}, u)$ 
9:      $\text{append}(X, -\textit{grad\_u} / \text{norm}(\textit{grad\_u}))$ 
10:  end for
11:   $\textit{div\_X} \leftarrow \text{ComputeDivergence}(X, \textit{mesh})$ 
12:   $\phi \leftarrow \text{SolveLinearSystem}(L, \textit{div\_X})$ 
13:  return  $\phi - \phi[\textit{source\_vertex}]$  ▷ Shift so source is 0
14: end function

```

20.20.5 5. Parameter Selection

- **Time Step (t):** In the Heat Method, t is typically set to the square of the average edge length. Too small t leads to numerical noise; too large t leads to smoothing errors.
- **Solver:** Sparse direct solvers (like SuperLU or CHOLMOD) are essential for the Laplacian systems.

20.20.6 6. Complexity

- **MMP Algorithm:** $O(n^2)$ in the worst case, but $O(n \log n)$ in practice.
- **Heat Method:** $O(n)$ after an initial $O(n^{1.5})$ matrix factorization. Subsequent queries from different sources are extremely fast.

20.20.7 7. Usages

- **Surface Parameterization:** Normalizing distances for texture mapping.
- **Mesh Segmentation:** Grouping vertices based on their distance from a set of “seeds.”
- **Shape Matching:** Using geodesic “fingerprints” to compare 3D models.
- **Texture Synthesis:** Propagation of patterns along a surface.
- **Medicine:** Measuring distances along the folds of the brain or surface of an organ.

20.20.8 8. Testing Methods and Edge Cases

- **Flat Mesh:** The results should match the Euclidean distance exactly.
- **Sphere:** Geodesics between any two points should follow “Great Circles.”
- **Concave Regions:** Verify that the path correctly “bends” into valleys rather than jumping across them.
- **Mesh Resolution:** Test how the accuracy of the Heat Method improves as the mesh is refined.
- **Boundaries:** Ensure the algorithm handles the edges of a surface without “leaking” or crashing.

20.20.9 9. References

- Mitchell, J. S. B., Mount, D. M., & Papadimitriou, C. H. [MMP87] “The discrete geodesic problem.” SIAM Journal on Computing.
- Crane, K., Weischedel, C., & Wardetzky, M. [CWW13b] “Geodesics in Heat: A New Approach to Computing Distance Based on Heat Flow.” ACM TOG.
- Surazhsky, V., Surazhsky, O., Kirsanov, D., Gortler, S. J., & Hoppe, H. (2005). “Fast exact and approximate geodesics on meshes.” SIGGRAPH.
- Wikipedia: Geodesic.

20.21 Medial Axis Transform

20.21.1 1. Overview

The Medial Axis of a shape is the set of all points within the shape that have more than one closest point on the shape’s boundary. Informally, it can be thought of as the “skeleton” or “midline” of the shape. Originally proposed by Harry Blum in 1967 [Blu67] as a tool for biological shape recognition, the Medial Axis Transform (MAT) is now a cornerstone of shape analysis, computer vision, and geometric modeling.

20.21.2 2. Definitions

Definition 20.79 (Medial Axis). The locus of the centers of all maximal inscribed circles that touch the boundary in at least two points.

Definition 20.80 (Maximal Inscribed Circle). A circle that is entirely contained within the shape and is not contained in any other such circle.

Definition 20.81 (Medial Axis Transform). The Medial Axis combined with the radius function (the radius of the maximal inscribed circle at each point), which together form a complete representation of the shape.

Definition 20.82 (Bifurcation Point). A point on the medial axis where the skeleton branches into multiple directions.

20.21.3 3. Theory

The Medial Axis can be understood through the **grassfire analogy** [Blu67]: if the boundary of a shape (represented by dry grass) is set on fire simultaneously at all points, the fire will propagate inward at a constant speed. The points where the fire fronts from different parts of the boundary meet and extinguish each other form the medial axis.

Mathematically, the medial axis of a polygon is composed of segments of **straight lines** (bisectors of two edges) and **parabolic arcs** (bisectors of an edge and a vertex). For 2D shapes, the medial axis is a 1D graph structure that topologically summarizes the 2D region.

Example 20.83 (Medial Axis of a Rectangle). Consider a rectangle with vertices $(0, 0)$, $(4, 0)$, $(4, 2)$, $(0, 2)$. The medial axis consists of a horizontal line segment from $(1, 1)$ to $(3, 1)$ (the bisector of the top and bottom edges) and two diagonal segments connecting the endpoints to the corners. Specifically, from $(1, 1)$ the axis branches to $(0, 0)$ and $(0, 2)$, and from $(3, 1)$ it branches to $(4, 0)$ and $(4, 2)$. The resulting skeleton is shaped like an “I” with bifurcation points at $(1, 1)$ and $(3, 1)$.

20.21.4 4. Pseudocode

Algorithm 62 Medial Axis

```

1: function MEDIALAXIS(polygon)
2:    $VD \leftarrow \text{Voronoi}(\text{polygon.edges}, \text{polygon.vertices})$ 
3:    $\text{skeleton\_edges} \leftarrow []$ 
4:   for  $\text{edge} \in VD.\text{edges}$  do
5:     if  $\text{IsInside}(\text{edge}, \text{polygon})$  then
6:        $\text{append}(\text{skeleton\_edges}, \text{edge})$ 
7:     end if
8:   end for
9:    $\text{pruned\_skeleton} \leftarrow \text{Prune}(\text{skeleton\_edges}, \text{threshold})$ 
10:  return  $\text{pruned\_skeleton}$ 
11: end function
12:
13: function ISINSIDE( $\text{edge}, \text{polygon}$ )
14:  return  $\text{PointInPolygon}(\text{edge.midpoint}, \text{polygon})$ 
15: end function

```

20.21.5 5. Parameter Selection

- **Pruning Threshold:** A crucial parameter that determines which branches are kept. Without pruning, even a tiny bump on the boundary creates a long branch in the medial axis.
- **Voronoi Precision:** Robustness is key, especially when dealing with nearly collinear segments or sharp corners.

20.21.6 6. Complexity

- **Time Complexity:** $O(n \log n)$ for a polygon with n vertices, leveraging the fact that the medial axis is a subset of the Voronoi diagram of the polygon's edges and vertices.
- **Space Complexity:** $O(n)$ to store the resulting graph structure.

20.21.7 7. Usages

- **Character Recognition (OCR):** Extracting the “bone” structure of letters for identification.
- **Shape Simplification:** Representing complex 2D shapes by their 1D skeletons for faster processing.
- **Motion Planning:** Finding the safest path for a robot (the medial axis is as far away from the walls as possible).
- **Mesh Generation:** Using the skeleton to guide the placement of elements in a structured grid.
- **Animation:** Creating skeletal rigs for 2D character deformation.

20.21.8 8. Testing Methods and Edge Cases

- **Convex Polygons:** The medial axis is purely composed of straight line segments.
- **Concave Polygons:** Contains parabolic segments where a vertex is the closest point to one side.
- **Noise on Boundary:** Verify that pruning correctly handles small irregularities without destroying the main structure.
- **Circular Shapes:** The medial axis of a perfect circle is a single point at its center.
- **Parallel Edges:** The medial axis is exactly halfway between the edges.

20.21.9 9. References

- Blum, H. [Blu67] “A Transformation for Extracting New Descriptors of Shape.” Models for the Perception of Speech and Visual Form.
- Chin, F., Snoeyink, J., & Wang, C. A. (1999). “Finding the Medial Axis of a Simple Polygon in Linear Time.” Discrete & Computational Geometry.
- Wikipedia: Medial axis.

20.22 Straight Skeleton

20.22.1 1. Overview

The straight skeleton of a polygon is a 1D skeleton structure topologically similar to the medial axis, but composed entirely of straight line segments. It is formed by a “wavefront propagation” process where the edges of the polygon move inward parallel to themselves at a constant speed. The paths traced by the vertices of the shrinking polygon form the straight skeleton.

20.22.2 2. Definitions

Definition 20.84 (Straight Skeleton). The locus of vertices as the edges of a polygon move inward at a uniform speed.

Definition 20.85 (Wavefront). The shrinking polygon boundary at any point in time during the propagation.

Definition 20.86 (Bisector). The ray originating from a polygon vertex that bisects the angle formed by its two incident edges.

Definition 20.87 (Edge Event). An event where an edge shrinks to zero length and disappears, causing its two neighboring vertices to merge into one.

Definition 20.88 (Split Event). An event where a vertex moves into and splits an edge of the wavefront into two separate parts.

20.22.3 3. Theory

Unlike the medial axis, which can contain curved (parabolic) segments, the straight skeleton is piecewise linear because each point on the skeleton is the intersection of two straight-line edge trajectories.

The construction of a straight skeleton is typically modeled as a **kinetic process**. We keep track of the time until the next “event” (either an edge disappearing or an edge being split). As events occur, the topology of the wavefront changes, and new events are calculated. This process continues until the entire polygon has collapsed.

Example 20.89 (Straight Skeleton of a Triangle). Consider a triangle with vertices A , B , C . As the three edges move inward at unit speed, the three bisectors meet at a single point (the incenter). The straight skeleton is a star with three edges connecting the incenter to the three original vertices. There are three edge events (one per original edge disappearing) but no split events. The time to collapse is r/v where r is the inradius and v is the wavefront speed.

20.22.4 4. Pseudocode

20.22.5 5. Parameter Selection

- **Precision:** Highly sensitive to numerical precision during event time calculations, especially for nearly parallel edges.
- **Degeneracies:** Multiple events occurring at the exact same time (e.g., in a square or a regular hexagon) must be handled carefully.

Algorithm 63 Straight Skeleton

```
1: function STRAIGHTSKELETON(polygon)
2:   events  $\leftarrow$  PriorityQueue()
3:   for  $v \in \text{polygon.vertices}$  do
4:     events.push(CalculateNextEvent( $v, \text{polygon}$ ))
5:   end for
6:   skeleton_edges  $\leftarrow$  []
7:   while  $\neg \text{events.empty}()$  do
8:     event  $\leftarrow$  events.pop()
9:     if event.type = "EDGE" then
10:      HandleEdgeEvent(event, skeleton_edges, events)
11:     else if event.type = "SPLIT" then
12:      HandleSplitEvent(event, skeleton_edges, events)
13:     end if
14:     UpdateNeighbors(event, events)
15:   end while
16:   return skeleton_edges
17: end function
```

20.22.6 6. Complexity

- **Time Complexity:** $O(n \log n)$ for simple polygons, using more advanced data structures (like kinetic priority queues). A simpler $O(n^2)$ approach is often implemented.
- **Space Complexity:** $O(n)$ to store the polygon wavefront and the resulting skeleton edges.

20.22.7 7. Usages

- **Roof Construction:** Defining the ridges and valleys of a roof for a given house footprint.
- **Corridor Generation:** Creating paths in architectural layouts.
- **Offsetting:** Generating inward/outward offsets of polygons (mitered offsets).
- **Computer Graphics:** Creating “puffy” 3D effects from 2D shapes (Beveling).
- **GIS:** Generating centerlines of road networks.

20.22.8 8. Testing Methods and Edge Cases

- **Convex Polygons:** In this case, the straight skeleton and the medial axis are topologically similar, though the straight skeleton remains linear.
- **Concave Polygons:** Split events are common and must be handled to ensure the skeleton correctly reflects the topology.
- **Holes:** The straight skeleton can be extended to polygons with holes, where the outer boundary moves in and the holes move out.
- **Collinear Edges:** Edges that are almost in a straight line can lead to events far into the future (numerical instability).
- **Symmetry:** Regular polygons should result in a perfectly symmetric skeleton meeting at a single point.

20.22.9 9. References

- Aichholzer, O., Aurenhammer, F., Alberts, D., & Gärtner, B. (1995). “A novel type of skeleton for polygons.” *Journal of Universal Computer Science*.
- Eppstein, D., & Erickson, J. (1999). “Raising Roofs, Formulating Foundations, and Dissecting Designs: Techniques for Discrete Dirichlet Problems.” *Discrete & Computational Geometry*.
- Wikipedia: Straight skeleton.

20.23 Mesh Cut Loops

20.23.1 1. Overview

Mesh cut loops (or cut graphs) are sets of edges on a surface mesh that, when removed, transform the mesh into a single topological disk (a genus-0 surface with a single boundary). Identifying these loops is a prerequisite for many surface parameterization algorithms (like BFF [CWW17] or LSCM), as these algorithms require the surface to be homeomorphic to a disk before they can flatten it into 2D UV space.

20.23.2 2. Definitions

Definition 20.90 (Genus). The number g of “handles” on a surface. A mesh with genus g requires $2g$ non-separating loops to be cut into a disk.

Definition 20.91 (Fundamental Loops). A basis for the first homology group of the surface; these loops “wrap around” the handles or holes.

Definition 20.92 (Cut Graph). A set of edges such that the remaining mesh is a topological disk.

Definition 20.93 (Tree-Cotree Decomposition). A technique for constructing a cut graph using a spanning tree of the primal mesh and a spanning tree of the dual mesh.

20.23.3 3. Theory

For a closed manifold mesh with genus g , the Euler characteristic is $\chi = V - E + F = 2 - 2g$. A cut graph that reduces this surface to a disk must contain $2g$ independent cycles that meet at a single vertex (forming a “wedge of circles”).

A robust method for finding a cut graph is the **Tree-Cotree Decomposition**:

1. Compute a **Spanning Tree** T of the vertices and edges of the mesh.
2. Compute a **Dual Spanning Tree** T^* of the faces and dual edges, but only using dual edges whose primal counterparts are NOT in T .
3. The edges that are in neither T nor T^* form the **Cut Graph**.

This algorithm ensures that the cut graph is minimal and that cutting along it results in exactly one connected component with no handles.

Example 20.94 (Cut Graph on a Torus). A torus has genus $g = 1$, so its Euler characteristic is $\chi = 0$. The tree-cotree decomposition produces a cut graph with exactly $2g = 2$ independent cycles. These two cycles correspond to the meridian (wrapping around the tube) and the longitude (wrapping through the hole). Cutting along both cycles opens the torus into a single rectangle (topological disk), which can then be parameterized.

20.23.4 4. Pseudocode

Algorithm 64 Find Cut Graph

```

1: function FINDCUTGRAPH(mesh)
2:   primal_tree  $\leftarrow$  ComputeSpanningTree(mesh.vertices, mesh.edges)
3:   candidate_dual  $\leftarrow$  [e | e  $\in$  mesh.edges  $\wedge$  e  $\notin$  primal_tree]
4:   dual_tree  $\leftarrow$  ComputeDualSpanningTree(mesh.faces, candidate_dual)
5:   cut_graph_edges  $\leftarrow$  []
6:   for e  $\in$  mesh.edges do
7:     if e  $\notin$  primal_tree  $\wedge$  e_dual(e)  $\notin$  dual_tree then
8:       append(cut_graph_edges, e)
9:     end if
10:  end for
11:  return cut_graph_edges
12: end function

```

20.23.5 5. Parameter Selection

- **Root Selection:** The choice of root for the primal and dual trees affects the shape of the cut graph but not its topological property.
- **Edge Weights:** Using weights (e.g., edge lengths) in the spanning tree calculation (Kruskal’s algorithm) can result in “shorter” or more “geometric” cut paths.

20.23.6 6. Complexity

- **Time Complexity:** $O(E \log E)$ or $O(E)$ depending on the spanning tree algorithm used.
- **Space Complexity:** $O(E)$ to store the primal and dual tree structures.

20.23.7 7. Usages

- **Surface Parameterization (UV Unwrapping):** Cutting a torus or a complex organic model so it can be laid flat in 2D.
- **Texture Mapping:** Defining the “seams” of a 3D model where the texture coordinates are discontinuous.
- **Mesh Morphing:** Aligning the topology of two different meshes before interpolating between them.
- **Computational Topology:** Calculating the homology or Betti numbers of a surface.
- **Geometry Compression:** Cutting a mesh into a single spanning triangle strip or a predictable layout.

20.23.8 8. Testing Methods and Edge Cases

- **Genus-0 Sphere:** The cut graph should be empty (or a single point/edge if a boundary is forced).
- **Torus (Genus-1):** The cut graph should consist of two loops (meridian and longitude) meeting at a vertex.

- **Multiple Components:** Ensure the algorithm handles disconnected parts of the mesh correctly.
- **Boundary Handling:** If the mesh already has a boundary, the cut graph only needs to “connect” the boundary to the handles.
- **Watertight Check:** After cutting, the resulting mesh should have exactly one connected component and its Euler characteristic should be 1.

20.23.9 9. References

- Erickson, J., & Whittlesey, K. (2005). “Greedy optimal homotopy and homology generators.” SODA.
- Gu, X., & Yau, S. T. (2002). “Computing conformal structures of surfaces.” Communications in Information and Systems.
- Eppstein, D. (2003). “Dynamic generators of topologically distinct cycles.” Proceedings of the 15th Canadian Conference on Computational Geometry.
- Wikipedia: Cut graph.

20.24 Mesh Tunnel Loops

20.24.1 1. Overview

Mesh tunnel loops (or handle loops) are non-separating cycles on a surface mesh that “wrap around” the handles of the surface. For a surface of genus g , there are $2g$ such fundamental loops. Identifying these loops is essential for computational topology, mesh editing (e.g., “closing” a handle), and understanding the global connectivity of complex shapes. Tunnel loops are often computed as part of a homology basis.

20.24.2 2. Definitions

Definition 20.95 (Handle). A topological feature equivalent to a bridge or a donut hole.

Definition 20.96 (Tunnel Loop). A cycle that goes “through” a handle.

Definition 20.97 (Handle Loop). A cycle that goes “around” a handle (orthogonal to the tunnel loop).

Definition 20.98 (Non-Separating Cycle). A cycle that, when removed, does not split the surface into two disconnected components.

Definition 20.99 (Homology Basis). A minimal set of cycles that generate all possible non-contractible loops on the surface.

20.24.3 3. Theory

Tunnel and handle loops come in pairs for each handle of a manifold mesh. The most common algorithm for finding them is based on the **Tree-Cotree Decomposition**:

1. Compute a **Primal Spanning Tree** T of the vertices and edges.
2. Compute a **Dual Spanning Tree** T^* of the faces and dual edges, using only dual edges whose primal edges are NOT in T .

3. The remaining edges $L = E \setminus (T \cup T^*)$ are the **generator edges**.
4. Each generator edge $e \in L$ completes exactly one cycle in $T \cup \{e\}$. These cycles form a basis for the first homology group $H_1(M)$.

For a mesh with genus g , there will be exactly $2g$ generator edges, each defining one fundamental loop.

20.24.4 4. Pseudocode

Algorithm 65 Find Tunnel Loops

```

1: function FINDTUNNELLOOPS(mesh)
2:   primal_tree  $\leftarrow$  ComputeSpanningTree(mesh.vertices, mesh.edges)
3:   candidate_dual  $\leftarrow$  [e_dual(e) | e  $\in$  mesh.edges  $\wedge$  e  $\notin$  primal_tree]
4:   dual_tree  $\leftarrow$  ComputeDualSpanningTree(mesh.faces, candidate_dual)
5:   generators  $\leftarrow$  []
6:   for e  $\in$  mesh.edges do
7:     if e  $\notin$  primal_tree  $\wedge$  e_dual(e)  $\notin$  dual_tree then
8:       append(generators, e)
9:     end if
10:  end for
11:  loops  $\leftarrow$  []
12:  for gen_edge  $\in$  generators do
13:    path  $\leftarrow$  primal_tree.GetPath(gen_edge.v1, gen_edge.v2)
14:    append(loops, path + [gen_edge])
15:  end for
16:  return loops
17: end function

```

20.24.5 5. Parameter Selection

- **Weighting:** Using edge lengths as weights in the spanning tree (Kruskal’s algorithm) ensures that the resulting loops are “short” and follow the geometric features of the handle.
- **Basis Type:** The tree-cotree method produces a **homology basis**. More advanced algorithms can produce a **canonical basis** (where loops meet at a single vertex) or **greedy optimal basis** (shortest possible loops).

20.24.6 6. Complexity

- **Time Complexity:** $O(E \log E)$ or $O(E \log V)$ for the spanning tree construction.
- **Space Complexity:** $O(E)$ to store the primal and dual trees.

20.24.7 7. Usages

- **Mesh Simplification:** Removing handles to reduce the genus of a mesh (e.g., making a model “watertight” for 3D printing).
- **Shape Analysis:** Calculating Betti numbers to distinguish between different topological classes.
- **Texture Mapping:** Using handle loops as natural boundaries for UV charts.

- **Animation:** Constraining deformations to preserve the topological structure of a character (e.g., not “merging” fingers).
- **Medical Imaging:** Identifying and measuring the size of holes or passages in anatomical structures (like blood vessels or lung airways).

20.24.8 8. Testing Methods and Edge Cases

- **Sphere (Genus-0):** The algorithm should find zero tunnel loops.
- **Torus (Genus-1):** Should find exactly two loops (the “donut” hole and the “tube” wrap).
- **Double Torus (Genus-2):** Should find exactly four loops.
- **Open Mesh (Boundary):** Loops that wrap around a boundary (hole) should be handled correctly (they are part of the relative homology).
- **Disconnected Mesh:** The algorithm must be applied to each connected component independently.

20.24.9 9. References

- Erickson, J., & Whittlesey, K. (2005). “Greedy optimal homotopy and homology generators.” SODA.
- Dey, T. K., Sun, J., & Wang, Y. (2010). “Approximating Cycles in a Shortest Homology Basis.” *Discrete & Computational Geometry*.
- Gu, X., & Yau, S. T. (2008). “Computational Conformal Geometry.” International Press.
- Wikipedia: Homology (mathematics).

20.25 Iterative Closest Point (ICP) Registration

20.25.1 1. Overview

Point cloud registration is the problem of aligning two or more point sets into a common coordinate frame. This is a fundamental task in 3D scanning, *Simultaneous Localization and Mapping* (SLAM), multi-view reconstruction, and robotics, where measurements from different sensors or viewpoints must be fused into a single coherent model [BM92]. The **Iterative Closest Point** (ICP) algorithm, introduced by Besl and McKay [BM92] and independently by Chen and Medioni [CM92], is the most widely used method for rigid registration. ICP iteratively establishes correspondences between points in the source and target clouds, then computes the rigid transformation that best aligns them, converging to a local minimum of the mean squared error (MSE) objective.

20.25.2 2. Definitions

Definition 20.100 (Point Cloud Registration). Given a source point set $P = \{p_1, \dots, p_n\}$ and a target point set $Q = \{q_1, \dots, q_m\}$ in $\mathbb{R}^3$, registration is the problem of finding a rigid transformation (R, t) with $R \in SO(3)$ and $t \in \mathbb{R}^3$ that minimizes the distance between corresponding points.

Definition 20.101 (Rigid Transformation). A mapping $T : \mathbb{R}^3 \rightarrow \mathbb{R}^3$ of the form $T(x) = Rx + t$, where R is an orthogonal matrix with $\det(R) = 1$ (i.e., $R \in SO(3)$) and t is a translation vector. Rigid transformations preserve Euclidean distances: $\|T(x) - T(y)\| = \|x - y\|$ for all $x, y \in \mathbb{R}^3$.

Definition 20.102 (Closest Point Correspondence). For each source point $p_i \in P$, the closest point in the target set Q is defined as $q_j = \arg \min_{q \in Q} \|p_i - q\|$. The index map $j(i)$ encodes this correspondence.

Definition 20.103 (Mean Squared Error Objective). The ICP objective is to find the transformation (R, t) that minimizes:

$$E(R, t) = \frac{1}{n} \sum_{i=1}^n \|Rp_i + t - q_{j(i)}\|^2$$

where $j(i)$ denotes the index of the closest target point to p_i .

20.25.3 3. Theory

The ICP algorithm alternates between two steps: (1) assigning correspondences by finding closest points, and (2) computing the optimal rigid transformation that minimizes the MSE for those correspondences.

Closest Point Assignment

Given the current estimate of the source points, each source point p_i is matched to its nearest neighbor $q_{j(i)}$ in the target set Q . This nearest-neighbor query is accelerated using a **KD-tree** [BM92], reducing the per-iteration cost from $O(nm)$ to $O(n \log m)$.

Optimal Rotation via SVD

Once correspondences are established, the centroids of the source and matched target points are computed:

$$\mu_P = \frac{1}{n} \sum_{i=1}^n p_i, \quad \mu_Q = \frac{1}{n} \sum_{i=1}^n q_{j(i)}.$$

The points are centered by subtracting their respective centroids:

$$\tilde{p}_i = p_i - \mu_P, \quad \tilde{q}_i = q_{j(i)} - \mu_Q.$$

The **cross-covariance matrix** H is then formed:

$$H = \sum_{i=1}^n \tilde{p}_i \tilde{q}_i^T = \tilde{P}^T \tilde{Q}$$

where $\tilde{P}$ and $\tilde{Q}$ are the $n \times 3$ centered point matrices.

Applying the **Singular Value Decomposition** (SVD) to H :

$$H = U \Sigma V^T$$

where U and V are 3×3 orthogonal matrices and Σ is a diagonal matrix of singular values.

The optimal rotation is:

$$R = V^T U^T$$

Theorem 20.104 (Reflection Handling). *If $\det(R) < 0$, the solution introduces a reflection (improper rotation). To enforce a proper rotation ($\det(R) = 1$), the last column of V (equivalently, the last row of V^T) is negated before recomputing R .*

After computing R , the optimal translation is:

$$t = \mu_Q - R \mu_P$$

Iterative Refinement

The source points are updated: $p_i \leftarrow Rp_i + t$, and the process repeats. The transformation is accumulated across iterations into a single 4×4 homogeneous matrix. Convergence is detected when the change in mean error between successive iterations falls below a specified `tolerance`.

Theorem 20.105 (ICP Convergence). *The ICP algorithm monotonically decreases the MSE objective $E(R, t)$ at each iteration. Since $E \geq 0$ is bounded below, the sequence of objective values converges to a local minimum.*

Proof. At iteration k , the closest point assignment step finds correspondences $j_k(i)$ that minimize the distance for each p_i independently:

$$\sum_{i=1}^n \|p_i - q_{j_k(i)}\|^2 \leq \sum_{i=1}^n \|p_i - q_{j_{k-1}(i)}\|^2$$

since $j_k(i)$ is the nearest neighbor of p_i in Q . The SVD step then finds the rotation R_k and translation t_k that minimize $\sum_{i=1}^n \|R_k p_i + t_k - q_{j_k(i)}\|^2$ over all rigid transformations. Therefore:

$$E_k = \frac{1}{n} \sum_{i=1}^n \|R_k p_i + t_k - q_{j_k(i)}\|^2 \leq \frac{1}{n} \sum_{i=1}^n \|p_i - q_{j_k(i)}\|^2 \leq \frac{1}{n} \sum_{i=1}^n \|R_{k-1} p_i^{(k-1)} + t_{k-1} - q_{j_{k-1}(i)}\|^2 = E_{k-1}.$$

The sequence $\{E_k\}$ is non-increasing and bounded below by zero, so it converges. The limit is a *local* minimum because each step is greedy and no global search over correspondences is performed. $\square$

Example 20.106 (Aligning Two Partially Overlapping Squares). Consider two overlapping unit squares in $\mathbb{R}^3$ lying in the xy -plane. The target Q has vertices at $(0, 0, 0)$, $(1, 0, 0)$, $(1, 1, 0)$, $(0, 1, 0)$. The source P is the same square rotated by 15° about the z -axis and translated by $(0.3, 0.1, 0)$:

$$R_{\text{true}} = \begin{pmatrix} \cos 15^\circ & -\sin 15^\circ & 0 \\ \sin 15^\circ & \cos 15^\circ & 0 \\ 0 & 0 & 1 \end{pmatrix}, \quad t_{\text{true}} = \begin{pmatrix} 0.3 \\ 0.1 \\ 0 \end{pmatrix}.$$

Iteration 1: Each source vertex is matched to its nearest target vertex. The centroids are computed: $\mu_P \approx (0.36, 0.19, 0)$, $\mu_Q = (0.5, 0.5, 0)$. After centering, the cross-covariance matrix

H is formed, and SVD yields $R_1 \approx \begin{pmatrix} 0.966 & 0.259 & 0 \\ -0.259 & 0.966 & 0 \\ 0 & 0 & 1 \end{pmatrix}$, close to R_{true}^{-1} . The source points are

updated, reducing the MSE.

Iteration 2: With updated source positions, new correspondences are found. The rotation refinement is smaller. The mean error drops further.

After approximately 5–8 iterations (depending on sampling density), the algorithm converges with $\text{MSE} < 10^{-5}$, recovering a transformation close to $(R_{\text{true}}^{-1}, -R_{\text{true}}^{-1}t_{\text{true}})$.

20.25.4 4. Pseudocode

20.25.5 5. Parameter Selection

- `max.iterations` (k_{max}): Maximum number of ICP iterations. Typical values range from 10 to 50. For well-aligned inputs, convergence is achieved in fewer iterations. Setting this too low may prevent convergence; too high wastes computation.

Algorithm 66 Iterative Closest Point (ICP) Registration

```

1: function ICP( $P, Q, k_{\max}, \epsilon$ )
2:    $T \leftarrow I_4$  ▷ Accumulated  $4 \times 4$  homogeneous transform
3:    $e_{\text{prev}} \leftarrow \infty$ 
4:   Build KD-tree  $\mathcal{T}$  from target points  $Q$ 
5:   for  $\ell = 1$  to  $k_{\max}$  do
6:      $(d, j) \leftarrow \mathcal{T}.\text{Query}(P)$  ▷ Closest-point query;  $j_i$  is index of nearest target to  $p_i$ 
7:      $e \leftarrow \frac{1}{n} \sum_{i=1}^n d_i$  ▷ Mean distance
8:     if  $|e_{\text{prev}} - e| < \epsilon$  then
9:       break
10:    end if
11:     $e_{\text{prev}} \leftarrow e$ 
12:     $\mu_P \leftarrow \frac{1}{n} \sum_{i=1}^n p_i$ 
13:     $\mu_Q \leftarrow \frac{1}{n} \sum_{i=1}^n q_{j_i}$ 
14:     $H \leftarrow \sum_{i=1}^n (p_i - \mu_P)(q_{j_i} - \mu_Q)^T$ 
15:     $(U, \Sigma, V^T) \leftarrow \text{SVD}(H)$ 
16:     $R \leftarrow V^T U^T$ 
17:    if  $\det(R) < 0$  then
18:      Negate the last column of  $V$ 
19:       $R \leftarrow V^T U^T$ 
20:    end if
21:     $t \leftarrow \mu_Q - R \mu_P$ 
22:     $P \leftarrow R \cdot P + t$  ▷ Transform source points
23:    Update accumulated transform:  $T \leftarrow \begin{pmatrix} R & t \\ 0 & 1 \end{pmatrix} \cdot T$ 
24:  end for
25:  return  $(P_{\text{aligned}}, T)$ 
26: end function

```

- **tolerance (ϵ):** Convergence threshold on the change in mean distance error between successive iterations. A value of 10^{-5} works well for most applications. For noisy data, a larger tolerance (e.g., 10^{-3}) avoids oscillation.
- **KD-tree Structure:** The KD-tree is built once over the target points and reused every iteration. For very large point clouds, approximate nearest-neighbor structures (e.g., random projection trees) can replace exact KD-tree queries.

20.25.6 6. Complexity

- **Time Complexity:** $O(k \cdot n \log n)$, where k is the number of iterations until convergence and $n = \max(|P|, |Q|)$. Each iteration involves an $O(n \log n)$ nearest-neighbor query (via KD-tree), $O(n)$ centroid computation, and an $O(n)$ SVD of a 3×3 matrix (constant time).
- **Space Complexity:** $O(n + m)$ to store the source and target point sets and the KD-tree.

20.25.7 7. Usages

- **3D Scanning:** Aligning multiple scans of an object taken from different viewpoints into a complete model.
- **SLAM:** Aligning successive LiDAR or depth-camera frames to track robot motion and build maps.
- **Multi-View Reconstruction:** Registering point clouds extracted from different camera views to form a dense 3D reconstruction.
- **Medical Imaging:** Aligning pre-operative CT/MRI models with intra-operative patient geometry for surgical navigation.
- **Quality Inspection:** Comparing a manufactured part's point cloud to its CAD model to detect deviations.

Implemented in `compgeom.mesh.surface.registration.MeshRegistration`.

20.25.8 8. Testing Methods and Edge Cases

- **Identity Transformation:** Source and target are identical; ICP should converge in one or zero iterations with zero error.
- **Known Rigid Transform:** Apply a known rotation and translation to generate the source, then verify that ICP recovers the inverse transformation within tolerance.
- **Partial Overlap:** Source and target share only a subset of points. ICP may converge to an incorrect alignment; testing should verify that the overlapping region is well-aligned.
- **Noise Robustness:** Add Gaussian noise to source points and verify that ICP still converges to a reasonable alignment.
- **Reflection Input:** A source that is a mirror image of the target. The reflection-handling logic ($\det(R) < 0$) should prevent convergence to a reflected solution.
- **Degenerate Configurations:** Coplanar or collinear point sets may yield ambiguous rotations. The algorithm should still produce a valid (if non-unique) alignment.
- **Empty or Mismatched Sets:** Empty source or target should raise an appropriate error.

20.25.9 9. References

- Besl, P. J., & McKay, N. D. [BM92] “A method for registration of 3-D shapes.” IEEE TPAMI, 14(2), 239–256.
- Chen, Y., & Medioni, G. [CM92] “Object modelling by registration of multiple range images.” Image and Vision Computing, 10(3), 145–155.
- Rusinkiewicz, S., & Levoy, M. (2001) “Efficient variants of the ICP algorithm.” 3DIM.
- Wikipedia: Iterative closest point.

Chapter 21

Half-Edge Data Structure

- 21.1 Half-Edge Representation
- 21.2 Vertices, Half-Edges, and Faces
- 21.3 Twin, Next, and Prev Operators
- 21.4 Boundary and Manifold Handling
- 21.5 Adjacency and Neighborhood Queries
- 21.6 Traversal Operations
- 21.7 Comparison with Other Representations
- 21.8 Incidence and Storage Complexity
- 21.9 Construction from Polygon Soup
- 21.10 Applications in Mesh Processing
- 21.11 Exercises

Chapter 22

Mesh Generation

- 22.1 Computational Meshes
- 22.2 Simplicial Complexes
- 22.3 Triangular and Tetrahedral Meshes
- 22.4 Mesh Quality Measures
- 22.5 Delaunay Mesh Generation
- 22.6 Constrained Delaunay Meshing
- 22.7 Delaunay Refinement
- 22.8 Ruppert's Algorithm
- 22.9 Chew's Algorithm
- 22.10 Advancing-Front Methods
- 22.11 Adaptive Meshing
- 22.12 Mesh Optimization
- 22.13 Finite-Element Applications
- 22.14 Exercises
- 22.15 Project: Adaptive Mesh Generator

Chapter 23

Voxel Geometry and OpenVDB

Voxel geometry represents three-dimensional shape on a regular grid of volume elements, or voxels. Where a mesh stores geometry on its boundary, a voxel representation samples the volume itself, making topological queries such as inside/outside tests, boolean operations, and morphological changes trivial and robust. The cost is memory: a dense uniform grid grows as the cube of its resolution. Voxel geometry became practical at high resolution through *sparse* representations, most prominently the OpenVDB library and its hierarchical tree of fixed-size blocks [Mus13]. This chapter develops the dense and sparse representations, signed distance fields, and the conversions between voxel and mesh form. Mesh voxelization itself is treated in Section 20.4.

23.1 Voxel Grids

23.1.1 1. Overview

A **voxel grid** is the 3D analogue of a pixel image: a rectangular array of cubic cells aligned to an axis-aligned bounding box. Each cell either records a binary occupancy (occupied or empty) or stores a scalar, vector, or other attribute sampled at the cell center. Grids support constant-time random access and simple neighbor iteration, which makes them the substrate for most volumetric algorithms [AM01, NT03].

23.1.2 2. Definitions

Definition 23.1 (Voxel). A *voxel* is a volume element: the cubic cell centered at a grid point. The grid resolution (n_x, n_y, n_z) and the voxel size s determine the box that the grid represents.

Definition 23.2 (Binary and Attribute Grids). A *binary grid* stores occupancy per voxel (0 empty, 1 occupied). An *attribute grid* stores values such as a signed distance, density, or velocity field.

Definition 23.3 (Conservative Voxelization). A voxelization is *conservative* if a voxel is occupied whenever any part of the surface passes through it; otherwise it is *exact* (a voxel is occupied only when it is pierced by the surface) [NT03].

23.1.3 3. Theory

A dense grid of resolution n^3 costs n^3 entries and thus $O(n^3)$ bytes. The occupied surface of a triangle mesh passes through only $O(n^2)$ voxels, so for high resolution almost all storage is wasted on empty space. This observation motivates the sparse schemes of Section 23.3 and the hierarchical trees of Section 23.4. The resolution must balance aliasing (too coarse) against

memory and computation time (too fine); adaptive sampling that keeps dense cells near the surface and coarsens empty regions is the resolution used in production systems [Mus13].

23.1.4 4. Pseudo code

```
function IsINSIDE( $p$ , grid, occ)
     $c \leftarrow \text{ToCell}(p, \text{grid})$ 
    return occ[ $c$ ]
end function
```

23.1.5 5. Parameters Selections

- **Resolution:** choose n so that the smallest surface feature spans several voxels; alias errors appear below 3 voxels per feature.
- **Bounds:** an axis-aligned bounding box must contain the whole object; padding avoids boundary artifacts.
- **Data type:** one bit per voxel for occupancy, 8–32 bits per voxel for scalar attributes.

23.1.6 6. Complexity

Dense grids: $O(n^3)$ space, $O(1)$ access, $O(n^3)$ build. A surface of $O(n^2)$ cells can be voxelized in $O(n^2)$ time plus $O(n^3)$ for the grid itself.

23.1.7 7. Usages

- **Physics simulation:** grid-based fluids, collision and pressure fields.
- **Medical imaging:** CT and MRI produce data natively as grids.
- **Boolean and morphological operations:** union, intersection, erosion, and dilation are voxel-wise.

23.1.8 8. Testing methods and Edge cases

- **Surface vs. solid:** verify whether boundary voxels are classified as inside.
- **Resolution sensitivity:** re-run the pipeline at two resolutions and compare the extracted surfaces.
- **Boundary alignment:** voxels exactly on the box boundary must be handled consistently.

23.1.9 9. References

- Akenine-Möller, T. (2001). “Fast 3D triangle-box overlap testing”. *Journal of Graphics Tools*.
- Nooruddin, F. S., & Turk, G. (2003). “Simplification and repair of polygonal models using volumetric techniques”. *IEEE TVCG*.
- Wikipedia: [Voxel](#)

23.2 Signed Distance Fields

23.2.1 1. Overview

A **signed distance field** (SDF) stores, at every voxel, the signed distance to the nearest surface, negative inside and positive outside. The zero set of the field is the surface itself, so blending, boolean operations, and collision queries reduce to arithmetic on the sampled field [OF03].

23.2.2 2. Definitions

Definition 23.4 (Signed Distance Field). A *signed distance field* is a function $d : \mathbb{R}^3 \rightarrow \mathbb{R}$ with $|d(x)| = \text{dist}(x, \Gamma)$, where Γ is the surface, and $\text{sgn}(d)$ indicates the side of Γ .

Definition 23.5 (Narrow Band). A *narrow band* stores d only within w voxels of Γ (typically $w = 3$); outside the band the field is clamped to $\pm w s$. This bounds the stored volume to the neighborhood of the surface [OF03, Mus13].

23.2.3 3. Theory

An SDF is usually constructed either by computing the distance to a triangle mesh with a fast sweeping or fast marching method, or by reinitializing an arbitrary level-set function so that $|\nabla d| = 1$. Once it is a true signed distance, operations are exact up to the grid resolution: the surface is extracted as the zero level set, and distance queries are direct voxel lookups. Level-set evolution updates d with a velocity field, moving the surface while preserving its implicit character [OF03].

23.2.4 4. Pseudo code

```

function BUILDSDF( $\Gamma$ , grid,  $w$ )
    initialize voxels near  $\Gamma$  with exact distances
    propagate distances outward with fast sweeping
    clamp all values to  $[-w s, +w s]$ 
    return  $d$ 
end function

```

23.2.5 5. Parameters Selections

- **Band width** w : $w = 3$ is the standard OpenVDB default; larger bands smooth boolean operations at higher cost.
- **Renormalization**: enforce $|\nabla d| = 1$ periodically so the field stays a true distance.

23.2.6 6. Complexity

Fast sweeping computes the distance to a grid of N voxels in $O(N)$ passes of $O(N)$ work; reinitialization of the narrow band is $O(w n^2)$ per sweep for an n^3 grid.

23.2.7 7. Usages

- **Boolean operations**: min / max of SDFs implement union/intersection.
- **Collision and proximity**: distance queries become lookup + clamp.
- **Level-set surface evolution** for sculpting and fluid boundaries [OF03].

23.2.8 8. Testing methods and Edge cases

- **Accuracy:** compare against analytic distance for a sphere.
- **Sign consistency:** verify the inside/outside convention matches the mesh orientation.
- **Thin features:** features thinner than 2s may vanish from the field.

23.2.9 9. References

- Osher, S., & Fedkiw, R. (2003). “Level Set Methods and Dynamic Implicit Surfaces”. Springer.
- Wikipedia: [Signed distance function](#)

23.3 Sparse Voxel Representations

23.3.1 1. Overview

Sparse voxel representations store only occupied cells, typically in a **sparse voxel octree** (SVO): a tree that subdivides the box into octants, recursing only where occupied cells exist. Hierarchical sparse volumes such as OpenVDB push the idea further, storing dense fixed-size blocks at the leaves and summarized tiles in the interior [Mus13].

23.3.2 2. Definitions

Definition 23.6 (Sparse Voxel Octree). An *SVO* is an octree whose nodes store the geometry and material of the cells they represent; nodes whose subtree is empty are pruned. Resolution increases only where the object actually exists.

Definition 23.7 (Tile). A *tile* is an internal node value that summarizes a uniform region of the volume (for example a constant SDF value) without expanding it into children. Tiles are the key to storing large homogeneous regions cheaply [Mus13].

23.3.3 3. Theory

An SVO with n occupied voxels uses $O(n \log n)$ nodes and supports ray-marching and level-of-detail rendering directly on the tree. OpenVDB’s tree generalizes the octree to arbitrary branching and depth (up to five levels) and adds *active* states: every node or tile is either active (stores topology) or inactive (implicitly background), so constant regions are represented by single tiles instead of millions of voxels [Mus13].

23.3.4 4. Pseudo code

```
function INSERT(tree, c, value)
    descend to the leaf containing cell c, expanding empty nodes
    set the leaf voxel value and mark the voxel active
    return tree
end function
```

23.3.5 5. Parameters Selections

- **Branching factor and depth:** octrees use 8; OpenVDB uses 16-child levels with 8^3 leaves.
- **Tile threshold:** collapse a subtree into a tile when all children share a value.

23.3.6 6. Complexity

Insertion and lookup descend $O(\log n)$ levels (bounded by the fixed tree depth in OpenVDB). Memory is proportional to the number of active voxels and tiles, not the bounding-box volume.

23.3.7 7. Usages

- **High-resolution volume rendering:** SVO ray tracing of gigavoxel scenes.
- **Production VFX:** OpenVDB effects in film pipelines [Mus13].

23.3.8 8. Testing methods and Edge cases

- **Deep trees:** verify behavior at the maximum supported depth.
- **Empty regions:** inserting into a fully empty tree must create only the minimal path.
- **Tile/leaf transitions:** adjacent active and inactive regions must share correct boundaries.

23.3.9 9. References

- Museth, K. (2013). “VDB: High-resolution sparse volumes with dynamic topology”. ACM Transactions on Graphics.
- Wikipedia: [Sparse voxel octree](#)

23.4 OpenVDB: Hierarchical Sparse Volumes

23.4.1 1. Overview

OpenVDB is an open-source C++ library, originated at DreamWorks and now under the Academy Software Foundation, that implements a hierarchical, sparse, dynamic topology data structure for volumetric data [Mus13]. It is the de facto standard for production voxel geometry, supporting narrow-band signed distance fields, boolean operations, smoothing, morphing, and level-set tracking at billion-voxel effective resolutions.

23.4.2 2. Definitions

Definition 23.8 (VDB Tree). A *VDB tree* has at most five levels: a root node with a hash-mapped set of children, up to three internal levels, and leaf nodes. Leaves store dense $8 \times 8 \times 8$ blocks; internal nodes store the states of their children as fixed-size arrays, and may hold a constant value (a tile) instead of children [Mus13].

Definition 23.9 (Active State). A node, tile, or voxel is *active* if its value is explicitly set and participates in topology; inactive elements are implicitly equal to the background value. All tree operations respect and maintain these states.

23.4.3 3. Theory

Each level of the tree multiplies the coordinate space by 16 in each dimension, so five levels span coordinates of 2^{19} per axis (about 524,288 voxels) while a leaf stores only 512 voxels. Because interior levels can be represented by tiles, a large flat region costs the same as a single voxel. Random access descends at most four pointer hops, and topology is dynamic: voxels can be inserted or deleted in expected $O(1)$ amortized time through the hash-mapped root, which makes level-set evolution efficient even as the interface moves [Mus13].

23.4.4 4. Pseudo code

```
function GETVALUE(tree,  $x$ )  
   $n \leftarrow \text{root}(\text{tree})$   
  for level  $L = 1 \dots 5$  do  
     $i \leftarrow \text{coord2index}_L(x)$   
    if  $n$  holds a tile then  
      return tile value  
    else if  $n[i]$  is a child then  
       $n \leftarrow n[i]$   
    else  
      return background value  
    end if  
  end for  
  return leaf voxel value at  $x$   
end function
```

23.4.5 5. Parameters Selections

- **Background value:** choose the "empty" value (e.g., $\pm\infty$ for SDF grids) consistently; it determines inactive semantics.
- **Tree configuration:** leaf log-dimension 3 and internal log-dimension 4 are defaults; larger leaf blocks trade memory for iteration speed.
- **Narrow-band mode:** for SDF grids, band width 3 is standard [Mus13].

23.4.6 6. Complexity

Access is $O(1)$ (at most five level hops, hash lookup at the root). Memory is proportional to active voxels and tiles; a 3D SDF of effective resolution 4096^3 typically requires only a few megabytes.

23.4.7 7. Usages

- **Visual effects:** level-set sculpting, liquid surfaces, and volumetric fire/smoke in film [Mus13].
- **Robotics and simulation:** signed distance collision and meshing pipelines.
- **Medical and geometry processing:** fast boolean operations on scanned geometry.

23.4.8 8. Testing methods and Edge cases

- **Bounding:** writing outside the current tree extent must grow the root's hash map correctly.
- **Tile collapse:** coalescing a region must not corrupt neighboring active voxels.
- **Level-set reinitialization:** verify the band stays $|\nabla d| = 1$ after many operations.

23.4.9 9. References

- Museth, K. (2013). "VDB: High-resolution sparse volumes with dynamic topology". ACM Transactions on Graphics.
- OpenVDB: <https://www.openvdb.org>
- Wikipedia: [OpenVDB](#)

23.5 Voxel-to-Mesh Conversion

23.5.1 1. Overview

Converting a voxel field back into a mesh recovers an explicit surface from an implicit or occupancy representation. The standard algorithm is **marching cubes**, which extracts the zero level set of an SDF (or the boundary of an occupancy grid) by triangulating each cell independently [LC87].

23.5.2 2. Definitions

Definition 23.10 (Marching Cubes). *Marching cubes* enumerates, for every grid cell, the 2^8 cases determined by which of the eight corner values are below the isovalue, and emits the corresponding polygon from a precomputed case table.

Definition 23.11 (Surface Nets). *Surface nets* place one vertex per crossing cell and connect adjacent vertices into a quad mesh, relaxing the vertices toward the surface; the result avoids some of marching cubes' topology artifacts [Gib98].

23.5.3 3. Theory

Marching cubes produces watertight surfaces for well-sampled SDFs but has ambiguous cases (15 base cases, 256 with symmetry and inversion) that must be resolved consistently to avoid cracks. Because the surface is the zero set of a sampled field, the mesh resolution is bounded by the grid resolution, and features thinner than one voxel are lost. Surface nets produce smoother results with fewer triangles at the same resolution [Gib98].

23.5.4 4. Pseudo code

```

function MARCHINGCUBES( $d, \rho$ )
   $M \leftarrow$  empty mesh
  for each cell with corners  $c_0 \dots c_7$  do
    case  $\leftarrow \sum_i 2^i [d(c_i) < \rho]$ 
    emit triangles of case from the lookup table
  end for
  return  $M$ 
end function

```

23.5.5 5. Parameters Selections

- **Iso-value ρ :** 0 for a signed distance field; 0.5 for a binary grid.
- **Vertex placement:** linear interpolation on edges vs. central placement in the cell.
- **Ambiguity resolution:** use the asymptotic-decider criterion for consistent topology.

23.5.6 6. Complexity

Marching cubes runs in $O(N)$ time for N cells and outputs $O(N)$ triangles; output is proportional to the surface area, not the volume.

23.5.7 7. Usages

- **Surface extraction** from CT/MRI scans.
- **Mesh generation** from SDF sculpting and level-set simulations.
- **Mesh repair**: voxelize a broken mesh, then extract a clean, watertight one.

23.5.8 8. Testing methods and Edge cases

- **Watertightness**: verify no cracks at cell boundaries (ambiguity cases).
- **Normals**: orient triangles consistently from the field gradient.
- **Empty grids**: fields with no sign change must produce no triangles.

23.5.9 9. References

- Lorensen, W. E., & Cline, H. E. (1987). “Marching cubes: A high resolution 3D surface construction algorithm”. Computer Graphics (SIGGRAPH).
- Gibson, S. F. F. (1998). “Constrained elastic surface nets”. MICCAI.
- Wikipedia: [Marching cubes](#)

23.6 Applications

- **Visual effects**: volumetric fluids, explosions, and organic sculpting in production using OpenVDB [Mus13].
- **Autonomous driving and robotics**: occupancy grids and signed-distance collision maps for perception and planning.
- **Medical imaging**: segmentation, surface extraction, and morphometry from volumetric scans [LC87].
- **Computational fabrication**: voxel models drive 3D printing and material assignment.

Part VIII

Motion, Visibility, and Robotics

Chapter 24

Visibility

24.1 Visibility Problems

24.2 Visibility inside Polygons

24.3 Visibility Polygons

24.4 Computing Visibility from a Point

24.5 Guarding Polygons

24.6 Visibility Graphs

24.7 Shortest Paths among Obstacles

24.8 Geodesic Paths

24.9 Applications

24.10 Exercises

24.11 Distance Transforms and Shortest Paths

24.12 Fast Marching Method

24.12.1 Overview

The Fast Marching Method (FMM) is a numerical technique for solving the **Eikonal equation**, a non-linear first-order partial differential equation that describes the evolution of a wave front. It is used to compute the shortest travel time (or distance) from a source point to all other points in a domain, even when the “speed” of travel varies across space. It can be viewed as a continuous version of Dijkstra’s algorithm [Set96, Tsi95].

24.12.2 Definitions

Definition 24.1 (Eikonal Equation). The Eikonal equation $|\nabla u(x)| = F(x)$ relates the arrival time $u(x)$ of a wave front to the reciprocal of the speed $F(x)$ at each point x in the domain. For a uniform speed $F(x) \equiv 1$, the solution $u(x)$ is the Euclidean distance from the source.

Definition 24.2 (Isotropic Speed). An isotropic speed $F(x)$ is one that depends only on the spatial location, not on the direction of travel. This contrasts with anisotropic speeds that depend on the gradient direction, as encountered in crystal growth or seismic wave propagation.

Definition 24.3 (Upwind Scheme). An upwind numerical discretization computes the solution at a point using values from “upstream” locations where the solution is already smaller (earlier in time). This ensures that information propagates causally, from regions of smaller u to regions of larger u .

24.12.3 Theory

FMM simulates the propagation of a front by moving from points where the travel time is already known to points where it is unknown. To maintain efficiency, points are categorized into three sets:

1. **Accepted:** Points where the time is correctly calculated and fixed.
2. **Trial:** Points on the boundary of the Accepted set; their times are estimated and kept in a priority queue.
3. **Far:** Points that have not been reached yet.

The key to FMM is the **upwind discretization** of the gradient $|\nabla u|$. In 2D, on a Cartesian grid with spacing h , the discretization is:

$$\max(D_{ij}^{-x}u, -D_{ij}^{+x}u, 0)^2 + \max(D_{ij}^{-y}u, -D_{ij}^{+y}u, 0)^2 = F_{ij}^2$$

where $D_{ij}^{-x}u = (u_{i-1,j} - u_{i,j})/h$ and $D_{ij}^{+x}u = (u_{i+1,j} - u_{i,j})/h$ are backward and forward finite differences. The $\max(\cdot, 0)$ operators ensure that only neighbors with smaller arrival times contribute, enforcing the upwind condition.

Theorem 24.4 (FMM Correctness). *For an isotropic speed field $F > 0$ and a single source point, the Fast Marching Method Algorithm 67 computes the viscosity solution of the Eikonal equation Theorem 24.1. That is, at convergence $U[p]$ equals the shortest travel time from the source to every grid point p .*

Proof. The proof proceeds by induction on the order in which points are moved from Trial to Accepted. Let p be the point popped from the priority queue with minimum value $U[p]$. All Accepted neighbors of p have $U[q] \leq U[p]$ by the priority queue invariant. Since FMM processes points in non-decreasing order of U , no later Accepted point can provide a shorter path to p . The upwind update then solves the quadratic Eikonal stencil using only Accepted or equal-valued neighbors, yielding the correct viscosity solution by the comparison principle for first-order PDEs. $\square$

24.12.4 Algorithm

Example 24.5 (FMM on a Uniform Grid). Consider a 5×5 grid with constant speed $F = 1$ and source at the center pixel $(2, 2)$. FMM proceeds as follows: initially $U(2, 2) = 0$ and the four neighbors are Trial with $U = 1$. The algorithm pops one of them (say $(1, 2)$) to Accepted, then updates its FAR neighbors. After processing, the distance field approximates concentric diamonds (the ℓ^∞ -ball) with values $1, \sqrt{2}, 2, \sqrt{5}, \dots$ radiating outward, converging to the true Euclidean distance field as the grid is refined.

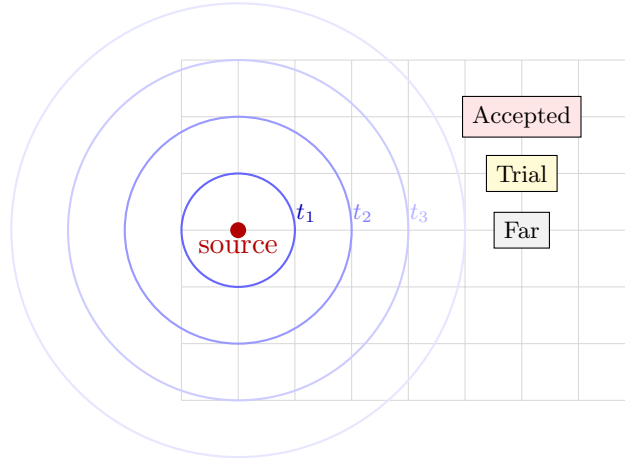


Figure 24.1: Fast Marching Method: the wavefront propagates from the source. Points are categorized as Accepted, Trial, or Far.

Algorithm 67 Fast Marching Method

```

1: function FASTMARCHING(source, speed_field)
2:    $U \leftarrow \text{Array}(\infty)$ 
3:    $\text{status} \leftarrow \text{Array}(FAR)$ 
4:    $U[\text{source}] \leftarrow 0$ 
5:    $\text{status}[\text{source}] \leftarrow ACCEPTED$ 
6:    $\text{trial\_queue} \leftarrow \text{PriorityQueue}()$ 
7:   for neighbor  $\in \text{GetNeighbors}(\text{source})$  do
8:      $U[\text{neighbor}] \leftarrow \text{CalculateEikonal}(\text{neighbor}, U, \text{speed\_field})$ 
9:      $\text{status}[\text{neighbor}] \leftarrow TRIAL$ 
10:     $\text{trial\_queue.push}(\text{neighbor}, U[\text{neighbor}])$ 
11:  end for
12:  while not  $\text{trial\_queue.empty}()$  do
13:     $p \leftarrow \text{trial\_queue.pop\_min}()$ 
14:     $\text{status}[p] \leftarrow ACCEPTED$ 
15:    for neighbor  $\in \text{GetNeighbors}(p)$  do
16:      if  $\text{status}[\text{neighbor}] = ACCEPTED$  then continue
17:    end if
18:     $\text{old\_val} \leftarrow U[\text{neighbor}]$ 
19:     $\text{new\_val} \leftarrow \text{CalculateEikonal}(\text{neighbor}, U, \text{speed\_field})$ 
20:    if  $\text{status}[\text{neighbor}] = FAR$  then
21:       $U[\text{neighbor}] \leftarrow \text{new\_val}$ 
22:       $\text{status}[\text{neighbor}] \leftarrow TRIAL$ 
23:       $\text{trial\_queue.push}(\text{neighbor}, \text{new\_val})$ 
24:    else if  $\text{new\_val} < \text{old\_val}$  then
25:       $U[\text{neighbor}] \leftarrow \text{new\_val}$ 
26:       $\text{trial\_queue.update\_priority}(\text{neighbor}, \text{new\_val})$ 
27:    end if
28:  end for
29: end while
30:  return  $U$ 
31: end function

```

24.12.5 Parameter Selections

- **Grid Resolution:** Finer grids produce more accurate wave fronts but require more memory and time.
- **Speed Function (F):** Can be constant (1.0 for Euclidean distance) or varying (to simulate terrain or obstacles).
- **Solver Order:** First-order FMM is standard; higher-order versions use more neighbors to achieve better accuracy.

24.12.6 Complexity

Theorem 24.6 (FMM Complexity). *The Fast Marching Method Algorithm 67 runs in $O(N \log N)$ time and $O(N)$ space, where N is the number of grid points.*

Proof. Each of the N grid points is moved from Far to Trial to Accepted exactly once, contributing a constant number of neighbor updates. Each update involves a constant-size quadratic solve and a priority queue operation (insert, decrease-key, or pop). A binary heap gives $O(\log N)$ per operation, yielding $O(N \log N)$ total. Space is $O(N)$ for the arrays U , $status$, and the heap. $\square$

24.12.7 Usages

- **Geodesic Distance on Meshes:** Computing distances along a surface (e.g., finding the shortest path over a mountain).
- **Image Segmentation:** Active contours (snakes) that expand to find object boundaries.
- **Robotics:** Path planning in complex environments with different “terrains” (e.g., water vs. land).
- **Medical Imaging:** Bone or organ segmentation in CT/MRI scans.
- **Seismology:** Simulating earthquake wave propagation.

24.12.8 Testing Methods and Edge Cases

- **Constant Speed:** The resulting time field should be exactly proportional to the Euclidean distance from the source.
- **Obstacles:** Set speed to zero (or $F \rightarrow \infty$) at obstacles; the front should flow around them.
- **Varying Speed:** Use a simple linear speed gradient to verify that the front bends (refraction) correctly according to Snell’s law.
- **Grid Boundaries:** Ensure the algorithm handles the edges of the domain without crashing.
- **Multiple Sources:** Initialize multiple points to 0; the result should be a Voronoi-like partition of arrival times.

24.12.9 References

- Sethian, J. A. (1996). “A fast marching level set method for monotonically advancing fronts.” *Proceedings of the National Academy of Sciences* [Set96].
- Tsitsiklis, J. N. (1995). “Efficient algorithms for globally optimal trajectories.” *IEEE Transactions on Automatic Control* [Tsi95].
- Kimmel, R., & Sethian, J. A. (1998). “Computing geodesic paths on manifolds.” *Proceedings of the National Academy of Sciences* [KS98].
- Wikipedia: Fast marching method.

24.13 Distance Map

24.13.1 Overview

A distance map (or distance transform) is a representation of a digital image or geometric scene where each point stores its distance to the nearest boundary or obstacle. For a binary image, the distance map calculates the distance from every background pixel to the closest foreground pixel. It is a fundamental tool in image processing, computer vision, and motion planning [RP66].

24.13.2 Definitions

Definition 24.7 (Distance Transform). A Distance Transform (DT) is an operator applied to binary images that produces a grayscale image where each pixel value encodes its distance to the nearest foreground (object) pixel under a chosen metric d .

Definition 24.8 (Signed Distance Function). A Signed Distance Function (SDF) is a variation of the distance transform where points inside a shape receive negative distances and points outside receive positive distances: $u(x) = \pm \min_{y \in \partial\Omega} \|x - y\|$, with the sign indicating inside/outside.

Definition 24.9 (Euclidean Distance Map). A Euclidean Distance Map (EDM) is a map where distances are computed using the true Euclidean metric $d(x, y) = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2}$, as opposed to chamfer approximations.

Definition 24.10 (Chamfer Distance). A Chamfer distance is an approximation of Euclidean distance using integer-valued masks applied in two passes. Common variants include the 3-4 chamfer (weights 3 for diagonals, 4 for axis-aligned) and the 5-7-11 chamfer for higher accuracy [RP66].

24.13.3 Theory

The most efficient algorithms for computing the Euclidean distance map are based on **separable filters** or **two-pass scanning** [MRH02, FH12].

1. **Chamfer Matching (Two-Pass)**: A simple algorithm that passes a small mask over the image twice (top-to-bottom and bottom-to-top). Each pixel is updated using the values of its neighbors plus the mask weight.
2. **Meijster’s Algorithm (Separable)**: A more accurate $O(N)$ algorithm that decomposes the 2D problem into 1D problems. First, it computes the distance transform for each row. Then, it uses the row results to compute the final 2D distances using a parabolic lower envelope approach.

3. **Fast Marching Method** Algorithm 67: Can also be used to build a distance map by treating the boundary as a wave front propagating at unit speed.

The key insight behind Meijster’s and Felzenszwalb’s linear-time algorithms is the observation that the squared Euclidean distance transform can be decomposed as:

$$D^2(p) = \min_{q \in \text{foreground}} [(p_x - q_x)^2 + (p_y - q_y)^2] = \min_{q_y} \left[(p_y - q_y)^2 + \min_{q_x} (p_x - q_x)^2 \right]$$

The inner minimization over q_x can be solved independently for each row in $O(N)$ time using a left-to-right and right-to-left sweep. The outer minimization reduces to computing the lower envelope of a family of parabolas, which can also be done in linear time [FH12].

Theorem 24.11 (Linear-Time Distance Transform). *The Euclidean distance transform of an N -pixel binary image can be computed in $O(N)$ time and $O(N)$ space using the separable algorithm of Meijster [MRH02] or Felzenszwalb and Huttenlocher [FH12].*

Proof. The row-wise pass performs two linear scans (forward and backward) over each of the H rows, each scan touching W pixels, for $O(WH) = O(N)$ total. The column-wise parabolic envelope computation processes each of the W columns independently; in each column, the lower envelope of H parabolas is computed by the linear-time algorithm of Felzenszwalb and Huttenlocher, yielding $O(WH) = O(N)$ total. The overall complexity is therefore $O(N)$ in both time and space. $\square$

24.13.4 Algorithm: Meijster’s Algorithm (1D Pass)

Algorithm 68 Meijster’s Distance Transform

```

1: function DISTANCEMAP2D(binary_grid)
2:   width, height  $\leftarrow$  grid.size
3:   for  $y \leftarrow 0$  to  $height - 1$  do
4:     for  $x \leftarrow 0$  to  $width - 1$  do
5:       if grid[ $x$ ][ $y$ ] = FOREGROUND then
6:         G[ $x$ ][ $y$ ]  $\leftarrow$  0
7:       else
8:         G[ $x$ ][ $y$ ]  $\leftarrow$   $\infty$ 
9:       end if
10:    end for
11:    for  $x \leftarrow 1$  to  $width - 1$  do
12:      G[ $x$ ][ $y$ ]  $\leftarrow$  min(G[ $x$ ][ $y$ ], G[ $x - 1$ ][ $y$ ] + 1)
13:    end for
14:    for  $x \leftarrow width - 2$  downto 0 do
15:      G[ $x$ ][ $y$ ]  $\leftarrow$  min(G[ $x$ ][ $y$ ], G[ $x + 1$ ][ $y$ ] + 1)
16:    end for
17:  end for
18:  final_map  $\leftarrow$  array[width][height]
19:  for  $x \leftarrow 0$  to  $width - 1$  do
20:    final_map[ $x$ ]  $\leftarrow$  ComputeParabolicEnvelope(G[ $x$ ])
21:  end for
22:  return  $\sqrt{\text{final\_map}}$ 
23: end function

```

Example 24.12 (Distance Transform of a 1D Signal). Consider the binary signal $[0, 0, 1, 0, 0, 0, 1, 0]$ where 1 denotes foreground. The row-wise forward/backward sweep gives $G = [0, 0, 0, 1, 2, 3, 0, 1]$. The parabolic envelope then yields the squared distances: $[0, 0, 0, 1, 1, 1, 0, 1]$, and the final Euclidean distance map is $[0, 0, 0, 1, 1, 1, 0, 1]$. Note that pixels equidistant from two foreground pixels are correctly assigned.

24.13.5 Parameter Selections

- **Metric:** Choose between Euclidean (accurate), Manhattan (fast), or Chebyshev (fast) based on the application’s needs.
- **Sign:** Decide if an SDF Theorem 24.8 is needed (requires identifying “inside” vs. “outside”).

24.13.6 Complexity

- **Time Complexity:** $O(N)$, where N is the total number of pixels/grid points, using Meijster’s Algorithm 68 or Felzenszwalb’s algorithm Theorem 24.11.
- **Space Complexity:** $O(N)$ to store the distance values.

24.13.7 Usages

- **Robotics:** Path planning using the “Artificial Potential Field” method (moving away from high-distance regions).
- **Image Processing:** Skeletonization (medial axis extraction) and morphological thinning.
- **Computer Graphics:** Rendering anti-aliased text and icons using Signed Distance Fields (SDF fonts).
- **Computer Vision:** Template matching and object recognition (Chamfer matching).
- **Medical Imaging:** Calculating the thickness of tissues or the distance between organs.

24.13.8 Testing Methods and Edge Cases

- **Single Point:** The distance map should look like a perfect radial gradient (concentric circles).
- **Empty Image:** All distances should be infinity (or a defined maximum).
- **Full Image:** All distances should be zero.
- **Thin Lines:** Verify that the algorithm correctly identifies the closest point even for 1-pixel-wide features.
- **Concave Shapes:** Verify that the “shadowing” effect of concavities is handled correctly.

24.13.9 References

- Rosenfeld, A., & Pfaltz, J. L. (1966). “Sequential operations in digital picture processing.” *Journal of the ACM* [RP66].
- Meijster, A., Roerdink, J. B., & Hesselink, W. H. (2002). “A general algorithm for computing distance transforms in linear time.” *Mathematical Morphology and its Applications to Image and Signal Processing* [MRH02].

- Felzenszwalb, P.F., & Huttenlocher, D.P. (2012). “Distance Transforms of Sampled Functions.” *Theory of Computing* [FH12].
- Wikipedia: Distance transform.

24.14 Dijkstra’s Algorithm

24.14.1 Overview

Dijkstra’s algorithm is a fundamental graph-search algorithm that finds the shortest paths between nodes in a graph. For a given source node, the algorithm finds the shortest distance to every other node. It is the discrete equivalent of the Fast Marching Method [Section 24.12](#) and serves as the backbone of modern navigation and network routing systems [\[Dij59\]](#).

24.14.2 Definitions

Definition 24.13 (Graph). A weighted graph $G = (V, E)$ consists of a finite set of vertices V and a set of edges $E \subseteq V \times V$, each associated with a non-negative weight $w : E \rightarrow \mathbb{R}_{\geq 0}$ representing distance, cost, or time.

Definition 24.14 (Shortest Path). The shortest path from a source s to a vertex v in a weighted graph G is a path $\pi = (s = v_0, v_1, \dots, v_k = v)$ minimizing $w(\pi) = \sum_{i=0}^{k-1} w(v_i, v_{i+1})$.

Definition 24.15 (Relaxation). Relaxation is the process of testing whether a known path to a vertex v through an intermediate vertex u is shorter than the currently known best path to v . If $\text{dist}[u] + w(u, v) < \text{dist}[v]$, then $\text{dist}[v]$ is updated.

24.14.3 Theory

The algorithm operates by maintaining a set of “visited” nodes and a set of “unvisited” nodes. Initially, the distance to the source is 0, and the distance to all other nodes is infinity. In each step, the algorithm:

1. Selects the unvisited node with the smallest known distance.
2. “Relaxes” [Theorem 24.15](#) all of its neighbors: for each neighbor, it checks if going through the current node provides a shorter path than the previously known best path.
3. Marks the current node as visited.

This greedy strategy works because the weights are non-negative; once a node is visited, its distance is guaranteed to be the shortest possible. Formally, this relies on the following principle.

Theorem 24.16 (Dijkstra Correctness). *When Dijkstra’s algorithm [Algorithm 69](#) terminates, for every vertex v , $\text{distances}[v]$ equals the length of the shortest path from source to v , provided all edge weights are non-negative.*

Proof. We prove by induction on the number of extractions from the priority queue. Let S denote the set of vertices that have been extracted (visited). **Base case:** $S = \{\text{source}\}$ and $\text{distances}[\text{source}] = 0$, which is correct. **Inductive step:** Assume all vertices in S have correct shortest-path distances. Let u be the next extracted vertex with $\text{distances}[u] = d$. We claim d is optimal. Any path from *source* to u not using vertices in S must cross the $S/\bar{S}$ boundary at some edge (s', t) with $s' \in S, t \notin S$. By non-negativity, $\text{distances}[t] \leq \text{distances}[s'] + w(s', t) \leq d$ since the priority queue extracted u with minimum key. Hence no alternative path can be shorter, and $\text{distances}[u] = d$ is optimal. After relaxing u ’s neighbors, all vertices in $S \cup \{u\}$ maintain correct distances, completing the induction. $\square$

Remark 24.17. Dijkstra's algorithm is a special case of the Bellman–Ford algorithm restricted to non-negative weights. It can also be derived from the label-correcting framework of [CLRS09].

24.14.4 Algorithm

Algorithm 69 Dijkstra's Shortest Path Algorithm

```

1: function DIJKSTRA(graph, source)
2:   distances  $\leftarrow \{v : \infty \mid v \in \text{graph.vertices}\}$ 
3:   previous  $\leftarrow \{v : \text{None} \mid v \in \text{graph.vertices}\}$ 
4:   distances[source]  $\leftarrow 0$ 
5:   pq  $\leftarrow$  PriorityQueue()
6:   pq.push(source, 0)
7:   while not pq.empty() do
8:     u, dist_u  $\leftarrow$  pq.pop_min()
9:     if dist_u > distances[u] then continue
10:    end if
11:    for (v, weight)  $\in$  graph.neighbors(u) do
12:      new_dist  $\leftarrow$  distances[u] + weight
13:      if new_dist < distances[v] then
14:        distances[v]  $\leftarrow$  new_dist
15:        previous[v]  $\leftarrow$  u
16:        pq.push(v, new_dist)
17:      end if
18:    end for
19:  end while
20:  return distances, previous
21: end function

```

Example 24.18 (Dijkstra on a Small Graph). Consider the graph with vertices $\{A, B, C, D\}$ and edges: $A \rightarrow B$ (weight 1), $A \rightarrow C$ (weight 4), $B \rightarrow C$ (weight 2), $B \rightarrow D$ (weight 6), $C \rightarrow D$ (weight 3), with source A .

1. Initialize: $\text{dist} = \{A : 0, B : \infty, C : \infty, D : \infty\}$.
2. Extract A (dist 0). Relax neighbors: $\text{dist}[B] = 1$, $\text{dist}[C] = 4$.
3. Extract B (dist 1). Relax: $\text{dist}[C] = \min(4, 1 + 2) = 3$, $\text{dist}[D] = 1 + 6 = 7$.
4. Extract C (dist 3). Relax: $\text{dist}[D] = \min(7, 3 + 3) = 6$.
5. Extract D (dist 6). No unvisited neighbors.

Final distances: $\{A : 0, B : 1, C : 3, D : 6\}$. The shortest path $A \rightarrow B \rightarrow C \rightarrow D$ has total weight $1 + 2 + 3 = 6$.

24.14.5 Parameter Selections

- **Data Structure:** Using a **Fibonacci Heap** or **Binary Heap** for the priority queue is critical for performance. Binary heaps are most common in practice.
- **Edge Weights:** If any edge weight is negative, the algorithm fails; the Bellman–Ford algorithm should be used instead.

24.14.6 Complexity

Theorem 24.19 (Dijkstra Complexity). *With a binary heap, Dijkstra’s algorithm Algorithm 69 runs in $O((E + V) \log V)$ time. With a Fibonacci heap, it runs in $O(E + V \log V)$ time. In both cases, space is $O(V + E)$.*

Proof. Each vertex is extracted from the priority queue at most once, contributing $O(\log V)$ per extraction with a binary heap. Each edge is relaxed at most once (when its tail is extracted), and each relaxation may trigger a decrease-key or insert operation costing $O(\log V)$. Total: $O(V \log V + E \log V) = O((V + E) \log V)$. With a Fibonacci heap, the decrease-key operation is $O(1)$ amortized, giving $O(V \log V + E) = O(E + V \log V)$. $\square$

24.14.7 Usages

- **GPS Navigation:** Finding the fastest route between two locations.
- **Network Routing:** Protocols like OSPF (Open Shortest Path First) use Dijkstra to route data packets across the internet.
- **Social Networks:** Finding “degrees of separation” between people.
- **Robotics:** Pathfinding for autonomous vehicles on a roadmap.
- **Game Development:** AI movement (often using A*, which is a heuristic-based extension of Dijkstra).

24.14.8 Testing Methods and Edge Cases

- **Disconnected Graphs:** Nodes that cannot be reached from the source should maintain a distance of infinity.
- **Single-Node Graph:** Distance to itself is 0.
- **Parallel Edges:** The algorithm should always select the edge with the minimum weight between two nodes.
- **Cycles:** Dijkstra correctly handles cycles as long as all weights are non-negative.
- **Path Reconstruction:** Ensure the `previous` pointers correctly trace the full path back to the source.

24.14.9 References

- Dijkstra, E. W. (1959). “A note on two problems in connexion with graphs.” *Numerische Mathematik* [Dij59].
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). “Introduction to Algorithms.” MIT Press [CLRS09].
- Wikipedia: Dijkstra’s algorithm.

24.15 K-Geodesics

24.15.1 Overview

K-Geodesics is an adaptation of the K-Means clustering algorithm to the surface of a 3D mesh. Instead of using Euclidean distance to cluster points in 3D space, K-Geodesics uses **geodesic distance** (the shortest path along the surface). This allows for the segmentation of a mesh into regions that are intrinsically meaningful, such as the limbs of a character or the functional parts of a machine [Llo82].

24.15.2 Definitions

Definition 24.20 (Geodesic Centroid). The geodesic centroid (or Fréchet mean) of a set of vertices on a mesh is the vertex that minimizes the sum of squared geodesic distances to all other vertices in the set: $c^* = \arg \min_{c \in V} \sum_{v \in S} d_g(c, v)^2$, where d_g denotes geodesic distance.

Definition 24.21 (Geodesic Voronoi Region). The geodesic Voronoi region of center c_i is the set $V_i = \{v \in V \mid d_g(v, c_i) \leq d_g(v, c_j) \text{ for all } j \neq i\}$, partitioning the mesh vertices according to their nearest center under the geodesic metric.

24.15.3 Theory

The K-Geodesics algorithm follows the standard iterative process of Lloyd’s algorithm [Llo82]:

1. **Initialization:** Pick k initial seed vertices as centers.
2. **Assignment:** Assign every vertex on the mesh to the cluster of its nearest center using a multi-source **Fast Marching Method** Algorithm 67 or the **Heat Method**.
3. **Update:** For each cluster, find a new center. This is the “Fréchet mean” on the surface Theorem 24.20. Since finding the exact mean is expensive, it is often approximated by picking the vertex in the cluster that minimizes the average distance to all other cluster members.
4. **Repeat:** Continue until the centers no longer change or a maximum number of iterations is reached.

Theorem 24.22 (K-Geodesics Monotone Energy Decrease). *At each iteration of the K-Geodesics algorithm Algorithm 70, the total geodesic energy $E = \sum_{i=1}^k \sum_{v \in V_i} d_g(v, c_i)^2$ does not increase.*

Proof. In the assignment step, each vertex is reassigned to its nearest center, which can only decrease or maintain its contribution to the energy. In the update step, the centroid of each cluster (by definition Theorem 24.20) minimizes the sum of squared distances within that cluster. Hence both steps are non-increasing in energy, and their composition is non-increasing. Since the energy is bounded below by zero, the algorithm converges in a finite number of iterations for discrete meshes. $\square$

24.15.4 Algorithm

Example 24.23 (K-Geodesics on a Cylinder). Consider a cylinder mesh with $k = 2$ clusters and seed centers placed at the top and bottom rims. The assignment step computes geodesic distances from both centers. The Voronoi partition Theorem 24.21 divides the cylinder into a top half and a bottom half along the geodesic midline. After centroid updates, the two centers migrate toward the centroids of their respective halves. Euclidean K-Means would also produce this partition for a cylinder, but for a bent or folded shape, K-Geodesics would correctly separate geometrically close but geodesically distant regions (e.g., the two ends of a U-shaped mesh).

Algorithm 70 K-Geodesics Clustering

```

1: function KGEODESICS(mesh, k)
2:   centers  $\leftarrow$  random.sample(mesh.vertices, k)
3:   while not converged do
4:     (distances, labels)  $\leftarrow$  MultiSourceGeodesic(mesh, centers)
5:     new_centers  $\leftarrow$  []
6:     for i  $\leftarrow$  0 to k - 1 do
7:       cluster_v  $\leftarrow$  {v | labels[v] = i}
8:       best_v  $\leftarrow$  FindLocalCentroid(mesh, cluster_v)
9:       new_centers.append(best_v)
10:    end for
11:    if new_centers = centers then break
12:    end if
13:    centers  $\leftarrow$  new_centers
14:  end while
15:  return labels, centers
16: end function

```

24.15.5 Parameter Selections

- **Number of Clusters (k):** Depends on the desired level of segmentation.
- **Distance Solver:** The **Heat Method** [CWW13b] is recommended for the assignment step because it is extremely fast after an initial factorization.
- **Initialization:** Using **K-Means++** logic (picking seeds that are far from each other) significantly improves convergence speed and result quality.

24.15.6 Complexity

- **Assignment:** $O(N)$ using the Heat Method or $O(N \log N)$ using Fast Marching.
- **Update:** $O(k \cdot N_{cluster})$ to find new centers.
- **Total:** $O(I \cdot N)$ where I is the number of iterations.

24.15.7 Usages

- **Mesh Segmentation:** Dividing a model into organic parts (e.g., body, head, arms).
- **Texture Atlas Generation:** Partitioning a complex mesh into k nearly-flat charts for UV unwrapping.
- **Shape Analysis:** Identifying repeating structural motifs on a surface.
- **Remote Sensing:** Clustering geographic regions based on travel time over terrain.
- **Remeshing:** Using the centers as a basis for generating a coarse, high-quality version of the mesh.

24.15.8 Testing Methods and Edge Cases

- **Convex Shapes:** For a sphere, the regions should look like a spherical Voronoi diagram.
- **Long Thin Regions:** Verify that the algorithm correctly segments limbs (which Euclidean K-Means might merge if they are physically close but geodesically far).

- **Number of Iterations:** Monitor the total “energy” (sum of distances) to ensure it decreases monotonically Theorem 24.22.
- **Empty Clusters:** Handle cases where a center becomes so isolated that no vertices are assigned to it.
- **Disconnected Components:** The algorithm should be applied to each component separately or handle infinity distances correctly.

24.15.9 References

- Peyré, G., & Cohen, L.D. (2006). “Geodesic remeshing using front propagation.” *International Journal of Computer Vision* [PC06].
- Crane, K., Weischedel, C., & Wardetzky, M. (2013). “Geodesics in Heat: A New Approach to Computing Distance Based on Heat Flow.” *ACM TOG* [CWW13b].
- Lloyd, S. (1982). “Least squares quantization in PCM.” *IEEE Transactions on Information Theory* [Llo82].
- Wikipedia: K-means clustering.

24.16 Fréchet Distance

24.16.1 Overview

The Fréchet distance is a measure of similarity between two curves (trajectories) that takes into account the location and ordering of points along the curves. It is often described using the **dog-leash analogy**: imagine a person walking along one curve and a dog walking along the other. The Fréchet distance is the minimum length of a leash needed to connect the two as they traverse their respective paths from start to finish. It is a more robust measure of curve similarity than the Hausdorff distance because it respects the parameterization of the curves [Fré06].

24.16.2 Definitions

Definition 24.24 (Discrete Fréchet Distance). The discrete Fréchet distance between two polygonal chains $P = (p_1, \dots, p_n)$ and $Q = (q_1, \dots, q_m)$ is

$$d_F(P, Q) = \min_{\alpha, \beta} \max_{t \in [0, 1]} \|P(\alpha(t)) - Q(\beta(t))\|$$

where $\alpha : [0, 1] \rightarrow [0, n-1]$ and $\beta : [0, 1] \rightarrow [0, m-1]$ are non-decreasing surjections (reparameterizations) [Fré06, AG95].

Definition 24.25 (Free Space Diagram). The free space diagram for two curves P and Q with threshold ϵ is the set $\mathcal{F}_\epsilon = \{(i, j) \mid \|p_i - q_j\| \leq \epsilon\} \subseteq [0, n] \times [0, m]$. The Fréchet distance is at most ϵ if and only if there exists a monotone path from $(0, 0)$ to (n, m) entirely within $\mathcal{F}_\epsilon$.

Definition 24.26 (Hausdorff Distance). The directed Hausdorff distance from P to Q is $h(P, Q) = \max_{p \in P} \min_{q \in Q} \|p - q\|$. The symmetric Hausdorff distance is $d_H(P, Q) = \max(h(P, Q), h(Q, P))$. Unlike the Fréchet distance, it does not account for the ordering of points along the curves.

24.16.3 Theory

For two curves P and Q of lengths n and m , the discrete Fréchet distance $d_F(P, Q)$ is the minimum cost of a “coupling” between the two sequences. This is typically solved using **dynamic programming**.

1. Let $dp[i][j]$ be the Fréchet distance between the prefix $P[0 \dots i]$ and $Q[0 \dots j]$.
2. The base case is $dp[0][0] = \text{dist}(P[0], Q[0])$.
3. The recurrence relation is: $dp[i][j] = \max(\text{dist}(P[i], Q[j]), \min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1]))$.
4. The final result is $dp[n-1][m-1]$.

For continuous curves, the problem is solved by searching for a “monotone path” from $(0, 0)$ to $(1, 1)$ in the **Free Space Diagram** Theorem 24.25.

Theorem 24.27 (Fréchet DP Correctness). *The dynamic programming recurrence for the discrete Fréchet distance Theorem 24.24 correctly computes $d_F(P, Q)$ in $O(nm)$ time.*

Proof. The proof is by strong induction on $i + j$. The recurrence considers all valid reparametrizations: at step (i, j) , the coupling can advance on P only ($dp[i-1][j]$), on Q only ($dp[i][j-1]$), or on both ($dp[i-1][j-1]$). Taking the minimum of these three predecessors ensures we find the coupling that minimizes the maximum leash length. The outer max ensures the leash is long enough to accommodate the current pair. The base case $dp[0][0] = \|P[0] - Q[0]\|$ is correct by definition. By induction, $dp[i][j]$ equals the optimal Fréchet distance for prefixes $P[0..i]$ and $Q[0..j]$. $\square$

24.16.4 Algorithm

Algorithm 71 Discrete Fréchet Distance

```

1: function DISCRETEFRECHET( $P, Q$ )
2:    $n \leftarrow |P|, m \leftarrow |Q|$ 
3:    $dp \leftarrow \text{array}[n][m]$ , initialized to  $-1$ 
4:    $dp[0][0] \leftarrow \text{Distance}(P[0], Q[0])$ 
5:   for  $i \leftarrow 1$  to  $n - 1$  do
6:      $dp[i][0] \leftarrow \max(dp[i-1][0], \text{Distance}(P[i], Q[0]))$ 
7:   end for
8:   for  $j \leftarrow 1$  to  $m - 1$  do
9:      $dp[0][j] \leftarrow \max(dp[0][j-1], \text{Distance}(P[0], Q[j]))$ 
10:  end for
11:  for  $i \leftarrow 1$  to  $n - 1$  do
12:    for  $j \leftarrow 1$  to  $m - 1$  do
13:       $cost \leftarrow \text{Distance}(P[i], Q[j])$ 
14:       $dp[i][j] \leftarrow \max(cost, \min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1]))$ 
15:    end for
16:  end for
17:  return  $dp[n-1][m-1]$ 
18: end function

```

Example 24.28 (Fréchet Distance of Two Triangular Paths). Consider $P = \{(0, 0), (1, 1), (2, 0)\}$ and $Q = \{(0, 0), (1, 2), (2, 0)\}$. The midpoint of P deviates by 1 unit, while Q 's midpoint deviates by 2 units. The discrete Fréchet distance is computed by the DP table:

- $dp[0][0] = 0$.
- $dp[1][0] = \|P_1 - Q_0\| = \sqrt{2}$; $dp[0][1] = \|P_0 - Q_1\| = \sqrt{5}$.
- $dp[1][1] = \max(\|P_1 - Q_1\|, \min(dp[0][1], dp[0][0], dp[1][0])) = \max(1, 0) = 1$.
- $dp[2][1] = \max(\|P_2 - Q_1\| = \sqrt{5}, \min(dp[1][1] = 1, dp[1][0] = \sqrt{2}, dp[2][0] = 2)) = \max(\sqrt{5}, 1) = \sqrt{5}$.
- $dp[2][2] = \max(\|P_2 - Q_2\| = 0, \min(dp[1][2], dp[1][1], dp[2][1])) = 0$.

Wait, let us recompute more carefully. Actually $dp[2][2] = \max(0, \min(dp[1][2], dp[1][1], dp[2][1]))$. We need $dp[1][2] = \max(\|P_1 - Q_2\| = \sqrt{2}, \min(dp[0][2], dp[0][1], dp[1][1]))$. Since $dp[1][1] = 1$, we get $dp[1][2] = \max(\sqrt{2}, 1) = \sqrt{2}$. Thus $dp[2][2] = \max(0, \min(\sqrt{2}, 1, \sqrt{5})) = 1$. The Fréchet distance is $\boxed{1}$, which is the leash length needed to connect the walkers at the midpoints of the two curves.

24.16.5 Parameter Selections

- **Distance Metric:** Euclidean distance is standard, but Manhattan or Chebyshev distances can also be used.
- **Normalization:** Curves should be normalized for scale or rotation if required by the application.

24.16.6 Complexity

- **Time Complexity:** $O(n \cdot m)$ for curves with n and m vertices, as each entry of the DP table Theorem 24.27 is computed in $O(1)$.
- **Space Complexity:** $O(n \cdot m)$ to store the DP table. Can be reduced to $O(\min(n, m))$ using a rolling array.

24.16.7 Usages

- **Trajectory Analysis:** Comparing the paths taken by animals, vehicles, or sports players.
- **Handwriting Recognition:** Comparing a drawn stroke to a template of letters.
- **Speech Recognition:** Time-aligning two audio signals (Dynamic Time Warping is a similar concept).
- **Bioinformatics:** Comparing the backbones of proteins or the shapes of molecules.
- **Computer Vision:** Shape matching for object tracking.

24.16.8 Testing Methods and Edge Cases

- **Identical Curves:** The distance should be 0.
- **Reversed Curves:** If one curve is the reverse of the other, the distance should be large (unlike Hausdorff distance).
- **Different Sampling Rates:** Test curves where one has many more vertices than the other but covers the same path.
- **Large Deviations:** A single outlier point in one curve should correctly increase the Fréchet distance.
- **Looping Curves:** Verify that the algorithm handles self-intersecting paths correctly.

24.16.9 References

- Fréchet, M. (1906). “Sur quelques points du calcul fonctionnel.” *Rendiconti del Circolo Matematico di Palermo* [Fré06].
- Alt, H., & Godau, M. (1995). “Computing the Fréchet distance between two polygonal curves.” *International Journal of Computational Geometry & Applications* [AG95].
- Eiter, T., & Mannila, H. (1994). “Computing discrete Fréchet distance.” Technical Report [EM94b].
- Wikipedia: Fréchet distance.

Chapter 25

Art Gallery Problem

The art gallery problem is a visibility question that has become one of the most studied problems in computational geometry: given the floor plan of an art gallery, modeled as a simple polygon, how many stationary guards are needed so that every point of the gallery is visible from at least one guard? Chvátal’s theorem gives a tight bound of $\lfloor n/3 \rfloor$ guards for an n -vertex polygon, and the structural connection between guarding, polygon triangulation, vertex coloring, and polygon kernels makes the problem a rich source of theory and algorithms. This chapter develops the classical results, the guard-placement algorithm, and the closely related notion of a polygon kernel.

25.1 Art Gallery Guard Placement

25.1.1 1. Overview

The **Art Gallery Problem** (AGP) is a classic problem in computational geometry that asks for the minimum number of guards required to monitor every point within a given art gallery (represented as a simple polygon) [O’R87]. A guard at a point P can “see” any point Q if the line segment PQ lies entirely within the polygon.

25.1.2 2. Definitions

Definition 25.1 (Art Gallery). An *art gallery* is a simple polygon P with n vertices.

Definition 25.2 (Visibility). A point Q is *visible* from a point P inside a polygon P if the line segment $\overline{PQ}$ lies entirely within the polygon: $\overline{PQ} \subseteq P$.

Definition 25.3 (Guard Set). A *guard set* is a set of points $\{G_1, G_2, \dots, G_k\}$ such that every point in P is visible from at least one G_i . The *guard number* $\gamma(P)$ is the minimum cardinality of a guard set.

Theorem 25.4 (Chvátal’s Art Gallery Theorem). For a simple polygon with n vertices, $\lfloor n/3 \rfloor$ guards are always sufficient and sometimes necessary to guard the polygon [Chw75].

Proof. The proof proceeds by **polygon triangulation and 3-coloring** [Fis78].

1. **Triangulate** the polygon into $n - 2$ triangles using an algorithm like ear clipping or monotone decomposition.
2. **3-color** the vertices of the resulting triangulation graph such that no two adjacent vertices share the same color. Since the dual graph of a triangulated simple polygon is a tree, the triangulation graph is chordal and therefore 3-colorable.

3. **Count** the number of vertices for each of the three colors. Since there are n vertices total, by the pigeonhole principle, the color with the minimum count has at most $\lfloor n/3 \rfloor$ vertices.
4. **Place** a guard at every vertex of this minimum-count color. Since every triangle in the triangulation must contain all three colors, every triangle is visible from at least one guard, and hence the entire polygon is guarded.

The bound is tight: Chvátal constructed “comb” polygons with $3k$ vertices that require exactly $k = \lfloor n/3 \rfloor$ guards [Chv75]. $\square$

Remark 25.5. For **orthogonal polygons** (all edges horizontal or vertical), the bound improves to $\lfloor n/4 \rfloor$ guards, proved by O’Rourke [O’R87].

Example 25.6 (Art Gallery with 6 Vertices). Consider the simple polygon with vertices $(0, 0), (4, 0), (4, 3), (2, 1), (0, 3)$ (5 vertices). After triangulation we have 3 triangles. A 3-coloring of the vertices yields color classes of sizes at most $\lfloor 5/3 \rfloor = 1$. Placing a single guard at a vertex of the minority color (e.g., at $(2, 1)$) guards the entire polygon.

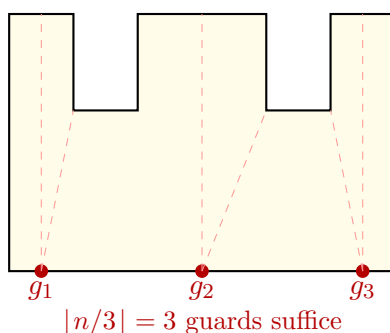


Figure 25.1: Art gallery problem: a comb polygon with $n = 12$ vertices requires $\lfloor 12/3 \rfloor = 4$ guards in the worst case.

25.1.3 4. Pseudo code

```

1: function PLACEGUARDS(polygon)                                ▷ 1. Triangulate the polygon
2:   triangles  $\leftarrow$  Triangulate(polygon)
                                                                    ▷ 2. Build the triangulation graph
3:   graph  $\leftarrow$  BuildTriangulationGraph(triangles)
                                                                    ▷ 3. 3-Color the graph (using DFS since it's a chordal graph)
4:   colors  $\leftarrow$  {v: None for v in polygon.vertices}
5:   DFS_Coloring(graph, start_vertex, colors)
                                                                    ▷ 4. Partition vertices by color
6:   color_groups  $\leftarrow$  [], [], []
7:   for v, c in colors.items() do
8:     color_groups[c].append(v)
9:   end for
                                                                    ▷ 5. Select the smallest group
10:  guard_positions  $\leftarrow$  min(color_groups, key=len)
11:  return guard_positions
12: end function

```

25.1.4 5. Parameters Selections

Definition 25.7 (Vertex vs. Point Guarding). *Vertex guarding* restricts guards to polygon vertices. *Point guarding* allows guards anywhere in the interior. Point guarding might reduce the total count in some cases, but the Chvátal bound $\lfloor n/3 \rfloor$ remains the same for worst-case bounds.

Definition 25.8 (Triangulation Algorithm Choice). The choice of triangulation (ear clipping vs. monotone decomposition) does not affect the 3-coloring result but does affect the $O(n \log n)$ vs. $O(n)$ construction time.

25.1.5 6. Complexity

Theorem 25.9 (Art Gallery Algorithm Complexity). *The guard placement algorithm runs in $O(n \log n)$ time using monotone decomposition for triangulation, followed by $O(n)$ for graph construction and 3-coloring. The space is $O(n)$.*

25.1.6 7. Usages

- **Physical Security:** Optimizing the placement of cameras or motion sensors in buildings.
- **Wireless Networking:** Determining the minimum number of Wi-Fi access points or cellular towers to cover a given region.
- **Robotics:** Path planning for a surveillance robot that needs to maintain visibility of its surroundings.
- **Environmental Monitoring:** Placing sensors to detect smoke or pollutants in complex indoor spaces.

25.1.7 8. Testing methods and Edge cases

- **Convex Polygons:** $n = 3, 4, 5$ all require only 1 guard ($\lfloor 5/3 \rfloor = 1$).
- **Comb Polygons:** “Comb” shapes (ortho-polygons) reach the upper bound of $\lfloor n/3 \rfloor$.
- **Star Polygons:** By definition, star polygons only require 1 guard (placed at the kernel).
- **Orthogonal Polygons:** For polygons where all edges are horizontal or vertical, the bound is tighter: $\lfloor n/4 \rfloor$ guards are sufficient (Hoffman’s Theorem).
- **Holes:** Polygons with holes require significantly more guards, and the problem becomes NP-hard.

25.1.8 9. References

- Chvátal, V. (1975). “A combinatorial theorem in plane geometry”. *Journal of Combinatorial Theory, Series B*.
- Fisk, S. (1978). “A short proof of Chvátal’s watchman theorem”. *Journal of Combinatorial Theory, Series B*.
- O’Rourke, J. (1987). “Art Gallery Theorems and Algorithms”. Oxford University Press.
- [Wikipedia: Art gallery problem](#)

25.2 Polygon Kernel and Star-Shaped Polygons

25.2.1 1. Overview

The **kernel** of a simple polygon is the set of points from which every point of the polygon is visible. A polygon whose kernel is non-empty is called **star-shaped**. Computing the kernel answers visibility questions such as “is there a point that sees the whole polygon?”, which arises in art gallery problems, motion planning for a point robot inside a polygon, and star-shaped polygonization [Las96].

25.2.2 2. Definitions

Definition 25.10 (Kernel of a Polygon). The *kernel* of a simple polygon P is

$$\ker(P) = \{p \in P \mid \text{the segment } \overline{pq} \subset P \text{ for all } q \in P\}.$$

Definition 25.11 (Star-Shaped Polygon). A polygon is *star-shaped* if it has a non-empty kernel: there exists a point p such that every point of the polygon is visible from p . The kernel need not be a single point; it is itself a (possibly degenerate) convex polygon.

25.2.3 3. Theory

Let P be given in counter-clockwise order. The kernel is the intersection of the closed left half-planes defined by the directed edges of P :

$$\ker(P) = \bigcap_i H(e_i), \quad H(e_i) = \{x \mid \text{orient2d}(v_i, v_{i+1}, x) \geq 0\}.$$

For a simple polygon this half-plane intersection is always convex and non-empty exactly when P is star-shaped. In general the kernel is computed by half-plane intersection in $O(n \log n)$ time; for the special case of polygon kernels there is a linear-time $O(n)$ algorithm using a deque, due to Lee and Preparata [LP79]. A point $p \in \ker(P)$ need only be checked against the vertices: p sees the entire polygon if and only if $\overline{pv_i} \subset P$ for every vertex v_i .

25.2.4 4. Pseudo code

```

function KERNEL( $P$ )
   $H \leftarrow \emptyset$ 
  for each directed edge  $(v_i, v_{i+1})$  of  $P$  do
    add the left half-plane of  $(v_i, v_{i+1})$  to  $H$ 
  end for
  return HalfPlaneIntersection( $H$ )
end function

```

25.2.5 5. Parameters Selections

- **Vertex order:** assume a consistent counter-clockwise ordering; reverse the half-plane side if the polygon is clockwise.
- **Exact predicates:** use robust orientation tests, since near-degenerate edges decide membership of kernel boundary points.
- **Collinear edges:** consecutive collinear edges define the same half-plane and may be merged.

25.2.6 6. Complexity

The general half-plane intersection approach runs in $O(n \log n)$ time and $O(n)$ space. The specialized deque algorithm of Lee and Preparata computes the kernel in optimal $O(n)$ time [LP79].

25.2.7 7. Usages

- **Art gallery:** the kernel identifies points from which a single guard observes the entire polygon.
- **Motion planning:** a point robot can navigate between any two points of a star-shaped region without leaving it.
- **Star-shaped polygonization:** connecting points to a common kernel point produces a star-shaped polygonization of a point set [Las96].

25.2.8 8. Testing methods and Edge cases

- **Convex polygon:** the kernel equals the polygon itself.
- **L-shaped polygon:** the kernel is the region common to both arms.
- **Spiral polygon:** the kernel is empty.
- **Clockwise input:** verify the half-plane orientation is flipped correctly.

25.2.9 9. References

- Laszlo, M. J. (1996). “Computational Geometry and Computer Graphics in C++”. Prentice Hall.
- Lee, D. T., & Preparata, F. P. (1979). “An optimal algorithm for finding the kernel of a polygon”. Journal of the ACM.
- Wikipedia: [Star-shaped polygon](#)

Chapter 26

Motion Planning

Motion planning asks whether an object can move from a start state to a goal state without colliding with obstacles. The central idea is to replace motion of a physical object by a path for a point in a higher-dimensional *configuration space*.

26.1 Configuration Space

A configuration records all degrees of freedom of a moving object. A point robot translating in the plane has configuration $q = (x, y) \in \mathbb{R}^2$. A rigid body translating and rotating in the plane has

$$q = (x, y, \theta) \in \mathbb{R}^2 \times S^1,$$

while a rigid body in three dimensions has six degrees of freedom, $q = (x, y, z, \alpha, \beta, \gamma) \in \mathbb{R}^3 \times SO(3)$.

The configuration space $\mathcal{C}$ contains all possible configurations. The obstacle region $\mathcal{C}_{\text{obs}}$ consists of configurations in which the robot intersects an obstacle. The free configuration space is

$$\mathcal{C}_{\text{free}} = \mathcal{C} \setminus \mathcal{C}_{\text{obs}}.$$

Motion planning then becomes the problem of finding a continuous path $\tau : [0, 1] \rightarrow \mathcal{C}_{\text{free}}$ with $\tau(0) = q_{\text{start}}$ and $\tau(1) = q_{\text{goal}}$.

26.2 Configuration-Space Obstacles

For a translating robot R and workspace obstacle O , collision-free translations can be computed using the Minkowski difference:

$$\mathcal{C}_{\text{obs}} = O \oplus (-R).$$

Thus a finite-sized robot can be replaced by a point, while the obstacles are expanded by the reflected robot. For a rotating robot, this construction is performed for each orientation, producing a slice of the higher-dimensional configuration-space obstacle.

Definition 26.1 (Free Path). A free path is a continuous map $\tau : [0, 1] \rightarrow \mathcal{C}_{\text{free}}$. It is *feasible* if it connects the start and goal configurations and satisfies any required constraints, such as joint limits or velocity bounds.

26.3 Exact Planning Methods

26.3.1 Cell Decomposition

Cell decomposition partitions $\mathcal{C}_{\text{free}}$ into simple cells whose interiors are either entirely free or entirely blocked. A connectivity graph is built by connecting adjacent free cells. A graph search gives a sequence of cells, after which a continuous path is constructed through them. Exact decompositions provide completeness, but their complexity grows rapidly with the dimension and algebraic complexity of the configuration space.

26.3.2 Visibility Graphs

For a polygonal point robot in the plane, a visibility graph uses the start, goal, and obstacle vertices as nodes. Two nodes are connected when the segment between them lies in free space. Dijkstra's algorithm or A* then finds a shortest collision-free path in the graph. Visibility graphs are exact for Euclidean shortest paths among polygonal obstacles.

26.4 Roadmaps and Sampling-Based Planning

Sampling-based planners avoid explicitly constructing high-dimensional obstacle boundaries. They sample configurations, reject samples in $\mathcal{C}_{\text{obs}}$, and connect nearby collision-free samples.

Algorithm 72 Probabilistic Roadmap Planner

```

1: function BUILDROADMAP( $\mathcal{C}$ ,  $N$ )
2:    $G \leftarrow$  empty graph
3:   for  $i = 1$  to  $N$  do
4:     Sample  $q$  from  $\mathcal{C}$ 
5:     if  $q \in \mathcal{C}_{\text{free}}$  then
6:       Add  $q$  to  $G$ 
7:       for nearby vertex  $p$  in  $G$  do
8:         if the local path from  $p$  to  $q$  is collision-free then
9:           Add edge  $(p, q)$  to  $G$ 
10:        end if
11:      end for
12:    end if
13:  end for
14:  return  $G$ 
15: end function

```

Probabilistic roadmaps (PRM) are effective for repeated queries in a fixed environment. Rapidly-exploring random trees (RRT) grow from the start toward unexplored regions and are useful for single-query planning and systems with complex dynamics. Both methods are probabilistically complete under standard sampling and local-planning assumptions, but they do not automatically return the shortest path.

26.5 Planning Constraints and Validation

Collision checking must test the entire local path, not only its endpoints. Practical planners also handle robot joint limits, self-collision, narrow passages, dynamic obstacles, clearance requirements, and kinodynamic limits on velocity and acceleration. A returned path should be checked independently, then shortened or smoothed only with collision tests after every modification.

26.6 Applications

Motion planning is used in robot manipulation, autonomous vehicles, computer animation, computer games, surgical navigation, warehouse automation, and molecular modeling. Configuration-space reasoning also connects polygon offsets, Minkowski operations, shortest paths, visibility, and geometric collision detection.

Part IX

Shape Analysis and Geometric Data

Chapter 27

Sampling

27.1 Poisson Disk Sampling

27.1.1 Overview

Poisson disk sampling is a technique for generating a set of points in a multidimensional space such that no two points are closer to each other than a specified minimum distance r . The resulting distribution has a “blue noise” characteristic: the points are tightly packed but avoid the structured look of a grid and the clumping found in pure random (white noise) sampling. This makes it ideal for computer graphics, procedural generation, and sensor placement [Coo86].

27.1.2 Definitions

Definition 27.1 (Minimum Distance (r)). The radius of an exclusion disk around each point. No two generated points may be closer than r to each other, ensuring a minimum spacing constraint.

Definition 27.2 (Blue Noise). A random distribution where low-frequency components are minimized in the power spectrum, resulting in a visually pleasing and uniform distribution. Formally, the Fourier transform of the point set has negligible energy below a cutoff frequency determined by the density [Coo86].

Definition 27.3 (Bridson’s Algorithm). A fast $O(n)$ algorithm for generating Poisson disk samples in any dimension, introduced by Robert Bridson [Bri07]. It uses a background grid for $O(1)$ proximity queries and an active list for incremental growth.

Definition 27.4 (Rejection Sampling). A naive approach where random points are generated and discarded if they fall within distance r of any existing point. The acceptance rate decreases rapidly as the domain fills, making it $O(n^2)$ in the worst case [LD08].

27.1.3 Theory

Bridson’s algorithm [Bri07] is the standard for efficient Poisson disk sampling:

1. **Initialize:** Place an initial point randomly in the domain and add it to an **active list**.
2. **Grid:** Initialize a background grid with cell size $r/\sqrt{d}$ (where d is dimension) to ensure each cell can contain at most one point. This guarantees that proximity checks only need to examine the 9 (in 2D) or 27 (in 3D) neighboring cells.
3. **Active Sampling:** While the active list is not empty:
 - Pick a random point P from the active list.

- Generate up to k candidates in an annulus $[r, 2r]$ around P .
- For each candidate Q , check if it is within distance r of any existing points (using the grid for $O(1)$ lookups).
- If Q is valid, add it to the grid and the active list.
- If no candidates are found after k trials, remove P from the active list.

Theorem 27.5 (Complexity of Bridson’s Algorithm). *Bridson’s algorithm generates n Poisson disk samples in expected $O(n)$ time and $O(n)$ space, assuming a fixed minimum distance r and dimension d .*

Proof. Each of the n points is processed at most once from the active list. For each active point, at most k candidates are generated, and each candidate requires $O(1)$ time for the grid-based neighborhood query (since at most 3^d cells need to be checked). When a candidate is accepted, the grid insertion is $O(1)$. A point is removed from the active list after k failed trials, so the total number of candidate tests is at most $kn = O(n)$. The grid stores at most $O(n)$ points in $O(n)$ cells. Thus the total expected time is $O(n)$ and space is $O(n)$ [Bri07]. $\square$

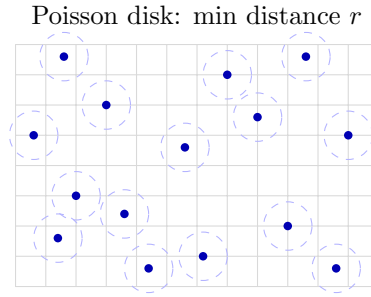


Figure 27.1: Poisson disk sampling: points are placed with minimum distance r between any pair.

Example 27.6 (Poisson Disk Sampling Trace on a 4×4 Grid). Consider a small domain $[0, 4] \times [0, 4]$ with $r = 1.5$ and $k = 8$.

1. Seed: Place $P_1 = (2.0, 2.0)$. Grid cell size = $1.5/\sqrt{2} \approx 1.06$, so the grid is 4×4 .
2. Active list: $[P_1]$. Pick P_1 , generate candidates around it. Suppose $Q_1 = (0.8, 1.2)$ passes the distance check ($|P_1 Q_1| \approx 1.44 < 1.5$ —rejected). Try again: $Q_2 = (3.3, 1.1)$, distance $\approx 1.35 < 1.5$ —rejected. After 8 failures, remove P_1 from active list.
3. Since only one point was placed and no new points accepted, the result is $\{(2.0, 2.0)\}$.

For a larger domain with more seed attempts, the algorithm fills the space more densely, with each new point guaranteed to be at least r from all others.

27.1.4 Pseudocode

27.1.5 Parameter Selection

- **Radius (r):** Determines the density of the points.
- **Trials (k):** Typically set to 30. Higher k results in tighter packing but increases computation time.
- **Variable Radius:** The algorithm can be adapted to use a varying $r(x, y)$ based on an importance map or image brightness.

Algorithm 73 Poisson Disk Sampling

```

function POISSONDISKSAMPLING(width, height, r, k=30)                                ▷ 1. Initialize Grid
    cell_size  $\leftarrow r/\sqrt{2}$ 
    cols  $\leftarrow \lceil \text{width}/\text{cell\_size} \rceil$ 
    rows  $\leftarrow \lceil \text{height}/\text{cell\_size} \rceil$ 
    grid  $\leftarrow$  array[cols][rows] initialized to None

    start_p  $\leftarrow$  Point(random(width), random(height))                                ▷ 2. Seed Point
    active_list  $\leftarrow$  [start_p]
    InsertIntoGrid(start_p, grid, cell_size)
    points  $\leftarrow$  [start_p]

    ▷ 3. Main Loop
    while active_list is not empty do
        p  $\leftarrow$  random.choice(active_list)
        found  $\leftarrow$  False
        for k in range(k) do                                ▷ Generate candidate in annulus [r, 2r]
            q  $\leftarrow$  GenerateRandomInAnnulus(p, r, 2 · r)
            if IsInBounds(q, width, height) and IsValid(q, grid, r, cell_size) then
                points.append(q)
                active_list.append(q)
                InsertIntoGrid(q, grid, cell_size)
                found  $\leftarrow$  True
                break
            end if
        end for
        if  $\neg$ found then
            active_list.remove(p)
        end if
    end while
    return points
end function

```

27.1.6 Complexity

- **Time Complexity:** $O(n)$ where n is the number of points generated, thanks to the grid-based spatial lookup (Theorem 27.5).
- **Space Complexity:** $O(n)$ to store the points and the grid.

27.1.7 Usages

- **Computer Graphics:** Placing objects (like trees or grass) in a natural-looking distribution.
- **Rendering:** Anti-aliasing and jittered sampling for ray tracing.
- **Image Processing:** Halftoning and stippling (representing an image with dots).
- **Procedural Content Generation:** Generating city layouts or dungeon rooms.
- **Physical Simulation:** Initializing particles for fluid or granular simulations.

27.1.8 Testing Methods and Edge Cases

- **Distance Constraint:** Verify that for all pairs (P_i, P_j) , $\text{dist}(P_i, P_j) \geq r$.
- **Boundary Handling:** Ensure points are correctly generated near the edges of the domain.
- **Clumping:** Check the Fourier transform of the point set to verify the blue noise profile (lack of low-frequency spikes).
- **Empty Result:** If the domain is smaller than r , the algorithm should return at most one point.
- **Grid Overflow:** Ensure the grid coordinates are correctly clamped to the array bounds.

27.1.9 References

- Bridson, R. (2007). “Fast Poisson disk sampling in arbitrary dimensions”. SIGGRAPH Sketches.
- Cook, R. L. (1986). “Stochastic sampling in computer graphics”. ACM Transactions on Graphics.
- Lagae, A., & Dutré, P. (2008). “A comparison of methods for generating Poisson disk distributions”. Computer Graphics Forum.
- Wikipedia: Poisson disk sampling

27.2 Halton Points

27.2.1 Overview

Halton sequences are a type of low-discrepancy sequence used to generate points in a d -dimensional space. Unlike pseudo-random numbers, which can clump together, low-discrepancy sequences are designed to be “more uniform than random,” ensuring that every part of the domain is sampled evenly [Hal60]. Halton points are a generalization of the 1D van der Corput sequence [VdC35] to higher dimensions using different prime numbers as bases.

27.2.2 Definitions

Definition 27.7 (Low-Discrepancy Sequence). A sequence of points $\{x_i\}_{i=1}^N$ in $[0, 1]^d$ where the discrepancy D_N (a measure of non-uniformity) satisfies $D_N = O(N^{-1}(\log N)^d)$. This is near-optimal by the Schmidt theorem bound, and achieves faster convergence than pseudo-random sequences in Quasi-Monte Carlo integration [Nie92].

Definition 27.8 (Quasi-Monte Carlo (QMC)). Integration and simulation methods that use low-discrepancy sequences instead of random numbers. QMC achieves a convergence rate of $O(N^{-1})$ for integrands of bounded variation, compared to $O(N^{-1/2})$ for standard Monte Carlo [Hal60].

Definition 27.9 (Van der Corput Sequence). A 1D sequence in base b formed by reversing the digits of the base- b representation of integers. The n -th element is $\phi_b(n) = \sum_{k=0}^{\infty} a_k b^{-(k+1)}$ where $n = \sum_{k=0}^{\infty} a_k b^k$ is the base- b expansion of n [VdC35].

Definition 27.10 (Discrepancy). A mathematical measure of how “non-uniform” a set of points is. For a point set $P = \{x_1, \dots, x_N\}$ in $[0, 1]^d$, the star discrepancy is:

$$D_N^* = \sup_{\mathbf{b} \in [0, 1]^d} \left| \frac{|P \cap [0, \mathbf{b})|}{N} - \text{Vol}([0, \mathbf{b})) \right|.$$

27.2.3 Theory

The Halton sequence in d dimensions is constructed by using the first d prime numbers as bases $(p_1, p_2, \dots, p_d)$. The i -th point in the sequence is:

$$x_i = (\phi_{p_1}(i), \phi_{p_2}(i), \dots, \phi_{p_d}(i))$$

where $\phi_b(i)$ is the **radical inverse function** in base b .

To compute $\phi_b(i)$:

1. Write i in base b : $i = \sum a_k b^k$.
2. Reverse the digits and place them after a decimal point: $\phi_b(i) = \sum a_k b^{-(k+1)}$.

Because the prime bases are co-prime, the coordinates in different dimensions are effectively “de-correlated,” filling the d -dimensional unit hypercube uniformly.

Theorem 27.11 (Discrepancy Bound for Halton Sequences). *For the first N points of the d -dimensional Halton sequence, the star discrepancy satisfies:*

$$D_N^* = O\left(\frac{1}{N} \prod_{i=1}^d \log p_i \cdot \log N\right) = O\left(\frac{(\log N)^{d+1}}{N}\right).$$

This is significantly better than the $O(N^{-1/2})$ convergence of pseudo-random Monte Carlo [Hal60, Nie92].

Proof. The proof proceeds by the Koksma–Hlawka inequality, which bounds the integration error by the product of the discrepancy D_N^* and the variation of the integrand. For the Halton sequence, the discrepancy is bounded using the Erdős–Turán inequality on the digital sequence construction. Each coordinate ϕ_{p_i} is a van der Corput sequence in base p_i , with discrepancy $O(\log p_i/N)$. By the product property of digital sequences [Nie92], the multi-dimensional discrepancy is the product of the one-dimensional discrepancies, giving the stated bound. $\square$

Example 27.12 (First Five Halton Points in 2D). Using bases $p_1 = 2$ and $p_2 = 3$:

n	$\phi_2(n)$	$\phi_3(n)$	(x, y)
1	$0.1_2 = 0.5$	$0.1_3 = 0.\bar{3}$	$(0.500, 0.333)$
2	$0.01_2 = 0.25$	$0.2_3 = 0.\bar{6}$	$(0.250, 0.667)$
3	$0.11_2 = 0.75$	$0.01_3 = 0.1\bar{1}$	$(0.750, 0.111)$
4	$0.001_2 = 0.125$	$0.12_3 = 0.4\bar{4}$	$(0.125, 0.444)$
5	$0.101_2 = 0.625$	$0.22_3 = 0.\bar{7}$	$(0.625, 0.778)$

Notice how the points are evenly spread across the unit square with no clumping, unlike pseudo-random sampling.

27.2.4 Pseudocode

Algorithm 74 Halton Sequence Generator

```

function HALTONSEQUENCE(num_points, dimensions)
   $primes \leftarrow \text{GetFirstPrimes}(\text{dimensions})$ 
   $points \leftarrow []$ 
  for  $i$  in range(1, num_points + 1) do
     $point \leftarrow []$ 
    for  $d$  in range(dimensions) do
       $point.append(\text{RadicalInverse}(i, primes[d]))$ 
    end for
     $points.append(point)$ 
  end for
  return  $points$ 
end function

```

```

function RADICALINVERSE(n, base)
   $val \leftarrow 0$ 
   $inv\_base \leftarrow 1.0/base$ 
   $f \leftarrow inv\_base$ 
  while  $n > 0$  do
     $val \leftarrow val + (n \bmod base) \cdot f$ 
     $n \leftarrow \lfloor n/base \rfloor$ 
     $f \leftarrow f \cdot inv\_base$ 
  end while return  $val$ 
end function

```

27.2.5 Parameter Selection

- **Prime Bases:** Must use unique prime numbers for each dimension. Using non-primes or identical bases will lead to strong correlations (linear patterns).
- **Burn-in (Leaping):** For higher dimensions, the first few hundred points can show patterns. “Scrambled Halton” or skipping the initial points can improve uniformity [Nie92].

27.2.6 Complexity

- **Time Complexity:** $O(N \cdot D \cdot \log_b N)$ to generate N points in D dimensions. Since b is small, this is very fast.
- **Space Complexity:** $O(D)$ to store the current point (or $O(N \cdot D)$ to store the full sequence).

27.2.7 Usages

- **Quasi-Monte Carlo Integration:** Estimating high-dimensional integrals with faster convergence ($O(1/N)$) than standard Monte Carlo ($O(1/\sqrt{N})$).
- **Computer Graphics:** Sampling light sources and pixels for ray tracing and path tracing.
- **Physics Simulation:** Initializing particles in a way that minimizes clumping.
- **Global Optimization:** Sampling the search space for black-box optimization algorithms.
- **Financial Modeling:** Pricing complex derivatives where high-dimensional integrals are involved.

27.2.8 Testing Methods and Edge Cases

- **Range Check:** All coordinates must be in the interval $[0, 1)$.
- **Uniformity:** Test using the “Chi-Squared” test or by calculating the actual discrepancy of the set.
- **Dimension Correlation:** Plot 2D projections of high-dimensional sequences (e.g., dim 10 vs dim 11) to check for “ghosting” patterns.
- **Large N :** Ensure the radical inverse function doesn’t lose precision for very large indices.
- **Base Selection:** Verify that using the same base for two dimensions results in points lying on a single line.

27.2.9 References

- Halton, J. H. (1960). “On the efficiency of certain quasi-random sequences of points in evaluating multi-dimensional integrals”. *Numerische Mathematik*.
- Niederreiter, H. (1992). “Random Number Generation and Quasi-Monte Carlo Methods”. SIAM.
- Van der Corput, J. G. (1935). “Verteilungsfunktionen”. *Proc. Akad. Wet. Amsterdam*.
- Wikipedia: Halton sequence

27.3 Uniform Random Point Generation

27.3.1 Overview

Generating points uniformly at random inside a basic shape—square, rectangle, circle, or sphere—is a building block for Monte Carlo simulation, rendering, and randomized algorithms. Two families of methods are standard: **rejection sampling**, which works for any shape, and exact **transform** methods that map uniform variables onto the shape without waste [Dev86]. For the square and rectangle the task is trivial (independent coordinates); for the circle and sphere a naive coordinate-based approach is biased, so the Jacobian of the mapping must be corrected.

27.3.2 Definitions

Definition 27.13 (Uniform Point Set). A point set is *uniform* in a region $\Omega \subseteq \mathbb{R}^d$ if the number of points in every measurable subset $A \subseteq \Omega$ is proportional to $\text{Vol}(A)$. Equivalently, the points are drawn from the uniform measure on Ω .

Definition 27.14 (Rejection Sampling). To sample uniformly in a bounded region Ω , sample candidate points uniformly in a bounding box $B \supseteq \Omega$ and *accept* those that lie inside Ω . The acceptance rate is $\text{Vol}(\Omega)/\text{Vol}(B)$.

27.3.3 Theory

For the axis-aligned rectangle $[x_1, x_2] \times [y_1, y_2]$, two independent uniform coordinates $u, v \sim \text{Unif}[0, 1]$ give the exact uniform point $(x_1 + u\Delta x, y_1 + v\Delta y)$.

For the disk of radius R , uniform coordinates in $[0, 1]^2$ must be transformed with the radial density of a disk: radii cluster near the center if drawn uniformly, so the inverse of the cumulative radial measure is used.

Theorem 27.15 (Uniform Disk and Sphere Sampling). *Let $u, v, w \sim \text{Unif}[0, 1]$ be independent. Then*

- $(\sqrt{u}R \cos 2\pi v, \sqrt{u}R \sin 2\pi v)$ is uniform in the disk of radius R ;
- $u^{1/3}R$ times a unit direction (drawn by Marsaglia's method below) is uniform in the ball of radius R ;
- $(2x\sqrt{1-x^2-y^2}, 2y\sqrt{1-x^2-y^2}, 1-2(x^2+y^2))$ for (x, y) uniform in the unit disk is uniform on the unit sphere [Mar72].

Proof sketch. The first claim follows from the Jacobian of the polar map: the density of a disk is uniform in the area element $r dr d\theta$, so the radius must be distributed as $\sqrt{u}$. The last claim is Marsaglia's method: a uniform point in the unit disk, lifted to the sphere by the inverse stereographic mapping scaled by $r \mapsto (2\sqrt{1-r^2}, 1-2r^2)$, preserves the uniform area measure [Mar72]. $\square$

Rejection sampling's acceptance rate is the volume ratio: $\pi/4 \approx 0.785$ for a disk in its bounding square, $\pi/6 \approx 0.524$ for a ball in its bounding cube. The expected number of trials to accept n points is n/rate .

For the *boundary* of a polygon the uniform measure is arc length, not area. A point sampled uniformly on the boundary is obtained by choosing an edge with probability proportional to its length and then sampling uniformly along that edge; equivalently, invert a uniform variate on the cumulative perimeter length (arc-length parametrization). For the rectangle $[x_1, x_2] \times [y_1, y_2]$ with width $w = x_2 - x_1$ and height $h = y_2 - y_1$, the edge probabilities are w, h, w, h over the total perimeter $2(w + h)$, so longer sides receive proportionally more points and no corner or side is over- or under-sampled.

27.3.4 Pseudocode

27.3.5 Parameter Selection

- **Rejection vs. Transform:** use rejection for arbitrary shapes (polygons, curved regions); use the exact transforms for disk/ball/sphere to avoid waste.
- **Seed:** fix the RNG seed for reproducibility in tests.

Algorithm 75 Uniform Random Points in Basic Shapes

```

function UNIFORMINRECTANGLE(x1, x2, y1, y2, n)
    return  $[(x_1 + u \cdot (x_2 - x_1), y_1 + v \cdot (y_2 - y_1)) \text{ for } (u, v) \text{ in Rand2D}(n)]$ 
end function

function UNIFORMINCIRCLE(cx, cy, R, n)
    return  $[(cx + \sqrt{u}R \cos 2\pi v, cy + \sqrt{u}R \sin 2\pi v) \text{ for } (u, v) \text{ in Rand2D}(n)]$ 
end function

function UNIFORMONSPHERE(n)
    pts  $\leftarrow []$ 
    while |pts| < n do
        x  $\leftarrow$  Unif[-1, 1], y  $\leftarrow$  Unif[-1, 1]
        if  $x^2 + y^2 < 1$  then ▷ reject points outside the unit disk
            s  $\leftarrow \sqrt{1 - x^2 - y^2}$ 
            pts.append((2xs, 2ys, 1 - 2(x2 + y2)))
        end if
    end while
    return pts
end function

function UNIFORMONRECTANGLEBOUNDARY(x1, x2, y1, y2, n)
    w  $\leftarrow x_2 - x_1$ , h  $\leftarrow y_2 - y_1$ 
    perim  $\leftarrow 2 \cdot (w + h)$ 
    pts  $\leftarrow []$ 
    for i in range(n) do
        s  $\leftarrow$  Unif[0, perim] ▷ arc length along the boundary
        if s < w then
            p  $\leftarrow (x_1 + s, y_1)$ 
        else if s < w + h then
            p  $\leftarrow (x_2, y_1 + (s - w))$ 
        else if s < 2w + h then
            p  $\leftarrow (x_2 - (s - w - h), y_2)$ 
        else
            p  $\leftarrow (x_1, y_2 - (s - 2w - h))$ 
        end if
        pts.append(p)
    end for
    return pts
end function

function UNIFORMONPOLYGONBOUNDARY(polygon, n)
    L  $\leftarrow$  edge lengths of polygon ▷ edges in CCW order
    cum  $\leftarrow$  cumulative sums of L
    pts  $\leftarrow []$ 
    for i in range(n) do
        s  $\leftarrow$  Unif[0, cum[last]]
        j  $\leftarrow$  first index with cum[j] > s ▷ binary search
        t  $\leftarrow (s - cum[j - 1]) / L[j]$ 
        p  $\leftarrow (1 - t) \cdot v_j + t \cdot v_{j+1}$ 
        pts.append(p)
    end for
    return pts
end function

```

27.3.6 Complexity

- **Transform Methods:** $O(n)$ time and space; no waste.
- **Rejection Sampling:** expected $O(n/\text{rate})$ time; worst-case unbounded, but the geometric distribution bound applies.
- **Boundary Sampling:** $O(n)$ for the rectangle (constant work per point); $O(n \log m)$ for a general m -gon via binary search on the cumulative perimeter lengths.

27.3.7 Usages

- **Monte Carlo Integration:** estimating areas and volumes (hit-or-miss sampling).
- **Rendering:** sampling light sources and area lights.
- **Initialization:** seeding particle and agent simulations.
- **Randomized Geometry:** point-in-polygon testing and random inputs for testing geometric algorithms.

27.3.8 Testing Methods and Edge Cases

- **Uniformity:** use a Chi-squared test on a sub-grid of the shape, or a 1D marginal (e.g., the x -marginal of a disk is uniform).
- **Boundary:** points on the boundary should be rare; include or exclude boundary deterministically.
- **Radius Distribution:** verify the disk histogram of r/R is linear (density of r proportional to r).
- **Zero-area / degenerate shapes:** a rectangle with $x_1 = x_2$ or $R = 0$ must return the boundary or an empty set deterministically.
- **Boundary density:** for a $w \times h$ rectangle, the number of points on the two horizontal sides must be in ratio $w : h$ (not $1 : 1$); points on the boundary must never fall inside the shape.

27.3.9 References

- Devroye, L. (1986). “Non-Uniform Random Variate Generation”. Springer.
- Marsaglia, G. (1972). “Choosing a point from the surface of a sphere”. The Annals of Mathematical Statistics.

27.4 Random Line Segment Generation

27.4.1 Overview

Generating random line segments in 2D or 3D is used for testing segment intersection and arrangement algorithms, for simulating sensor and cable networks, and for constructing random inputs [dBCvKO08]. The natural model samples the two endpoints independently from a uniform distribution over the domain, so that every segment is equally likely up to the position of its two endpoints. An alternative model fixes the segment length and samples a random direction, which is the right model for fibers, cracks, and insect paths.

27.4.2 Definitions

Definition 27.16 (Random Segment Model). A segment in $\mathbb{R}^d$ is a pair of endpoints a, b . In the *independent-endpoint model*, a and b are drawn independently from a uniform distribution over the domain Ω . In the *fixed-length model*, a length ℓ , a midpoint (or base point), and a uniform direction determine the segment.

27.4.3 Theory

The independent-endpoint model is exact: because each endpoint is uniform in Ω , the pair (a, b) is uniform in $\Omega \times \Omega$. A fixed length requires a uniform random direction, which in 2D is the angle $\theta \sim \text{Unif}[0, 2\pi)$ and in 3D a unit vector sampled uniformly on the sphere. A cheap 3D method picks a point in the unit ball and normalizes it (rejection of the z -axis bias of naive polar coordinates), or uses Marsaglia’s method of Section 27.3.

Theorem 27.17 (Uniform Endpoints in a Box). *If a, b are independent and uniform in a box $\prod_i [x_i, y_i]$, then the endpoints and the segment are uniformly distributed over all segments whose endpoints lie in the box. The marginal density of the segment midpoint is not uniform: it concentrates near the center, since midpoints arise from many endpoint pairs.*

27.4.4 Pseudocode

Algorithm 76 Random Line Segment Generation

```

function RANDOMSEGMENT2D(x1, x2, y1, y2, n)
    return [RandomSegmentInBox( $x_1, x_2, y_1, y_2$ ) for  $\_$  in range( $n$ )]
end function

function RANDOMSEGMENTINBOX(x1, x2, y1, y2, n)
     $a \leftarrow \text{UniformInRectangle}(x_1, x_2, y_1, y_2, 1)[0]$ 
     $b \leftarrow \text{UniformInRectangle}(x_1, x_2, y_1, y_2, 1)[0]$ 
    return ( $a, b$ )
end function

function RANDOMFIXEDLENGTHSEGMENT2D(cx, cy, ell, n)
    return [(( $cx, cy$ ), ( $cx + \ell \cos \theta, cy + \ell \sin \theta$ )) for  $\theta \sim \text{Unif}[0, 2\pi)$  in range( $n$ )]
end function

function RANDOMSEGMENT3D(x1, x2, y1, y2, z1, z2, n)
     $pts \leftarrow []$ 
    while  $|pts| < 2n$  do
         $u, v, w \sim \text{Unif}[0, 1)$ 
         $p \leftarrow (x_1 + u(x_2 - x_1), y_1 + v(y_2 - y_1), z_1 + w(z_2 - z_1))$ 
         $pts.append(p)$ 
    end while
    return [( $pts[2i], pts[2i + 1]$ ) for  $i$  in range( $n$ )]
end function

```

27.4.5 Parameter Selection

- **Endpoint model vs. fixed length:** choose the independent endpoint model for intersection and arrangement stress tests; fixed length for fiber and crack models.

- **Seed:** fix the RNG seed for reproducible random inputs.

27.4.6 Complexity

- **Time:** $O(n)$ to generate n segments.
- **Space:** $O(n)$ to store the segments; $O(1)$ streaming.

27.4.7 Usages

- **Testing segment intersection** algorithms with large random inputs.
- **Simulating sensor links**, cables, and cracks.
- **Arrangement** experiments and **sweep-line** stress tests.

27.4.8 Testing Methods and Edge Cases

- **Endpoint Range:** verify all endpoints lie in the box.
- **Count:** verify exactly n segments are returned.
- **Distinctness:** for the independent model, degenerate zero-length segments are possible but should be rare; decide whether to keep or reject them.
- **3D Direction:** for fixed length in 3D, verify the direction vectors have norm ℓ and are isotropically distributed.

27.4.9 References

- Devroye, L. (1986). “Non-Uniform Random Variate Generation”. Springer.
- de Berg, M., Cheong, O., van Kreveld, M., & Overmars, M. (2008). “Computational Geometry: Algorithms and Applications”. Springer.

27.5 Random Polygon Generation

27.5.1 Overview

Generating random convex and concave (simple) polygons in 2D provides stress tests for polygon algorithms: triangulation, point-in-polygon, convex partitioning, and boolean operations [dBCvKO08]. The two standard constructions are **angle-sorted points on a curve** for convex polygons, and **angular sorting around a center** for concave polygons. Both run in $O(n \log n)$ time and are easy to implement correctly.

27.5.2 Definitions

Definition 27.18 (Random Convex Polygon). A convex polygon with n vertices sampled by sorting n angles uniformly in $[0, 2\pi)$ and placing the corresponding points on the unit circle (or an ellipse/randomly scaled radius).

Definition 27.19 (Random Simple Polygon). A simple (non-self-intersecting) polygon built by ordering points by their polar angle around their centroid, giving a star-shaped polygon; a further random radial perturbation can introduce reflex (concave) vertices while preserving simplicity.

27.5.3 Theory

Sorting points by polar angle around any point inside their convex hull produces a simple polygon: the edges of the star-shaped polygon do not cross. For the convex case, points on a strictly convex curve (a circle or ellipse) sorted by angle give a convex polygon. The polygon is convex if and only if every interior angle is less than π ; an $O(n)$ check of the turn direction at each vertex certifies this.

Theorem 27.20 (Star-Shaped Polygonization). *If $P = \{p_1, \dots, p_n\}$ are in general position and c is a point in their convex hull, then the polygon obtained by connecting the points sorted by polar angle around c is simple and star-shaped with kernel containing c [dBCvKO08].*

Proof sketch. Every edge of the sorted polygon bounds a triangle with apex c that lies inside the polygon; adjacent triangles meet only along shared edges, so the polygon is simple and every point is visible from c . $\square$

27.5.4 Pseudocode

Algorithm 77 Random Polygon Generation

```

function RANDOMCONVEXPOLYGON( $n$ )
   $\theta \leftarrow \text{sort}(\text{Unif}[0, 2\pi) \text{ for } \_ \text{ in range}(n))$ 
   $r \leftarrow \text{Unif}[0.5, 1.0) \text{ for } \_ \text{ in range}(n)$ 
  return  $[(r_i \cos \theta_i, r_i \sin \theta_i) \text{ for } i \text{ in range}(n)]$ 
end function

function RANDOMSIMPLEPOLYGON( $n$ )
   $p \leftarrow \text{RandomConvexPolygon}(n)$ 
   $c \leftarrow \text{mean}(p)$ 
   $q \leftarrow \text{sort}(p, \text{key} = \text{angle around } c)$ 
  for  $i$  in range( $n$ ) do
    if reflex vertex at  $q_i$  then                                 $\triangleright$  flip to concave by radial move
       $q_i \leftarrow c + (1 + \varepsilon_i) \cdot (q_i - c)$ 
    end if
  end for
  return  $q$ 
end function

```

27.5.5 Parameter Selection

- **Radius range:** keep $r \in [0.5, 1.0)$ so points stay away from the origin and the polygon is not degenerate.
- **Perturbation:** small ε_i moves in the radial direction create reflex vertices without destroying simplicity; large moves can cause self-intersection.

27.5.6 Complexity

- **Time:** $O(n \log n)$ for sorting the angles; $O(n)$ for the perturbation and the convexity check.
- **Space:** $O(n)$.

27.5.7 Usages

- **Testing triangulation, point-in-polygon, and boolean operations** on large random inputs.
- **Benchmarking** convex partitioning (Keil–Snoeyink) and art gallery algorithms on concave polygons.

27.5.8 Testing Methods and Edge Cases

- **Simplicity**: verify no two edges intersect (except adjacent edges at shared vertices).
- **Convexity**: for the convex generator, check all interior angles $< \pi$ (positive turn direction at every vertex).
- **Vertex Count**: the polygon has exactly n distinct vertices.
- **Degenerate cases**: reject duplicate angles and points at the origin to keep the polygon simple.

27.5.9 References

- de Berg, M., Cheong, O., van Kreveld, M., & Overmars, M. (2008). “Computational Geometry: Algorithms and Applications”. Springer.
- Devroye, L. (1986). “Non-Uniform Random Variate Generation”. Springer.

27.6 Random Walk

27.6.1 Overview

A random walk is a mathematical object, known as a stochastic or random process, that describes a path consisting of a succession of random steps on some mathematical space such as the integers or a 2D/3D grid. In computational geometry and physics, random walks are used to model Brownian motion, diffusion, and to approximate the solutions of various partial differential equations (PDEs) through Monte Carlo methods [Pea05].

27.6.2 Definitions

Definition 27.21 (Step Size (ℓ)). The distance moved in a single step of the random walk. This may be fixed or drawn from a distribution (e.g., Gaussian).

Definition 27.22 (Lattice Walk). A random walk restricted to a discrete grid (e.g., integer coordinates). At each step, the walker moves to one of the neighboring lattice points with prescribed probabilities.

Definition 27.23 (Isotropic Walk). A walk where every direction is equally likely. In 2D, the step direction θ is uniformly distributed on $[0, 2\pi)$.

Definition 27.24 (Mean Squared Displacement (MSD)). A measure of how far the walker deviates from its starting point over time. For a simple d -dimensional random walk with step size ℓ , $\text{MSD}(n) = \langle |\mathbf{r}_n - \mathbf{r}_0|^2 \rangle = d \cdot \ell^2 \cdot n$, so $\text{MSD} \propto t$ [Spi76].

27.6.3 Theory

The theory of random walks is closely related to the diffusion equation $\frac{\partial \phi}{\partial t} = D \nabla^2 \phi$ [Spi76].

1. **1D Walk:** A walker starts at 0 and moves +1 or -1 with probability 1/2. After n steps, the expected position is 0, but the standard deviation is $\sqrt{n}$.
2. **2D/3D Walk:** The walker moves in a random direction. Polya's Theorem [P21] states that a random walk on a 2D lattice is “recurrent” (it will return to the start with probability 1), while in 3D it is “transient” (it may never return).
3. **Walk on Meshes:** A random walk on a mesh jumps from a vertex to one of its adjacent vertices. The probability of choosing a neighbor can be uniform (1/degree) or weighted by edge length or cotangent weights to simulate specific geometric properties.

Theorem 27.25 (Polya's Recurrence Theorem). *A simple random walk on the $\mathbb{Z}^d$ lattice is recurrent (returns to the origin with probability 1) for $d = 1$ and $d = 2$, and transient (returns with probability less than 1) for $d \geq 3$. Specifically, the return probability in $d = 3$ is approximately 0.3405 [P21].*

Proof. The probability of return is related to the Green's function $G(\mathbf{0}) = \sum_{n=0}^{\infty} P_n(\mathbf{0} \rightarrow \mathbf{0})$, which is the expected number of visits to the origin. The walk is recurrent if and only if $G(\mathbf{0}) = \infty$. For a d -dimensional walk, the return probability at step $2n$ is:

$$P_{2n}(\mathbf{0} \rightarrow \mathbf{0}) \sim \frac{C_d}{n^{d/2}}$$

for some constant $C_d > 0$. The series $\sum_{n=1}^{\infty} n^{-d/2}$ converges if and only if $d/2 > 1$, i.e., $d > 2$. Therefore, $G(\mathbf{0}) = \infty$ for $d \leq 2$ (recurrent) and $G(\mathbf{0}) < \infty$ for $d \geq 3$ (transient) [Spi76]. $\square$

Example 27.26 (1D Random Walk: Expected Position After 100 Steps). Consider a 1D random walk starting at position 0, taking steps of +1 or -1 with equal probability. After $n = 100$ steps:

- Expected position: $\mathbb{E}[X_{100}] = 0$.
- Standard deviation: $\sigma = \sqrt{100} = 10$.
- Expected squared displacement: $\mathbb{E}[X_{100}^2] = 100$.

This means that while the walker is equally likely to be at any position from -100 to +100, most outcomes will cluster within about 10 units of the origin.

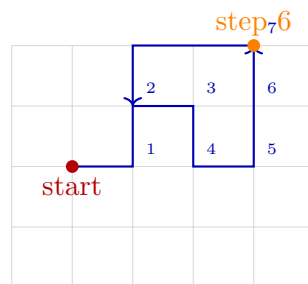


Figure 27.2: Random walk on a grid: at each step, move to a uniformly random neighboring vertex.

Algorithm 78 2D Random Walk

```
function RANDOMWALK2D(start_point, num_steps, step_size)
    current  $\leftarrow$  start_point
    path  $\leftarrow$  [current]
    for i in range(num_steps) do                                      $\triangleright$  1. Choose random direction
         $\theta \leftarrow \text{random.uniform}(0, 2\pi)$ 
                                                 $\triangleright$  2. Update position

        dx  $\leftarrow$  step_size  $\cdot$  cos( $\theta$ )
        dy  $\leftarrow$  step_size  $\cdot$  sin( $\theta$ )
        current  $\leftarrow$  Point(current.x + dx, current.y + dy)
        path.append(current)
    end for
    return path
end function
```

27.6.4 Pseudocode**Simple 2D Random Walk (Continuous)****Random Walk on a Mesh**

Algorithm 79 Random Walk on a Mesh

```
function MESHRANDOMWALK(mesh, start_v, num_steps)
    current_v  $\leftarrow$  start_v
    visited_v  $\leftarrow$  [current_v]
    for i in range(num_steps) do
        neighbors  $\leftarrow$  mesh.GetNeighbors(current_v)            $\triangleright$  1. Choose a random neighbor
        current_v  $\leftarrow$  random.choice(neighbors)
        visited_v.append(current_v)
    end for
    return visited_v
end function
```

27.6.5 Parameter Selection

- **Step Size:** Constant step size vs. variable (Gaussian) step size.
- **Connectivity:** On a grid, 4-connectivity (von Neumann) vs. 8-connectivity (Moore).
- **Weights:** In weighted random walks, the probability of a step is proportional to some value (e.g., $e^{-\Delta E/kT}$).

27.6.6 Complexity

- **Time Complexity:** $O(n)$ where n is the number of steps.
- **Space Complexity:** $O(n)$ to store the full path, or $O(1)$ if only the final position is needed.

27.6.7 Usages

- **Monte Carlo Simulation:** Estimating areas or volumes of complex shapes (Hit-or-Miss Monte Carlo).

- **Mesh Analysis:** Using random walks to calculate “closeness” between vertices or to partition meshes.
- **Generative Art:** Creating organic, branching patterns (e.g., Diffusion-Limited Aggregation).
- **Network Science:** Identifying communities in a graph or calculating PageRank.
- **Finance:** Modeling stock price movements (Geometric Brownian Motion).
- **Physics:** Simulating the movement of gas particles or the propagation of heat.

27.6.8 Testing Methods and Edge Cases

- **Mean Position:** Verify that after many trials, the average final position of an isotropic walk is the starting point.
- **Scaling Law:** Verify that the distance from the start grows as $\sqrt{n}$ on average.
- **Boundaries:** Test how the walk behaves when it hits a wall (e.g., reflection vs. absorption).
- **Mesh Boundaries:** Ensure the walker doesn’t “fall off” the edge of an open mesh.
- **Looping:** Observe how often the walker visits the same vertex in 2D vs. 3D.

27.6.9 References

- Pearson, K. (1905). “The Problem of the Random Walk”. *Nature*.
- Polya, G. (1921). “Über eine Aufgabe der Wahrscheinlichkeitsrechnung betreffend die Irrfahrt im Straßennetz”. *Mathematische Annalen*.
- Spitzer, F. (1976). “Principles of Random Walk”. Springer.
- Wikipedia: Random walk

27.7 Self-Avoiding Walk

27.7.1 Overview

A self-avoiding walk (SAW) is a sequence of moves on a lattice (such as a 2D or 3D grid) that does not visit the same point more than once. Unlike a simple random walk, which can cross its own path, a SAW must satisfy the constraint of self-exclusion. This exclusion makes SAWs significantly more difficult to analyze and simulate, as the “future” path depends on the entire “history” of the walk [MS96].

27.7.2 Definitions

Definition 27.27 (Lattice). A regular grid of points (e.g., square, hexagonal, cubic). The lattice determines the set of allowed moves at each step.

Definition 27.28 (Length (n)). The number of steps in the walk. The walk consists of $n + 1$ vertices and n edges.

Definition 27.29 (Connective Constant (μ)). A lattice-specific constant describing the exponential growth rate of the number c_n of possible SAWs of length n : $c_n \sim A\mu^n n^{\gamma-1}$ as $n \rightarrow \infty$, where γ is a universal critical exponent depending only on the dimension [MS96].

Definition 27.30 (End-to-End Distance). The Euclidean distance between the starting point and the final point of the walk. For a SAW of length n , the mean end-to-end distance scales as $\langle R^2 \rangle \sim n^{2\nu}$ where ν is the Flory exponent ($\nu \approx 3/4$ in $d = 2$) [Flo53].

27.7.3 Theory

The number of simple random walks of length n on a d -dimensional lattice is exactly $(2d)^n$. For SAWs, there is no simple formula. The number of SAWs c_n is known to behave as $c_n \sim A\mu^n n^{\gamma-1}$, where μ depends on the lattice and γ is a “universal” exponent that depends only on the dimension [MS96].

Generating SAWs is a challenge due to **attrition**: if you build a walk step-by-step randomly, you often get “trapped” where all neighboring points have already been visited. This leads to several simulation strategies:

1. **Simple Sampling**: Generate random walks and discard them if they intersect themselves. (Extremely inefficient for large n).
2. **Pivot Algorithm**: Start with a valid SAW and apply a symmetry operation (rotation or reflection) to a portion of the walk. If the result is still self-avoiding, accept it. This is the most efficient known method for generating long SAWs.
3. **Rosenbluth Method**: A weighted sampling method that avoids immediate self-intersections but suffers from diminishing weights.

Theorem 27.31 (Connective Constant Bounds). *For the square lattice $\mathbb{Z}^2$, the connective constant satisfies $\mu \leq 2 + \sqrt{2} \approx 3.414$ (the Madras–Slade bound). Exact values are known only for self-avoiding walks on certain special lattices. For the hexagonal lattice, $\mu = \sqrt{2 + \sqrt{2}} \approx 1.848$ [MS96].*

Proof. The upper bound $\mu \leq 2 + \sqrt{2}$ is proved using the lace expansion technique, which decomposes the SAW into a random walk plus correction terms that account for self-avoidance. The key identity relates the two-point function of the SAW to that of the simple random walk through a renormalization argument. For the hexagonal lattice, the exact value follows from a combinatorial argument using the correspondence between SAWs and the zero-field Ising model at the critical temperature [MS96]. $\square$

Example 27.32 (Number of SAWs on $\mathbb{Z}^2$ for Small n). The exact counts c_n of self-avoiding walks on the square lattice starting from the origin:

n	c_n
1	4
2	12
3	36
4	100
5	284
6	780

From these values, one can estimate $\mu \approx c_{n+1}/c_n \rightarrow \mu \approx 2.64$ as n grows, converging to the true value $\mu \approx 2.638$.

27.7.4 Pseudocode

Simple Recursive SAW Generator

Note that Algorithm 80 uses exhaustive backtracking. For large n , the **pivot algorithm** is far more efficient, achieving near- $O(n)$ amortized cost per sample [MS96].

Algorithm 80 Generate Self-Avoiding Walk (Recursive)

```

function GENERATESAW(path, n, lattice)
  if len(path) = n then return path
  end if
  current ← path[-1]
  neighbors ← lattice.GetNeighbors(current)
  random.shuffle(neighbors)
  for all neighbor ∈ neighbors do
    if neighbor ∉ path then
      result ← GenerateSAW(path + [neighbor], n, lattice)
      if result ≠ None then return result
    end if
  end for
  return None
end function

```

▷ Shuffle neighbors to explore randomly

▷ 1. Recursive attempt

▷ 2. Dead end (Backtrack) **return** None

27.7.5 Parameter Selection

- **Lattice Type:** Square (4 neighbors) and Hexagonal (3 neighbors) are common in 2D.
- **Algorithm:** Use the **Pivot Algorithm** for generating very long SAWs (thousands of steps) as it is much faster than simple recursion or sampling.

27.7.6 Complexity

- **Simple Recursion:** $O(\mu^n)$ in the worst case, as it is an exhaustive search.
- **Pivot Algorithm:** $O(n)$ or even $O(n^{0.5})$ per successful update, making it feasible for $n \approx 10^6$.
- **Space Complexity:** $O(n)$ to store the coordinates of the walk.

27.7.7 Usages

- **Polymer Physics:** SAWs are the standard model for “linear polymers” in a good solvent, where the exclusion principle represents the “excluded volume” of atoms [Flo53].
- **Proteins:** Modeling the backbone of a protein during folding.
- **Network Science:** Analyzing paths in social or communication networks where nodes shouldn’t be repeated.
- **Material Science:** Studying the distribution of chains in a porous medium.
- **Combinatorics:** A fundamental problem in counting and probability.

27.7.8 Testing Methods and Edge Cases

- **No Intersections:** Verify that all points in the generated path are unique.
- **Connectivity:** Ensure each point in the path is a valid neighbor of the previous point on the lattice.

- **Connective Constant:** Verify that for small n , the number of generated SAWs matches known exact counts (Theorem [27.32](#)).
- **Lattice Boundaries:** If the walk is confined to a box, ensure it handles the boundaries correctly.
- **Trap Detection:** Verify the algorithm correctly backtracks or terminates when the walker is surrounded by visited points.

27.7.9 References

- Madras, N., & Slade, G. (1996). “The Self-Avoiding Walk”. Birkhäuser.
- Flory, P. J. (1953). “Principles of Polymer Chemistry”. Cornell University Press.
- Kennedy, T. (2002). “A faster implementation of the pivot algorithm for self-avoiding walks”. Journal of Statistical Physics.
- Wikipedia: Self-avoiding walk

Chapter 28

Shape Representation and Reconstruction

- 28.1 Shape Representation
- 28.2 Implicit and Parametric Representations
- 28.3 Reconstruction from Point Clouds
- 28.4 Surface Reconstruction
- 28.5 Signed Distance Functions
- 28.6 Level Sets and Front Propagation
- 28.7 Poisson Surface Reconstruction
- 28.8 Applications
- 28.9 Exercises

Chapter 29

Geometric Matching

29.1 Comparing Geometric Objects

29.2 Hausdorff Distance

29.3 Fréchet Distance

29.4 Discrete Fréchet Distance

29.5 Shape Matching

29.6 Point-Set Registration

29.7 Rigid Transformations

29.8 Iterative Closest Point

29.9 Geometric Hashing

29.10 Applications in Computer Vision

29.11 Exercises

Chapter 30

Computational Topology

30.1 Mesh Analysis and Topology

30.2 Mesh Connected Components

30.2.1 1. Overview

Finding the connected components of a mesh is the process of partitioning the mesh into sets of faces or vertices that are reachable from each other through edge connectivity. A single 3D file often contains multiple disjoint geometric objects (e.g., a table and four separate chairs). Identifying these components is a standard preprocessing step for mesh analysis [BKP⁺10], repair, and selective editing.

30.2.2 2. Definitions

Definition 30.1 (Connected Component). A maximal subgraph of a mesh where any two nodes (vertices or faces) are connected by a path of adjacency relations.

Definition 30.2 (Adjacency). In a mesh, two faces are *edge-adjacent* if they share an edge. Two vertices are adjacent if they share an edge.

Definition 30.3 (Graph Traversal). The process of visiting all reachable elements in a graph starting from a seed point, following a systematic exploration strategy such as breadth-first or depth-first search.

30.2.3 3. Theory

The problem of finding connected components in a mesh is equivalent to finding the connected components of its dual graph (where nodes are faces) or its primal graph (where nodes are vertices).

Theorem 30.4 (Component Count). *Let M be a mesh with n vertices and m edges. The connected components of M can be computed in $O(n + m)$ time using either breadth-first search or depth-first search.*

Proof. Each vertex and edge is visited at most once by BFS or DFS. The algorithm maintains a set of unvisited vertices, and each vertex is removed from this set exactly once. For each vertex, all its incident edges are examined, contributing $O(\deg(v))$ work. Summing over all vertices gives $\sum_v O(\deg(v)) = O(m)$ by the Handshaking Lemma [Die17]. Combined with the $O(n)$ initialization, the total is $O(n + m)$. $\square$

Standard Breadth-First Search (BFS) or Depth-First Search (DFS) algorithms are used:

1. Initialize a set of unvisited faces.
2. While there are unvisited faces:
 - Pick an unvisited face as a “seed.”
 - Start a new component.
 - Perform a BFS/DFS to find all faces reachable from the seed via shared edges.
 - Mark all reached faces as visited and add them to the current component.

For manifold meshes, face-connectivity is usually sufficient. For non-manifold or point cloud data, vertex-connectivity might be required.

Example 30.5 (Connected Components). Consider a mesh consisting of a tetrahedron (4 vertices, 4 faces) and a disconnected triangle (3 vertices, 1 face). The algorithm starts at an arbitrary face of the tetrahedron and discovers all 4 faces via adjacency. It then picks the lone triangle as a new seed, finding 1 face. The result is two components: one with 4 faces and one with 1 face. The Euler characteristic of the first component is $\chi_1 = 4 - 6 + 4 = 2$ (sphere-like), and the second is $\chi_2 = 3 - 3 + 1 = 1$ (disk).

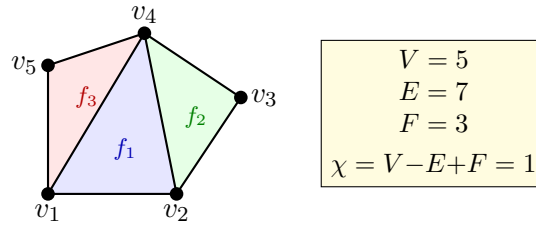


Figure 30.1: A triangle mesh with vertices v_i , edges, and faces f_j . The Euler characteristic $\chi = V - E + F$ is a topological invariant.

30.2.4 4. Pseudocode

30.2.5 5. Parameter Selection

- **Connectivity Type:** **Edge-connectivity** (faces sharing an edge) is standard. **Vertex-connectivity** (faces sharing only a vertex) is “weaker” and will result in fewer, larger components.
- **Data Structure:** Using a Half-Edge [GS85] structure or an Adjacency List makes finding neighbors $O(1)$, leading to optimal performance.

30.2.6 6. Complexity

- **Time Complexity:** $O(F)$ where F is the number of faces, as each face and each adjacency relationship is visited a constant number of times.
- **Space Complexity:** $O(F)$ to store the visited status and the resulting component lists.

30.2.7 7. Usages

- **Mesh Repair:** Identifying and removing small “floating” noise components (e.g., isolated triangles) from a scan.

Algorithm 81 Find Mesh Components

```

1: function FINDMESHCOMPONENTS(mesh)
2:   unvisited  $\leftarrow$  set(range(num_faces))
3:   components  $\leftarrow$  []
4:   while unvisited do
5:     seed  $\leftarrow$  unvisited.pop()
6:     current_component  $\leftarrow$  [seed]
7:     queue  $\leftarrow$  [seed]
8:     while queue do
9:       f  $\leftarrow$  queue.pop(0)
10:      for neighbor  $\in$  GetAdjacentFaces(f, mesh) do
11:        if neighbor  $\in$  unvisited then
12:          unvisited.remove(neighbor)
13:          current_component.append(neighbor)
14:          queue.append(neighbor)
15:        end if
16:      end for
17:    end while
18:    components.append(current_component)
19:  end while
20:  return components
21: end function

```

- **Selective Editing:** Selecting an entire object by clicking on a single triangle (e.g., “Select Linked” in Blender).
- **3D Printing:** Ensuring that a model consists of a single connected “shell” or identifying internal voids.
- **Rigging and Animation:** Automatically grouping parts of a character (e.g., separating the body from the clothing or eyes).
- **Physics Simulation:** Treating each connected component as a separate rigid body for collision and dynamics.

30.2.8 8. Testing Methods and Edge Cases

- **Single Watertight Object:** Should return exactly one component containing all faces.
- **Disconnected Objects:** Verify that multiple separate objects are correctly identified as distinct components.
- **Points/Lines:** Ensure the algorithm handles “degenerate” components like isolated vertices or edges.
- **Non-Manifold Edges:** Verify that connectivity is correctly traced even if three or more faces share an edge.
- **Large Meshes:** Test performance on meshes with millions of triangles to ensure linear scaling.

30.2.9 9. References

- Hopcroft, J., & Tarjan, R. (1973). “Efficient algorithms for graph manipulation.” Communications of the ACM.
- Botsch, M., Kobbelt, L., Pauly, M., Alliez, P., & Lévy, B. [BKP⁺10] “Polygon Mesh Processing.” CRC Press.
- Wikipedia: Connected component (graph theory).

30.3 Mesh Boundary Detection

30.3.1 1. Overview

Mesh boundary detection is the process of identifying vertices and edges that form the boundary of a surface mesh [BKP⁺10]. In a manifold mesh, every edge is shared by exactly two faces. Boundary edges are those that are part of only one face. Identifying these boundaries is essential for surface parameterization, texture mapping, and mesh editing.

30.3.2 2. Definitions

Definition 30.6 (Surface Mesh). A collection of vertices, edges, and faces that represent a 2-dimensional surface embedded in 3-dimensional space.

Definition 30.7 (Boundary Edge). An edge that is incident to exactly one face. In a half-edge data structure [GS85], this corresponds to a half-edge whose `twin` is null.

Definition 30.8 (Boundary Vertex). A vertex that is incident to at least one boundary edge.

Definition 30.9 (Boundary Loop). A connected cycle of boundary edges and vertices that define a hole or the perimeter of the mesh.

30.3.3 3. Theory

The fundamental principle of mesh boundary detection is counting the face-edge incidence. For a given mesh M with F faces:

1. Iterate through all faces.
2. For each face, extract its oriented edges (e.g., $(v_1, v_2), (v_2, v_3), (v_3, v_1)$).
3. Store the occurrence of each edge in a hash map, ignoring the orientation (i.e., (v_i, v_j) is the same as (v_j, v_i)).
4. Edges that appear only **once** in the total count are boundary edges.

Theorem 30.10 (Boundary Edge Characterization). *In a closed 2-manifold mesh, every edge is shared by exactly two faces. An edge e is a boundary edge if and only if it is incident to exactly one face.*

Proof. By definition of a 2-manifold with boundary, each edge is incident to either one face (boundary edge) or two faces (interior edge). The edge count algorithm enumerates all face-edge incidences. An interior edge appears twice (once per adjacent face), while a boundary edge appears exactly once. Hence, edges with count equal to 1 are precisely the boundary edges. $\square$

Once the boundary edges are identified, they can be chained together by matching endpoints to form one or more boundary loops.

Example 30.11 (Boundary Detection). Consider a single triangle with vertices (v_0, v_1, v_2) and edges $\{(v_0, v_1), (v_1, v_2), (v_2, v_0)\}$. The edge count map stores each edge with count 1. All three edges are boundary edges, and chaining them produces a single boundary loop (v_0, v_1, v_2) . For a tetrahedron (4 triangles), every edge is shared by exactly 2 faces, so the count is always 2 and no boundary edges are found.

30.3.4 4. Pseudocode

Algorithm 82 Detect Mesh Boundaries

```

1: function DETECTMESHBOUNDARIES(mesh)
2:   edge_counts  $\leftarrow \{\}$ 
3:   for face  $\in$  mesh.faces do
4:     for i = 0  $\rightarrow$  len(face) - 1 do
5:        $v_1, v_2 \leftarrow \text{SortedPair}(\text{face}[i], \text{face}[(i + 1) \bmod \text{len}(\text{face})])$ 
6:       edge_counts[ $(v_1, v_2)$ ]  $\leftarrow$  edge_counts.get( $(v_1, v_2)$ , 0) + 1
7:     end for
8:   end for
9:   boundary_edges  $\leftarrow [e \mid e, c \in \text{edge\_counts.items}() \text{ if } c = 1]$ 
10:  loops  $\leftarrow []$ 
11:  visited_edges  $\leftarrow \{\}$ 
12:  for start_edge  $\in$  boundary_edges do
13:    if start_edge  $\in$  visited_edges then
14:      continue
15:    end if
16:    current_loop  $\leftarrow [\text{start\_edge}[0], \text{start\_edge}[1]]$ 
17:    visited_edges.add(start_edge)
18:    while True do
19:      next_edge  $\leftarrow \text{FindNextBoundaryEdge}(\text{current\_loop}[-1], \text{boundary\_edges}, \text{visited\_edges})$ 
20:      if next_edge = None or next_edge = start_edge then
21:        break
22:      end if
23:      next_v  $\leftarrow \text{next\_edge}[1]$  if next_edge[0] = current_loop[-1] else next_edge[0]
24:      current_loop.append(next_v)
25:      visited_edges.add(next_edge)
26:    end while
27:    loops.append(current_loop)
28:  end for
29:  return loops
30: end function

```

30.3.5 5. Parameter Selection

- **Mesh Representation:** Using a Half-Edge [GS85] data structure makes boundary detection trivial, as boundary edges are typically stored as “null” half-edges or explicit boundary objects.
- **Sorting:** When using an indexed face set, sorting the vertex IDs for each edge (e.g., $(\min(u, v), \max(u, v))$) ensures that the count correctly accounts for both orientations of the same geometric edge.

30.3.6 6. Complexity

- **Time Complexity:** $O(F)$, where F is the number of faces in the mesh. Chaining the loops takes $O(E_{\text{boundary}})$.
- **Space Complexity:** $O(E)$ to store the edge counts.

30.3.7 7. Usages

- **Surface Parameterization (UV Mapping):** BFF and Harmonic Mapping require the mesh boundary to be identified and mapped to a 2D shape.
- **Mesh Repair:** Identifying and filling holes in scanned 3D models.
- **Texture Blending:** Applying effects specifically at the boundaries of different surface materials.
- **Physical Simulation:** Applying boundary conditions (like fixing a boundary in place) for Finite Element analysis.

30.3.8 8. Testing Methods and Edge Cases

- **Watertight Mesh:** A perfectly closed mesh (like a sphere) should return zero boundary edges.
- **Open Sheet:** A mesh like a flat square should return a single boundary loop containing all perimeter edges.
- **Non-Manifold Edges:** Edges shared by more than two faces (count > 2) should be flagged as errors or handled separately.
- **Duplicate Faces:** Two faces covering the same three vertices will result in edge counts of 2, making them appear “watertight” even if they are isolated.
- **Multiple Holes:** Verify that each hole is correctly identified as a separate loop.

30.3.9 9. References

- Botsch, M., Kobbelt, L., Pauly, M., Alliez, P., & Lévy, B. [BKP⁺10] “Polygon Mesh Processing.” CRC Press.
- Guibas, L. J., & Stolfi, J. [GS85] “Primitives for the manipulation of general subdivisions.” ACM Transactions on Graphics.
- Wikipedia: Boundary (topology).

30.4 Mesh Orientability

30.4.1 1. Overview

Mesh orientability is a property of a surface mesh that determines whether a consistent “outside” and “inside” (or a consistent normal direction) can be defined for the entire surface. An orientable surface allows every face to be oriented so that any two adjacent faces share an edge with opposite orientations. Non-orientable surfaces, such as the Möbius strip or the Klein bottle, cannot be assigned a consistent orientation [BKP⁺10].

30.4.2 2. Definitions

Definition 30.12 (Face Orientation). The order in which the vertices of a face are listed (e.g., (v_1, v_2, v_3)), which determines the face normal using the right-hand rule.

Definition 30.13 (Orientable Surface). A surface where face orientations can be made consistent: for every edge shared by faces F_1 and F_2 , the orientations of F_1 and F_2 must traverse the edge in opposite directions.

Definition 30.14 (Inconsistent Edge). An edge where both incident faces traverse the edge in the same direction (e.g., both use (u, v)).

Definition 30.15 (Two-Sided Surface). An orientable manifold mesh effectively has two sides (e.g., the inside and outside of a sphere).

30.4.3 3. Theory

The process of orienting a mesh is a graph-search problem.

1. Represent the mesh as a dual graph where each node is a face and edges connect adjacent faces (those sharing an edge).
2. Select an initial face and assign it an arbitrary orientation (e.g., CCW).
3. Propagate this orientation to its neighbors: if face F_1 uses edge (u, v) , then its neighbor F_2 **must** use edge (v, u) .
4. If at any point we encounter a neighbor that has already been oriented in a way that conflicts with its current neighbor, the mesh is **non-orientable**.

Theorem 30.16 (Orientability Characterization). *A connected 2-manifold surface S is orientable if and only if it does not contain a Möbius strip as a sub-surface. Equivalently, S is orientable if and only if every edge is traversed by its two incident faces in opposite directions.*

Proof. If S is orientable, we can assign consistent face orientations, so every edge is traversed in opposite directions by its two incident faces. Conversely, if every edge is traversed in opposite directions, the BFS propagation never encounters a conflict, and we obtain a consistent global orientation. The non-orientable case requires the existence of an orientation-reversing cycle, which is topologically equivalent to a Möbius strip. $\square$

Mathematically, a manifold surface is orientable if and only if it does not contain a Möbius strip as a sub-surface.

Example 30.17 (Orienting a Triangle Mesh). Consider a tetrahedron with faces $F_1 = (v_0, v_1, v_2)$, $F_2 = (v_0, v_2, v_3)$, $F_3 = (v_0, v_3, v_1)$, $F_4 = (v_1, v_3, v_2)$. Starting at F_1 with CCW orientation, edge (v_0, v_2) is traversed from v_0 to v_2 . Face F_2 must then traverse this edge as (v_2, v_0) , which is satisfied by (v_0, v_2, v_3) . Propagating to F_3 via edge (v_0, v_3) , we require (v_3, v_0) , satisfied by (v_0, v_3, v_1) . All four faces receive consistent orientations, confirming the tetrahedron is orientable.

30.4.4 4. Pseudocode

30.4.5 5. Parameter Selection

- **Start Face:** Any face can be used as a starting point. For non-closed meshes, starting at a boundary can be helpful.
- **Components:** Orientation must be checked and performed independently for each connected component of the mesh.

Algorithm 83 Orient Mesh

```
1: function ORIENTMESH(mesh)
2:   oriented_faces  $\leftarrow \{\}$ 
3:   face_directions  $\leftarrow \{\}$ 
4:   for start_face  $\in$  mesh.faces do
5:     if start_face  $\in$  oriented_faces then
6:       continue
7:     end if
8:     queue  $\leftarrow [start\_face]$ 
9:     oriented_faces.add(start_face)
10:    while queue do
11:      current_face  $\leftarrow queue.pop(0)$ 
12:      for (neighbor_face, shared_edge)  $\in$  GetNeighbors(current_face, mesh) do
13:        if neighbor_face  $\notin$  oriented_faces then
14:          new_orientation  $\leftarrow$  MatchOrientation(current_face, neighbor_face, shared_edge)
15:          face_directions[neighbor_face]  $\leftarrow$  new_orientation
16:          oriented_faces.add(neighbor_face)
17:          queue.append(neighbor_face)
18:        else
19:          if  $\neg$ IsConsistent(current_face, neighbor_face, shared_edge) then
20:            return False, “Mesh is non-orientable (Möbius strip topology)”
21:          end if
22:        end if
23:      end for
24:    end while
25:  end for
26:  return True, face_directions
27: end function

28: function ISCONSISTENT(f1, f2, edge)
29:   u, v  $\leftarrow$  GetEdgeVertices(f1, edge)
30:   u', v'  $\leftarrow$  GetEdgeVertices(f2, edge)
31:   return (u = v')  $\wedge$  (v = u')
32: end function
```

30.4.6 6. Complexity

- **Time Complexity:** $O(F)$, where F is the number of faces, as each face is visited exactly once in the BFS.
- **Space Complexity:** $O(F)$ to store the face orientations and the BFS queue.

30.4.7 7. Usages

- **Rendering:** Backface culling and lighting calculations depend on correct, consistent normals.
- **Slicing for 3D Printing:** Determining what is “solid” vs “void” requires consistent orientations.
- **Surface Offsetting:** Offsetting a surface requires moving vertices in a consistent normal direction.
- **Fluid Simulation:** Pressure forces must be applied consistently along the surface normals.

30.4.8 8. Testing Methods and Edge Cases

- **Möbius Strip:** A classic non-orientable manifold; the algorithm should detect the inconsistency.
- **Klein Bottle:** A closed non-orientable surface (though it must self-intersect in 3D).
- **Watertight Sphere:** Orienting one face should correctly orient the entire sphere.
- **Disconnected Mesh:** Ensure the algorithm handles multiple separate objects in the same file.
- **Non-Manifold Mesh:** Orientability is generally defined for manifolds; non-manifold edges (shared by 3+ faces) make orientability ambiguous.

30.4.9 9. References

- Botsch, M., Kobbelt, L., Pauly, M., Alliez, P., & Lévy, B. [BKP⁺10] “Polygon Mesh Processing.” CRC Press.
- Guibas, L. J., & Stolfi, J. [GS85] “Primitives for the manipulation of general subdivisions.” ACM Transactions on Graphics.
- Wikipedia: Orientability.

30.5 Manifold Verification

30.5.1 1. Overview

Manifold verification is the process of checking whether a 3D surface mesh satisfies the topological constraints of a 2nd-order manifold. A manifold surface is one that “locally looks like a flat plane” at every point. Manifold meshes are highly desirable in geometric modeling because they have well-defined orientations, consistent normals, and are compatible with standard data structures like the half-edge mesh [GS85].

30.5.2 2. Definitions

Definition 30.18 (Edge Manifold). An edge is *edge-manifold* if it is shared by exactly two faces (for a closed surface) or one face (for a boundary edge).

Definition 30.19 (Vertex Manifold). A vertex is *vertex-manifold* if its surrounding faces form a single, connected “fan” or “disk.”

Definition 30.20 (Watertight). A closed manifold mesh with no holes: every edge has exactly two incident faces. Equivalently, the mesh has no boundary edges.

Definition 30.21 (Euler Characteristic). For a manifold mesh with V vertices, E edges, and F faces, the Euler characteristic is $\chi = V - E + F$. For a sphere-like surface, $\chi = 2$.

30.5.3 3. Theory

To verify that a mesh is a manifold, we check three primary conditions [BKP⁺10]:

1. **Edge Condition:** Every edge must be incident to either one or two faces. Edges with three or more incident faces (non-manifold edges) are topologically ambiguous.
2. **Vertex Condition:** For each vertex, the set of faces sharing that vertex must be edge-connected. If a vertex belongs to two separate “cones” that only touch at the vertex itself, it is non-manifold.
3. **Face Consistency:** For each edge, the orientation of its two incident faces must be compatible (e.g., if one face uses (u, v) , the other must use (v, u)).

Theorem 30.22 (Manifold Verification Correctness). *A mesh M is a 2-manifold if and only if it satisfies the edge condition, the vertex condition, and the face consistency condition simultaneously.*

Proof. ($\Rightarrow$) If M is a 2-manifold, then by definition every point has a neighborhood homeomorphic to a half-plane (boundary) or full plane (interior). This implies each edge has at most two incident faces (edge condition), each vertex has a single fan of faces (vertex condition), and the orientations are globally consistent (face consistency).

($\Leftarrow$) If all three conditions hold, the BFS orientation propagation succeeds globally (face consistency). The edge condition ensures no edge has more than two faces. The vertex condition ensures each vertex has a single connected neighborhood. Together, these guarantee that every point has the required local topological structure. $\square$

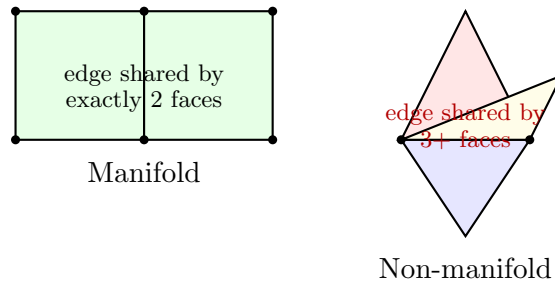


Figure 30.2: (Left) Manifold mesh: every edge is shared by exactly two faces. (Right) Non-manifold: an edge shared by three faces violates the manifold property.

Algorithm 84 Is Manifold

```

1: function ISMANIFOLD(mesh)
2:   edge_faces  $\leftarrow$  GetEdgeFaceMap(mesh)
3:   for (edge, faces)  $\in$  edge_faces.items() do
4:     if len(faces) > 2 then
5:       return False, “Non-manifold edge detected”
6:     end if
7:   end for
8:   for vertex  $\in$  mesh.vertices do
9:     neighbor_faces  $\leftarrow$  GetFacesSharingVertex(vertex, mesh)
10:    if  $\neg$ IsConnected(neighbor_faces, vertex) then
11:      return False, “Non-manifold vertex detected”
12:    end if
13:  end for
14:  if  $\neg$ HasConsistentOrientation(mesh) then
15:    return False, “Inconsistent face orientations”
16:  end if
17:  return True, “Mesh is manifold”
18: end function

19: function ISCONNECTED(faces, vertex)
20:   components  $\leftarrow$  FindFaceComponents(faces, vertex)
21:   return len(components) = 1
22: end function

```

30.5.4 4. Pseudocode**30.5.5** 5. Parameter Selection

- **Tolerance:** Floating-point precision should be considered when identifying “duplicate” vertices that should have been merged.
- **Boundary Handling:** A manifold with a boundary is still a manifold; the condition for boundary edges (1 face) and boundary vertices (faces form a single half-disk) must be handled.

30.5.6 6. Complexity

- **Time Complexity:** $O(F)$, where F is the number of faces, to build the edge-face map and check vertex connectivity.
- **Space Complexity:** $O(E + F)$ to store the connectivity information.

30.5.7 7. Usages

- **3D Printing:** Slicing software requires watertight, manifold meshes to correctly identify “inside” vs. “outside” volumes.
- **Physical Simulation:** Finite Element Method (FEM) and Fluid Dynamics simulations often fail on non-manifold geometry.
- **Mesh Boolean Operations:** Algorithms like Union/Intersection are much more robust on manifold meshes.

- **Surface Parameterization:** Algorithms like BFF or LSCM rely on a consistent half-edge structure, which requires a manifold mesh.

30.5.8 8. Testing Methods and Edge Cases

- **T-Junctions:** A vertex lying on an edge of another face (without sharing it) is geometrically non-manifold.
- **Pinch Points:** Two separate mesh components touching at a single vertex.
- **Self-Intersections:** A mesh can be topologically manifold while still being geometrically self-intersecting (this is usually checked separately).
- **Isolated Vertices:** Vertices with zero incident faces.
- **Inconsistent Normals:** A “Möbius strip” style mesh where orientations cannot be made consistent.

30.5.9 9. References

- Guibas, L. J., & Stolfi, J. [GS85] “Primitives for the manipulation of general subdivisions.” ACM Transactions on Graphics.
- Botsch, M., Kobbelt, L., Pauly, M., Alliez, P., & Lévy, B. [BKP⁺10] “Polygon Mesh Processing.” CRC Press.
- Wikipedia: Manifold.

30.6 Euler Characteristic

30.6.1 1. Overview

The Euler characteristic (χ) is a topological invariant, a number that describes a topological space’s shape or structure regardless of the way it is bent or twisted. For a 3D surface mesh (a polyhedral surface), it relates the number of vertices (V), edges (E), and faces (F). It is a fundamental tool for classifying surfaces and verifying the topological integrity of meshes [Eul58, Poi95].

30.6.2 2. Definitions

Definition 30.23 (Euler Formula). For a convex polyhedron (topologically equivalent to a sphere), the Euler formula states that $\chi = V - E + F = 2$.

Definition 30.24 (Genus). The genus g of a surface is the number of “handles” or “holes” through a surface (e.g., a torus has $g = 1$, a sphere has $g = 0$).

Definition 30.25 (Boundary Components). The number b of connected boundary loops in a mesh.

Definition 30.26 (Topological Invariant). A property that remains unchanged under continuous deformation (homeomorphism).

30.6.3 3. Theory

For a general connected orientable surface, the Euler characteristic is related to its genus and boundary count by the formula:

$$\chi = V - E + F = 2 - 2g - b$$

Theorem 30.27 (Euler–Poincaré Formula). *For a connected orientable 2-manifold with genus g and b boundary components, the Euler characteristic satisfies $\chi = V - E + F = 2 - 2g - b$. In particular, for a closed surface ($b = 0$), $\chi = 2 - 2g$.*

Proof. We prove this by induction on the genus g . For $g = 0$ (a sphere), the classical Euler formula gives $\chi = 2$. We establish the formula for $g = 0, b > 0$ by noting that each boundary component is obtained by removing a disk, which reduces χ by 1. Hence $\chi = 2 - b$.

For the inductive step, suppose the formula holds for genus g . To obtain a surface of genus $g+1$, we remove two disks from the surface and attach a cylinder (tube) between the resulting boundary circles. Removing two disks decreases χ by 2, and attaching the cylinder (which has $\chi = 0$) does not change χ . The genus increases by 1. Thus $\chi_{g+1} = \chi_g - 2 = (2 - 2g) - 2 = 2 - 2(g + 1)$, completing the induction. $\square$

This allows us to determine the global topology of a mesh simply by counting its elements:

- **Sphere:** $V - E + F = 2$ ($g = 0, b = 0$).
- **Disk:** $V - E + F = 1$ ($g = 0, b = 1$).
- **Torus:** $V - E + F = 0$ ($g = 1, b = 0$).
- **Cylinder:** $V - E + F = 0$ ($g = 0, b = 2$).

The Gauss–Bonnet theorem further relates the Euler characteristic to the total curvature of the surface: $\int_M K dA = 2\pi\chi$. In the discrete case, this means the sum of angle defects at all vertices is exactly $2\pi\chi$, where the angle defect at vertex v is defined as $\delta(v) = 2\pi - \sum_i \theta_i$, with θ_i being the angles of the faces incident to v .

Example 30.28 (Euler Characteristic Computations). 1. **Tetrahedron:** $V = 4, E = 6, F = 4$. Thus $\chi = 4 - 6 + 4 = 2$, confirming it is homeomorphic to a sphere ($g = 0$).

2. **Cube:** $V = 8, E = 12, F = 6$. Thus $\chi = 8 - 12 + 6 = 2$.

3. **Torus (mesh approximation):** A torus mesh with V vertices satisfies $V - E + F = 0$, regardless of resolution. For instance, a coarse torus with $V = 16, E = 32, F = 16$ gives $\chi = 0$.

4. **Two disjoint spheres:** $V = 8, E = 12, F = 8$ (two tetrahedra). $\chi = 8 - 12 + 8 = 4 = 2 \times 2$, consistent with the formula for $C = 2$ components.

30.6.4 4. Pseudocode

30.6.5 5. Parameter Selection

- **Element Counting:** Ensure that duplicate vertices or edges are correctly merged before counting, as they will artificially deflate/inflate χ .
- **Component Handling:** If the mesh has C disconnected components, the formula becomes $\chi = V - E + F = 2C - 2g - b$.

Algorithm 85 Calculate Euler Characteristic

```
1: function CALCULATEEULERCHARACTERISTIC(mesh)
2:    $V \leftarrow \text{len}(\text{mesh.vertices})$ 
3:    $F \leftarrow \text{len}(\text{mesh.faces})$ 
4:    $\text{edges} \leftarrow \{\}$ 
5:   for  $\text{face} \in \text{mesh.faces}$  do
6:     for  $i = 0 \rightarrow \text{len}(\text{face}) - 1$  do
7:        $v_1 \leftarrow \text{face}[i]$ 
8:        $v_2 \leftarrow \text{face}[(i + 1) \bmod \text{len}(\text{face})]$ 
9:        $\text{edges.add}(\text{tuple}(\text{sorted}((v_1, v_2))))$ 
10:    end for
11:  end for
12:   $E \leftarrow \text{len}(\text{edges})$ 
13:   $\chi \leftarrow V - E + F$ 
14:  return  $\chi$ 
15: end function
```

30.6.6 6. Complexity

- **Time Complexity:** $O(F)$, where F is the number of faces, to iterate through the mesh and identify unique edges.
- **Space Complexity:** $O(E)$ to store the set of unique edges.

30.6.7 7. Usages

- **Mesh Verification:** Detecting if a mesh has unexpected holes or disconnected parts.
- **Surface Classification:** Automatically determining if a scanned object is a “donut” (torus) or a “ball” (sphere).
- **Topology-Preserving Operations:** Ensuring that operations like mesh simplification or remeshing do not change the number of holes in the model.
- **Shape Matching:** Using χ as a fast, first-pass filter to distinguish between objects with different topologies.
- **Theoretical Physics:** Used in string theory and condensed matter physics to classify manifold structures.

30.6.8 8. Testing Methods and Edge Cases

- **Platonic Solids:** Verify that the Tetrahedron, Cube, Octahedron, Dodecahedron, and Icosahedron all yield $\chi = 2$.
- **Open Meshes:** A single triangle has $V = 3, E = 3, F = 1 \Rightarrow \chi = 1$ (a disk).
- **Disconnected Parts:** A mesh consisting of two separate spheres should have $\chi = 4$.
- **Non-Orientable Surfaces:** For a Möbius strip, $\chi = 0$ ($V - E + F = 0$, $g_{\text{non-orient}} = 1, b = 1$).
- **Non-Manifold Edges:** Ensure the edge counting logic correctly handles edges shared by 3+ faces (though the standard formula is defined for 2-manifolds).

30.6.9 9. References

- Euler, L. [Eul58] “Elementa doctrinae solidorum.” Novi Commentarii Academiae Scientiarum Petropolitanae.
- Poincaré, H. [Poi95] “Analysis situs.” Journal de l’École Polytechnique.
- Botsch, M., Kobbelt, L., Pauly, M., Alliez, P., & Lévy, B. [BKP⁺10] “Polygon Mesh Processing.” CRC Press.
- Wikipedia: Euler characteristic.

30.7 Node Degree and Valence

30.7.1 1. Overview

Node degree calculation is the process of counting the number of edges incident to each vertex (node) in a graph or mesh. In the context of 3D meshes, vertex degree (often called **valence**) is a key indicator of mesh quality and topological regularity. Vertices with unusual degrees (extraordinary vertices) can cause artifacts in subdivision and smoothing algorithms [BKP⁺10].

30.7.2 2. Definitions

Definition 30.29 (Valence (Degree)). The number of edges connected to a vertex v , denoted $\deg(v)$.

Definition 30.30 (Regular Vertex). A vertex with the “ideal” degree for its mesh type: degree 6 for triangle meshes, degree 4 for quad meshes.

Definition 30.31 (Extraordinary Vertex). A vertex with a degree other than the regular degree. Also called a singularity.

Definition 30.32 (Average Degree). The sum of all degrees divided by the number of vertices. For a closed triangle mesh, the average degree approaches 6 as $V \rightarrow \infty$.

30.7.3 3. Theory

Theorem 30.33 (Handshaking Lemma). *The sum of all vertex degrees in a mesh is equal to twice the number of edges: $\sum_{i=1}^V \deg(v_i) = 2E$.*

Proof. Each edge (u, v) contributes exactly 1 to $\deg(u)$ and 1 to $\deg(v)$. Summing over all edges, the total contribution to the degree sum is $2E$. $\square$

For a closed 2-manifold triangle mesh, the average degree is related to the Euler characteristic χ :

$$\bar{d} = 6 - \frac{6\chi}{V}$$

Theorem 30.34 (Average Degree of Triangle Mesh). *For a closed 2-manifold triangle mesh with V vertices, E edges, and Euler characteristic χ , the average vertex degree is $\bar{d} = 6 - 6\chi/V$.*

Proof. By the Handshaking Lemma [Die17], $\bar{d} = 2E/V$. For a triangle mesh, each face has 3 edges and each edge is shared by 2 faces, so $3F = 2E$, giving $F = 2E/3$. Substituting into Euler’s formula $\chi = V - E + F$:

$$\chi = V - E + \frac{2E}{3} = V - \frac{E}{3}$$

Hence $E = 3(V - \chi)$, and $\bar{d} = 2E/V = 6(V - \chi)/V = 6 - 6\chi/V$. $\square$

As the number of vertices $V \rightarrow \infty$, the average degree approaches 6.

In a Half-Edge [GS85] data structure, calculating the degree involves “circulating” around the vertex using the `next` and `twin` pointers until the starting half-edge is reached again.

Example 30.35 (Valence Distribution). Consider an icosahedron: $V = 12$, $E = 30$, $F = 20$. Every vertex has degree 5 (all vertices are regular for this polyhedron). The average degree is $\bar{d} = 2 \times 30/12 = 5$, which matches the formula $\bar{d} = 6 - 6(2)/12 = 5$. In a refined triangle mesh with genus 0 and many vertices, the average degree approaches 6, but the 12 original icosahedral vertices remain extraordinary with degree 5.

30.7.4 4. Pseudocode

Using an Adjacency List:

Algorithm 86 Calculate Vertex Degrees

```

1: function CALCULATEDEGREES(mesh)
2:   degrees  $\leftarrow$   $[0] \times \text{num\_vertices}$ 
3:   for  $(v_1, v_2) \in \text{mesh.edges}$  do
4:     degrees[ $v_1$ ]  $\leftarrow$  degrees[ $v_1$ ] + 1
5:     degrees[ $v_2$ ]  $\leftarrow$  degrees[ $v_2$ ] + 1
6:   end for
7:   return degrees
8: end function

```

Using a Half-Edge Structure:

Algorithm 87 Get Vertex Degree

```

1: function GETVERTEXDEGREE(mesh, v)
2:   count  $\leftarrow$  0
3:   start_he  $\leftarrow$  v.halfedge
4:   curr_he  $\leftarrow$  start_he
5:   loop
6:     count  $\leftarrow$  count + 1
7:     curr_he  $\leftarrow$  curr_he.twin.next
8:     if curr_he = start_he then
9:       break
10:    end if
11:  end loop
12:  return count
13: end function

```

30.7.5 5. Parameter Selection

- **Boundary Handling:** Vertices on the boundary of an open mesh naturally have lower degrees (typically 2 or 3). They should be flagged or handled separately in quality metrics.
- **Directed vs. Undirected:** For general graphs, degrees are split into **In-Degree** and **Out-Degree**. For standard surface meshes, edges are considered undirected.

30.7.6 6. Complexity

- **Time Complexity:** $O(E)$ to iterate through all edges, or $O(V \cdot \bar{d})$ to circulate around each vertex. Both are linear in the size of the mesh.

- **Space Complexity:** $O(V)$ to store the degree values.

30.7.7 7. Usages

- **Mesh Quality Analysis:** Identifying regions with poor connectivity that might cause simulation errors.
- **Subdivision Surfaces:** Stencils for Loop [Loo87] or Catmull-Clark [CC78] subdivision change based on the vertex degree to maintain smoothness.
- **Mesh Simplification:** Algorithms like Quadratic Error Metrics (QEM) often prioritize removing vertices based on their degree and geometric impact.
- **Network Science:** Identifying “hubs” (nodes with very high degrees) in social or infrastructure graphs.
- **Topology Verification:** Checking if a mesh is “regular” (e.g., all vertices have degree 6).

30.7.8 8. Testing Methods and Edge Cases

- **Simple Polyhedra:** Verify that all vertices of a cube have degree 3, and all vertices of an icosahedron have degree 5.
- **Regular Grid:** A flat triangle grid should have interior vertices of degree 6.
- **Boundary Vertices:** Test on an open square; corner vertices should have degree 2 and edge vertices degree 3.
- **Non-Manifold Vertices:** Verify the count when multiple “fans” of triangles meet at a single vertex.
- **Isolated Vertices:** Ensure vertices with zero edges are handled (degree 0).

30.7.9 9. References

- Diestel, R. [Die17] “Graph Theory.” Springer.
- Botsch, M., Kobbelt, L., Pauly, M., Alliez, P., & Lévy, B. [BKP⁺10] “Polygon Mesh Processing.” CRC Press.
- Wikipedia: Degree (graph theory).

30.8 Mesh Coloring

30.8.1 1. Overview

Mesh coloring is the process of assigning labels (colors) to the elements of a mesh (typically vertices or faces) such that no two adjacent elements share the same label. This is a direct application of graph coloring to the connectivity of a mesh. While it is often used for visual aesthetics, its primary technical purpose is to identify independent sets of elements that can be processed in parallel without race conditions.

30.8.2 2. Definitions

Definition 30.36 (*k*-Coloring). An assignment of k colors to vertices such that adjacent vertices have different colors.

Definition 30.37 (Vertex Graph). A graph where vertices of the mesh are nodes and edges of the mesh are graph edges.

Definition 30.38 (Dual Graph (Face Graph)). A graph where each face of the mesh is a node and edges connect faces that share an edge.

Definition 30.39 (Chromatic Number). The chromatic number $\chi(G)$ is the minimum number of colors needed to color a graph G .

30.8.3 3. Theory

For a 2D surface mesh (which is essentially a planar or surface-embedded graph), several key theorems apply:

Theorem 30.40 (Four-Color Theorem). *Any planar graph can be colored with at most 4 colors [WP67]. Since most surface meshes are locally planar, 4 to 6 colors are typically sufficient.*

Proof. The Four-Color Theorem was originally proved by Appel and Haken (1976) using computer-assisted case analysis, and later given a simpler (though still computer-assisted) proof by Robertson, Sanders, Seymour, and Thomas (1997). The key idea is that every planar graph has a vertex of degree at most 5, which allows an inductive argument: remove such a vertex, color the smaller graph by induction, then restore the vertex using one of the 4 colors not used by its at-most-5 neighbors (by the pigeonhole principle, at least one color is available among 4). The hard part is handling the finite set of unavoidable configurations. $\square$

Theorem 30.41 (Brooks' Theorem). *The chromatic number is at most the maximum vertex degree Δ , unless the graph is a complete graph or an odd cycle.*

The most common algorithm is the **Greedy Coloring** approach:

1. Iterate through vertices in a specific order.
2. For the current vertex, check the colors of its already-colored neighbors.
3. Assign the smallest available color that is not used by any neighbor.

Example 30.42 (Greedy Coloring Trace). Consider a triangle mesh with vertices v_0, v_1, v_2, v_3 forming two triangles (v_0, v_1, v_2) and (v_1, v_2, v_3) . Ordering by descending degree: v_1 (deg 4), v_2 (deg 4), v_0 (deg 2), v_3 (deg 2). Color v_1 with 0 (no colored neighbors). Color v_2 with 1 (neighbor v_1 uses 0). Color v_0 with 1 (neighbors v_1 uses 0, v_2 uses 1; but v_0 is only adjacent to v_1 and v_2). Wait, v_0 is adjacent to both v_1 and v_2 in triangle (v_0, v_1, v_2) , so its neighbors use colors $\{0, 1\}$; assign color 2. Color v_3 similarly with color 2 (neighbors v_1 and v_2 use $\{0, 1\}$). Total: 3 colors, matching the fact that the chromatic number of a triangle mesh is at most 4 by Theorem 30.40.

30.8.4 4. Pseudocode

30.8.5 5. Parameter Selection

- **Ordering Strategy:** Sorting vertices by degree (Largest First) often results in fewer total colors than a random ordering.
- **Saturation Degree (DSATUR):** A more advanced strategy that colors vertices with the most uniquely colored neighbors first.

Algorithm 88 Color Mesh

```

1: function COLORMESH(mesh, strategy = “GREEDY”)
2:   colors  $\leftarrow \{v : -1 \mid v \in \text{mesh.vertices}\}$ 
3:   ordered_vertices  $\leftarrow \text{SortByDegree}(\text{mesh.vertices}, \text{descending} = \text{True})$ 
4:   for v  $\in$  ordered_vertices do
5:     neighbor_colors  $\leftarrow \{\}$ 
6:     for neighbor  $\in$  GetNeighbors(v, mesh) do
7:       if colors[neighbor]  $\neq -1$  then
8:         neighbor_colors.add(colors[neighbor])
9:       end if
10:    end for
11:    color  $\leftarrow 0$ 
12:    while color  $\in$  neighbor_colors do
13:      color  $\leftarrow$  color + 1
14:    end while
15:    colors[v]  $\leftarrow$  color
16:  end for
17:  return colors
18: end function

```

30.8.6 6. Complexity

- **Time Complexity:** $O(V \cdot \Delta)$, where V is the number of vertices and Δ is the average vertex degree (typically $\Delta \approx 6$ for triangular meshes).
- **Space Complexity:** $O(V)$ to store the assigned color for each vertex.

30.8.7 7. Usages

- **Parallel Computing (GPGPU):** When performing operations like mesh smoothing (Mean Curvature Flow [GH86, Gra87]) on a GPU, vertices of the same color can be updated simultaneously because they don’t share data with each other.
- **Implicit Solvers:** Partitioning the Laplacian matrix into independent blocks for faster parallel solves.
- **Visualizing Mesh Structure:** Highlighting different regions or patches for debugging.
- **Texture Mapping:** Assigning different texture atlases to different parts of a mesh to avoid overlaps.

30.8.8 8. Testing Methods and Edge Cases

- **Adjacency Check:** After coloring, iterate through every edge (u, v) and verify that $\text{color}[u] \neq \text{color}[v]$.
- **Color Count:** Verify that the total number of colors used is reasonably small (e.g., < 10 for typical meshes).
- **Disconnected Components:** Ensure the algorithm handles multiple separate mesh objects correctly.
- **High-Valence Vertices:** Check performance and color count on meshes with “star” patterns (vertices with many neighbors).

- **Face Coloring:** Test by applying the same logic to the dual graph (coloring faces instead of vertices).

30.8.9 9. References

- Welsh, D. J. A., & Powell, M. B. [WP67] “An upper bound for the chromatic number of a graph and its application to timetabling problems.” The Computer Journal.
- Brélaz, D. (1979). “New methods to color the vertices of a graph.” Communications of the ACM.
- Wikipedia: Graph coloring.

30.9 Mesh Reordering

30.9.1 1. Overview

Mesh reordering is the process of permuting the indices of the vertices (and faces) of a mesh to improve the performance of numerical solvers and data access patterns. The most common objective is to minimize the **bandwidth** of the sparse matrices derived from the mesh (like the Laplacian). Reducing the bandwidth ensures that non-zero entries are clustered close to the main diagonal, leading to faster computations and more efficient CPU cache usage [CM69].

30.9.2 2. Definitions

Definition 30.43 (Bandwidth). For a matrix A , the bandwidth is the maximum value of $|i - j|$ such that $A_{ij} \neq 0$.

Definition 30.44 (Permutation). A mapping P that reassigns each vertex index i to a new index $j = P(i)$.

Definition 30.45 (Profile). The total number of zeros within the band of the matrix; smaller profiles are generally better.

Definition 30.46 (Adjacency Graph). The graph where vertices of the mesh are nodes and edges represent connections between them.

30.9.3 3. Theory

The most widely used reordering technique is the Reverse Cuthill–McKee (RCM) algorithm [CM69]. It is a refinement of a breadth-first search (BFS) that carefully chooses the starting node and the order of neighbors.

1. Find a “pseudo-peripheral” vertex (one that is far from other vertices, often by using a heuristic).
2. Perform a BFS starting from this vertex.
3. In each step of the BFS, sort the neighbors by their degree (number of connections) in ascending order.
4. Once all vertices are visited, reverse the resulting permutation. Reversing is key because it significantly reduces the matrix profile compared to the standard CM order.

Example 30.47 (RCM Reordering). Consider a path graph $v_0 - v_1 - v_2 - v_3 - v_4$. Starting BFS from the peripheral vertex v_0 : the permutation is $[v_0, v_1, v_2, v_3, v_4]$. Reversing gives $[v_4, v_3, v_2, v_1, v_0]$. The bandwidth of the adjacency matrix is 1 in both cases (since it is already optimal for a path). However, for a more complex mesh like a 2D grid, RCM produces a bandwidth of $O(\sqrt{N})$ rather than $O(N)$.

30.9.4 4. Pseudocode

Algorithm 89 RCM Reorder

```

1: function RCMREORDER(mesh)
2:    $v \leftarrow \text{FindPseudoPeripheralVertex}(\textit{mesh})$ 
3:    $R \leftarrow [v]$ 
4:    $\textit{visited} \leftarrow \{v\}$ 
5:   for  $\textit{current\_v} \in R$  do
6:      $\textit{neighbors} \leftarrow \text{GetNeighbors}(\textit{current\_v}, \textit{mesh})$ 
7:      $\textit{sorted\_neighbors} \leftarrow \text{sorted}([n \mid n \in \textit{neighbors} \text{ if } n \notin \textit{visited}], \textit{key} = \lambda n : \text{len}(\text{GetNeighbors}(n, \textit{mesh})))$ 
8:     for  $n \in \textit{sorted\_neighbors}$  do
9:        $R.\text{append}(n)$ 
10:       $\textit{visited}.\text{add}(n)$ 
11:    end for
12:  end for
13:  return  $\text{list}(\text{reversed}(R))$ 
14: end function

```

30.9.5 5. Parameter Selection

- **Starting Vertex:** The choice of the first vertex is critical. A bad starting point leads to high bandwidth. Heuristics like the Gibbs–Poole–Stockmeyer (GPS) algorithm are often used to find a good starting point.
- **Metric:** While bandwidth reduction is the most common goal, other orderings (like Nested Dissection) are better for parallel solvers or multifrontal methods.

30.9.6 6. Complexity

- **Time Complexity:** $O(V \cdot \Delta \log \Delta)$, where V is vertices and Δ is average degree. Since $\Delta \approx 6$ for meshes, this is essentially $O(V)$.
- **Space Complexity:** $O(V + E)$ to store the adjacency list and the visit status.

30.9.7 7. Usages

- **Sparse Linear Solvers:** Essential preprocessing step for Cholesky or LU factorization of Laplacian systems.
- **FEM Simulations:** Reducing memory access time and improving cache hit rates in structural or fluid simulations.
- **Data Compression:** Reordered meshes often have more predictable connectivity, allowing for better compression (e.g., in the PLY or OBJ formats).
- **Rendering:** Improving the vertex cache hit rate during triangle strip or fan rendering.

30.9.8 8. Testing Methods and Edge Cases

- **Bandwidth Calculation:** Compare the bandwidth of the Laplacian matrix before and after reordering. A successful RCM should show a significant reduction.
- **Graph with Disconnected Components:** The algorithm must be applied to each component separately and concatenated.
- **Highly Regular Mesh:** A perfect grid should be reordered into a diagonal-like pattern.
- **Very Large Mesh:** Ensure the algorithm handles millions of vertices without excessive memory overhead.
- **Degenerate Mesh:** Test on meshes with very high-valence vertices (where bandwidth reduction is harder).

30.9.9 9. References

- Cuthill, E., & McKee, J. [CM69] “Reducing the bandwidth of sparse symmetric matrices.” ACM National Conference.
- George, A. (1971). “Computer implementation of the finite element method.” Stanford Technical Report.
- Liu, W. H., & Sherman, A. H. (1976). “Comparative analysis of the Cuthill–McKee and the reverse Cuthill–McKee algorithms for sparse matrices.” SIAM Journal on Numerical Analysis.
- Wikipedia: Cuthill–McKee algorithm.

30.10 Mesh Rigidity

30.10.1 1. Overview

Mesh rigidity is the study of whether a framework of vertices and edges (a mesh) can be continuously deformed while preserving the lengths of all its edges. A mesh is **rigid** if the only possible motions are rigid body motions (translation and rotation). If it can be deformed in other ways, it is **flexible**. Rigidity analysis is critical for structural engineering, robotics, and molecular biology.

30.10.2 2. Definitions

Definition 30.48 (Bar-and-Joint Framework). A graph where edges represent rigid bars and vertices represent flexible joints.

Definition 30.49 (Rigidity Matrix). A matrix R that relates the velocities of vertices to the rates of change of edge lengths. For each edge (i, j) , the corresponding row contains $(P_i - P_j)$ at columns $3i$ through $3i + 2$ and $(P_j - P_i)$ at columns $3j$ through $3j + 2$.

Definition 30.50 (Infinitesimal Rigidity). A linear approximation where a mesh is rigid if the only vertex velocities that preserve edge lengths (to first order) are rigid body motions.

Definition 30.51 (Degrees of Freedom). The dimension of the kernel of the rigidity matrix. For a rigid 3D framework, $\text{DOF} = 6$ (3 translation + 3 rotation).

30.10.3 3. Theory

The rigidity of a mesh is determined by the **Rigidity Matrix** R . For each edge (i, j) , there is a row in R containing the vector $(P_i - P_j)$ at columns corresponding to P_i and $(P_j - P_i)$ at columns for P_j .

A framework is infinitesimally rigid if the rank of R is $3V - 6$ (in 3D) or $2V - 3$ (in 2D).

- If $\text{rank}(R) < 3V - 6$, the mesh has internal mechanisms (it is flexible).
- If $\text{rank}(R) = 3V - 6$, the mesh is rigid.
- If the number of edges $E > 3V - 6$, the mesh is **redundantly rigid** or over-constrained.

Theorem 30.52 (Maxwell's Counting Rule / Laman's Theorem in 2D). *A graph $G = (V, E)$ in 2D is generically rigid if and only if $E = 2V - 3$ and for every subgraph $H = (V', E')$ with $|V'| \geq 2$, $E' \leq 2|V'| - 3$.*

Proof. The necessity direction is straightforward: if $E > 2|V'| - 3$ for some subgraph, then that subgraph has too few constraints relative to its vertices, so it must be flexible. The sufficiency direction (the harder direction) was proved by Laman [Lam70] using a combinatorial characterization based on matroid theory. The key insight is that the generic rigidity matroid on a graph G coincides with the sparsity matroid defined by the counting condition, which can be checked efficiently using the pebble game algorithm. $\square$

Maxwell's Counting Rule provides a combinatorial necessary condition: a graph must have at least $3V - 6$ edges to be rigid in 3D, but this is not sufficient (as the edges might be poorly distributed).

Example 30.53 (Rigidity Analysis). 1. **Tetrahedron:** $V = 4$, $E = 6$, $3V - 6 = 6$. The rigidity matrix has rank 6, so the tetrahedron is rigid. It is the simplest rigid 3D structure.

2. **Square with no diagonal:** $V = 4$, $E = 4$, $3V - 6 = 6$. Since $E < 3V - 6$, the square is under-constrained and flexible (it can be deformed into a rhombus). Adding one diagonal gives $E = 5 < 6$, still flexible. Adding both diagonals gives $E = 6 = 3V - 6$, and the resulting framework is rigid.

3. **Icosahedron:** $V = 12$, $E = 30$, $3V - 6 = 30$. Exactly the right count, and the icosahedron is rigid.

30.10.4 4. Pseudocode

30.10.5 5. Parameter Selection

- **Numerical Tolerance:** Since rank calculation is sensitive to noise, a threshold based on the machine epsilon or the smallest non-zero singular value must be used.
- **Generic vs. Specific Position:** A mesh might be rigid in general but flexible in a specific degenerate configuration (e.g., all vertices on a line). Generic rigidity depends only on the graph topology.

30.10.6 6. Complexity

- **Matrix Construction:** $O(E)$.
- **Rank Calculation:** $O(E \cdot V^2)$ using SVD. For large meshes, sparse solvers or **Pebble Game** algorithms (for combinatorial rigidity) are used.

Algorithm 90 Analyze Mesh Rigidity

```
1: function ANALYZEMESHRIGIDITY(mesh)
2:    $V \leftarrow \text{len}(\text{mesh.vertices})$ 
3:    $E \leftarrow \text{len}(\text{mesh.edges})$ 
4:   if  $E < 3 \cdot V - 6$  then
5:     return “Flexible (under-constrained)”
6:   end if
7:    $R \leftarrow \text{SparseMatrix}(E, 3 \cdot V)$ 
8:   for (row_idx, edge)  $\in \text{enumerate}(\text{mesh.edges})$  do
9:      $i, j \leftarrow \text{edge}$ 
10:     $p_i \leftarrow \text{mesh.vertices}[i]$ 
11:     $p_j \leftarrow \text{mesh.vertices}[j]$ 
12:     $\text{diff} \leftarrow p_i - p_j$ 
13:     $R[\text{row\_idx}, 3i : 3i + 3] \leftarrow \text{diff}$ 
14:     $R[\text{row\_idx}, 3j : 3j + 3] \leftarrow -\text{diff}$ 
15:   end for
16:    $\text{rank} \leftarrow \text{CalculateRank}(R)$ 
17:    $\text{dof} \leftarrow 3 \cdot V - \text{rank}$ 
18:   if  $\text{dof} = 6$  then
19:     return “Rigid”
20:   else
21:     return “Flexible with  $\text{dof} - 6$  internal mechanisms”
22:   end if
23: end function
```

30.10.7 7. Usages

- **Structural Engineering:** Verifying that a bridge or building truss is stable and won’t collapse.
- **Robotics:** Analyzing the workspace and mobility of parallel manipulators.
- **Molecular Biology:** Studying the flexibility of protein structures to understand how they bind to other molecules.
- **Sensor Networks:** Determining if a set of distance measurements is sufficient to uniquely locate all sensors (Localization).
- **Computer Graphics:** Constraining mesh deformations to be “as-rigid-as-possible” (ARAP) for realistic animation.

30.10.8 8. Testing Methods and Edge Cases

- **Tetrahedron:** The simplest rigid 3D structure ($V = 4, E = 6, 3V - 6 = 6$).
- **Square:** A 2D square with 4 edges is flexible ($V = 4, E = 4, 2V - 3 = 5$ required); adding a diagonal makes it rigid.
- **Collinear Vertices:** Verify that the algorithm detects flexibility when vertices are aligned, even if the graph is combinatorially rigid.
- **Disconnected Mesh:** The rank check should reflect the sum of rigid body motions for each component (6 DOF per component).
- **Over-constrained Mesh:** Ensure the algorithm handles $E > 3V - 6$ correctly.

30.10.9 9. References

- Asimow, L., & Roth, B. (1978). “The rigidity of graphs.” Transactions of the American Mathematical Society.
- Laman, G. [Lam70] “On graphs and rigidity of plane skeletal structures.” Journal of Engineering Mathematics.
- Connelly, R. (1980). “The rigidity of polyhedra.” SIAM Journal on Mathematical Analysis.
- Wikipedia: Structural rigidity.

30.11 Mesh Decimation via Quadric Error Metrics

30.11.1 1. Overview

Mesh decimation is the process of reducing the number of faces (and vertices) in a triangle mesh while preserving its geometric shape as closely as possible. This is a fundamental operation in computer graphics and geometric computing. High-resolution meshes captured by 3D scanners or generated by subdivision surfaces often contain millions of triangles, far more than necessary for real-time rendering, transmission, or simulation. Mesh decimation enables Level-of-Detail (LOD) management, where coarser versions of a model are used when it occupies a small portion of the screen, and finer versions when it is viewed up close. Other critical applications include reducing the cost of Finite Element Method (FEM) simulations, accelerating ray tracing, and lowering memory and bandwidth requirements for networked 3D applications.

The Quadric Error Metric (QEM) method, introduced by Garland and Heckbert [GH97], is one of the most widely used decimation algorithms. It proceeds by iteratively collapsing edges, where each collapse merges two vertices into one. The key insight is that the geometric error introduced by an edge collapse can be efficiently approximated using 4×4 symmetric matrices called *quadrics*, enabling near-optimal vertex placement at each step.

30.11.2 2. Definitions

Definition 30.54 (Edge Collapse). An edge collapse is an operation that merges two adjacent vertices u and v into a single vertex $\bar{v}$, removing the edge (u, v) and all faces incident to it. Formally, the map $u \mapsto \bar{v}$, $v \mapsto \bar{v}$ is applied to all faces, and degenerate faces (those with fewer than three distinct vertices) are discarded.

Definition 30.55 (Quadric Error Metric). Given a vertex v , its quadric matrix Q_v is a 4×4 symmetric matrix that summarizes the squared distances from v to a set of planes. For a point $\bar{v}$ in homogeneous coordinates $\bar{v}_h = (\bar{v}_x, \bar{v}_y, \bar{v}_z, 1)^\top$, the quadric error is $\Delta(\bar{v}) = \bar{v}_h^\top Q_v \bar{v}_h$. The quadric for a single plane $\mathbf{p} = (a, b, c, d)^\top$ (where $a^2 + b^2 + c^2 = 1$) is $K_{\mathbf{p}} = \mathbf{p} \mathbf{p}^\top$.

Definition 30.56 (Link Condition). The link of a vertex v , denoted $\text{Link}(v)$, is the set of edges connecting the neighbors of v that are not incident to v itself. An edge (u, v) satisfies the link condition if $\text{Link}(u) \cap \text{Link}(v) \subseteq \text{Link}(u, v)$, where $\text{Link}(u, v)$ is the set of edges in the link of both u and v that are not the edge (u, v) itself. This condition ensures that the local topology is consistent after the collapse.

Definition 30.57 (Manifold Preservation). A decimation step *preserves manifoldness* if, after the edge collapse, every edge remains incident to at most two faces and every vertex retains a single connected fan of incident faces.

30.11.3 3. Theory

The QEM algorithm assigns each vertex v a 4×4 symmetric matrix Q_v that encodes the sum of squared distances from v to the planes of all faces incident to v .

Initial Quadric Computation

For each face with unit outward normal $\mathbf{n} = (n_x, n_y, n_z)^\top$ and offset $d = -\mathbf{n} \cdot \mathbf{v}_0$ (where $\mathbf{v}_0$ is any vertex of the face), define the plane vector $\mathbf{p} = (n_x, n_y, n_z, d)^\top$. The fundamental quadric of this face is:

$$K_{\mathbf{p}} = \mathbf{p} \mathbf{p}^\top = \begin{pmatrix} a^2 & ab & ac & ad \\ ab & b^2 & bc & bd \\ ac & bc & c^2 & cd \\ ad & bd & cd & d^2 \end{pmatrix}$$

The quadric of a vertex v is the sum of the fundamental quadrics of all faces incident to v :

$$Q_v = \sum_{\mathbf{p} \in \text{Faces}(v)} K_{\mathbf{p}}$$

Boundary Penalty Quadrics

Mesh boundaries require special treatment to prevent edges from collapsing inward and creating visual artifacts or topological anomalies. For each boundary edge (u, v) , a boundary penalty quadric is constructed as follows. Let $\mathbf{f}$ be the face incident to (u, v) with normal $\mathbf{n}_f$, and let $\mathbf{e}$ be the unit direction along (u, v) . The boundary constraint plane has normal:

$$\mathbf{n}_b = \frac{\mathbf{e} \times \mathbf{n}_f}{\|\mathbf{e} \times \mathbf{n}_f\|}$$

This plane is perpendicular to both the face normal and the edge direction, thus penalizing movement that would fold the boundary. The boundary penalty quadric $K_b = w_b \mathbf{p}_b \mathbf{p}_b^\top$ (with a large weight w_b , typically 10^3) is added to the quadrics of both u and v .

Optimal Vertex Placement

For an edge (u, v) , the combined quadric is $Q = Q_u + Q_v$. The optimal collapse position $\bar{v}$ minimizes the quadric error:

$$\bar{v} = \arg \min_{\mathbf{x}} \mathbf{x}_h^\top Q \mathbf{x}_h$$

where $\mathbf{x}_h = (x, y, z, 1)^\top$. If Q restricted to the upper-left 3×3 block is invertible, the minimum is obtained by solving the linear system $Q_{3 \times 3} \bar{v} = -\mathbf{q}$, where $\mathbf{q}$ is the upper-right 3×1 block of Q . In practice, a midpoint heuristic $\bar{v} = (u + v)/2$ is often used as a robust fallback, and the actual error at this position is evaluated via $\Delta(\bar{v}) = \bar{v}_h^\top Q \bar{v}_h$.

Edge Collapse Priority Queue

All edges are inserted into a min-heap keyed by their collapse cost $\Delta(\bar{v})$. At each step, the edge with the smallest cost is popped. Stale entries (where one of the endpoints has already been collapsed) are discarded. This greedy strategy ensures that the least disruptive collapses are performed first.

Link Condition Check

Before collapsing (u, v) , the algorithm verifies that the link condition holds. A simplified check ensures that u and v share at most two faces; if they share more than two, the collapse could create a non-manifold edge. If the link condition is violated, the edge is skipped.

Theorem 30.58 (Manifold Preservation Under Link-Condition-Satisfying Collapse). *Let M be a 2-manifold triangle mesh (with or without boundary). If an edge collapse $(u, v) \rightarrow \bar{v}$ satisfies the link condition and neither u nor v is itself the result of a previous collapse into a non-adjacent vertex, then the resulting mesh M' remains a 2-manifold.*

Proof. We verify that the three manifold conditions (from Theorem 30.22) hold after the collapse.

Edge condition: Before the collapse, every edge in M is incident to at most two faces. The collapse removes all faces incident to (u, v) and replaces v with u in all remaining faces. The link condition guarantees that no new non-manifold edge is created: if an edge (u, w) existed before the collapse, the faces incident to (u, w) are either faces already incident to u (unchanged) or faces that were incident to v and have been redirected to u . Since $\text{Link}(u) \cap \text{Link}(v) \subseteq \text{Link}(u, v)$, the redirected faces do not create a third face incident to any edge.

Vertex condition: For any vertex $w \neq u, v$, its incident faces are unchanged. For u , the incident faces after the collapse are: (i) the original faces of u that were not degenerated, plus (ii) the faces of v redirected to u that were not degenerated. The link condition ensures these two sets of faces form a single connected fan around u . Specifically, the redirected faces share edges with the original faces of u precisely through $\text{Link}(u) \cap \text{Link}(v)$, which is a connected subset of u 's link.

Face consistency: Since the mesh M was orientable before the collapse, and the collapse only relabels vertex indices without reversing any face winding, the orientations remain consistent. The degenerate faces (those reduced to fewer than three distinct vertices) are simply removed, which does not affect consistency.

Therefore, M' satisfies all three conditions and is a 2-manifold. $\square$

Example 30.59 (Quadric Computation for a Simple Vertex). Consider a vertex v shared by two faces:

- Face F_1 with vertices $(0, 0, 0)$, $(1, 0, 0)$, $(0, 1, 0)$. The outward normal is $\mathbf{n}_1 = (0, 0, 1)$, and with $\mathbf{v}_0 = (0, 0, 0)$, the plane is $\mathbf{p}_1 = (0, 0, 1, 0)^\top$.
- Face F_2 with vertices $(0, 0, 0)$, $(0, 1, 0)$, $(0, 0, 1)$. The outward normal is $\mathbf{n}_2 = (1, 0, 0)$, and with $\mathbf{v}_0 = (0, 0, 0)$, the plane is $\mathbf{p}_2 = (1, 0, 0, 0)^\top$.

The fundamental quadrics are:

$$K_1 = \mathbf{p}_1 \mathbf{p}_1^\top = \begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix}, \quad K_2 = \mathbf{p}_2 \mathbf{p}_2^\top = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

The quadric for v is $Q_v = K_1 + K_2 = \text{diag}(1, 0, 1, 0)$.

Now consider an edge (v, w) where Q_w has been computed similarly. The combined quadric is $Q = Q_v + Q_w$. Evaluating the error at the midpoint $\bar{v} = (v + w)/2$ gives $\Delta(\bar{v}) = \bar{v}_h^\top Q \bar{v}_h$, which determines the collapse priority.

30.11.4 4. Pseudocode

30.11.5 5. Parameter Selection

- **Target Face Count:** The primary user parameter. Setting this too low will cause severe geometric distortion; a reasonable starting point is 10%–20% of the original face count.

Algorithm 91 Mesh Decimation via QEM

```

1: function DECIMATE(mesh, target_faces)
2:   if  $|mesh.faces| \leq target\_faces$  then
3:     return mesh
4:   end if
5:   vertices  $\leftarrow$  ExtractPositions(mesh)
6:   faces  $\leftarrow$  BuildFaceMap(mesh)
7:    $Q \leftarrow$  array of  $|V|$  zero  $4 \times 4$  matrices
8:                                      $\triangleright$  Compute initial quadrics from face planes
9:   for  $f \in$  faces do
10:     $\mathbf{n}, d \leftarrow$  ComputePlane( $f$ )
11:     $\mathbf{p} \leftarrow (\mathbf{n}_x, \mathbf{n}_y, \mathbf{n}_z, d)^\top$ 
12:     $K \leftarrow \mathbf{p} \mathbf{p}^\top$ 
13:    for  $v_i \in f$  do
14:       $Q[v_i] \leftarrow Q[v_i] + K$ 
15:    end for
16:  end for
17:                                      $\triangleright$  Add boundary penalty quadrics
18:  edge_faces  $\leftarrow$  CountEdgeFaces(faces)
19:  for  $(u, v) \in$  edge_faces where count = 1 do
20:     $\mathbf{n}_b \leftarrow$  BoundaryNormal( $u, v, f$ )
21:     $K_b \leftarrow w_b \cdot \mathbf{p}_b \mathbf{p}_b^\top$ 
22:     $Q[u] \leftarrow Q[u] + K_b$ 
23:     $Q[v] \leftarrow Q[v] + K_b$ 
24:  end for
25:                                      $\triangleright$  Build edge priority queue
26:  heap  $\leftarrow \emptyset$                                       $\triangleright$  min-heap of  $(cost, u, v)$ 
27:  for  $(u, v) \in$  all edges do
28:     $cost \leftarrow$  ComputeEdgeCost( $u, v, Q$ )
29:    HeapPush(heap,  $(cost, u, v)$ )
30:  end for
31:                                      $\triangleright$  Iteratively collapse edges
32:  collapsed  $\leftarrow \{\}$ 
33:  removed  $\leftarrow 0$ 
34:  while heap  $\neq \emptyset \wedge$  removed  $< |F| - target\_faces$  do
35:     $cost, u, v \leftarrow$  HeapPop(heap)
36:    if  $u \in$  collapsed or  $v \in$  collapsed then
37:      continue
38:    end if
39:    if  $|Faces(u) \cap Faces(v)| > 2$  then
40:      continue                                      $\triangleright$  Link condition violated
41:    end if
42:     $\bar{v} \leftarrow$  OptimalPosition( $u, v, Q$ )
43:    vertices[ $u$ ]  $\leftarrow \bar{v}$ 
44:     $Q[u] \leftarrow Q[u] + Q[v]$ 
45:    collapsed[ $v$ ]  $\leftarrow u$ 
46:    for  $f \in Faces(v)$  do
47:      new_f  $\leftarrow$  replace  $v$  with  $u$  in  $f$ 
48:      if  $|new\_f| < 3$  then
49:        RemoveFace(faces,  $f$ )
50:        removed  $\leftarrow$  removed + 1
51:      else
52:        faces[ $f$ ]  $\leftarrow$  new_f
53:      end if
54:    end for
55:  end while
56:  return ReconstructMesh(vertices, faces, collapsed)

```


- **Boundary Penalty Weight:** A large weight ($w_b \approx 10^3$) prevents boundary edges from collapsing inward. Increasing it further preserves boundary shape more aggressively at the cost of fewer possible collapses.
- **Optimal Position Strategy:** The midpoint heuristic is robust but suboptimal. Solving the 3×3 linear system for the true minimum yields better geometric fidelity but may place the new vertex far from both originals, which can cause crossing faces if the mesh has thin features.
- **Link Condition Strictness:** The simplified check ($|\text{Faces}(u) \cap \text{Faces}(v)| \leq 2$) is fast but conservative. A full link condition check using the link graph is more permissive but more expensive.

30.11.6 6. Complexity

- **Initial Quadric Computation:** $O(F)$, where F is the number of faces, since each face contributes a 4×4 outer product to each of its three vertices.
- **Edge Queue Construction:** $O(E \log E)$ to insert all E edges into the min-heap.
- **Decimation Loop:** Each iteration pops one heap entry and may push updated entries for neighboring edges. With lazy deletion (stale entries are discarded), each edge is processed at most a constant number of times. The total number of iterations is at most $F - \text{target_faces}$. Each iteration performs $O(1)$ quadric additions and a constant number of face updates. The overall cost of the loop is $O(n \log n)$, where $n = |V| + |F|$.
- **Overall:** $O(n \log n)$.

30.11.7 7. Usages

- **Level of Detail (LOD):** Generating progressively simpler versions of a mesh for real-time rendering; the GPU selects the appropriate resolution based on screen-space coverage.
- **3D Printing:** Reducing mesh complexity to fit within file size limits or to speed up slicing.
- **Finite Element Analysis:** Creating coarser meshes for faster preliminary simulations while retaining geometric accuracy in critical regions.
- **Networked Graphics:** Compressing meshes for transmission over bandwidth-limited connections (e.g., mobile AR/VR).
- **Collision Detection:** Using simplified bounding meshes for broad-phase collision queries before refining with the full mesh.
- **Game Engines:** Dynamically adjusting mesh detail based on GPU load and frame rate targets.

30.11.8 8. Testing Methods and Edge Cases

- **Single Triangle:** The simplest mesh; no edge should be collapsed if the target face count equals 1.
- **Tetrahedron:** A watertight mesh with 4 faces. Collapsing any edge removes exactly one face, which should be verified by checking the output face count.
- **Boundary-Only Mesh** (e.g., a single triangle): Ensure that boundary penalty quadrics are applied and that the boundary does not collapse inward.

- **Non-Manifold Input:** Edges shared by 3+ faces should be detected by the link condition check and skipped.
- **Already at Target:** If the input face count is $\leq target_faces$, the algorithm must return the mesh unmodified.
- **Degenerate Faces:** After a collapse, faces with fewer than three distinct vertices must be removed and counted toward the reduction budget.
- **Disconnected Components:** The algorithm should work correctly on meshes with multiple disconnected parts.
- **Large Reduction Factor:** Reducing a mesh to 1% of its original face count should still produce a topologically valid (manifold) mesh, even if geometric fidelity is low.

30.11.9 9. References

- Garland, M., & Heckbert, P. S. [GH97] “Surface simplification using quadric error metrics.” *Proceedings of SIGGRAPH 1997*, 209–216.
- Garland, M. (1999). “Quadric-based polygonal surface simplification.” Ph.D. dissertation, Carnegie Mellon University.
- Hoppe, H. (1999). “New quadric metric for simplifying meshes with appearance attributes.” *Proceedings of Visualization 1999*.
- Botsch, M., Kobbelt, L., Pauly, M., Alliez, P., & Lévy, B. [BKP⁺10] “Polygon Mesh Processing.” CRC Press.
- Luebke, D., Erikson, C. (1997). “View-dependent simplification of arbitrary polygonal environments.” *Proceedings of SIGGRAPH 1997*.

30.12 Spectral Geometry and Shape Analysis

30.13 Shape Spectra

30.13.1 Overview

The spectrum of a shape refers to the set of eigenvalues of its Laplace–Beltrami operator. In geometric modeling, these eigenvalues (and their corresponding eigenfunctions) act as a “geometric DNA” or “fingerprint” that describes the intrinsic shape of a manifold. Shape space spectra are used for shape matching, classification, and analysis because they are invariant under isometric transformations (bending without stretching). The field is often summarized by the question posed by Mark Kac: “Can one hear the shape of a drum?” [Kac66].

30.13.2 Definitions

Definition 30.60 (Laplace–Beltrami Operator). The Laplace–Beltrami operator Δ_M on a Riemannian manifold (M, g) is the generalization of the Laplacian to curved spaces. In local coordinates, $\Delta_M f = \frac{1}{\sqrt{|g|}} \partial_i \left(\sqrt{|g|} g^{ij} \partial_j f \right)$, where g_{ij} is the metric tensor and $|g|$ its determinant.

Definition 30.61 (Shape Spectrum). The shape spectrum is the non-decreasing sequence of eigenvalues $0 = \lambda_0 \leq \lambda_1 \leq \lambda_2 \leq \dots$ satisfying $\Delta_M \phi_k = \lambda_k \phi_k$ on a closed compact manifold M of area A . The eigenvalues are also called the Dirichlet spectrum.

Definition 30.62 (Shape-DNA). Shape-DNA is a shape descriptor formed by the first k eigenvalues of the Laplace–Beltrami operator, typically normalized by the total area $\lambda_k \cdot A$ to enable comparison across differently-sized shapes [RWP06].

Definition 30.63 (Heat Kernel). The heat kernel $K_t(x, y)$ on a manifold M is the fundamental solution to the heat equation $\partial_t u = \Delta_M u$ with initial condition $u(0, x) = \delta(x)$. It admits the spectral expansion $K_t(x, y) = \sum_{k=0}^{\infty} e^{-\lambda_k t} \phi_k(x) \phi_k(y)$.

30.13.3 Theory

The Laplacian spectrum encodes information about the area, perimeter, and topology of a shape.

1. **Weyl’s Law:** The growth rate of eigenvalues relates to the volume (area) of the shape: $\lambda_n \sim \frac{4\pi n}{Area}$ as $n \rightarrow \infty$ for a 2D domain.
2. **Isometry Invariance:** Two isometric shapes have identical spectra. However, the converse is not always true (there exist non-isometric “isospectral” shapes [Kac66]).
3. **Fundamental Tone:** λ_1 (the Fiedler value) relates to the “width” or “length” of the shape and is used for spectral clustering and graph partitioning.

The spectrum is computed by discretizing the Laplacian on a mesh using **cotangent weights** and solving a generalized eigenvalue problem: $L\Phi = \Lambda M\Phi$, where L is the stiffness matrix and M is the mass matrix.

Definition 30.64 (Cotangent Laplacian). The cotangent Laplacian discretizes Δ_M on a triangle mesh. For a vertex i with one-ring neighbors $\{j\}$, the stiffness matrix entry is $L_{ij} = -\frac{1}{2}(\cot \alpha_{ij} + \cot \beta_{ij})$ for each edge (i, j) , where α_{ij} and β_{ij} are the angles opposite to edge (i, j) in the two adjacent triangles. The diagonal is $L_{ii} = -\sum_{j \neq i} L_{ij}$.

Theorem 30.65 (Weyl’s Asymptotic Law). *For a bounded domain $\Omega \subset \mathbb{R}^2$ with area $|\Omega|$, the eigenvalues of the Dirichlet Laplacian satisfy*

$$\lambda_n \sim \frac{4\pi n}{|\Omega|} \quad \text{as } n \rightarrow \infty.$$

Proof. The proof uses the phase-space counting argument (Weyl’s original method). The number of eigenvalues $\leq \lambda$ equals the number of lattice points (k_1, k_2) satisfying $\frac{\pi^2 k_1^2}{L_1^2} + \frac{\pi^2 k_2^2}{L_2^2} \leq \lambda$ for a rectangular domain. The area of this ellipse in (k_1, k_2) -space is $\frac{|\Omega|}{\pi^2} \cdot \pi \lambda = \frac{|\Omega| \lambda}{\pi}$, giving $N(\lambda) \sim \frac{|\Omega|}{4\pi} \lambda$. Inverting gives $\lambda_n \sim \frac{4\pi n}{|\Omega|}$. The result extends to arbitrary domains via a variational argument (comparing with inscribed/inscribed rectangles). $\square$

Remark 30.66. Weyl’s law Theorem 30.65 shows that the high-frequency part of the spectrum is determined entirely by the area, while lower eigenvalues encode finer geometric information (perimeter, topology, curvature). This motivates using only the first $k \ll N$ eigenvalues as shape descriptors.

30.13.4 Algorithm

Example 30.67 (Shape-DNA of a Unit Square). For a unit square $\Omega = [0, 1]^2$ with Dirichlet boundary conditions, the eigenvalues are $\lambda_{m,n} = \pi^2(m^2 + n^2)$ for $m, n \in \mathbb{Z}_{>0}$. The first five are:

- $\lambda_{1,1} = 2\pi^2 \approx 19.74$ (fundamental tone)
- $\lambda_{1,2} = \lambda_{2,1} = 5\pi^2 \approx 49.35$ (doublet)

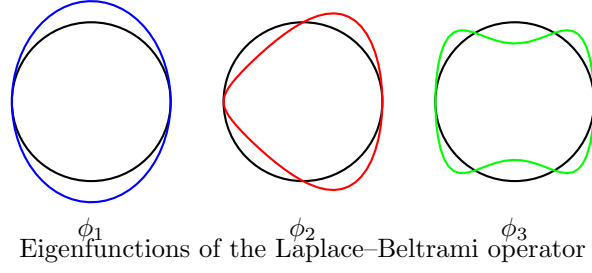


Figure 30.3: First three eigenfunctions of the Laplacian on a circle. The eigenvalues λ_k form the shape spectrum.

Algorithm 92 Shape-DNA Computation

```

1: function COMPUTESHAPEDNA(mesh, k)
2:    $L, M \leftarrow \text{BuildDiscreteLaplacian}(\text{mesh})$        $\triangleright L$ : cotangent stiffness;  $M$ : lumped mass
3:    $\text{eigenvalues}, \text{eigenfunctions} \leftarrow \text{scipy.sparse.linalg.eigsh}(L, M, k=k, \sigma=-1e-6)$ 
4:    $\text{area} \leftarrow \text{mesh.CalculateTotalArea}()$ 
5:    $\text{normalized\_dna} \leftarrow \text{eigenvalues} \times \text{area}$ 
6:   return  $\text{normalized\_dna}, \text{eigenfunctions}$ 
7: end function

```

- $\lambda_{2,2} = 8\pi^2 \approx 78.96$
- $\lambda_{1,3} = \lambda_{3,1} = 10\pi^2 \approx 98.70$ (doublet)

The doublet structure reflects the four-fold symmetry of the square. For a rectangle of area 1 but side ratio 1 : 2, the eigenvalues are $\pi^2(m^2 + 4n^2)$, and the first few differ, showing how the spectrum encodes aspect ratio.

30.13.5 Parameter Selections

- **Number of Eigenvalues (k):** Typically 10–100. More eigenvalues provide more detail but are more sensitive to mesh noise.
- **Normalization:** Normalizing by λ_1 or the total area is necessary to compare shapes of different sizes Theorem 30.62.
- **Lumped vs. Consistent Mass:** Using a diagonal lumped mass matrix is faster and usually sufficient for shape analysis.

30.13.6 Complexity

- **Matrix Construction:** $O(F)$ where F is the face count.
- **Eigen-decomposition:** $O(N^{1.5})$ for sparse matrices using Lanczos or LOBPCG methods.
- **Space Complexity:** $O(N \cdot k)$ to store the eigenfunctions.

30.13.7 Usages

- **Shape Retrieval:** Searching a 3D database for models that “look like” a query object [RWP06].
- **Symmetry Detection:** Identifying self-isometries of a shape.

- **Mesh Segmentation:** Partitioning a mesh into functional parts based on the nodal sets of eigenfunctions.
- **Graph Partitioning:** Using the second eigenvector (Fiedler vector) to split a mesh into two balanced pieces.
- **Style Analysis:** Classifying models into categories (e.g., “human,” “animal,” “chair”) based on their spectral signatures [Lev06].

30.13.8 Testing Methods and Edge Cases

- **Simple Geometries:** Verify that the spectrum of a sphere or a cube matches known analytical or high-precision values Theorem 30.67.
- **Area Invariance:** Ensure that scaling a mesh and then normalizing the spectrum yields the same DNA Theorem 30.62.
- **Mesh Refinement:** Verify that the eigenvalues converge as the mesh resolution increases.
- **Disconnected Components:** The number of zero eigenvalues ($\lambda = 0$) should match the number of connected components.
- **Isospectral Shapes:** Test on known pairs of isospectral shapes (like the “drums” of Gordon, Webb, and Wolpert [Kac66]) to see if they can be distinguished by other means.

30.13.9 References

- Kac, M. (1966). “Can one hear the shape of a drum?” *American Mathematical Monthly* [Kac66].
- Reuter, M., Wolter, F. E., & Peinecke, N. (2006). “Laplace–Beltrami spectra as ‘Shape-DNA’ of surfaces and solids.” *Computer-Aided Design* [RWP06].
- Levy, B. (2006). “Laplace-Beltrami Eigenfunctions towards an Optimal Tool for Geology and Geometry.” *SMA* [Lev06].
- Botsch, M., et al. (2010). “Polygon Mesh Processing.” AK Peters [BKP+10].
- Wikipedia: Spectral shape analysis.

30.14 Shape Space Spectra

30.14.1 Overview

Shape Space Spectra (or SpectralNet) is a deep-learning-based framework for learning spectral descriptors for shape analysis [S⁺25].

Traditional spectral descriptors (like HKS or WKS) are based on the Laplace–Beltrami spectrum of a single shape. Shape Space Spectra learns how these descriptors change as a shape is deformed. This provides a more robust way to perform tasks like shape matching and retrieval across non-rigid shapes [B⁺17].

Definition 30.68 (Shape Space Descriptor). A shape space descriptor is a learned map $f_\theta : (x, \theta) \mapsto \mathbb{R}^k$ that takes a point x on a surface and a deformation parameter θ , and produces a k -dimensional spectral descriptor. The parameters θ are learned end-to-end to maximize discriminability across non-rigid shape correspondences.

30.14.2 Implementation Details

In `CompGeom`, the `ShapeSpaceSpectralNet` provides:

1. **Point-Cloud Basis:** Operates on 3D point cloud representations of shapes.
2. **Latent Deformations:** Integrates shape deformation parameters (θ) to learn how the spectrum responds to change.
3. **Eigen-Descriptor Computation:** Generates k spectral descriptors for each point.

Remark 30.69. The key advantage of shape space spectra over fixed descriptors (like Shape-DNATheorem 30.62) is that the learned descriptors adapt to the distribution of deformations encountered in a dataset, rather than relying on a fixed spectral truncation. This makes them more robust to noise, partial observations, and topology changes.

30.14.3 Usage

Algorithm 93 Shape Space SpectralNet Usage

```

1: from compgeom.mesh.algorithms.shape_space_spectra import
   ShapeSpaceSpectralNet
2: import torch

3: # num_eigen: number of spectral descriptors to learn
4: net = ShapeSpaceSpectralNet(num_eigen=10)

5: # x: point cloud (N, 3), theta: deformation parameters (N, D)
6: x = torch.randn(100, 3)
7: theta = torch.randn(100, 8)
8: descriptors = net(x, theta)

```

Example 30.70 (Shape Space Spectra on a Bent Cylinder). Consider a cylinder that is gradually bent from straight to U-shaped, parameterized by a bending angle $\theta \in [0, \pi]$. Traditional Shape-DNATheorem 30.62 changes as the cylinder bends (because the spectrum shifts), but the changes are global and non-local. A shape space spectral descriptor $f_\theta(x, \theta)$ learns to produce stable per-vertex features: at $\theta = 0$ (straight cylinder), the descriptor is uniform along the axis; as θ increases, the descriptor for points on the inner bend diverges from those on the outer bend, capturing the local geometric distortion in a way that is invariant to isometric deformation.

30.14.4 References

- Sharp, N., et al. “SpectralNet: Learning Spectral Descriptors for Shape Analysis.” *ACM Transactions on Graphics (SIGGRAPH)*, 2025/2026 [S⁺25].
- Bronstein, M. M., et al. “Geometric Deep Learning: Grids, Groups, Graphs, Geodesics, and Gauges.” 2017 [B⁺17].

30.15 Odeco Frame Fields

30.15.1 Overview

Odeco Frame Fields is a method for designing 3D anisotropic frame fields in tetrahedral meshes using Orthonormal Decomposable (Odeco) tensors [Z⁺25].

A frame field is a volumetric structure that assigns a 3D orthogonal frame to each point in a volume. Frame fields are essential for generating high-quality quad-dominant or hex-dominant meshes. Standard frame field representations (like 4th-order tensors) can be difficult to control. The Odeco representation provides a more direct way to specify the orientation and stretching of the local frame [P⁺14].

Definition 30.71 (Odeco Tensor). An Orthonormal Decomposable (Odeco) tensor $\mathbf{Q} \in \mathbb{R}^{3 \times 3}$ is a symmetric positive-definite tensor that admits an eigendecomposition $\mathbf{Q} = \mathbf{R} \text{diag}(\sigma_1, \sigma_2, \sigma_3) \mathbf{R}^T$, where $\mathbf{R} \in SO(3)$ is a rotation matrix and $\sigma_1 \geq \sigma_2 \geq \sigma_3 > 0$ are the principal stretching factors. The eigenvectors of $\mathbf{Q}$ define the local frame.

Definition 30.72 (Frame Field). A frame field on a tetrahedral mesh $\mathcal{T}$ is a map $\mathcal{F} : \mathcal{T} \rightarrow SO(3) \times \mathbb{R}_{>0}^3$ assigning to each tetrahedron a rotation (orientation) and three stretching magnitudes (anisotropy). Equivalently, it can be represented as an Odeco tensor field $\{\mathbf{Q}_e\}$ over the edges or elements of $\mathcal{T}$.

30.15.2 Implementation Details

The `OdecoFrameField` implementation in `CompGeom` allows for:

1. **Boundary Alignment:** The frames can be explicitly aligned to boundary normals.
2. **Anisotropy Control:** The stretching magnitudes along each frame axis can be specified.
3. **Smoothness Optimization:** The orientations and stretching can be propagated through the volume via a smoothing process.

Theorem 30.73 (Odeco Frame Field Smoothness). *The Odeco frame field smoothness energy $E_{\text{smooth}} = \sum_{(e_1, e_2) \in \mathcal{E}} w_{e_1, e_2} \|\mathbf{Q}_{e_1} - \mathbf{R}_{e_1 \rightarrow e_2} \mathbf{Q}_{e_2} \mathbf{R}_{e_1 \rightarrow e_2}^T\|^2$ is minimized when adjacent frames are related by parallel transport along the mesh, where $\mathbf{R}_{e_1 \rightarrow e_2}$ is the rotation that maps the reference frame of e_1 to that of e_2 .*

Proof. The energy measures the Frobenius norm of the difference between the tensor at e_1 and the parallel-transported tensor from e_2 . Since the Frobenius norm is invariant under simultaneous rotation $\|\mathbf{A}\|_F = \|\mathbf{R}\mathbf{A}\mathbf{R}^T\|_F$, the energy is well-defined regardless of the choice of reference frame for each element. The global minimizer of this quadratic energy is the solution to a sparse linear system on the mesh, which can be solved efficiently by preconditioned conjugate gradient. $\square$

30.15.3 Usage

Example 30.74 (Frame Field on a Cube). Consider a unit cube meshed with tetrahedra. We align the boundary faces to the coordinate axes: the front/back faces have frame $(e_1, e_2, e_3) = (x, y, z)$, the left/right faces have (y, z, x) , and the top/bottom have (z, x, y) . The Odeco smoothness solver Theorem 30.73 propagates these orientations into the interior, creating a smoothly varying frame field. The resulting field has a singularity at the cube center (since the four boundary orientations cannot be globally consistent), which is detected as a point where the frame rotation is ill-defined. This singularity structure guides the subsequent hexahedral remeshing.

Remark 30.75. Frame field singularities are analogous to vector field singularities on surfaces (Poincaré–Hopf theorem). Their number and type are constrained by the Euler characteristic of the domain. Controlling singularity placement is critical for generating high-quality hex meshes [BLP⁺13].

Algorithm 94 Odeco Frame Field Design

```
1: from compgeom.mesh.volume.odeco_frame_field import OdecoFrameField
2: from compgeom.mesh.volume.tetmesh.tetmesh import TetMesh
3: import numpy as np

4: # Load a tetrahedral mesh
5: mesh = TetMesh.from_file("volume.obj")

6: # Initialize and align to boundary
7: off = OdecoFrameField(mesh)
8: off.align_to_boundary(normals, indices)

9: # Propagate across the volume
10: off.solve_smoothness(iterations=10)
```

30.15.4 References

- Zhu, Y., et al. “Designing 3D Anisotropic Frame Fields with Odeco Tensors.” *ACM Transactions on Graphics (SIGGRAPH)*, 2025 [Z⁺25].
- Panozzo, D., et al. “Frame Fields: An Orientation-aware Surface Representation.” *ACM Transactions on Graphics (SIGGRAPH)*, 2014 [P⁺14].
- Bommes, D., et al. (2013). “Quad-Mesh Generation and Processing: A Survey.” *Computer Graphics Forum* [BLP⁺13].

30.16 Discrete Exterior Calculus

30.17 Hodge Star Computation

30.17.1 Overview

The Hodge star operator ($*$) is a fundamental tool in Riemannian geometry and exterior calculus that establishes a correspondence between k -forms and $(n - k)$ -forms. In Discrete Exterior Calculus (DEC) [DHLM05, CWW13a], the Hodge star operator is represented as a diagonal matrix that maps discrete forms on a primal mesh to discrete forms on its dual mesh. It is essential for solving partial differential equations (PDEs), such as the Poisson equation or Maxwell’s equations, on triangle meshes.

30.17.2 Definitions

Definition 30.76 (Discrete k -Form). A scalar value associated with each k -simplex of a mesh (0-form: vertex, 1-form: edge, 2-form: face). More precisely, a discrete k -form is an element of the dual space $\Omega^k(K) = C^k(K; \mathbb{R})$, where C^k is the space of k -cochains on the simplicial complex K .

Definition 30.77 (Primal Mesh). The original triangular mesh, consisting of vertices (V), edges (E), and faces (F). The primal mesh serves as the domain on which discrete differential operators are defined.

Definition 30.78 (Dual Mesh). Typically the Voronoi diagram or the circumcentric dual of the primal mesh. Each primal k -simplex corresponds to a dual $(n - k)$ -simplex, establishing a bijection between the primal and dual cell complexes.

Definition 30.79 (Hodge Star $(\star)$). An operator that maps a discrete form on a primal simplex to its dual simplex, preserving “flow” or “intensity” across the two. The Hodge star $\star_k: \Omega^k(K) \rightarrow \Omega^{n-k}(K^\star)$ encodes the local metric information of the mesh.

30.17.3 Theory

In 2D (surface meshes), the discrete Hodge star operators are typically defined as follows:

- **0-star** $(\star_0)$: Maps a value at a primal vertex to its dual Voronoi cell. $\star_0 = \frac{\text{Area}(\text{Dual Cell})}{\text{Area}(\text{Primal Vertex})} = \text{Area}(\text{Dual Cell})$.
- **1-star** $(\star_1)$: Maps a value on a primal edge to its dual Voronoi edge. $\star_1 = \frac{\text{Length}(\text{Dual Edge})}{\text{Length}(\text{Primal Edge})}$. This is the famous cotangent weight ratio: $\frac{1}{2}(\cot \alpha + \cot \beta)$.
- **2-star** $(\star_2)$: Maps a value on a primal face to its dual Voronoi vertex. $\star_2 = \frac{\text{Area}(\text{Dual Vertex})}{\text{Area}(\text{Primal Face})} = \frac{1}{\text{Area}(\text{Primal Face})}$.

Theorem 30.80 (Cotangent Weight Formula). *The discrete 1-Hodge star on a triangulated surface with circumcentric dual is given by the cotangent weights:*

$$(\star_1)_e = \frac{1}{2}(\cot \alpha_e + \cot \beta_e),$$

where α_e and β_e are the angles opposite to edge e in the two adjacent triangles. These weights are non-negative if and only if the two triangles sharing edge e form an intrinsic Delaunay triangulation.

Proof. Consider an edge $e = (u, v)$ shared by two triangles $\triangle uvw_1$ and $\triangle uvw_2$. The dual edge $e^\star$ connects the circumcenters c_1, c_2 of the two triangles. Since the circumcenter lies at the intersection of perpendicular bisectors, the dual edge $e^\star$ is perpendicular to the primal edge e . By elementary trigonometry, the length of the dual edge satisfies

$$|e^\star| = \frac{1}{2}|e|(\cot \alpha_e + \cot \beta_e),$$

where $\alpha_e = \angle uw_1v$ and $\beta_e = \angle uw_2v$. Dividing by $|e|$ yields the stated ratio. The non-negativity condition follows from the fact that $\cot \theta > 0$ for $\theta < \pi/2$ and $\cot \theta < 0$ for $\theta > \pi/2$ [PP93]. $\square$

Example 30.81 (Cotangent Weight on an Equilateral Triangle). Consider two equilateral triangles sharing an edge, forming a rhombus. Each opposite angle is 60° , so the cotangent weight is:

$$(\star_1)_e = \frac{1}{2}(\cot 60^\circ + \cot 60^\circ) = \frac{1}{2} \left(\frac{1}{\sqrt{3}} + \frac{1}{\sqrt{3}} \right) = \frac{1}{\sqrt{3}} \approx 0.577.$$

This is the canonical value for a regular triangulation, and the resulting Laplacian matrix reduces to the standard 5-point stencil on a regular grid.

These operators allow for the definition of the Discrete Laplace-Beltrami Operator as $\Delta = \star_0^{-1} d^\top \star_1 d$, where d is the discrete exterior derivative [CWW13a].

30.17.4 Pseudocode

Algorithm 95 computes all three diagonal Hodge star matrices in a single pass over the mesh elements. The dominant cost is the Voronoi area and angle computations per element.

Algorithm 95 Compute Hodge Star Operators

```
function COMPUTEHODGESTARS(mesh) ▷ 1. 0-Star (Vertex areas)
    star0 ← DiagonalMatrix( $n\_vertices$ )
    for all  $v \in \text{mesh.vertices}$  do
        star0[ $v, v$ ] ← ComputeVoronoiArea( $v, mesh$ )
    end for

▷ 2. 1-Star (Edge cotangent weights)
    star1 ← DiagonalMatrix( $n\_edges$ )
    for all  $edge \in \text{mesh.edges}$  do
         $\alpha, \beta \leftarrow \text{GetOppositeAngles}(edge, mesh)$ 
        star1[ $edge, edge$ ] ←  $0.5 \cdot (\cot(\alpha) + \cot(\beta))$ 
    end for

▷ 3. 2-Star (Face areas)
    star2 ← DiagonalMatrix( $n\_faces$ )
    for all  $face \in \text{mesh.faces}$  do
        star2[ $face, face$ ] ←  $1.0 / \text{ComputeFaceArea}(face, mesh)$ 
    end for
    return star0, star1, star2
end function
```

30.17.5 Parameter Selection

- **Dual Mesh Choice:** The **Circumcentric Dual** is preferred because primal edges are perpendicular to dual edges, simplifying the star operator to a simple ratio.
- **Robustness:** Cotangent weights can become negative if the mesh contains obtuse triangles. Using an intrinsic Delaunay triangulation or an epsilon-clamping can mitigate this [PP93].

30.17.6 Complexity

- **Time Complexity:** $O(F)$, where F is the number of faces, to calculate all local geometric properties (areas, lengths, angles).
- **Space Complexity:** $O(V + E + F)$ to store the diagonal matrices.

30.17.7 Usages

- **Surface Parameterization:** Constructing the Laplacian matrix for Harmonic Mapping or LSCM.
- **Mesh Smoothing:** Implementing Mean Curvature Flow using the DEC Laplacian.
- **Fluid Simulation on Surfaces:** Solving the Navier-Stokes equations using discrete forms.
- **Geodesic Distances:** Implementing the Heat Method for distance calculation.
- **Texture Synthesis:** Generating procedural textures by solving reaction-diffusion equations on meshes.

30.17.8 Testing Methods and Edge Cases

- **Flat Plane:** On a regular grid, the cotangent weights should simplify to the standard 5-point Laplacian stencil.

- **Equilateral Triangles:** All cotangent weights should be exactly $\cot(60^\circ) = 1/\sqrt{3}$.
- **Obtuse Triangles:** Verify that negative cotangent weights are handled or expected (they can cause instability in some solvers).
- **Boundaries:** Vertices and edges on the boundary have only one incident face; their Hodge star contributions must be halved or handled carefully.

30.17.9 References

- Desbrun, M., Hirani, A. N., Leok, M., & Marsden, J. E. (2005). “Discrete Exterior Calculus”. arXiv preprint.
- Crane, K., Weischedel, C., & Wardetzky, M. (2013). “Digital Geometry Processing with Discrete Exterior Calculus”. SIGGRAPH Course.
- Pinkall, U., & Polthier, K. (1993). “Computing discrete minimal surfaces and their conjugates”. Computing.
- Wikipedia: Discrete exterior calculus

30.18 Exterior Derivative

30.18.1 Overview

The exterior derivative (d) is a fundamental operator in differential geometry that generalizes the concepts of gradient, curl, and divergence to higher-dimensional forms. In Discrete Exterior Calculus (DEC) [DHL05, CWW13a], the exterior derivative is a sparse matrix that acts as a topological operator, mapping discrete k -forms on a mesh to discrete $(k+1)$ -forms. It depends purely on the connectivity of the mesh (its boundary relationships) and is independent of the mesh’s geometric coordinates.

30.18.2 Definitions

Definition 30.82 (Discrete k -Form (ω^k)). A vector of scalar values, where each entry is associated with a k -simplex of the mesh (0-simplex: vertex, 1-simplex: edge, 2-simplex: face). The space of all discrete k -forms is denoted Ω^k .

Definition 30.83 (Chain Complex). A sequence of vector spaces of k -chains connected by boundary operators (∂):

$$\dots \xrightarrow{\partial_{k+1}} C_k \xrightarrow{\partial_k} C_{k-1} \xrightarrow{\partial_{k-1}} \dots$$

satisfying the fundamental property $\partial_k \circ \partial_{k+1} = 0$ for all k .

Definition 30.84 (Cochain Complex). A sequence of vector spaces of k -forms connected by exterior derivatives (d):

$$\dots \xrightarrow{d_{k-1}} \Omega^k \xrightarrow{d_k} \Omega^{k+1} \xrightarrow{d_{k+1}} \dots$$

The cochain complex is dual to the chain complex, and the exterior derivative d_k is the transpose of the boundary operator ∂_{k+1} .

Definition 30.85 (Boundary Relationship (∂)). The boundary of a 1-simplex (edge) is its two 0-simplices (vertices). The boundary of a 2-simplex (triangle) is its three 1-simplices (edges). More generally, ∂_k maps each k -simplex to a formal sum of its $(k-1)$ -faces with appropriate orientations.

30.18.3 Theory

The discrete exterior derivative d_k is the **dual of the boundary operator** ∂_{k+1} from the primal mesh. Specifically, it maps a value on a k -simplex to its incident $(k+1)$ -simplices based on orientation.

- **0-derivative** (d_0): Maps a value at vertices to its incident edges. $d_0(\phi)_{(u,v)} = \phi_v - \phi_u$. This is the discrete equivalent of the **Gradient**.
- **1-derivative** (d_1): Maps a value on edges to its incident faces. $d_1(\alpha)_{(u,v,w)} = \alpha_{uv} + \alpha_{vw} + \alpha_{wu}$ (where indices are oriented). This is the discrete equivalent of the **Curl**.

Theorem 30.86 (Null Property of the Exterior Derivative). *For any discrete k -form ω on a simplicial complex, the composition $d_{k+1} \circ d_k = 0$. That is, the derivative of a derivative is always zero.*

Proof. We verify this directly. Let ϕ be a 0-form (vertex values). Then $(d_0\phi)_{(u,v)} = \phi_v - \phi_u$. Applying d_1 to this 1-form yields, for a face (u, v, w) :

$$(d_1 d_0 \phi)_{(u,v,w)} = (\phi_v - \phi_u) + (\phi_w - \phi_v) + (\phi_u - \phi_w) = 0.$$

This holds for every face, so $d_1 d_0 = 0$ as a matrix product. In higher dimensions, the argument is analogous: each $(k+2)$ -simplex has exactly two $(k+1)$ -faces that contain each k -face, and their contributions cancel due to opposite orientations. This is the discrete version of the identities $\nabla \times (\nabla \phi) = 0$ and $\nabla \cdot (\nabla \times A) = 0$ [DHLM05]. $\square$

Example 30.87 (Exterior Derivative on a Single Triangle). Let a triangle have vertices u, v, w with a 0-form $\phi(u) = 1, \phi(v) = 3, \phi(w) = 0$. The 1-form $d_0\phi$ assigns values to oriented edges:

$$(d_0\phi)_{uv} = 3 - 1 = 2, \quad (d_0\phi)_{vw} = 0 - 3 = -3, \quad (d_0\phi)_{wu} = 1 - 0 = 1.$$

Applying d_1 :

$$(d_1 d_0 \phi)_{uvw} = 2 + (-3) + 1 = 0,$$

confirming the null property $d_1 \circ d_0 = 0$.

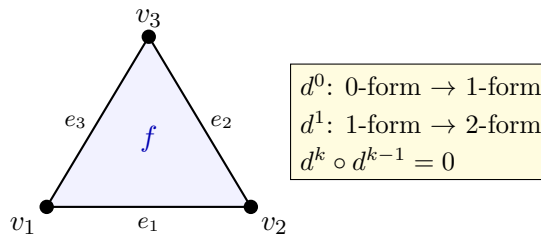


Figure 30.4: Discrete exterior derivative: d^0 maps vertex values to edge differences; d^1 maps edge values to face circulations.

30.18.4 Pseudocode

Note that Algorithm 96 requires only the mesh connectivity (vertex-edge and edge-face incidence), not any geometric information such as vertex positions or edge lengths.

Algorithm 96 Compute Exterior Derivatives

```

function COMPUTEEXTERIORDERIVATIVES(mesh)                                ▷ 1. d0 (0-forms to 1-forms)
     $d0 \leftarrow \text{SparseMatrix}(n\_edges, n\_vertices)$ 
    for all  $edge \in \text{mesh.edges}$  do
         $u, v \leftarrow edge.start\_vertex, edge.end\_vertex$ 
         $d0[edge\_id, v] \leftarrow 1$ 
         $d0[edge\_id, u] \leftarrow -1$ 
    end for

                                                                                         ▷ 2. d1 (1-forms to 2-forms)
     $d1 \leftarrow \text{SparseMatrix}(n\_faces, n\_edges)$ 
    for all  $face \in \text{mesh.faces}$  do
        for all  $(edge\_id, orientation) \in \text{GetOrientedEdges}(face)$  do
             $d1[face\_id, edge\_id] \leftarrow orientation$ 
        end for
    end for
    return  $d0, d1$ 
end function

```

30.18.5 Parameter Selection

- **Orientation:** All simplices must be consistently oriented. Primal edges are typically oriented by vertex ID (e.g., $u \rightarrow v$ if $u < v$). Primal faces are usually CCW.
- **Sparse Representation:** Since each simplex has only a few neighbors, d matrices are extremely sparse; CSR or CSC formats are essential for performance.

30.18.6 Complexity

- **Time Complexity:** $O(F)$, where F is the number of faces, as we iterate through each face and edge relationship once.
- **Space Complexity:** $O(E + F)$ to store the sparse matrix entries.

30.18.7 Usages

- **Vector Field Processing:** Decomposing vector fields into curl-free and divergence-free components (Hodge Decomposition).
- **Surface Parameterization:** Representing gradients of scalar functions used in UV mapping.
- **Fluid Simulation:** Enforcing divergence-free conditions on flow velocities.
- **Solving PDEs:** Formulating the Poisson or Heat equation using the DEC Laplacian $\Delta = \star^{-1} d^\top \star d$.
- **Topology Analysis:** Computing Betti numbers or identifying mesh “holes” using homology.

30.18.8 Testing Methods and Edge Cases

- **d o d Property:** Verify that $d_1 \times d_0$ results in a matrix where all entries are near zero (floating point) or exactly zero (integer).

- **Single Triangle:** On a single triangle, the sum of edge values around the boundary must match the face derivative.
- **Disconnected Mesh:** The matrices should naturally decompose into block-diagonal structures for each component.
- **Boundaries:** Ensure that orientation is handled correctly for edges on the mesh boundary.

30.18.9 References

- Desbrun, M., Hirani, A. N., Leok, M., & Marsden, J. E. (2005). “Discrete Exterior Calculus”. arXiv preprint.
- Crane, K., Weischedel, C., & Wardetzky, M. (2013). “Digital Geometry Processing with Discrete Exterior Calculus”. SIGGRAPH Course.
- Wikipedia: Exterior derivative

30.19 Hodge Decomposition

30.19.1 Overview

The Hodge-Helmholtz Decomposition (often simply Hodge Decomposition) is a fundamental theorem in vector calculus and differential geometry that states any vector field can be uniquely decomposed into three orthogonal components: a curl-free component (the gradient of a scalar potential), a divergence-free component (the curl of a vector potential), and a harmonic component (representing flow around topological “holes”). In the context of 2D surface meshes, it provides a powerful way to analyze and manipulate tangent vector fields [CWW13a, TLHD03].

30.19.2 Definitions

Definition 30.88 (Exact Component (ω_{exact})). The gradient of a scalar function α , $\omega_{\text{exact}} = d\alpha$. These are **curl-free**, meaning $d(\omega_{\text{exact}}) = d(d\alpha) = 0$ by Theorem 30.86. In $\mathbb{R}^3$ notation, $\omega_{\text{exact}} = \nabla\alpha$.

Definition 30.89 (Co-exact Component ($\omega_{\text{co-exact}}$)). The “rotational” part, represented as $\omega_{\text{co-exact}} = \star d\beta$. These are **divergence-free**, meaning $\delta(\omega_{\text{co-exact}}) = 0$, where $\delta = \star^{-1}d^\top\star$ is the codifferential. In $\mathbb{R}^3$ notation, $\omega_{\text{co-exact}} = \nabla \times \mathbf{A}$ for some vector potential $\mathbf{A}$.

Definition 30.90 (Harmonic Component (ω_{harmonic})). A field that is both curl-free and divergence-free, $d\omega = \delta\omega = 0$. These depend on the topology of the mesh—specifically, on the Betti numbers of the surface. On a closed surface of genus g , there are exactly $2g$ linearly independent harmonic 1-forms [PP03].

30.19.3 Theory

In Discrete Exterior Calculus (DEC), the decomposition is performed by solving two separate Poisson equations [CWW13a]:

1. **Gradient Part:** Minimize the energy $\|\omega - d\alpha\|^2$ to find the scalar potential α . This leads to the linear system $\Delta\alpha = \delta\omega$, where $\delta = \star^{-1}d^\top\star$ is the codifferential (divergence).
2. **Curl Part:** Minimize the energy $\|\omega - \delta\beta\|^2$ to find the dual potential β . This leads to the linear system $\Delta\beta = d\omega$, where d is the curl.
3. **Harmonic Part:** The remainder $\gamma = \omega - d\alpha - \delta\beta$ is the harmonic component.

Orthogonality between these components is guaranteed by the $d \circ d = 0$ property (Theorem 30.86) and the definition of the Hodge star.

Theorem 30.91 (Helmholtz-Hodge Decomposition). *For any 1-form ω on a compact, oriented Riemannian 2-manifold, there exist unique forms α (0-form), β (2-form), and γ (harmonic 1-form) such that:*

$$\omega = d\alpha + \star d\beta + \gamma,$$

where $d\alpha$ is exact (curl-free), $\star d\beta$ is co-exact (divergence-free), and γ is harmonic. Moreover, the three components are mutually orthogonal under the L^2 inner product:

$$\langle d\alpha, \star d\beta \rangle = \langle d\alpha, \gamma \rangle = \langle \star d\beta, \gamma \rangle = 0.$$

Proof. We construct the decomposition explicitly. Given a 1-form ω :

Step 1 (Exact part): Solve $\Delta\alpha = \delta\omega$ for α , where $\Delta = d^\top \star_1^{-1} d + d_1^\top \star_2 d_1^{-1} \dots$ is the discrete Laplacian. Define $\omega_{\text{exact}} = d\alpha$. Then $\delta(\omega - d\alpha) = \delta\omega - \Delta\alpha = 0$.

Step 2 (Co-exact part): Solve $\Delta\beta = d(\omega - d\alpha)$ for β . Define $\omega_{\text{co-exact}} = \star d\beta$. By Theorem 30.86, $d(\star d\beta) = 0$, so the co-exact part is divergence-free.

Step 3 (Harmonic part): Set $\gamma = \omega - d\alpha - \star d\beta$. By construction, $d\gamma = d\omega - d(d\alpha) - d(\star d\beta) = d\omega - 0 - 0 = d\omega - d\omega = 0$, and similarly $\delta\gamma = 0$.

Orthogonality follows from $\langle d\alpha, \star d\beta \rangle = \langle \alpha, d^\top \star d\beta \rangle = \langle \alpha, \Delta\beta \rangle$ and the self-adjointness of Δ [TLHD03]. $\square$

Example 30.92 (Hodge Decomposition on a Torus). Consider a uniform vector field ω flowing around the major circumference of a torus (genus $g = 1$). Since the field has no local variation, both $d\omega$ and $\delta\omega$ are zero, meaning ω is itself harmonic: $\omega = \gamma$. The exact and co-exact components are both zero. This demonstrates that harmonic components capture global topological flow—they cannot be expressed as local gradients or curls [PP03].

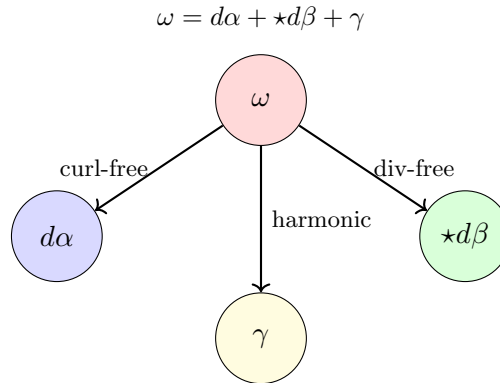


Figure 30.5: Hodge decomposition: any 1-form ω splits uniquely into exact, co-exact, and harmonic components.

30.19.4 Pseudocode

As shown in Algorithm 97, the algorithm relies on Algorithms 95 and 96 as subroutines and requires solving two sparse linear systems.

30.19.5 Parameter Selection

- **Boundary Conditions:** For meshes with boundaries, the decomposition is not unique unless boundary conditions are specified (e.g., $d\alpha$ is normal or tangent to the boundary).

Algorithm 97 Hodge Decomposition of a 1-Form

```

function HODGEDecompose(omega, mesh)                                ▷ 1. Compute discrete operators
    d0, d1 ← ComputeExteriorDerivatives(mesh)                        ▷ Algorithm 96
    star0, star1, star2 ← ComputeHodgeStars(mesh)                    ▷ Algorithm 95
                                                                    ▷ 2. Find Exact Component ( $d\alpha$ )

    Laplacian_0 ←  $d0^\top \cdot star1 \cdot d0$ 
    rhs_exact ←  $d0^\top \cdot star1 \cdot omega$ 
     $\alpha$  ← SolveSparse(Laplacian_0, rhs_exact)
    exact_part ←  $d0 \cdot \alpha$ 

                                                                    ▷ 3. Find Co-exact Component ( $\star d\beta$ )

    Laplacian_1 ←  $d1 \cdot star1^{-1} \cdot d1^\top$ 
    rhs_coexact ←  $d1 \cdot omega$ 
     $\hat{\beta}$  ← SolveSparse(Laplacian_1, rhs_coexact)
    coexact_part ←  $star1^{-1} \cdot d1^\top \star 2^{-1} \cdot \hat{\beta}$ 

                                                                    ▷ 4. Harmonic Part

    harmonic_part ←  $omega - exact\_part - coexact\_part$ 
    return exact_part, coexact_part, harmonic_part
end function

```

- **Sparse Solver:** Cholesky factorization (via `scipy.sparse.linalg.factorized`) is recommended as the Laplacian matrices are symmetric and positive semi-definite.

30.19.6 Complexity

- **Time Complexity:** $O(E^{1.5})$ where E is the number of edges, dominated by the sparse linear solver performance.
- **Space Complexity:** $O(E)$ to store the discrete operators and the field vectors.

30.19.7 Usages

- **Vector Field Design:** Creating smooth vector fields for hair, fur, or pen-and-ink rendering by manipulating the potentials.
- **Surface Parameterization:** Constructing “conformal” maps by ensuring the gradient field of UV coordinates is harmonic.
- **Fluid Animation:** Extracting the rotational (co-exact) part of a flow for vortex visualization or removing divergence to maintain volume.
- **Shape Matching:** Using harmonic components as topological signatures for 3D models.
- **Meteorology:** Analyzing global wind patterns into geostrophic (exact) and ageostrophic components.

30.19.8 Testing Methods and Edge Cases

- **Pure Gradient Field:** If $\omega = d\phi$ for some random scalar ϕ , then the co-exact and harmonic parts should be nearly zero.
- **Zero-Genus Mesh:** On a mesh with the topology of a sphere (genus 0), the harmonic part should be exactly zero.
- **Topological Holes:** On a torus (genus 1), a non-zero harmonic component should appear as a field that “wraps” around the hole without having any divergence or curl.

- **Exactness Verification:** Verify that $d_1 \times \text{exact_part} \approx 0$ and $\delta \times \text{coexact_part} \approx 0$.

30.19.9 References

- Polthier, K., & Preuss, E. (2003). “Identifying vector field singularities using a discrete Hodge decomposition”. Visualization and Mathematics III.
- Tong, Y., Lombeyda, S., Hirani, A. N., & Desbrun, M. (2003). “Discrete Multiscale Vector Field Decomposition”. ACM TOG.
- Crane, K. (2013). “Digital Geometry Processing with Discrete Exterior Calculus”. SIGGRAPH Course.
- Wikipedia: Helmholtz decomposition

30.20 Discrete Morse Theory

30.20.1 Overview

Discrete Morse Theory, developed by Robin Forman [For98, For02], is a combinatorial version of the classical Morse theory used to study the topology of a manifold by analyzing the critical points of a smooth function defined on it. In the discrete setting, it provides a powerful framework for simplifying complex simplicial complexes (like meshes) while preserving their fundamental topological properties (homology). It is used for topological data analysis, mesh compression, and identifying “features” like ridges and valleys in geometric data [GNPB08].

30.20.2 Definitions

Definition 30.93 (Simplicial Complex (K)). A collection of simplices (vertices, edges, faces) such that every face of a simplex is also in K . A simplicial complex encodes both the combinatorial structure (which elements are connected) and the dimensional hierarchy of the mesh.

Definition 30.94 (Discrete Morse Function (f)). A function $f: K \rightarrow \mathbb{R}$ that assigns a real number to every simplex in K such that for each simplex σ :

1. $|\{\tau^{(p+1)} > \sigma : f(\tau) \leq f(\sigma)\}| \leq 1$,
2. $|\{\nu^{(p-1)} < \sigma : f(\nu) \geq f(\sigma)\}| \leq 1$.

That is, at most one “upper neighbor” has a value no larger, and at most one “lower neighbor” has a value no smaller. A simplex where both counts are zero is called **critical**.

Definition 30.95 (Gradient Vector Field). A pairing of simplices into “gradient arrows” ($\alpha^p < \beta^{p+1}$) where $f(\alpha) \geq f(\beta)$. Each non-critical simplex belongs to exactly one gradient pair, and each gradient pair connects simplices of adjacent dimensions.

Definition 30.96 (Critical Simplex). A simplex that is not part of any gradient pair. Critical simplices correspond to the topological features of the underlying space: 0-dimensional critical simplices (minima) correspond to connected components, 1-dimensional (saddles) to handles, and 2-dimensional (maxima) to voids.

Definition 30.97 (Morse Complex). A reduced complex formed only by the critical simplices and their connectivity. The Morse-Smale complex further incorporates the gradient flow lines between critical points, providing a complete decomposition of the manifold into cells.

30.20.3 Theory

The core idea of Discrete Morse Theory is the **Discrete Gradient Vector Field** [For98].

1. A simplex α^p can be paired with an incident simplex β^{p+1} of higher dimension.
2. If every simplex is paired, the complex can be “collapsed” to a single point without changing its homotopy type.
3. Any simplex that *cannot* be paired is a **critical simplex**.
4. The number of critical simplices of dimension p (denoted m_p) provides an upper bound on the p -th Betti number β_p (Morse Inequalities): $m_p \geq \beta_p$.

Theorem 30.98 (Morse-Euler Relation). *For any discrete Morse function f on a finite simplicial complex K , the alternating sum of the counts of critical simplices equals the Euler characteristic:*

$$\sum_{p=0}^n (-1)^p m_p = \chi(K),$$

where m_p is the number of critical p -simplices.

Proof. The proof follows from the Morse complex construction. Each critical simplex contributes $(-1)^p$ to the alternating sum. Since the Morse complex is homotopy equivalent to the original complex, its Euler characteristic equals $\chi(K)$. For a finite complex K with v vertices, e edges, and f faces, we have $\chi(K) = v - e + f$. The gradient pairing removes one p -simplex and one $(p+1)$ -simplex simultaneously, which does not change the alternating sum $v - e + f$. Therefore, the sum over critical simplices alone must equal $\chi(K)$ [For02]. $\square$

Example 30.99 (Morse Function on a Sphere). Consider a geodesic sphere with a height function $f(v) = z$ -coordinate. A “perfect” discrete Morse function on a sphere should yield exactly two critical simplices: one vertex (the global minimum at the south pole, $m_0 = 1$) and one face (the global maximum at the north pole, $m_2 = 1$). The Morse-Euler relation gives $m_0 - m_1 + m_2 = 1 - 0 + 1 = 2 = \chi(S^2)$, which is correct for a sphere.

By constructing an appropriate Morse function, one can extract a **1D skeleton** (the Morse-Smale complex) that summarizes the flow and connectivity of a scalar field (like elevation) over a surface [GNPB08].

30.20.4 Pseudocode

Greedy Construction of a Discrete Gradient Field

Algorithm 98 produces a valid discrete gradient field, but not necessarily one with the fewest critical points. More sophisticated algorithms [GNPB08] use cancellation operations to merge pairs of critical simplices and further simplify the Morse complex.

30.20.5 Parameter Selection

- **Initial Function:** The choice of the input scalar field (e.g., height, curvature, or random) determines which geometric features are identified as critical points.
- **Simplification Threshold:** Small “noise” critical points can be removed by canceling pairs of critical points (e.g., a small hill and a small saddle) using Morse cancellations.

Algorithm 98 Greedy Construction of Discrete Gradient Field

```

function GREEDYGRADIENTFIELD(mesh)
    unpaired  $\leftarrow$  set(mesh.all_simplices)
    gradient_pairs  $\leftarrow$  []
    critical_simplices  $\leftarrow$  []
    while unpaired is not empty do
         $\alpha \leftarrow$  FindSimplexWithOneNeighbor(unpaired)
        if  $\alpha \neq \text{None}$  then
             $\beta \leftarrow$  GetOnlyNeighbor( $\alpha$ , unpaired)
            gradient_pairs.append(( $\alpha$ ,  $\beta$ ))
            unpaired.remove( $\alpha$ )
            unpaired.remove( $\beta$ )
        else
             $\alpha \leftarrow$  unpaired.pop()
            critical_simplices.append( $\alpha$ )
        end if
    end while
    return gradient_pairs, critical_simplices
end function

```

30.20.6 Complexity

- **Time Complexity:** $O(N \log N)$ to sort simplices by function value, followed by $O(N)$ for the greedy pairing, where N is the total number of simplices.
- **Space Complexity:** $O(N)$ to store the simplices and the gradient field.

30.20.7 Usages

- **Topological Data Analysis (TDA):** Computing persistent homology and identifying stable structural features in noisy data.
- **Mesh Compression:** Reducing a complex mesh to a much smaller Morse complex that has the same topology.
- **Terrain Analysis:** Automatically identifying peaks (maximums), pits (minimums), and passes (saddles) in geographic maps.
- **Scientific Visualization:** Constructing Morse-Smale complexes to segment a volume based on the behavior of a physical field (e.g., velocity or pressure).
- **Feature Extraction:** Detecting ridge and valley lines on 3D models.

30.20.8 Testing Methods and Edge Cases

- **Sphere:** A perfect discrete Morse function on a sphere should yield exactly two critical simplices: one vertex (min) and one face (max).
- **Torus:** Should yield one vertex, two edges (loops), and one face.
- **Euler Characteristic:** Verify the Morse-Euler relation (Theorem 30.98): $\sum (-1)^p m_p = \chi$, where m_p is the number of critical p -simplices.
- **Noise Handling:** Test the algorithm on a noisy field to ensure it produces many small critical points, which can then be simplified.

- **Boundaries:** Ensure the theory correctly accounts for simplices on the boundary of the complex.

30.20.9 References

- Forman, R. (1998). “Discrete Morse Theory”. *Advances in Mathematics*.
- Forman, R. (2002). “A user’s guide to discrete Morse theory”. *Sém. Lothar. Combin.*
- Gyulassy, A., Natarajan, V., Pascucci, V., & Bremer, P. T. (2008). “A Practical Approach to Morse-Smale Complex Computation: Quasi-complexes and Geometric Methods”. *IEEE TVCG*.
- Wikipedia: Discrete Morse theory

Part X

Advanced Algorithmic Techniques

Chapter 31

Randomized Geometric Algorithms

- 31.1 Why Randomization Helps
- 31.2 Randomized Incremental Construction
- 31.3 Backward Analysis
- 31.4 Conflict Graphs
- 31.5 Clarkson–Shor Technique
- 31.6 Random Sampling
- 31.7 ε -Nets
- 31.8 Applications to Convex Hulls
- 31.9 Applications to Delaunay Triangulations
- 31.10 Applications to Linear Programming
- 31.11 Exercises

Chapter 32

Approximation Algorithms in Geometry

- 32.1 Why Approximation?
- 32.2 Geometric Approximation
- 32.3 ε -Approximations
- 32.4 Coresets
- 32.5 Approximate Nearest Neighbors
- 32.6 Locality-Sensitive Hashing
- 32.7 Well-Separated Pair Decomposition
- 32.8 Approximate Distance Queries
- 32.9 Geometric Clustering
- 32.10 k -Center and k -Median Problems
- 32.11 Minimum Enclosing Shapes
- 32.12 High-Dimensional Problems
- 32.13 Exercises
- 32.14 Project: Approximate Nearest-Neighbor Search

Chapter 33

Packing Algorithms

33.1 Circle Packing

33.1.1 Overview

Circle packing is the study of placing non-overlapping circles within a given container (like a square or another circle) or on a surface (like a mesh) such that the total area of the circles is maximized or the density is optimized. In computational geometry, it is a classic problem in both optimization and discrete geometry, with applications ranging from material science to map labeling and data visualization. The theory of circle packing was developed extensively by Stephenson [Ste05], building on the foundational work of Koebe [Koe36] and the influential exposition of Thurston [Thu85].

33.1.2 Definitions

Definition 33.1 (Packing Density). The **packing density** of a configuration of n circles $\{C_1, \dots, C_n\}$ in a container Ω is the ratio

$$\delta = \frac{\sum_{i=1}^n \text{Area}(C_i)}{\text{Area}(\Omega)},$$

where the areas of C_i are the portions lying inside Ω .

Definition 33.2 (Optimal Packing). An **optimal packing** is a configuration that achieves the maximum possible density for a given number of circles or a given container size. For n congruent circles in the plane, the problem of determining the optimal arrangement is open for most values of $n > 7$.

Definition 33.3 (Circle Contact Graph). The **circle contact graph** of a packing is a graph $G = (V, E)$ where each node $v_i \in V$ corresponds to a circle C_i and an edge $(v_i, v_j) \in E$ exists if and only if circles C_i and C_j are tangent (touching but not overlapping).

Definition 33.4 (Hexagonal Packing). **Hexagonal packing** is the densest possible packing of congruent circles in an infinite 2D plane, achieving a density of $\delta = \frac{\pi}{2\sqrt{3}} \approx 0.9069$ (approximately 90.69%).

33.1.3 Theory

Finding the optimal packing of n circles in a container is a **non-convex optimization** problem. The most common approaches are:

1. **Greedy Placement**: Placing circles one by one at the best available position (e.g., using a “front-advancing” algorithm).

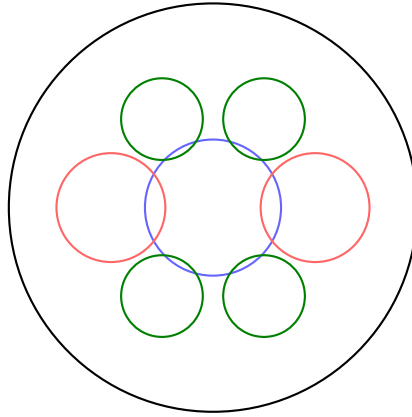
2. **Force-Directed Layout:** Treating circles as repelling particles and using a physics simulation to find an equilibrium state where they are packed tightly.
3. **Circle Packing on Surfaces:** Using **conformal mapping** and the Circle Packing Theorem to represent any planar graph as a set of tangent circles.
4. **Vortex-based Packing:** Packing circles into a spiral or vortex to fill a circular container.

For identical circles, the hexagonal lattice is optimal in 2D. For circles of different sizes (Apollonian gaskets), the packing can achieve 100% density in the limit.

Theorem 33.5 (Circle Packing Theorem). (*Koebe–Andreev–Thurston*) *Every connected planar graph G can be represented as the tangency graph of a circle packing in the plane (or on the sphere), and such a representation is unique up to Möbius transformations.*

Proof. The proof proceeds in two stages. First, Andreev’s theorem provides existence for triangulations using a discrete analogue of the Riemann mapping theorem. Second, Koebe’s original result [Koe36] establishes that every planar graph is a subgraph of a triangulation. Thurston [Thu85] gave an influential expository account combining these ideas with conformal mapping theory. See Stephenson [Ste05, Chapter 3] for a self-contained modern treatment. $\square$

Corollary 33.6. *The hexagonal lattice packing of congruent circles in $\mathbb{R}^2$ is optimal among all packings of congruent circles, with density $\frac{\pi}{2\sqrt{3}}$.*



Non-overlapping disks, maximally packed

Figure 33.1: Circle packing: placing non-overlapping disks of varying radii inside a bounding region.

Example 33.7 (Greedy Packing Trace). Consider packing circles of radii $r_1 = 1.0$, $r_2 = 0.8$, $r_3 = 0.6$ into a 4×3 rectangle, sorted in decreasing order:

1. Place C_1 at $(1.0, 1.0)$ —the first position that fits within the container boundary.
2. Place C_2 at $(2.8, 1.0)$ —the leftmost position with y -coordinate equal to the lowest feasible row, and $\text{dist}((2.8, 1.0), (1.0, 1.0)) = 1.8 = 1.0 + 0.8$ (tangent to C_1).
3. Place C_3 at $(1.0, 2.6)$ —directly above C_1 , with $\text{dist} = 1.6 = 1.0 + 0.6$ (tangent to C_1), and $\text{dist}((1.0, 2.6), (2.8, 1.0)) = 2.0 > 0.8 + 0.6$ (non-overlapping with C_2).

The final packing density is $\frac{\pi(1.0^2+0.8^2+0.6^2)}{12} = \frac{2.0\pi}{12} \approx 52.4\%$.

33.1.4 Pseudocode

Simple Greedy Packing (into a rectangle)

```

1: function GREEDYCIRCLEPACKING(container, radii)
2:   radii.sort(reverse = True)
3:   placed_circles  $\leftarrow$  []
4:   for  $r$  in radii do
5:     best_p  $\leftarrow$  FindCandidatePosition(container, placed_circles,  $r$ )
6:     if best_p  $\neq$  None then
7:       placed_circles.append(Circle(best_p,  $r$ ))
8:     end if
9:   end for
10:  return placed_circles
11: end function
12: function FINDCANDIDATEPOSITION(container, others,  $r$ )
13:  for  $p$  in SamplePoints(container) do
14:    if  $\neg$  IsWithin( $p, r$ , container) then
15:      continue
16:    end if
17:    is_overlapping  $\leftarrow$  False
18:    for  $c$  in others do
19:      if Distance( $p, c.p$ )  $< (r + c.r)$  then
20:        is_overlapping  $\leftarrow$  True
21:        break
22:      end if
23:    end for
24:    if  $\neg$  is_overlapping then
25:      return  $p$ 
26:    end if
27:  end for
28:  return None
29: end function

```

33.1.5 Parameter Selection

- **Radii Distribution:** Can be fixed (monodisperse) or varying (polydisperse).
- **Optimization Iterations:** In force-directed methods, more iterations lead to tighter packing but increase computation time.

33.1.6 Complexity

- **Greedy Approach:** $O(n^2)$ if each placement is checked against all previous circles. This can be reduced to $O(n \log n)$ using a spatial index like a quadtree.
- **Force-Directed:** $O(i \cdot n \log n)$ where i is the number of iterations and n is the number of circles.

33.1.7 Usages

- **Manufacturing:** Minimizing waste when cutting circular parts from a rectangular sheet of material.

- **Cartography:** Dorling Cartograms, where regions are represented by circles whose size is proportional to a variable (e.g., population).
- **Data Visualization:** Packing circles into categories to represent hierarchy (e.g., D3.js circle packing).
- **Material Science:** Modeling the structure of crystals or granular materials.
- **Logo Design and Art:** Generative art and “bubble” patterns.

33.1.8 Testing Methods and Edge Cases

- **Overlap Check:** Verify that no two circles have an intersection.
- **Boundary Check:** Ensure all circles are fully within the container.
- **Density Calculation:** Measure the final packing efficiency.
- **Single Circle:** Should be placed in the center of the container.
- **Radius > Container:** Should result in zero placed circles.
- **Varying Container Shapes:** Test packing into circles, triangles, or complex polygons.

33.1.9 References

- Stephenson, K. (2005). “Introduction to Circle Packing: The Theory of Discrete Analytic Functions”. Cambridge University Press.
- Koebe, P. (1936). “Kontaktprobleme der Konformen Abbildung”. *Berichte Sächs. Akad. Wiss. Leipzig*.
- Thurston, W. (1985). “The Finite Riemann Mapping Theorem”. *International Congress of Mathematicians*.
- Wikipedia: [Circle packing](#)

33.2 Rectangle Packing

33.2.1 Overview

Rectangle packing is the problem of arranging a set of rectangles of varying sizes within a larger container rectangle such that no two rectangles overlap and the total wasted space is minimized (or the density is maximized). This is a classic NP-hard optimization problem with wide-ranging applications in manufacturing, logistics, and computer graphics. Comprehensive surveys of 2D packing methods are given by Jylänki [Jyl10] and Lodi, Martello, and Monaci [LMM02].

33.2.2 Definitions

Definition 33.8 (Bin Packing). The **bin packing** problem asks: given a set of objects with prescribed sizes and a set of bins of fixed capacity, pack all objects into the minimum number of bins such that no two objects in the same bin overlap. The 2D version is strongly NP-hard.

Definition 33.9 (Strip Packing). The **strip packing** problem asks: given a set of rectangles and a container of fixed width W and unbounded height, arrange the rectangles (without rotation or with 90° rotation) to minimize the total height used. The decision version is NP-complete.

Definition 33.10 (Guillotine Cut). A **guillotine cut** is a straight-line cut that goes entirely from one edge of a rectangular piece to the opposite edge, dividing it into two smaller rectangles. A packing is *guillotine-cuttable* if it can be obtained by a sequence of such cuts.

Definition 33.11 (Skyline). A **skyline** representation tracks the upper boundary of the packed rectangles as a piecewise-constant function of x . New rectangles are placed at the lowest point on the skyline, and the skyline is updated accordingly.

33.2.3 Theory

Because rectangle packing is NP-hard, heuristic and metaheuristic algorithms are used in practice.

1. **Shelf Algorithms:** Rectangles are placed in “shelves” (rows). When a rectangle doesn’t fit in the current shelf, a new shelf is started above it. (e.g., Next-Fit Decreasing Height).
2. **Guillotine Algorithms:** The container is recursively split into two smaller rectangles using horizontal or vertical cuts.
3. **MaxRects:** Maintains a list of all maximal free rectangular areas. A new rectangle is placed into one of these areas, and the remaining free spaces are recalculated.
4. **Skyline Algorithms:** Tracks the top boundary of the packed rectangles. New rectangles are placed on the lowest part of the skyline.

Sorting the rectangles by height or area before packing (e.g., First-Fit Decreasing) significantly improves the performance of most heuristics.

Theorem 33.12 (NP-Hardness of Rectangle Packing). *The 2D bin packing problem and the 2D strip packing problem are both NP-hard in the strong sense.*

Proof. The NP-hardness of 2D bin packing follows by reduction from the 3-Partition problem, which is strongly NP-complete. Given a multiset $S = \{a_1, \dots, a_{3m}\}$ of integers with $\frac{B}{4} < a_i < \frac{B}{2}$ and $\sum a_i = mB$, construct $3m$ rectangles of sizes $a_i \times 1$ to be packed into m bins of size $B \times 1$. The constraint $\frac{B}{4} < a_i < \frac{B}{2}$ ensures exactly three rectangles must fit per bin, reducing 3-Partition to bin packing; see [LMM02, Section 8]. $\square$

Example 33.13 (MaxRects Trace). Consider packing rectangles $A = 3 \times 2$, $B = 2 \times 2$, $C = 1 \times 1$ into a 5×4 bin using Best Short Side Fit (BSSF):

1. Free list: $\{(0, 0, 5, 4)\}$. Sort: $A(3 \times 2)$, $B(2 \times 2)$, $C(1 \times 1)$ by decreasing height.
2. Place A at $(0, 0)$. Short-side fit: $\min(5 - 3, 4 - 2) = 2$. Split free rect $(0, 0, 5, 4)$:
 - Right remainder: $(3, 0, 2, 4)$
 - Top remainder: $(0, 2, 3, 2)$
3. Place B at $(3, 0)$ in free rect $(3, 0, 2, 4)$. Fit: $\min(0, 2) = 0$. Split:
 - Top remainder: $(3, 2, 2, 2)$
4. Place C at $(0, 2)$ in free rect $(0, 2, 3, 2)$. Fit: $\min(2, 1) = 1$.

Total density: $\frac{6+4+1}{20} = 55\%$.

```
1: function MAXRECTS(bin_width, bin_height, rectangles)
2:   free_rects  $\leftarrow$  [Rectangle(0, 0, bin_width, bin_height)]
3:   placed_rects  $\leftarrow$  []
4:   rectangles.sort(key =  $\lambda r$ :  $r.h$ , reverse = True)
5:   for  $r$  in rectangles do
6:     best_fit  $\leftarrow$  None
7:     min_short_side_fit  $\leftarrow$   $\infty$ 
8:     for  $f$  in free_rects do
9:       if  $f.w \geq r.w$  and  $f.h \geq r.h$  then
10:        fit  $\leftarrow$  min( $f.w - r.w$ ,  $f.h - r.h$ )
11:        if fit < min_short_side_fit then
12:          min_short_side_fit  $\leftarrow$  fit
13:          best_fit  $\leftarrow$   $f$ 
14:        end if
15:      end if
16:    end for
17:    if best_fit  $\neq$  None then
18:       $r.x, r.y \leftarrow$  best_fit.x, best_fit.y
19:      placed_rects.append( $r$ )
20:      new_free  $\leftarrow$  []
21:      for  $f$  in free_rects do
22:        new_free.extend(SplitFreeRect( $f, r$ ))
23:      end for
24:      free_rects  $\leftarrow$  PruneRedundant(new_free)
25:    end if
26:  end for
27:  return placed_rects
28: end function
```

33.2.4 Pseudocode

MaxRects (BSSF — Best Short Side Fit)

33.2.5 Parameter Selection

- **Rotation:** Allowing 90° rotation can significantly increase packing density.
- **Heuristic Choice:** **MaxRects** is generally considered the best-performing heuristic for 2D packing, while **Guillotine** is faster but less efficient.

33.2.6 Complexity

- **Heuristic Packing:** $O(n^2)$ for simple heuristics like First-Fit. MaxRects can be $O(n^3)$ in the worst case as the free list grows.
- **Optimal Packing:** Exponential $O(2^n)$, only feasible for small n (e.g., $n < 20$).

33.2.7 Usages

- **Manufacturing:** Cutting parts from a sheet of wood, metal, or fabric (Nesting).
- **Sprite Sheet Generation:** Packing many small game icons into a single large texture to improve GPU performance.

- **Logistics:** Loading pallets or shipping containers.
- **UI Layout:** Arranging windows or widgets in a tile-based interface.
- **VLSI Design:** Floorplanning of electronic components on a chip.

33.2.8 Testing Methods and Edge Cases

- **Overlap Check:** Verify that no two rectangles in the final layout intersect.
- **Container Boundary:** Ensure all rectangles are within the bin.
- **Density Calculation:** Calculate (Total Rectangle Area) / (Container Area).
- **Large Rectangles:** Test cases where one rectangle takes up most of the bin.
- **Thin Rectangles:** Test “sliver” rectangles that are hard to pack.
- **Rotation:** Verify that rotating a rectangle 90° is handled correctly by the splitting logic.

33.2.9 References

- Jylänki, J. (2010). “A Thousand Ways to Pack the Bin — A Practical Approach to Two-Dimensional Rectangle Bin Packing”. Technical Report.
- Lodi, A., Martello, S., & Monaci, M. (2002). “Two-dimensional packing problems: A survey”. European Journal of Operational Research.
- Wikipedia: [Packing problems](#)

33.3 Hopper Optimization

33.3.1 Overview

Hopper optimization is a specialized technique for packing non-convex polygons into a rectangular container (the “hopper”) such that the vertical height used is minimized. Unlike standard bin packing, which often uses bounding boxes, hopper optimization uses the exact geometry of the polygons, often leveraging the **No-Fit Polygon (NFP)** to find the tightest possible nesting. It is widely used in the textile, leather, and metal industries where material waste must be minimized. See Bennell and Oliveira [BO08] for a comprehensive tutorial and Burke et al. [BHKW07] for robust NFP computation.

33.3.2 Definitions

Definition 33.14 (Hopper). A **hopper** is a semi-infinite rectangular region $\{(x, y) \mid 0 \leq x \leq W, y \geq 0\}$ of fixed width W and unbounded height, into which polygons are placed from the top and allowed to slide down under gravity.

Definition 33.15 (No-Fit Polygon). The **No-Fit Polygon (NFP)** $\text{NFP}(A, B)$ of two convex polygons A (the *stationary* piece) and B (the *orbiting* piece) is the locus of all positions of a reference point on B such that B is translated to touch A without overlapping. The interior of $\text{NFP}(A, B)$ represents infeasible placements; the boundary represents tangent placements.

Definition 33.16 (Gravity Heuristic). The **gravity heuristic** places polygons into the hopper one at a time, allowing each polygon to “fall” as far down as possible (minimizing y), then as far left as possible (minimizing x), subject to non-overlap constraints with previously placed polygons and the hopper walls.

33.3.3 Theory

The core of hopper optimization is determining the **feasible placement** of a new polygon P_{new} relative to already placed polygons $\{P_1, \dots, P_k\}$.

1. **NFP Calculation:** For every pair of polygons (P_i, P_{new}) , the No-Fit Polygon $\text{NFP}_{i,\text{new}}$ is computed. This polygon defines the forbidden region for the reference point of P_{new} .
2. **Feasible Space:** The set of all possible positions for P_{new} is the area inside the hopper minus the union of all NFPs.
3. **Optimization:** The algorithm searches for a position in the feasible space that minimizes the current maximum height of the packing.

A common approach is the **Bottom-Left-Fill (BLF)** heuristic, which always places the next polygon at the lowest and then leftmost available position.

Theorem 33.17 (NFP Correctness). *For two convex polygons A with n vertices and B with m vertices, the NFP can be computed in $O(n + m)$ time. The NFP is itself a convex polygon with at most $n + m$ vertices, and a placement of B relative to A is valid if and only if the reference point of B lies outside $\text{int}(\text{NFP}(A, B))$.*

Proof. The NFP of two convex polygons is obtained by computing the Minkowski sum of A and the reflected polygon $-B$. Since the Minkowski sum of two convex polygons with n and m vertices is a convex polygon with at most $n + m$ vertices, and can be computed in $O(n + m)$ time by merging edge angle sequences, the result follows. A reference point placement is valid precisely when B does not overlap A , which corresponds to the reference point lying outside the interior of the Minkowski sum. $\square$

33.3.4 Pseudocode

```

1: function HOPPEROPTIMIZE(hopper_width, polygons)
2:   polygons.sort(key =  $\lambda p: p.\text{area}$ , reverse = True)
3:   placed_polygons  $\leftarrow$  []
4:   for  $p_{\text{new}}$  in polygons do
5:     best_pos  $\leftarrow$  FindLowestPosition( $p_{\text{new}}$ , placed_polygons, hopper_width)
6:      $p_{\text{new}}.\text{position} \leftarrow$  best_pos
7:     placed_polygons.append( $p_{\text{new}}$ )
8:   end for
9:   return GetMaxHeight(placed_polygons)
10: end function
11: function FINDLOWESTPOSITION( $p_{\text{new}}$ , placed, width)
12:   forbidden_region  $\leftarrow$  Union([ComputeNFP( $p_i, p_{\text{new}}$ ) |  $p_i \in$  placed])
13:   return  $\text{argmin}_{p \in \text{forbidden\_region.boundary}}(p.y, p.x)$ 
14: end function

```

33.3.5 Parameter Selection

- **Sorting Rule:** Decreasing area or decreasing height are standard.
- **Rotation:** Allowing discrete rotations (e.g., 0° , 90° , 180° , 270°) or continuous rotation significantly improves results but increases complexity.
- **NFP Precision:** For complex curved shapes, polygons are often simplified or sampled.

33.3.6 Complexity

- **NFP Calculation:** $O(n \cdot m)$ for two convex polygons with n and m vertices. For non-convex, it can be much higher.
- **Packing Heuristic:** $O(k^2)$ where k is the number of polygons, assuming NFP lookups are optimized.

33.3.7 Usages

- **Textile Industry:** Nesting clothing patterns onto a roll of fabric.
- **Sheet Metal Cutting:** Arranging mechanical parts on a metal plate for laser or waterjet cutting.
- **Leather Industry:** Packing parts onto irregular animal hides (which includes avoiding defects).
- **Shipbuilding:** Cutting large steel plates for hulls.
- **Furniture Manufacturing:** Maximizing the use of plywood or wood planks.

33.3.8 Testing Methods and Edge Cases

- **Convex vs. Non-Convex:** Verify that the algorithm handles interlocking U-shapes or L-shapes correctly.
- **Container Width:** Ensure no polygon extends beyond the hopper walls.
- **Overlap Check:** Rigorous verification that no two polygons intersect.
- **Hopper Height:** The primary metric to minimize.
- **Large Quantity:** Test with hundreds of small pieces to ensure stability.

33.3.9 References

- Bennell, J. A., & Oliveira, J. F. (2008). “The geometry of nesting problems: A tutorial”. *European Journal of Operational Research*.
- Burke, E. K., Hellier, R. S., Kendall, G., & Whitwell, G. (2007). “Complete and robust no-fit polygon generation for the irregular cutting and packing problem”. *European Journal of Operational Research*.
- Wikipedia: [Nesting \(process\)](#)

Chapter 34

Dynamic and Kinetic Geometry

- 34.1 Dynamic Geometric Problems
- 34.2 Dynamic Point Sets
- 34.3 Dynamic Convex Hulls
- 34.4 Dynamic Proximity Structures
- 34.5 Moving Geometric Objects
- 34.6 Kinetic Data Structures
- 34.7 Certificates and Events
- 34.8 Kinetic Sorting
- 34.9 Kinetic Convex Hulls
- 34.10 Kinetic Closest Pairs
- 34.11 Applications to Tracking and Simulation
- 34.12 Exercises

Part XI

Applications

Chapter 35

Computational Geometry for Computer Graphics

- 35.1 Geometric Modeling
- 35.2 Spatial Acceleration Structures
- 35.3 Bounding Volume Hierarchies
- 35.4 Ray–Object Intersection
- 35.5 Hidden-Surface Problems
- 35.6 Clipping
- 35.7 Mesh Processing
- 35.8 Collision Detection
- 35.9 Real-Time Geometry
- 35.10 GPU-Based Geometric Computation
- 35.11 Case Study: Geometry Pipeline
- 35.12 Project

Chapter 36

Computational Geometry for GIS

- 36.1 Geographic Data
- 36.2 Spatial Indexing
- 36.3 Map Overlay
- 36.4 Point-in-Region Queries
- 36.5 Proximity Analysis
- 36.6 Voronoi-Based Service Regions
- 36.7 Terrain Representation
- 36.8 Triangulated Irregular Networks
- 36.9 Digital Elevation Models
- 36.10 Map Generalization
- 36.11 Large-Scale Spatial Data
- 36.12 Case Study: Geographic Facility Location
- 36.13 Project

Chapter 37

Computational Geometry for Robotics

- 37.1 Robot Geometry
- 37.2 Collision Detection
- 37.3 Configuration Spaces
- 37.4 Path Planning
- 37.5 Sensor Geometry
- 37.6 Localization
- 37.7 Mapping
- 37.8 SLAM and Geometric Structures
- 37.9 Manipulation Planning
- 37.10 Autonomous Navigation
- 37.11 Case Study: Mobile-Robot Planning
- 37.12 Project

Chapter 38

Computational Geometry for Scientific Computing

38.1 Geometry in Numerical Simulation

38.2 Mesh Generation

38.3 Finite-Element Geometry

38.4 Domain Discretization

38.5 Adaptive Refinement

38.6 Particle Methods

38.7 Nearest-Neighbor Computations

38.8 Spatial Decomposition

38.9 Parallel Geometric Algorithms

38.10 Large-Scale Scientific Geometry

38.11 Case Study: PDE Mesh Construction

38.12 Project

38.13 Stochastic and Meshless PDE Methods

38.14 Stochastic PDE Solvers

38.14.1 Overview

Stochastic PDE solvers use Monte Carlo methods to solve partial differential equations (like Laplace or Poisson) on geometric domains [SC20].

Unlike standard grid-based solvers (Finite Element/Difference), which require a high-quality global mesh, stochastic solvers like **Walk on Spheres (WoS)** and **Walk on Stars (WoSt)**

can solve for the value at a single point without constructing a global system of equations. This is particularly useful for complex or dirty geometries.

Definition 38.1 (Walk on Spheres). Walk on Spheres is a Monte Carlo method for solving the Laplace equation $\Delta u = 0$ in a domain Ω . Starting from an interior point x , a random walk iteratively jumps to a uniformly random point on the largest inscribed sphere centered at the current position, until the walk reaches the boundary $\partial\Omega$, where the Dirichlet data is evaluated.

Definition 38.2 (Walk on Stars). Walk on Stars generalizes WoS by allowing jumps to uniformly random points on the intersection of the largest inscribed sphere with a cone defined by the star-shaped region around the current point. This extension enables estimation of derivatives of the solution and supports Neumann boundary conditions [Y+25].

38.14.2 Implementation Details

The CompGeom implementation includes:

1. Walk on Spheres (WoS):

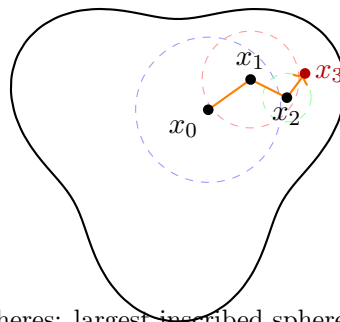
- **Laplace Solver:** Estimates $u(x)$ where $\Delta u = 0$.
- **Poisson Solver:** Estimates $u(x)$ where $\Delta u = f$ using the Walk on Balls variant with the Green's function representation.

2. Walk on Stars (WoSt):

- **Gradient Estimator:** Robustly estimates $\nabla u(x)$ using the Boundary Integral Equation (BIE).

Theorem 38.3 (WoS Convergence). *For a bounded domain Ω with Lipschitz boundary and continuous Dirichlet data g , the Walk on Spheres estimator converges almost surely to the unique harmonic solution $u(x)$ of $\Delta u = 0$ in Ω with $u|_{\partial\Omega} = g$, as the number of random walks $M \rightarrow \infty$.*

Proof. The proof relies on the mean value property of harmonic functions. At each step, the expected value of u at a random point on the sphere of radius r centered at x equals $u(x)$ by harmonicity. Therefore, the Walk on Spheres process is a martingale. By the martingale convergence theorem, the process converges as the number of steps grows. The boundary hitting point converges to $\partial\Omega$ because the domain is bounded, so the inscribed sphere radii decrease. The strong law of large numbers then gives almost sure convergence of the Monte Carlo average over M independent walks. $\square$



Walk on Spheres: largest inscribed sphere at each step

Figure 38.1: Walk on Spheres: from each interior point, jump to a random point on the largest inscribed sphere.

Algorithm 99 Stochastic PDE Solver Usage

```

1: from compgeom.mesh.algorithms.stochastic_pde import WalkOnSpheres,
   WalkOnStars
2: from compgeom import Point3D

3: # Initialize on a surface mesh
4: wos = WalkOnSpheres(mesh)
5: value = wos.solve_laplace(Point3D(0.5, 0.5, 0.5), boundary_func)

6: # Robust gradient estimation
7: wost = WalkOnStars(mesh)
8: grad = wost.solve_gradient(Point3D(0.5, 0.5, 0.5), boundary_func)

```

38.14.3 Usage

Example 38.4 (WoS on a Unit Disk). Consider the Laplace equation $\Delta u = 0$ on the unit disk $\Omega = \{(x, y) : x^2 + y^2 < 1\}$ with boundary data $g(\theta) = \cos(2\theta)$. The exact solution is $u(r, \theta) = r^2 \cos(2\theta)$. To estimate $u(0.5, 0)$ via WoS, we start at $(0.5, 0)$, which has inscribed sphere radius 0.5. A random walk jumps to a point on this sphere, then from the new position computes its inscribed sphere, and so on until hitting $\partial\Omega$. Repeating for $M = 10,000$ walks and averaging the boundary values yields an estimate close to $u(0.5, 0) = 0.25 \cdot 1 = 0.25$, demonstrating the convergence stated in Theorem 38.3.

Remark 38.5. The variance of WoS estimators decreases as $O(1/M)$ where M is the number of random walks. For Laplace’s equation, the walk is guaranteed to terminate almost surely for bounded domains. For Poisson’s equation, an additional correction term accumulates along each walk, increasing variance.

38.14.4 References

- Sawhney, R., & Crane, K. “Monte Carlo Geometry Processing: A Meshless Approach to Geometric PDE.” *ACM Transactions on Graphics (SIGGRAPH)*, 2020 [SC20].
- Yu, Y., et al. “Robust Derivative Estimation with Walk on Stars.” *ACM Transactions on Graphics (SIGGRAPH)*, 2025 [Y+25].

38.15 Wave Kernel Signature**38.15.1 Overview**

The Wave Kernel Signature (WKS) is a multi-scale geometric descriptor for vertices on a 3D mesh. Like the Heat Kernel Signature (HKS), it is based on the spectral properties of the Laplace–Beltrami operator. However, while HKS models heat diffusion (essentially a low-pass filter), WKS models the evolution of a quantum particle (wave function) with a specific energy distribution. This makes WKS better at capturing high-frequency geometric details and provides superior performance in non-rigid shape matching [ASC11].

38.15.2 Definitions

Definition 38.6 (Schrödinger Equation). The time-dependent Schrödinger equation on a Riemannian manifold (M, g) governs the evolution of a quantum state: $i\hbar \frac{\partial \psi}{\partial t} = -\Delta_M \psi$, where Δ_M is the Laplace–Beltrami operator Theorem 24.1 and $\hbar$ is the reduced Planck constant (set to 1 in the geometric setting).

Definition 38.7 (Wave Kernel Signature). The Wave Kernel Signature at vertex x and energy scale e is defined as

$$W(x, e) = \sum_{k=1}^{\infty} g_e(\lambda_k)^2 \phi_k(x)^2$$

where $\{\lambda_k\}$ and $\{\phi_k\}$ are the eigenvalues and orthonormal eigenfunctions of the Laplace–Beltrami operator, and g_e is an energy band-pass filter centered at energy e [ASC11].

Definition 38.8 (Energy Distribution). The energy distribution (or spectral filter) $g_e(\lambda)$ is typically a log-normal distribution:

$$g_e(\lambda) = \exp\left(-\frac{(\log e - \log \lambda)^2}{2\sigma^2}\right)$$

This ensures that the descriptor is sensitive to relative energy differences, providing a balanced representation of both coarse and fine geometric features [ASC11, SOG09].

38.15.3 Theory

The Wave Kernel Signature $W(x, e)$ at vertex x and energy scale e is defined as the average probability of a particle with energy distribution g_e being found at x :

$$W(x, e) = \sum_{k=1}^{\infty} g_e(\lambda_k)^2 \phi_k(x)^2$$

where λ_k and ϕ_k are the eigenvalues and eigenfunctions of the Laplace–Beltrami operator.

The filter g_e is usually a log-normal distribution Theorem 38.8:

$$g_e(\lambda) = \exp\left(-\frac{(\log e - \log \lambda)^2}{2\sigma^2}\right)$$

This ensures that the descriptor is sensitive to relative energy differences, providing a balanced representation of both coarse and fine geometric features.

The WKS can be interpreted as the diagonal of the spectral projection operator onto the energy band $[e - \sigma, e + \sigma]$. Unlike the HKS (which integrates over all time and thus acts as a low-pass filter), the WKS selects a specific frequency band, making it a “band-pass” descriptor that captures geometric information at a particular scale.

Theorem 38.9 (WKS Isometry Invariance). *The Wave Kernel Signature Theorem 38.7 is invariant under isometric deformations of the surface. That is, if (M, g) and (M, g') are isometric Riemannian manifolds with corresponding eigenpairs $\{(\lambda_k, \phi_k)\}$ and $\{(\lambda'_k, \phi'_k)\}$, then $W(x, e) = W'(f(x), e)$ for any isometry $f : M \rightarrow M'$.*

Proof. An isometry preserves the Laplace–Beltrami operator: $\Delta_{g'} = \Delta_g$ after the pullback by f . Hence the eigenvalues λ_k are identical ($\lambda'_k = \lambda_k$), and the eigenfunctions satisfy $\phi'_k(f(x)) = \phi_k(x)$. The WKS formula depends only on λ_k , $g_e(\lambda_k)$, and $\phi_k(x)^2$, all of which are invariant under the isometry. Therefore $W'(f(x), e) = \sum_k g_e(\lambda_k)^2 \phi'_k(f(x))^2 = \sum_k g_e(\lambda_k)^2 \phi_k(x)^2 = W(x, e)$. $\square$

Observation 38.10. While WKS Theorem 38.9 is isometry-invariant, it is *not* scale-invariant. Scaling the metric by a constant factor c scales eigenvalues as $\lambda_k \rightarrow c^{-2}\lambda_k$, which shifts the WKS along the energy axis. In contrast, the HKS with logarithmic time normalization achieves approximate scale invariance [BK10].

Algorithm 100 Wave Kernel Signature

```

1: function COMPUTEWKS(mesh, num_basis=100, num_scales=100)
2:    $L, M \leftarrow \text{BuildDiscreteLaplacian}(\text{mesh})$ 
3:    $\text{evals}, \text{Phis} \leftarrow \text{scipy.sparse.linalg.eigsh}(L, M, k=\text{num\_basis}, \sigma=-1e-6)$ 
4:    $e_{\min} \leftarrow \log(\text{evals}[1])$  ▷ Skip zero eigenvalue
5:    $e_{\max} \leftarrow \log(\text{evals}[-1])$ 
6:    $\text{scales} \leftarrow \text{np.linspace}(e_{\min}, e_{\max}, \text{num\_scales})$ 
7:    $\sigma \leftarrow (e_{\max} - e_{\min}) / \text{num\_scales} \times 7.0$  ▷ Heuristic width
8:    $\text{wks} \leftarrow \text{np.zeros}((\text{num\_vertices}, \text{num\_scales}))$ 
9:   for  $i, e \in \text{enumerate}(\text{scales})$  do
10:     $\text{weights} \leftarrow \exp(-(e - \log(\text{evals}))^2 / (2\sigma^2))$ 
11:     $\text{weights} \leftarrow \text{weights} / \sum(\text{weights})$ 
12:     $\text{wks}[:, i] \leftarrow (\text{Phis}^2) \cdot \text{weights}$ 
13:   end for
14:   return  $\text{wks}$ 
15: end function

```

38.15.4 Algorithm

Example 38.11 (WKS on a Sphere). On a unit sphere, the eigenvalues of the Laplace–Beltrami operator are $\lambda_k = k(k+1)$ for $k = 0, 1, 2, \dots$, with eigenfunctions being the spherical harmonics Y_k^m . Since the sphere is completely symmetric, the WKS at every vertex is identical: $W(x, e) = \sum_{k=0}^{\infty} g_e(k(k+1))^2 \sum_{m=-k}^k |Y_k^m(x)|^2$. By the addition theorem for spherical harmonics, $\sum_m |Y_k^m(x)|^2 = (2k+1)/(4\pi)$, which is constant in x . Therefore W is the same at all vertices—as expected for a shape with trivial isometry group orbits.

38.15.5 Parameter Selections

- **Number of Eigenfunctions (k):** Typically 100–300. More eigenfunctions are needed than for HKS to capture the “wave” behavior.
- **Filter Width (σ):** Controls the trade-off between spatial and spectral localization.
- **Scale Range:** Usually spans the entire computed spectrum in log-space.

38.15.6 Complexity

- **Eigen-decomposition:** $O(N^{1.5})$ for sparse matrices using Lanczos or LOBPCG solvers.
- **Signature Calculation:** $O(N \cdot k \cdot S)$ where S is the number of scales.
- **Space Complexity:** $O(N \cdot S)$ to store the descriptors.

38.15.7 Usages

- **Non-Rigid Shape Matching:** Finding corresponding points between different poses of the same object (e.g., a human walking) [ASC11].
- **Shape Retrieval:** Creating a global descriptor by pooling WKS values across the mesh [B⁺08].
- **Symmetry Detection:** Identifying vertices with similar geometric contexts.
- **Segment Classification:** Training machine learning models to identify parts of a mesh (e.g., “head,” “arm,” “leg”).

38.15.8 Testing Methods and Edge Cases

- **Isometry Invariance**Theorem 38.9: Verify that WKS is identical for a mesh in two different isometric poses.
- **Scale Invariance**: WKS is NOT naturally scale-invariant (unlike HKS with normalization). Test how it behaves when the mesh is scaled.
- **Locality**: Verify that WKS correctly identifies high-curvature regions (like fingertips or nose) as having distinct signatures.
- **Low Resolution**: Ensure the signature remains stable even on coarse triangulations.
- **Spectral Truncation**: Observe how the descriptor quality degrades as the number of basis functions k is reduced.

38.15.9 References

- Aubry, M., Schlickewei, U., & Cremers, D. (2011). “The wave kernel signature: A quantum mechanical approach to shape analysis.” *ICCV Workshops* [ASC11].
- Sun, J., Ovsjanikov, M., & Guibas, L. (2009). “A concise and provably informative multi-scale signature based on heat diffusion.” *Computer Graphics Forum* [SOG09].
- Bronstein, M. M., & Kokkinos, I. (2010). “Scale-invariant heat kernel signatures for non-rigid shape retrieval.” *CVPR* [BK10].
- Wikipedia: Spectral shape analysis.

Part XII

Modern and Research-Level Topics

Chapter 39

High-Dimensional Computational Geometry

39.1 Geometry beyond Three Dimensions

39.2 High-Dimensional Convex Hulls

39.3 Voronoi and Delaunay Structures

39.4 Nearest-Neighbor Search

39.5 Dimensionality Reduction

39.6 Johnson–Lindenstrauss Lemma

39.7 Random Projections

39.8 Concentration of Measure

39.9 Curse of Dimensionality

39.10 Applications to Machine Learning

39.11 Exercises

Chapter 40

Geometry of Data and Machine Learning

- 40.1 Geometric View of Data
- 40.2 Metric Spaces
- 40.3 Nearest-Neighbor Classification
- 40.4 Clustering Geometry
- 40.5 Convex Geometry in Machine Learning
- 40.6 Kernel Geometry
- 40.7 Manifold Learning
- 40.8 Geometric Embeddings
- 40.9 Graph-Based Geometric Learning
- 40.10 Optimal Transport: A Geometric Introduction
- 40.11 Computational Geometry for AI
- 40.12 Research Directions
- 40.13 Project

Chapter 41

Research Frontiers

- 41.1 Robust and Certified Geometry
- 41.2 Massive Geometric Data Sets
- 41.3 Streaming Geometry
- 41.4 External-Memory Geometry
- 41.5 Geometric Algorithms for Autonomous Systems
- 41.6 Geometry in Machine Learning
- 41.7 Topological Data Analysis
- 41.8 Shape and Surface Reconstruction
- 41.9 Geometric Deep Learning
- 41.10 Differentiable Geometry
- 41.11 Geometry Processing
- 41.12 Open Problems in Computational Geometry
- 41.13 How to Read Computational Geometry Research
- 41.14 Designing Computational Experiments
- 41.15 From Graduate Study to Research
- 41.16 Computational Algebra and Similarity
- 41.17 Buchberger's Algorithm
 - 41.17.1 Overview

Buchberger's algorithm is a fundamental method in computational algebraic geometry for computing a **Gröbner basis** of a polynomial ideal. Much like the Gaussian elimination

algorithm for linear systems, Buchberger's algorithm transforms a set of multivariate polynomials into a "standard" form that makes it easy to solve systems of equations, perform polynomial division, and analyze the properties of the underlying algebraic variety. The algorithm was first described by Bruno Buchberger in his 1965 PhD thesis [Buc65] and has since become a cornerstone of computational algebra, extensively treated in standard references such as Cox, Little, and O'Shea [CLO07].

41.17.2 Definitions

Definition 41.1 (Polynomial Ideal). Let k be a field and let $f_1, \dots, f_m \in k[x_1, \dots, x_n]$ be a set of polynomials. The **polynomial ideal** generated by these polynomials is the set

$$I = \langle f_1, \dots, f_m \rangle = \left\{ \sum_{i=1}^m h_i f_i \mid h_i \in k[x_1, \dots, x_n] \right\}.$$

The polynomials $f_1, \dots, f_m$ are called the *generators* of I .

Definition 41.2 (Monomial Order). A **monomial order** on $k[x_1, \dots, x_n]$ is a total ordering $\succ$ on the set of monomials $x^\alpha = x_1^{\alpha_1} \cdots x_n^{\alpha_n}$ (where $\alpha \in \mathbb{N}^n$) that satisfies:

1. $x^\alpha \succ 1$ for all $\alpha \neq \mathbf{0}$.
2. If $x^\alpha \succ x^\beta$, then $x^{\alpha+\gamma} \succ x^{\beta+\gamma}$ for all $\gamma \in \mathbb{N}^n$.

Common choices include **lexicographic** (Lex) and **graded reverse lexicographic** (Degrevlex). For two monomials x^α and x^β , the Lex order compares first by x_1 -exponent, then x_2 , etc., while Degrevlex first compares by total degree and then breaks ties by the last differing exponent in reverse.

Definition 41.3 (Leading Term). Given a monomial order $\succ$ and a nonzero polynomial $f = \sum_{\alpha} c_{\alpha} x^{\alpha}$, the **leading term** $\text{LT}(f)$ is the term $c_{\alpha} x^{\alpha}$ with the greatest α under $\succ$. Similarly, $\text{LM}(f)$ denotes the leading monomial (coefficient removed) and $\text{LC}(f)$ the leading coefficient.

Definition 41.4 (Gröbner Basis). A finite set $G = \{g_1, \dots, g_t\} \subset I$ is a **Gröbner basis** for the ideal $I = \langle f_1, \dots, f_m \rangle$ with respect to a monomial order $\succ$ if

$$\langle \text{LT}(g_1), \dots, \text{LT}(g_t) \rangle = \langle \text{LT}(I) \rangle,$$

where $\text{LT}(I) = \{\text{LT}(f) \mid f \in I \setminus \{0\}\}$ is the set of all leading terms of nonzero polynomials in I .

Definition 41.5 (S-Polynomial). Given two nonzero polynomials f, g and a monomial order $\succ$, the **S-polynomial** is defined as

$$S(f, g) = \frac{L}{\text{LT}(f)} f - \frac{L}{\text{LT}(g)} g,$$

where $L = \text{lcm}(\text{LM}(f), \text{LM}(g))$ is the least common multiple of the leading monomials of f and g . The S-polynomial is constructed precisely to cancel the leading terms of f and g .

41.17.3 Theory

The algorithm starts with an initial set of generators G . It systematically checks all pairs of polynomials (f, g) in G and calculates their S-polynomial:

$$S(f, g) = \frac{L}{\text{LT}(f)} f - \frac{L}{\text{LT}(g)} g$$

where $L = \text{lcm}(\text{LM}(f), \text{LM}(g))$.

The S-polynomial is then reduced by the current basis G using multivariate polynomial division. If the remainder is non-zero, it means G is not yet a Gröbner basis, and the remainder is added to G . This process continues until the S-polynomial of every pair in G reduces to zero.

Theorem 41.6 (Buchberger’s Criterion). *A finite generating set G for an ideal I is a Gröbner basis with respect to a monomial order $\succ$ if and only if for every pair $f_i, f_j \in G$, the remainder of the S -polynomial $S(f_i, f_j)$ on division by G is zero; that is, $S(f_i, f_j) \xrightarrow{G} 0$ for all $i \neq j$.*

Proof. The forward direction is straightforward: if G is a Gröbner basis, then $\text{LT}(G)$ generates $\text{LT}(I)$, so every S -polynomial reduces to zero by the division algorithm. The converse relies on the fact that if $S(f_i, f_j) \rightarrow 0$ for all pairs, then the leading terms of G generate $\text{LT}(I)$, which is shown by induction on the monomial order applied to an arbitrary $h \in I$; see [CLO07, Chapter 2] for the complete proof. $\square$

Theorem 41.7 (Termination of Buchberger’s Algorithm). *Buchberger’s algorithm terminates after finitely many steps for any input set of polynomials F and any admissible monomial order.*

Proof. Each time a nonzero remainder r is produced, r is added to G , and $\langle \text{LT}(G) \rangle$ strictly increases in the monomial ideal lattice. Since every monomial ideal is finitely generated (by Dickson’s lemma), this chain of strict inclusions must stabilize, and the algorithm terminates. $\square$

Generator	Leading	Remainder
f_1	x^2y	$S(\cancel{f_1}, f_2) \rightarrow r$
f_2	xy^2	

Figure 41.1: Buchberger’s algorithm: S -polynomials are formed from pairs of generators and reduced.

41.17.4 Pseudocode

```

1: function BUCHBERGER( $F$ , monomial_order)
2:    $G \leftarrow F$ 
3:   pairs  $\leftarrow \{(f_i, f_j) \mid f_i, f_j \in G, i < j\}$ 
4:   while pairs  $\neq \emptyset$  do
5:      $(f, g) \leftarrow \text{pairs.pop}()$ 
6:      $S \leftarrow \text{S.Polynomial}(f, g, \text{monomial\_order})$ 
7:     remainder  $\leftarrow \text{MultivariateDivision}(S, G, \text{monomial\_order})$ 
8:     if remainder  $\neq 0$  then
9:       for each existing_g in  $G$  do
10:        pairs.add((existing_g, remainder))
11:      end for
12:       $G.\text{append}(\text{remainder})$ 
13:    end if
14:  end while
15:  return  $G$ 
16: end function

```

Example 41.8 (Computing an S -Polynomial). Let $f = x^2y - 1$ and $g = xy^2 - 1$ with Lex order ($x \succ y$). Then $\text{LT}(f) = x^2y$ and $\text{LT}(g) = xy^2$, so $L = \text{lcm}(x^2y, xy^2) = x^2y^2$. The S -polynomial is

$$S(f, g) = \frac{x^2y^2}{x^2y}f - \frac{x^2y^2}{xy^2}g = y \cdot f - x \cdot g = (x^2y^2 - y) - (x^2y^2 - x) = x - y.$$

Since $x - y$ is nonzero, it would be added to the basis G during Section 41.17.4.

41.17.5 Parameter Selection

- **Monomial Order: Lexicographic** (Lex) is used for solving systems of equations, while **Graded Reverse Lexicographical** (Degrevlex) is generally much faster for computing the basis.
- **Selection Heuristic:** Choosing pairs with the smallest LCM of leading terms can improve performance.

41.17.6 Complexity

- **Time Complexity:** In the worst case, the algorithm's complexity can be **doubly exponential** in the number of variables n . More precisely, for a zero-dimensional ideal with n variables and maximum degree d , the number of polynomials in a Gröbner basis can be $O(d^{2^n})$.
- **Space Complexity:** Can also grow extremely large as many intermediate polynomials are generated.

41.17.7 Usages

- **Solving Polynomial Systems:** Transforming a complex system into a Lex-ordered Gröbner basis (which looks like a “triangular” system) and solving by back-substitution.
- **Surface Implicitization:** Converting a parametric surface (e.g., Bézier or NURBS) into an implicit equation $f(x, y, z) = 0$.
- **Collision Detection:** Determining if two curved surfaces (defined by algebraic equations) intersect.
- **Robot Kinematics:** Solving the inverse kinematics problem for robot arms with multiple joints.
- **Motion Planning:** Checking for intersections between objects in configuration space.

41.17.8 Testing Methods and Edge Cases

- **Linear Systems:** On a system of linear equations, Buchberger should perform identically to Gaussian elimination.
- **Single Variable:** On polynomials in x , it should behave like the Euclidean algorithm for finding the GCD.
- **Inconsistent Systems:** If the system has no solution, the Gröbner basis should contain the constant 1.
- **Redundant Generators:** The algorithm should eliminate redundant polynomials.
- **Monomial Order Sensitivity:** Verify that the resulting basis is correct for different orders (Lex vs. Degrevlex).

41.17.9 References

- Buchberger, B. (1965). “An Algorithm for Finding the Basis Elements of the Residue Class Ring of a Zero Dimensional Polynomial Ideal”. PhD Thesis.
- Cox, D., Little, J., & O’Shea, D. (2007). “Ideals, Varieties, and Algorithms”. Springer.
- Wikipedia: [Buchberger’s algorithm](#)

41.18 Resultant-Based Elimination

41.18.1 Overview

Resultant-based elimination is a symbolic algebraic technique used to eliminate variables from a system of polynomial equations. The **resultant** of two polynomials is a scalar value (or another polynomial) that is zero if and only if the two original polynomials share a common root. This concept dates to the work of Sylvester [Sy140] and has been extensively developed in modern computational algebra [CLO07]. By calculating the resultant with respect to one variable, we “project” the intersection of the two polynomials onto a lower-dimensional space. It is a faster alternative to Gröbner bases for many problems in surface implicitization and collision detection [Man94].

41.18.2 Definitions

Definition 41.9 (Resultant). Let $f(x) = a_n x^n + \cdots + a_0$ and $g(x) = b_m x^m + \cdots + b_0$ be two polynomials in $k[x]$ with coefficients in a field k . The **resultant** $\text{Res}(f, g, x)$ is a polynomial in the coefficients a_i, b_j that vanishes if and only if f and g have a common root in some extension field of k , or equivalently, $\gcd(f, g)$ has positive degree.

Definition 41.10 (Sylvester Matrix). For $f(x) = a_n x^n + \cdots + a_0$ and $g(x) = b_m x^m + \cdots + b_0$, the **Sylvester matrix** is the $(m+n) \times (m+n)$ matrix formed by stacking m shifted copies of the coefficient vector of f followed by n shifted copies of the coefficient vector of g :

$$\mathbf{S}(f, g) = \begin{pmatrix} a_n & a_{n-1} & \cdots & a_0 & 0 & \cdots & 0 \\ 0 & a_n & a_{n-1} & \cdots & a_0 & \cdots & 0 \\ \vdots & & & \ddots & & \ddots & \vdots \\ 0 & \cdots & 0 & a_n & a_{n-1} & \cdots & a_0 \\ b_m & b_{m-1} & \cdots & b_0 & 0 & \cdots & 0 \\ 0 & b_m & b_{m-1} & \cdots & b_0 & \cdots & 0 \\ \vdots & & & \ddots & & \ddots & \vdots \\ 0 & \cdots & 0 & b_m & b_{m-1} & \cdots & b_0 \end{pmatrix}$$

The resultant is $\text{Res}(f, g, x) = \det(\mathbf{S}(f, g))$.

Definition 41.11 (Elimination). **Elimination** is the process of reducing a system of n polynomial equations in n variables to a system of fewer variables. In the context of resultants, computing $\text{Res}_x(f, g)$ eliminates the variable x , yielding a polynomial in the remaining variables whose roots encode the x -coordinates of intersection points.

41.18.3 Theory

The resultant of two polynomials $f(x) = \sum a_i x^i$ and $g(x) = \sum b_j x^j$ is the determinant of the Sylvester matrix.

For example, if $f(x, y)$ and $g(x, y)$ are two implicit curves, their resultant $\text{Res}_x(f, g)$ is a polynomial in y whose roots are the y -coordinates of all intersection points. This allows us to solve for y first and then back-substitute to find x .

For **Implicitization**, given parametric equations $x = X(t)$, $y = Y(t)$, $z = Z(t)$, we form the polynomials $f_1 = x - X(t)$, $f_2 = y - Y(t)$, $f_3 = z - Z(t)$ and calculate the resultant $\text{Res}_t(f_1, f_2, f_3)$ to eliminate the parameter t .

Theorem 41.12 (Resultant Characterization). Let $f, g \in k[x]$ be nonzero polynomials of degrees n and m respectively. Then $\text{Res}(f, g, x) = 0$ if and only if f and g have a common root in the algebraic closure $\bar{k}$, or both $\text{LC}(f) = 0$ and $\text{LC}(g) = 0$.

Proof. The result follows from the equivalence: f and g share a common root if and only if there exist nonzero polynomials u, v with $\deg(u) < m$ and $\deg(v) < n$ such that $uf + vg = 0$. This linear system in the coefficients of u and v has a nontrivial solution if and only if the $(m+n) \times (m+n)$ Sylvester matrix is singular, which occurs precisely when its determinant (the resultant) is zero. See [CLO07, Chapter 3]. $\square$

Example 41.13 (Resultant of Two Quadratics). Let $f(x) = x^2 + px + q$ and $g(x) = x^2 + rx + s$ (both degree $n = m = 2$). The Sylvester matrix is 4×4 :

$$\mathbf{S} = \begin{pmatrix} 1 & p & q & 0 \\ 0 & 1 & p & q \\ 1 & r & s & 0 \\ 0 & 1 & r & s \end{pmatrix}$$

Computing the determinant:

$$\text{Res}(f, g, x) = (q - s)^2 - (p - r)(ps - qr).$$

Setting this to zero yields the condition under which the two quadratics share a root.

41.18.4 Pseudocode

```

1: function RESULTANT( $f, g, x$ )
2:    $S \leftarrow \text{BuildSylvesterMatrix}(f, g, x)$ 
3:    $\text{res} \leftarrow \text{determinant}(S)$ 
4:   return res
5: end function
6: function BUILDSYLVESTERMATRIX( $f, g, x$ )
7:   Build matrix rows by shifting the coefficient vectors:
8:    $[a_n, a_{n-1}, \dots, a_0, 0, 0]$ 
9:    $[0, a_n, a_{n-1}, \dots, a_0, 0]$ 
10:   $\vdots$ 
11:   $[b_m, b_{m-1}, \dots, b_0, 0, 0]$ 
12:   $[0, b_m, b_{m-1}, \dots, b_0, 0]$ 
13:  return  $S$ 
14: end function

```

41.18.5 Parameter Selection

- **Matrix Type:** Sylvester matrices are large and sparse. Bezout or Dixon matrices are much smaller for higher-order polynomials but are more complex to construct.
- **Numerical Precision:** If coefficients are floating-point numbers, calculating the determinant can be numerically unstable. Using symbolic computation or arbitrary-precision arithmetic is recommended.

41.18.6 Complexity

- **Time Complexity:** $O(d^3)$ where $d = \deg(f) + \deg(g)$, based on the cost of the determinant calculation of the $(m+n) \times (m+n)$ Sylvester matrix.
- **Space Complexity:** $O(d^2)$ to store the Sylvester matrix.

41.18.7 Usages

- **Surface Implicitization:** Converting parametric Bézier/NURBS surfaces into implicit $f(x, y, z) = 0$ equations for faster inside/outside tests.
- **Curve/Surface Intersection:** Finding the exact points where a line (ray) intersects a curved surface.
- **Collision Detection:** Determining if two curved objects intersect by checking if their resultant has any real roots.
- **Robot Kinematics:** Solving inverse kinematics for complex linkages.
- **Computer Aided Design (CAD):** Boolean operations on curved surfaces.

41.18.8 Testing Methods and Edge Cases

- **Two Lines:** The resultant of two linear equations $a_1x + b_1$ and $a_2x + b_2$ should be $a_1b_2 - a_2b_1$.
- **Parallel Curves:** If curves never intersect, the resultant should be a non-zero constant.
- **Coincident Curves:** If two curves are the same, the resultant will be identically zero.
- **Leading Coefficient Zero:** Ensure the algorithm handles cases where the leading coefficient vanishes for certain values of the other variables.
- **Large Degrees:** Verify that the determinant calculation remains robust for polynomials of degree 10+.

41.18.9 References

- Sylvester, J. J. (1840). “A method of determining by inspection the form of the combined roots of two algebraic equations”. *Philosophical Magazine*.
- Cox, D., Little, J., & O’Shea, D. (2007). “Ideals, Varieties, and Algorithms”. Springer.
- Manocha, D. (1994). “Solving systems of polynomial equations using resultants”. SIG-GRAPH Course.
- Wikipedia: [Resultant](#)

Appendix A

Mathematical Reference

A.1 Linear Algebra

A.2 Vector Calculus

A.3 Determinants

A.4 Affine Geometry

A.5 Convexity

A.6 Probability

A.7 Graph Theory

A.8 Asymptotic Analysis

Appendix B

Algorithms and Data Structures

B.1 Sorting

B.2 Binary Search

B.3 Balanced Search Trees

B.4 Priority Queues

B.5 Hashing

B.6 Graph Algorithms

B.7 Union–Find

B.8 Randomized Algorithms

Appendix C

Geometry Formula Reference

C.1 Vector Operations

C.2 Orientation Tests

C.3 Distance Formulas

C.4 Intersection Formulas

C.5 Circle Predicates

C.6 Areas and Volumes

C.7 Coordinate Transformations

C.8 Barycentric Coordinates

Appendix D

Implementation Guide

- D.1 Designing a Geometry Kernel
- D.2 Choosing Numeric Types
- D.3 Floating-Point Error
- D.4 Exact Predicates
- D.5 Testing Geometric Software
- D.6 Generating Adversarial Test Cases
- D.7 Benchmarking
- D.8 Visualization and Debugging

Appendix E

Computational Geometry Software

E.1 CGAL

E.2 GEOSE

E.3 Boost.Geometry

E.4 Qhull

E.5 Triangle

E.6 TetGen

E.7 SciPy Spatial Algorithms

E.8 Python Geometry Ecosystem

E.9 Choosing a Library

Appendix F

Graduate Projects

- F.1 Convex Hull Laboratory
- F.2 Sweep-Line Intersection Engine
- F.3 Voronoi/Delaunay Package
- F.4 Spatial Database
- F.5 Mesh Generator
- F.6 Collision-Detection System
- F.7 Robot Path Planner
- F.8 Point-Cloud Reconstruction
- F.9 GIS Spatial Analysis System
- F.10 Persistent-Homology Explorer
- F.11 GPU Geometry Engine
- F.12 Research Replication Project

Appendix G

Selected Solutions and Hints

Bibliography

- [ABE98] N. Amenta, M. Bern, and D. Eppstein. The crust and the β -skeleton: Combinatorial curve reconstruction. *Graphical Models and Image Processing*, 60(2):125–135, 1998.
- [ACH⁺91] E. M. Arkin, L. P. Chew, D. P. Huttenlocher, K. Kedem, and J. S. Mitchell. An efficiently computable metric for comparing polygonal shapes. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 1991.
- [AG95] H. Alt and M. Godau. Computing the Fréchet distance between two polygonal curves. *International Journal of Computational Geometry & Applications*, 1995.
- [AM01] T. Akenine-Möller. Fast 3D triangle-box overlap testing. *Journal of Graphics Tools*, 2001.
- [And79] A. M. Andrew. Another efficient algorithm for convex hulls in two dimensions. *Information Processing Letters*, 1979.
- [ASC11] M. Aubry, U. Schlickewei, and D. Cremers. The wave kernel signature: A quantum mechanical approach to shape analysis. In *ICCV Workshops*, 2011.
- [Ata85] M. J. Atallah. On symmetry detection. *IEEE Transactions on Computers*, 1985.
- [Aur91] F. Aurenhammer. Voronoi diagrams: A survey of a fundamental geometric data structure. *ACM Computing Surveys*, 1991.
- [B68] P. Bézier. How Renault uses numerical control for car body design and tooling. In *SAE Paper*, 1968.
- [B⁺08] A. M. Bronstein et al. *Numerical Geometry of Non-Rigid Shapes*. Springer, 2008.
- [B⁺17] M. M. Bronstein et al. Geometric deep learning: Grids, groups, graphs, geodesics, and gauges. *arXiv preprint*, 2017.
- [BBP01] H. Brönnimann, C. Burnikel, and S. Pion. Interval arithmetic yields efficient dynamic filters for computational geometry. In *Proceedings of the 17th Annual Symposium on Computational Geometry*, pages 165–174, 2001.
- [BCKO08] M. de Berg, O. Cheong, M. van Kreveld, and M. Overmars. *Computational Geometry: Algorithms and Applications*. Springer-Verlag, 2008.
- [BDH96] C. B. Barber, D. P. Dobkin, and H. Huhdanpaa. The Quickhull algorithm for convex hulls. *ACM Transactions on Mathematical Software (TOMS)*, 1996.
- [Ben75] J. L. Bentley. Multidimensional binary search trees used for associative searching. *Communications of the ACM*, 1975.
- [Ben77] J. L. Bentley. Algorithms for Klee’s measure problem. Technical report, Technical Report, 1977.

- [BHKW07] E. K. Burke, R. S. Hellier, G. Kendall, and G. Whitwell. Complete and robust no-fit polygon generation for the irregular cutting and packing problem. *European Journal of Operational Research*, 2007.
- [BK10] M. M. Bronstein and I. Kokkinos. Scale-invariant heat kernel signatures for non-rigid shape retrieval. In *CVPR*, 2010.
- [BKP⁺10] M. Botsch, L. Kobbelt, M. Pauly, P. Alliez, and B. Lévy. *Polygon Mesh Processing*. CRC Press, 2010.
- [BKSS90] N. Beckmann, H. P. Kriegel, R. Schneider, and B. Seeger. The R^* -tree: An efficient and robust access method for points and rectangles. In *ACM SIGMOD*, 1990.
- [BLP⁺13] D. Bommes, B. Lévy, N. Pietroni, E. Puppo, C. Silva, M. Tarini, and R. Zayer. State of the art in quad-meshing. *Eurographics STAR*, 2013.
- [Blu67] H. Blum. A transformation for extracting new descriptors of shape. In *Models for the Perception of Speech and Visual Form*, 1967.
- [BM92] P. J. Besl and N. D. McKay. A method for registration of 3-d shapes. In *IEEE Transactions on Pattern Analysis and Machine Intelligence*, volume 14, pages 239–256, 1992.
- [BMR⁺99] V. Bernardini, J. Mittleman, H. Rushmeier, C. Silva, and G. Taubin. Ball-pivoting algorithm for surface reconstruction. In *IEEE Transactions on Visualization and Computer Graphics*, volume 5, pages 349–359, 1999.
- [BO79] J. L. Bentley and T. A. Ottmann. Algorithms for reporting and counting geometric intersections. *IEEE Transactions on Computers*, 1979.
- [BO08] J. A. Bennell and J. F. Oliveira. The geometry of nesting problems: A tutorial. *European Journal of Operational Research*, 2008.
- [Bow81] A. Bowyer. Computing Dirichlet tessellations. *The Computer Journal*, 1981.
- [Bri07] R. Bridson. Fast Poisson disk sampling in arbitrary dimensions. In *SIGGRAPH Sketches*, 2007.
- [Bri12] K. Bringmann. New upper bounds for Klee’s measure problem in small dimensions. *Computational Geometry*, 2012.
- [BS76] J. L. Bentley and M. I. Shamos. Divide-and-conquer in multidimensional space. In *ACM Symposium on Theory of Computing (STOC)*, 1976.
- [BS07] A. I. Bobenko and B. A. Springborn. A discrete Laplace–Beltrami operator for simplicial surfaces. *Discrete & Computational Geometry*, 2007.
- [Buc65] B. Buchberger. *An Algorithm for Finding the Basis Elements of the Residue Class Ring of a Zero Dimensional Polynomial Ideal*. PhD thesis, University of Innsbruck, 1965.
- [CB78] M. Cyrus and J. Beck. Generalized two- and three-dimensional clipping. *Computers and Graphics*, 3(1):23–28, 1978.
- [CC78] E. Catmull and J. Clark. Recursively generated B-spline surfaces on arbitrary topological meshes. *Computer-Aided Design*, 1978.

-
- [CF90] B. Chazelle and J. Friedman. A deterministic view of random sampling and its use in geometry. *Combinatorica*, 10(3):229–249, 1990.
 - [CGL85] B. Chazelle, L. J. Guibas, and D. T. Lee. The power of geometric duality. *BIT Numerical Mathematics*, 1985.
 - [Cha85] B. Chazelle. On the convex layers of a planar set. *IEEE Transactions on Information Theory*, 1985.
 - [Cha96] T. M. Chan. Optimal output-sensitive convex hull algorithms in two and three dimensions. *Discrete & Computational Geometry*, 1996.
 - [Chv75] V. Chvátal. A combinatorial theorem in plane geometry. *Journal of Combinatorial Theory, Series B*, 1975.
 - [CL06] R. R. Coifman and S. Lafon. Diffusion maps. *Applied and Computational Harmonic Analysis*, 2006.
 - [CLO07] D. Cox, J. Little, and D. O’Shea. *Ideals, Varieties, and Algorithms*. Springer, 2007.
 - [CLRS09] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein. *Introduction to Algorithms*. MIT Press, 2009.
 - [CM69] E. Cuthill and J. McKee. Reducing the bandwidth of sparse symmetric matrices. In *ACM National Conference*, 1969.
 - [CM92] Y. Chen and G. Medioni. Object modeling by registration of multiple range images. In *IEEE International Conference on Robotics and Automation*, pages 2724–2729, 1992.
 - [Coo86] R. L. Cook. Stochastic sampling in computer graphics. *ACM Transactions on Graphics*, 1986.
 - [CWW13a] K. Crane, C. Weischedel, and M. Wardetzky. Digital geometry processing with discrete exterior calculus. In *SIGGRAPH Course*, 2013.
 - [CWW13b] K. Crane, C. Weischedel, and M. Wardetzky. Geodesics in heat: A new approach to computing distance based on heat flow. *ACM Transactions on Graphics*, 2013.
 - [CWW17] K. Crane, C. Weischedel, and M. Wardetzky. Boundary first flattening. *ACM Transactions on Graphics*, 2017.
 - [dBCvKO08] M. de Berg, O. Cheong, M. van Kreveld, and M. Overmars. *Computational Geometry: Algorithms and Applications*. Springer, 3rd edition, 2008.
 - [Dev86] L. Devroye. *Non-Uniform Random Variate Generation*. Springer, 1986.
 - [Dev02] O. Devillers. The delaunay hierarchy. *International Journal of Foundations of Computer Science*, 13(2):163–180, 2002.
 - [Dey98] T. K. Dey. Improved bounds on planar k-sets and related problems. *Discrete & Computational Geometry*, 19(3):373–382, 1998.
 - [DHLM05] M. Desbrun, A. N. Hirani, M. Leok, and J. E. Marsden. Discrete exterior calculus. *arXiv preprint*, 2005.
 - [Die17] R. Diestel. *Graph Theory*. Springer, 2017.

- [Dij59] E. W. Dijkstra. A note on two problems in connexion with graphs. *Numerische Mathematik*, 1959.
- [DS65] H. Davenport and A. Schinzel. A combinatorial problem connected with differential equations. *American Journal of Mathematics*, 1965.
- [Edd77] W. F. Eddy. A new convex hull algorithm for planar sets. *ACM Transactions on Mathematical Software (TOMS)*, 1977.
- [Ede80] H. Edelsbrunner. Dynamic data structures for orthogonal intersection queries. Technical report, Technical University of Graz, 1980.
- [Ede85] H. Edelsbrunner. Computing the extreme distances between two convex polygons. *Journal of Algorithms*, 1985.
- [Ede87] H. Edelsbrunner. *Algorithms in Combinatorial Geometry*. Springer-Verlag, 1987.
- [EM90] H. Edelsbrunner and E. P. Mücke. Simulation of simplicity: A technique to cope with degenerate cases in geometric algorithms. *ACM Transactions on Graphics*, 9(1):66–104, 1990.
- [EM94a] H. Edelsbrunner and E. P. Mücke. Three-dimensional alpha shapes. *ACM Transactions on Graphics*, 13(1):43–72, 1994.
- [EM94b] T. Eiter and H. Mannila. Computing discrete Fréchet distance. Technical report, Technical University of Vienna, 1994.
- [EOS86] H. Edelsbrunner, J. O’Rourke, and R. Seidel. Constructing arrangements of lines and hyperplanes with applications. *SIAM Journal on Computing*, 1986.
- [Eri04] C. Ericson. *Real-Time Collision Detection*. Morgan Kaufmann, 2004.
- [ES83] H. Edelsbrunner and R. Seidel. Voronoi diagrams and arrangements. *Discrete & Computational Geometry*, 1:25–44, 1983.
- [Eul58] L. Euler. Elementa doctrinae solidorum. *Novi Commentarii Academiae Scientiarum Petropolitanae*, 1758.
- [Far02] G. Farin. *Curves and Surfaces for CAGD: A Practical Guide*. Morgan Kaufmann, 2002.
- [FB74] R. A. Finkel and J. L. Bentley. Quad trees: A data structure for retrieval on composite keys. *Acta Informatica*, 1974.
- [FBF77] J. H. Friedman, J. L. Bentley, and R. A. Finkel. An algorithm for finding best matches in logarithmic expected time. *ACM Transactions on Mathematical Software*, 1977.
- [FH12] P. F. Felzenszwalb and D. P. Huttenlocher. Distance transforms of sampled functions. *Theory of Computing*, 2012.
- [Fis78] S. Fisk. A short proof of chvátal’s watchman theorem. *Journal of Combinatorial Theory, Series B*, 1978.
- [FKN80] H. Fuchs, Z. M. Kedem, and B. F. Naylor. On visible surface generation by a priori tree structures. In *Proceedings of the 7th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH)*, pages 124–133, 1980.

-
- [Flo53] P. J. Flory. *Principles of Polymer Chemistry*. Cornell University Press, 1953.
- [For87] S. Fortune. A sweepline algorithm for Voronoi diagrams. *Algorithmica*, 1987.
- [For98] R. Forman. Discrete Morse theory. *Advances in Mathematics*, 1998.
- [For02] R. Forman. A user’s guide to discrete Morse theory. *Sém. Lothar. Combin.*, 2002.
- [Fré06] M. Fréchet. Sur quelques points du calcul fonctionnel. *Rendiconti del Circolo Matematico di Palermo*, 1906.
- [FSBS07] M. Fisher, B. Springborn, A. Bobenko, and P. Schröder. An algorithm for the construction of intrinsic Delaunay triangulations with applications to digital geometry processing. *ACM Transactions on Graphics*, 2007.
- [FVW93] S. Fortune and C. J. Van Wyk. Efficient exact arithmetic for computational geometry. In *Proceedings of the 9th Annual Symposium on Computational Geometry*, pages 163–172, 1993.
- [FVW96] S. Fortune and C. J. Van Wyk. Static analysis yields efficient exact integer arithmetic for computational geometry. *ACM Transactions on Graphics*, 15(3):223–248, 1996.
- [GH86] M. Gage and R. S. Hamilton. The heat equation shrinking convex plane curves. *Journal of Differential Geometry*, 1986.
- [GH97] M. Garland and P. S. Heckbert. Surface simplification using quadric error metrics. *Proceedings of the 24th Annual Conference on Computer Graphics and Interactive Techniques*, pages 209–216, 1997.
- [GH98] G. Greiner and K. Hormann. Efficient clipping of arbitrary polygons. *ACM Transactions on Graphics*, 1998.
- [Gib98] S. F. F. Gibson. Constrained elastic surface nets: Generating smooth surfaces from binary segmented data. In *Proceedings of Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, pages 888–898, 1998.
- [GJK88] E. G. Gilbert, D. W. Johnson, and S. S. Keerthi. A fast procedure for computing the distance between complex objects in three-dimensional space. *IEEE Journal on Robotics and Automation*, 1988.
- [GNPB08] A. Gyulassy, V. Natarajan, V. Pascucci, and P. T. Bremer. A practical approach to Morse-Smale complex computation. In *IEEE Transactions on Visualization and Computer Graphics*, 2008.
- [Gol91] D. Goldberg. What every computer scientist should know about floating-point arithmetic. *ACM Computing Surveys*, 23(1):5–48, 1991.
- [Gra72] R. L. Graham. An efficient algorithm for determining the convex hull of a finite planar set. *Information Processing Letters*, 1972.
- [Gra87] M. A. Grayson. The heat equation shrinks embedded plane curves to round points. *Journal of Differential Geometry*, 1987.
- [GS85] L. Guibas and J. Stolfi. Primitives for the manipulation of general subdivisions and the computation of Voronoi diagrams. *ACM Transactions on Graphics*, 1985.

- [GS87] L. J. Guibas and R. Seidel. Computing convolutions by reciprocal search. *Discrete & Computational Geometry*, 1987.
- [Gut84] A. Guttman. R-trees: A dynamic index structure for spatial searching. In *ACM SIGMOD*, 1984.
- [H⁺18] Y. Hu et al. fTetWild: Robust tetrahedral meshing in the wild. In *ACM Transactions on Graphics (SIGGRAPH)*, 2018.
- [HA01] K. Hormann and A. Agathos. The point in polygon problem for arbitrary polygons. *Computational Geometry*, 2001.
- [Hal60] J. H. Halton. On the efficiency of certain quasi-random sequences of points in evaluating multi-dimensional integrals. *Numerische Mathematik*, 1960.
- [Her89] J. Hershberger. Finding the upper envelope of n line segments in $O(n \log n)$ time. *Information Processing Letters*, 1989.
- [Hig86] P. T. Highnam. Optimal algorithms for finding the symmetries of a planar point set. *Information Processing Letters*, 1986.
- [Hil91] D. Hilbert. Über die stetige abbildung einer linie auf ein flächenstück. *Mathematische Annalen*, 1891.
- [HMY04] D. Halperin, K. Mehlhorn, and C. K. Yap. Robust geometric computation. In J. E. Goodman and J. O'Rourke, editors, *Handbook of Computational Geometry*, pages 1119–1152. CRC Press, 2 edition, 2004.
- [HW87] D. Haussler and E. Welzl. ε -nets and simplex range queries. *Discrete & Computational Geometry*, 2(2):127–151, 1987.
- [Jyl10] J. Jylänki. A thousand ways to pack the bin - a practical approach to two-dimensional rectangle bin packing. Technical report, Technical Report, 2010.
- [Kac66] M. Kac. Can one hear the shape of a drum? *American Mathematical Monthly*, 1966.
- [KBH06] M. Kazhdan, M. Bolitho, and H. Hoppe. Poisson surface reconstruction. *Proceedings of the Fourth Eurographics Symposium on Geometry Processing*, pages 61–70, 2006.
- [Kle77] V. Klee. Can the measure of $\cup[a_i, b_i]$ be computed in less than $O(n \log n)$ steps? *American Mathematical Monthly*, 1977.
- [KLN91] M. Karasick, D. Lieber, and L. R. Nackman. Efficient delaunay triangulation using rational arithmetic. *ACM Transactions on Graphics*, 10(1):71–91, 1991.
- [KMP⁺08] L. Kettner, K. Mehlhorn, S. Pion, S. Schirra, and C. K. Yap. Classroom examples of robustness problems in geometric computations. *Computational Geometry: Theory and Applications*, 40(1):61–90, 2008.
- [Koe36] P. Koebe. Kontaktprobleme der konformen abbildung. *Berichte Sächs. Akad. Wiss. Leipzig*, 1936.
- [KS98] R. Kimmel and J. A. Sethian. Computing geodesic paths on manifolds. *Proceedings of the National Academy of Sciences*, 1998.

-
- [Lam70] G. Laman. On graphs and rigidity of plane skeletal structures. *Journal of Engineering Mathematics*, 1970.
- [Las96] M. J. Laszlo. *Computational Geometry and Computer Graphics in C++*. Prentice Hall, 1996.
- [Law77] C. L. Lawson. Software for C^1 surface interpolation. *Mathematical Software*, 1977.
- [LC87] W. E. Lorensen and H. E. Cline. Marching cubes: A high resolution 3d surface construction algorithm. In *ACM SIGGRAPH Computer Graphics*, volume 21, pages 163–169, 1987.
- [LD81] D. T. Lee and R. L. Drysdale. Generalization of Voronoi diagrams in the plane. *SIAM Journal on Computing*, 10(1):73–87, 1981.
- [LD08] A. Lagae and P. Dutré. A comparison of methods for generating Poisson disk distributions. *Computer Graphics Forum*, 2008.
- [Lev06] B. Levy. Laplace-Beltrami eigenfunctions towards an optimal tool for geology and geometry. In *SMA*, 2006.
- [Llo82] S. Lloyd. Least squares quantization in PCM. *IEEE Transactions on Information Theory*, 1982.
- [LMM02] A. Lodi, S. Martello, and M. Monaci. Two-dimensional packing problems: A survey. *European Journal of Operational Research*, 2002.
- [Loo87] C. Loop. Smooth subdivision surfaces based on triangle meshes. Master’s thesis, University of Utah, 1987.
- [LP79] D. T. Lee and F. P. Preparata. An optimal algorithm for finding the kernel of a polygon. *Journal of the ACM*, 26(4):702–712, 1979.
- [LP83] T. Lozano-Pérez. Spatial planning: A configuration space approach. *IEEE Transactions on Computers*, 1983.
- [Lue78] G. S. Lueker. A data structure for orthogonal range queries. In *IEEE Symposium on Foundations of Computer Science (FOCS)*, 1978.
- [Man94] D. Manocha. Solving systems of polynomial equations using resultants. In *SIGGRAPH Course*, 1994.
- [Mar72] G. Marsaglia. Choosing a point from the surface of a sphere. *The Annals of Mathematical Statistics*, 43(2):645–646, 1972.
- [Mat92] J. Matoušek. Reporting points in halfspaces. *Computational Geometry: Theory and Applications*, 2(3):169–186, 1992.
- [Mat95] J. Matoušek. Tight upper bounds for the discrepancy of half-spaces. *Discrete & Computational Geometry*, 13(3–4):593–601, 1995.
- [Mat99] J. Matoušek. *Geometric Discrepancy: An Illustrated Guide*. Springer, 1999.
- [Meg83] N. Megiddo. Linear-time algorithms for linear programming in R^3 and related problems. *SIAM Journal on Computing*, 1983.
- [Mei75] G. H. Meisters. Polygons have ears. *The American Mathematical Monthly*, 1975.

- [MJFS01] B. Moon, H. V. Jagadish, C. Faloutsos, and J. H. Saltz. Analysis of the clustering properties of the Hilbert space-filling curve. *IEEE Transactions on Knowledge and Data Engineering*, 2001.
- [MMP87] J. S. B. Mitchell, D. M. Mount, and C. H. Papadimitriou. The discrete geodesic problem. *SIAM Journal on Computing*, 1987.
- [Mor66] G. M. Morton. A computer-oriented geodetic data base and a new technique in file sequencing. Technical report, IBM Technical Report, 1966.
- [MRH02] A. Meijster, J. B. Roerdink, and W. H. Hesselink. A general algorithm for computing distance transforms in linear time. In *Mathematical Morphology and its Applications to Image and Signal Processing*, 2002.
- [MS96] N. Madras and G. Slade. *The Self-Avoiding Walk*. Birkhäuser, 1996.
- [Mul90] K. Mulmuley. A fast planar partition algorithm, I. *Journal of Symbolic Computation*, 10:253–266, 1990.
- [Mus13] K. Museth. VDB: High-resolution sparse volumes with dynamic topology. *ACM Transactions on Graphics*, 32(3):1–22, 2013.
- [MY20] Y. A. Malkov and D. A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4):824–836, 2020.
- [NAT90] B. Naylor, J. Amanatides, and W. Thibault. Merging bsp trees yields polyhedral set operations. In *Proceedings of the 17th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH)*, pages 115–124, 1990.
- [Nie92] H. Niederreiter. *Random Number Generation and Quasi-Monte Carlo Methods*. SIAM, 1992.
- [NT03] F. S. Nooruddin and G. Turk. Simplification and repair of polygonal models using volumetric techniques. *IEEE Transactions on Visualization and Computer Graphics*, 2003.
- [OBCS⁺12] M. Ovsjanikov, M. Ben-Chen, J. Solomon, A. Butscher, and L. Guibas. Functional maps: A flexible representation of maps between shapes. *ACM Transactions on Graphics*, 2012.
- [OBSC00] A. Okabe, B. Boots, K. Sugihara, and S. N. Chiu. *Spatial Tessellations: Concepts and Applications of Voronoi Diagrams*. Wiley, 2nd edition, 2000.
- [OF03] S. Osher and R. Fedkiw. *Level Set Methods and Dynamic Implicit Surfaces*. Springer, 2003.
- [O’R87] J. O’Rourke. *Art Gallery Theorems and Algorithms*. Oxford University Press, 1987.
- [OvL81] M. H. Overmars and J. van Leeuwen. Maintenance of configurations in the plane. *Journal of Computer and System Sciences*, 1981.
- [OY91] M. H. Overmars and C. K. Yap. New upper bounds in Klee’s measure problem. *SIAM Journal on Computing*, 1991.
- [P21] G. Pólya. Über eine aufgabe der Wahrscheinlichkeitsrechnung betreffend die Irrfahrt im Straßennetz. *Mathematische Annalen*, 1921.

-
- [P⁺14] D. Panozzo et al. Frame fields: An orientation-aware surface representation. In *ACM Transactions on Graphics (SIGGRAPH)*, 2014.
 - [PC06] G. Peyré and L. D. Cohen. Geodesic remeshing using front propagation. *International Journal of Computer Vision*, 2006.
 - [Pea90] G. Peano. Sur une courbe, qui remplit toute une aire plane. *Mathematische Annalen*, 1890.
 - [Pea05] K. Pearson. The problem of the random walk. *Nature*, 1905.
 - [PM92] W. B. Pennebaker and J. L. Mitchell. *JPEG: Still Image Data Compression Standard*. Van Nostrand Reinhold, 1992.
 - [Poi95] H. Poincaré. Analysis situs. *Journal de l'École Polytechnique*, 1895.
 - [PP93] U. Pinkall and K. Polthier. Computing discrete minimal surfaces and their conjugates. *Computing*, 1993.
 - [PP03] K. Polthier and E. Preuss. Identifying vector field singularities using a discrete Hodge decomposition. In *Visualization and Mathematics III*, 2003.
 - [Pri91] D. M. Priest. Algorithms for arbitrary precision floating-point arithmetic. In *Proceedings of the 10th IEEE Symposium on Computer Arithmetic*, pages 132–143, 1991.
 - [PS85] F. P. Preparata and M. I. Shamos. *Computational Geometry: An Introduction*. Springer-Verlag, 1985.
 - [PT97] L. Piegl and W. Tiller. *The NURBS Book*. Springer, 1997.
 - [R⁺12] J. F. Remacle et al. A frontal Delaunay quad mesh generator. *International Journal for Numerical Methods in Engineering*, 2012.
 - [RP66] A. Rosenfeld and J. L. Pfaltz. Sequential operations in digital picture processing. *Journal of the ACM*, 1966.
 - [RWP06] M. Reuter, F. E. Wolter, and N. Peinecke. Laplace-Beltrami spectra as Shape-DNA of surfaces and solids. *Computer-Aided Design*, 2006.
 - [S⁺25] N. Sharp et al. SpectralNet: Learning spectral descriptors for shape analysis. In *ACM Transactions on Graphics (SIGGRAPH)*, 2025.
 - [SA95] M. Sharir and P. K. Agarwal. *Davenport-Schinzel Sequences and Their Geometric Applications*. Cambridge University Press, 1995.
 - [Sag94] H. Sagan. *Space-Filling Curves*. Universitext, 1994.
 - [Sam84] H. Samet. The quadtree and related hierarchical data structures. *ACM Computing Surveys*, 1984.
 - [Sam06] H. Samet. *Foundations of Multidimensional and Metric Data Structures*. Elsevier, 2006.
 - [SC20] R. Sawhney and K. Crane. Monte carlo geometry processing: A meshless approach to geometric PDE. In *ACM Transactions on Graphics (SIGGRAPH)*, 2020.

- [Sei91] R. Seidel. A simple and fast incremental randomized algorithm for computing trapezoidal decompositions and for point location. *Computational Geometry: Theory and Applications*, 1(1):51–64, 1991.
- [Set96] J. A. Sethian. A fast marching level set method for monotonically advancing fronts. *Proceedings of the National Academy of Sciences*, 1996.
- [SH74] I. E. Sutherland and G. W. Hodgman. Reentrant polygon clipping. *Communications of the ACM*, 17(1):32–42, 1974.
- [SH75] M. I. Shamos and D. Hoey. Closest-point problems. In *IEEE Symposium on Foundations of Computer Science (FOCS)*, 1975.
- [She97] J. R. Shewchuk. Adaptive precision floating-point arithmetic and fast robust geometric predicates. *Discrete & Computational Geometry*, 1997.
- [She98] J. R. Shewchuk. *Delaunay Refinement Mesh Generation*. PhD thesis, Carnegie Mellon University, 1998.
- [SOG09] J. Sun, M. Ovsjanikov, and L. Guibas. A concise and provably informative multi-scale signature based on heat diffusion. *Computer Graphics Forum*, 2009.
- [Spi76] F. Spitzer. *Principles of Random Walk*. Springer, 1976.
- [Ste05] K. Stephenson. *Introduction to Circle Packing: The Theory of Discrete Analytic Functions*. Cambridge University Press, 2005.
- [Sun01] D. Sunday. Inclusion of a point in a polygon. *Journal of Graphics Tools*, 2001.
- [SUW64] M. L. Stein, S. M. Ulam, and M. B. Wells. A visual display of some properties of the distribution of primes. *The American Mathematical Monthly*, 1964.
- [Sy140] J. J. Sylvester. A method of determining by inspection the form of the combined roots of two algebraic equations. *Philosophical Magazine*, 1840.
- [Thu85] W. Thurston. The finite Riemann mapping theorem. In *International Congress of Mathematicians*, 1985.
- [TLHD03] Y. Tong, S. Lombeyda, A. N. Hirani, and M. Desbrun. Discrete multiscale vector field decomposition. In *ACM Transactions on Graphics*, 2003.
- [Tou83] G. T. Toussaint. Solving geometric problems with the rotating calipers. In *Proceedings of MELECON '83*, 1983.
- [Tsi95] J. N. Tsitsiklis. Efficient algorithms for globally optimal trajectories. *IEEE Transactions on Automatic Control*, 1995.
- [Tuk75] J. W. Tukey. Mathematics and the picturing of data. In *Proceedings of the International Congress of Mathematicians*, 1975.
- [Tut63] W. T. Tutte. How to draw a graph. *Proceedings of the London Mathematical Society*, 1963.
- [Vat92] B. R. Vatti. A generic solution to polygon clipping. *Communications of the ACM*, 1992.
- [VdC35] J. G. Van der Corput. Verteilungsfunktionen. *Proc. Akad. Wet. Amsterdam*, 1935.

- [Vor08] G. Voronoi. Nouvelles applications des paramètres continus à la théorie des formes quadratiques. *Journal für die reine und angewandte Mathematik*, 1908.
- [Wat81] D. F. Watson. Computing the n-dimensional Delaunay tessellation with application to Voronoi polytopes. *The Computer Journal*, 1981.
- [Wel91] E. Welzl. Smallest enclosing disks (balls and ellipsoids). In *Lecture Notes in Computer Science*, 1991.
- [WLW22] X. Wei, J. Liu, and J. Wang. CoACD: Collision-oriented approximate convex decomposition. In *ACM Transactions on Graphics (SIGGRAPH)*, 2022.
- [WP67] D. J. A. Welsh and M. B. Powell. An upper bound for the chromatic number of a graph and its application to timetabling problems. *The Computer Journal*, 1967.
- [Y⁺25] Y. Yu et al. Robust derivative estimation with walk on stars. In *ACM Transactions on Graphics (SIGGRAPH)*, 2025.
- [Yap88] C. K. Yap. Symbolic treatment of geometric degeneracies. *Journal of Symbolic Computation*, 10(3-4):349–370, 1988.
- [Yap90] C. K. Yap. A geometric consistency theorem for a symbolic perturbation scheme. *Journal of Computer and System Sciences*, 40(1):2–18, 1990.
- [Yap97] C. K. Yap. Towards exact geometric computation. *Computational Geometry: Theory and Applications*, 7(1-2):3–23, 1997.
- [Z⁺25] Y. Zhu et al. Designing 3D anisotropic frame fields with Odeco tensors. In *ACM Transactions on Graphics (SIGGRAPH)*, 2025.

Index

- $\lambda_s(n)$, 157
- x -node, 147
- y -node, 147
- 3D scanning, 240

- AABB, 129
- adjacency list, 314
- affine heat flow, 227
- alpha-shape, 212
- anchor point, 13
- Andrew's algorithm, 16
- angle-bounded remeshing, 226
- anisotropy, 347
- approximate convex decomposition, 201
- arrangements, 159
- art gallery problem, 277
- AVL tree, 85

- backwards analysis, 148
- bandwidth, 332
- barycentric coordinates, 49
- Bellman–Ford algorithm, 269
- Betti number, 353, 354, 358
- Bezier surface, 189
- bin packing, 370
- binary heap, 270
- binary space partitioning, 93
- bioinformatics, 275
- bit-interleaving, 99
- blue noise, 289
- Bottom-Left-Fill, 374
- boundary alignment, 347
- boundary detection, 316
- boundary edge, 316
- boundary integral equation, 388
- boundary loop, 316
- boundary operator, 351
- boundary penalty quadric, 338
- breadth-first search, 313
- Brownian motion, 302
- Buchberger's algorithm, 400

- can one hear the shape of a drum, 342
- Catmull-Clark subdivision, 329
- Chamfer matching, 267
- character recognition, 42
- chromatic number, 330
- circle contact graph, 367
- circle packing, 367
- circumcentric dual, 348
- circumradius, 212
- clipping, 42
- cochain, 348
- codifferential, 354
- collinear points, 15
- collision detection, 35, 80, 90, 182
- comb polygon, 278
- computational geometry, 76
- computational topology, 238
- computer graphics, 76, 83, 267
- computer vision, 38, 42, 267, 275
- configuration space, 35
- conformal parameterization, 216
- conjugate gradient, 347
- connected component, 313, 345
- constrained Delaunay triangulation, 62
- convex decomposition, 44
- convex hull, 13, 212
- convex hull peeling, 21
- cotangent weight, 349
- cotangent weights, 343
- Cox-de Boor recursion, 192
- cross-covariance matrix, 241
- CSG, 29
- curl, 351
- curl-free, 354
- cusp vertex, 52

- DAG, 147
- database indexing, 86
- Davenport–Schinzel sequence, 157
- DCEL, 30
- Degrevlex, 403

- Delaunay property, 54
- Delaunay triangulation, 54, 212
- depth, 141
- depth-first search, 313
- differential geometry, 351
- diffusion, 302
- diffusion distance, 222
- diffusion equation, 303
- Dijkstra's algorithm, 261, 268, 269
 - greedy, 268
- Dirichlet boundary condition, 343
- Dirichlet spectrum, 342
- discrepancy, 293
- discrete exterior calculus, 348
- discrete Morse theory, 357
- distance map, 265
- distance transform, 265
- divergence, 351
- divergence-free, 354
- divide and conquer, 19, 125, 160
- dog-leash analogy, 273
- dual graph, 313, 319
- duality, 32
- duplicate points, 15
- dynamic programming, 46, 274

- ear, 49
- ear clipping, 49
- edge collapse, 337
- edge flip, 54
- eigen-decomposition, 344, 391
- eigenfunction, 342, 390
- Eikonal equation, 261
- elimination, 404
- envelope complexity, 160
- epsilon-sample, 209
- Euclidean distance, 125, 180
- Euler characteristic, 322, 324, 327, 347, 358
- Euler formula, 324
- exact geometric computation, 9
- expansion, 9
- expansion arithmetic, 8
- exterior calculus, 348
- exterior derivative, 351
- extraordinary vertex, 327

- face-edge incidence, 316
- farthest-point Voronoi diagram, 116
- Fast Marching Method, 261, 263
- Fibonacci heap, 269, 270
- Fiedler value, 343
- Fiedler vector, 345

- finite element method, 337
- fixed-radius neighbor search, 91
- floating-point filter, 8
- Flory exponent, 306
- FMM
 - Accepted set, 262
 - Far set, 262
 - point classification, 262
 - Trial set, 262
- force-directed layout, 368
- Fréchet distance, 273, 274
- Fréchet mean, 271
- frame field, 346
- frame field singularity, 347
- free space diagram, 273
- functional maps, 220
- fundamental quadric, 338

- game development, 270
- Gauss–Bonnet theorem, 325
- general position, 30
- generalized winding number, 196
- genus, 324
- geodesic distance, 222, 229, 264, 271
- GIS, 42, 76, 80, 83
- GIS clipping, 29
- GJK algorithm, 180
- GPS navigation, 270
- Gröbner basis, 400, 401
- gradient, 351
- gradient estimation, 388
- graph, 268
- graph coloring, 329
- graph partitioning, 343, 345
- graph search, 319
- gravity heuristic, 373
- greedy coloring, 330
- Green's function, 388
- grid resolution, 264
- guillotine cut, 371

- half-edge, 314, 317, 328
- half-plane discrepancy, 163
- half-plane intersection, 280
- Halton sequence, 292
- handwriting recognition, 275
- harmonic component, 354
- harmonic mapping, 217
- Hausdorff distance, 273
- heat kernel, 222, 343
- Heat Kernel Signature, 345, 389
- Heat Method, 271, 272

-
- hex meshing, 347
 - hexagonal packing, 367
 - hexahedral remeshing, 347
 - hidden surface removal, 94
 - Hilbert curve, 96
 - Hodge decomposition, 353, 354
 - Hodge star, 348
 - homology, 353, 357
 - homology basis, 238
 - hopper, 373
 - hopper optimization, 373

 - illegal edge, 59
 - image compression, 80
 - image processing, 265, 267
 - image segmentation, 264
 - implicitization, 404
 - in-circle test, 54
 - interval tree, 84
 - intrinsic Delaunay edge, 120
 - intrinsic Delaunay triangulation, 119
 - inverse Ackermann function, 157
 - isometry invariance, 342, 343, 390
 - isospectral shapes, 343, 345
 - Iterative Closest Point, 240

 - K-Geodesics, 272
 - k-level, 163
 - K-Means clustering, 271
 - K-Means++, 272
 - k-nearest neighbors, 128
 - Kac, Mark, 342
 - KD-tree, 73, 128, 241, 244
 - Keil–Snoeyink algorithm, 46
 - kernel of a polygon, 280
 - kinetic data structure, 159
 - Klee’s measure problem, 165

 - Laplace equation, 387, 388
 - Laplace–Beltrami operator, 342, 389, 390
 - Laplace–Beltrami operator, 223, 349, 355
 - lazy propagation, 87
 - level of detail, 337
 - LiDAR, 207
 - line arrangement, 30
 - link condition, 337
 - Lipschitz boundary, 388
 - Lloyd’s algorithm, 271
 - locality preservation, 96
 - Loop subdivision, 329
 - low-discrepancy sequence, 292
 - lower envelope, 157, 160

 - machine learning, 76
 - manifold, 321
 - manifold verification, 321
 - marching cubes, 257
 - martingale, 388
 - martingale convergence theorem, 388
 - mass matrix, 343
 - matrix construction, 344
 - MaxRects, 372
 - mean curvature flow, 223
 - mean value property, 388
 - medial axis, 209, 231, 267
 - medical imaging, 264, 267
 - Meijster’s algorithm, 266
 - mesh decimation, 337
 - mesh refinement, 197
 - mesh segmentation, 272, 345
 - minimization diagram, 160
 - minimum bounding box, 138
 - Minkowski difference, 180
 - Minkowski sum, 33
 - monomial order, 401
 - monotone decomposition, 52
 - Monte Carlo estimation, 165
 - Monte Carlo method, 302, 387
 - morphological operations, 35
 - Morse theory, 357
 - Morse–Smale complex, 357
 - Morton curve, 99
 - motion planning, 32
 - multi-view reconstruction, 240

 - nearest neighbor, 128
 - nearest neighbor search, 74
 - nesting, 373
 - network routing, 270
 - No-Fit Polygon, 373
 - non-convex polygon, 33
 - non-obtuse triangulation, 63
 - non-rigid shape matching, 391
 - non-zero winding rule, 134
 - normalization, 344
 - NURBS, 192

 - octree, 77
 - Odeco frame fields, 346, 348
 - Odeco tensor, 346
 - onion peeling, 21
 - OpenVDB, 255
 - optimal packing, 367
 - orientability, 318
 - orientation test, 16

- orthogonal matrix, 240
- orthogonal polygon, 278
- orthogonal range query, 131
- outlier detection, 23
- output-sensitive algorithm, 18

- packing density, 367
- parallel transport, 347
- partial differential equations, 348
- Peano curve, 102
- photogrammetry, 207
- physical security, 279
- plane sweep, 52
- Poincaré–Hopf theorem, 347
- point cloud, 207, 240, 346
- point location, 146
 - walking, 151
- Poisson disk sampling, 289
- Poisson equation, 348, 387, 388
- Polya’s recurrence theorem, 303
- polygon triangulation, 277
- polynomial ideal, 401
- primal graph, 313
- primal mesh, 348
- priority queue, 262

- quad meshing, 347
- quad-edge data structure, 60
- quadric, 337
- quadric error metric, 337
- quadtrees, 77
- quasi-Monte Carlo, 293

- R-tree, 81, 129
- random polygon generation, 300
- random segment generation, 298
- random walk, 302
- randomized incremental construction, 147
- range minimum query, 88
- range query, 78
- range search, 32, 131
- range tree, 132
- rational arithmetic, 9
- ray tracing, 86
- rectangle packing, 370
- red-black tree, 85
- reflectional symmetry, 37
- reflex vertex, 45
- rejection sampling, 296
- remeshing, 272
- resource allocation, 86
- resultant, 404

- Reverse Cuthill–McKee, 332
- Riemannian geometry, 348
- robotics, 38, 240, 279
 - path planning, 264, 267, 270
- rotating calipers, 138, 181
- rotational symmetry, 37

- S-polynomial, 401
- scale invariance, 390
- Schrödinger equation, 389
- segment tree, 86, 141, 166
- seismology, 264
- self-avoiding walk, 305
- separable filter, 265
- separating axis, 180
- shape analysis, 272
- shape matching, 342
- shape retrieval, 344, 391
- shape spectrum, 342
- Shape-DNA, 343, 344
- signed distance field, 253
- Simulation of Simplicity, 10
- singular value decomposition, 241
- singularity, 327
- skyline algorithm, 371
- SLAM, 240
- smallest enclosing circle, 178
- smoothness optimization, 347
- snake scan, 104
- social networks, 270
- space-filling curve, 96, 102
- space-partitioning, 73
- sparse voxel octree, 254
- spatial database, 83
- spatial query, 77
- spectral clustering, 343
- spectral projection, 390
- SpectralNet, 345, 346
- spherical Voronoi diagram, 272
- spiral walk, 107
- star polygon, 279
- star-shaped polygon, 280
- Steiner point, 63
- stiffness matrix, 343
- stochastic PDE solver, 387
- straight skeleton, 234
- strip packing, 370
- support points, 178
- surface implicitization, 403
- surface reconstruction, 207
- sweep line, 143
- sweep-line, 141, 143

- sweep-line algorithm, 166
- sweep-line paradigm, 25
- Sylvester matrix, 404
- symmetry, 344
- symmetry detection, 344, 391

- tetrahedral mesh, 346
- texture atlas, 272
- time complexity
 - $O((n + k) \log n)$, 144
 - $O(n \log h)$, 18
 - $O(n \log n)$, 14, 56, 126
 - $O(n^2)$, 20, 23, 50, 56, 60
- tolerance, 242
- topological data analysis, 357
- trajectory analysis, 275
- trapezoid, 146
- trapezoidal map, 146, 147
- triangle merging, 44
- triangle to quad conversion, 203
- triangle-box intersection, 194
- Tukey depth, 21
- two-pass algorithm, 265

- uniform grid, 89
- uniform random sampling, 295

- union measure, 165
- union of rectangles, 25
- upper envelope, 160

- valence, 327
- van der Corput sequence, 292
- visibility graph, 32
- VLSI design, 86
- Voronoi diagram, 56, 113
- Voronoi vertex, 136
- voxel geometry, 251
- voxel grid decimation, 205
- voxelization, 194, 251

- Walk on Spheres, 387, 388
- Walk on Stars, 387, 388
- wave front, 261
- Wave Kernel Signature, 345, 389, 391
- Welzl's algorithm, 177
- Weyl's law, 343
- winding number, 134
- wireless networking, 279

- Yamabe equation, 216

- Z-order, 99
- zigzag curve, 104